ГЛУБОКОЕ ОПИСАНИЕ ПРОЕКТА

EH/Unwind Analyzer: полная техническая карта проекта от чтения bytes до PE-модели, RUNTIME_FUNCTION, UNWIND_INFO, handler layer, эвристик, dynamic evidence, отчетов, тестирования и границ применимости.

1. Паспорт проекта

Пункт	Описание
Название	EH/Unwind Analyzer
Версия	1.0.0
Язык	C17
Целевая платформа анализа	Windows PE32+ AMD64/x64. Static analysis cross-platform, dynamic validation Windows x64 only.
Предмет	Exception/unwind metadata: PE sections, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handlers, chain info, placement, reuse, dynamic exception evidence.
Результат	Console output, CSV, Ghidra CSV, JSON, HTML, findings, score, suspicion, confidence, mitigations, verdict, manual validation steps.
Главная ценность	Не черный ящик, а объяснимая модель: где сломалась координатная система, карта функций, unwind metadata или handler layer.

Этот документ описывает весь проект целиком. Memory protection рассматривается только как один из разделов, а не как центральная тема.

2. Назначение проекта

Проект создан для анализа той части PE32+ x64 файла, которая отвечает за exception handling и stack unwinding. На Windows x64 механизм обработки исключений и построения call stack опирается на таблицу функций и UNWIND_INFO. Если эти структуры повреждены, скрыты, перенаправлены или размещены нетипично, reverse engineer может получить ложные границы функций, неверный call stack или не понять реальный handler path.

Обычный антивирус решает другую задачу: быстро классифицировать объект для пользователя или SOC pipeline. Этот проект не пытается заменить антивирус. Его задача - объяснить конкретный слой: что лежит в .pdata/.xdata, какие функции описаны, можно ли прочитать metadata, где handler, какие признаки говорят о повреждении или обфускации.

Поэтому вывод проекта состоит не из одного слова, а из evidence: structured findings, severity, family, affected_count, representative_rva, score, confidence, mitigations и manual steps.

3. Что проект делает и чего не делает

Область	Статус	Пояснение
PE32+ AMD64	делает	Проверяет MZ/PE, Machine 0x8664, magic 0x20b, Optional Header, sections, data directories.
PE32 x86	не делает	x86 SEH model другая; применять x64 UNWIND_INFO эвристики нельзя.
RVA/raw mapping	делает	Различает virtual layout и bytes on disk, учитывает headers и zero-fill.
RUNTIME_FUNCTION	делает	Читает таблицу, проверяет ranges, OOB, sortedness, overlaps, duplicate begins.
UNWIND_INFO	делает	Разбирает v1/v2 header, codes, operands, handler, chain, anomalies.
Memory protection	частично	Использует section characteristics для UNWIND_INFO и handler placement, но не полный PE-hardening audit.
Dynamic behavior	частично	Windows Debug API evidence по exceptions, handlers, VEH/VCH/UEF и dynamic function tables.
Full malware sandbox	не делает	Dynamic mode наблюдательный и ограниченный, не полная песочница.
Full disassembler	не делает	Не дизассемблирует весь .text; работает с PE/unwind metadata.
C++ EH language data	частично	Видит handler RVA, но не заявляет полный разбор всех compiler-specific descriptors.

4. Архитектура высокого уровня

Архитектура проекта построена как последовательный конвейер. Каждый слой получает модель предыдущего слоя и не перепрыгивает через проверки. Это важно, потому что нельзя правильно анализировать handler target, если еще не ясно, корректно ли UnwindInfoAddress мапится в файл.

Проект разделен на небольшие модули. `util.c` не знает PE-семантики, он только безопасно читает bytes и пишет файлы. `pe.c` не знает эвристик, он строит PeImage. `unwind.c` не выставляет score, он декодирует metadata. `heuristics.c` не читает файл напрямую, он анализирует готовую модель. `report.c` не меняет модель, а сериализует ее.

```
input file bytes
  -> util: safe Buf reads, little-endian values, SHA-256
  -> pe: PE32+ AMD64 headers, section table, RVA mapping
  -> unwind: exception directory, RUNTIME_FUNCTION, UNWIND_INFO
  -> heuristics: findings, score, confidence, mitigations, verdict
  -> dynamic_win: optional Windows runtime evidence
  -> report: console, CSV, Ghidra CSV, JSON, HTML
```

5. Модули проекта

Модуль	Основные функции	За что отвечает
main.c	parse_args, main, print_one, build_invocation	CLI, порядок анализа, ownership, cleanup, exit codes, вызов всех этапов.
util.c	buf_init_from_file, buf_read_*, StrBuf, file_write_atomic, sha256_hex	Безопасный ввод/вывод, little-endian чтение, строки, SHA-256, атомарная запись.
pe.c	pe_parse, pe_rva_to_fileoff, pe_find_section_for_rva	PE32+ parser, section validation, SizeOfImage/Headers, RVA->file mapping.
unwind.c	unwind_read_runtime_functions_ex, unwind_parse	Чтение .pdata, RUNTIME_FUNCTION, UNWIND_INFO, opcodes, handlers, chain info.
heuristics.c	heuristics_analyze, finalize_verdict	27 findings, score, thresholds, confidence, mitigations, evidence families.
dynamic_win.c	dynamic_collect_windows	Windows Debug API, breakpoints, exceptions, VEH/VCH/UEF, dynamic function tables.
report.c	report_write_csv/json/html, assessment_summary	Отчеты, escaping, metadata validation, manual steps, assessment.
app_info.c	app_version, app_build_*, app_utc_timestamp	Version/schema/build/timestamp metadata.

6. Главный поток выполнения main.c

Главный файл проекта важен не только как `main`. Он показывает, как все модули соединены. Сначала CLI разбирает опции: dump, limit, ops, heuristics, CSV, Ghidra CSV, JSON, HTML, follow-chain, strict ABI, dynamic, timeout, trace steps и target arguments после `--`.

Затем файл читается в `Buf`. После этого `pe_parse` строит `PeImage`. Если PE вообще некорректен, анализ прекращается. Далее `unwind_read_runtime_functions_ex` пытается прочитать exception directory. Если проблема OOM - остановка. Если directory malformed - проект продолжает с частичным structural report и выставляет `exception_directory_oob`.

После чтения function table выделяются массивы `UnwindSummary` и `us_ok`. Для каждой записи вызывается `unwind_parse`. Потом при `--heuristics` строится `HeuristicReport`. При `--dynamic` запускается Windows Debug API collector. В конце report writers получают уже готовые модели и пишут CSV/JSON/HTML.

Этап main.c	Что создается	Что освобождается
`buf_init_from_file`	owned memory + Buf	buf_free_owned
`pe_parse`	PeImage.sections	pe_free
`unwind_read_runtime_functions_ex`	RuntimeFunction* rf	free(rf)
`calloc summaries`	UnwindSummary* us, bool* us_ok	free(us), free(us_ok)
`heuristics_analyze`	HeuristicReport.details	heuristics_free
`dynamic_collect_windows`	DynamicReport.details	dynamic_report_free
`metadata/report`	StrBuf invocation, input_display	sb_free, free

7. Почему C и C17

C выбран потому, что задача низкоуровневая: parser работает с bytes, offsets, little-endian числами, alignment, integer overflow, ручным memory ownership и системным ABI. В C эти операции видны явно. Это хорошо для ВКР, потому что демонстрирует понимание бинарного формата, а не только умение вызвать библиотеку.

C++ мог бы дать классы и RAII, Python мог бы ускорить прототипирование, но текущая задача выигрывает от прозрачной C-модели: структура данных, функция чтения, проверка границы, cleanup. При этом проект не пишет небезопасный C вслепую: есть bounds checks, safe arithmetic, `StrBuf.failed`, атомарная запись, sanitizers и fuzz/mutation tests.

8. Сборка и инженерная дисциплина

Проект собирается CMake 3.23+ как C17 project. Основной статический library target - `ehua_core`, executable - `eh_unwind_analyzer`. Включены строгие предупреждения: MSVC `/W4 /permissive- /utf-8`, GCC/Clang `-Wall -Wextra -Wpedantic -Wconversion -Wshadow -Wformat=2`. При включенном `EHUA_WARNINGS_AS_ERRORS` предупреждения становятся ошибками.

Есть профили sanitizer, coverage и optional libFuzzer harness. Sanitizer и coverage одновременно запрещены, потому что это разные режимы инструментирования. На Windows проверяется 64-bit build. Для MinGW выставляется `__USE_MINGW_ANSI_STDIO`, чтобы printf formatting был согласован.

CMake option	Смысл
`EHUA_WARNINGS_AS_ERRORS`	Считать warnings ошибками.
`EHUA_ENABLE_SANITIZERS`	AddressSanitizer + UndefinedBehaviorSanitizer для GCC/Clang.
`EHUA_ENABLE_COVERAGE`	Coverage instrumentation.
`EHUA_BUILD_FUZZER`	Сборка Clang libFuzzer parser harness.
`BUILD_TESTING`	Unit tests, regression samples, validators, mutation fuzz.

9. util.c: фундамент безопасного парсинга

`util.c` - нижний слой. Его задача - сделать чтение недоверенного файла предсказуемым. Файл читается целиком в память, `Buf` хранит `data` и `size`. Любая попытка чтения проходит через `buf_read`, которая проверяет, что offset и length не выходят за границы.

Функции `buf_read_u16/u32/u64` читают little-endian значения явно. Это важнее, чем кажется: PE формат little-endian, а C struct cast зависел бы от packing, alignment и endianness машины. Такой parser был бы хрупким и опасным.

`StrBuf` нужен для накопления ошибок, JSON/HTML/CSV и details. У него есть `failed`, чтобы OOM не превращался в тихую порчу строки. `file_write_atomic` пишет через temporary sibling file и replacement, чтобы сбой записи не уничтожил предыдущий отчет. `sha256_hex` нужен для воспроизводимости отчета.

Функция	Смысл	Почему важна
`buf_read`	Проверяет диапазон и копирует bytes.	Главная защита от OOB read.
`buf_read_u16/u32/u64`	Читает little-endian числа.	Не зависит от struct packing.
`sb_append/sb_appendf`	Наращивает строку с контролем failed.	Отчеты и errors не ломаются при OOM.
`file_write_atomic`	Пишет файл через временный sibling.	Не портит старый отчет при interrupted write.
`sha256_hex`	Считает SHA-256 input.	Report provenance и reproducibility.

10. pe.c: построение PE-модели

`pe_parse` проверяет DOS header MZ, `e_lfanew`, PE signature, COFF header, Machine, Optional Header и section table. Проект поддерживает только AMD64: `IMAGE_FILE_MACHINE_AMD64 = 0x8664`. Optional Header должен быть PE32+: magic `0x20b`.

Из Optional Header читаются `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `SizeOfHeaders`, `NumberOfRvaAndSizes`, Exception Data Directory. Затем читаются section headers: name, VirtualSize, VirtualAddress, SizeOfRawData, PointerToRawData, Characteristics.

После чтения секций проект не доверяет им слепо. Он считает raw OOB, virtual OOB, raw overlaps, virtual overlaps, misalignment. Это PE validation layer, без которого нельзя надежно рассуждать о RUNTIME_FUNCTION или UNWIND_INFO.

Проверка pe.c	Зачем
`SizeOfImage != 0`	Без виртуальной границы нельзя проверять RVA.
`SizeOfHeaders > 0 и <= file_size`	Headers должны физически существовать в файле.
`SectionAlignment/FileAlignment != 0`	Выравнивание необходимо для image/file layout.
`DataDirectory достаточно большой`	Exception Directory index 3 должен существовать.
`section raw_end <= file_size`	Секция не должна заявлять несуществующие bytes.
`section virtual_end <= SizeOfImage`	Секция должна помещаться в image.
`raw/virtual overlaps`	Один диапазон не должен иметь неоднозначную интерпретацию.
`SizeOfImage % SectionAlignment`	Image layout должен быть согласован с alignment.
`ImageBase 64K alignment`	Нормальное Windows image base выравнивание.

11. RVA, VA, file offset и zero-fill

RVA, VA и file offset - разные сущности. RVA - относительный виртуальный адрес внутри образа. VA - реальный адрес в процессе: `ImageBase + RVA`. File offset - смещение от начала файла на диске. Большая часть ошибок в PE parsers возникает из-за смешивания этих понятий.

Проект учитывает, что виртуальная раскладка image и физическое расположение bytes различаются. Секция может иметь больший VirtualSize, чем SizeOfRawData. Тогда loader создаст zero-fill хвост в памяти, но в файле этих bytes нет. Статический анализатор не имеет права читать этот хвост из файла.

pe_find_section_for_rva отвечает на вопрос 'какой virtual section содержит RVA'. pe_rva_to_fileoff отвечает на другой вопрос: 'можно ли прочитать bytes по этому RVA из файла'. Если delta >= raw_size, mapping отклоняется. Если RVA попадает в headers (RVA < SizeOfHeaders), он мапится напрямую.

```
RVA classification:
  section span = [VirtualAddress, VirtualAddress + max(VirtualSize, SizeOfRawData))

Static file read:
  delta = RVA - VirtualAddress
  if delta >= SizeOfRawData: bytes are virtual zero-fill, not present in file
  else file_offset = PointerToRawData + delta

Headers:
  if RVA < SizeOfHeaders and RVA < file_size: file_offset = RVA
```

12. Границы проекта: file, image, section, function, unwind

Граница	Источник	Что проверяет
file_size	Реальный размер файла	Нельзя читать bytes за этой границей.
SizeOfImage	Optional Header	RVA image structures должны находиться внутри virtual image.
SizeOfHeaders	Optional Header	Headers являются отдельной RVA-областью.
Section raw range	PointerToRawData + SizeOfRawData	Физическое размещение секции.
Section virtual range	VirtualAddress + max(VirtualSize, RawSize)	Virtual placement секции.
Function range	BeginAddress/EndAddress	Границы функции в RUNTIME_FUNCTION.
Unwind bytes	UNWIND_INFO header/codes/tail	Структура должна физически читаться.
Protection bits	Section characteristics	Execute/write placement metadata/handler.

13. Section characteristics и memory protection

Memory protection - это права страниц памяти: read, write, execute. В PE эти свойства отражены через section characteristics. Проект читает characteristics каждой секции и использует биты IMAGE_SCN_MEM_EXECUTE и IMAGE_SCN_MEM_WRITE.

Но это не полный аудит всех секций. Проект не выдает отдельный finding за .text writable или .rdata executable, если эти секции не участвуют в unwind/handler placement. Это важно: предмет ВКР - EH/unwind metadata, а не общий PE-hardening.

Что реально проверяется: где лежит UNWIND_INFO, executable ли эта секция, writable ли она, похоже ли имя секции на .xdata/.rdata/.pdata; куда указывает HandlerRVA, executable ли target, writable ли target; есть ли combo вроде writable metadata + writable handler.

Случай	Finding
UNWIND_INFO в executable секции	unwind_in_exec_section
UNWIND_INFO в writable секции	unwind_writable_section
UNWIND_INFO не в .xdata/.rdata/.pdata-like секции	unwind_unusual_section
HandlerRVA в non-executable/unknown секции	handler_nonexec_section
HandlerRVA в writable секции	handler_writable_section
Writable metadata + writable handler	writable_metadata_handler_combo
.text writable, но не участвует в EH/unwind	отдельного finding нет
Runtime VirtualProtect меняет protection	прямого finding нет

14. Exception Directory и .pdata

Exception Directory - это Data Directory index 3. В PE32+ x64 он обычно указывает на `.pdata`, где лежит массив `RUNTIME_FUNCTION`. Каждая запись занимает 12 bytes: BeginAddress, EndAddress, UnwindInfoAddress.

Проект проверяет, что directory имеет согласованные RVA/Size. Оба нуля означают absence. Один ноль и другой не ноль - malformed. RVA должен мапиться в file offset. `off + size` должен помещаться в file_size. Size должен быть не меньше 12 и кратен 12.

Если directory malformed из-за структурной ошибки, проект выставляет `exception_directory_oob` и может продолжить частичный отчет. Это лучше, чем просто завершиться без объяснения, потому что сама unreadable directory является evidence.

15. RUNTIME_FUNCTION как карта функций

`RUNTIME_FUNCTION` связывает диапазон кода функции с UNWIND_INFO. Диапазон `[BeginAddress, EndAddress)` должен быть корректным, не пустым и находиться внутри `SizeOfImage`. `UnwindInfoAddress` должен указывать на metadata.

Таблица должна быть пригодной для поиска по BeginAddress. Поэтому проверяются sortedness, overlaps и duplicate begins. Если один RIP может принадлежать нескольким функциям, virtual unwind становится неоднозначным.

Поле	Нормальный смысл	Проблемы
`BeginAddress`	RVA начала функции	За SizeOfImage, duplicate begin, Begin >= End
`EndAddress`	RVA конца функции	End <= Begin, End > SizeOfImage, overlap
`UnwindInfoAddress`	RVA UNWIND_INFO	0, OOB, not mappable, unusual/writable/executable placement

16. unwind.c: декодирование UNWIND_INFO

`unwind_parse` читает UNWIND_INFO по RVA. Сначала RVA мапится в file offset. Затем читается 4-byte header: version, flags, prolog size, count codes, frame register, frame offset. Версия должна быть 1 или 2.

Массив unwind codes хранится как 2-byte slots, причем общее число slots выравнивается до четного. Каждый opcode имеет свое число slots. Parser проверяет, не выходит ли opcode за `CountOfCodes`, и только после этого читает operands.

Parser не просто печатает текст. Он строит `UnwindSummary`: fixed_stack_bytes, pushed_nonvol_count, saved_nonvol_mask, saved_xmm_mask, has_handler, handler_rva, has_chain, chain, anomalies, parse_error.

UWOP	Что означает
`PUSH_NONVOL`	Сохранение nonvolatile GPR через push.
`ALLOC_LARGE / ALLOC_SMALL`	Выделение stack space.
`SET_FPREG`	Установка frame pointer.
`SAVE_NONVOL / FAR`	Сохранение nonvolatile GPR по offset.
`SAVE_XMM / SAVE_XMM128`	Сохранение XMM-регистров.
`PUSH_MACHFRAME`	Machine frame при interrupt/exception context.
`v2 EPILOG / SPARE`	Поддержка UNWIND_INFO version 2 descriptors.

17. Anomalies и strict ABI

Проект разделяет parse failure и anomaly. Parse failure означает, что структура не может быть надежно прочитана. Anomaly означает, что структура прочитана, но внутри есть странности: unknown version, bad opcode, bad operand, code offset outside prolog, chain loop, handler OOB, alignment issue.

`--strict-abi` добавляет формальные Microsoft x64 ABI diagnostics: register class, reserved opinfo, push order, chain restriction, save offset. Важно: strict ABI warnings видимы, но сами по себе не повышают suspicion score,

потому что реальные toolchains могут иметь совместимые нестандартности.

Аномалия	Смысл
UA_TRUNCATED	Структура обрезана или выходит за файл.
UA_BAD_OPCODE	Невалидный opcode или недопустимое сочетание version/opcode.
UA_BAD_OPERAND	Некорректные flags, frame register, operand.
UA_CODEOFFSET_OUTSIDE	CodeOffset больше prolog size.
UA_CHAIN_LOOP / DEPTH	Chained info зациклен или слишком глубок.
UA_HANDLER_OOB	HandlerRVA выходит за SizeOfImage.
UA_ALIGNMENT	UNWIND_INFO RVA не выровнен на 4 bytes.
UA_ABI_*	Strict-only diagnostics.

18. Handler layer

Handler layer появляется, когда flags содержат UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER. После unwind codes, с учетом выравнивания, находится HandlerRVA. Это потенциальный control-flow target при exception handling.

Проект проверяет не только наличие handler, но и его placement. Handler должен указывать в executable code. Если секция unknown или non-executable, появляется finding. Если handler target writable, это отдельный finding. Если один handler обслуживает множество функций, считается handler dominance.

Language-specific data после handler не разбираются полностью, потому что это зависит от runtime/compiler. Проект честно ограничен structural handler pointer and placement analysis.

19. CHAININFO

UNW_FLAG_CHAININFO означает, что tail UNWIND_INFO содержит chained RUNTIME_FUNCTION, а не handler RVA. Это нормальный механизм, но опасный для parser: chain может быть некорректным, зацикленным или бесконечным.

Проект ограничивает chain depth: default 8, CLI 1..16. Он запоминает посещенные unwind RVA и фиксирует loop/depth anomalies. При follow-chain summary дочерней metadata добавляется к текущей, но это не превращается в полноценный graph analysis.

20. heuristics.c: модель evidence

`heuristics_analyze` получает уже разобранные структуры: `PeImage`, массив `RuntimeFunction`, массив `UnwindSummary`, массив `us_ok`. Он не читает файл напрямую и не перепарсивает PE. Его задача - превратить structural facts в findings и score.

Сначала копируются PE counters: raw OOB, virtual OOB, overlaps, misalignment, exception directory OOB. Затем проход по function table считает invalid ranges, RF OOB, UnwindInfoRVA OOB, unsortedness, parse failures, anomalies, ABI warnings, placement, handler counts и reuse maps.

Для reuse/dominance используются hash maps по UnwindInfoRVA и HandlerRVA. После прохода таблица сортируется по BeginAddress для проверки overlaps и duplicate begins. Затем начисляются score и findings по правилам evidence matrix.

Family	Что покрывает
`table_integrity`	PE layout, exception directory, RUNTIME_FUNCTION ranges, OOB, overlaps, sortedness.
`unwind_integrity`	UNWIND_INFO parse failures and anomalies.
`metadata_placement`	Section names/permissions for UNWIND_INFO and handlers.
`unwind_reuse`	Массовое повторное использование одного UnwindInfoRVA.
`handler_dominance`	Доминирование одного HandlerRVA.

21. Все 27 findings

Finding	Family	Что означает	Почему важно
pe_section_raw_oob	table_integrity	Raw-диапазон секции выходит за размер файла.	Секция заявляет bytes за пределами file_size. Это ломает физическое чтение из файла.
pe_section_virtual_oob	table_integrity	Virtual-диапазон секции выходит за SizeOfImage.	Секция заявляет RVA-область за пределами виртуального image.
pe_section_raw_overlap	table_integrity	Raw-диапазоны секций пересекаются.	Одни bytes файла имеют несколько секционных интерпретаций.
pe_section_virtual_overlap	table_integrity	Virtual-диапазоны секций пересекаются.	Один RVA может иметь неоднозначные section permissions/role.
pe_section_misaligned	table_integrity	Нарушены alignment/header инварианты.	Проверяются FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders, order и ImageBase alignment.
pe_exception_directory_oob	table_integrity	Exception directory битая или недоступная.	Directory не помещается, не мапится, неполная или имеет некратный размер.
no_runtime_functions	table_integrity	В executable x64 image нет RUNTIME_FUNCTION.	Для обычного x64 кода это странно, но leaf-only/resource cases возможны.
invalid_ranges	table_integrity	EndAddress <= BeginAddress.	Function table entry не задает корректный полуоткрытый диапазон функции.
rf_oob	table_integrity	Begin/End функции выходят за SizeOfImage.	Function table ссылается на код вне виртуального образа.
unwind_rva_oob	table_integrity	UnwindInfoAddress недоступен.	RVA нулевой, вне image или не читается из raw bytes.
rf_unsorted	table_integrity	Таблица не отсортирована по BeginAddress.	Нарушается ожидание пригодной для поиска function table.
rf_overlaps	table_integrity	Диапазоны функций пересекаются.	Один RIP может относиться к нескольким функциям.
duplicate_begin	table_integrity	Несколько функций имеют одинаковый BeginAddress.	Конфликт идентичности функции в таблице.
unwind_parse_fail	unwind_integrity	UNWIND_INFO не разобран.	Metadata заявлена, но header/codes/tail недоступны или обрезаны.
unwind_anomalies	unwind_integrity	UNWIND_INFO разобран, но содержит anomalies.	Версия, opcode, operands, code offsets, chain, handler или alignment выглядят странно.
unwind_reuse_strong	unwind_reuse	Массовое reuse одного UnwindInfoRVA.	При n > 200 и ratio > 0.30 это похоже на centralized metadata pattern.
handler_dominance	handler_dominance	Один handler доминирует.	При handler_count > 50 и ratio > 0.80 один target обслуживает почти все функции.
unwind_reuse_moderate	unwind_reuse	Умеренное reuse, подтвержденное другим сигналом.	При ratio > 0.10 и n > 200 finding появляется только при корреляции.
unwind_in_exec_section	metadata_placement	UNWIND_INFO лежит в executable секции.	Metadata оказалась в области, предназначенной для кода.
unwind_unusual_section	metadata_placement	UNWIND_INFO лежит в необычной секции.	Секция не похожа на .xdata/.rdata/.pdata или родственные им имена.
unwind_writable_section	metadata_placement	UNWIND_INFO лежит в writable секции.	Metadata, влияющая на unwind/handler, потенциально изменяема.
handler_nonexec_section	metadata_placement	HandlerRVA указывает в non-executable/unknown секцию.	Handler должен быть code path; non-exec target требует проверки.
handler_writable_section	metadata_placement	HandlerRVA указывает в writable секцию.	Endpoint обработки исключений потенциально изменяем.
placement_exec_nonexec_handler_combo	metadata_placement	Executable metadata + non-exec handler.	Две placement странности на соседних слоях усиливают друг друга.
placement_integrity_combo	unwind_integrity	Placement anomaly + integrity problem.	Странное размещение подтверждается parse/range проблемами.

Finding	Family	Что означает	Почему важно
`reuse_dominance_combo`	unwind_reuse	Reuse metadata + dominant handler.	Похоже на centralized dispatcher/interceptor style.
`writable_metadata_handler_combo`	metadata_placement	Writable metadata + writable handler.	Mutable metadata и mutable control-flow endpoint одновременно.

22. Почему именно эти признаки

Признаки отбирались не для количества, а для покрытия цепочки доверия. Нижний слой - PE layout. Затем exception directory и function table. Затем UNWIND_INFO. Затем handler placement. Затем reuse/dominance и combo correlations.

Если убрать PE section findings, можно ошибочно доверять координатной системе. Если убрать function table findings, можно пропустить поврежденную карту функций. Если убрать unwind findings, нельзя понять, применима ли metadata. Если убрать placement findings, пропадут executable/writable/unusual секции. Если убрать reuse/dominance, пропадут централизованные dispatcher-style patterns.

Добавлять признаки вроде общего `.text writable` можно, но это уже другой продукт - PE hardening analyzer. В текущем проекте признак должен отвечать на вопрос: влияет ли он на доверие к EH/unwind механизму?

23. Score, thresholds, confidence, mitigations

Score - engineering suspiciousness 0..100. Это не ML probability. Вес зависит от тяжести нарушения и риска false positive. Нарушения, которые ломают чтение или function map, имеют высокий вес. Contextual placement/reuse signals имеют средний или небольшой вес.

Пороги тоже инженерные. 1 процент отделяет единичный glitch от pattern для table OOB/ranges. 5 процентов отделяют точечные unwind anomalies от массовой проблемы. Reuse 10/30 процентов работает только при n > 200, чтобы маленькая выборка не давала ложный вывод. Handler dominance требует handler_count > 50 и ratio > 0.80.

Confidence отделен от score. Он растет при нескольких evidence families, strong/medium findings и hard table/unwind issues. Он снижается mitigations: single_signal_only, small_sample_size, placement_only_signal, single_family_only. Итоговый confidence ограничен 0.05..0.90.

Score range	Verdict	Смысл
`0..14`	no_strong_static_anomalies	Нет сильных static EH/unwind anomalies.
`15..39`	low_suspicion	Есть слабые/контекстные признаки.
`40..69`	moderate_suspicion	Несколько серьезных или коррелирующих признаков.
`70..100`	high_suspicion	Сильная static картина повреждения/обфускации EH/unwind.

24. dynamic_win.c: runtime evidence

Dynamic mode запускает target под Windows Debug API. Это опасная граница: файл реально исполняется, поэтому untrusted samples должны запускаться в изолированной VM. Цель dynamic mode - не sandbox всего поведения, а сбор evidence по exception handling.

Collector ведет список модулей, ставит software breakpoints, наблюдает Debug API events, отслеживает exceptions, first/second chance flow, handler hits, VEH/VCH/UEF registrations, RtlAddFunctionTable и RtlInstallFunctionTableCallback. После handler hit может собираться bounded RIP trace.

Breakpoints требуют re-arm logic: после INT3 надо восстановить оригинальный byte, сдвинуть RIP, выполнить оригинальную инструкцию и поставить breakpoint снова. Проект ведет counters: attempts, failures, capacity drops, pending rearm failures, dropped events.

Dynamic field/event	Смысл
`exceptions_seen / first_chance / second_chance`	Сколько exception events видел debugger.
`unwind_handler_bp_set / hits`	Breakpoints на handlers из static UNWIND_INFO и их срабатывания.
`veh/vch/uef registrations/hits`	Vectored/unhandled exception handler layer.
`rtl_add_function_table_calls`	Runtime function tables, которых могло не быть on disk.
`instrumentation_complete`	Насколько полно удалось поставить instrumentation.
`trace`	Ограниченная цепочка RIP после handler hit.

25. report.c: отчеты и assessment

`report.c` отвечает за вывод, но не должен менять модель. Он печатает brief, CSV, Ghidra CSV, JSON и HTML. CSV дает flat table, Ghidra CSV помогает импортировать функции и comments, JSON дает machine-readable модель, HTML дает удобный self-contained dashboard.

Report metadata включает version, schema, timestamp UTC, SHA-256 input, input_size, command_line, build compiler/platform/mode, strict_abi, follow_chain, max_chain_depth. Это нужно для воспроизводимости.

JSON/HTML writers валидируют metadata и model bounds. Строки экранируются. HTML имеет offline CSP и не тянет remote assets. Assessment layer объединяет static evidence, runtime evidence и correlation в case_strength.

Формат	Для чего
`Console`	Быстрый запуск и краткий вывод.
`CSV`	Табличная обработка function/unwind data.
`Ghidra CSV`	Импорт ranges/comments в reverse-engineering workflow.
`JSON`	Автоматизация, schema validation, semantic validation.
`HTML`	Человеческий self-contained отчет с русскими пояснениями и manual steps.

26. Manual validation steps

Проект не должен оставлять пользователя с голым score. Manual validation steps говорят, что делать дальше. Если есть table_integrity issues - проверить section ranges и .pdata. Если metadata placement - проверить section permissions и role. Если handler dominance - открыть dominant handler. Если dynamic handler hit - сопоставить runtime address и static RVA.

Это делает проект полезным для reverse engineer: отчет превращается в карту дальнейшей ручной работы.

27. app_info.c и воспроизводимость

`app_info.c` возвращает version, report schema version, compiler, platform, build mode и UTC timestamp. Это маленький модуль, но он важен для сдачи: отчет должен объяснять, какой версией инструмента и с какими build settings был получен результат.

28. CLI и пользовательские режимы

Опция	Что делает
`--dump`	Печатает RUNTIME_FUNCTION entries.
`--limit N`	Ограничивает число dump-записей.
`--ops`	Показывает decoded unwind operations.
`--heuristics`	Запускает heuristic assessment.
`--csv`	Пишет full CSV.
`--ghidra-csv`	Пишет CSV для Ghidra importer.
`--json`	Пишет JSON report.
`--html`	Пишет self-contained HTML report.
`--no-follow-chain`	Отключает рекурсивное следование CHAININFO.
`--strict-abi`	Включает formal ABI diagnostics.
`--max-chain-depth`	Задает chain depth 1..16.
`--dynamic`	Запускает Windows dynamic validation.
`--timeout-ms`	Задает hard timeout dynamic session.
`--trace-steps`	Задает RIP samples после handler hit.
`--`	Все после этого - аргументы target process.

29. Тестирование

Проект имеет несколько уровней тестирования. Unit tests проверяют unwind, heuristics, mitigations, combos, edge cases, report JSON/HTML. Regression samples проверяют реальные PE32+ образцы: benign_minimal, benign_richer, suspicious_eh_unwind_tamper, suspicious_section_raw_oob, suspicious_unwind_oob, suspicious_unwind_writable.

Есть JSON schema и semantic validation. Есть mutation fuzz на 120 deterministic parser mutations. Есть dynamic validator tests и Windows smoke scenarios для VEH/VCH/UEF/RtlAddFunctionTable. Есть release metadata/hygiene checks. Есть sanitizer, coverage и optional libFuzzer.

Тестовый слой	Что ловит
`unit_unwind`	UNWIND_INFO parsing, opcodes, handlers, chain.
`unit_heuristics*`	Findings, score, mitigations, combos, context.
`unit_report_json/html`	Serialization, escaping, metadata.
`regression_samples`	Поведение на corpus PE samples.
`report_semantics`	Cross-field invariants JSON report.
`mutation_fuzz`	Robustness на поврежденных parser inputs.
`dynamic smoke`	Windows runtime handler/API evidence.

30. Ghidra workflow

Ghidra CSV нужен, чтобы результаты analyzer не остались только в отчете. Пользователь генерирует `--ghidra-csv`, затем запускает importer в Ghidra. Importer валидирует columns/ranges, использует transaction, skips overlapping functions, сохраняет existing user/imported names, добавляет comments/bookmarks для parse failures или anomalies.

Это важная часть целевой аудитории: reverse engineer получает не только текстовый отчет, но и перенос данных в рабочий инструмент.

31. Практические сценарии анализа

Сценарий	Типичный результат	Что делать
`Нормальный MSVC/MinGW PE`	Низкий score, metadata в .xdata/.rdata, handlers в .text.	Использовать как baseline.
`Обрезанный файл`	raw_oob, exception_directory_oob, parse_fail.	Проверить sample integrity.
`Packer/protector`	unusual/writable/executable placement, reuse, handler dominance.	Открыть dominant handler, включить dynamic mode.
`Runtime function tables`	Static может быть чистым, dynamic ловит RtlAddFunctionTable.	Сопоставить runtime table base/count.
`Resource-only image`	Нет executable sections, no runtime functions как note.	Не завышать вывод.

32. Ограничения проекта

Проект честно ограничен PE32+ AMD64. Он не анализирует PE32 x86, ARM64, ELF, Mach-O. Он не является полноценным loader, не делает import resolution, relocation emulation, TLS execution, unpacking, deobfuscation или full disassembly.

Проект не делает полный аудит всех memory protections. Он проверяет section characteristics только в контексте UNWIND_INFO и handler placement. Он не делает runtime VirtualQueryEx snapshot и не отслеживает все VirtualProtect changes.

Dynamic mode исполняет target и потому требует изоляции. Он не является malware sandbox. Он собирает bounded EH/unwind-related evidence.

Score - инженерная шкала, не статистическая вероятность. Verdict - триаж EH/unwind suspiciousness, не доказательство malware/benign.

33. Почему проект сложнее обычного CRUD/сайта

В CRUD-сайте данные обычно приходят через формы и хранятся в базе по известной схеме. В этом проекте вход - hostile binary file, где каждый offset, size, RVA и count может быть специально поврежден. Ошибка parser logic дает crash или ложный security finding.

Сложность проекта в согласовании нескольких координатных систем: file offset, RVA, VA, headers, raw section range, virtual section range, function range, unwind bytes, handler target. Ошибка между ними меняет смысл анализа.

Еще одна сложность - explainability. Проект не просто считает число, а хранит findings, families, severity, representative RVA, affected count, confidence, mitigations, manual steps, dynamic correlation и разные форматы отчетов.

34. Почему проект нужен при наличии Windows/антивирусов

Windows использует эти структуры, но не объясняет их исследователю как отчет. Антивирус может быть закрытым verdict-движком. Проект делает другое: показывает, какие PE/unwind/handler структуры выглядят несогласованно, и дает проверяемое объяснение.

Для reverse engineer и malware analyst полезно видеть не только 'подозрительно', а почему: exception directory unreadable, RUNTIME_FUNCTION overlap, UnwindInfo in writable section, handler non-executable, strong reuse, dominant handler. Это сокращает ручной анализ.

35. Ответы комиссии

Вопрос	Ответ
`Почему C?`	Потому что задача про bytes, offsets, endian, ABI, overflow и memory ownership. C показывает это явно.
`Почему не Python?`	Python удобен для прототипа, но скрыл бы часть низкоуровневых решений и контроля памяти.
`Почему только x64?`	Windows x64 имеет RUNTIME_FUNCTION/UNWIND_INFO модель. x86 SEH требует отдельного проекта.
`Почему 27 findings?`	Они покрывают цепочку доверия PE layout -> function table -> unwind -> handler -> correlation.
`Почему не все PE признаки?`	Фокус ВКР - EH/unwind metadata, а не общий PE-hardening scanner.
`Это антивирус?`	Нет. Это explainable analyzer для reverse-engineering triage.
`Score доказывает malware?`	Нет. Score показывает suspiciousness EH/unwind модели и требует manual review.
`Права памяти проверяются?`	Да, но только для UNWIND_INFO/handler placement, не как полный audit всех секций.

36. Итог

EH/Unwind Analyzer - это системный C17-проект для explainable анализа Windows PE32+ x64 EH/unwind metadata. Он безопасно читает недоверенный PE-файл, строит PE-модель, различает RVA/raw/header/zero-fill, валидирует sections и exception directory, читает RUNTIME_FUNCTION, декодирует UNWIND_INFO, анализирует handler layer, формирует 27 findings, score, confidence, mitigations и verdict, при необходимости собирает Windows Debug API evidence и выпускает воспроизводимые отчеты. Его предмет - доверие к EH/unwind слою, а не замена антивируса, loader, full disassembler или sandbox.

37. Как мыслить проект целиком

Проект надо понимать не как набор независимых проверок, а как цепочку восстановления доверия к нескольким координатным системам. На входе есть только bytes файла. Эти bytes сами по себе ничего не гарантируют: любой размер, offset, RVA, count или флаг мог быть испорчен. Поэтому первая задача проекта - не найти вирус, а построить безопасную модель файла, где каждое обращение к bytes предварительно проверено.

После чтения bytes проект постепенно поднимается по слоям. Сначала он доказывает, что файл вообще похож на PE32+ AMD64. Затем строит карту секций. Затем переводит RVA в file offset только там, где такие bytes реально есть на диске. Затем читает Exception Directory как массив RUNTIME_FUNCTION. Затем для каждой функции читает UNWIND_INFO. Затем оценивает, где лежат metadata и handlers, не пересекаются ли диапазоны, не слишком ли много функций ссылаются на один и тот же unwind или handler. В конце проект не говорит магическое 'bad', а показывает evidence.

Такая архитектура важна, потому что ошибка на раннем слое заражает все последующие выводы. Если RVA неправильно переведен в file offset, то decoder UNWIND_INFO прочитает чужие bytes. Если SizeOfImage игнорируется, функция может казаться валидной, хотя указывает за пределы image. Если секции пересекаются, один и тот же RVA может получить разные permissions. Поэтому проект постоянно разделяет: что известно из headers, что реально читается из файла, что предполагается loader-ом, а что требует ручной проверки.

Главная идея: проект не пытается угадать намерение автора бинарника. Он проверяет согласованность PE/unwind модели и показывает, где доверие к этой модели ломается.

38. Полный поток данных

Шаг	Что принимает	Что строит	Что может пойти не так
`1. File bytes`	Путь к EXE/DLL	Buf: указатель + размер	Файл не читается, слишком мал, поврежден.
`2. PE parser`	Buf	PeImage: machine, image base, headers, sections, exception directory	Не PE, не x64, не PE32+, OOB headers, неверные alignment.
`3. RVA mapper`	PeImage + RVA	file offset или отказ	RVA попал в zero-fill tail, вне image, вне секций и headers.
`4. Runtime table reader`	Exception Directory	Массив RuntimeFunction	Directory не кратна 12, не мапится, выходит за raw bytes.
`5. Unwind decoder`	UnwindInfoRVA	UnwindSummary	Bad version, truncated slots, bad opcode, bad handler/chain tail.
`6. Heuristics`	PeImage + rf + summaries	27 findings, score, confidence, mitigations	Сигналы могут быть одиночными или требовать корреляции.
`7. Dynamic mode`	Static model + запуск target	DynamicReport	Только Windows, target может не пройти нужный exception path.
`8. Reports`	Все модели	Console, CSV, Ghidra CSV, JSON, HTML	Нужно экранировать строки и сохранить cross-field consistency.

39. Где проходит граница доверия

В проекте есть несколько trust boundaries. Первая - между файловой системой и parser-ом: файл полностью недоверенный, поэтому `buf_read` проверяет, что `off + len` не выходит за `b->size`. Это базовая защита от чтения за пределами массива.

Вторая граница - между PE headers и реальностью. PE header может заявить, что секция начинается в одном месте и имеет огромный размер. Проект не обязан верить этому слепо. Он фиксирует raw OOB, virtual OOB, overlaps и misalignment как признаки нарушения модели.

Третья граница - между virtual image и физическим файлом. Loader Windows может создать bytes в памяти, которых нет в файле, например zero-fill часть секции. Static parser не может прочитать эти bytes из файла, поэтому `pe_rva_to_fileoff` возвращает отказ, а не делает вид, что данные существуют.

Четвертая граница - dynamic mode. Статический анализ не исполняет target. Dynamic mode уже запускает target под debugger-ом, поэтому это отдельный режим с timeout, event limits, trace limits и явной пометкой в отчете.

40. Модульная архитектура в реальном коде

Модуль	Ключевые структуры/функции	Зачем нужен
`util.c / util.h`	`Buf`, `StrBuf`, `buf_read`, `file_write_atomic`, `sha256_hex`	Фундамент безопасного чтения, динамических строк, атомарной записи отчетов и воспроизводимости.
`pe.c / pe.h`	`PeImage`, `PeSection`, `pe_parse`, `pe_rva_to_fileoff`, `pe_find_section_for_rva`	Строит PE-модель и считает структурные нарушения секций и Exception Directory.
`unwind.c / unwind.h`	`RuntimeFunction`, `UnwindSummary`, `unwind_read_runtime_functions_ex`, `unwind_parse`	Читает .pdata и декодирует UNWIND_INFO, включая flags, slots, handler и chain.
`heuristics.c / heuristics.h`	`HeuristicReport`, `HeuristicFinding`, `heuristics_analyze`	Превращает структурные факты в findings, score, confidence, mitigations и verdict.
`dynamic_win.c / dynamic.h`	`DynamicReport`, `DynamicEvent`, `dynamic_collect_windows`	Собирает runtime evidence через Windows Debug API: exceptions, handlers, VEH/VCH/UEF, dynamic function tables.
`report.c / report.h`	`ReportMetadata`, report writers, assessment helpers	Сериализует результаты, строит assessment, manual validation steps, HTML/JSON/CSV.
`main.c`	`Args`, `parse_args`, `main`	Оркестратор: CLI, порядок вызовов, ownership, cleanup, exit status.
`app_info.c / app_info.h`	version/schema/compiler/platform/timestamp	Добавляет воспроизводимую metadata в отчеты.

41. Ownership и управление памятью

Проект написан на C, поэтому архитектура должна явно отвечать на вопрос: кто владеет памятью и кто ее освобождает. `buf_init_from_file` выделяет память под bytes файла, а `buf_free_owned` освобождает ее. `pe_parse` выделяет массив sections, а `pe_free` освобождает его. `unwind_read_runtime_functions_ex` возвращает heap array, который освобождает caller. `heuristics_analyze` и `dynamic_report_init` владеют внутренними StrBuf, которые освобождаются отдельными free-функциями.

В `main.c` порядок cleanup построен так, чтобы ранняя ошибка не оставляла выделенную память. Это не косметика. В parser-е hostile binary inputs часто приводят к ранним выходам: сломан DOS header, bad optional header, OOM на sections, invalid exception directory. Каждый такой путь должен корректно освобождать уже созданные объекты.

Это один из признаков инженерной сложности проекта: здесь нельзя надеяться на garbage collector. Надо проектировать lifetime объектов так же внимательно, как и алгоритмы анализа.

42. Почему именно PE32+ AMD64

Проект специально ограничен PE32+ AMD64. PE32+ означает 64-bit Optional Header, его magic равен `0x20b`. AMD64 означает COFF Machine `0x8664`. Эти два поля проверяются в `pe_parse`, и если они не совпадают, проект не продолжает анализ.

Это правильное ограничение, потому что Windows x64 exception handling устроен не так, как x86 SEH. В x64 есть Exception Directory с RUNTIME_FUNCTION, а каждая запись указывает на UNWIND_INFO. В x86 другая модель обработчиков и стековых frames, поэтому те же признаки были бы либо неверными, либо вводили бы комиссию в заблуждение.

Таким образом, отказ от PE32/x86 - не недоработка, а честное сужение предметной области. Проект лучше глубоко и корректно анализирует один ABI, чем поверхностно заявляет поддержку всего.

43. Проверка machine, magic, data directory и section table

Слой PE	Что проверяется	Почему важно
`DOS header`	MZ magic и `e_lfanew`	Без этого нельзя найти NT headers. Если offset выходит за файл, все дальнейшие чтения опасны.
`NT signature`	`PE\0\0`	Подтверждает, что по `e_lfanew` лежит PE header, а не случайные bytes.
`COFF Machine`	`0x8664`	Гарантирует, что применима AMD64 unwind model.
`Optional magic`	`0x20b`	Гарантирует PE32+ layout и offsets полей 64-bit Optional Header.
`NumberOfRvaAndSizes`	Должен включать Exception Directory	Если directory table короче, читать entry 3 нельзя.
`Exception DataDirectory`	RVA/Size entry 3	Это вход в RUNTIME_FUNCTION table.
`Section headers`	Каждый заголовок 40 bytes должен помещаться в файл	Иначе секционная карта частично выдумана.
`Section alignment`	FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders, order	Эти поля задают соответствие между disk layout и memory image.

Да, проект проверяет согласованность machine/magic/data directory/section table. Часть нарушений останавливает parser, часть фиксируется как findings через counters в PeImage.

44. Что такое границы в этом проекте

Когда в признаках говорится 'выход за границы', речь не об одной универсальной границе. В PE/unwind анализе есть несколько разных границ, и каждая защищает свою координатную систему.

Граница	Кто ее задает	Что значит выход
`file_size`	Фактический размер файла на диске	Raw offset/size просит bytes, которых в файле нет.
`SizeOfImage`	PE Optional Header, проверяется parser-ом	RVA структуры указывает за пределы виртуального image, который loader должен создать.
`SizeOfHeaders`	PE Optional Header	RVA в начале image может указывать в headers, а не в секцию.
`raw section range`	PointerToRawData + SizeOfRawData	Physical bytes секции пересекаются, отсутствуют или выходят за file_size.
`virtual section range`	VirtualAddress + max(VirtualSize, SizeOfRawData)	RVA секции пересекаются или выходят за image.
`function range`	BeginAddress..EndAddress в RUNTIME_FUNCTION	Функция пустая, обратная, пересекается с другой или уходит за image.
`unwind blob range`	UNWIND_INFO header/count/slots/tail	Decoder не может безопасно прочитать все заявленные bytes.
`handler target range`	HandlerRVA после unwind codes	Handler указывает за image или в секцию без execute permission.

Эти границы установлены форматом PE/COFF, Windows x64 exception handling ABI и фактическим размером файла. Разработчик проекта не придумывает сам факт границ. Он выбирает, какие нарушения считать отдельными findings и какие веса им дать.

Совпадение 'как на диске' и 'как в памяти' не означает, что virtual address обязан численно совпадать с file offset. Нужна корректная формула: RVA попадает в секцию, из него вычитается VirtualAddress, получается delta, затем delta прибавляется к PointerToRawData. Если delta больше SizeOfRawData, это virtual tail, который loader заполнит нулями, но в файле bytes нет.

45. RVA, file offset, headers и zero-fill

RVA - это адрес относительно ImageBase внутри виртуального image. File offset - физическое смещение в bytes файла на диске. Они не равны в общем случае. Например, секция может иметь VirtualAddress `0x1000`, PointerToRawData `0x400`; тогда RVA `0x1010` соответствует file offset `0x410`, если delta `0x10` лежит внутри SizeOfRawData.

`pe_find_section_for_rva` ищет секцию по виртуальному диапазону. В текущей реализации размер диапазона берется как `max(VirtualSize, SizeOfRawData)`. Это практичный компромисс для анализа реальных PE, где заголовки могут быть странными, но чтение bytes все равно дополнительно ограничено SizeOfRawData.

`pe_rva_to_fileoff` делает более строгую вещь: если RVA попал в секцию, но delta больше или равна raw_size, функция возвращает false. Это означает: в памяти такая область может существовать как zero-fill, но в файле ее нельзя прочитать. Также функция отдельно разрешает RVA внутри SizeOfHeaders: headers тоже являются частью image и могут быть легальной RVA-областью.

Это одна из важнейших тонкостей проекта: найти секцию по RVA и прочитать bytes из файла - не одно и то же действие. Проект разделяет эти операции.

46. PE validation findings: первые шесть признаков

Finding	Триггер	Смысл	Почему это серьезно
`pe_section_raw_oob`	raw_ptr + raw_size > file_size	Секция заявляет bytes за концом файла.	Static parser не может безопасно читать заявленную область.
`pe_section_virtual_oob`	VirtualAddress + span > SizeOfImage	Секция выходит за виртуальный image.	RVA могут указывать туда, где loader не обязан иметь image bytes.
`pe_section_raw_overlap`	Raw ranges двух секций пересекаются	Одни file bytes принадлежат двум секциям.	Может скрывать неоднозначную интерпретацию данных.
`pe_section_virtual_overlap`	Virtual ranges двух секций пересекаются	Один RVA попадает в несколько секций.	Permissions и роль RVA становятся неоднозначными.
`pe_section_misaligned`	alignment/order/header invariants violated	Нарушены правила раскладки PE.	Parser не может спокойно доверять секционной карте.
`pe_exception_directory_oob`	Exception Directory нулевая/неполная/OOB/некратная	Вход в .pdata поврежден.	Нельзя корректно прочитать RUNTIME_FUNCTION table.

47. Exception Directory и RUNTIME_FUNCTION

Exception Directory - это запись DataDirectory[3] в Optional Header. Она задает RVA и размер области, где для PE32+ AMD64 лежит таблица RUNTIME_FUNCTION. Обычно это .pdata, но проект не обязан опираться только на имя секции. Он берет RVA/Size из directory и пытается прочитать массив записей.

Каждая RUNTIME_FUNCTION запись занимает 12 bytes: BeginAddress, EndAddress, UnwindInfoAddress. Begin/End - это RVA диапазон функции в виртуальном image. Диапазон мыслится как полуоткрытый: Begin включительно, End не включительно. Это удобно для проверки пересечений: две функции не должны описывать один и тот же RIP.

Если Exception Directory не кратна 12 bytes, не мапится в raw bytes, выходит за файл или имеет недочитанный хвост, reader возвращает malformed. `main.c` не падает, а помечает `img.exception_directory_oob = true` и пытается сделать частичный структурный отчет. Это хороший инженерный выбор: даже битый объект может дать полезное evidence.

48. Проверки таблицы функций

Finding	Проверка	Логика
`no_runtime_functions`	n == 0, Exception Directory пустая, но есть executable section	Для x64 image с кодом отсутствие .pdata может быть странным, но leaf-only case возможен, поэтому это medium.
`invalid_ranges`	EndAddress <= BeginAddress	Запись не описывает нормальную функцию: размер нулевой или отрицательный.
`rf_oob`	Begin >= SizeOfImage или End > SizeOfImage	Function table ссылается на код вне image.
`unwind_rva_oob`	UnwindInfoAddress == 0 или >= SizeOfImage	Функция указывает на отсутствующую unwind metadata.
`rf_unsorted`	Текущий Begin меньше предыдущего	Таблица не соответствует ожидаемому sorted layout.
`rf_overlaps`	После сортировки Begin следующей < max_end	Один RIP может соответствовать нескольким функциям.
`duplicate_begin`	После сортировки одинаковый BeginAddress	Две записи конкурируют за начало одной функции.

Важно: `rf_unsorted` и `rf_overlaps` - разные признаки. Несортированность говорит о порядке таблицы. Overlap говорит о пересечении самих диапазонов после сортировки. Можно иметь несортированную таблицу без пересечений, а можно иметь отсортированную таблицу с overlapping ranges.

Порог 1% для части range/unwind ошибок нужен для различения единичной порчи и системной проблемы. Одна запись из тысячи может быть поврежденным артефактом или редкой особенностью, но больше 1% уже говорит, что нарушена не точка, а слой таблицы.

49. UNWIND_INFO: что именно декодируется

UNWIND_INFO описывает, как восстановить состояние стека при выходе из функции во время exception handling или stack walking. В проекте decoder читает header: version, flags, prolog size, count of unwind codes, frame register и frame offset. Затем читает массив unwind slots с выравниванием до четного количества слотов.

Decoder поддерживает documented v1 operations и v2 epilog/spare descriptors. Для каждого operation он понимает, сколько slots занимает запись. Это критично: если ошибиться с количеством slots, следующие bytes будут интерпретированы как другой opcode, handler RVA или chain record.

Проект не просто печатает opcodes. Он считает fixed_stack_bytes, pushed_nonvol_count, masks сохраненных GPR/XMM registers, наличие handler, наличие chain и набор anomalies. Это превращает raw bytes в компактную смысловую модель функции.

Поле	Что означает	Как используется
`version`	Версия UNWIND_INFO	Разрешены v1/v2, неизвестная версия дает anomaly/fail.
`flags`	EHANDLER, UHANDLER, CHAININFO	Определяет, какой tail лежит после codes: handler RVA или chained RUNTIME_FUNCTION.
`prolog_size`	Размер prolog в bytes	CodeOffset unwind op не должен выходить за prolog.
`count_codes`	Количество unwind slots	Задает размер массива codes и offset tail.
`frame_reg/frame_off`	Frame pointer model	Проверяется на согласованность и ABI warnings в strict mode.
`fixed_stack_bytes`	Оценка фиксированного изменения RSP	Помогает понять stack frame без полного исполнения кода.
`handler_rva`	Target exception/termination handler	Проверяется на OOB, section execute/write placement и dominance.
`chain`	Связанная RUNTIME_FUNCTION запись	Follow-chain mode объединяет часть summary и ловит loops/depth.

50. Anomalies внутри UNWIND_INFO

Anomaly	Что означает	Почему не всегда сразу finding
`UA_VERSION_UNKNOWN`	Версия не 1 и не 2	Обычно parse fail, потому что decoder не знает layout.
`UA_TRUNCATED`	Header/codes/operand/tail недочитаны	Становится `unwind_parse_fail`, если нельзя безопасно продолжать.
`UA_BAD_OPCODE`	Opcode неизвестен для версии	Decoder фиксирует нарушение и пытается не читать лишнего.
`UA_BAD_OPERAND`	Флаги/регистры/handler/chain fields противоречивы	Может быть anomaly, severity зависит от распространенности.
`UA_CODEOFFSET_OUTSIDE`	CodeOffset больше prolog_size	Metadata описывает действие за пределами пролога.
`UA_CODEOFFSET_ORDER`	CodeOffset идут в неожиданном порядке	Может указывать на странную или ручную metadata.
`UA_ALIGNMENT`	Unwind RVA не выровнен на 4	UNWIND_INFO обычно DWORD-aligned.
`UA_CHAIN_INVALID`	Chained record имеет плохие ranges/RVA	Ломает доверие к chain layer.
`UA_CHAIN_LOOP`	Follow-chain обнаружил цикл	Без depth limit decoder мог бы уйти в бесконечность.
`UA_CHAIN_DEPTH_EXCEEDED`	Превышена допустимая глубина	Защита от слишком длинных или злонамеренных chain.
`UA_HANDLER_OOB`	Handler RVA выходит за SizeOfImage	Handler target не может быть нормальным code path внутри image.
`UA_OPERAND_SANITY`	Размеры alloc/save выглядят нелепо	Сигнал sanity, не абсолютное доказательство.
`UA_ABI_*`	Формальные strict ABI warnings	В проекте не повышают score сами по себе, потому что production toolchains бывают гибкими.

51. Handler layer

Handler layer начинается там, где UNWIND_INFO flags содержат `UNW_FLAG_EHANDLER` или `UNW_FLAG_UHANDLER`. После aligned unwind codes лежит 32-bit HandlerRVA. Проект читает этот RVA, сохраняет в UnwindSummary и дальше использует для проверки placement и dominance.

Почему handler важен: обработчик исключений - это control-flow endpoint. Если сотни функций ведут к одному handler, это может быть нормальный runtime helper, но может быть и centralized dispatcher в protector/obfuscator/interceptor. Поэтому проект не делает автоматический вывод по одному факту reuse, а учитывает section placement и другие static signals.

`handler_nonexec_section` появляется, когда handler target не найден в секции или секция не executable. Нормальный handler должен исполняться как код. `handler_writable_section` появляется, когда target лежит в writable секции: сам факт не равен malware, но это сильный повод проверить, почему endpoint обработки исключений изменяем.

52. Section characteristics и memory protection

Раздел memory protection в проекте не является отдельным полным hardening scanner. Он используется точечно: чтобы понять, разумно ли расположены UNWIND_INFO и handler target. Проверяются флаги section characteristics: execute и write. Для отчета также выводятся readable/writable/executable для секций.

UNWIND_INFO обычно должен находиться в data-like readable области: .xdata, .rdata, иногда .pdata или toolchain-specific xdata-like sections. Он не должен исполняться как код, потому что это metadata. Если он лежит в executable section, появляется `unwind_in_exec_section`. Если в writable section - `unwind_writable_section`.

Handler, наоборот, должен быть executable, потому что это код. Поэтому non-executable handler target сильнее подозрителен, чем просто необычное имя секции. Writable handler target тоже важен: если code endpoint изменяем, возможна runtime patching/JIT/protector логика.

Честная граница: проект не проверяет все секции на W^X, DEP, ASLR, NX compatibility и не делает VirtualQueryEx snapshot. Он проверяет rights там, где они влияют на EH/unwind модель.

53. Placement heuristics

Finding	Нормальное ожидание	Почему finding появляется
`unwind_in_exec_section`	Metadata в non-exec data section	Executable metadata смешивает code и service data.
`unwind_unusual_section`	.xdata/.rdata/.pdata или похожее имя	Section name не похож на unwind/data placement.
`unwind_writable_section`	Metadata read-only после загрузки	Mutable unwind metadata может менять stack walking/exception behavior.
`handler_nonexec_section`	Handler target в executable code	Non-exec target не выглядит как исполняемый handler.
`handler_writable_section`	Handler target обычно не writable	Mutable handler endpoint требует ручной проверки.

54. Reuse и dominance

Reuse-сигналы не являются простыми счетчиками 'больше - значит плохо'. Проект учитывает размер выборки. `unwind_reuse_moderate` рассматривается только при `n > 200` и ratio > 0.10, причем должен быть другой static signal. Это сделано, чтобы не ругать маленькие учебные binaries, где повторение metadata может быть естественным.

`unwind_reuse_strong` появляется при `n > 200` и ratio > 0.30. Это означает, что более 30% функций разделяют один UnwindInfoRVA. Для больших binaries такая концентрация уже похожа на специальный pattern, особенно если рядом есть handler dominance или placement anomalies.

`handler_dominance` требует handler_count > 50 и ratio > 0.80. То есть речь не про две функции с одним handler, а про десятки обработчиков, где один target доминирует. В коде дополнительно есть ослабление: если dominant handler лежит в executable code-like section и нет другого static context, finding не добавляется.

55. Все 27 findings по слоям

Слой	Findings
`PE layout`	pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob
`Function table`	no_runtime_functions, invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin
`Unwind integrity`	unwind_parse_fail, unwind_anomalies
`Reuse/dominance`	unwind_reuse_strong, unwind_reuse_moderate, handler_dominance
`Metadata placement`	unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section
`Correlation combos`	placement_exec_nonexec_handler_combo, placement_integrity_combo, reuse_dominance_combo, writable_metadata_handler_combo

Почему 27, а не 15 или 50: это не произвольный красивый номер. Текущие findings покрывают минимально необходимые точки отказа цепочки PE layout -> function table -> unwind info -> handler layer -> cross-signal correlation. Добавлять признаки вне этой цепочки можно, но тогда проект превратится в общий PE scanner и потеряет фокус ВКР.

Важная защита на комиссии: 27 - это текущая версия evidence model. Она не утверждает, что других PE-признаков в мире нет. Она утверждает, что для цели ВКР отобраны признаки, которые непосредственно влияют на доверие к EH/unwind metadata.

56. Подробная матрица признаков

Finding	Что проверяет	Почему важно	Что проверять вручную
`pe_section_raw_oob`	Raw-диапазон секции выходит за размер файла.	Секция заявляет bytes за пределами file_size. Это ломает физическое чтение из файла.	Открыть section table, сравнить PointerToRawData/SizeOfRawData с file_size.
`pe_section_virtual_oob`	Virtual-диапазон секции выходит за SizeOfImage.	Секция заявляет RVA-область за пределами виртуального image.	Сравнить VirtualAddress + span с SizeOfImage.
`pe_section_raw_overlap`	Raw-диапазоны секций пересекаются.	Одни bytes файла имеют несколько секционных интерпретаций.	Построить raw ranges и увидеть пересечение file bytes.
`pe_section_virtual_overlap`	Virtual-диапазоны секций пересекаются.	Один RVA может иметь неоднозначные section permissions/role.	Построить RVA ranges и проверить ambiguity section permissions.
`pe_section_misaligned`	Нарушены alignment/header инварианты.	Проверяются FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders, order и ImageBase alignment.	Проверить FileAlignment, SectionAlignment, SizeOfHeaders, SizeOfImage, order sections.
`pe_exception_directory_oob`	Exception directory битая или недоступная.	Directory не помещается, не мапится, неполная или имеет некратный размер.	Проверить DataDirectory[3], кратность 12, RVA mapping, raw readability.
`no_runtime_functions`	В executable x64 image нет RUNTIME_FUNCTION.	Для обычного x64 кода это странно, но leaf-only/resource cases возможны.	Проверить, есть ли настоящий executable code и насколько он leaf-only.
`invalid_ranges`	EndAddress <= BeginAddress.	Function table entry не задает корректный полуоткрытый диапазон функции.	Отсортировать .pdata и найти записи с End <= Begin.
`rf_oob`	Begin/End функции выходят за SizeOfImage.	Function table ссылается на код вне виртуального образа.	Сравнить Begin/End каждой функции с SizeOfImage.
`unwind_rva_oob`	UnwindInfoAddress недоступен.	RVA нулевой, вне image или не читается из raw bytes.	Проверить UnwindInfoAddress: не 0, внутри image, мапится в raw bytes.
`rf_unsorted`	Таблица не отсортирована по BeginAddress.	Нарушается ожидание пригодной для поиска function table.	Сравнить порядок BeginAddress в исходной таблице.
`rf_overlaps`	Диапазоны функций пересекаются.	Один RIP может относиться к нескольким функциям.	После сортировки проверить пересечение ranges.
`duplicate_begin`	Несколько функций имеют одинаковый BeginAddress.	Конфликт идентичности функции в таблице.	Найти BeginAddress, встречающиеся больше одного раза.
`unwind_parse_fail`	UNWIND_INFO не разобран.	Metadata заявлена, но header/codes/tail недоступны или обрезаны.	Открыть UNWIND_INFO bytes, проверить CountOfCodes, operands, tail.
`unwind_anomalies`	UNWIND_INFO разобран, но содержит anomalies.	Версия, opcode, operands, code offsets, chain, handler или alignment выглядят странно.	Сравнить opcodes, prolog offsets, flags, chain и handler с ABI.
`unwind_reuse_strong`	Массовое reuse одного UnwindInfoRVA.	При n > 200 и ratio > 0.30 это похоже на centralized metadata pattern.	Посмотреть dominant UnwindInfoRVA и несколько referencing functions.
`handler_dominance`	Один handler доминирует.	При handler_count > 50 и ratio > 0.80 один target обслуживает почти все функции.	Открыть dominant handler в Ghidra и понять, runtime helper это или dispatcher.
`unwind_reuse_moderate`	Умеренное reuse, подтвержденное другим сигналом.	При ratio > 0.10 и n > 200 finding появляется только при корреляции.	Проверить второй corroborating signal, без него finding не должен быть сильным.
`unwind_in_exec_section`	UNWIND_INFO лежит в executable секции.	Metadata оказалась в области, предназначенной для кода.	Проверить секцию UnwindInfoRVA и ее characteristics.
`unwind_unusual_section`	UNWIND_INFO лежит в необычной секции.	Секция не похожа на .xdata/.rdata/.pdata или родственные им имена.	Понять, почему metadata не в .xdata/.rdata/.pdata-like секции.
`unwind_writable_section`	UNWIND_INFO лежит в writable секции.	Metadata, влияющая на unwind/handler, потенциально изменяема.	Проверить, есть ли JIT/runtime/protector причина writable metadata.
`handler_nonexec_section`	HandlerRVA указывает в non-executable/unknown секцию.	Handler должен быть code path; non-exec target требует проверки.	Дизассемблировать target и проверить execute permission.

Finding	Что проверяет	Почему важно	Что проверять вручную
`handler_writable_section`	HandlerRVA указывает в writable секцию.	Endpoint обработки исключений потенциально изменяем.	Проверить, почему handler code endpoint writable.
`placement_exec_nonexec_handler_combo`	Executable metadata + non-exec handler.	Две placement странности на соседних слоях усиливают друг друга.	Проверить обе стороны: metadata placement и handler target.
`placement_integrity_combo`	Placement anomaly + integrity problem.	Странное размещение подтверждается parse/range проблемами.	Сопоставить placement anomaly с parse/range damage.
`reuse_dominance_combo`	Reuse metadata + dominant handler.	Похоже на centralized dispatcher/interceptor style.	Проверить centralized metadata и centralized handler вместе.
`writable_metadata_handler_combo`	Writable metadata + writable handler.	Mutable metadata и mutable control-flow endpoint одновременно.	Проверить mutable metadata и mutable handler target как единый case.

57. Score: что означают цифры

Цифры score - это инженерные веса, а не физические константы и не математически доказанная вероятность. Их задает разработчик модели, но не произвольно: веса сгруппированы по силе evidence. Нарушения table/unwind integrity имеют самые высокие веса, placement anomalies - medium/high в зависимости от характера, reuse/dominance - умеренные без корреляции, combo findings добавляют усиление, когда два слоя подтверждают друг друга.

Почему `+25`, а не `+47`: шкала ограничена 0..100. Вес 25 означает сильный отдельный сигнал, но оставляет место для других независимых signals. Если поставить 47, одиночный finding слишком быстро приблизит verdict к high_suspicion и испортит triage. Почему не `+26`: можно было бы, но это не изменит качественный класс модели. Важны не отдельные единицы, а группы весов и пороги verdict.

Именно поэтому в проекте есть `docs/evidence_matrix.md` и `docs/threshold_calibration.md`: они фиксируют правила, объясняют их и предлагают reproducible score sweep. Проект честно говорит, что текущие defaults не являются ROC-calibrated статистикой на огромном датасете.

Группа	Типичные веса	Смысл
`Сильная структурная поломка`	+20..+25	OOB, overlap, unreadable directory, parse fail в заметной доле.
`Средняя структурная странность`	+8..+15	Misalignment, single-family issues, unusual placement.
`Reuse/dominance`	+10..+20	Становится значимым на больших n и/или при corroboration.
`Combo`	+8..+10	Не заменяет основные findings, а усиливает связку signals.
`Confidence bonuses`	+0.04..+0.11	Уверенность растет от количества families и strength.
`Mitigation penalties`	-0.06..-0.12	Уверенность падает при одиночных/малых/placement-only signals.

58. Verdict, evidence level и confidence

`score` показывает suspiciousness по статической EH/unwind модели. `suspicion` - это score / 100. `confidence` - это уверенность интерпретации, а не вероятность malware. Это принципиально: sample может иметь высокий score из-за поврежденной metadata, но проект не утверждает, что это точно вредонос.

Static evidence level строится так: strong, если есть минимум два strong findings или score >= 70; medium, если есть strong finding, минимум два medium findings или score >= 30; weak, если есть хоть какой-то signal ниже этих порогов; none, если findings нет и score 0.

Verdict по score: `high_suspicion` от 70, `moderate_suspicion` от 40, `low_suspicion` от 15, иначе `no_strong_static_anomalies`. Эти пороги дают триажную шкалу, а не судебное доказательство. Поэтому report дополнительно содержит manual validation steps.

59. Mitigations и ослабление выводов

Mitigation	Когда применяется	Зачем нужна
`single_signal_only`	Ровно один non-strong finding	Один слабый сигнал не должен выглядеть как сильное дело.
`small_sample_size`	Меньше 32 функций, есть findings, нет strong finding	На маленькой выборке ratios легко переоценить.
`placement_only_signal`	Только unusual placement без integrity/range/exec/write damage	Редкий layout может быть benign.
`single_family_only`	Нет strong finding и evidence из одной family	Нужна cross-layer corroboration.

Mitigations показывают зрелость проекта. Простой scanner мог бы только добавлять score. Здесь модель умеет говорить: сигнал есть, но он изолирован; относись осторожно. Это важно для защиты ВКР, потому что показывает понимание false positives.

60. Dynamic validation

Dynamic mode - Windows-only часть проекта. Она запускает target как child process под debugger-ом и собирает bounded evidence. Цель не в том, чтобы полностью анализировать malware behavior, а в том, чтобы проверить runtime признаки, связанные с exception handling и unwind.

Модуль наблюдает first-chance и second-chance exceptions, hits handler-ов из статического UNWIND_INFO, registrations/hits VEH/VCH/UEF, вызовы RtlAddFunctionTable и RtlInstallFunctionTableCallback, а также короткий RIP trace после handler hit. Эти данные помогают ответить: статическая странность реально проявляется во время exception path или пока видна только в файле.

Dynamic session имеет timeout, event cap, trace cap и флаг completeness. Если session incomplete, runtime evidence level capped to weak и не усиливает final confidence как полноценная корреляция. Это честно: неполный запуск не должен подтверждать сильный case.

Dynamic field	Смысл
`exceptions_seen / first_chance_seen / second_chance_seen`	Показывает, были ли исключения и доходили ли до second chance.
`unwind_handler_bp_set / unwind_handler_hits`	Сколько handler breakpoints поставлено и сколько реально сработало.
`veh_registrations / veh_hits`	Vectored exception handler registration/hit evidence.
`vch_registrations / vch_hits`	Vectored continue handler registration/hit evidence.
`uef_set_calls / uef_hits`	Unhandled exception filter evidence.
`rtl_add_function_table_calls`	Runtime добавляет function table вручную.
`rtl_install_function_table_cb_calls`	Runtime ставит callback-based function table.
`trace`	Ограниченный список RIP после handler hit.

61. Static-dynamic correlation

Correlation layer специально консервативный. Он не утверждает per-function proof graph. Он возвращает true только при достаточно естественной связи: например, static handler-focused evidence плюс runtime handler/VEH/VCH/UEF hits; или static placement anomalies плюс runtime dynamic function table activity.

Final assessment в JSON/HTML учитывает static evidence level, runtime evidence level, correlation flag, dynamic_signal_count и weakening_rule_count. Если correlation есть и хотя бы один слой strong, case becomes `strong_correlated_case`. Если evidence strong только на одном слое без корреляции, это `strong_unconfirmed_case`.

Такая модель защищает от двух ошибок: не переоценивать static-only anomalies и не игнорировать runtime evidence, когда оно действительно подтверждает static подозрения.

62. Reports как часть архитектуры

Отчетность в проекте - не декоративный слой. JSON содержит исходные coordinates, sections, pe_validation, heuristic_analysis, dynamic_validation, assessment и functions. Это делает результат пригодным для повторной проверки, автоматического тестирования и включения в диплом.

CSV полезен для табличного просмотра. Ghidra CSV специально рассчитан на импорт в Ghidra: begin/end/name/comment. HTML нужен для человека: self-contained report без remote scripts/styles/fonts, с offline CSP и русскими текстами для key findings.

Report writer валидирует metadata: schema version, tool version, timestamp, input sha256, input size, command line, build compiler/platform/mode, strict_abi, follow_chain, max_chain_depth. Это важно для воспроизводимости: через месяц можно понять, каким билдом и с какими flags был получен отчет.

63. Manual validation steps

Проект осознанно добавляет manual validation steps. Это правильный подход для reverse engineering tooling: analyzer сужает поле поиска, но финальный вывод должен подтверждать человек. Steps зависят от findings. Например, при table integrity issues нужно проверить .pdata ranges в PE viewer. При unwind parse fail - открыть UNWIND_INFO bytes и сравнить с prolog. При handler dominance - открыть handler target в Ghidra.

Эта часть особенно важна для комиссии: проект не маскируется под 'искусственный интеллект, который сам все знает'. Он показывает технический evidence и говорит, что именно проверить дальше.

64. Тестирование и качество

Проект тестируется на нескольких уровнях. Unit tests проверяют unwind decoder, heuristics, mitigations, context-sensitive combos, report JSON/HTML, core edge cases и matrix report. Regression samples проверяют реальные small binaries: benign_minimal, benign_richer и несколько suspicious samples с raw OOB, unwind OOB, writable unwind, EH/unwind tamper.

Python validators проверяют JSON schema и semantic invariants. Это важно: отчет может быть синтаксически валидным JSON, но внутренне противоречивым. Например, functions length должен соответствовать runtime function count, sections должны иметь ожидаемые fields, assessment должен быть согласован с heuristic/dynamic blocks.

Mutation fuzzing гоняет детерминированные повреждения parser-а. Это напрямую соответствует threat model: входной PE недоверенный и может быть обрезан, испорчен, иметь странные sizes/offsets. Sanitizer/CI options в CMake помогают ловить memory errors.

Тип проверки	Что доказывает
`unit_unwind`	UNWIND_INFO decoder правильно читает operations, flags, anomalies.
`unit_heuristics*`	Findings, score, mitigations, combos и context работают ожидаемо.
`unit_report_json/html`	Reports строятся и содержат нужные blocks.
`run_regression_samples.py`	Baseline samples дают ожидаемые verdict/findings и проходят schema/semantics.
`run_mutation_fuzz.py`	Parser устойчив к поврежденным входам.
`dynamic validators`	Windows dynamic reports имеют ожидаемые runtime evidence fields.
`release hygiene/metadata`	Пакет не теряет version/schema/build metadata.

65. Почему это сложнее CRUD/сайта

В CRUD или системе бронирования предметная область обычно задается чистыми структурами: пользователь, бронь, услуга, дата, база данных. Ошибка чаще всего связана с формой, SQL, API или UI. В этом проекте вход - hostile binary file. Он может намеренно заставлять parser ошибиться через переполнения, неверные offsets, нулевые sizes, пересечения и ложные RVA.

Здесь сложность не в количестве экранов, а в корректном чтении низкоуровневого формата. Нужно одновременно понимать C memory safety, little-endian чтение, PE/COFF, Windows x64 ABI, exception directory,

RUNTIME_FUNCTION, UNWIND_INFO, section permissions, static/dynamic correlation и сериализацию отчетов.

Еще одна сложность - explainability. Проект должен не просто вернуть число, а объяснить, на каком слое сломалась модель и почему это важно для reverse engineer-а.

66. Почему проект нужен, если есть Windows и антивирусы

Windows использует эти структуры, но не выдает reverse engineer-у учебный и объяснимый отчет: где именно Exception Directory не читается, какие function ranges пересекаются, какой handler доминирует, почему confidence ослаблена. Windows loader и exception dispatcher решают runtime-задачу, а не задачу анализа и документирования.

Антивирус может иметь закрытые внутренние эвристики, но для обычного пользователя и студента это часто black box. Даже в компаниях analyst не всегда имеет доступ к internal scoring rules. Этот проект ценен как открытая, воспроизводимая и объяснимая модель узкой области EH/unwind.

Крупные компании могут не выкладывать аналогичные инструменты, потому что такие эвристики являются частью detection know-how: scoring, thresholds, correlation rules, confidence model, test corpus, final verdict logic. В твоем проекте эти правила открыты для проверки, что и делает его учебно и исследовательски полезным.

67. Честные ограничения проекта

Ограничение	Почему это нормально
`Только PE32+ AMD64`	Фокус на одной ABI-модели повышает глубину и корректность.
`Нет PE32/x86`	x86 SEH требует другой архитектуры признаков.
`Нет ELF/Mach-O`	Это другие object formats и exception models.
`Нет full disassembly`	Цель - metadata layer, а не восстановление всего кода.
`Нет полноценного malware verdict`	Score относится к EH/unwind suspiciousness, не к вредоносности файла вообще.
`Dynamic только Windows`	Используются Windows Debug API и x64 exception behavior.
`Нет полной runtime memory protection map`	Section characteristics проверяются для unwind/handler placement.
`Thresholds engineering defaults`	Есть reproducible sweep path, но нет заявления о статистической оптимальности.

68. Как отвечать комиссии про признаки

Вопрос	Сильный ответ
`Почему именно эти признаки?`	Они покрывают цепочку доверия EH/unwind: PE layout, function table, unwind integrity, handler placement, reuse/dominance и cross-signal combos.
`Почему не больше?`	Потому что цель не общий PE scanner. Признаки вне EH/unwind можно добавить, но это расширит предметную область и потребует новых тестов/обоснований.
`Кто установил границы?`	Базовые границы заданы PE/COFF, Windows x64 ABI и file_size. Проект выбирает, какие нарушения этих границ фиксировать как findings.
`Кто установил веса?`	Разработчик модели как engineering defaults, закрепленные в evidence_matrix и проверяемые regression/score_sweep.
`Почему score не вероятность?`	Нет большого statistically calibrated corpus. Поэтому confidence - инженерная уверенность интерпретации, а не вероятность malware.
`Почему memory protection не полностью?`	Потому что права секций нужны здесь для проверки размещения UNWIND_INFO/handler, а не для полного hardening audit.
`Почему dynamic не sandbox?`	Он собирает bounded EH/unwind-related evidence, а не все поведение процесса.

69. Итоговая техническая формула проекта

EH/Unwind Analyzer = безопасный PE32+ AMD64 parser + строгая модель RVA/raw/image границ + reader RUNTIME_FUNCTION + decoder UNWIND_INFO + handler/placement/reuse heuristics + explainable scoring/confidence + optional Windows runtime evidence + проверяемые JSON/HTML/CSV reports.

Приложение. Термины

Термин	Значение
PE	Portable Executable, формат EXE/DLL Windows.
PE32+	64-bit Optional Header, magic 0x20b.
AMD64	x64 Machine 0x8664.
RVA	Relative Virtual Address внутри image.
VA	Virtual Address = ImageBase + RVA.
file offset	Смещение в bytes от начала файла.
SizeOfImage	Размер виртуального образа.
SizeOfHeaders	Размер headers области.
zero-fill	Virtual bytes, которых нет on disk и которые loader заполнит нулями.
Exception Directory	DataDirectory[3], вход в RUNTIME_FUNCTION table.
RUNTIME_FUNCTION	BeginAddress, EndAddress, UnwindInfoAddress.
UNWIND_INFO	Metadata для восстановления stack frame.
EHANDLER/UHANDLER	Флаги наличия handler tail.
CHAININFO	Флаг chained RUNTIME_FUNCTION.
finding	Структурированный признак проекта.
confidence	Уверенность интерпретации static verdict, не вероятность malware.

ДОПОЛНЕНИЕ К ГЛУБОКОМУ ОПИСАНИЮ ПРОЕКТА

70. Как ВКР безопасно читает Exception Directory

Этот раздел добавлен в конец документа, чтобы явно показать не абстрактную теорию parser-а, а реальный путь, который выполняет текущая ВКР. Важно: проект не читает Exception Directory 'на глаз'. Он проходит по цепочке PE-структур, на каждом этапе проверяет границы, а потом отдельно анализирует каждую запись RUNTIME_FUNCTION и соответствующий UNWIND_INFO.

Ниже ссылки указаны в формате `src/file.c:line-range`. Это не пересказ по памяти, а привязка к текущему коду проекта.

Короткая формула реализации: `DOS Header -> e_lfanew -> NT Headers -> COFF File Header -> Optional Header PE32+ -> NumberOfRvaAndSizes -> DataDirectory[Exception] -> section/RVA mapping -> file offset -> RUNTIME_FUNCTION[] -> UNWIND_INFO -> heuristics findings`.

70.1. Цепочка чтения PE-структур

Шаг	Что делает ВКР	Где в коде
1	Читает DOS Header, проверяет `MZ`, берет `e_lfanew` из offset `0x3c`. Если DOS-заголовок плохой, parser не идет дальше.	`src/pe.c:11-17`, вызов из `src/pe.c:40-45`
2	Переходит по `e_lfanew`, проверяет, что там помещаются PE signature и COFF header.	`src/pe.c:45`
3	Проверяет PE signature `PE\0\0`. Если сигнатура не совпала, это не тот PE-заголовок.	`src/pe.c:51`
4	Читает COFF File Header: `Machine`, `NumberOfSections`, `SizeOfOptionalHeader`.	`src/pe.c:56-63`
5	Проверяет архитектуру: поддерживается только `IMAGE_FILE_MACHINE_AMD64`, то есть Windows x64.	`src/pe.c:67-70`
6	Проверяет, что Optional Header полностью помещается в файл и достаточно большой для нужных полей.	`src/pe.c:73-80`
7	Проверяет `OptionalHeader.Magic == 0x20b`, то есть PE32+.	`src/pe.c:83-90`
8	Читает ключевые поля PE32+: `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `SizeOfHeaders`, `NumberOfRvaAndSizes`.	`src/pe.c:97-116`
9	Проверяет, что таблица data directories вообще содержит Exception Directory entry.	`src/pe.c:121-124`
10	Вычисляет смещение `DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]` и читает `VirtualAddress/Size`.	`src/pe.c:127-134`

70.2. Проверка секций перед чтением directory

После чтения заголовков проект строит карту секций. Это нужно не только для красивого отчета. Без section table нельзя надежно сопоставлять RVA с file offset и нельзя понимать, в какой области лежит UNWIND_INFO или handler.

ВКР читает каждый section header и сохраняет `name`, `VirtualSize`, `VirtualAddress`, `SizeOfRawData`, `PointerToRawData`, `Characteristics`. Затем проверяет raw-выходы за файл, virtual-выходы за `SizeOfImage`, raw/virtual overlaps, alignment, порядок секций и согласованность `SizeOfHeaders`.

Проверка	Зачем нужна	Где в коде
`Чтение section headers`	Чтобы построить карту raw/virtual диапазонов и прав секций.	`src/pe.c:139-165`
`FileAlignment/SectionAlignment/SizeOfImage/SizeOfHeaders`	Чтобы понять, можно ли доверять раскладке файла и image.	`src/pe.c:174-188`
`Raw range OOB`	Секция не должна заявлять bytes за пределами физического файла.	`src/pe.c:193-196`
`Virtual range OOB`	Секция не должна выходить за `SizeOfImage`.	`src/pe.c:194-197`
`Raw/virtual overlaps`	Одни bytes или один RVA не должны иметь несколько конфликтующих интерпретаций.	`src/pe.c:203-220`
`Сохранение counters в PeImage`	Эти counters позже превращаются в findings.	`src/pe.c:245-255`

70.3. Что ВКР делает с VirtualAddress и Size

Для Exception Directory проект сначала фиксирует пару `exception_rva` и `exception_size` в `PeImage`. Уже на уровне PE parser-а он отмечает грубую несогласованность: если одно поле нулевое, а другое нет, или если `RVA + Size` выходит за `SizeOfImage`, выставляется `exception_directory_oob`.

Но окончательное чтение таблицы выполняется позже в `unwind_read_runtime_functions_ex`. Там логика строже: оба поля нули означают отсутствие статической таблицы исключений; один ноль при втором ненулевом - malformed; ненулевая пара должна быть маппируемой в file offset, физически помещаться в файл и иметь размер, совместимый с массивом RUNTIME_FUNCTION.

Условие	Реакция проекта	Где в коде
`RVA == 0 && Size == 0`	Таблица отсутствует; это не ошибка само по себе, `out_count = 0`.	`src/unwind.c:623-625`
`RVA == 0 || Size == 0`	Malformed: поля должны быть либо оба нулевые, либо оба ненулевые.	`src/unwind.c:626-629`
`RVA + Size > SizeOfImage`	Помечается `exception_directory_oob` в PE-модели.	`src/pe.c:230-235`
`RVA не переводится в file offset`	Malformed: directory нельзя безопасно прочитать из файла.	`src/unwind.c:631-634`
`file_offset + Size > file_size`	Malformed: заявленная область выходит за физические bytes файла.	`src/unwind.c:635-638`
`Size < 12`	Malformed: область меньше одной записи RUNTIME_FUNCTION.	`src/unwind.c:639-642`
`Size % 12 != 0`	Malformed: размер не кратен 12 bytes, то есть не является корректным массивом RUNTIME_FUNCTION.	`src/unwind.c:643-646`
`n = Size / 12`	Количество записей вычисляется только после всех проверок выше.	`src/unwind.c:647`

Нюанс для защиты: в текущей ВКР размер Exception Directory, не кратный 12, не имеет отдельного finding-кода. Он фиксируется как malformed directory и попадает в общий finding `pe_exception_directory_oob`.

70.4. Как RVA переводится в file offset

`pe_rva_to_fileoff` не делает простое `file_offset = RVA`. Сначала ищется секция, содержащая RVA. Если секция найдена, вычисляется delta от начала секции. Чтение разрешается только если delta лежит внутри `SizeOfRawData`. Если RVA попадает в virtual tail, который loader заполнит нулями, static parser отказывается читать bytes из файла, потому что этих bytes на диске нет.

Отдельно поддерживается область headers: если RVA меньше `SizeOfHeaders` и физически помещается в файл, RVA мапится напрямую в file offset. Это важно, потому что headers тоже находятся в начале виртуального image.

Механизм	Что защищает	Где в коде
`Поиск секции по RVA`	Понимание, какому virtual range принадлежит адрес.	`src/pe.c:265-275`
`Запрет чтения zero-fill tail`	Parser не читает bytes, которых нет в файле.	`src/pe.c:279-286`
`Проверка сложения raw pointer + delta`	Защита от overflow и выхода за файл.	`src/pe.c:287-290`
`Поддержка SizeOfHeaders`	RVA может указывать в headers, а не в обычную секцию.	`src/pe.c:294-300`

70.5. Чтение массива RUNTIME_FUNCTION

После того как Exception Directory прошел проверки, проект выделяет массив `RuntimeFunction` на `n = Size / 12` записей. Затем каждая запись читается как три DWORD-поля: `BeginAddress`, `EndAddress`, `UnwindInfoAddress`. Если хотя бы одно поле очередной записи не читается, массив освобождается и чтение считается malformed.

Важно: на этом этапе проект только безопасно достает записи из файла. Смысловые проверки диапазонов выполняются дальше в heuristics layer, потому что именно там формируются findings, severity, score и representative RVA.

Действие	Смысл	Где в коде
`Выделение массива RuntimeFunction`	Готовит структуру для всех записей .pdata.	`src/unwind.c:647-653`
`Чтение трех DWORD-полей`	Каждая запись = Begin, End, UnwindInfo RVA.	`src/unwind.c:655-663`
`Сохранение в rf[i]`	Дальше эти поля будут проверяться heuristics и report layers.	`src/unwind.c:664-667`
`Malformed при недочитанной записи`	Parser не продолжает на частично прочитанной записи.	`src/unwind.c:657-662`

70.6. Что происходит, если directory malformed

ВКР не падает без объяснения. `main.c` вызывает `unwind_read_runtime_functions_ex`. Если результат не `RUNTIME_FUNCTION_READ_OK`, но это не OOM, проект выставляет `img.exception_directory_oob = true`, печатает сообщение о частичном структурном отчете и продолжает настолько, насколько может.

Потом `heuristics_analyze` переносит этот флаг в `HeuristicReport` и добавляет finding `pe_exception_directory_oob`. Поэтому пользователь видит не просто ошибку чтения, а структурированный признак в отчете.

Слой	Что делает	Где в коде
`Orchestration`	Вызов reader-а и обработка malformed/OOM.	`src/main.c:306-321`
`PE flag -> heuristic report`	Копирует `img->exception_directory_oob` в report model.	`src/heuristics.c:610`
`Finding`	Создает `pe_exception_directory_oob` с severity `high`.	`src/heuristics.c:825-831`

70.7. Проверка каждой RUNTIME_FUNCTION после чтения

После чтения .pdata проект не считает записи автоматически корректными. В `heuristics_analyze` каждая запись проходит проверки range integrity. Это уже не чтение bytes, а смысловая проверка карты функций.

Finding / counter	Что проверяет	Где в коде
`rf_invalid_ranges -> invalid_ranges`	`EndAddress <= BeginAddress`: запись не задает корректный полуоткрытый диапазон функции.	counter: `src/heuristics.c:652-654`, finding: `src/heuristics.c:850-855`
`rf_oob -> rf_oob`	`BeginAddress` или `EndAddress` выходят за `SizeOfImage`.	counter: `src/heuristics.c:655-658`, finding: `src/heuristics.c:857-862`
`rf_unwind_oob -> unwind_rva_oob`	`UnwindInfoAddress` нулевой или за пределами image.	counter: `src/heuristics.c:659-662`, finding: `src/heuristics.c:864-869`
`rf_unsorted -> rf_unsorted`	Таблица не идет по возрастанию `BeginAddress`.	counter: `src/heuristics.c:664-672`, finding: `src/heuristics.c:871-876`
`rf_overlaps -> rf_overlaps`	После сортировки диапазоны функций пересекаются.	counter: `src/heuristics.c:746-765`, finding: `src/heuristics.c:878-883`
`rf_duplicate_begins -> duplicate_begin`	Несколько записей имеют одинаковый `BeginAddress`.	counter: `src/heuristics.c:758-761`, finding: `src/heuristics.c:885-890`

70.8. Доступность и разбор UNWIND_INFO

Для каждой записи RUNTIME_FUNCTION `main.c` вызывает `unwind_parse` по `rf[i].unwind_rva`. Сам `unwind_parse` снова не читает bytes напрямую: сначала `parse_unwindinfo_header` переводит UnwindInfo RVA через `pe_rva_to_fileoff`, затем проверяется header, версия, flags, размер массива unwind codes, operands, tail, handler и chain.

Если decoder не может безопасно прочитать структуру, появляется parse failure. Если структура читается, но содержит странности - например неизвестную версию, плохие operands, нарушение порядка CodeOffset, плохой chain или handler OOB - это попадает в anomalies.

Проверка	Смысл	Где в коде
Цикл по функциям	Для каждого rf[i] вызывается unwind_parse.	src/main.c:351-355
RVA -> file offset для UNWIND_INFO	UNWIND_INFO не читается, если RVA не маппится в физические bytes.	src/unwind.c:56-64
Alignment и header	Фиксируется невыровненный RVA и читается 4-byte header.	src/unwind.c:190-193
Version/flags sanity	Разрешены v1/v2, проверяются EHANDLER/UHANDLER/CHAININFO combinations.	src/unwind.c:207-229
Unwind codes range	Массив unwind slots должен помещаться в файл.	src/unwind.c:233-237
Opcode/operand parsing	Для каждого opcode проверяется slots count и operands.	src/unwind.c:254-516
CHAININFO tail	Читает chained RUNTIME_FUNCTION и проверяет его ranges/RVA.	src/unwind.c:520-536
Handler tail	Читает handler_rva и проверяет выход за SizeOfImage.	src/unwind.c:540-550
Follow-chain protection	Ловит chain loops и depth exceeded.	src/unwind.c:563-604
Findings	unwind_parse_fail и unwind_anomalies попадают в structured findings.	src/heuristics.c:893-905

70.9. Проверка секций для UNWIND_INFO и handler

Дополнительно ВКР проверяет роль и права секций именно в EH/unwind-контексте. Для UNWIND_INFO ожидается metadata/data-like область, обычно .xdata, .rdata, .pdata или похожая секция. Для handler target ожидается executable code. Поэтому проект проверяет executable/writable characteristics для этих адресов.

Проверка	Что означает	Где в коде
Section для UnwindInfoAddress	Ищется секция, где лежит UNWIND_INFO.	src/heuristics.c:693
unwind_in_exec_section	UNWIND_INFO лежит в executable section.	counter: src/heuristics.c:704-706, finding: src/heuristics.c:965-970
unwind_writable_section	UNWIND_INFO лежит в writable section.	counter: src/heuristics.c:708-710, finding: src/heuristics.c:980-986
Section для handler	Ищется секция, куда указывает handler_rva.	src/heuristics.c:715
handler_nonexec_section	Handler target не в executable section или секция неизвестна.	counter: src/heuristics.c:716-718, finding: src/heuristics.c:988-993
handler_writable_section	Handler target находится в writable section.	counter: src/heuristics.c:720-722, finding: src/heuristics.c:995-1000

70.10. Честные нюансы реализации

Вопрос	Как в ВКР
Есть ли отдельный finding exception_directory_bad_size?	Нет. Некратный 12 размер фиксируется как malformed Exception Directory и в отчете выходит через общий pe_exception_directory_oob.
Проверяется ли, что Begin/End лежат именно в executable code section?	Не отдельным finding. Проект проверяет Begin/End внутри SizeOfImage, сортировку, пересечения и duplicate begins. Section начала функции выводится в report layer, но executable-права function range не являются отдельным признаком.
Проверяется ли доступность UNWIND_INFO?	Да. unwind_parse переводит UnwindInfo RVA через pe_rva_to_fileoff, проверяет header, codes, operands, handler/chain tail и не читает zero-fill bytes.
Проверяется ли роль секций?	Частично и по теме ВКР: для UNWIND_INFO и handler placement. Полный аудит всех секций как PE-hardening scanner не заявляется.
Что происходит при malformed directory?	Проект не падает молча: ставит exception_directory_oob, продолжает частичный отчет и формирует finding.

70.11. Формулировка для защиты

ВКР читает Exception Directory по безопасной цепочке PE-структур: сначала валидирует DOS/NT/COFF/Optional Header, проверяет PE32+ AMD64 и наличие data directory entry, затем читает VirtualAddress и Size, проверяет согласованность нулевых/ненулевых полей, границы SizeOfImage, возможность RVA-to-file-offset mapping, физическую доступность bytes в файле и кратность размера 12 байтам. Только после этого строится массив RUNTIME_FUNCTION. Каждая запись затем проходит отдельные проверки диапазонов, OOB, сортировки, пересечений, UnwindInfoAddress, доступности и корректности UNWIND_INFO, chain/handler data и placement прав секций. Поэтому parser не доверяет заголовкам слепо, а постепенно доказывает корректность каждого следующего шага.

ДОПОЛНЕНИЕ: ТРИ СОСТАВНЫЕ ЧАСТИ ПРОЕКТА

71. Архитектура проекта как три блока

Проект удобно и правильно объяснять как систему из трех крупных блоков: parser, static analysis и dynamic validation. Это не просто три красивых слова для презентации. Такое деление отражает реальные границы ответственности в коде: сначала недоверенные bytes превращаются в структурированную модель, затем модель оценивается статическими эвристиками, затем при необходимости часть выводов проверяется во время выполнения на Windows.

Главная мысль для защиты: parser отвечает за корректность чтения, static analysis отвечает за объяснимые признаки и оценку, dynamic validation отвечает за runtime evidence. Если смешать эти роли, проект станет неясным: parser начнет 'угадывать подозрительность', static analysis начнет притворяться исполнением, а dynamic mode начнет выглядеть как sandbox. В текущей ВКР эти роли разделены намного аккуратнее.

Общая цепочка: `input PE bytes -> parser model -> static findings/score/confidence -> optional dynamic evidence -> JSON/HTML/CSV report and assessment`.

71.1. Цели трех блоков

Блок	Главная цель	Что принимает	Что выдает	За что не отвечает
`Parser`	Безопасно прочитать недоверенный PE32+ AMD64 файл и построить структурную модель без выхода за границы.	Путь к файлу, bytes файла, PE headers, section table, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO RVA.	`Buf`, `PeImage`, массив `RuntimeFunction`, массив `UnwindSummary`, `us_ok[]`, parse errors/anomalies.	Не делает финальный verdict и не утверждает suspiciousness сам по себе.
`Static analysis`	Преобразовать parsed model в объяснимые findings, score, confidence, mitigations и static verdict.	`PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`, `us_ok[]`.	`HeuristicReport`: 27 findings, families, severity, score, suspicion, confidence, evidence level, manual review flag.	Не исполняет файл и не доказывает runtime behavior.
`Dynamic validation`	Запустить target под Windows debugger-ом и собрать runtime evidence по exception/unwind behavior.	Путь к target, argv, static model, timeout, trace_steps.	`DynamicReport`: exceptions, handler hits, VEH/VCH/UEF, dynamic function table calls, events, trace.	Не является полноценной malware sandbox и не заменяет static parser.

71.2. Где эти блоки находятся в коде

Блок	Основные файлы	Ключевые функции/структуры
`Parser foundation`	`src/util.c`, `include/util.h`	`Buf`, `StrBuf`, `buf_init_from_file`, `buf_read`, `buf_read_u32`, `file_write_atomic`, `sha256_hex`.
`PE parser`	`src/pe.c`, `include/pe.h`	`PeImage`, `PeSection`, `pe_parse`, `pe_find_section_for_rva`, `pe_rva_to_fileoff`.
`Unwind parser`	`src/unwind.c`, `include/unwind.h`	`RuntimeFunction`, `UnwindSummary`, `unwind_read_runtime_functions_ex`, `unwind_parse`.
`Static analysis`	`src/heuristics.c`, `include/heuristics.h`	`HeuristicReport`, `HeuristicFinding`, `heuristics_analyze`, scoring constants, thresholds, mitigations.
`Dynamic validation`	`src/dynamic_win.c`, `include/dynamic.h`	`DynamicReport`, `DynamicEvent`, `dynamic_collect_windows`, breakpoint/event logic.
`Output/assessment bridge`	`src/report.c`, `include/report.h`	`report_write_json_ex`, `report_write_html`, `assessment_summary`, static-dynamic correlation.
`Orchestration`	`src/main.c`	`parse_args`, `main`, вызовы parser -> static -> dynamic -> reports`.

72. Блок 1: Parser

72.1. Цель parser-a

Цель parser-a - не найти malware и не выставить score. Его цель ниже уровнем и важнее для надежности: взять недоверенный бинарный файл и безопасно превратить его bytes в осмысленные структуры PE/unwind. Слово 'безопасно' здесь означает: каждое чтение должно знать offset, size и границу буфера; каждый RVA должен быть сопоставлен с file offset только если bytes реально существуют в файле; каждый заявленный размер должен проверяться до чтения.

Parser в этой ВКР состоит из нескольких уровней. `util.c` дает безопасные примитивы чтения. `pe.c` строит PE-модель. `unwind.c` читает Exception Directory как массив RUNTIME_FUNCTION и декодирует UNWIND_INFO. Эти части вместе дают не verdict, а фактическую карту файла, на которую уже может опираться static analysis.

Parser layer	Задача	Код
`Byte layer`	Прочитать файл в `Buf`, читать маленькие значения только через checked reads.	`src/util.c:13-67`, declarations: `include/util.h:20-27`
`PE header layer`	Проверить DOS, NT, Machine AMD64, OptionalHeader magic PE32+.	`src/pe.c:11-90`, constants: `include/pe.h:16-19`
`Image layout layer`	Прочитать `ImageBase`, `SizeOfImage`, `SizeOfHeaders`, alignments, sections.	`src/pe.c:95-165`
`Section validation layer`	Посчитать raw/virtual OOB, overlaps, misalignment, exception directory OOB.	`src/pe.c:174-255`
`RVA mapping layer`	Найти секцию по RVA, запретить чтение zero-fill tail, учитывать headers.	`src/pe.c:265-300`
`Runtime function layer`	Проверить `Exception Directory`, кратность 12, прочитать записи RUNTIME_FUNCTION.	`src/unwind.c:611-671`
`UNWIND_INFO layer`	Декодировать header, flags, unwind codes, operands, chain, handler.	`src/unwind.c:56-604`

72.2. Как parser защищается от битого файла

Входной PE-файл в проекте рассматривается как hostile input. Это значит, что нельзя доверять `NumberOfSections`, `SizeOfOptionalHeader`, `SizeOfImage`, `PointerToRawData`, `SizeOfRawData`, `Exception Directory Size`, `CountOfCodes` и любым RVA. Любое из этих полей может быть повреждено или специально подобрано так, чтобы parser прочитал за пределами буфера.

Поэтому parser сначала проверяет физическую границу файла через `buf_read`, затем логическую границу image через `SizeOfImage`, затем секционную карту через raw/virtual ranges, затем возможность RVA-to-file-offset mapping. Такой порядок важен: нельзя читать UNWIND_INFO, пока не доказано, что его RVA вообще можно связать с bytes на диске.

Опасность	Как parser отвечает	Пример кода
`Offset указывает за файл`	`buf_read` возвращает false, чтение не выполняется.	`src/util.c:42-48`
`Не PE или неверный заголовок`	`pe_parse` останавливается на DOS/NT checks.	`src/pe.c:40-51`
`Не та архитектура`	Только AMD64 `0x8664` и PE32+ `0x20b`; иначе unsupported.	`src/pe.c:67-90`
`Секции заявляют невозможные диапазоны`	Считаются counters raw_oob, virtual_oob, overlaps, misaligned.	`src/pe.c:193-220`
`Exception Directory выходит за image`	Выставляется `exception_directory_oob`.	`src/pe.c:230-255`
`RVA попал в zero-fill tail`	`pe_rva_to_fileoff` возвращает false; bytes не читаются.	`src/pe.c:279-286`
`Exception Directory не похож на массив RUNTIME_FUNCTION`	Проверяется `Size >= 12` и `Size % 12 == 0`.	`src/unwind.c:639-647`
`UNWIND_INFO claims too many slots`	Проверяется `codes_off + codes_bytes <= b->size`.	`src/unwind.c:233-237`

72.3. Что parser выдает наверх

Результат parser-a - это не строка текста, а несколько C-структур. `PeImage` хранит image base, size of image, size of headers, alignments, exception directory, sections и counters структурных проблем. Массив `RuntimeFunction` хранит begin/end/unwind RVA. Массив `UnwindSummary` хранит то, что удалось извлечь из UNWIND_INFO: version, flags, prolog size, stack model, handler, chain, fixed stack bytes, saved registers, anomalies и parse_error.

Именно это разделяет parser и static analysis. Parser говорит: 'я прочитал вот такие структуры, а здесь чтение невозможно или есть anomalies'. Static analysis дальше решает, какие из этих фактов становятся findings и как они влияют на score/confidence.

72.4. Границы parser-а

Parser не обязан и не должен сам говорить, что файл вредоносный. Если parser начнет выставлять security verdict, он смешает слой чтения формата со слоем интерпретации. В текущей ВКР это разделено: parser строит модель, heuristics оценивает модель.

73. Блок 2: Static analysis

73.1. Цель static analysis

Цель static analysis - взять уже распарсенную модель и превратить ее в объяснимое evidence. На этом уровне проект отвечает на вопрос: какие части EH/unwind-модели выглядят несогласованно, поврежденно, необычно или требующими ручной проверки? Static analysis не запускает файл, но оно умеет связать разные признаки: PE layout, function table, UNWIND_INFO, handler placement, reuse/dominance и combo-сигналы.

Ценность static block в том, что он делает выводы не одной строкой. Он формирует `HeuristicReport`: score 0..100, suspicion, confidence, evidence families, findings, severity, affected_count, representative_rva, mitigations, evidence_level, verdict и флаг manual review.

Static sublayer	Что делает	Где в коде
`Evidence model`	Определяет семьи evidence и структуру finding/report.	`include/heuristics.h:17-92`
`Weights and thresholds`	Фиксирует веса findings, confidence constants и пороги reuse/dominance.	`src/heuristics.c:31-84`
`Analysis entry point`	Главная функция static block.	`src/heuristics.c:590`
`Table integrity checks`	Проверяет invalid ranges, OOB, unsorted, overlaps, duplicate begins.	`src/heuristics.c:647-765`, findings: `src/heuristics.c:850-890`
`Unwind integrity checks`	Проверяет parse fail и anomalies.	counters: `src/heuristics.c:728-732`, findings: `src/heuristics.c:893-905`
`Placement checks`	Проверяет sections для UNWIND_INFO и handler: exec/write/nonexec.	`src/heuristics.c:690-722`, findings: `src/heuristics.c:965-1000`
`Reuse/dominance checks`	Считает reuse ratios для UnwindInfoRVA и handler target.	ratios: `src/heuristics.c:779-785`, findings: `src/heuristics.c:909-960`
`Combo checks`	Усиливает вывод при сочетании независимых признаков.	`src/heuristics.c:1003-1022`
`Verdict/confidence`	Считает evidence level, confidence, mitigations, verdict.	`src/heuristics.c:504-588`

73.2. Что именно считается статическим признаком

Статический признак в проекте - это не любое странное поле PE. Признак должен относиться к теме ВКР: доверие к EH/unwind metadata. Поэтому первая группа findings покрывает PE layout, потому что без корректной секционной карты нельзя доверять RVA. Вторая группа покрывает RUNTIME_FUNCTION, потому что это карта функций. Третья группа покрывает UNWIND_INFO, потому что это инструкция для восстановления кадра. Четвертая группа покрывает handler placement и reuse, потому что handler layer может показывать exception redirection или dispatcher-style поведение.

Static analysis intentionally не превращен в общий PE scanner. Например, он не делает полный аудит всех imports, TLS callbacks, resources, relocations или общую W^X-политику всех секций. Он проверяет права секций там, где это связано с UNWIND_INFO и handler targets.

Группа признаков	Что защищает	Примеры findings
`PE layout`	Координатную систему file/raw/RVA/image.	`pe_section_raw_oob`, `pe_section_virtual_oob`, `pe_section_raw_overlap`, `pe_exception_directory_oob`
`Function table`	Карту функций для RIP -> function range.	`invalid_ranges`, `rf_oob`, `rf_unsorted`, `rf_overlaps`, `duplicate_begin`

Группа признаков	Что защищает	Примеры findings
`Unwind integrity`	Возможность корректно прочитать и интерпретировать UNWIND_INFO.	`unwind_parse_fail`, `unwind_anomalies`
`Metadata placement`	Разумность секции и прав для UNWIND_INFO/handler.	`unwind_in_exec_section`, `unwind_writable_section`, `handler_nonexec_section`
`Reuse/dominance`	Централизацию metadata или handler targets.	`unwind_reuse_strong`, `unwind_reuse_moderate`, `handler_dominance`
`Combos`	Корреляцию нескольких статических странностей.	`placement_integrity_combo`, `reuse_dominance_combo`, `writable_metadata_handler_combo`

73.3. Как static analysis делает вывод осторожным

Статический блок не просто суммирует баллы. Он еще оценивает силу evidence. Confidence увеличивается от количества evidence families, strong findings и medium findings. Но confidence уменьшается, если сигнал одиночный, выборка маленькая, признак только placement-only или все evidence из одной family. Это важно для защиты от false positives.

Например, один необычный section name сам по себе не должен звучать как 'вредонос'. Но необычная секция плюс parse fail плюс writable handler уже формируют более сильную картину. Именно для этого в проекте есть families, combos и mitigations.

Механизм	Для чего нужен	Код
`Evidence families`	Показывают, на каких слоях есть подтверждение.	`include/heuristics.h:14-20`, names: `src/heuristics.c:367-391`
`Score`	Численная triage-шкала suspiciousness EH/unwind-модели.	`src/heuristics.c:31-64`, scoring calls throughout `src/heuristics.c:790-1022`
`Evidence level`	Переводит counts/score в none/weak/medium/strong.	`src/heuristics.c:504-512`
`Confidence`	Оценивает уверенность интерпретации, не вероятность malware.	`src/heuristics.c:531-572`
`Mitigations`	Ослабляют вывод при одиночных/малых/односемейных сигналах.	`src/heuristics.c:542-568`
`Manual review`	Флаг, что нужен ручной анализ в Ghidra/debugger.	`src/heuristics.c:575-582`

73.4. Связь static analysis с отчетами

Static analysis сам формирует `HeuristicReport`, но пользователю нужен отчет. Поэтому `report.c` сериализует static findings в JSON/HTML/CSV и строит assessment block. Assessment объединяет static evidence level, runtime evidence level, dynamic_signal_count, final_confidence и final_verdict.

Это важная архитектурная деталь: отчет не просто печатает внутренние структуры. Он делает результат проверяемым: можно открыть JSON, увидеть конкретный finding, affected_count, representative_rva, section, confidence, weakening rules и manual validation steps.

74. Блок 3: Dynamic validation

74.1. Цель dynamic validation

Цель dynamic validation - не заменить static analysis и не стать полноценной sandbox. Его задача уже: проверить, проявляются ли EH/unwind-related сигналы во время выполнения. Например, статически найденный handler может никогда не сработать на обычном path, а может активно использоваться как dispatcher. Dynamic block собирает evidence, который помогает отличить 'странность в файле' от 'странность, реально участвующая в exception flow'.

Dynamic block доступен только на Windows, потому что он опирается на Windows Debug API и x64 exception behavior. Он запускает target как child process, ставит breakpoints на интересующие handler/API addresses, слушает debug events, считает first/second chance exceptions, handler hits, VEH/VCH/UEF registrations/hits, calls to dynamic function table APIs и записывает bounded RIP trace.

Dynamic sublayer	Что делает	Где в коде
`Data model`	Описывает `DynamicEvent`, `DynamicReport`, limits events/trace.	`include/dynamic.h:26-117`

Dynamic sublayer	Что делает	Где в коде
`Entry point`	Главная функция dynamic mode.	`src/dynamic_win.c:657`
`Process launch`	Формирует command line и запускает target под debugger-ом.	`src/dynamic_win.c:499-710`
`Debug loop`	Ждет debug events, обрабатывает process/dll/exception/exit.	`src/dynamic_win.c:752-1145`
`Breakpoints`	Ставит/снимает software breakpoints и перевооружает их.	`src/dynamic_win.c:200-238`, `src/dynamic_win.c:1050-1118`
`API hooks`	Ставит breakpoints на VEH/VCH/UEF и dynamic function table APIs.	`src/dynamic_win.c:561-593`
`Event recording`	Добавляет structured runtime events с cap.	`src/dynamic_win.c:416-430`
`Trace`	Записывает ограниченный RIP trace после handler hit.	`src/dynamic_win.c:617-636`, `src/dynamic_win.c:1083`

74.2. Какие runtime-сигналы собирает dynamic block

Dynamic block смотрит не весь процесс вообще, а конкретные сигналы, связанные с exception handling и unwind. Это соответствует фокусу ВКР. Он считает исключения, различает first chance и second chance, фиксирует попадания в handlers из static UNWIND_INFO, наблюдает регистрации vectored/unhandled handlers и видит runtime registration of function tables.

Runtime signal	Что означает	Где хранится/пишется
`Exceptions`	Были ли exception events и какого типа они были.	`DynamicReport.exceptions_seen`, `first_chance_seen`, `second_chance_seen`; model: `include/dynamic.h:58-63`
`Unwind handler hits`	Сработал ли handler, найденный статикой в UNWIND_INFO.	`unwind_handler_bp_set`, `unwind_handler_hits`; `include/dynamic.h:65-68`
`VEH/VCH`	Регистрируются или срабатывают vectored handlers.	`veh_registrations`, `veh_hits`, `vch_registrations`, `vch_hits`; `include/dynamic.h:70-73`
`UEF`	Ставится или срабатывает unhandled exception filter.	`uef_set_calls`, `uef_hits`; `include/dynamic.h:74-75`
`Dynamic function tables`	Процесс добавляет runtime unwind/function table.	`rtl_add_function_table_calls`, `rtl_install_function_table_cb_calls`; `include/dynamic.h:77-78`
`Events`	Structured timeline с type/module/thread/address/args.	`DynamicEvent` and `events[]`; `include/dynamic.h:28-39`, `include/dynamic.h:82`
`Trace`	Короткая RIP-цепочка после handler hit.	`trace_count`, `trace[256]`; `include/dynamic.h:84-86`

74.3. Как dynamic block работает пошагово

Сначала `main.c` проверяет CLI options: `--dynamic`, `--timeout-ms`, `--trace-steps`. Значения по умолчанию: timeout 15000 ms, trace 64 шага. Затем `main.c` передает в `dynamic_collect_windows` не только путь к target, но и результаты static parser-а: `PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`, `us_ok[]`. Это важно: dynamic block использует статически найденные handler addresses для постановки breakpoints.

После запуска target под debugger-ом dynamic block обновляет список модулей, ставит API breakpoints, ставит breakpoints на handler addresses из UNWIND_INFO, затем в debug loop обрабатывает события. Когда breakpoint срабатывает, block понимает его kind: AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback или static unwind handler. Для handler hits он может начать single-step trace.

Шаг	Что происходит	Код
`CLI`	`--dynamic`, timeout и trace_steps разбираются в arguments.	`src/main.c:176-229`
`Call from main`	Передаются target path, argv, static model, timeout, trace_steps.	`src/main.c:405-408`
`CreateProcess`	Target запускается как debuggee.	`src/dynamic_win.c:657-710`
`Module/API setup`	Обновляются modules и ставятся API breakpoints.	`src/dynamic_win.c:789-816`, `src/dynamic_win.c:561-593`
`Static handler bps`	Breakpoints ставятся на handlers из `UnwindSummary`.	`src/dynamic_win.c:796-804`

Шаг	Что происходит	Код
`Debug event loop`	Обрабатываются create/load/unload/exit/exception events.	`src/dynamic_win.c:752-1145`
`Breakpoint hit`	Снимается breakpoint, читается context/registers, определяется kind.	`src/dynamic_win.c:832-1032`
`Rearm/continue`	Breakpoint перевооружается после single-step, процесс продолжается.	`src/dynamic_win.c:1050-1136`
`Final status`	Заполняются timed_out, target_exited, ok, instrumentation flags.	`src/dynamic_win.c:1143-1202`

74.4. Почему dynamic validation ограничен

Dynamic block специально ограничен timeout, event cap, trace cap и completeness flags. Это не слабость, а честная граница: инструмент не должен выдавать неполный debug run за доказательство. В report layer incomplete session понижает runtime evidence до weak.

Report layer оценивает dynamic evidence отдельно. `dynamic_session_complete` требует, чтобы session ran, ok, not timed out, target exited and instrumentation complete. Если session неполная, runtime evidence level не усиливается как полноценное подтверждение. Корреляция static-dynamic считается только если runtime evidence связан с static findings, например handler-focused static evidence плюс handler/VEH/VCH/UEF hits, или placement anomalies плюс runtime dynamic function table activity.

Report mechanism	Что делает	Код
`dynamic_signal_count`	Считает количество runtime signal families.	`src/report.c:326-336`
`dynamic_session_complete`	Проверяет полноту dynamic run.	`src/report.c:338-345`
`static-dynamic correlation`	Связывает static handler/placement findings с runtime hits/API calls.	`src/report.c:347-359`
`runtime_evidence_level`	Переводит runtime signals в none/weak/medium/strong.	`src/report.c:362-374`
`case_strength`	Строит итоговую силу case: correlated/unconfirmed/weak/no case.	`src/report.c:377-392`
`assessment_summary`	Считает final verdict/confidence для JSON/HTML.	`src/report.c:405-440`
`JSON dynamic block`	Сериализует dynamic_validation and assessment.	`src/report.c:782-860`

75. Как три блока связаны в main.c

`main.c` - это оркестратор. Он не содержит всю логику parser-a, static analysis или dynamic validation внутри себя. Его задача - вызвать блоки в правильном порядке, передать результаты дальше и корректно освободить память. Порядок важен: dynamic block не запускается до parser-a, потому что ему нужны static handler addresses; reports не пишутся до heuristics/dynamic, потому что assessment зависит от их результатов.

Этап в main	Что вызывает	Зачем
`Read file`	`buf_init_from_file`	Получить bytes файла как `Buf`.
`PE parse`	`pe_parse`	Построить `PeImage` и section model.
`Runtime functions`	`unwind_read_runtime_functions_ex`	Получить `.pdata` как массив `RuntimeFunction`.
`Unwind parse loop`	`unwind_parse` для каждого `rf[i]`	Получить `UnwindSummary[]` и `us_ok[]`.
`Static analysis`	`heuristics_analyze`	Сформировать `HeuristicReport`.
`Dynamic optional`	`dynamic_collect_windows`	Собрать `DynamicReport`, если включен `--dynamic`.
`Reports`	`report_write_csv`, `report_write_json_ex`, `report_write_html`	Выдать результаты человеку и инструментам.
`Cleanup`	`heuristics_free`, `dynamic_report_free`, `pe_free`, `buf_free_owned`	Освободить память и не оставить ресурсных хвостов.

76. Что говорить комиссии

Проект состоит из трех блоков. Parser нужен, чтобы безопасно превратить недоверенные bytes PE32+ AMD64 в проверенную модель структур. Static analysis нужен, чтобы из этой модели получить объяснимые findings, score, confidence, mitigations и static verdict. Dynamic validation нужен, чтобы на Windows проверить, проявляются ли EH/unwind-related подозрения во время выполнения: handler hits, VEH/VCH/UEF, first/second chance exceptions и dynamic function table APIs. Такое разделение делает архитектуру проверяемой: каждый блок имеет свою цель, вход, выход и границы ответственности.

Вопрос комиссии	Ответ
`Почему три блока?`	Потому что чтение формата, интерпретация признаков и runtime-проверка - разные задачи с разными рисками и разными результатами.
`Почему parser отдельно?`	Чтобы не смешивать безопасность чтения bytes с эвристическими выводами. Parser должен быть максимально строгим и предсказуемым.
`Почему static отдельно?`	Чтобы findings, weights, thresholds, confidence и mitigations были прозрачной evidence model, а не побочным эффектом parser-a.
`Почему dynamic отдельно?`	Потому что запуск target - отдельная trust boundary. Он нужен только для runtime evidence и требует Windows/debugger/timeout limits.
`Что является главным результатом parser-a?`	`PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`, `us_ok[]` и parse/anomaly signals.
`Что является главным результатом static?`	`HeuristicReport`: 27 findings, score, confidence, evidence level, verdict, manual review flag.
`Что является главным результатом dynamic?`	`DynamicReport`: debug events, handler hits, VEH/VCH/UEF, dynamic function table calls, trace and completeness flags.

ПРОФЕССИОНАЛЬНОЕ ТЕХНИЧЕСКОЕ ОПИСАНИЕ РАБОТЫ ПРОЕКТА

77. Почему описание нельзя сводить к набору if

Поверхностный взгляд на проект может создать ошибочное впечатление, что он состоит из простых проверок вида 'если поле плохое, добавить признак'. Это неверная оценка уровня задачи. В системном бинарном анализе отдельная проверка действительно может выглядеть как короткий `if`, но ценность проекта определяется не длиной одной ветки, а корректной композицией многих проверок, адресных преобразований, ABI-инвариантов, обработкой поврежденных входов и сохранением объяснимой evidence model.

Проект работает не с доверенной JSON-структурой и не с формой сайта. Его вход - недоверенный PE-файл, где любое поле может быть повреждено: offset, RVA, count, size, alignment, section range, unwind code, handler RVA. Ошибка в одном месте меняет смысл всех последующих слоев. Например, неверное сопоставление RVA и file offset приведет к чтению не того UNWIND_INFO; ошибка в границах секций исказит права handler target; ошибка в таблице функций даст ложные overlaps или пропустит реальную аномалию.

Поэтому сложность проекта является не UI-сложностью, а системной: нужно удерживать несколько координатных систем, безопасно читать байты, строить промежуточные модели, различать parse failure и suspicious finding, не исполнять файл в static mode, а в dynamic mode явно перейти через границу запуска target под debugger-ом.

Правильная формулировка: проект реализует не 'набор if', а конвейер восстановления доверия к EH/unwind-слою PE32+ AMD64: bytes -> PE image model -> function table model -> unwind model -> evidence model -> optional runtime evidence -> reproducible report.

78. Формальная модель конвейера

Проект можно описать как последовательность преобразований состояния. На каждом этапе есть вход, выход, инварианты и failure mode. Следующий этап не должен предполагать, что предыдущий 'примерно прав': он получает только проверенную модель или явно зафиксированную ошибку.

Этап	Вход	Выход	Ключевой инвариант
`File loading`	path	`Buf {data, size}`	Все дальнейшие чтения должны быть ограничены `b->size`; реализация в `src/util.c:13-67`.
`PE parsing`	`Buf`	`PeImage`	Файл является PE32+ AMD64; headers, sections, alignments и data directory прочитаны в проверенных границах; `src/pe.c:36-255`.
`RVA mapping`	`PeImage + RVA`	file offset или отказ	RVA читается из файла только если есть реальные raw bytes; zero-fill не читается; `src/pe.c:277-300`.
`Runtime table reading`	Exception Directory	`RuntimeFunction[]`	Directory - массив 12-byte записей; `Size % 12 == 0`; `src/unwind.c:611-671`.
`UNWIND_INFO decoding`	`UnwindInfoAddress`	`UnwindSummary`	Header, slots, operands, handler/chain tail читаются с проверкой границ; `src/unwind.c:56-604`.
`Static evidence`	Parsed models	`HeuristicReport`	Каждая аномалия классифицируется как finding/family/severity/score/confidence; `src/heuristics.c:590-1029`.
`Dynamic evidence`	Static model + target	`DynamicReport`	Runtime events собираются в bounded debugger session; `src/dynamic_win.c:657-1202`.
`Reporting`	Models + metadata	JSON/HTML/CSV	Отчеты сериализуются с validation, escaping, atomic write; `src/report.c:556-1515`, `src/util.c:190`.

79. Parser как система инвариантов

Профессиональный parser бинарного формата отличается от учебного тем, что он не 'читает поле', а доказывает допустимость чтения. В проекте это выражено через несколько уровней инвариантов. Первый уровень - физический: `off + len` не должен выходить за `file_size`. Второй - структурный: DOS/NT/COFF/Optional Header должны быть согласованы. Третий - виртуальный: RVA должен лежать внутри image и сопоставляться с секцией или headers. Четвертый - предметный: Exception Directory должен быть массивом RUNTIME_FUNCTION, а UNWIND_INFO должен соответствовать layout Windows x64 ABI.

Это важно для защиты, потому что одна строка `if (size % 12 != 0)` сама по себе проста, но она появляется только после цепочки проверок: файл прочитан, PE header найден, OptionalHeader достаточно большой, DataDirectory существует, RVA/Size прочитаны, область не выходит за image, RVA маппится в raw bytes, область помещается в file_size. Сложность здесь в порядке доказательства, а не в синтаксисе отдельного условия.

Инвариант	Что предотвращает	Где реализовано
`Checked byte reads`	Чтение за пределами `Buf`.	`buf_read`, `buf_read_u16/u32/u64`: `src/util.c:42-67`
`PE identity`	Интерпретацию произвольного файла как PE32+ AMD64.	DOS/NT/Machine/Magic: `src/pe.c:11-90`, constants: `include/pe.h:16-19`
`Header bounds`	Переход по `e_lfanew` или OptionalHeader за конец файла.	`src/pe.c:45`, `src/pe.c:73-80`
`Image layout consistency`	Нулевой `SizeOfImage`, плохие headers, несогласованные alignments.	`src/pe.c:109-123`, `src/pe.c:174-193`
`Section range consistency`	Raw/virtual OOB и неоднозначное отображение секций.	`src/pe.c:193-224`
`RVA-to-file mapping correctness`	Чтение virtual zero-fill как будто это raw bytes.	`src/pe.c:277-300`
`Exception table shape`	Интерпретацию произвольной области как RUNTIME_FUNCTION[].	`src/unwind.c:623-647`
`UNWIND_INFO grammar`	Чтение неверного tail/operand layout.	`slots_for_code`: `src/unwind.c:24-53`, parse loop: `src/unwind.c:254-516`

80. Адресные пространства и почему это не тривиально

В проекте одновременно существуют несколько адресных пространств: file offset, RVA, VA, raw section range, virtual section range, headers range, function range, unwind blob range и handler target. Ошибка возникает, когда их начинают смешивать. Например, RVA не является file offset. Section `VirtualSize` важен для memory layout, а `SizeOfRawData` - для чтения bytes из файла. `SizeOfImage` ограничивает виртуальный образ, но не гарантирует физическую читаемость bytes.

ВКР специально разделяет эти понятия. `pe_find_section_for_rva` отвечает на вопрос 'какой section role/permissions соответствует RVA'. `pe_rva_to_fileoff` отвечает на другой вопрос: 'можно ли этот RVA физически прочитать из файла'. Вторая операция строже первой, потому что static parser не может читать zero-fill bytes, которые loader создаст только в памяти.

Координата	Что означает	Ошибка при смешении
`file offset`	Смещение в bytes от начала файла.	Можно прочитать за пределами файла или не те bytes.
`RVA`	Адрес внутри image относительно ImageBase.	Можно принять виртуальный адрес за физическое смещение.
`VA`	`ImageBase + RVA` в процессе.	В static parser-е VA без загрузки process не является file offset.
`raw section range`	`PointerToRawData + SizeOfRawData`.	Можно считать виртуальный tail существующим на диске.
`virtual section range`	`VirtualAddress + span`.	Можно не заметить overlap или выход за `SizeOfImage`.
`headers range`	`0..SizeOfHeaders` внутри image.	Можно ошибочно требовать, чтобы любой RVA лежал в обычной секции.
`function range`	`BeginAddress..EndAddress` в RUNTIME_FUNCTION.	Можно получить неоднозначный RIP -> function mapping.
`unwind blob range`	Header, codes, aligned slots, tail.	Можно прочитать handler или chain из неверного offset.

81. Алгоритмическая часть static analysis

Static analysis - это не плоский список условий. Внутри есть несколько алгоритмических задач: подсчет структурных counters из `PeImage`, проход по RUNTIME_FUNCTION для локальных проверок, построение hash maps для reuse/dominance, сортировка копии function ranges для overlap/duplicate detection, вычисление ratios, применение thresholds, построение findings и расчет confidence с weakening rules.

Например, overlap нельзя надежно проверить только сравнением с предыдущей исходной записью, потому что таблица может быть несортированной. Поэтому проект создает массив `RfIdx`, сортирует его через `qsort`, затем проверяет duplicate begins и overlaps на отсортированном представлении. Для reuse используется собственная hash map, чтобы найти максимальное количество ссылок на один `UnwindInfoRVA` и один handler target.

Задача	Техническая реализация	Сложность/смысл
`Local range checks`	Один проход по `rf[i]`: invalid ranges, OOB, unwind OOB.	O(n); `src/heuristics.c:647-662`.

Задача	Техническая реализация	Сложность/смысл
Reuse counting	HMap unwind_counts, HMap handler_counts, hmap_inc.	O(n) expected; src/heuristics.c:169-275, src/heuristics.c:630-685.
Sorted overlap detection	Копия ranges в RfIdx, qsort, проверка begin < max_end.	O(n log n); src/heuristics.c:741-765.
Ratio computation	unwind_reuse_ratio, handler_reuse_ratio, safe_ratio.	Нужно избежать деления на ноль и переоценки малых выборок; src/heuristics.c:775-785.
Threshold logic	1%, 5%, 10%, 30%, 80% thresholds по типам signals.	Инженерная evidence matrix; src/heuristics.c:80-84, uses: src/heuristics.c:844-981.
Finding construction	add_finding сохраняет code/family/severity/message/location/count.	Делает вывод объяснимым, а не только числовым; src/heuristics.c:408-449.
Confidence and mitigation	Бонусы/штрафы за families, strong/medium, isolated/small/placement-only.	Снижает false positives; src/heuristics.c:514-588.

82. UNWIND_INFO decoder как предметная логика ABI

Декодер UNWIND_INFO - не простое чтение структуры фиксированного размера. Формат содержит переменную длину: header задает CountOfCodes, каждый opcode занимает разное количество 2-byte slots, количество slots выравнивается, а tail зависит от flags. Если выставлен CHAININFO, после codes лежит chained RUNTIME_FUNCTION; если выставлен EHANDLER/UHANDLER, там лежит handler RVA и далее language-specific data. Поэтому decoder должен вычислять layout, а не просто читать подряд несколько полей.

Проект учитывает v1/v2 особенности, slots count для операций, проверяет version, flags, frame register, prolog offsets, order of code offsets, operands, alloc/save sanity, CHAININFO restrictions, handler OOB, loops и depth при follow-chain. Это уже предметная логика Windows x64 unwind ABI.

Элемент decoder-а	Почему он сложнее обычного чтения	Код
Opcode slot sizing	Разные unwind opcodes занимают 1, 2 или 3 slots; v2 epilog имеет отдельную логику.	src/unwind.c:24-53
Header parsing	Version/flags/prolog/count/frame fields задают дальнейший layout.	src/unwind.c:165-237
Operand parsing	ALLOC/SAVE/FAR/XMM/MACHFRAME читают разные operands и sanity-checks.	src/unwind.c:254-516
Tail dispatch	Tail зависит от flags: handler или chained record.	src/unwind.c:520-550
Chain following	Нужны loop/depth guards и merge summary.	src/unwind.c:563-604
Anomaly taxonomy	Разные причины не сваливаются в одну ошибку: version, truncation, opcode, operand, order, chain, handler, ABI.	include/unwind.h:53-69
Parse failure preservation	parse_error сохраняет причину, чтобы report был объяснимым.	src/unwind.c:143-163, include/unwind.h:43

83. Evidence model как инженерная система, а не магическое число

Static result в проекте не сводится к числу score. Score - лишь один из выходов evidence model. Настоящая модель содержит families, severity, affected_count, representative_rva, mitigation rules, evidence level, confidence and manual review policy. Это важно, потому что один и тот же score без контекста малоценен: аналитик должен видеть, какие слои повреждены и насколько признаки независимы.

Профессиональная часть здесь - в разделении suspiciousness и confidence. suspicion нормализуется из score. confidence строится отдельно: растет при нескольких families и strong findings, но падает при single signal, small sample, placement-only или single-family cases. Это показывает, что проект не делает грубый детектор, а учитывает неопределенность выводов.

Компонент	Профессиональный смысл	Код
Finding code	Стабильный идентификатор признака для отчета, тестов и защиты.	HeuristicFinding.code: include/heuristics.h:25-33
Family	Показывает слой evidence: table, unwind, placement, reuse, handler dominance.	enum HeuristicEvidenceFamily: include/heuristics.h:14-20
Severity	Отделяет weak/medium/high signals от количества occurrences.	add_finding: src/heuristics.c:408-449

Компонент	Профессиональный смысл	Код
`Affected count`	Показывает масштаб проблемы, а не только факт ее наличия.	`HeuristicFinding.affected_count`: `include/heuristics.h:31`
`Representative RVA`	Дает точку входа для ручной проверки в Ghidra/PE viewer.	`remember_location`: `src/heuristics.c:287-299`
`Mitigations`	Формально ослабляют вывод при неоднозначных условиях.	`src/heuristics.c:480-495`, uses: `src/heuristics.c:542-568`
`Evidence level`	Переводит набор findings в none/weak/medium/strong.	`src/heuristics.c:504-512`
`Confidence`	Инженерная уверенность интерпретации, не вероятность malware.	`src/heuristics.c:531-572`

84. Dynamic validation как событийная debugger-система

Dynamic part нельзя описывать как 'запустить файл'. Реализация использует Windows Debug API и работает как event-driven state machine. Target запускается как debuggee, затем проект обрабатывает поток событий: create process, load/unload DLL, exception events, breakpoint hits, single-step events, process exit, timeout. Внутри этого цикла нужно управлять breakpoints, временно снимать их, корректировать RIP, перевооружать после single-step и продолжать процесс через `ContinueDebugEvent`.

Важная профессиональная граница: dynamic validation не объявляет себя sandbox. Он собирает bounded evidence именно по EH/unwind теме: handler hits, VEH/VCH/UEF registration/hits, RtlAddFunctionTable/RtlInstallFunctionTableCallback, first/second chance exceptions, trace after handler hit. Runtime session имеет timeout, event cap, trace cap и completeness flags.

Механизм	Что усложняет реализацию	Код
`Debuggee launch`	Нужно корректно собрать command line и запустить процесс с debug flags.	`src/dynamic_win.c:499-710`
`Debug loop`	Нужно обрабатывать разные event types и не зависнуть.	`src/dynamic_win.c:752-1145`
`Breakpoints`	Software breakpoint меняет байт в target process; его надо снять и вернуть.	`src/dynamic_win.c:165-238`
`API instrumentation`	Breakpoints ставятся на runtime APIs только когда modules готовы.	`src/dynamic_win.c:561-593`
`Static handler breakpoints`	Static UNWIND_INFO дает адреса handler-ов, dynamic ставит на них breakpoints.	`src/dynamic_win.c:796-804`
`Event recording`	События ограничены `DYNAMIC_MAX_EVENTS`, overflow фиксирует incomplete instrumentation.	`include/dynamic.h:26-39`, `src/dynamic_win.c:416-430`
`Trace`	После handler hit собирается bounded RIP trace, не бесконечная трассировка.	`src/dynamic_win.c:617-636`, `src/dynamic_win.c:1083`
`Completion semantics`	Incomplete/timeout session не должна усиливать вывод как полноценное подтверждение.	`src/dynamic_win.c:1143-1202`, `src/report.c:338-374`

85. Report layer как часть корректности

Отчетный слой в проекте не является декоративным выводом. Он выполняет несколько инженерных функций: проверяет model validity и metadata, экранирует JSON/HTML, пишет файлы атомарно, сохраняет build/version/schema/input hash, формирует manual validation steps и assessment summary. Это важно, потому что результат analyzer-а должен быть воспроизводимым и проверяемым.

Если analyzer нашел finding, но не может показать evidence, координату, affected count и manual step, такой результат слаб для защиты и для реального reverse engineering. Поэтому report layer - это часть архитектуры доверия, а не просто печать текста.

Функция report layer	Зачем нужна	Код
`Metadata validation`	Нельзя писать отчет без schema/tool/build/input metadata.	`src/report.c:556-620`
`Model validation`	Защищает report writer от противоречивых внутренних моделей.	`src/report.c:596-620`
`JSON escaping`	Исключает сломанный JSON при спецсимволах в строках.	`src/report.c:255-301`
`HTML escaping + CSP`	Исключает HTML injection и внешние зависимости отчета.	`src/report.c:933-964`, CSP: `src/report.c:1194`
`Assessment summary`	Объединяет static/dynamic evidence в final verdict/confidence.	`src/report.c:405-440`
`Manual validation steps`	Дает аналитику конкретные следующие действия.	`src/report.c:465-519`

Функция report layer	Зачем нужна	Код
`Atomic write`	Не портит предыдущий отчет при сбое записи.	`src/util.c:190`, uses: `src/report.c:212`, `src/report.c:918`, `src/report.c:1515`

86. Инженерная сложность проекта

Сложность ВКР можно обосновывать по нескольким измерениям. Во-первых, предметная сложность: PE32+ AMD64, Windows x64 exception handling, RUNTIME_FUNCTION, UNWIND_INFO, handler flags, chain info, section characteristics. Во-вторых, надежность: файл недоверенный, parser должен быть устойчивым к OOB, overlaps, malformed sizes, zero-fill, truncated structures. В-третьих, evidence model: 27 findings распределены по слоям и превращаются в score/confidence/mitigations. В-четвертых, dynamic validation: debugger event loop, breakpoints, runtime APIs, trace, timeout, completeness.

Даже по объему реализации видно, что проект не является коротким скриптом. Основные C-файлы содержат тысячи строк: dynamic_win.c - 1124 строки, report.c - 1436, heuristics.c - 948, unwind.c - 635, main.c - 488, pe.c - 272, util.c - 317. Но главное не количество строк, а то, что каждая часть решает отдельную системную задачу и связана с остальными через явные структуры.

Критерий сложности	Как проявляется в проекте
`Hostile input`	Входной PE может быть поврежденным или намеренно противоречивым; parser не доверяет header fields.
`Multiple coordinate systems`	file offset, RVA, VA, raw/virtual sections, headers, image, function ranges, unwind blob.
`ABI-specific logic`	UNWIND_INFO layout зависит от version, flags, slots, opcodes, chain/handler tail.
`Non-flat evidence`	Findings имеют code/family/severity/location/count; confidence и mitigations отделены от score.
`Algorithmic checks`	Sort for overlaps, hash maps for reuse/dominance, thresholds by prevalence and sample size.
`Runtime instrumentation`	Debugger loop, breakpoints, API hooks, handler hits, trace, timeout, incomplete session semantics.
`Reproducible output`	JSON/HTML/CSV reports with schema/version/hash/build metadata and manual validation steps.

87. Как отвечать на выпад 'это просто несколько if'

Ответ: отдельная проверка может выглядеть как простой `if`, но проектная сложность находится в доказательстве корректности всей цепочки: безопасное чтение недоверенного PE, согласование file/RVA/image координат, декодирование переменной структуры UNWIND_INFO, построение evidence model, защита от false positives через mitigations и runtime validation через Windows debugger. Если убрать любой слой, результат станет либо небезопасным, либо необъяснимым, либо непроверяемым.

Возражение комиссии	Технический ответ
`Parser - это несколько if.`	Parser проверяет не одно поле, а цепочку инвариантов: физический файл, PE identity, image layout, section ranges, RVA mapping, Exception Directory shape, UNWIND_INFO grammar.
`Статика - это набор условий.`	Статика строит evidence model: families, severity, locations, counts, ratios, thresholds, score, confidence, mitigations and manual review.
`Динамика - это просто запуск файла.`	Dynamic mode - debugger event system: CreateProcess debuggee, WaitForDebugEvent loop, breakpoints, API hooks, exception events, trace, timeout and completeness semantics.
`Можно написать скрипт.`	Скрипт может прочитать нормальный PE. ВКР обрабатывает malformed PE, сохраняет причины ошибок, не читает zero-fill, не смешивает RVA/file offset и формирует воспроизводимый отчет.
`Почему это сложнее CRUD?`	CRUD работает с доверенной схемой данных. Здесь вход - hostile binary format, где каждое поле может ломать последующие интерпретации и security findings.

88. Итоговая профессиональная формулировка

EH/Unwind Analyzer - это не скрипт для чтения нескольких PE-полей, а системный C17-инструмент для анализа доверия к Windows x64 EH/unwind metadata. Он реализует безопасный parser недоверенного PE32+ AMD64 файла, строгую модель file/RVA/image/section границ, decoder переменного формата UNWIND_INFO, статическую evidence model с 27 findings, score, confidence и mitigations, а также optional Windows Debug API validation для runtime exception/unwind behavior. Сложность проекта заключается в согласовании этих слоев и сохранении объяснимого, воспроизводимого вывода, а не в длине отдельного условия `if`.

РАСШИРЕННАЯ БЛОЧНАЯ РЕДАКЦИЯ ОПИСАНИЯ

Этот раздел специально разделяет работу проекта по инженерным блокам. Его задача - не смешивать parser, static analysis, dynamic validation и reporting в одну кучу, а показать границы ответственности каждого блока, входы, выходы, типичные граничные случаи и точные ссылки на код.

89. Карта блоков и границы ответственности

В проекте есть несколько самостоятельных блоков. Они связаны последовательно, но каждый отвечает за свой уровень реальности. Parser не решает, вредоносный файл или нет; он строит проверенную модель PE. Unwind reader не оценивает подозрительность; он извлекает function table или сообщает, что таблица отсутствует либо повреждена. Static analysis не запускает файл; он превращает структурные факты в evidence. Dynamic validation не заменяет static analysis; он добавляет наблюдения runtime поведения. Report layer не просто печатает текст; он сохраняет доказательную базу в воспроизводимом виде.

Такое разделение важно для защиты архитектуры. Если комиссия спрашивает, где сложность, ответ не в том, что каждый `if` длинный, а в том, что проект удерживает границы между блоками и не позволяет ошибке одного уровня незаметно стать ложным выводом другого уровня.

Блок	Главный вопрос	Вход	Выход
`1. File/byte safety`	Можно ли безопасно читать эти байты?	Путь к файлу	`Buf` и checked reads
`2. PE parser`	Является ли файл допустимой PE32+ AMD64 моделью?	`Buf`	`PeImage`
`3. Exception Directory`	Есть ли таблица функций и можно ли ее читать?	`PeImage.exception_rva/size`	`RuntimeFunction[]` или empty/partial state
`4. UNWIND_INFO parser`	Можно ли разобрать unwind metadata каждой функции?	`RuntimeFunction.unwind_rva`	`UnwindSummary[]`
`5. Static evidence`	Какие признаки нарушений видны без запуска?	`PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`	`HeuristicReport`
`6. Dynamic validation`	Подтверждается ли часть поведения при запуске под debugger?	target process + static model	`DynamicReport`
`7. Report layer`	Как доказательно сохранить результат?	Все модели и metadata	JSON/HTML/CSV report

90. Блок 1 - безопасный вход и byte-level чтение

Первый блок проекта отвечает за самую нижнюю гарантию: ни одна структура PE не читается напрямую из произвольного указателя. Файл сначала превращается в `Buf`, а затем все чтение идет через checked helpers. Это принципиально: вредоносный или поврежденный PE может указать offset за пределами файла, сделать размер слишком большим, поставить e_lfanew в конец файла или создать переполнение при сложении offset + size.

Поэтому проект работает как cautious parser. Он не говорит: 'в PE должно быть поле, значит оно есть'. Он говорит: 'перед чтением поля я доказываю, что эти bytes физически есть в файле'. Реализация чтения находится в `src/util.c:13-67`: загрузка файла, `buf_read`, `buf_read_u8/u16/u32/u64`. Именно этот слой защищает остальные блоки от выхода за границы буфера.

Проверка	Что защищает	Как объяснять комиссии
`off + len <= file_size`	Out-of-bounds read при поврежденных offsets/sizes.	Бинарный файл - недоверенный вход; parser не имеет права читать за пределы реальных bytes.
`Typed reads через helpers`	Ошибки endian/размера поля и повторение ручного кода.	Все 16/32/64-bit поля читаются единообразно и проверяемо.
`Ошибки через StrBuf err`	Потерю причины отказа.	Проект не просто падает, а сохраняет диагностическую причину.

91. Блок 2 - PE parser: от файла к модели образа

PE parser отвечает не за поиск 'плохости', а за построение модели `PeImage`. Сначала он проверяет DOS header и `e_lfanew`, затем NT signature, COFF header, machine type, OptionalHeader magic, базовые размеры image/header, alignment и количество data directories. После этого он читает section table и вычисляет структурные counters: raw OOB, virtual OOB, raw overlaps, virtual overlaps, misalignment, exception directory OOB.

Проект поддерживает именно PE32+ AMD64. Это видно по `IMAGE_FILE_MACHINE_AMD64 0x8664` и `IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b` в `include/pe.h:18-19`, а также по проверкам в `src/pe.c:67-89`. PE32 x86 не является текущей целевой архитектурой, потому что x64 EH/unwind имеет другую модель: RUNTIME_FUNCTION и UNWIND_INFO являются центральной частью темы ВКР.

Шаг parser-а	Что проверяется	Код
`DOS/e_lfanew`	Файл начинается как MZ/PE, offset PE header не выходит за файл.	`src/pe.c:40-48`
`PE signature`	По `e_lfanew` действительно лежит `PE\0\0`.	`src/pe.c:50-54`
`COFF Machine`	Только AMD64, не x86/ARM/другой формат.	`src/pe.c:56-70`
`OptionalHeader`	Header помещается в файл, magic равен PE32+ `0x20b`.	`src/pe.c:72-90`
`Image fields`	`ImageBase`, `SizeOfImage`, `SizeOfHeaders`, alignments, directory count.	`src/pe.c:92-123`
`Section table`	Секции читаются как модель raw/virtual ranges + characteristics.	`src/pe.c:146-193`
`Section invariants`	Raw/virtual OOB, overlaps, alignment violations.	`src/pe.c:193-224`
`Exception directory bounds`	RVA/Size directory согласованы и не выходят за `SizeOfImage`.	`src/pe.c:230-255`

92. Блок 3 - адресные пространства: file offset, RVA, image и headers

Одна из главных тонкостей проекта - не путать физическое расположение bytes в файле и виртуальное расположение объекта после загрузки loader-ом. `RVA` - это адрес относительно `ImageBase` внутри образа. `file offset` - это смещение в самом файле. Они часто связаны через section table, но не равны друг другу.

Проект использует две разные операции. `pe_find_section_for_rva` отвечает на вопрос, к какой секции относится RVA в виртуальном образе. `pe_rva_to_fileoff` отвечает на более строгий вопрос: можно ли реально прочитать bytes из файла. Если RVA попал в virtual tail секции, где `delta >= SizeOfRawData`, parser отказывается читать, потому что loader заполнит эту область нулями только в памяти. Если RVA попал в headers range `0..SizeOfHeaders`, он может маппиться напрямую на file offset. Это реализовано в `src/pe.c:277-300`.

Ключевой ответ комиссии: совпадение границ не означает, что raw layout и virtual layout должны быть одинаковыми. Они должны быть согласованы через PE section table. Для virtual проверок важен `VirtualAddress/VirtualSize/SizeOfImage`, для чтения bytes из файла важен `PointerToRawData/SizeOfRawData/file_size`.

93. Блок 4 - Exception Directory: отсутствует, повреждена или читается

Exception Directory - это запись `DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]`. На x64 она обычно указывает на таблицу `RUNTIME_FUNCTION`, то есть на `.pdata`-подобную область. Но в реальных файлах возможны три больших сценария: directory отсутствует, directory противоречива, directory присутствует и читается.

В проекте случай отсутствия Exception Directory учтен явно. Если `img->exception_rva == 0` и `img->exception_size == 0`, функция `unwind_read_runtime_functions_ex` возвращает `RUNTIME_FUNCTION_READ_OK`, оставляет `out_rf = NULL` и `out_count = 0`. Это находится в `src/unwind.c:620-625`. То есть отсутствие directory само по себе не является parser crash и не является malformed state.

Дальше static layer уточняет смысл отсутствия. Если `n == 0`, directory отсутствует, но в image есть исполняемая секция, проект добавляет finding `no_runtime_functions`, потому что для x64 executable image полное отсутствие runtime functions может быть допустимо только в ограниченных случаях, например leaf-only code, и требует ручного подтверждения. Это `src/heuristics.c:833-839`. Если `n == 0` и исполняемых секций нет, проект не усиливает подозрение, а добавляет note: это согласуется с resource-only или forwarding image. Это

src/heuristics.c:840-842.

Сценарий	Как проект реагирует	Почему это правильно
RVA=0 и Size=0	Runtime table считается отсутствующей, read result OK, count 0.	В PE zero/zero directory обычно означает отсутствие таблицы, а не повреждение.
RVA=0, Size!=0 или наоборот	Malformed exception directory, read result MALFORMED.	Directory не может одновременно быть отсутствующей и иметь размер/адрес.
RVA не маппится в raw bytes	Malformed, diagnostic message, partial report.	Static parser не должен читать virtual-only или несуществующую область.
Directory выходит за file range	Malformed, затем finding pe_exception_directory_oob.	Таблица заявлена так, что ее невозможно безопасно прочитать из файла.
Size < 12	Malformed.	Одна запись RUNTIME_FUNCTION занимает 12 bytes.
Size % 12 != 0	Malformed.	Таблица должна быть массивом целых 12-byte записей.
Все checks пройдены	Читается n = Size / 12 записей.	Размер directory превращается в количество функций только после проверок.

Если чтение directory вернуло MALFORMED, main.c не уничтожает весь анализ. Он печатает ошибку, выставляет img.exception_directory_oob = true, продолжает partial structural report и ставит exit_code = 1. Это важно: проект сохраняет диагностическую ценность даже на поврежденных файлах. Код: src/main.c:306-324; finding затем создается в src/heuristics.c:825-831.

94. Блок 5 - RUNTIME_FUNCTION table: карта функций

Когда Exception Directory успешно прочитана, она превращается в массив RuntimeFunction { begin_rva, end_rva, unwind_rva }. Эта таблица является картой функций для Windows x64 unwind machinery. По ней система понимает, какая функция покрывает текущий RIP и где лежит metadata для выхода из кадра.

Проект проверяет эту карту не только на физическую читаемость, но и на логическую согласованность: EndAddress должен быть больше BeginAddress, диапазоны должны лежать внутри SizeOfImage, UnwindInfoAddress не должен быть нулевым или выходить за image, таблица должна быть отсортирована, ranges не должны пересекаться, BeginAddress не должен дублироваться. Эти checks рождают findings invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin.

Нюанс: некоторые проверки делаются в исходном порядке, а overlap/duplicate надежнее проверять после сортировки. Поэтому проект строит RfIdx, сортирует и затем проверяет пересечения. Это не просто 'сравнить соседние строки из файла', потому что сам файл может быть несортированным.

Finding	Что означает	Точная причина важности
invalid_ranges	EndAddress <= BeginAddress.	Функция с нулевой или отрицательной длиной не задает корректный диапазон RIP.
rf_oob	Begin/End выходят за SizeOfImage.	Function range ссылается за пределы виртуального образа.
unwind_rva_oob	UnwindInfoAddress равен 0 или вне image.	Невозможно найти metadata для virtual unwind.
rf_unsorted	BeginAddress нарушает возрастающий порядок.	Windows lookup и бинарный поиск ожидают корректную таблицу.
rf_overlaps	Диапазоны функций пересекаются.	Один RIP может неоднозначно принадлежать нескольким функциям.
duplicate_begin	Несколько записей начинаются с одного RVA.	Это создает неоднозначную карту функций.

95. Блок 6 - UNWIND_INFO: декодирование переменной ABI-структуры

UNWIND_INFO - это не структура фиксированного размера. В ней есть header, массив unwind codes переменной длины, выравнивание, а затем tail, смысл которого зависит от flags. Если стоит UNW_FLAG_CHAININFO, tail содержит chained RUNTIME_FUNCTION. Если стоит UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, tail содержит handler RVA и затем language-specific data.

Проект поэтому не может просто прочитать sizeof(UNWIND_INFO). Он вычисляет количество slots для каждого opcode, проверяет operands, границы, order, ABI restrictions, chained records, handler target и depth/loop guards. Так появляются unwind_parse_fail и unwind_anomalies, а также более точные internal anomaly flags:

UA_VERSION_UNKNOWN, UA_TRUNCATED, UA_BAD_OPCODE, UA_BAD_OPERAND, UA_CHAIN_LOOP, UA_HANDLER_OOB, UA_ABI_* и другие. Таксономия находится в include/unwind.h:51-69.

Тонкость	Как проект ее учитывает
Разные opcodes занимают разное число slots	slots_for_code в src/unwind.c:24-53 определяет layout чтения.
Header задает дальнейший layout	Header parsing и базовые checks находятся в src/unwind.c:165-237.
Operands зависят от opcode/opinfo	Operand decoder находится в src/unwind.c:254-516.
Tail зависит от flags	Handler/chain tail читается в src/unwind.c:520-550.
Chained unwind может образовать цикл	Follow-chain защищен depth/loop checks в src/unwind.c:563-604.
Parse error должен быть объяснимым	parse_error хранит причину отказа; src/unwind.c:143-163.

96. Блок 7 - размещение metadata и права секций

Проект не заявляет, что он является полным memory-protection auditor для всего PE. Он делает более узкую и правильную для темы ВКР проверку: соответствует ли размещение EH/unwind metadata и handler targets их ожидаемой роли. Это важно, потому что .pdata/.xdata/.rdata обычно являются read-only metadata, а handler code должен лежать в исполняемой области.

Для UnwindInfoRVA проект смотрит, в какой секции лежит metadata. Если секция отсутствует, имеет необычное имя, executable или writable characteristics, это усиливает подозрение через findings unwind_unusual_section, unwind_in_exec_section, unwind_writable_section. Для handler RVA проект проверяет, что target лежит в executable секции; handler в non-executable/unknown секции дает handler_nonexec_section, handler в writable секции дает handler_writable_section. Код размещения находится в src/heuristics.c:690-724, findings - в src/heuristics.c:964-1002.

Точная защитная формулировка: проект проверяет не 'все права всех секций вообще', а права и роль секций применительно к EH/unwind данным: где лежит UNWIND_INFO, где лежит handler и не возникает ли опасная комбинация writable/executable/unusual placement.

97. Блок 8 - Static evidence: признаки, веса, confidence и корреляции

Static analysis собирает не один признак, а набор evidence families. Сначала он переносит структурные counters из PeImage, затем проходит по runtime functions, считает invalid/OOB/unwind failures, собирает hash maps для reuse/dominance, сортирует ranges для overlaps, вычисляет ratios и добавляет findings. Каждому finding присваивается code, family, severity, affected_count и representative location.

Веса не являются математической константой природы. Их задает разработчик как engineering calibration: сильные нарушения инвариантов получают больше баллов, слабые или контекстные признаки - меньше. Например, выход за SizeOfImage сильнее, чем умеренно необычное имя секции; массовое reuse сильнее одиночного случая; комбинация placement + integrity сильнее каждого признака отдельно. В проекте это реализовано как score constants и thresholds в src/heuristics.c:31-84, а финальный verdict/confidence строится в src/heuristics.c:514-588.

Часть evidence model	Почему нужна
score	Компактная мера суммарной подозрительности, удобная для verdict.
family	Показывает слой проблемы: PE table, unwind integrity, metadata placement, reuse, handler dominance.
severity	Отделяет серьезность типа нарушения от количества повторений.
affected_count	Показывает масштаб: один случай или системный паттерн.
representative_rva	Дает точку для ручной проверки в дизассемблере.
confidence	Отдельно показывает уверенность вывода, а не просто score.
mitigations	Ослабляют вывод, если признак одиночный, выборка мала или сигнал placement-only.
combo findings	Фиксируют корреляцию независимых признаков: placement + integrity, reuse + dominance и т.д.

98. Блок 9 - Dynamic validation: что именно проверяется при запуске

Dynamic block не является антивирусной песочницей и не обещает полное поведенческое покрытие. Его задача уже: проверить runtime-события, связанные с exception/unwind темой. Он запускает target под Windows Debug API, ставит breakpoints на интересующие API и handler targets, обрабатывает debug events, собирает bounded trace и фиксирует incomplete/timeout state.

Именно поэтому dynamic evidence в отчете должно читаться как дополнительная проверка, а не как замена статике. Если dynamic session incomplete, timeout или overflow, report layer не должен делать вид, что runtime evidence полное. Код debugger loop находится в `src/dynamic_win.c:752-1145`, breakpoints - `src/dynamic_win.c:165-238`, event model - `include/dynamic.h:26-86`, final completeness - `src/dynamic_win.c:1143-1202`.

Runtime signal	Что показывает
`Handler breakpoint hit`	Статически найденный handler реально достигнут во время исполнения.
`VEH/VCH/UEF registration/hit`	Процесс использует user-mode exception handling APIs.
`RtlAddFunctionTable/RtlInstallFunctionTableCallback`	Runtime function table может добавляться динамически.
`First/second chance exception`	Происходит exception flow, важный для проверки EH behavior.
`Trace after handler hit`	Можно увидеть короткую цепочку RIP после события, не полную трассировку всего процесса.
`Timeout/incomplete`	Динамический вывод ограничен и должен ослабляться в отчете.

99. Блок 10 - Report layer: почему отчет является частью системы

Report layer нужен не для красоты. Он превращает внутренние структуры в доказуемый результат: сохраняет версию schema, build info, input hash, список секций с правами, findings с location/count/severity, static/dynamic correlation, assessment summary и manual validation steps. Это делает работу воспроизводимой: другой человек может открыть отчет и понять, откуда взялся verdict.

Проект учитывает технические риски генерации отчетов: JSON strings экранируются, HTML strings экранируются, CSP ограничивает выполнение внешнего кода, запись файлов делается атомарно. Ключевые места: JSON escaping `src/report.c:255-301`, HTML escaping `src/report.c:933-964`, CSP `src/report.c:1194`, report validation `src/report.c:556-620`, atomic write `src/util.c:190`.

100. Дополнительные граничные случаи, которые важно уметь объяснить

Ситуация	Как трактовать в проекте
`PE32 x86 вместо PE32+ x64`	Не поддерживается намеренно: тема проекта завязана на Windows x64 RUNTIME_FUNCTION/UNWIND_INFO; parser откажет по magic/machine.
`Exception Directory полностью отсутствует`	Не crash. `RVA=0, Size=0` -> count 0; дальше либо `no_runtime_functions`, либо neutral note, зависит от executable sections.
`Directory частично нулевая`	Malformed: один из пары RVA/Size нулевой, другой нет; это противоречивая data directory.
`Directory указывает в headers`	RVA внутри `SizeOfHeaders` может маппиться на file offset; parser это учитывает.
`RVA попадает в virtual zero-fill tail`	Virtual address может существовать в памяти, но bytes отсутствуют в файле; static read запрещается.
`Section raw range выходит за файл`	Finding `pe_section_raw_oob`; нельзя доверять чтению bytes из такой секции.
`Section virtual range выходит за SizeOfImage`	Finding `pe_section_virtual_oob`; виртуальная карта image противоречива.
`Raw/virtual ranges пересекаются`	Finding raw/virtual overlap; отображение становится неоднозначным.
`UNWIND_INFO в writable секции`	Finding `unwind_writable_section`; metadata может быть изменяемой, что необычно для read-only служебных таблиц.
`Handler в non-executable секции`	Finding `handler_nonexec_section`; handler target должен быть кодом.
`Много функций с одним UnwindInfo`	Reuse может быть нормальным в малых масштабах, но массовый reuse похож на dispatcher/obfuscation pattern.
`Dynamic ничего не поймал`	Это не автоматически 'чисто': событие могло не исполниться, session могла быть incomplete, нужен static evidence и manual validation.

101. Короткая защитная речь по архитектуре

Мой проект построен блочно. Первый блок безопасно читает недоверенный binary input. Второй блок строит PE32+ AMD64 модель и проверяет согласованность headers, sections, data directories и image boundaries. Третий блок отдельно разбирает Exception Directory и корректно различает отсутствие таблицы, противоречивую таблицу и читаемую таблицу RUNTIME_FUNCTION. Четвертый блок декодирует Windows x64 UNWIND_INFO как переменную ABI-структуру с flags, slots, opcodes, chain и handlers. Пятый блок строит static evidence model из 27 признаков, учитывая severity, prevalence, correlations, score, confidence и mitigations. Шестой блок добавляет optional runtime validation через Windows Debug API. Седьмой блок формирует воспроизводимые отчеты с metadata, escaping, validation и manual steps. Поэтому это не набор случайных проверок, а законченный анализатор EH/unwind слоя PE-файла.

ПОШАГОВАЯ РАБОТА БЛОКОВ ПРОЕКТА

Этот раздел добавлен как развернутая инженерная расшифровка работы проекта. Он не смешивает parser, static analysis, dynamic validation и report layer. Каждый блок описан как последовательность шагов: что происходит, как это реализовано в коде, зачем это нужно для анализа PE/EH/unwind и что происходит при ошибке или граничном случае.

Главная мысль для защиты: проект сложен не потому, что каждая отдельная проверка выглядит длинной, а потому что результат зависит от правильной последовательности шагов. Нельзя декодировать UNWIND_INFO, пока не доказано, что RVA корректно переводится в file offset; нельзя делать verdict, пока findings не разделены по семействам, severity, confidence и mitigating conditions; нельзя усиливать вывод динамикой, если debug session incomplete.

102. Блок main/orchestration - как программа проводит файл через pipeline

Цель блока: управлять всей цепочкой анализа, не смешивая чтение файла, parsing, static evidence, dynamic evidence и report writing. Вход блока: аргументы командной строки, путь к PE-файлу, optional flags `--heuristics`, `--dynamic`, `--json`, `--html`, `--csv`. Выход блока: exit code, console output, optional отчеты и очищенная память.

Шаг 1. Разбор командной строки

Что происходит: Программа сначала понимает, что именно пользователь попросил: просто разобрать PE, вывести dump, показать unwind operations, включить heuristics, включить dynamic mode, записать JSON/HTML/CSV.

Как работает: Функция `parse_args` заполняет структуру `Args`: выставляет значения по умолчанию, затем проходит по `argv` и распознает флаги. Для числовых параметров используются `parse_long_range` и `parse_u32_range`, чтобы пользователь не передал отрицательное или слишком большое значение.

Зачем нужно: Это отделяет управление режимами от логики анализа. Без этого parser начал бы зависеть от деталей консоли, а dynamic block - от ручного разбора строк.

Нюанс/ошибка: Если флаг неизвестен или число вне диапазона, программа прекращает работу с кодом ошибки `2`, а не запускает анализ в неопределенном режиме.

Код: `src/main.c:176-242`

Шаг 2. Загрузка входного PE в память

Что происходит: После разбора аргументов файл открывается в binary mode и полностью читается в `Buf`.

Как работает: `buf_init_from_file` узнает размер файла через `fseek/ftell`, выделяет память, читает ровно `sz` bytes и возвращает `Buf { data, size }` вместе с owned pointer для дальнейшего освобождения.

Зачем нужно: PE-parser работает с произвольными смещениями, поэтому ему нужен быстрый доступ к bytes. Но эти bytes остаются недоверенными: наличие буфера не означает, что любая структура внутри корректна.

Нюанс/ошибка: Если файл не открылся, размер не получен, память не выделилась или `fread` прочитал не весь файл, анализ останавливается. Это правильнее, чем строить вывод на неполном input.

Код: `src/main.c:283-288, src/util.c:13-35`

Шаг 3. Запуск PE parser-а

Что происходит: Загруженный `Buf` передается в `pe_parse`, который должен построить `PeImage` или объяснить, почему это невозможно.

Как работает: `main` заранее создает `StrBuf err`, чтобы parser мог вернуть понятную причину: не MZ, неверная PE signature, unsupported machine, optional header out of range, плохие alignments и так далее.

Зачем нужно: Оркестратор не должен сам знать детали PE headers. Его задача - вызвать специализированный блок и остановиться, если базовая модель образа не построена.

Нюанс/ошибка: Если `pe_parse` вернул false, программа освобождает буфер и завершает работу. Дальше идти нельзя: без `PeImage` невозможно корректно понимать RVA, sections и directory.

Код: `src/main.c:290-299, src/pe.c:36-255`

Шаг 4. Чтение Exception Directory как function table

Что происходит: После построения `PeImage` программа пытается извлечь массив `RUNTIME_FUNCTION` из Exception Directory.

Как работает: `unwind_read_runtime_functions_ex` получает `exception_rva` и `exception_size`, проверяет сценарии отсутствия/повреждения/чтения и возвращает `rf` плюс `rf_count`.

Зачем нужно: Это отдельный блок, потому что Exception Directory - уже не общий PE header, а предметная x64 EH/unwind структура.

Нюанс/ошибка: Если directory отсутствует (`RVA=0` и `Size=0`), это OK и `rf_count=0`. Если directory противоречива или не читается, `main` ставит `img.exception_directory_oob = true`, продолжает partial structural report и выставляет `exit_code=1`.

Код: `src/main.c:301-324, src/unwind.c:611-671`

Шаг 5. Выделение массивов под результаты unwind parsing

Что происходит: Для каждой записи `RUNTIME_FUNCTION` нужно хранить результат декодирования `UNWIND_INFO` и флаг успешности.

Как работает: `main` выделяет `UnwindSummary* us` и `bool* us_ok`. Даже если `rf_count == 0`, выделяется минимум 1 элемент, чтобы `calloc(0)` не дал неоднозначное поведение на разных CRT.

Зачем нужно: Так static layer дальше получает согласованные массивы: `rf[i]` описывает функцию, `us[i]` описывает ее unwind metadata, `us_ok[i]` говорит, можно ли этому summary доверять.

Нюанс/ошибка: Если память не выделилась, программа аккуратно освобождает уже созданные структуры и завершает работу. Это важно для C-проекта: отказ памяти не должен превращаться в use-after-free или leak cascade.

Код: `src/main.c:328-346`

Шаг 6. Декодирование UNWIND_INFO для каждой функции

Что происходит: Программа проходит по всем `RUNTIME_FUNCTION` и пытается разобрать `rf[i].unwind_rva`.

Как работает: На каждую запись создается локальный buffer ошибки `perr`, вызывается `unwind_parse`, результат кладется в `us[i]`, а success/fail - в `us_ok[i]`.

Зачем нужно: Так проект отделяет 'таблица функций прочиталась' от 'metadata конкретной функции корректна'. В реальном PE часть записей может быть нормальной, а часть битой.

Нюанс/ошибка: Если один `UNWIND_INFO` не разобрался, весь анализ не падает. Ошибка превращается в `us_ok[i]=false`, а static layer позже добавит `unwind_parse_fail`.

Код: `src/main.c:351-361, src/unwind.c:165-608`

Шаг 7. Запуск static heuristics

Что происходит: Если включен `--heuristics`, программа строит `HeuristicReport` по parsed PE, runtime functions и unwind summaries.

Как работает: Функция `heuristics_analyze` считает counters, ratios, findings, score, confidence, verdict и mitigation notes. После успешного анализа `main` печатает brief summary и сохраняет указатель `hr_ptr` для report layer.

Зачем нужно: Static heuristics превращают сырые технические факты в объяснимую evidence model. Без этого пользователь увидел бы только таблицы адресов, но не понял бы, какие нарушения важнее.

Нюанс/ошибка: Если heuristic analysis не смог выделить память или модель противоречива, программа пишет ошибку и продолжает только те outputs, которые еще возможны.

Код: `src/main.c:382-399, src/heuristics.c:590-1028`

Шаг 8. Optional dynamic validation

Что происходит: Если включен `--dynamic`, программа запускает target под Windows Debug API и собирает runtime-события.

Как работает: `dynamic_collect_windows` получает static model (`img`, `rf`, `us`, `us_ok`) и использует ее для установки breakpoints на handler targets, а также на API, связанные с exception handling и dynamic function tables.

Зачем нужно: Dynamic block не заменяет static analysis. Он отвечает на другой вопрос: проявляются ли некоторые EH/unwind-паттерны во время исполнения.

Нюанс/ошибка: Если dynamic session failed или timed out, это фиксируется в `DynamicReport` и `exit_code`, но static result не уничтожается.

Код: `src/main.c:401-425, src/dynamic_win.c:657-1202`

Шаг 9. Подготовка metadata и запись отчетов

Что происходит: Когда analysis завершен, программа собирает timestamp, command line, input hash, build info и пишет JSON/HTML/CSV, если пользователь попросил.

Как работает: Metadata создается только для JSON/HTML, затем report writer проверяет metadata/model validity и экранирует строки. CSV/Ghidra CSV пишутся отдельными функциями.

Зачем нужно: Отчет нужен не как 'красивый вывод', а как воспроизводимое доказательство: версия инструмента, hash input, schema version, flags и findings должны быть сохранены вместе.

Нюанс/ошибка: Если report не записался, программа не скрывает проблему: печатает ошибку и возвращает non-zero exit code.

Код: `src/main.c:450-510, src/report.c:556-704, src/report.c:933-964`

Шаг 10. Освобождение ресурсов

Что происходит: В конце `main` освобождает все owned ресурсы: heuristic details, dynamic details, buffers, arrays, PE sections и исходный файл.

Как работает: Каждый ресурс освобождается функцией своего слоя: `heuristics_free`, `dynamic_report_free`, `pe_free`, `buf_free_owned`, `sb_free`.

Зачем нужно: В C это часть корректности архитектуры. Анализатор бинарников часто гоняют пакетно на многих файлах, поэтому утечки памяти и dangling pointers быстро становятся реальной проблемой.

Нюанс/ошибка: Даже при ошибках в середине pipeline код старается освобождать уже выделенные структуры. Это отличает сдаваемый C-проект от одноразового прототипа.

Код: `src/main.c:511-526`

103. Блок util/buffer safety - как проект безопасно читает bytes

Цель блока: дать всем остальным блокам надежный способ читать недоверенный бинарный файл и собирать диагностические строки. Вход блока: путь к файлу, offset/length чтения, форматированные сообщения. Выход блока: `Buf`, typed read helpers, `StrBuf`, atomic file write.

Шаг 1. Открытие файла строго в binary mode

Что происходит: Файл открывается через `fopen(path, "rb")`, чтобы Windows не преобразовывала переводы строк и не меняла byte stream.

Как работает: В бинарном анализе каждый byte имеет значение. PE header, offsets, signatures и tables должны читаться точно такими, какие лежат на диске.

Зачем нужно: Если открыть файл как text, на Windows возможны преобразования CR/LF и EOF semantics, что разрушило бы offsets.

Нюанс/ошибка: Если `fopen` не сработал, функция возвращает false без частично заполненного `Buf`.

Код: `src/util.c:13-17`

Шаг 2. Определение размера файла

Что происходит: Перед выделением памяти код переходит в конец файла, читает `ftell`, затем возвращается в начало.

Как работает: Это дает parser-у физическую границу: `b->size`. Все дальнейшие проверки OOB опираются именно на нее.

Зачем нужно: Граница файла - базовая защита от чтения мусора за пределами выделенной памяти.

Нюанс/ошибка: Если `ftell` вернул отрицательное значение или `fseek` failed, файл не анализируется.

Код: `src/util.c:19-23`

Шаг 3. Выделение owned buffer

Что происходит: Код выделяет память под весь файл и сохраняет pointer в `owned_mem`, чтобы потом его можно было освободить.

Как работает: Даже для пустого файла выделяется минимум 1 byte (`alloc_size = sz > 0 ? sz : 1u`), чтобы поведение `malloc(0)` не зависело от реализации.

Зачем нужно: Parser получает стабильный массив bytes. Он не держит открытый файл и не зависит от внешнего состояния файловой системы.

Нюанс/ошибка: Если `malloc` или `fread` failed, память освобождается и наружу не выходит поврежденный `Buf`.

Код: `src/util.c:24-35`

Шаг 4. Checked чтение произвольного диапазона

Что происходит: Любой блок, который хочет прочитать bytes по offset, вызывает `buf_read`.

Как работает: Сначала проверяется `off > b->size`, затем `len > b->size - off`. Второе условие специально написано так, чтобы избежать integer overflow при `off + len`.

Зачем нужно: Это фундамент безопасности parser-а. Злонамеренный PE может указать offset около `SIZE_MAX`; прямое `off + len` могло бы переполниться и стать маленьким числом.

Нюанс/ошибка: Если диапазон не помещается, функция возвращает false, а вызывающий блок превращает это в parse error или finding.

Код: `src/util.c:42-48`

Шаг 5. Typed reads little-endian

Что происходит: PE хранит числовые поля в little-endian. Проект читает `u8/u16/u32/u64` через отдельные helpers.

Как работает: Например, `buf_read_u32` читает 4 bytes в temporary array и вызывает `le32`. Это отделяет byte order от логики parser-а.

Зачем нужно: Так PE parser не повторяет ручное сложение bytes на каждом поле и не рискует случайно прочитать значение в host-endian неверно.

Нюанс/ошибка: Если нужных bytes нет, typed read возвращает false, и структура не считается прочитанной.

Код: `src/util.c:50-70`

Шаг 6. StrBuf как безопасный accumulation buffer

Что происходит: Ошибки и детали отчета собираются в growable string buffer.

Как работает: `sb_reserve` расширяет память с проверкой overflow, `sb_appendn` проверяет `SIZE_MAX`, `sb_appendf` сначала вычисляет нужный размер через `vsnprintf`.

Зачем нужно: Это важно, потому что analyzer должен сохранять причины ошибок, findings и diagnostic details, но не должен ломаться на длинном выводе.

Нюанс/ошибка: Если память закончилась или форматирование failed, `sb->failed` помечается, и report/model validity может это увидеть.

Код: `src/util.c:72-188`

Шаг 7. Atomic write для отчетов

Что происходит: Когда отчет записывается в файл, проект сначала пишет temporary file, затем заменяет destination.

Как работает: На Windows используется `MoveFileExA` с `MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH`; на POSIX - `rename`.

Зачем нужно: Это защищает от ситуации, когда программа упала или диск дал ошибку посередине записи и оставил битый итоговый report.

Нюанс/ошибка: Если запись или rename failed, temporary file удаляется, а вызывающий report writer получает false.

Код: `src/util.c:190-230`

104. Блок PE parser - как bytes превращаются в модель PE32+ AMD64

Цель блока: построить `PeImage`, не доверяя ни одному полю файла до проверки его границ и согласованности. Вход блока: `Buf` с bytes файла. Выход блока: `PeImage` с headers, sections, exception directory и counters структурных нарушений.

Шаг 1. Проверка DOS header и `e_lfanew`

Что происходит: Parser проверяет MZ signature и читает `e_lfanew` - offset до PE header.

Как работает: `read_dos_lfanew` читает `u16` по offset 0 и `u32` по offset `0x3c`. После этого `pe_parse` проверяет, что по `lfanew` физически помещаются PE signature и COFF header.

Зачем нужно: Без этого любой файл можно было бы ошибочно интерпретировать как PE. `e_lfanew` - первое недоверенное смещение, поэтому его сразу ограничивают размером файла.

Нюанс/ошибка: Если MZ нет или `lfanew + 4 + 20 > file_size`, parser возвращает ошибку и не идет дальше.

Код: `src/pe.c:11-18, src/pe.c:40-48`

Шаг 2. Проверка PE signature

Что происходит: По адресу `e_lfanew` должен лежать magic `PE\0\0`.

Как работает: Код читает `u32` и сравнивает с `PE_NT_MAGIC`. Проверка идет после bounds check, поэтому чтение безопасно.

Зачем нужно: MZ alone недостаточно: многие файлы могут иметь MZ-stub, но не быть корректным PE.

Нюанс/ошибка: Если signature неверная, все дальнейшие offsets не имеют смысла.

Код: `src/pe.c:50-54`

Шаг 3. Чтение COFF header

Что происходит: Parser читает `Machine`, `NumberOfSections` и `SizeOfOptionalHeader`.

Как работает: Эти поля определяют архитектуру, количество section headers и размер optional header. Они нужны, чтобы понять, где дальше находятся sections и data directories.

Зачем нужно: COFF header - переход от generic PE identity к конкретной архитектуре и layout.

Нюанс/ошибка: Если хотя бы одно поле не читается, parser останавливается: значит header обрезан или `e_lfanew` указывает не туда.

Код: `src/pe.c:56-65`

Шаг 4. Фильтр архитектуры AMD64

Что происходит: Проект принимает только `IMAGE_FILE_MACHINE_AMD64` и PE32+ magic `0x20b`.

Как работает: Сначала отсекается machine type, затем позже optional header magic. Оба условия должны совпасть.

Зачем нужно: Тема ВКР - x64 EH/unwind. На x86 exception handling устроен иначе; там нет той же обязательной `.pdata`/`UNWIND_INFO` модели.

Нюанс/ошибка: Если комиссия спросит про PE32: текущий проект осознанно ограничен PE32+ AMD64, потому что признаки построены вокруг Windows x64 unwind ABI.

Код: `src/pe.c:67-70, include/pe.h:18-19`

Шаг 5. Проверка OptionalHeader bounds и размера

Что происходит: Parser вычисляет начало OptionalHeader и проверяет, что он целиком помещается в файл и достаточно велик для нужных PE32+ fields.

Как работает: Проверка `opthdr + opt_size > b->size` предотвращает чтение за пределы файла. Проверка `opt_size < 0x70` гарантирует, что можно добраться до DataDirectory.

Зачем нужно: OptionalHeader называется optional исторически, но для executable PE он содержит критичные поля image layout.

Нюанс/ошибка: Если optional header обрезан, parser не пытается угадывать значения.

Код: `src/pe.c:72-80`

Шаг 6. Чтение key image fields

Что происходит: Parser читает `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `SizeOfHeaders`, `NumberOfRvaAndSizes`.

Как работает: Offsets берутся по layout PE32+: например, `ImageBase` по `opthdr + 0x18`, `SizeOfImage` по `opthdr + 0x38`.

Зачем нужно: Эти поля задают координатную систему виртуального образа и правила сопоставления sections.

Нюанс/ошибка: Если fields не читаются, дальнейшие проверки sections и RVA невозможны.

Код: `src/pe.c:92-108`

Шаг 7. Проверка базовых инвариантов image

Что происходит: Parser отбрасывает нулевой `SizeOfImage`, плохой `SizeOfHeaders`, нулевые alignments и недостаточное количество data directories.

Как работает: Здесь устанавливается, что image вообще имеет конечную виртуальную границу, headers лежат в файле, а exception directory entry существует в массиве DataDirectory.

Зачем нужно: Без `SizeOfImage` нельзя определить, вышел ли RVA за образ. Без `NumberOfRvaAndSizes` нельзя безопасно обращаться к Exception Directory.

Нюанс/ошибка: Если `NumberOfRvaAndSizes` меньше `IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1`, это не 'просто нет exception directory', а отсутствие самой записи directory.

Код: `src/pe.c:109-123`

Шаг 8. Чтение Exception DataDirectory entry

Что происходит: Parser вычисляет offset записи `DataDirectory[3]` и читает пару `VirtualAddress/Size`.

Как работает: Сначала проверяется, что `dd_rel + 8 <= opt_size`, затем читаются `exc_rva` и `exc_sz`.

Зачем нужно: На x64 именно эта пара ведет к таблице `RUNTIME_FUNCTION`. Но parser пока только сохраняет координаты, а не читает таблицу.

Нюанс/ошибка: Если запись directory лежит за OptionalHeader, parser прекращает работу: layout заголовков противоречив.

Код: `src/pe.c:126-137`

Шаг 9. Проверка и чтение section headers

Что происходит: Parser вычисляет начало section table и проверяет, что `NumberOfSections * 40` не переполняет размер и помещается в файл.

Как работает: Каждая секция читается в `PeSection`: имя, `VirtualSize`, `VirtualAddress`, `SizeOfRawData`, `PointerToRawData`, `Characteristics`.

Зачем нужно: Sections связывают virtual image layout с raw bytes на диске. Без них нельзя переводить RVA в file offset и нельзя оценивать права секций.

Нюанс/ошибка: Если section table выходит за файл, parser не строит частичную модель: дальнейшая интерпретация была бы опасной.

Код: `src/pe.c:139-172`

Шаг 10. Проверка alignment-инвариантов

Что происходит: Parser считает `misaligned`, если нарушены правила `FileAlignment`, `SectionAlignment`, `SizeOfImage`, `SizeOfHeaders`, `ImageBase` или порядок sections.

Как работает: В коде учтен compact alignment случай: если `SectionAlignment < 0x1000`, то `FileAlignment` должен совпадать с ним. Проверяются power-of-two и допустимые диапазоны.

Зачем нужно: Alignment показывает, что layout создан нормальным linker/loader-compatible способом. Нарушения могут быть ошибкой, packed layout или deliberate malformed PE.

Нюанс/ошибка: Это не всегда фатальная ошибка parser-а, поэтому счетчик сохраняется как будущий finding `pe_section_misaligned`.

Код: `src/pe.c:180-197, src/pe.c:205-213`

Шаг 11. Проверка raw и virtual границ секций

Что происходит: Для каждой секции parser проверяет, не выходит ли raw range за file size и virtual range за `SizeOfImage`, а также не пересекаются ли ranges секций.

Как работает: Raw end считается как `PointerToRawData + SizeOfRawData`. Virtual span берется как `max(VirtualSize, SizeOfRawData)`, потому что разные toolchains могут по-разному задавать размеры.

Зачем нужно: Raw OOB ломает безопасное чтение файла; virtual OOB ломает карту образа; overlap делает mapping неоднозначным.

Нюанс/ошибка: Эти нарушения не всегда прекращают parse, потому что analyzer должен уметь report-ить malformed PE. Они сохраняются в `PeImage` как counters.

Код: `src/pe.c:198-227`

Шаг 12. Проверка Exception Directory на выход за image

Что происходит: Если `exc_rva` или `exc_sz` ненулевые, parser проверяет, что оба ненулевые и `exc_rva + exc_sz <= SizeOfImage`.

Как работает: Здесь проверяется виртуальная допустимость directory. Физическая читаемость через file offset будет проверена позже в `unwind_read_runtime_functions_ex`.

Зачем нужно: Это разделение важно: `SizeOfImage` отвечает за virtual boundary, а `file_size/raw_size` - за физическое чтение bytes.

Нюанс/ошибка: Если directory частично нулевая или выходит за image, выставляется `exception_directory_oob`, который позже станет finding.

Код: `src/pe.c:230-255`

Шаг 13. RVA lookup и RVA-to-file mapping

Что происходит: `pe_find_section_for_rva` ищет секцию, покрывающую RVA; `pe_rva_to_fileoff` переводит RVA в file offset только если bytes реально есть в файле.

Как работает: Если RVA внутри секции, считается `delta = rva - section.VirtualAddress`; если `delta >= raw_size`, mapping запрещается, потому что это virtual zero-fill tail. Если RVA внутри headers (`rva < SizeOfHeaders`), offset равен RVA.

Зачем нужно: Это ключевая тонкость проекта. В памяти loader может создать нули, но static parser не имеет права читать bytes, которых нет на диске.

Нюанс/ошибка: Если RVA не маппится, вызывающий блок получает false и фиксирует malformed/unreadable состояние.

Код: `src/pe.c:265-300`

105. Блок Exception Directory и RUNTIME_FUNCTION - как строится карта функций

Цель блока: превратить Exception Directory в массив `RuntimeFunction` и корректно отличить отсутствие таблицы от поврежденной таблицы. Вход блока: `PeImage.exception_rva`, `PeImage.exception_size`, исходный `Buf`. Выход блока: `RuntimeFunction* rf`, `rf_count`, либо partial structural finding.

Шаг 1. Инициализация результата чтения

Что происходит: Функция сначала проверяет указатели и сбрасывает `*out_rf = NULL`, `*out_count = 0`.

Как работает: Это гарантирует, что вызывающий код не увидит мусорный pointer при ошибке.

Зачем нужно: В C важно заранее привести outputs к безопасному состоянию, особенно если дальше возможны ранние return.

Нюанс/ошибка: Если аргументы функции некорректны, возвращается `RUNTIME_FUNCTION_READ_MALFORMED`.

Код: `src/unwind.c:617-622`

Шаг 2. Сценарий полного отсутствия directory

Что происходит: Если `exception_rva == 0` и `exception_size == 0`, таблица считается отсутствующей, а не поврежденной.

Как работает: Reader возвращает OK, count остается 0. Позже heuristics смотрит, есть ли executable sections: если есть, добавляет `no_runtime_functions`; если нет, добавляет нейтральную note.

Зачем нужно: Это корректно для PE: zero/zero directory обычно означает 'таблицы нет'. Но для x64 executable image это требует контекста, потому что unwind metadata часто нужна для non-leaf функций.

Нюанс/ошибка: Важно говорить на защите: отсутствие directory не ломает проект; оно обрабатывается как отдельный сценарий.

Код: `src/unwind.c:623-625, src/heuristics.c:833-842`

Шаг 3. Сценарий частично нулевой directory

Что происходит: Если `RVA` нулевой, а `Size` ненулевой, или наоборот, directory противоречива.

Как работает: Reader пишет в `err`: 'RVA and size must both be zero or non-zero' и возвращает malformed.

Зачем нужно: Одна половина пары не имеет смысла без другой: размер без адреса не указывает на bytes, адрес без размера не задает область.

Нюанс/ошибка: В `main` это затем превращается в partial structural report и finding `pe_exception_directory_oob`.

Код: `src/unwind.c:626-629, src/pe.c:230-235`

Шаг 4. Перевод RVA directory в file offset

Что происходит: Если directory заявлена как присутствующая, ее RVA нужно перевести в физический offset файла.

Как работает: Reader вызывает `pe_rva_to_fileoff`. Эта функция учитывает sections, headers и запрещает чтение virtual zero-fill tail.

Зачем нужно: Так проект не делает ошибку 'RVA равен offset'. Он доказывает, что bytes directory реально лежат на диске.

Нюанс/ошибка: Если mapping невозможен, directory считается malformed/unreadable.

Код: `src/unwind.c:630-634, src/pe.c:277-300`

Шаг 5. Проверка физического размера directory

Что происходит: Даже если начало directory маппится, весь диапазон `off + exception_size` должен помещаться в `b->size`.

Как работает: Это отдельная проверка, потому что первый byte может быть валиден, а конец таблицы может выходить за файл.

Зачем нужно: Чтение массива записей без этой проверки дало бы OOB read при последней записи.

Нюанс/ошибка: Если диапазон выходит за файл, reader возвращает malformed, а `main` продолжает partial report.

Код: `src/unwind.c:635-638`

Шаг 6. Проверка минимального размера и кратности 12

Что происходит: Одна `RUNTIME_FUNCTION` запись занимает 12 bytes: BeginAddress, EndAddress, UnwindInfoAddress.

Как работает: Если `Size < 12`, таблица не содержит ни одной полной записи. Если `Size % 12 != 0`, область не является чистым массивом записей.

Зачем нужно: Эта проверка защищает от неверной архитектуры, повреждения, truncation или deliberate obfuscation.

Нюанс/ошибка: Если `Size=0`, этот путь не достигается: zero/zero был обработан раньше как absence.

Код: `src/unwind.c:639-647`

Шаг 7. Выделение массива `RuntimeFunction`

Что происходит: После всех проверок количество записей вычисляется как `n = exception_size / 12`, и выделяется массив `calloc(n, sizeof(RuntimeFunction))`.

Как работает: Память выделяется только когда известно, что `n` получено из проверенного размера directory.

Зачем нужно: Так reader не выделяет массив по недоверенному числу из файла напрямую, а выводит количество из уже проверенного размера.

Нюанс/ошибка: Если памяти нет, возвращается `RUNTIME_FUNCTION_READ_OOM`, и `main` завершает анализ как hard error.

Код: `src/unwind.c:647-652`

Шаг 8. Чтение каждой записи таблицы

Что происходит: Reader проходит по `i=0..n-1` и читает три `u32`: begin RVA, end RVA, unwind RVA.

Как работает: Offset записи считается как `off + i * 12`; поля читаются через checked `buf_read_u32`.

Зачем нужно: На этом этапе reader еще не решает, хороши ли ranges. Он только извлекает таблицу в структурированную C-модель.

Нюанс/ошибка: Если чтение одной записи не удалось, массив освобождается и возвращается malformed.

Код: `src/unwind.c:654-671`

106. Блок UNWIND_INFO parser - как декодируется переменная ABI-структура

Цель блока: разобрать Windows x64 `UNWIND_INFO`, сохранив summary действий пролога, handler/chain metadata и anomaly flags. Вход блока: `RuntimeFunction.unwind_rva`, `Buf`, `PeImage`, parser options. Выход блока: `UnwindSummary`, `us_ok`, optional text dump of operations.

Шаг 1. Подготовка options и output

Что происходит: Parser очищает `UnwindSummary`, настраивает `max_chain_depth`, `follow_chain`, `strict_abi`.

Как работает: Если options не переданы, используются default: follow chain включен, max depth 8, strict ABI выключен. Max depth принудительно ограничен 16.

Зачем нужно: UNWIND_INFO может иметь chained records. Без ограничения глубины malformed chain мог бы вызвать рекурсивный обход без конца.

Нюанс/ошибка: Если входные pointers неверные, `unwind_parse` сразу возвращает false.

Код: `src/unwind.c:165-185`

Шаг 2. Проверка alignment и чтение header

Что происходит: Parser проверяет, выровнен ли `unwind_rva` на 4 bytes, и пытается прочитать первые 4 bytes header.

Как работает: Header читается через `pe_rva_to_fileoff`: сначала RVA превращается в file offset, затем проверяется, что 4 bytes помещаются в файл.

Зачем нужно: Это снова показывает разделение: virtual address должен быть читаем из raw bytes. Нельзя просто взять RVA как offset.

Нюанс/ошибка: Если header не читается, выставляется `UA_TRUNCATED`, parse failure сохраняет причину.

Код: `src/unwind.c:187-195, src/unwind.c:56-72`

Шаг 3. Разбор полей header

Что происходит: Из первого byte берется `version` и `flags`, из следующих - `prolog_size`, `count_codes`, `frame_reg`, `frame_off`.

Как работает: Это bit-level decoding: low 3 bits - version, high 5 bits - flags; y frame byte low nibble - register, high nibble - scaled offset.

Зачем нужно: Эти поля определяют, как читать остальную структуру: сколько unwind code slots есть и какой tail ожидается.

Нюанс/ошибка: Если на этом этапе ошибиться в битах, весь последующий parser будет читать wrong layout.

Код: `src/unwind.c:197-210`

Шаг 4. Проверка версии, flags и frame register

Что происходит: Parser принимает version 1 или 2, проверяет неизвестные flags, конфликт `CHAININFO` с handler flags и согласованность frame register.

Как работает: Например, если `frame_reg == 0`, `frame_off` должен быть 0. В strict ABI mode дополнительно проверяется класс nonvolatile register.

Зачем нужно: Эти checks ловят не только truncation, но и семантически странные комбинации, которые могут указывать на битую или вручную собранную metadata.

Нюанс/ошибка: Не все странности сразу fatal. Часть помечается anomaly flags, чтобы static layer оценил их как findings.

Код: `src/unwind.c:211-229`

Шаг 5. Вычисление размера массива unwind codes

Что происходит: `CountOfCodes` задает количество 2-byte slots, но на диске массив выравнивается до четного количества slots.

Как работает: Код считает `slots_even = (count + 1) & ~1`, `codes_off = uoff + 4`, `codes_bytes = slots_even * 2`, затем проверяет, что область помещается в файл.

Зачем нужно: Это важный ABI nuance: после codes может лежать tail, и его offset зависит от выравнивания.

Нюанс/ошибка: Если codes array выходит за файл, parser возвращает parse fail, а static layer позже увидит `unwind_parse_fail`.

Код: `src/unwind.c:231-239`

Шаг 6. Определение длины каждого opcode

Что происходит: Каждый unwind opcode занимает разное количество slots. Parser обязан знать, сколько slots пропустить.

Как работает: `slots_for_code` возвращает 1/2/3 slots для разных `UWOP_*`. Для version 2 учитываются epilog/spare descriptors.

Зачем нужно: Это причина, почему нельзя читать `UNWIND_INFO` как фиксированную структуру. Layout зависит от содержимого.

Нюанс/ошибка: Если opcode неизвестен или занимает больше slots, чем осталось в `CountOfCodes`, parser ставит anomaly/parse fail.

Код: `src/unwind.c:24-53, src/unwind.c:254-275`

Шаг 7. Проверка order и prolog offsets

Что происходит: Для обычных descriptors parser проверяет, что `code_offset` не выходит за размер prolog и что offsets идут в ожидаемом порядке.

Как работает: В strict ABI mode дополнительно проверяется порядок push/nonvol operations и restrictions для chain info.

Зачем нужно: UNWIND_INFO должен описывать действия пролога функции. Если offset указывает за prolog, metadata не соответствует реальному смыслу.

Нюанс/ошибка: Часть ABI violations не ломает parsing, но снижает доверие к metadata через `unwind_anomalies` или ABI warnings.

Код: `src/unwind.c:276-295`

Шаг 8. Декодирование operands и расчет summary

Что происходит: Для каждого opcode parser читает нужные operands и обновляет summary: fixed stack bytes, pushed nonvolatile registers, masks saved registers/XMM.

Как работает: Например, alloc operations увеличивают оценку fixed stack; save operations отмечают saved register; machframe учитывает frame effect.

Зачем нужно: Это делает output не просто 'valid/invalid', а полезной моделью пролога, которую можно сравнивать с disassembly.

Нюанс/ошибка: Если operand нелогичен или выходит за допустимые ожидания, parser ставит anomaly flags.

Код: `src/unwind.c:297-516`

Шаг 9. Разбор tail: chain или handler

Что происходит: После aligned unwind codes parser смотрит flags. Если есть `CHAININFO`, tail читается как chained `RUNTIME_FUNCTION`; если EHANDLER/UHANDLER - как handler RVA.

Как работает: Для chain проверяются begin/end/unwind bounds. Для handler проверяется, не выходит ли handler RVA за `SizeOfImage`.

Зачем нужно: Tail - место, где unwind metadata связывается с обработчиком исключения. Это сердце темы ВКР.

Нюанс/ошибка: Если tail обрезан, parse fail. Если handler RVA вне image, это anomaly `UA_HANDLER_OOB`.

Код: `src/unwind.c:520-551`

Шаг 10. Follow-chain и защита от loops

Что происходит: Если включен `follow_chain`, parser идет по chained unwind records, но хранит seen RVAs и ограничивает depth.

Как работает: Для каждого chained `unwind_rva` временно вызывает `unwind_parse` с `follow_chain=false`, затем объединяет fixed stack/register summary.

Зачем нужно: Chained unwind используется легально, но malformed chain может быть циклической или слишком глубокой. Проект должен не зависнуть.

Нюанс/ошибка: При loop ставится `UA_CHAIN_LOOP`, при превышении глубины - `UA_CHAIN_DEPTH_EXCEEDED`.

Код: `src/unwind.c:559-604`

Шаг 11. Финальный результат parse

Что происходит: Если структура разобрана, `out->valid = true` и функция возвращает true.

Как работает: `UnwindSummary` содержит version, flags, prolog, frame register, handler/chain flags, handler RVA, stack estimate, masks и anomaly bitmask.

Зачем нужно: Дальше heuristics может отличить 'не разобралось вообще' от 'разобралось, но содержит anomalies'.

Нюанс/ошибка: Это важно для false positives: parse failure и ABI warning - разные уровни evidence.

Код: `src/unwind.c:607-608, include/unwind.h:22-43`

107. Блок static heuristics - как факты превращаются в findings, score и confidence

Цель блока: превратить parsed structures в объяснимую evidence model из признаков, severity, affected counts, score, confidence, mitigations и verdict. Вход блока: `PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`, `us_ok[]`. Выход блока: `HeuristicReport` с findings, counters, verdict и manual review flag.

Шаг 1. Инициализация report и перенос PE counters

Что происходит: Static analysis начинает с пустого `HeuristicReport` и переносит counters, найденные parser-ом: raw/virtual OOB, overlaps, misaligned, exception_directory_oob.

Как работает: Так PE parser и heuristics остаются разделены: parser считает факты, heuristics решает, как оценить их как evidence.

Зачем нужно: Это делает архитектуру чище. Parser не обязан знать веса и verdict, а heuristics не перечитывает headers.

Нюанс/ошибка: Если входные массивы inconsistent (`n > 0`, но `rf/us/us_ok == NULL`), функция возвращает false.

Код: `src/heuristics.c:590-610, include/heuristics.h:42-93`

Шаг 2. Создание hash maps для reuse analysis

Что происходит: Перед проходом по функциям создаются `HMap unwind_counts` и `HMap handler_counts`.

Как работает: Они нужны, чтобы считать, сколько записей используют один и тот же `UnwindInfoRVA` или один и тот же handler RVA.

Зачем нужно: Массовое reuse может быть признаком dispatcher-style metadata или защиты, но его нельзя увидеть одной проверкой range.

Нюанс/ошибка: Если размер слишком большой или memory allocation failed, analysis прекращается с ошибкой, а не строит неполные ratios.

Код: `src/heuristics.c:630-644`

Шаг 3. Первый проход по RUNTIME_FUNCTION

Что происходит: Для каждой записи проверяются `End <= Begin`, выход Begin/End за `SizeOfImage`, нулевой или OOB `UnwindInfoRVA`, а также нарушение сортировки.

Как работает: Counters увеличиваются, а `remember_location` сохраняет representative RVA для будущего finding.

Зачем нужно: Это базовый слой table integrity. Он отвечает на вопрос: можно ли вообще доверять карте функций.

Нюанс/ошибка: Если одна запись плохая, анализ не падает. Счетчик позволит показать масштаб: один случай или массовая проблема.

Код: `src/heuristics.c:646-670`

Шаг 4. Подсчет reuse `UnwindInfoRVA`

Что происходит: Каждый `unwind_rva` добавляется в hash map.

Как работает: Позже из map будет найден dominant value и вычислены reuse/unique ratios.

Зачем нужно: Если многие функции указывают на один и тот же unwind blob, это может быть нормальной оптимизацией в малом масштабе, но в большом масштабе требует объяснения.

Нюанс/ошибка: Проект специально не делает finding сразу: он сначала собирает статистику и контекст.

Код: `src/heuristics.c:671-677`

Шаг 5. Подсчет handler dominance

Что происходит: Если `UNWIND_INFO` parsed successfully и содержит handler, handler RVA добавляется в отдельную hash map.

Как работает: Считается не просто количество handlers, а максимальная доля одного handler target среди всех handlers.

Зачем нужно: Один dominant handler может быть нормальным runtime helper, но вместе с другими признаками похож на central dispatcher.

Нюанс/ошибка: Если handler не parsed или metadata failed, handler dominance для этой записи не считается, чтобы не строить вывод на мусоре.

Код: `src/heuristics.c:679-688`

Шаг 6. Проверка placement и прав секций

Что происходит: Для каждого успешно parsed `UNWIND_INFO` проект смотрит, где лежит unwind metadata и handler target.

Как работает: Unwind metadata считается необычной, если не лежит в `.xdata/.rdata/.pdata`-подобной секции, лежит в executable или writable секции. Handler считается проблемным, если он не в executable section или в writable section.

Зачем нужно: Это не общий аудит всех секций, а предметная проверка роли EH/unwind данных. Metadata обычно read-only data, handler - executable code.

Нюанс/ошибка: Если секция не найдена, это тоже учитывается как unusual/nonexec, потому что RVA не имеет ясной роли в image map.

Код: `src/heuristics.c:690-724`

Шаг 7. Учет parse failures и anomalies

Что происходит: Если `us_ok[i] == false`, увеличивается `unwind_parse_fail`; если parsed, но есть non-ABI anomaly flags, увеличивается `unwind_anomalies`; ABI warnings считаются отдельно.

Как работает: Так проект различает уровни: 'структуру невозможно прочитать' сильнее, чем 'структура читается, но имеет подозрительные поля'.

Зачем нужно: Это снижает риск грубого verdict. Не всякое ABI warning равно attack.

Нюанс/ошибка: Если strict ABI включен, может появиться больше warnings, но они не смешиваются с hard parse failures.

Код: `src/heuristics.c:727-736`

Шаг 8. Сортировка ranges для overlaps/duplicates

Что происходит: После первого прохода создается массив `RfIdx`, сортируется по begin RVA и проверяется на duplicate begins и overlaps.

Как работает: Нельзя надежно искать overlap только в исходном порядке, потому что сама таблица может быть несортированной.

Зачем нужно: Это алгоритмическая часть static analysis: нужно построить вторичное представление данных, чтобы проверить инвариант.

Нюанс/ошибка: Если не хватает памяти на sort array, analysis останавливается, потому что overlap results были бы неполными.

Код: `src/heuristics.c:740-765`

Шаг 9. Превращение PE counters в findings

Что происходит: Raw OOB, virtual OOB, overlaps, misalignment и exception directory OOB получают finding codes и score weights.

Как работает: `add_finding` сохраняет code/family/severity/message, `annotate_finding` добавляет location и affected_count.

Зачем нужно: Так результат становится объяснимым: не просто `score=25`, а `pe_section_raw_oob` с причиной и масштабом.

Нюанс/ошибка: Если finding limit достигнут, details все равно продолжают записываться в human-readable buffer.

Код: `src/heuristics.c:790-831`

Шаг 10. Обработка отсутствующей function table

Что происходит: Если `n == 0` и Exception Directory отсутствует, heuristics проверяет, есть ли executable section.

Как работает: Если executable section есть, добавляется `no_runtime_functions` со score 10. Если executable sections нет, добавляется нейтральная info note.

Зачем нужно: Это аккуратный компромисс: отсутствие `.pdata` может быть легальным, но для x64 executable требует ручной проверки.

Нюанс/ошибка: Так проект не делает ложное 'malware' только по отсутствию directory.

Код: `src/heuristics.c:833-842`

Шаг 11. Пороговые findings по процентам

Что происходит: Для invalid ranges, OOB, unwind parse failures и anomalies проект считает долю от общего числа записей.

Как работает: Порог `1%` отделяет единичную поломку от broad issue; `5%` для unwind anomalies означает более массовую проблему integrity.

Зачем нужно: Это важно, потому что один поврежденный entry в большой таблице и сотни поврежденных entries - разные случаи.

Нюанс/ошибка: Если `n=0`, `safe_ratio` защищает от деления на ноль.

Код: `src/heuristics.c:844-905, src/heuristics.c:80-84`

Шаг 12. Reuse/dominance findings с контекстом

Что происходит: Strong unwind reuse появляется при ratio > 30% и `n > 200`. Moderate reuse требует ratio > 10%, `n > 200` и corroboration другим signal.

Как работает: Handler dominance требует `handler_count > 50` и ratio > 80%, но имеет исключение для benign runtime-like dominant handler без другого suspicious context.

Зачем нужно: Проект специально не наказывает маленькие выборки. Иначе 3 функции с одним handler дали бы 100% dominance, но это статистически слабый вывод.

Нюанс/ошибка: Эти правила удобны для защиты: цифры не магические, а engineering thresholds, ограничивающие false positives.

Код: `src/heuristics.c:908-962`

Шаг 13. Placement findings и combo findings

Что происходит: Unwind in executable/writable/unusual section, handler in nonexec/writable section превращаются в findings; затем проверяются комбинации.

Как работает: Комбо усиливают вывод, когда независимые слои совпали: placement + integrity, reuse + dominance, writable metadata + writable handler.

Зачем нужно: Один слабый признак может иметь benign explanation. Два согласованных признака из разных семей труднее объяснить случайностью.

Нюанс/ошибка: Combo findings добавляют score только при реально наблюдаемой корреляции внутри report.

Код: `src/heuristics.c:964-1023`

Шаг 14. Финализация score, confidence и verdict

Что происходит: После всех findings проект вычисляет suspicion, evidence level, confidence, manual review flag и verdict.

Как работает: Confidence растет от количества evidence families и strong/medium findings, но снижается mitigation rules: single signal, small sample, placement-only, single-family evidence.

Зачем нужно: Это важнейшая защита от дилетантского 'нашел признак - значит плохо'. Проект показывает не только подозрительность, но и уверенность интерпретации.

Нюанс/ошибка: Вердикты привязаны к score: 70+ high, 40+ moderate, 15+ low, иначе no strong static anomalies.

Код: `src/heuristics.c:504-588`

108. Блок dynamic validation - как работает runtime-проверка под debugger-ом

Цель блока: собрать runtime evidence, связанное с exception handling, handler hits и dynamic function tables, не подменяя static analysis. Вход блока: путь к target, static model, timeout, trace_steps. Выход блока: `DynamicReport` с событиями, counters, trace и completeness flags.

Шаг 1. Инициализация dynamic report

Что происходит: Dynamic block очищает report, включает flags `enabled`, сохраняет timeout и ограничивает trace steps до 256.

Как работает: Также инициализируется `details`, где будут human-readable события.

Зачем нужно: Эта структура заранее отделяет 'запускался ли dynamic', 'успешно ли завершился', 'были ли timeouts', 'полна ли instrumentation'.

Нюанс/ошибка: Если details buffer не выделился, dynamic session не запускается.

Код: `src/dynamic_win.c:657-680, include/dynamic.h:41-90`

Шаг 2. Сбор command line для CreateProcess

Что происходит: Target может запускаться с аргументами после `--`. Их нужно корректно собрать в строку командной строки Windows.

Как работает: `build_cmdline` экранирует path и child args, затем `CreateProcessA` запускает debuggee с `DEBUG_ONLY_THIS_PROCESS`.

Зачем нужно: Нельзя просто склеить строки через пробел: пробелы и кавычки в path/args сломают запуск.

Нюанс/ошибка: Если command line не построилась или `CreateProcessA` failed, dynamic returns false с диагностикой.

Код: `src/dynamic_win.c:499-511, src/dynamic_win.c:681-710`

Шаг 3. Почему не используется CREATE_SUSPENDED

Что происходит: В коде явно указано, что process не создается suspended.

Как работает: Проект хочет получить стандартный поток Debug API events сразу после `CreateProcess`. На некоторых конфигурациях suspended debuggee может не выдать ожидаемый `CREATE_PROCESS_DEBUG_EVENT`.

Зачем нужно: Это практический нюанс debugger-программирования: неправильный flag может привести к deadlock ожидания events.

Нюанс/ошибка: Если комиссия спросит, это инженерное решение, основанное на поведении Windows Debug API.

Код: `src/dynamic_win.c:698-707`

Шаг 4. Обновление module list и постановка API breakpoints

Что происходит: При `CREATE_PROCESS_DEBUG_EVENT` и `LOAD_DLL_DEBUG_EVENT` dynamic block обновляет список модулей и пытается поставить breakpoints на интересующие API.

Как работает: API включают `AddVectoredExceptionHandler`, `AddVectoredContinueHandler`, `SetUnhandledExceptionFilter`, `RtlAddFunctionTable`, `RtlInstallFunctionTableCallback`.

Зачем нужно: Эти API связаны с exception handling и runtime function table registration. Их вызовы помогают увидеть динамическое изменение EH/unwind поведения.

Нюанс/ошибка: Если модуль еще не загружен, breakpoint откладывается до следующего refresh; это не fatal.

Код: `src/dynamic_win.c:789-817, src/dynamic_win.c:561-599`

Шаг 5. Постановка breakpoints на static handler targets

Что происходит: Static parser уже нашел handler RVAs в UNWIND_INFO. Dynamic block переводит их в runtime VA через `base_main + handler_rva` и ставит breakpoints.

Как работает: Так static evidence превращается в runtime watchpoints: если execution реально попадет в handler, проект увидит событие.

Зачем нужно: Это связь static и dynamic частей: dynamic не угадывает handlers, а использует результат parser/unwind decoder.

Нюанс/ошибка: Если breakpoint поставить не удалось, это best effort; некоторые handlers могут быть invalid/outside image.

Код: `src/dynamic_win.c:798-808`

Шаг 6. Как software breakpoint ставится технически

Что происходит: Breakpoint - это запись byte `0xCC` в память debuggee по нужному адресу.

Как работает: Код сначала читает original byte через `ReadProcessMemory`, затем пишет `0xCC`. Если page RX и запись не проходит, временно включает `PAGE_EXECUTE_READWRITE` через `VirtualProtectEx`, пишет byte и возвращает старую защиту.

Зачем нужно: Так работает классический INT3 breakpoint. Нужно сохранить original byte, чтобы потом восстановить инструкцию.

Нюанс/ошибка: Если запись не удалась, report увеличивает counters failures и instrumentation может стать incomplete.

Код: `src/dynamic_win.c:165-238`

Шаг 7. Основной event loop

Что происходит: Dynamic block крутит цикл `WaitForDebugEvent` до timeout, process exit или ошибки.

Как работает: Он обрабатывает process create, DLL load/unload, process exit и exceptions. Для обычных exceptions считает first/second chance и передает их target как `DBG_EXCEPTION_NOT_HANDLED`, чтобы сохранить нормальный exception dispatch flow.

Зачем нужно: Это важно: analyzer наблюдает, но не должен ломать естественную обработку исключений target-a.

Нюанс/ошибка: Если timeout достигнут, добавляется event `timeout`, session помечается timed_out и evidence позже ослабляется.

Код: `src/dynamic_win.c:765-845`

Шаг 8. Обработка breakpoint hit

Что происходит: Когда target попадает на INT3, Debug API сообщает `EXCEPTION_BREAKPOINT`.

Как работает: Проект ищет breakpoint по адресу. Если это его breakpoint, он восстанавливает original byte, перематывает RIP назад на адрес original instruction, включает trap flag для single-step и записывает событие.

Зачем нужно: Без перемотки RIP target продолжил бы с адреса после INT3 и пропустил бы оригинальную инструкцию.

Нюанс/ошибка: Если это первый системный breakpoint debugger-a, проект игнорирует его один раз. Если неизвестный breakpoint - передает как not handled.

Код: `src/dynamic_win.c:852-1043`

Шаг 9. Сбор аргументов API по Windows x64 calling convention

Что происходит: При hit на API breakpoint проект читает аргументы из registers: RCX, RDX, R8, R9.

Как работает: Так фиксируются handler pointers для VEH/VCH/UEF и параметры `RtlAddFunctionTable/RtlInstallFunctionTableCallback`.

Зачем нужно: Это уже не статический анализ файла, а runtime observation: видно, что процесс регистрирует обработчики или добавляет function table во время работы.

Нюанс/ошибка: Если handler pointer найден, на него может быть поставлен дополнительный custom breakpoint.

Код: `src/dynamic_win.c:902-983`

Шаг 10. Сбор hits handler-ов и trace

Что происходит: Если сработал static unwind handler breakpoint или custom VEH/VCH/UEF handler, счетчики hits увеличиваются и начинается bounded trace.

Как работает: Trace использует trap flag: после handler hit debuggee выполняет инструкции по одной, а проект записывает RIP до лимита `trace_steps`.

Зачем нужно: Это помогает увидеть, куда идет управление после handler-а, но не превращает проект в полный tracer всего процесса.

Нюанс/ошибка: Trace bounded, чтобы не зависнуть и не создать огромный report.

Код: `src/dynamic_win.c:984-1029, src/dynamic_win.c:617-640, src/dynamic_win.c:1082-1092`

Шаг 11. Re-arm breakpoint после single-step

Что происходит: После исполнения original instruction breakpoint нужно поставить обратно.

Как работает: Для этого используется pending rearm per thread: на single-step event проект снова пишет `0xCC` и продолжает процесс.

Зачем нужно: Без re-arm breakpoint сработал бы только один раз. С re-arm можно наблюдать повторные hits.

Нюанс/ошибка: Если thread завершился до re-arm или re-arm failed, instrumentation_complete становится false.

Код: `src/dynamic_win.c:1045-1107, src/dynamic_win.c:1111-1139`

Шаг 12. Финал dynamic session и cleanup

Что происходит: В конце выставляются `timed_out`, `target_exited`, `ok`, очищаются breakpoints, закрываются handles и освобождаются arrays.

Как работает: `ok` требует got_create, debug_loop_ok и отсутствие timeout. Если target не вышел сам, процесс аккуратно завершается и debug events drain-ятся.

Зачем нужно: Completion flags важны для report layer: incomplete dynamic session не должна усиливать вывод как полноценное runtime подтверждение.

Нюанс/ошибка: Если timed out или не было CREATE_PROCESS_DEBUG_EVENT, error message сохраняется в `err`.

Код: `src/dynamic_win.c:1143-1202`

109. Блок report layer - как проект превращает анализ в доказуемый результат

Цель блока: сохранить результат анализа так, чтобы его можно было воспроизвести, проверить и использовать для ручной валидации. Вход блока: `PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`, `HeuristicReport`, `DynamicReport`, `ReportMetadata`. Выход блока: JSON/HTML/CSV отчеты, console summary, manual validation steps.

Шаг 1. Подготовка report metadata

Что происходит: Перед JSON/HTML `main` собирает display path, command line, UTC timestamp, SHA-256 input и build/tool info.

Как работает: Эти данные помещаются в `ReportMetadata`: tool version, schema version, input hash, input size, compiler/platform/build mode, strict ABI/follow chain/max depth.

Зачем нужно: Без metadata отчет трудно защищать: непонятно, какой бинарник анализировался и с какими настройками.

Нюанс/ошибка: Если metadata собрать не удалось, JSON/HTML не пишутся, чтобы не создать неполный отчет.

Код: `src/main.c:450-480`

Шаг 2. Проверка metadata validity

Что происходит: Report writer проверяет, что metadata не пустая, schema version правильная, hash похож на 64 hex chars, timestamp имеет формат `YYYY-MM-DDTHH:MM:SSZ`, max chain depth в диапазоне 1..16.

Как работает: Это защита от внутренней ошибки программы: report не должен тихо сохранять битую служебную информацию.

Зачем нужно: Так отчет становится надежным артефактом, а не просто текстовым дампом.

Нюанс/ошибка: Если metadata invalid, writer возвращает false и добавляет ошибку.

Код: `src/report.c:556-594`

Шаг 3. Проверка model validity

Что происходит: Перед записью report проверяется согласованность моделей: sections pointer, limits findings/mitigations, dynamic event/trace caps, instrumentation counters.

Как работает: Например, если `dr->instrumentation_complete == true`, но при этом есть dropped events или breakpoint failures, model считается противоречивой.

Зачем нужно: Это не дает report layer случайно выдать incomplete dynamic evidence как complete.

Нюанс/ошибка: Если model invalid, report не пишется.

Код: `src/report.c:596-620`

Шаг 4. Экранирование JSON

Что происходит: Все строки JSON проходят через `json_escape`.

Как работает: Backslash, quotes, newlines, control chars и невалидный UTF-8 кодируются безопасно.

Зачем нужно: Файл пути, section name или message могут содержать спецсимволы. Без escaping JSON стал бы невалидным.

Нюанс/ошибка: Если строка содержит invalid UTF-8, символ кодируется как unicode escape, а не ломает весь отчет.

Код: `src/report.c:255-301`

Шаг 5. Запись ключевых PE и section facts в JSON

Что происходит: JSON report сохраняет input, metadata, image fields, exception directory, PE validation counters и section list.

Как работает: Для каждой секции сохраняются name, virtual address/size, raw offset/size, characteristics, readable/writable/executable flags.

Зачем нужно: Это позволяет комиссии или аналитику проверить findings: например, увидеть, почему handler считался non-executable или unwind metadata writable.

Нюанс/ошибка: Report не требует повторного запуска parser-a, чтобы увидеть базовые факты.

Код: `src/report.c:622-704`

Шаг 6. Static/dynamic assessment summary

Что происходит: Report layer оценивает completeness dynamic session, runtime evidence level, correlation static/dynamic и final verdict.

Как работает: Если dynamic session incomplete, runtime evidence не поднимается выше weak. Correlation появляется только когда static suspicious handler/placement совпадает с runtime hits/API calls.

Зачем нужно: Это защищает от переоценки динамики: один timeout или один неполный hit не должен превращаться в strong correlated case.

Нюанс/ошибка: Final confidence может увеличиться при complete dynamic signals и correlation, но capped до 0.98.

Код: `src/report.c:338-440`

Шаг 7. Manual validation steps

Что происходит: Report writer формирует список ручных шагов проверки в зависимости от findings.

Как работает: Например, при table integrity issues предлагает проверить `.pdata` ranges в PE viewer; при unwind anomalies - открыть UNWIND_INFO blob в hex view и сравнить с prolog в Ghidra; при handler dominance - проверить dominant handler target.

Зачем нужно: Это важная часть практической полезности: инструмент не только выносит verdict, но и говорит, что аналитику делать дальше.

Нюанс/ошибка: Если dynamic signals отсутствуют, но static findings есть, report предлагает повторить sample с `--dynamic` на representative exception path.

Код: `src/report.c:465-515`

Шаг 8. Экранирование HTML и CSP

Что происходит: HTML report экранирует &, <, >, quotes и invalid UTF-8; в head добавляется Content-Security-Policy.

Как работает: CSP запрещает внешние ресурсы, images, forms и base-uri, оставляя только inline style/script, которые report сам генерирует.

Зачем нужно: Это важно, потому что report может содержать строки из анализируемого файла. Нельзя позволять им становиться HTML/script injection.

Нюанс/ошибка: Если path или message содержит спецсимволы, они отображаются как текст, а не выполняются.

Код: `src/report.c:933-964, src/report.c:1192-1196`

Шаг 9. Atomic write output

Что происходит: Финальная запись файлов идет через temporary path и atomic replace.

Как работает: Это применяется report writers через utility function, чтобы итоговый JSON/HTML/CSV не остался наполовину записанным.

Зачем нужно: Для сдачи и практического использования важна сохранность результата: report либо старый, либо новый, но не битый промежуточный.

Нюанс/ошибка: Если atomic replace failed, temporary file удаляется и caller видит write error.

Код: `src/util.c:190-230`

110. Сквозные граничные случаи - что именно происходит в нестандартных ситуациях

Цель блока: показать, что проект не разваливается на нетипичных PE и не делает выводы без контекста. Вход блока: malformed PE, отсутствующие tables, strange sections, incomplete runtime. Выход блока: объяснимый отказ, finding, mitigation или neutral note.

Шаг 1. Файл не PE или не PE32+ AMD64

Что происходит: Если нет MZ/PE signature, machine не AMD64 или OptionalHeader magic не `0x20b`, parser возвращает ошибку.

Как работает: Проект не пытается анализировать неподходящую архитектуру, потому что его признаки завязаны на x64 unwind ABI.

Зачем нужно: Это честнее, чем поддерживать PE32 поверхностно и выдавать неверные findings.

Нюанс/ошибка: Ответ комиссии: ограничение архитектуры является осознанным scope проекта, а не недоработкой parser-a.

Код: `src/pe.c:40-90`

Шаг 2. Headers есть, но section table повреждена

Что происходит: Если section headers выходят за файл или не читаются, `pe_parse` возвращает false.

Как работает: Без section table нельзя корректно маппить RVA и оценивать права секций.

Зачем нужно: Такой файл не может быть надежно проанализирован на EH/unwind layer, потому что адресная модель сломана.

Нюанс/ошибка: Если section table читается, но содержит raw/virtual OOB/overlap/misalignment, parser не падает, а сохраняет counters для findings.

Код: `src/pe.c:139-172`

Шаг 3. Exception Directory отсутствует

Что происходит: Zero/zero directory приводит к `rf_count=0`, а затем к контекстной оценке.

Как работает: Если executable code есть, `no_runtime_functions` предлагает ручную проверку leaf-only объяснения. Если executable code нет, добавляется neutral note.

Зачем нужно: Это тонкая логика: отсутствие таблицы само по себе не равно вредоносность.

Нюанс/ошибка: Такой сценарий обязательно стоит проговорить на комиссии, потому что он показывает зрелость обработки edge cases.

Код: `src/unwind.c:623-625, src/heuristics.c:833-842`

Шаг 4. RVA внутри image, но bytes нет в файле

Что происходит: Если RVA попадает в virtual tail секции (`delta >= raw_size`), `pe_rva_to_fileoff` возвращает false.

Как работает: В памяти loader заполнит эту область нулями, но static parser читает файл, а не loaded process image.

Зачем нужно: Это предотвращает ложное чтение несуществующих bytes и ошибочную интерпретацию zero-fill как реального UNWIND_INFO.

Нюанс/ошибка: Если такой RVA нужен для Exception Directory или UNWIND_INFO, соответствующий блок фиксирует unreadable/malformed state.

Код: `src/pe.c:277-300`

Шаг 5. Таблица функций читается, но ranges плохие

Что происходит: Reader успешно извлек 12-byte records, но static layer находит `End<=Begin`, OOB, unsorted, overlaps или duplicate begins.

Как работает: Это нормальное разделение: raw table shape валиден, но смысловая карта функций повреждена.

Зачем нужно: Так проект не путает 'не смог прочитать bytes' и 'прочитал, но данные противоречат ABI/loader expectations'.

Нюанс/ошибка: Каждый тип получает свой finding, потому что причины и серьезность разные.

Код: `src/heuristics.c:646-670, src/heuristics.c:850-890`

Шаг 6. UNWIND_INFO parse failed только у части функций

Что происходит: Один failure не останавливает весь анализ.

Как работает: Каждый `us_ok[i]` хранит локальный результат. Heuristics затем считает количество failures и долю от `n`.

Зачем нужно: Это важно для реальных файлов: единичная битая запись и массово битая metadata - разные выводы.

Нюанс/ошибка: При доле выше порога score/severity увеличивается.

Код: `src/main.c:351-361, src/heuristics.c:727-736, src/heuristics.c:893-905`

Шаг 7. Dynamic session ничего не нашла

Что происходит: Отсутствие runtime hits не означает, что файл безопасен или что static findings ложные.

Как работает: Dynamic coverage зависит от выбранного execution path, input, timeout и completeness instrumentation.

Зачем нужно: Report layer поэтому рассматривает dynamic как дополнительный evidence layer и ослабляет incomplete sessions.

Нюанс/ошибка: Ответ комиссии: dynamic block подтверждает наблюдаемое поведение, но не доказывает отсутствие скрытого поведения.

Код: `src/report.c:362-374, src/report.c:405-440`

Шаг 8. Веса и thresholds выглядят 'ручными'

Что происходит: Да, веса задает разработчик как engineering calibration, но они не произвольны по смыслу.

Как работает: Hard structural violations получают больше баллов, контекстные placement signals меньше, массовость и корреляции усиливают score, mitigation rules снижают confidence.

Зачем нужно: Это типично для explainable heuristic analyzer: важна не магическая точность числа, а обоснованная и стабильная шкала.

Нюанс/ошибка: Если комиссия спросит '+25 или +26', ответ: одна единица не является физической константой; важны группы тяжести, пороги verdict и проверка на false positives.

Код: `src/heuristics.c:31-84, src/heuristics.c:514-588`

111. Как это объяснять комиссии простыми словами

Короткая версия: проект - это не один parser и не один набор проверок. Это pipeline, где каждый слой доказывает условия для следующего слоя. Сначала доказывается, что bytes можно читать. Потом доказывается, что это PE32+ AMD64. Потом строится карта sections и адресных пространств. Потом Exception Directory превращается в таблицу функций. Потом каждая функция получает UNWIND_INFO summary. Потом static heuristics превращают нарушения в объяснимые findings, score и confidence. Потом optional dynamic mode проверяет runtime-события через debugger. В конце report layer сохраняет результат так, чтобы его можно было проверить вручную.

Фраза для ответа на 'это просто if-ы': отдельный `if` проверяет один инвариант, но научная и инженерная часть проекта в том, что инварианты расположены в правильном порядке и относятся к разным координатным системам: file offset, RVA, image boundary, section raw range, section virtual range, function range, unwind blob и runtime VA. Если перепутать эти слои, analyzer начнет выдавать красивые, но ложные выводы.

КАРТА ИСХОДНИКОВ И СКВОЗНАЯ РАБОТА ПРОЕКТА

Этот раздел добавлен в конец глубокого описания по двум задачам: связать архитектурные блоки с конкретными файлами и строками исходников; затем описать всю работу проекта последовательно от загрузки бинарника до выдачи отчета.

112. Соответствие архитектурных блоков и исходных файлов

Архитектурный блок - это смысловая часть системы, а исходный файл - физическое место реализации. Чаще всего один блок имеет основной `.c` и основной `.h`, но есть сквозные блоки: например, `util` используется почти везде, `main` связывает все модули, а `report` собирает данные из parser/static/dynamic.

112.1. Файл -> какие блоки содержит

`src/main.c`

`47-88` - вспомогательная подготовка строк/UTF-8 для отображения пути и аргументов.

`89-116` - вывод версии и справки CLI.

`118-151` - сбор строки запуска для metadata отчета.

`154-173` - безопасный разбор числовых CLI-параметров.

`176-242` - разбор аргументов командной строки и включение режимов `--dump`, `--ops`, `--heuristics`, `--dynamic`, `--json`, `--html`, `--csv`.

`245-267` - печать одной записи `RUNTIME_FUNCTION` и ее `UNWIND_INFO` summary для dump-mode.

`270-527` - главный pipeline: загрузка файла, PE parse, чтение Exception Directory, parse UNWIND_INFO, static heuristics, dynamic validation, запись отчетов, cleanup.

`src/util.c` / `include/util.h`

`include/util.h:15-27` - контракт `Buf` и checked byte reads.

`src/util.c:13-35` - загрузка файла в память как binary buffer.

`src/util.c:42-70` - безопасные чтения `buf_read`, `buf_read_u8/u16/u32/u64`.

`include/util.h:39-66`, `src/util.c:72-188` - `StrBuf`: безопасный growable buffer для ошибок, details и report output.

`src/util.c:190-230` - atomic write временного файла с заменой итогового отчета.

`src/util.c:240-348` - SHA-256 для `input_sha256` в metadata отчета.

`src/pe.c` / `include/pe.h`

`include/pe.h:15-22` - PE constants: MZ, PE signature, AMD64, PE32+ magic, Exception Directory index.

`include/pe.h:24-31` - структура `PeSection`.

`include/pe.h:33-54` - структура `PeImage`: image fields, sections и counters структурных проблем.

`src/pe.c:5-34` - helpers: max, power-of-two, safe add, overlap.

`src/pe.c:36-256` - `pe_parse`: чтение DOS/NT/COFF/Optional Header, DataDirectory, section table, section invariants.

`src/pe.c:259-263` - освобождение `PeImage`.

`src/pe.c:265-301` - поиск секции по RVA и перевод RVA в file offset.

`src/unwind.c` / `include/unwind.h`

`include/unwind.h:16-20` - структура `RuntimeFunction`.

`include/unwind.h:22-43` - структура `UnwindSummary`.

`include/unwind.h:45-69` - flags и anomaly bitmask для UNWIND_INFO.

`include/unwind.h:72-82` - options parser-a и result type чтения runtime table.

`include/unwind.h:85-110` - публичные функции unwind parser-a.

`src/unwind.c:5-22` - имена регистров и строковое представление flags.

`src/unwind.c:24-53` - вычисление количества slots для unwind opcode.

`src/unwind.c:56-162` - чтение header, slots, chained record и сохранение parse failure.

`src/unwind.c:165-608` - основной `unwind_parse`: header, opcodes, operands, tail, handler, chain.

`src/unwind.c:611-681` - чтение Exception Directory как массива `RUNTIME_FUNCTION`.

`src/heuristics.c` / `include/heuristics.h`

`include/heuristics.h:17-23` - evidence families.

`include/heuristics.h:27-40` - `HeuristicFinding` и `HeuristicMitigation`.

`include/heuristics.h:42-93` - `HeuristicReport`.

`src/heuristics.c:31-84` - веса, confidence constants и thresholds.

`src/heuristics.c:86-145` - helpers для имен секций и executable-section checks.

`src/heuristics.c:159-275` - hash map для reuse/dominance подсчетов.

`src/heuristics.c:287-343` - representative locations, section issue lookup, sort comparator.

`src/heuristics.c:408-588` - создание findings, mitigations, evidence level, confidence и verdict.

`src/heuristics.c:590-1040` - основной static analysis: table integrity, unwind integrity, placement, reuse, combos, final score.

`src/heuristics.c:1040-1044` - освобождение `HeuristicReport`.

`src/dynamic_win.c` / `include/dynamic.h`

`include/dynamic.h:26-39` - `DynamicEvent` и event cap.

`include/dynamic.h:41-90` - `DynamicReport`: status flags, counters, events, trace, details.

`include/dynamic.h:92-110` - публичный API dynamic block.

`src/dynamic_win.c:31-153` - module list и module lookup.

`src/dynamic_win.c:155-397` - breakpoint storage, INT3 write/clear, self-check, pending rearm.

`src/dynamic_win.c:403-430` - append details и dynamic event storage.

`src/dynamic_win.c:449-457` - init/free dynamic report.

`src/dynamic_win.c:463-511` - безопасная сборка command line для `CreateProcessA`.

`src/dynamic_win.c:521-599` - поиск API адресов и постановка API breakpoints.

`src/dynamic_win.c:617-653` - trace after handler hit и отображение address -> RVA/section.

`src/dynamic_win.c:657-1209` - основной Windows Debug API session: CreateProcess, events, exceptions, breakpoint hits, cleanup.

`src/dynamic_win.c:1213-1253` - non-Windows stubs.

`src/report.c` / `include/report.h`

`include/report.h:18-31` - `ReportMetadata`.

`include/report.h:33-68` - API report layer.

`src/report.c:11-111` - helpers для masks, anomalies, comments и section lookup.

`src/report.c:113-122` - console brief.

`src/report.c:124-217` - CSV/Ghidra CSV writer.

`src/report.c:219-321` - UTF-8 validation и JSON escaping.

`src/report.c:326-440` - dynamic signal count, correlation, runtime evidence, final assessment.

`src/report.c:444-526` - manual validation steps.

`src/report.c:528-554` - console print heuristics.

`src/report.c:556-620` - metadata/model validation.

`src/report.c:622-923` - JSON writer.

`src/report.c:933-1149` - HTML escaping, Russian text mappings, manual/finding/mitigation text.

`src/report.c:1151-1526` - HTML writer: summary, PE info, sections, findings, functions, dynamic events.

`src/app_info.c` / `include/app_info.h`

`include/app_info.h:10-20` - version/schema/build/timestamp API.

`src/app_info.c:6-60` - версия инструмента, версия schema, compiler/platform/build mode, UTC timestamp.

112.2. Блок -> исходные файлы и диапазоны строк

main/orchestration

src/main.c:176-242, src/main.c:270-527; дополнительно metadata через src/app_info.c:6-60. Это блок управления всем pipeline.

buffer safety / util

include/util.h:15-66, src/util.c:13-230, src/util.c:240-348. Это не отдельный анализатор, а фундамент безопасного чтения bytes, строковых буферов, atomic write и hash metadata.

PE parser

include/pe.h:15-60, src/pe.c:36-256, src/pe.c:265-301. Это блок DOS/NT/COFF/Optional Header, DataDirectory, section table, alignments, RVA mapping.

Data Directory / Exception Directory evidence source

src/pe.c:126-137 читает DataDirectory[Exception]; src/pe.c:230-235 проверяет virtual boundary; src/unwind.c:623-647 различает absence/malformed/readable и проверяет size; src/report.c:669-680 выводит rva/size и validation flags. Важно: отдельного развернутого DataDirectoryEvidence объекта в отчете пока нет.

RUNTIME_FUNCTION table reader

include/unwind.h:16-20, include/unwind.h:78-106, src/unwind.c:611-681. Это блок превращения Exception Directory в массив записей по 12 bytes.

UNWIND_INFO parser

include/unwind.h:22-76, src/unwind.c:24-608. Это parser переменной ABI-структуры: slots, opcodes, operands, handler tail, chain tail, anomaly flags.

static heuristics

include/heuristics.h:17-104, src/heuristics.c:31-84, src/heuristics.c:408-588, src/heuristics.c:590-1040. Это блок findings, score, thresholds, confidence, mitigations, verdict.

dynamic validation

include/dynamic.h:26-110, src/dynamic_win.c:31-1209. Это Windows Debug API block: target launch, modules, breakpoints, API hooks, handler hits, trace, timeout, cleanup.

report layer

include/report.h:18-68, src/report.c:113-1526. Это console/CSV/JSON/HTML, metadata validation, escaping, assessment, manual validation steps.

version/build metadata

include/app_info.h:10-20, src/app_info.c:6-60. Этот блок питает report metadata: tool version, schema version, compiler/platform/build mode, UTC timestamp.

Как защищать архитектуру: блоки в описании - это не просто имена файлов. Блок показывает ответственность. Файл показывает место реализации. Один файл может содержать несколько подблоков, а один архитектурный блок может опираться на .h, .c, helper-функции и report-output в другом файле.

113. Сквозная пошаговая работа всего проекта

Этот раздел описывает работу проекта не по файлам, а во времени: что происходит с момента запуска инструмента до выдачи отчета. Это удобно учить как единую историю: входной PE-файл постепенно превращается в проверенную модель, затем в evidence, затем в отчет.

Шаг 1. Пользователь запускает инструмент

Что делает проект: Пользователь передает путь к PE-файлу и флаги: например `--heuristics`, `--dynamic`, `--json`, `--html`, `--csv`, `--dump`, `--ops`.

Зачем это нужно: Сначала нужно понять режим работы. Один и тот же analyzer может просто разобрать таблицу, вывести dump, построить static verdict, добавить dynamic evidence или записать отчеты.

Где в коде: `src/main.c:176-242`

Шаг 2. Программа задает значения по умолчанию

Что делает проект: `parse_args` выставляет `dump_limit=-1`, `follow_chain=true`, `max_chain_depth=8`, `timeout_ms=15000`, `trace_steps=64`.

Зачем это нужно: Даже если пользователь не указал все параметры, поведение остается определенным. Например, chain following включен, но глубина ограничена.

Где в коде: `src/main.c:176-183`

Шаг 3. CLI-параметры проверяются на диапазоны

Что делает проект: Числа вроде `--limit`, `--max-chain-depth`, `--timeout-ms`, `--trace-steps` проходят через safe range parsing.

Зачем это нужно: Это защищает от отрицательных значений, переполнений и бессмысленных настроек. Проект не запускает анализ в неопределенном режиме.

Где в коде: `src/main.c:154-173, src/main.c:204-231`

Шаг 4. Файл открывается как binary input

Что делает проект: После успешного разбора CLI проект вызывает `buf_init_from_file` и читает весь файл в `Buf { data, size }`.

Зачем это нужно: PE - бинарный формат. Нельзя читать его как текст: каждый byte, offset и размер должен остаться неизменным.

Где в коде: `src/main.c:283-288, src/util.c:13-35`

Шаг 5. Физическая граница файла становится известной

Что делает проект: `Buf.size` становится главным ограничителем всех дальнейших reads.

Зачем это нужно: Любой offset из PE-заголовков недоверенный. Проверка `file_size` нужна, чтобы не выйти за пределы памяти.

Где в коде: `include/util.h:15-18, src/util.c:42-48`

Шаг 6. Создается buffer диагностических ошибок

Что делает проект: Проект инициализирует `StrBuf err`, куда parser сможет положить причину отказа.

Зачем это нужно: Анализатор должен объяснять, что именно сломалось: не MZ, неверная PE signature, OptionalHeader OOB и т.д.

Где в коде: `src/main.c:290-291, src/util.c:72-188`

Шаг 7. PE parser проверяет DOS header

Что делает проект: `pe_parse` читает `MZ` и `e_lfanew`.

Зачем это нужно: `e_lfanew` - первый переход от DOS header к PE header. Если он поврежден, весь дальнейший layout бессмыслен.

Где в коде: `src/pe.c:11-18, src/pe.c:40-48`

Шаг 8. Проверяется PE signature

Что делает проект: По `e_lfanew` parser ищет `PE\0\0`.

Зачем это нужно: MZ еще не доказывает корректный PE. Нужно подтвердить NT headers.

Где в коде: `src/pe.c:50-54`

Шаг 9. Читается COFF header

Что делает проект: Parser читает `Machine`, `NumberOfSections`, `SizeOfOptionalHeader`.

Зачем это нужно: Эти поля говорят, какая архитектура у файла, сколько секций читать и где начинается optional header.

Где в коде: `src/pe.c:56-65`

Шаг 10. Отсекается неподдерживаемая архитектура

Что делает проект: Если `Machine != IMAGE_FILE_MACHINE_AMD64`, проект прекращает parse.

Зачем это нужно: ВКР анализирует Windows x64 EH/unwind metadata. Для x86 модель исключений другая, поэтому смешивать архитектуры нельзя.

Где в коде: `src/pe.c:67-70, include/pe.h:18-19`

Шаг 11. Проверяется OptionalHeader

Что делает проект: Parser убеждается, что OptionalHeader помещается в файл, достаточно велик и имеет magic PE32+ `0x20b`.

Зачем это нужно: Именно PE32+ содержит нужный layout для `ImageBase`, `SizeOfImage`, DataDirectory и x64 unwind contract.

Где в коде: `src/pe.c:72-90`

Шаг 12. Читаются ключевые поля image

Что делает проект: Parser читает `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `SizeOfHeaders`, `NumberOfRvaAndSizes`.

Зачем это нужно: Эти поля задают виртуальную координатную систему образа и правила перевода между file layout и memory layout.

Где в коде: `src/pe.c:92-108`

Шаг 13. Проверяются базовые image-инварианты

Что делает проект: Проект проверяет, что `SizeOfImage` не нулевой, headers лежат в файле, alignments не нулевые, DataDirectory содержит Exception entry.

Зачем это нужно: Без этих условий нельзя корректно говорить о границах image, секциях и Exception Directory.

Где в коде: `src/pe.c:109-123`

Шаг 14. Читается Exception DataDirectory

Что делает проект: Из `OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]` извлекаются `VirtualAddress` и `Size`.

Зачем это нужно: Это точка входа в `.pdata`/RUNTIME_FUNCTION table. Здесь важно помнить: `VirtualAddress` - это RVA, не VA и не file offset.

Где в коде: `src/pe.c:126-137`

Шаг 15. Читается section table

Что делает проект: Parser проверяет, что section headers помещаются в файл, затем читает имя, virtual size/address, raw size/offset, characteristics каждой секции.

Зачем это нужно: Секции связывают виртуальные адреса image с raw bytes на диске. Без них нельзя правильно перевести RVA в file offset.

Где в коде: `src/pe.c:139-172`

Шаг 16. Проверяется выравнивание и порядок секций

Что делает проект: Считаются нарушения `FileAlignment`, `SectionAlignment`, `SizeOfImage`, `SizeOfHeaders`, `ImageBase` и порядка `VirtualAddress`.

Зачем это нужно: Нормальный loader-compatible PE должен соблюдать эти инварианты. Нарушение не всегда fatal, но это evidence для structural findings.

Где в коде: `src/pe.c:180-197, src/pe.c:205-213`

Шаг 17. Проверяются raw/virtual границы секций

Что делает проект: Для каждой секции проверяется, не выходит ли raw range за размер файла, virtual range за `SizeOfImage`, и не пересекаются ли секции.

Зачем это нужно: Raw OOB ломает чтение bytes, virtual OOB ломает карту образа, overlaps делают отображение неоднозначным.

Где в коде: `src/pe.c:198-227`

Шаг 18. Проверяется виртуальная граница Exception Directory

Что делает проект: Если Exception Directory заявлена, проект проверяет, что RVA/Size согласованы и `RVA + Size <= SizeOfImage`.

Зачем это нужно: Это первая проверка directory на уровне image. Физическая читаемость из файла будет проверяться позже.

Где в коде: `src/pe.c:230-235`

Шаг 19. Формируется модель `PeImage`

Что делает проект: Parser сохраняет image fields, sections и counters нарушений в `PeImage`.

Зачем это нужно: С этого момента остальные блоки работают не с сырыми bytes headers, а с проверенной моделью PE.

Где в коде: `src/pe.c:237-256, include/pe.h:33-54`

Шаг 20. Exception Directory читается как таблица функций

Что делает проект: `main` вызывает `unwind_read_runtime_functions_ex`.

Зачем это нужно: Это переход от общего PE parser-а к предметному x64 EH/unwind слою.

Где в коде: `src/main.c:301-307, src/unwind.c:611-681`

Шаг 21. Обрабатывается отсутствие Exception Directory

Что делает проект: Если `exception_rva == 0` и `exception_size == 0`, reader возвращает OK и `rf_count=0`.

Зачем это нужно: Отсутствие directory не должно ломать parser. Но позже static heuristics решит, нормально ли это для image с executable sections.

Где в коде: `src/unwind.c:623-625, src/heuristics.c:833-842`

Шаг 22. RVA Exception Directory переводится в file offset

Что делает проект: Если directory присутствует, `pe_rva_to_fileoff` ищет секцию или headers range и вычисляет физический offset.

Зачем это нужно: Файл на диске не понимает VA. Для чтения bytes нужен file offset. RVA нельзя использовать как file offset напрямую.

Где в коде: `src/unwind.c:630-634, src/pe.c:277-300`

Шаг 23. Проверяется физическая читаемость directory

Что делает проект: Проект проверяет, что `file_offset + exception_size <= file_size`.

Зачем это нужно: Даже если начало таблицы маппится, конец может выходить за файл. Читать такую таблицу нельзя.

Где в коде: `src/unwind.c:635-638`

Шаг 24. Размер directory проверяется на формат RUNTIME_FUNCTION

Что делает проект: Проект проверяет `Size >= 12` и `Size % 12 == 0`, затем считает `n = Size / 12`.

Зачем это нужно: Одна запись `RUNTIME_FUNCTION` занимает 12 bytes. Если размер не кратен 12, таблица не является корректным массивом записей.

Где в коде: `src/unwind.c:639-647`

Шаг 25. Читаются записи RUNTIME_FUNCTION

Что делает проект: Для каждой записи читаются `BeginAddress`, `EndAddress`, `UnwindInfoAddress`.

Зачем это нужно: Это карта функций: какие RVA покрывает функция и где лежит ее `UNWIND_INFO`.

Где в коде: `src/unwind.c:654-671, include/unwind.h:16-20`

Шаг 26. При поврежденной directory сохраняется partial report

Что делает проект: Если reader вернул malformed, `main` выставляет `img.exception_directory_oob = true`, продолжает частичный structural report и ставит non-zero exit code.

Зачем это нужно: Даже поврежденный файл полезно описать: проект должен показать, что сломано, а не просто упасть.

Где в коде: `src/main.c:306-324`

Шаг 27. Печатается краткая сводка

Что делает проект: Проект выводит путь, ImageBase, SizeOfImage, SizeOfHeaders, число секций и число `RUNTIME_FUNCTION`.

Зачем это нужно: Это быстрый sanity-check для пользователя до длинных отчетов.

Где в коде: `src/main.c:326, src/report.c:113-122`

Шаг 28. Выделяются массивы `UnwindSummary` и `us_ok`

Что делает проект: На каждую runtime function выделяется место для результата parse UNWIND_INFO и флага успешности.

Зачем это нужно: Важно не смешивать две ситуации: таблица функций прочиталась, но конкретный `UNWIND_INFO` может быть битым.

Где в коде: `src/main.c:328-346`

Шаг 29. Для каждой функции запускается `unwind_parse`

Что делает проект: Цикл идет по `rf_count`, берет `rf[i].unwind_rva` и пытается разобрать UNWIND_INFO.

Зачем это нужно: Так каждая запись function table получает отдельное evidence: valid summary или parse failure.

Где в коде: `src/main.c:351-361`

Шаг 30. UNWIND_INFO RVA переводится в file offset

Что делает проект: Parser header-а вызывает `pe_rva_to_fileoff` для `unwind_rva`.

Зачем это нужно: Опять разделяются RVA и file offset. Виртуальный zero-fill tail не читается как bytes из файла.

Где в коде: `src/unwind.c:56-72`

Шаг 31. Разбирается header UNWIND_INFO

Что делает проект: Из первых 4 bytes извлекаются version, flags, prolog size, count of codes, frame register и frame offset.

Зачем это нужно: Эти поля определяют, как читать variable-length массив unwind codes и какой tail ожидать.

Где в коде: `src/unwind.c:187-239`

Шаг 32. Разбираются unwind opcodes и operands

Что делает проект: Parser идет по slots, вычисляет длину каждого opcode, читает operands и строит summary действий пролога.

Зачем это нужно: UNWIND_INFO не фиксированного размера. Layout зависит от opcodes, version и flags.

Где в коде: `src/unwind.c:24-53, src/unwind.c:254-516`

Шаг 33. Разбирается handler или chain tail

Что делает проект: После aligned slots parser читает chained `RUNTIME_FUNCTION` или handler RVA, если flags это требуют.

Зачем это нужно: Именно здесь metadata связывается с exception handler logic.

Где в коде: `src/unwind.c:520-551`

Шаг 34. При необходимости обходится chained unwind

Что делает проект: Если включен `follow_chain`, parser идет по chain, но ограничивает глубину и ловит loops.

Зачем это нужно: Malformed chain не должен зациклить analyzer. Поэтому есть max depth и seen set.

Где в коде: `src/unwind.c:559-604`

Шаг 35. Если включен dump, выводятся функции и операции

Что делает проект: В dump-mode `main` печатает записи и может повторно вызвать parser для текстового списка operations.

Зачем это нужно: Это учебно и диагностически полезно: пользователь видит не только итоговые findings, но и сырую карту функций.

Где в коде: `src/main.c:363-380, src/main.c:245-267`

Шаг 36. Если включены heuristics, создается static evidence

Что делает проект: `heuristics_analyze` получает `PeImage`, `rf`, `us`, `us_ok`, `rf_count`.

Зачем это нужно: Static analysis превращает технические факты в findings, counters, score, confidence и verdict.

Где в коде: `src/main.c:382-399, src/heuristics.c:590-1040`

Шаг 37. Static layer переносит PE structural counters

Что делает проект: Raw OOB, virtual OOB, overlaps, misalignment и exception_directory_oob становятся частью `HeuristicReport`.

Зачем это нужно: Так parser остается parser-ом, а оценка значимости нарушений происходит в heuristics.

Где в коде: `src/heuristics.c:590-610, src/heuristics.c:790-831`

Шаг 38. Static layer проверяет function table

Что делает проект: Проверяются invalid ranges, OOB ranges, OOB unwind RVA, сортировка, overlaps, duplicate begins.

Зачем это нужно: Это отвечает на вопрос: можно ли доверять карте функций, по которой Windows делает lookup при unwind.

Где в коде: `src/heuristics.c:646-670, src/heuristics.c:740-765, src/heuristics.c:850-890`

Шаг 39. Static layer проверяет UNWIND_INFO integrity

Что делает проект: Считаются parse failures, anomalies и ABI warnings.

Зачем это нужно: Так проект отличает unreadable metadata от readable, но suspicious metadata.

Где в коде: `src/heuristics.c:727-736, src/heuristics.c:893-905`

Шаг 40. Static layer проверяет placement и права секций

Что делает проект: Проверяется, где лежит `UNWIND_INFO`, является ли секция unusual/executable/writable, и лежит ли handler в executable/writable секции.

Зачем это нужно: Metadata обычно должна быть read-only data, а handler target должен быть executable code. Нарушения дают evidence.

Где в коде: `src/heuristics.c:690-724, src/heuristics.c:964-1002`

Шаг 41. Static layer считает reuse и dominance

Что делает проект: Hash maps считают, сколько функций используют один `UnwindInfoRVA` или один handler RVA.

Зачем это нужно: Массовое reuse/dominance может указывать на centralized dispatcher или защитную схему, но оценивается с порогами и контекстом.

Где в коде: `src/heuristics.c:159-275, src/heuristics.c:671-688, src/heuristics.c:908-962`

Шаг 42. Static layer добавляет combo findings

Что делает проект: Если независимые признаки совпадают, например placement + integrity или reuse + dominance, score усиливается.

Зачем это нужно: Комбинации важнее одиночных сигналов: их труднее объяснить случайностью или нестандартным benign layout.

Где в коде: `src/heuristics.c:1003-1023`

Шаг 43. Вычисляются score, confidence и verdict

Что делает проект: После findings проект считает suspicion, evidence level, confidence, weakening rules, manual review flag и verdict.

Зачем это нужно: Это делает вывод не бинарным, а объяснимым: есть подозрительность, уверенность и причины ослабления.

Где в коде: `src/heuristics.c:504-588`

Шаг 44. Если включена динамика, запускается debuggee

Что делает проект: `dynamic_collect_windows` строит command line и запускает target через Windows Debug API.

Зачем это нужно: Dynamic block проверяет runtime-события, связанные с exception handling, а не заменяет static parser.

Где в коде: `src/main.c:401-425, src/dynamic_win.c:657-710`

Шаг 45. Dynamic block ставит API breakpoints

Что делает проект: При загрузке модулей проект ищет API вроде `AddVectoredExceptionHandler`, `SetUnhandledExceptionFilter`, `RtlAddFunctionTable` и ставит breakpoints.

Зачем это нужно: Эти API показывают регистрацию exception handlers и runtime function tables.

Где в коде: `src/dynamic_win.c:521-599, src/dynamic_win.c:789-817`

Шаг 46. Dynamic block ставит breakpoints на static handlers

Что делает проект: Handler RVAs, найденные в UNWIND_INFO, переводятся в runtime VA через base image и получают breakpoints.

Зачем это нужно: Так static evidence проверяется в runtime: реально ли execution попадает в эти обработчики.

Где в коде: `src/dynamic_win.c:798-808`

Шаг 47. Debug event loop собирает события

Что делает проект: Проект обрабатывает create/load/unload/exit/exception/breakpoint/single-step events до timeout или exit.

Зачем это нужно: Это событийная debugger-система, а не просто `system("run exe")`.

Где в коде: `src/dynamic_win.c:765-845, src/dynamic_win.c:852-1143`

Шаг 48. Breakpoint hit обрабатывается корректно

Что делает проект: При INT3 проект восстанавливает original byte, перематывает RIP, включает trap flag и потом re-arm-ит breakpoint.

Зачем это нужно: Иначе target пропустил бы оригинальную инструкцию или breakpoint сработал бы только один раз.

Где в коде: `src/dynamic_win.c:852-1047, src/dynamic_win.c:1048-1139`

Шаг 49. Dynamic session получает completeness status

Что делает проект: В конце выставляются ok, `timed_out`, `target_exited`, `instrumentation_complete`, failures и counters.

Зачем это нужно: Неполная dynamic session не должна усиливать итог как полноценное runtime доказательство.

Где в коде: `src/dynamic_win.c:1143-1209, src/report.c:338-374`

Шаг 50. Опционально пишутся CSV и Ghidra CSV

Что делает проект: Если пользователь передал `--csv` или `--ghidra-csv`, проект пишет табличный export функций и unwind summary.

Зачем это нужно: CSV удобен для внешней проверки, сортировки и импорта в инструменты анализа.

Где в коде: `src/main.c:427-448, src/report.c:124-217`

Шаг 51. Готовится metadata для JSON/HTML

Что делает проект: Проект собирает command line, UTC timestamp, SHA-256 input, tool version, schema version, compiler/platform/build mode.

Зачем это нужно: Отчет должен быть воспроизводимым: нужно знать, какой файл, какой версией инструмента и с какими флагами анализировался.

Где в коде: `src/main.c:450-480, src/app_info.c:6-60, src/util.c:333-348`

Шаг 52. JSON writer проверяет model и metadata

Что делает проект: Перед записью JSON report проверяет корректность metadata, counters, dynamic limits и instrumentation consistency.

Зачем это нужно: Отчет не должен сохранять внутренне противоречивую модель как достоверный результат.

Где в коде: `src/report.c:556-620`

Шаг 53. JSON report сериализует evidence

Что делает проект: JSON содержит PE fields, exception directory rva/size, sections, heuristic counters, findings, dynamic events, assessment, functions.

Зачем это нужно: Это машинно-читаемая доказательная база. Часть evidence уже раскрыта, но отдельный подробный DataDirectoryEvidence object пока можно добавить как улучшение.

Где в коде: `src/report.c:622-923`

Шаг 54. HTML writer делает читаемый отчет

Что делает проект: HTML выводит summary, PE info, structural checks, section table, findings, function table, dynamic events.

Зачем это нужно: Это human-readable слой: он нужен комиссии и аналитику, чтобы быстро понять результат без чтения JSON.

Где в коде: `src/report.c:933-1526`

Шаг 55. Report layer формирует итоговую оценку

Что делает проект: Assessment объединяет static evidence level, runtime evidence level, correlation, final confidence и final verdict.

Зачем это нужно: Так проект не смешивает static и dynamic blindly: correlation усиливает вывод, incomplete dynamic ослабляет runtime evidence.

Где в коде: `src/report.c:326-440`

Шаг 56. Report layer добавляет manual validation steps

Что делает проект: В зависимости от findings отчет предлагает ручные проверки: PE viewer, Ghidra, handler target, runtime table arguments.

Зачем это нужно: Инструмент не заменяет аналитика полностью. Он показывает, куда смотреть дальше и почему.

Где в коде: `src/report.c:465-526`

Шаг 57. Файлы отчетов записываются атомарно

Что делает проект: Report output пишется через temporary file и replace.

Зачем это нужно: Это защищает от поломанного отчета при сбое записи.

Где в коде: `src/util.c:190-230`

Шаг 58. Память и ресурсы освобождаются

Что делает проект: В конце освобождаются heuristic report, dynamic report, buffers, arrays, PE sections и исходный file buffer.

Зачем это нужно: В C это часть надежности: analyzer должен корректно работать пакетно на множестве файлов.

Где в коде: `src/main.c:511-526`

Шаг 59. Возвращается exit code

Что делает проект: Если critical parse/write/dynamic error была, `exit_code` становится non-zero; если анализ прошел, возвращается 0.

Зачем это нужно: Exit code нужен для автоматизации, тестов и batch-mode запуска.

Где в коде: `src/main.c:270-527`

Главная сквозная формула проекта: bytes -> checked PE model -> Exception Directory -> RUNTIME_FUNCTION table -> UNWIND_INFO summaries -> static findings -> optional dynamic evidence -> reproducible reports -> cleanup/exit code.

DATA DIRECTORY VALIDATION И УСЛОВИЯ PIPELINE

Этот раздел уточняет сквозную схему из 59 шагов. Эти 59 шагов - не утверждение, что любой запуск всегда проходит все пункты. Это максимальная объяснительная модель полного анализа: статический разбор, опциональная динамика, опциональные отчеты и завершение. Реальный путь зависит от CLI-флагов, корректности входного PE, наличия Exception Directory, количества `RUNTIME_FUNCTION` entries и успешности отдельных слоев.

114. Валидация Data Directory и что именно делает ВКР

Validation - это проверка корректности данных по набору правил. Для PE/unwind такая проверка начинается не с `UNWIND_INFO`, а раньше: с Optional Header, Data Directory, `VirtualAddress/Size`, `SizeOfImage`, section table, file-offset mapping и размера таблицы `RUNTIME_FUNCTION`. Ниже - точная карта: что проект делает сейчас, где это находится в коде, зачем нужно, и где есть ограничение текущей реализации.

114.1. Проверка доступности entry `DataDirectory[Exception]`

Статус в ВКР: выполняется.

Что проверяется: parser не читает `DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]`, пока не убедится, что `NumberOfRvaAndSizes` больше индекса Exception Directory и что сама запись directory помещается внутри Optional Header.

Зачем нужно: если `NumberOfRvaAndSizes` меньше нужного индекса, parser не имеет права обращаться к этой записи. Иначе он будет читать bytes, которые по формату PE не принадлежат этому массиву, то есть фактически мусор за пределами заявленной структуры.

Где в коде: `src/pe.c:121-123` проверяет `number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1`; `src/pe.c:126-130` проверяет, что `dd_rel + 8 <= opt_size`; `include/pe.h:22` задает `IMAGE_DIRECTORY_ENTRY_EXCEPTION = 3`.

Как объяснять комиссии: ВКР не доверяет тому, что Exception Directory 'должна быть'. Сначала он проверяет, что Optional Header действительно содержит такую entry, и только потом читает пару RVA/Size.

114.2. Проверка пары `VirtualAddress/Size`

Статус в ВКР: выполняется.

Что проверяется: если `VirtualAddress = 0` и `Size = 0`, directory считается отсутствующей. Если одно поле нулевое, а другое нет, это считается противоречивой/malformed directory. Если оба ненулевые, дальше проверяется диапазон.

Зачем нужно: пара RVA/Size имеет смысл только вместе. Размер без адреса не указывает на таблицу. Адрес без размера не задает область чтения. Zero/zero - нормальный способ сказать, что такой каталог отсутствует.

Где в коде: `src/pe.c:230-235` выставляет `exception_directory_oob`, если пара противоречива или выходит за `SizeOfImage`; `src/unwind.c:623-629` отдельно различает absence (`RVA=0`, `Size=0`) и malformed (`RVA=0` xor `Size=0`).

Как это попадает в evidence: если reader видит malformed directory, `main` выставляет `img.exception_directory_oob = true` в `src/main.c:317-323`, а heuristics создает finding `pe_exception_directory_oob` в `src/heuristics.c:825-831`.

114.3. Проверка попадания Exception Directory в `SizeOfImage`

Статус в ВКР: выполняется.

Что проверяется: если directory присутствует, проект считает `exc_end = exc_rva + exc_sz` и проверяет, что конец не выходит за `SizeOfImage`.

Зачем нужно: `SizeOfImage` - это общая виртуальная граница загруженного PE-образа. Если directory указывает за эту границу, loader/анализатор не должен считать такую область нормальной частью image.

Где в коде: `src/pe.c:230-235`.

Связанные проверки дальше: уже после чтения function table static layer проверяет, что `BeginAddress/EndAddress` и `UnwindInfoAddress` каждой `RUNTIME_FUNCTION` записи не выходят за `SizeOfImage`; это `src/heuristics.c:651-660`.

Handler RVA внутри UNWIND_INFO также проверяется на выход за image в src/unwind.c:540-550.

114.4. Перевод RVA в section и file offset

Статус в ВКР: выполняется.

Что проверяется: когда нужно реально читать таблицу из файла, проект переводит Exception Directory RVA в file offset. Для этого он ищет секцию, чей virtual range покрывает RVA, либо допускает специальный случай RVA внутри headers.

Зачем нужно: RVA, VA и file offset - разные координаты. VirtualAddress из Data Directory - это RVA, а не offset в файле. Чтобы прочитать bytes на диске, нужно пройти через section table: file offset = PointerToRawData + (RVA - Section.VirtualAddress).

Где в коде: pe_find_section_for_rva находится в src/pe.c:265-274; pe_rva_to_fileoff находится в src/pe.c:277-300; Exception Directory reader вызывает этот перевод в src/unwind.c:630-634; UNWIND_INFO header reader использует тот же механизм в src/unwind.c:56-72.

Важный нюанс: если RVA попадает в virtual tail секции, где delta >= raw_size, ВКР отказывается читать bytes из файла. Это правильно, потому что loader заполнит такую область нулями в памяти, но этих bytes нет на диске. Код: src/pe.c:281-285.

Еще один нюанс: проект учитывает headers range. Если RVA меньше SizeOfHeaders и физически помещается в файл, pe_rva_to_fileoff маппит такой RVA напрямую. Код: src/pe.c:293-299.

114.5. Проверка размера таблицы RUNTIME_FUNCTION

Статус в ВКР: выполняется, но finding сейчас обобщенный.

Что проверяется: если Exception Directory присутствует и маппится в файл, проект проверяет, что размер области не меньше 12 bytes и кратен 12. Затем количество записей вычисляется как exception_size / 12.

Зачем нужно: на x64 одна RUNTIME_FUNCTION запись состоит из трех DWORD: BeginAddress, EndAddress, UnwindInfoAddress, то есть 12 bytes. Если размер не кратен 12, область не является корректным массивом записей.

Где в коде: src/unwind.c:639-647.

Как это попадает в evidence: если Size % 12 != 0, reader возвращает malformed с текстом ошибки Exception directory size is not a multiple of RUNTIME_FUNCTION size.. main помечает img.exception_directory_oob = true в src/main.c:317-323, а heuristics создает общий finding pe_exception_directory_oob. Сейчас в проекте нет отдельного finding code вида exception_directory_size_not_multiple_of_runtime_function. Это честное ограничение текущей evidence-модели.

Как отвечать комиссии: проверка есть, но детализация finding пока укрупненная. Улучшение на будущее - вывести отдельный evidence object с Size, entry_size=12, remainder=Size % 12 и отдельным кодом finding.

114.6. Проверка прав секции для Exception Directory / .pdata

Статус в ВКР: частично, но не как отдельный finding именно для Exception Directory.

Что проект уже делает: он сохраняет права всех секций в отчеты. В JSON для каждой секции выводятся readable, writable, executable, characteristics; это src/report.c:682-697. В HTML показывается таблица секций с правами R/W/X; это src/report.c:1307-1326.

Что проект проверяет как findings: он проверяет placement и права для UNWIND_INFO и handler targets. Если UNWIND_INFO лежит в executable/writable/unusual секции, появляются findings unwind_in_exec_section, unwind_writable_section, unwind_unusual_section. Если handler target лежит в non-executable или writable секции, появляются handler_nonexec_section, handler_writable_section. Код: src/heuristics.c:690-724, findings: src/heuristics.c:964-1002.

Чего сейчас нет как отдельного finding: проект не выделяет отдельную проверку 'сама Exception Directory / .pdata лежит в writable или executable секции' с отдельным кодом finding. То есть права секции можно увидеть в report sections table, но они не превращаются в отдельный Data Directory finding.

Зачем такая проверка была бы полезна: .pdata/Exception Directory - это служебная таблица runtime functions. Обычно она является read-only metadata. Если она writable или executable, это не обязательно вредоносность, но повод для внимания: packer, нестандартный linker, manual mapping, runtime patching или повреждение характеристик

секций.

Как честно формулировать: ВКР проверяет права секций на уровне EH/unwind metadata и handler targets, но не имеет отдельного finding именно для прав секции, содержащей Exception Directory. Это можно добавить как развитие evidence-модели.

114.7. Итог по шести проверкам Data Directory validation

1. Доступность entry через `NumberOfRvaAndSizes` - да, выполняется.

2. Проверка пары `VirtualAddress/Size` - да, выполняется; absence и malformed различаются.

3. Проверка попадания directory в `SizeOfImage` - да, выполняется.

4. Перевод RVA в section/file offset - да, выполняется через `pe_rva_to_fileoff`; учитываются sections, headers и отказ от virtual zero-fill bytes.

5. Проверка размера таблицы на кратность 12 - да, выполняется; но finding сейчас общий `pe_exception_directory_oob`, а не отдельный 'size not multiple'.

6. Проверка прав секции самой Exception Directory - частично: права секций выводятся, а placement/permissions проверяются для `UNWIND_INFO` и handler targets; отдельного finding именно для `.pdata`/Exception Directory permissions сейчас нет.

Главная защитная формулировка: 'ВКР уже реализует базовую и важную validation-цепочку Data Directory, но evidence-витрина может быть усилена отдельным DataDirectoryEvidence-блоком и отдельными finding codes для размера directory и прав секции `.pdata`.'

115. Главный принцип: pipeline охраняемый, а не линейный без условий

115.1. Что означает 'выполняется всегда'

В этом проекте 'выполняется всегда' нужно понимать аккуратно. Есть служебные запуски, например `--help` и `--version`, где программа вообще не должна читать PE-файл. Есть нормальный анализ PE, где сначала обязателен CLI parse, затем чтение файла, затем PE parse. Но даже в нормальном анализе каждый следующий слой запускается только если предыдущий слой дал достаточно модели для продолжения.

Поэтому правильная формулировка такая: обязательные шаги - это шаги, которые программа пытается выполнить при обычном анализе PE, если до них дошел pipeline. Опциональные шаги - это шаги, которые запускаются только по флагу или при наличии соответствующих данных. Условные шаги - это шаги, которые зависят от результата parser-а, наличия таблицы, количества записей или состояния dynamic session.

115.2. Почему нельзя говорить 'все 59 шагов всегда выполняются'

Потому что это было бы технически неверно. Например, без `--dynamic` не запускается Windows debugger. Без `--json` не пишется JSON. Без `--html` не пишется HTML. Если пользователь передал `--help`, программа печатает справку и завершает работу. Если файл не PE32+ AMD64, проект останавливается на PE parser-е. Если `Exception Directory` отсутствует, чтение `RUNTIME_FUNCTION` не дает записей, а цикл `UNWIND_INFO` фактически не проходит по функциям.

Но это не недостаток. Это нормальная архитектура analyzer-а: он не делает лишнюю опасную работу и не строит вывод, если нет корректной основы. На защите это нужно объяснять как плюс: pipeline проверяет условия перехода между слоями.

116. Карта обязательности шагов

116.1. Служебные запуски: `--help` и `--version`

Условие: пользователь запускает программу с `--help`, `-h` или `--version`.

Что выполняется: только начальный разбор аргументов и печать справки или версии. Входной бинарник не читается, PE parser не запускается, Exception Directory не анализируется, отчеты не пишутся.

Почему так правильно: справка и версия - это не режим анализа. Если пользователь просто спрашивает возможности инструмента, программа не должна требовать PE-файл и не должна выполнять потенциально долгий анализ.

Связь с 59 шагами: из полной схемы фактически выполняется только стартовая часть шага 1 и ранний выход. Шаги 4-59 не выполняются.

Код: `src/main.c:176-187`, `src/main.c:273-281`.

116.2. Обязательные шаги обычного анализа: 1-19

Условие: пользователь запустил нормальный анализ PE, то есть передал путь к файлу и не ушел в `--help`/`--version`.

Шаги 1-3 обязательны для нормального запуска: разбор CLI, defaults, проверка числовых параметров. Если CLI некорректный, анализ не начинается.

Шаги 4-6 обязательны после успешного CLI: открыть файл как binary input, загрузить его в `Buf`, создать диагностические buffers. Если файл не читается, PE parser не запускается.

Шаги 7-19 обязательны после успешной загрузки файла: DOS header, PE signature, COFF, архитектура AMD64, OptionalHeader PE32+, image fields, DataDirectory, sections, alignments, raw/virtual ranges, Exception Directory virtual boundary и формирование `PeImage`.

Если любой hard-check PE parser-а не проходит, pipeline завершает анализ раньше. Например, не MZ, не PE, не AMD64, не PE32+, OptionalHeader вне файла, section headers вне файла - это hard stop.

Код: `src/main.c:176-299`, `src/pe.c:36-256`.

116.3. Обязательные после PE parse, но зависящие от DataDirectory: 20-27

Условие: `pe_parse` успешно построил `PeImage`.

Шаг 20 запускается всегда после успешного PE parse: `main` вызывает `unwind_read_runtime_functions_ex`. Но результат этого вызова может быть разным: directory отсутствует, directory повреждена, directory читается.

Если `Exception Directory` отсутствует (`RVA=0`, `Size=0`), reader возвращает OK и `rf_count=0`. Это не crash. Дальше не будет чтения записей `RUNTIME_FUNCTION`, потому что записей нет.

Если `Exception Directory` присутствует, выполняются шаги перевода RVA в file offset, проверки физической читаемости, проверки `Size >= 12`, проверки `Size % 12 == 0`, вычисления `n = Size / 12` и чтения записей.

Если directory malformed, проект переходит в partial structural mode: выставляет `img.exception_directory_oob = true`, сообщает ошибку и продолжает там, где это возможно. Если ошибка OOM, это hard stop.

Код: `src/main.c:301-324`, `src/unwind.c:611-681`, `src/pe.c:277-300`.

116.4. Условные шаги UNWIND_INFO: 28-34

Условие: программа дошла до этапа после чтения `RUNTIME_FUNCTION` table.

Шаг 28 - выделение массивов `UnwindSummary` и `us_ok` - выполняется даже если `rf_count=0`, но размер берется безопасно как минимум 1 элемент. Это нужно для стабильного C-поведения.

Шаги 29-33, то есть реальный цикл `unwind_parse` по функциям, выполняются только если `rf_count > 0`. Если `RUNTIME_FUNCTION` entries нет, цикл не делает итераций.

Шаг 34 с dump-output выполняется только если пользователь включил `--dump`. Показ операций `UNWIND_INFO` внутри dump дополнительно зависит от `--ops` и успешного parse конкретной записи.

Код: `src/main.c:328-380`, `src/unwind.c:165-608`.

116.5. Статические эвристики: шаги 35-43

Условие: пользователь передал флаг `--heuristics`.

Если `--heuristics` не передан, проект все равно может разобрать PE, `RUNTIME_FUNCTION` и `UNWIND_INFO`, но не будет строить `HeuristicReport`: не будет score, confidence, verdict, findings, mitigations и static evidence level.

Если `--heuristics` включен, выполняются шаги переноса PE counters, проверки table integrity, проверки unwind integrity, placement checks, reuse/dominance analysis, combo findings и финализации score/confidence/verdict.

Если `rf_count=0`, эвристики все равно могут выполниться: например, `no_runtime_functions` появляется при отсутствующей Exception Directory и наличии executable sections. Если executable sections нет, добавляется нейтральная note.

Код: `src/main.c:388-399`, `src/heuristics.c:590-1040`.

116.6. Динамическая проверка: шаги 44-50

Условие: пользователь передал `--dynamic`.

Если `--dynamic` не передан, target не запускается, Windows Debug API не используется, breakpoints не ставятся, runtime events не собираются. Static analysis от этого не становится хуже - просто нет runtime evidence.

Если `--dynamic` включен, dynamic block запускается после static parsing stage, потому что ему нужны `PeImage`, `RuntimeFunction[]`, `UnwindSummary[]` и `us_ok[]`. Эти данные используются для постановки breakpoints на handlers, найденные в UNWIND_INFO.

Dynamic validation зависит от ОС и запуска target. На Windows он работает через Debug API. Если target не стартовал, не было `CREATE_PROCESS_DEBUG_EVENT`, произошел timeout, не удалось поставить breakpoints или instrumentation incomplete, это отражается в `DynamicReport` и позже ослабляет runtime evidence.

Важно: отсутствие dynamic hits не доказывает отсутствие проблемы. Это может означать, что нужный exception path не исполнился, timeout был слишком мал или instrumentation была неполной.

Код: `src/main.c:401-425`, `src/dynamic_win.c:657-1209`, `include/dynamic.h:26-110`.

116.7. CSV и Ghidra CSV: шаг 51

Условие: пользователь передал `--csv path` или `--ghidra-csv path`.

Если эти флаги не переданы, CSV не пишется. Это нормально: CSV - дополнительный export, а не обязательная часть анализа.

CSV может быть полезен, даже если `--heuristics` не включен: он выводит таблицу функций и unwind summary для внешней проверки или импорта в инструменты.

Код: `src/main.c:427-448`, `src/report.c:124-217`.

116.8. Metadata, JSON и HTML: шаги 52-54

Условие шага 52: пользователь передал `--json` или `--html`. Только тогда проект собирает report metadata: command line, timestamp, SHA-256 input, tool/schema/build info.

Условие шага 53: пользователь передал `--json`, metadata готова и model valid. Тогда пишется JSON report.

Условие шага 54: пользователь передал `--html`, metadata готова и model valid. Тогда пишется HTML report.

Если `--json`/`--html` не переданы, эти шаги не выполняются. Console output и, при необходимости, CSV могут существовать без JSON/HTML.

Если metadata не удалось собрать, JSON/HTML не должны записываться, потому что отчет без hash, timestamp, schema и build info был бы слабым доказательным артефактом.

Код: `src/main.c:450-510`, `src/report.c:556-620`, `src/report.c:622-923`, `src/report.c:1151-1526`.

116.9. Assessment и manual validation: шаги 55-56

Условие: assessment и manual validation steps формируются внутри report layer, когда пишется JSON/HTML и есть static report (`hr`) или dynamic report (`dr`).

Если `--heuristics` и `--dynamic` не включены, то `hr` и `dr` могут отсутствовать. Тогда итоговая оценка подозрительности не имеет полноценной evidence model, хотя базовые PE/function данные все еще могут быть выведены.

Manual validation steps зависят от findings и dynamic events. Например, если есть `rf_oob`, предлагается проверить `.pdata`; если есть handler dominance, предлагается открыть handler target в Ghidra; если есть `RtlAddFunctionTable`,

предлагается проверить runtime table arguments.

Код: `src/report.c:326-440`, `src/report.c:465-526`, `src/report.c:847-861`.

116.10. Atomic write, cleanup и exit code: шаги 57-59

Шаг 57 выполняется для тех файлов отчетов, которые реально запрошены и успешно дошли до записи. Если JSON не запрошен, atomic write JSON не выполняется. Если HTML не запрошен, atomic write HTML не выполняется.

Шаг 58 - cleanup - выполняется на нормальном пути в конце, а также частично на ранних ошибках: освобождаются только те ресурсы, которые уже были выделены. Это важно для C-кода.

Шаг 59 - возврат exit code - выполняется всегда, потому что любой процесс завершается кодом возврата. Значение зависит от результата: успешный анализ обычно возвращает 0, hard/partial errors могут дать non-zero.

Код: `src/util.c:190-230`, `src/main.c:511-526`, `src/main.c:270-527`.

117. Сводная карта режимов запуска

117.1. Минимальный статический запуск без отчетов

Пример: `ehua.exe sample.exe`.

Выполняется: CLI parse, file load, PE parser, Exception Directory reader, `RUNTIME_FUNCTION` reading, `UNWIND_INFO` parse, brief console output, cleanup, exit code.

Не выполняется: static heuristics, dynamic validation, JSON/HTML/CSV reports, assessment, manual validation steps.

Зачем нужен такой режим: быстро проверить, что файл является PE32+ AMD64 и что таблицы unwind metadata читаются.

117.2. Статический анализ с findings

Пример: `ehua.exe sample.exe --heuristics`.

Дополнительно выполняется: static heuristics, findings, score, confidence, verdict, manual_review flag, console print heuristics.

Не выполняется: dynamic validation и report files, если не переданы соответствующие флаги.

Зачем нужен такой режим: получить объяснимую static evidence model без запуска target.

117.3. Статический анализ с JSON/HTML отчетом

Пример: `ehua.exe sample.exe --heuristics --json out.json --html out.html`.

Дополнительно выполняется: metadata сбор, JSON writer, HTML writer, assessment, manual validation steps, atomic write output.

Зависит от условий: если metadata невалидна или model validation failed, отчет не должен записываться.

Зачем нужен такой режим: получить сдаваемый/проверяемый артефакт с evidence, findings и шагами ручной проверки.

117.4. Динамический режим

Пример: `ehua.exe sample.exe --heuristics --dynamic --html out.html`.

Дополнительно выполняется: запуск debuggee, module refresh, API breakpoints, static handler breakpoints, debug event loop, runtime events, trace, dynamic completeness status.

Зависит от условий: target должен запускаться; session может завершиться timeout; instrumentation может быть incomplete; нужный exception path может не исполниться.

Зачем нужен такой режим: добавить runtime evidence к static evidence, но не заменять static analysis.

117.5. Export-режим CSV/Ghidra CSV

Пример: `ehua.exe sample.exe --csv out.csv --ghidra-csv ghidra.csv`.

Дополнительно выполняется: CSV writer или Ghidra-compatible CSV writer.

Зависит от условий: таблица функций и unwind summaries должны быть доступны настолько, насколько их удалось разобрать; при write error exit code станет non-zero.

Зачем нужен такой режим: передать данные в таблицу, скрипты или reverse engineering tools.

117.6. Поврежденный файл или неподдерживаемая архитектура

Если файл не читается, pipeline останавливается до PE parser-а.

Если файл не MZ/PE, не AMD64 или не PE32+, pipeline останавливается на `pe_parse`.

Если обязательные headers повреждены, pipeline останавливается на parser-е.

Если PE parse прошел, но Exception Directory повреждена, возможен partial structural report: проект фиксирует `exception_directory_oob`, не доверяет runtime table, но сохраняет диагностическую ценность.

Если один `UNWIND_INFO` поврежден, весь анализ не падает: конкретная запись получает `us_ok=false`, а static layer позже считает `unwind_parse_fail`.

118. Как коротко отвечать комиссии

Полный pipeline я разложил на 59 логических шагов от запуска до отчета и exit code. Но эти шаги не означают, что каждый запуск всегда проходит все 59 пунктов. Обязательная часть - это guarded static pipeline до построения PE/EH/unwind модели. Дальше есть режимы: `--heuristics` включает findings/score/confidence, `--dynamic` включает Windows Debug API validation, `--json` и `--html` включают отчеты, `--csv` включает export. Если входной файл поврежден, pipeline либо останавливается на hard error, либо переходит в partial structural report. Такая условность не слабость, а правильная архитектура analyzer-a: каждый слой запускается только при наличии доказанной основы для следующего слоя.

Формула для защиты: '59 шагов - это максимальная объяснительная схема полного анализа. Реальный путь выбирается условиями: корректность входного PE, наличие Exception Directory, число RUNTIME_FUNCTION entries, флаги пользователя, успешность dynamic session и запрошенные форматы отчета.'

ПОЛНАЯ ПРОВЕРОЧНАЯ МЕХАНИКА ПРОЕКТА

Этот раздел отвечает на вопрос: как именно проект проверяет бинарник в проверочном смысле. Важно: 27 findings - это не 27 случайных if-ов по байтам файла. Проект сначала строит несколько моделей: файл как `Buf`, PE-образ как `PeImage`, таблицу функций как массив `RuntimeFunction`, unwind layer как массив `UnwindSummary`, затем уже из этих моделей выводит признаки, score, confidence, verdict и отчет.

119. Главная схема: от файла до findings

119.1. Что считается проверкой в этом проекте

Проверка в этом проекте - это не просто сравнение двух чисел. Каждая проверка отвечает на практический вопрос: можно ли безопасно читать структуру, принадлежит ли адрес допустимой области image, согласованы ли file-layout и virtual-layout, можно ли разобрать unwind metadata, указывает ли handler в разумное место, и не образуют ли отдельные странности общий подозрительный паттерн.

Логическая цепочка такая: `input file -> Buf -> PeImage -> sections -> Exception Directory -> RUNTIME_FUNCTION[] -> UnwindSummary[] -> counters -> findings -> score/confidence/verdict -> JSON/HTML evidence`.

Если комиссия говорит 'это же просто if-ы', правильный ответ: каждый if защищает переход между координатными системами или слоями формата. Без этих проверок parser будет читать мусор, путать RVA/file offset/VA, строить неверную карту функций и выдавать ложные выводы.

119.2. Где в исходниках находятся проверочные слои

`src/main.c` управляет запуском, режимами, порядком слоев и cleanup: `src/main.c:270-526`.

`src/util.c` читает файл и дает безопасные функции чтения little-endian чисел: `src/util.c:13-70`.

`src/pe.c` проверяет PE32+ x64, Optional Header, Data Directory и section table: `src/pe.c:36-256`.

`src/pe.c` также переводит `RVA -> file offset`: `src/pe.c:265-300`.

`src/unwind.c` читает Exception Directory как массив `RUNTIME_FUNCTION` и разбирает `UNWIND_INFO`: `src/unwind.c:56-72`, `src/unwind.c:165-608`, `src/unwind.c:611-681`.

`src/heuristics.c` превращает проверенные данные в counters, 27 findings, score, confidence и verdict: `src/heuristics.c:590-1038`, `src/heuristics.c:514-588`.

`src/report.c` делает evidence-витрину: metadata, PE validation, sections, findings, dynamic events, functions table: `src/report.c:622-920`, `src/report.c:1151-1520`.

120. Блок запуска: какие проверки вообще включены

120.1. CLI не просто принимает флаги, он задает границы анализа

Сначала проект разбирает аргументы командной строки. `--heuristics` включает формирование 27 static findings; `--dynamic` включает runtime evidence; `--json`, `--html`, `--csv`, `--ghidra-csv` включают соответствующие выводы; `--strict-abi`, `--no-follow-chain`, `--max-chain-depth` меняют глубину и строгость unwind-проверки.

Как именно: `parse_args` задает defaults и валидирует числовые параметры. Например, `--max-chain-depth` ограничен диапазоном 1..16, а `--trace-steps` диапазоном 0..256. Код: `src/main.c:176-242`.

Зачем: проверочный результат должен быть воспроизводимым. Если пользователь включил только базовый разбор, проект не должен делать вид, что построил heuristic verdict. Если включена динамика, проект должен явно показать, что это отдельный режим, зависящий от запуска target.

120.2. Сквозной порядок выполнения

После CLI проект всегда пытается прочитать файл, затем вызывает `pe_parse`, затем `unwind_read_runtime_functions_ex`, затем разбирает `UNWIND_INFO` для каждой записи. Код основной цепочки: `src/main.c:283-361`.

Heuristics запускаются только при `--heuristics`: `src/main.c:382-399`. Dynamic запускается только при `--dynamic`: `src/main.c:401-425`. Reports пишутся только по флагам export: `src/main.c:427-510`.

Зачем: это защищает проект от ложного обещания 'все всегда проверено'. Базовая модель PE/unwind строится всегда при обычном анализе, а findings/verdict создаются только когда пользователь явно включил heuristic layer.

121. Блок безопасного чтения файла

121.1. Как проект превращает файл в безопасный `Buf`

Файл открывается в binary mode, измеряется через `fseek/ftell`, полностью читается в память, затем сохраняется как `Buf { data, size }`. Код: `src/util.c:13-35`, структура `Buf`: `include/util.h:15-18`.

Зачем: все дальнейшие структуры PE читаются из одного стабильного массива байтов. Это важно для анализа поврежденных файлов: parser должен всегда знать размер входа и не зависеть от потокового чтения с меняющейся позицией.

121.2. Как защищается каждое чтение поля

Все чтения идут через `buf_read`, `buf_read_u16`, `buf_read_u32`, `buf_read_u64`. `buf_read` проверяет `off > size` и `len > size - off`, а только потом делает `memcpy`. Код: `src/util.c:42-70`.

Зачем: PE-файл может специально указывать смещения за пределы файла. Если читать напрямую, анализатор сам станет уязвимым. Здесь каждое поле читается только если bytes физически присутствуют в input file.

Смысловая гарантия следующему слою: если `pe.c` или `unwind.c` получили число через `buf_read_u32`, значит это число было прочитано из реально существующих 4 bytes, а не из памяти за пределами файла.

122. Блок PE container validation

122.1. DOS Header, PE Signature и COFF Header

Проект читает `MZ` и `e_lfanew`: `src/pe.c:11-19`, затем проверяет, что `e_lfanew + 4 + 20` помещается в файл: `src/pe.c:40-48`, затем читает PE signature: `src/pe.c:50-54`, затем COFF fields: `Machine`, `NumberOfSections`, `SizeOfOptionalHeader`: `src/pe.c:56-65`.

Зачем сравнивать magic и границы: без `MZ`/`PE` это не тот формат. Без проверки `e_lfanew` parser может прыгнуть по смещению, которое лежит за концом файла. Без корректного COFF header невозможно понять архитектуру и где начинается Optional Header.

122.2. Почему проект принимает только x64 PE32+

Проект сравнивает `Machine` с `IMAGE_FILE_MACHINE_AMD64` и `OptionalHeader.Magic` с `0x20b`. Код: `src/pe.c:67-90`, константы: `include/pe.h:18-19`.

Зачем: вся дальнейшая логика относится к x64 Exception Directory, `RUNTIME_FUNCTION` и Microsoft x64 `UNWIND_INFO`. Для PE32 формат exception/unwind другой или отсутствует в таком виде. Если не остановиться здесь, проект начнет интерпретировать чужой формат как x64 и получит бессмысленные findings.

122.3. Key Optional Header fields

Проект читает `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `SizeOfHeaders`, `NumberOfRvaAndSizes`: `src/pe.c:92-108`.

Зачем: это координатная система всего анализа. `SizeOfImage` задает виртуальную границу загруженного образа, `SizeOfHeaders` задает область заголовков, alignments объясняют допустимую раскладку секций, `NumberOfRvaAndSizes` говорит, можно ли вообще читать нужный Data Directory entry.

122.4. Доступность Exception Data Directory

Проект проверяет, что `NumberOfRvaAndSizes >= IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1`, затем проверяет, что сама 8-byte entry помещается в Optional Header. Код: `src/pe.c:121-130`, индекс directory: `include/pe.h:22`.

Зачем: нельзя читать `DataDirectory[3]`, если Optional Header сам говорит, что такого элемента нет или он выходит за `SizeOfOptionalHeader`. Это защита от чтения мусора внутри заголовка.

123. Блок построения PE image model

123.1. Как проект читает section table

После Optional Header проект вычисляет начало section headers, проверяет переполнение размера таблицы и ее попадание в файл: `src/pe.c:139-148`. Потом для каждой секции читает имя, `VirtualSize`, `VirtualAddress`, `SizeOfRawData`, `PointerToRawData`, `Characteristics`: `src/pe.c:156-172`.

Зачем: section table - мост между file layout и virtual image layout. Именно по ней проект позже понимает, где лежит code, где metadata, какие права у секции, и как переводить RVA в file offset.

123.2. Что сохраняется в `PeImage`

Модель `PeImage` хранит поля image, секции и counters structural validation: `section_raw_oob`, `section_virtual_oob`, `section_raw_overlaps`, `section_virtual_overlaps`, `section_misaligned`, `exception_directory_oob`. Структура: `include/pe.h:33-54`, заполнение: `src/pe.c:237-256`.

Зачем: дальше `heuristics.c` не перечитывает PE-заголовки. Он берет уже построенную модель и превращает counters в findings. Это разделяет parsing и decision layer: сначала факты, потом выводы.

124. Блок проверки секций

124.1. Как проверяется raw layout

Для каждой секции проект считает `raw_end = PointerToRawData + SizeOfRawData`. Если `SizeOfRawData > 0` и `raw_end > file_size`, увеличивается `raw_oob`. Код: `src/pe.c:198-204`.

Зачем сравнивать с размером файла: raw-диапазон - это физические bytes на диске. Если секция заявляет bytes за концом файла, parser не может безопасно читать эту область. Это становится finding `pe_section_raw_oob`: `src/heuristics.c:789-795`.

124.2. Как проверяется virtual layout

Проект считает `virtual_span = max(VirtualSize, SizeOfRawData)` и `virtual_end = VirtualAddress + virtual_span`. Если `virtual_end > SizeOfImage`, увеличивается `virtual_oob`. Код: `src/pe.c:200-204`.

Зачем сравнивать с `SizeOfImage`: loader рассматривает image как область `[0, SizeOfImage)`. Если секция выходит за эту область, виртуальная карта образа противоречива. Это становится finding `pe_section_virtual_oob`: `src/heuristics.c:796-802`.

124.3. Как проверяются пересечения section ranges

Проект сравнивает каждую секцию с предыдущими через `ranges_overlap_u64`. Отдельно проверяются raw-пересечения и virtual-пересечения. Код: функция overlap `src/pe.c:28-34`, цикл сравнений `src/pe.c:215-227`.

Зачем: если две секции накладываются на одни и те же bytes или одну и ту же virtual область, parser не может однозначно сказать, кому принадлежит RVA/offset. Raw overlap дает `pe_section_raw_overlap`, virtual overlap дает `pe_section_virtual_overlap`: `src/heuristics.c:803-816`.

124.4. Как проверяется alignment и порядок

Проект проверяет, что `FileAlignment` является допустимой степенью двойки или compact alignment, что `SectionAlignment >= FileAlignment`, что `SizeOfImage` кратен `SectionAlignment`, что `SizeOfHeaders` кратен `FileAlignment` и не меньше реально нужных заголовков, что `ImageBase` выровнен, что `VirtualAddress`, `PointerToRawData`, `SizeOfRawData` секций согласованы с alignments и что virtual addresses идут по возрастанию. Код: `src/pe.c:180-196`, `src/pe.c:205-213`.

Зачем: alignment - не косметика. Loader и tooling ожидают согласованную раскладку. Нарушение alignments часто означает повреждение, ручную сборку, packer layout или неправильный parser mode. Это дает finding `pe_section_misaligned`: `src/heuristics.c:817-824`.

125. Блок Data Directory и Exception Directory

125.1. Как читается `DataDirectory[Exception]`

Проект берет из Optional Header пару `VirtualAddress/Size` для Exception Directory: `src/pe.c:126-137`. Эти значения сохраняются как `img.exception_rva` и `img.exception_size`: `src/pe.c:245-246`.

Зачем: Exception Directory - входная точка в `.pdata`, где лежит x64 runtime function table. Если этот entry отсутствует или поврежден, дальше нельзя корректно построить карту функций.

125.2. Как проверяется пара `VirtualAddress/Size`

Если оба поля нулевые, каталог считается отсутствующим. Если одно поле нулевое, а другое нет, это malformed. Если оба ненулевые, проект проверяет конец `exc_rva + exc_size` относительно `SizeOfImage`. Код первичной проверки: `src/pe.c:230-235`; детальная проверка в reader: `src/unwind.c:623-629`.

Зачем: адрес без размера и размер без адреса не образуют валидную область. А выход `RVA+Size` за `SizeOfImage` означает, что directory указывает за пределы загруженного образа. Это ведет к `pe_exception_directory_oob`: `src/heuristics.c:825-832`.

125.3. Как malformed directory не ломает весь анализ

Если чтение Exception Directory возвращает malformed, `main` не падает как при OOM, а ставит `img.exception_directory_oob = true`, печатает сообщение и продолжает partial structural report. Код: `src/main.c:306-323`.

Зачем: даже если `.pdata` нельзя читать, PE headers и section validation все равно полезны. Проект показывает структурную проблему вместо того, чтобы просто завершиться без evidence.

126. Блок перевода координат `RVA -> file offset`

126.1. Как проект ищет секцию по RVA

`pe_find_section_for_rva` проходит по секциям и проверяет попадание RVA в virtual range `[VirtualAddress, VirtualAddress + max(VirtualSize, SizeOfRawData))`. Код: `src/pe.c:265-274`.

Зачем: большинство PE-указателей в headers являются RVA, а не file offset. Чтобы читать bytes из файла, нужно сначала понять, в какой секции находится RVA.

126.2. Как проект переводит RVA в физическое смещение

Если секция найдена, проект считает `delta = rva - Section.VirtualAddress`, проверяет, что `delta < SizeOfRawData`, потом считает `PointerToRawData + delta` через overflow-safe addition. Код: `src/pe.c:277-290`.

Зачем проверять `delta < raw_size`: часть секции может существовать только в памяти как zero-fill, но не иметь bytes в файле. Static parser не имеет права читать то, чего нет на диске.

Если RVA лежит внутри headers и физически помещается в файл, проект допускает прямой mapping `file offset = RVA`. Код: `src/pe.c:293-299`. Зачем: некоторые directory/data могут указывать в headers area, и это не надо автоматически считать секцией.

127. Блок чтения таблицы `RUNTIME_FUNCTION`

127.1. Как проект читает `.pdata` как массив записей

`unwind_read_runtime_functions_ex` проверяет отсутствие directory, malformed pair, mappability RVA, физическую читаемость диапазона, минимальный размер 12 bytes и кратность 12. Код: `src/unwind.c:611-647`.

Зачем размер 12: одна x64 `RUNTIME_FUNCTION` состоит из трех DWORD: `BeginAddress`, `EndAddress`, `UnwindInfoAddress`. Если directory size не кратен 12, это не корректный массив таких записей.

127.2. Как читается каждая запись

После вычисления `n = exception_size / 12` проект выделяет массив и для каждой записи читает три поля: begin, end, unwind. Код: `src/unwind.c:647-671`, структура `RuntimeFunction`: `include/unwind.h:16-20`.

Зачем: это карта функций образа. Каждая запись говорит: где начинается функция, где заканчивается и где лежит описание unwind. Следующие проверки уже работают не с raw bytes, а с этой картой.

128. Блок проверки RUNTIME_FUNCTION table integrity

128.1. Что проверяется в каждой runtime function entry

В цикле heuristics_analyze проект берет BeginAddress, EndAddress, UnwindInfoAddress каждой записи. Он проверяет EndAddress > BeginAddress, попадание begin/end в SizeOfImage, и что UnwindInfoAddress не равен 0 и не выходит за SizeOfImage. Код: src/heuristics.c:646-662.

Зачем: функция не может иметь отрицательную или нулевую длину, не должна выходить за границу image, а unwind pointer должен указывать на существующую область образа. Эти проверки дают findings invalid_ranges, rf_oob, unwind_rva_oob: src/heuristics.c:844-870.

128.2. Как проверяется порядок, overlap и duplicate begin

Проект сначала проверяет локальную сортировку по BeginAddress в исходном порядке таблицы: src/heuristics.c:664-670. Потом строит отдельный массив ranges, сортирует его и проверяет duplicate BeginAddress и пересечения через max_end: src/heuristics.c:740-773.

Зачем: Windows runtime lookup ожидает понятную function table. Если таблица не отсортирована, пересекается или имеет несколько функций с одинаковым началом, lookup/analysis может выбрать неправильную запись. Findings: rf_unsorted, rf_overlaps, duplicate_begin: src/heuristics.c:871-890.

128.3. Как проверяется отсутствие runtime functions

Если rf_count == 0, Exception Directory отсутствует и при этом в image есть executable section, проект создает finding no_runtime_functions. Код: проверка executable sections src/heuristics.c:134-144, finding src/heuristics.c:833-842.

Зачем: для x64 образа с исполняемым кодом полное отсутствие runtime function entries может быть нормальным только в ограниченных случаях, например leaf-only code. Поэтому проект не кричит 'вредоносно', а требует ручного подтверждения сложности кода.

129. Блок разбора UNWIND_INFO

129.1. Как проект находит и читает UNWIND_INFO

Для каждой runtime function main вызывает unwind_parse с rf[i].unwind_rva: src/main.c:351-361. Внутри parser сначала переводит UnwindInfoRVA в file offset, проверяет 4-byte header и читает его. Код: src/unwind.c:56-72, src/unwind.c:187-195.

Зачем: RUNTIME_FUNCTION только указывает на unwind metadata. Реальная проверка возможности unwinding начинается с того, что metadata физически доступна и имеет читаемый header.

129.2. Как проверяются version, flags, frame register и count

Parser извлекает version, flags, prolog_size, count_codes, frame_reg, frame_off: src/unwind.c:197-209. Затем допускает только version 1 или 2, проверяет неизвестные flags, конфликт CHAININFO с handler flags, frame offset без frame register и допустимость frame register. Код: src/unwind.c:211-229.

Зачем: эти поля задают интерпретацию всего хвоста структуры. Если version неизвестна или flags противоречат друг другу, parser может неправильно определить, где заканчивается codes array и где начинается handler/chain info.

129.3. Как проверяется массив unwind codes

Проект считает количество slots с выравниванием до четного числа, вычисляет codes_off и codes_bytes, проверяет, что массив помещается в файл. Код: src/unwind.c:231-239.

Дальше он идет по slots, читает каждый unwind slot, определяет opcode/opinfo, считает сколько slots занимает операция, проверяет, что операция не выходит за CountOfCodes, проверяет порядок CodeOffset и выход CodeOffset за prolog. Код: src/unwind.c:251-295.

Зачем: unwind opcodes являются компактным bytecode описания пролога. Если opcode занимает больше slots, чем осталось, parser не должен читать следующие bytes как операнды. Если code offset выходит за prolog или идет в

странном порядке, это признак поврежденного или нетипичного unwind описания.

129.4. Как проверяются операнды UWOP_*

Для каждой операции проект отдельно читает нужные operands и проверяет их смысл. Примеры: UWOP_ALLOC_LARGE требует 1 или 2 дополнительных slots и проверяет подозрительные размеры; SAVE_NONVOL, SAVE_XMM, SAVE_XMM128 проверяют наличие operand slots, регистры и offsets; PUSH_MACHFRAME проверяет opinfo. Код: src/unwind.c:346-510; расчет slot count: src/unwind.c:24-54; sanity helpers: src/unwind.c:113-128.

Зачем: это не просто синтаксис. Эти операции описывают, как восстановить stack/register state. Если operands обрезаны или абсурдны, virtual unwind может восстановить неверный кадр или вообще оборваться.

129.5. Как parse failures и anomalies становятся findings

Если unwind_parse возвращает false, main сохраняет us_ok[i] = false: src/main.c:351-356, а heuristics.c считает это как unwind_parse_fail: src/heuristics.c:727-729. Если parse успешен, но anomalies содержит не только ABI warnings, это считается unwind_anomalies: src/heuristics.c:730-735.

Findings создаются в src/heuristics.c:893-906. Зачем разделять fail и anomalies: fail означает, что metadata нельзя надежно разобрать; anomalies означает, что структура читается, но внутри есть странные поля, комбинации или encoding-patterns.

130. Блок handler layer и chained unwind

130.1. Как проект проверяет handler flags

Если flags содержат UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, parser читает 4-byte handler_rva после unwind codes. Если bytes не читаются, это parse fail. Если handler_rva >= SizeOfImage, ставится anomaly UA_HANDLER_OOB. Код: src/unwind.c:540-552, флаги: include/unwind.h:45-49.

Зачем: handler - это код/точка обработки исключения. Если handler RVA не помещается в image, Windows не сможет корректно перейти к нему как к части загруженного образа.

130.2. Как проект проверяет UNW_FLAG_CHAININFO

Если flags содержат CHAININFO, parser читает вложенную RUNTIME_FUNCTION из хвоста UNWIND_INFO, проверяет end > begin, попадание begin/end/unwind в SizeOfImage и наличие unwind_rva. Код: src/unwind.c:522-538, helper parse_chain_rf: src/unwind.c:130-140.

Зачем: chained unwind связывает одну unwind-запись с другой. Если chain битая, попытка follow-chain может уйти в неверную функцию, за границы image или в loop.

130.3. Как проект ограничивает follow-chain

Если follow_chain включен, parser идет по chain до max_chain_depth, запоминает уже виденные unwind RVA, ловит loop, объединяет stack/register effects и останавливается при превышении глубины. Код: src/unwind.c:559-608, CLI-limit --max-chain-depth: src/main.c:212-219.

Зачем: malicious или поврежденный файл может сделать chain-loop. Ограничение глубины и список seen не дают parser-у зависнуть или бесконечно ходить по одной цепочке.

131. Блок metadata placement и прав секций

131.1. Как проект понимает, где лежит UNWIND_INFO

Для каждой успешно разобранной записи проект ищет секцию по UnwindInfoRVA через pe_find_section_for_rva. Если секция не найдена, или имя секции не похоже на .xdata/.rdata/.pdata, счетчик unwind_unusual_section растет. Код: src/heuristics.c:690-704; allowed/plausible names: src/heuristics.c:120-126.

Зачем: unwind metadata обычно является служебными read-only данными. Если она лежит в неизвестной или code-like области, это может быть packer layout, manual mapping, interceptor metadata или повреждение section mapping. Finding: unwind_unusual_section: src/heuristics.c:972-978.

131.2. Как проект проверяет execute/write права для UNWIND_INFO

После нахождения секции проект проверяет bits IMAGE_SCN_MEM_EXECUTE и IMAGE_SCN_MEM_WRITE в Characteristics. Если metadata лежит в executable section, растет unwind_exec_section; если в writable, растет unwind_writable_section. Код: src/heuristics.c:704-710; findings: src/heuristics.c:965-986.

Зачем: служебная unwind metadata обычно не должна исполняться и не должна изменяться во время обычной работы. Execute повышает риск смешения metadata/code, write повышает риск runtime-patching. Это не автоматический приговор, но сильный повод для ручной проверки.

131.3. Как проект проверяет placement handler target

Если UNWIND_INFO имеет handler, проект ищет секцию по handler_rva. Если секции нет или она не executable, растет handler_nonexec_section. Если секция writable, растет handler_writable_section. Код: src/heuristics.c:714-723; findings: src/heuristics.c:988-1000.

Зачем: handler должен быть исполняемым кодом. Handler в non-executable/unknown области означает несогласованность metadata и memory protections. Handler в writable области может быть легитимен в JIT/runtime-сценарии, но для обычного PE это нетипично.

132. Блок reuse и dominance

132.1. Как проект считает повторное использование UNWIND_INFO

Внутри heuristics_analyze создается hash map unwind_counts, куда для каждой RUNTIME_FUNCTION добавляется ее UnwindInfoRVA. Код hash map: src/heuristics.c:146-170, инкремент в цикле: src/heuristics.c:671-677, вычисление max/unique/ratio: src/heuristics.c:775-780.

Зачем: если слишком много функций указывают на одну и ту же unwind-запись, это может быть нормальной оптимизацией для простых leaf/stub функций, а может быть centralized dispatcher/obfuscation pattern. Поэтому проект смотрит не на один reuse, а на долю и контекст.

132.2. Как проект отличает moderate и strong reuse

Strong reuse срабатывает, когда unwind_reuse_ratio > 0.30 и n > 200; moderate candidate - когда ratio > 0.10 и n > 200, но finding unwind_reuse_moderate создается только если есть другой статический сигнал. Код thresholds: src/heuristics.c:82-83; logic: src/heuristics.c:908-918, src/heuristics.c:946-963.

Зачем нужны n > 200 и корреляция: на маленькой таблице проценты легко обманывают. Если 3 из 10 функций используют одно metadata, это 30%, но статистически слабый сигнал. На большом sample reuse становится осмысленным.

132.3. Как проект считает handler dominance

Проект считает handler_counts только по успешно разобранным UNWIND_INFO, где есть handler. Затем вычисляет handler_reuse_ratio = max_handler_count / handler_count. Finding создается при handler_count > 50 и handler_reuse_ratio > 0.80, но с защитой от доброкачественного случая, когда dominant handler лежит в обычной executable code-like section и нет другого контекста. Код: src/heuristics.c:679-688, src/heuristics.c:782-785, src/heuristics.c:920-943.

Зачем: один общий handler может быть нормальным для runtime/SEH patterns, но если почти все handler-ссылки сходятся в одну точку и есть сопутствующие странности, это похоже на central dispatcher.

133. Блок correlation/combo findings

133.1. Почему проект проверяет комбинации, а не только одиночные признаки

Один признак часто можно объяснить benign-особенностью: необычная секция, small sample, нестандартный linker. Но когда разные слои подтверждают друг друга, вероятность случайности ниже. Поэтому проект создает combo findings, если странности совпадают в нескольких слоях.

Код combo rules: placement_exec_nonexec_handler_combo, placement_integrity_combo, reuse_dominance_combo, writable_metadata_handler_combo находятся в src/heuristics.c:1003-1022.

133.2. Смысл каждой корреляции

`placement_exec_nonexec_handler_combo`: unwind metadata в executable section совпадает с handler target в non-executable/unknown section. Смысл: сразу две роли секций выглядят перевернутыми.

`placement_integrity_combo`: странное размещение metadata подтверждается parse/anomaly/OOB проблемами. Смысл: это не просто необычное имя секции, а еще и структурные проблемы.

`reuse_dominance_combo`: сильное reuse metadata совпадает с dominant handler. Смысл: похоже на централизованную dispatcher-style схему.

`writable_metadata_handler_combo`: writable unwind metadata совпадает с writable handler targets. Смысл: изменяемость обнаружена сразу в metadata и control-flow target.

134. Как именно рождаются 27 findings

Ниже - не просто список признаков, а карта: какое условие проверяется, зачем оно нужно и где из него создается finding. Все 27 static findings создаются в `src/heuristics.c` через `add_finding`: `src/heuristics.c:408-430`, конкретные вызовы `src/heuristics.c:789-1022`.

1. `pe_section_raw_oob`

Как проверяется: `pe.c` считает raw end секции и сравнивает с file size; если raw bytes выходят за входной файл, counter переносится в heuristics.

Зачем это проверяется: Без этого parser может считать, что секция существует на диске, хотя bytes за концом файла. Это ломает file-offset reading.

Где в коде: `src/pe.c:198-204`, `src/heuristics.c:789-795`.

2. `pe_section_virtual_oob`

Как проверяется: Проект считает virtual end секции и сравнивает с `SizeOfImage`.

Зачем это проверяется: Если virtual range выходит за image, RVA-карта образа противоречива, а loader/analyzer не должен считать эту область нормальной частью image.

Где в коде: `src/pe.c:200-204`, `src/heuristics.c:796-802`.

3. `pe_section_raw_overlap`

Как проверяется: Каждая секция сравнивается с предыдущими по raw ranges.

Зачем это проверяется: Две секции не должны претендовать на одни и те же bytes файла: это делает принадлежность offset неоднозначной.

Где в коде: `src/pe.c:215-219`, `src/heuristics.c:803-809`.

4. `pe_section_virtual_overlap`

Как проверяется: Каждая секция сравнивается с предыдущими по virtual ranges.

Зачем это проверяется: Две секции не должны перекрывать одну RVA-область, иначе `RVA -> section` становится неоднозначным.

Где в коде: `src/pe.c:220-226`, `src/heuristics.c:810-816`.

5. `pe_section_misaligned`

Как проверяется: Проверяются `FileAlignment`, `SectionAlignment`, `SizeOfImage`, `SizeOfHeaders`, `ImageBase`, выравнивание секций и порядок `VirtualAddress`.

Зачем это проверяется: Эти проверки показывают, согласована ли PE-раскладка с правилами loader-а. Нарушения часто означают нестандартный/поврежденный layout.

Где в коде: `src/pe.c:180-196`, `src/pe.c:205-213`, `src/heuristics.c:817-824`.

6. `pe_exception_directory_oob`

Как проверяется: Проверяется доступность Data Directory entry, пара `VirtualAddress/Size`, попадание directory в `SizeOfImage`, mappability и читаемость `.pdata`, размер не меньше 12 и кратность 12.

Зачем это проверяется: Exception Directory - вход в function table. Если он поврежден, дальше нельзя доверять карте функций.

Где в коде: `src/pe.c:121-137`, `src/pe.c:230-235`, `src/unwind.c:623-647`, `src/main.c:317-323`, `src/heuristics.c:825-832`.

7. `no_runtime_functions`

Как проверяется: Если Exception Directory отсутствует, `rf_count == 0`, но есть executable section, проект создает warning-level finding.

Зачем это проверяется: Для x64 executable image отсутствие runtime functions может быть допустимо только для очень простого leaf-only кода; поэтому это не жесткая ошибка, а повод проверить код вручную.

Где в коде: `src/heuristics.c:134-144`, `src/heuristics.c:833-842`.

8. `invalid_ranges`

Как проверяется: Для каждой `RUNTIME_FUNCTION` проверяется `EndAddress > BeginAddress`.

Зачем это проверяется: Функция не может заканчиваться до начала или иметь нулевой диапазон; такая запись ломает function table semantics.

Где в коде: `src/heuristics.c:646-654`, `src/heuristics.c:850-856`.

9. `rf_oob`

Как проверяется: Проверяется, что `BeginAddress` и `EndAddress` лежат внутри `SizeOfImage`.

Зачем это проверяется: Function range должен описывать код внутри загруженного image. Выход за image означает неверную координатную систему или повреждение таблицы.

Где в коде: `src/heuristics.c:655-658`, `src/heuristics.c:857-863`.

10. `unwind_rva_oob`

Как проверяется: Проверяется, что `UnwindInfoAddress` не равен 0 и меньше `SizeOfImage`.

Зачем это проверяется: Function table без валидного указателя на `UNWIND_INFO` не дает Windows корректно выйти из кадра.

Где в коде: `src/heuristics.c:659-662`, `src/heuristics.c:864-870`.

11. `rf_unsorted`

Как проверяется: Проверяется, что begin addresses в исходной таблице не идут назад.

Зачем это проверяется: Function table обычно должна быть отсортирована для корректного lookup. Несортированность затрудняет runtime search и анализ.

Где в коде: `src/heuristics.c:664-670`, `src/heuristics.c:871-877`.

12. `rf_overlaps`

Как проверяется: Проект сортирует ranges и проверяет, не начинается ли следующий range раньше текущего максимального конца.

Зачем это проверяется: Один RVA кода не должен одновременно принадлежать нескольким функциям; overlap делает stack/unwind attribution неоднозначным.

Где в коде: `src/heuristics.c:740-773`, `src/heuristics.c:878-884`.

13. `duplicate_begin`

Как проверяется: После сортировки проверяются одинаковые `BeginAddress` у разных записей.

Зачем это проверяется: Две функции с одним началом конфликтуют как записи function table; runtime lookup и analyzer могут выбрать не ту запись.

Где в коде: `src/heuristics.c:754-762`, `src/heuristics.c:885-891`.

14. `unwind_parse_fail`

Как проверяется: Если `unwind_parse` не смог прочитать header/codes/operands/tail, `us_ok=false`, а heuristics считает parse fail.

Зачем это проверяется: Это означает, что metadata физически или синтаксически неразбираема, поэтому корректный unwind для этой функции не гарантирован.

Где в коде: `src/main.c:351-356`, `src/unwind.c:56-72`, `src/unwind.c:165-608`, `src/heuristics.c:727-729`, `src/heuristics.c:893-899`.

15. unwind_anomalies

Как проверяется: Если UNWIND_INFO читается, но внутри есть неизвестная version, bad opcode, bad operand, code offset outside prolog, chain loop/depth, handler OOB и другие non-ABI anomalies, счетчик растет.

Зачем это проверяется: Это показывает, что структура формально доступна, но ее логика странная или противоречит ожидаемой unwind-модели.

Где в коде: include/unwind.h:51-70, src/unwind.c:211-239, src/unwind.c:251-295, src/unwind.c:346-552, src/heuristics.c:730-735, src/heuristics.c:900-906.

16. unwind_reuse_strong

Как проверяется: Считается максимальное число функций, указывающих на один UnwindInfoRVA; finding срабатывает при ratio > 0.30 и n > 200.

Зачем это проверяется: На большой таблице массовое повторное использование одной unwind-записи может указывать на централизованную metadata схему, stubs или obfuscation pattern.

Где в коде: src/heuristics.c:671-677, src/heuristics.c:775-780, src/heuristics.c:910-918.

17. handler_dominance

Как проверяется: Считается, сколько handler-ссылок указывает на один HandlerRVA; finding срабатывает при handler_count > 50 и ratio > 0.80, если это не выглядит как benign runtime handler без другого контекста.

Зачем это проверяется: Один handler, обслуживающий почти все функции, может быть централизованным dispatcher-ом или interception point.

Где в коде: src/heuristics.c:679-688, src/heuristics.c:782-785, src/heuristics.c:920-943.

18. unwind_reuse_moderate

Как проверяется: Moderate reuse candidate появляется при ratio > 0.10 и n > 200, но finding создается только если есть другой статический сигнал.

Зачем это проверяется: Умеренный reuse сам по себе может быть нормальным, поэтому проект требует корреляцию с другими проблемами, чтобы не завышать вывод.

Где в коде: src/heuristics.c:908-909, src/heuristics.c:946-963.

19. unwind_in_exec_section

Как проверяется: UnwindInfoRVA сопоставляется с секцией; если у секции есть execute bit, счетчик растет.

Зачем это проверяется: Unwind metadata обычно должна быть read-only data, а не исполняемым кодом. Execute placement повышает подозрительность metadata layout.

Где в коде: src/heuristics.c:690-710, src/heuristics.c:965-971.

20. unwind_unusual_section

Как проверяется: Секция UNWIND_INFO сравнивается с ожидаемыми/похожими именами .xdata, .rdata, .pdata; отсутствие секции тоже считается unusual.

Зачем это проверяется: Необычное размещение не всегда ошибка, но оно требует объяснения, потому что нормальный toolchain обычно кладет unwind metadata в data/metadata sections.

Где в коде: src/heuristics.c:120-126, src/heuristics.c:690-704, src/heuristics.c:972-978.

21. unwind_writable_section

Как проверяется: Проверяется write bit секции, где лежит UNWIND_INFO.

Зачем это проверяется: Writable metadata может меняться во время выполнения. Для обычного PE это нетипично и требует проверки на JIT/runtime/packer/manual patching.

Где в коде: src/heuristics.c:708-710, src/heuristics.c:980-986.

22. handler_nonexec_section

Как проверяется: HandlerRVA сопоставляется с секцией; если секции нет или она не executable, счетчик растет.

Зачем это проверяется: Handler должен быть исполняемой точкой обработки. Non-executable/unknown target противоречит control-flow смыслу handler-а.

Где в коде: `src/heuristics.c:714-719`, `src/heuristics.c:988-994`.

23. `handler_writable_section`

Как проверяется: Проверяется write bit секции, куда указывает handler.

Зачем это проверяется: Handler в writable section может означать runtime-generated/modified code. Это не автоматическая вредоносность, но сильный manual-review повод.

Где в коде: `src/heuristics.c:720-723`, `src/heuristics.c:995-1000`.

24. `placement_exec_nonexec_handler_combo`

Как проверяется: Создается, когда `UNWIND_INFO` лежит в executable section, а handler target находится в non-executable/unknown section.

Зачем это проверяется: Комбинация показывает не одну странность, а переворот ролей сразу в metadata и handler layer.

Где в коде: `src/heuristics.c:1003-1007`.

25. `placement_integrity_combo`

Как проверяется: Создается, когда suspicious placement совпадает с parse/anomaly/unwind-RVA integrity проблемами.

Зачем это проверяется: Если странное размещение подтверждается повреждением структуры, общий вывод сильнее, чем каждый признак отдельно.

Где в коде: `src/heuristics.c:1008-1012`.

26. `reuse_dominance_combo`

Как проверяется: Создается, когда heavy `UNWIND_INFO` reuse совпадает с `handler_dominance`.

Зачем это проверяется: Смысл: metadata reuse и dominant handler вместе похожи на centralized dispatcher-style схему.

Где в коде: `src/heuristics.c:1014-1018`.

27. `writable_metadata_handler_combo`

Как проверяется: Создается, когда writable unwind metadata совпадает с writable handler targets.

Зачем это проверяется: Одновременная изменяемость metadata и handler target нетипична для обычного compiler-produced image и требует внимательной проверки.

Где в коде: `src/heuristics.c:1019-1022`.

135. Блок score, confidence и verdict

135.1. Как проект присваивает веса

Веса заданы в `src/heuristics.c:19-64`. В комментарии прямо указано, что это conservative engineering model, а не математически обученный classifier. Жесткие нарушения table/unwind integrity получают больше баллов, placement - medium/high, reuse/dominance - moderate unless strong, combo - дополнительное усиление.

Зачем нужны веса: пользователю нужен не только список признаков, но и приоритизация. `pe_exception_directory_oob`, `rf_overlaps`, `rf_oob`, `unwind_parse_fail` сильнее, потому что ломают саму возможность доверять карте функций или unwind. `pe_section_misaligned` слабее, потому что может быть особенностью layout и требует контекста.

135.2. Как thresholds превращают единичную ошибку в broad issue

`THRESHOLD_HIGH_PERCENT = 0.01` означает: больше 1% entries - уже не единичный glitch. `THRESHOLD_UNWIND_ANOM_HIGH = 0.05` означает: больше 5% anomalies - broad integrity issue. Reuse thresholds: 10% moderate, 30% strong. Handler dominance: 80%. Код: `src/heuristics.c:80-84`.

Зачем сравнивать проценты: одна битая запись может быть случайной особенностью или повреждением одного range. Массовый процент показывает системный паттерн.

135.3. Как строится confidence и verdict

`finalize_verdict` считает family count, strong/medium/weak findings, добавляет confidence за разные evidence families и hard table/unwind issues, снижает confidence за single signal, small sample, placement-only и single-family evidence. Код: `src/heuristics.c:514-573`.

Вердикт задается по score: `>=70 high_suspicion`, `>=40 moderate_suspicion`, `>=15 low_suspicion`, иначе `no_strong_static_anomalies`. Manual review включается при score >= 20 или при важных placement/parse signals. Код: `src/heuristics.c:575-587`.

Зачем: проект не должен выдавать 'опасно' только по одной слабой странности. Confidence специально понижается, если evidence мало или оно односемейное.

136. Блок evidence и отчетов

136.1. Как проект проверяет, что отчет можно писать

Перед JSON/HTML проект собирает metadata: tool version, schema, timestamp, SHA-256 input, input size, command line, compiler/platform/build mode, strict ABI/follow chain/max depth. Код: `src/main.c:450-480`.

Report layer валидирует metadata: schema version, SHA-256 length/hex, timestamp format, build mode, max chain depth. Код: `src/report.c:556-593`. Также валидируется модель: sections pointer, limits findings/mitigations/events/trace and instrumentation consistency: `src/report.c:596-620`.

Зачем: отчет должен быть доказательным артефактом. Без hash, времени, версии schema и параметров запуска невозможно воспроизвести анализ и защитить вывод.

136.2. Как evidence попадает в JSON/HTML

JSON выводит image fields, Exception Directory, PE validation counters, sections с правами R/W/X, heuristic counters, findings с code/family/severity/message/affected_count/RVA/section, dynamic events, assessment и functions table. Код: `src/report.c:622-920`.

HTML выводит summary cards, PE image, structural validation, sections table, assessment, findings, mitigation rules, dynamic check и full `RUNTIME_FUNCTION` table. Код: `src/report.c:1151-1520`.

Зачем: пользователь должен видеть не только финальный verdict, но и цепочку доказательств: какое поле прочитано, какой счетчик сработал, какая секция затронута и какой RVA является представителем проблемы.

137. Динамическая проверка: что она делает и чего не делает

137.1. Динамика не заменяет 27 static findings

27 findings создаются static layer в `src/heuristics.c`. Dynamic mode не добавляет эти 27 codes, а собирает runtime evidence: exception events, handler hits, VEH/VCH/UEF registrations/hits, dynamic function table registrations и trace после handler hit. Структура `DynamicReport`: `include/dynamic.h:41-90`.

Зачем: static analysis отвечает 'структура подозрительна или повреждена', dynamic analysis отвечает 'участвовали ли эти механизмы в реальном запуске'.

137.2. Как dynamic mode использует static parse

Перед запуском target dynamic layer собирает уникальные handler RVA из уже разобранных `UnwindSummary[]`: `src/dynamic_win.c:726-746`. При `CREATE_PROCESS_DEBUG_EVENT` получает base address target и ставит breakpoints на `base_main + handler_rva`: `src/dynamic_win.c:789-808`.

Зачем: статический handler RVA сам по себе не доказывает execution. Breakpoint показывает, был ли этот handler реально достигнут во время конкретного запуска.

137.3. Как dynamic mode следит за exception machinery

Dynamic layer запускает process под Windows Debug API через `CreateProcessA` с `DEBUG_ONLY_THIS_PROCESS`: `src/dynamic_win.c:657-715`, затем обрабатывает debug events, modules, exceptions, breakpoints и timeout: `src/dynamic_win.c:768-850`.

Он ставит API breakpoints на `AddVectoredExceptionHandler`, `AddVectoredContinueHandler`, `SetUnhandledExceptionFilter`, `RtlAddFunctionTable`, `RtlInstallFunctionTableCallback`:

`src/dynamic_win.c:561-599`. При hits читает register arguments из thread context и записывает events: `src/dynamic_win.c:902-984`.

Зачем: эти API показывают runtime exception redirection и runtime function table registration, которые static `.pdata` не всегда раскрывает полностью.

137.4. Как dynamic evidence влияет на итоговый assessment

`report.c` считает dynamic signals: handler hits, VEH/VCH/UEF, runtime function table calls, second-chance exceptions. Код: `src/report.c:326-335`. Complete session требует ran/ok/no timeout/target exited/instrumentation complete: `src/report.c:338-345`.

Static-dynamic correlation возникает, если suspicious static handler context совпал с handler hits или suspicious placement совпал с runtime function table calls. Код: `src/report.c:347-358`. Итоговый assessment повышает confidence за complete dynamic session и за correlation: `src/report.c:405-440`.

Зачем: динамика не говорит 'все чисто', если путь не исполнился. Она только усиливает вывод, когда runtime events подтверждают static подозрения.

138. Как отвечать комиссии коротко и точно

Проект проверяет бинарник как многоуровневый PE/unwind analyzer. Сначала он безопасно читает файл и проверяет, что это PE32+ x64. Затем строит PE image model, проверяет section raw/virtual layout, alignments и Exception Directory. После этого переводит RVA в file offsets, читает `.pdata` как массив `RUNTIME_FUNCTION`, проверяет диапазоны функций, сортировку, overlap и pointers на `UNWIND_INFO`. Потом разбирает каждую `UNWIND_INFO`: version, flags, count codes, opcodes, operands, handler data и chain info. Затем сопоставляет metadata и handler targets с секциями и правами R/W/X, считает reuse/dominance, строит combo correlations, score, confidence и verdict. 27 findings - это не отдельные случайные проверки, а результат этой цепочки evidence.

Главная защитная формула: 'Каждая проверка нужна не сама по себе, а чтобы защитить следующий слой анализа. Если не проверить границы файла, нельзя безопасно читать PE. Если не проверить `SizeOfImage`, нельзя доверять RVA. Если не проверить section table, нельзя переводить RVA в file offset. Если не проверить `.pdata`, нельзя доверять карте функций. Если не разобрать `UNWIND_INFO`, нельзя оценивать handler layer. Если не сопоставить metadata с секциями и правами, нельзя понять, нормальная это служебная информация или подозрительное размещение.'

МИКРОМАТРИЦА ПРОВЕРОК И ПОСЛЕДСТВИЙ

Этот раздел уточняет предыдущую схему. Здесь каждая важная проверка расписана в формате: как именно проверяется, зачем проверяется, что происходит при провале и какой finding/ошибка из этого возникает. Важно: не все проверки дают один из 27 findings. Ранние проверки PE-формата являются hard-stop gates: если они не пройдены, модель `PeImage` не построена, поэтому heuristic findings еще не могут быть корректно созданы.

139. Три типа результата проверки

139.1. Hard stop до модели PE

Как именно проверяется: Проверка выполняется до того, как проект построил `PeImage`: чтение файла, `MZ`, `PE`, AMD64, PE32+, Optional Header, section headers.

Для чего проверяется: Пока нет корректной PE-модели, нельзя честно создавать findings про `.pdata`, `RUNTIME_FUNCTION` и `UNWIND_INFO`, потому что еще нет координатной системы.

Если не проходит: Программа печатает ошибку и завершает анализ с non-zero exit code. Это не один из 27 findings, а отказ от анализа неподходящего/битого входа.

Код: `src/main.c:283-299`, `src/pe.c:36-148`.

139.2. Structural counter внутри `PeImage`

Как именно проверяется: Проверка проходит после чтения PE headers и section table: raw/virtual OOB, overlaps, misalignment, Exception Directory OOB.

Для чего проверяется: На этом уровне модель PE уже есть, поэтому проект может сохранить численный факт нарушения в `PeImage`.

Если не проходит: При `--heuristics` эти counters превращаются в findings семейства `table_integrity`: `pe_section_*`, `pe_exception_directory_oob`.

Код: `include/pe.h:48-53`, `src/pe.c:174-235`, `src/heuristics.c:789-832`.

139.3. Heuristic finding после построения unwind-модели

Как именно проверяется: Проверка выполняется над `RuntimeFunction[]`, `UnwindSummary[]`, `us_ok[]` и section rights.

Для чего проверяется: Здесь уже известны функции, unwind metadata, handler targets и секции, поэтому можно делать выводы о целостности и подозрительности.

Если не проходит: При провале создается один из 27 findings, добавляется вес, severity, family, affected_count и representative RVA.

Код: `src/heuristics.c:408-430`, `src/heuristics.c:590-1038`.

140. Ранние hard-stop проверки

140.1. Файл должен быть прочитан полностью

Как именно проверяется: `buf_init_from_file` открывает файл в binary mode, измеряет размер, выделяет память и читает ровно `size` bytes.

Для чего проверяется: Без полного byte buffer нельзя безопасно проверять смещения и границы структур.

Если не проходит: Если открыть/прочитать/выделить память не удалось, анализ прекращается. Finding не создается, потому что нет входных bytes.

Код: `src/util.c:13-35`, `src/main.c:283-288`.

140.2. Любое поле читается только если bytes существуют

Как именно проверяется: `buf_read` проверяет `off <= size` и `len <= size - off`, затем копирует bytes; `buf_read_u16/u32/u64` строят little-endian числа через `buf_read`.

Для чего проверяется: Это базовая защита parser-а от выхода за границы файла при злонамеренных offsets.

Если не проходит: Если bytes не помещаются, конкретный caller получает false и либо hard-stop, либо parse fail, либо malformed directory в зависимости от слоя.

Код: `src/util.c:42-70`.

140.3. DOS magic `MZ` и `e_lfanew`

Как именно проверяется: Проект читает word по offset 0 и сравнивает с `PE_DOS_MAGIC`, затем читает `e_lfanew` по offset `0x3c`.

Для чего проверяется: Это доказывает, что файл хотя бы начинает соответствовать PE container, и дает смещение к NT headers.

Если не проходит: Если `MZ` или `e_lfanew` не читаются, `pe_parse` возвращает false. Finding не создается: PE-модель еще не существует.

Код: `src/pe.c:11-19`, `src/pe.c:40-44`.

140.4. `e_lfanew + PE header` должен помещаться в файл

Как именно проверяется: Проверяется, что `(size_t)lfanew + 4 + 20 <= file_size`, то есть есть место на PE signature и COFF header.

Для чего проверяется: Без этой проверки parser мог бы перейти по `e_lfanew` за пределы файла.

Если не проходит: Hard stop: ошибка `PE header offset out of range`, findings не создаются.

Код: `src/pe.c:45-48`.

140.5. PE signature должна быть `PE\0\0`

Как именно проверяется: По `e_lfanew` читается DWORD и сравнивается с `PE_NT_MAGIC`.

Для чего проверяется: Это отделяет настоящий NT headers от произвольных bytes по указанному offset.

Если не проходит: Hard stop: `Invalid PE signature`, findings не создаются.

Код: `src/pe.c:50-54`, `include/pe.h:15-16`.

140.6. COFF header должен читаться

Как именно проверяется: Из COFF header читаются `Machine`, `NumberOfSections`, `SizeOfOptionalHeader`.

Для чего проверяется: Эти поля задают архитектуру, размер Optional Header и длину section table.

Если не проходит: Если поля не читаются, hard stop: parser не знает, как двигаться дальше.

Код: `src/pe.c:56-65`.

140.7. Архитектура должна быть AMD64

Как именно проверяется: `Machine` сравнивается с `IMAGE_FILE_MACHINE_AMD64`.

Для чего проверяется: Проект анализирует Microsoft x64 unwind. На другой архитектуре `.pdata`/unwind могут иметь другой смысл.

Если не проходит: Hard stop: `Unsupported Machine`; это не finding, а неподдерживаемый формат.

Код: `src/pe.c:67-70`, `include/pe.h:18`.

140.8. Optional Header должен помещаться в файл и иметь нужный минимум

Как именно проверяется: Проверяется `opthdr + opt_size <= file_size` и `opt_size >= 0x70`.

Для чего проверяется: Нужные поля PE32+ и Data Directory нельзя читать, если Optional Header короче или выходит за файл.

Если не проходит: Hard stop: `Optional header out of range` или `too small`, findings не создаются.

Код: `src/pe.c:72-80`.

140.9. `OptionalHeader.Magic` должен быть `0x20b`

Как именно проверяется: Читается magic по началу Optional Header и сравнивается с `IMAGE_NT_OPTIONAL_HDR64_MAGIC`.

Для чего проверяется: Это подтверждает PE32+ x64 layout, где offsets полей соответствуют текущему parser-у.

Если не проходит: Hard stop: PE32 или другой optional header не анализируется как x64 unwind.

Код: `src/pe.c:82-90`, `include/pe.h:19`.

140.10. Ключевые поля Optional Header должны читаться

Как именно проверяется: Читаются `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `SizeOfHeaders`, `NumberOfRvaAndSizes`.

Для чего проверяется: Эти поля задают все дальнейшие границы и координаты анализа.

Если не проходит: Если одно поле не читается, hard stop: parser не может строить корректную PE-модель.

Код: src/pe.c:92-108.

140.11. SizeOfImage, SizeOfHeaders, alignments должны иметь базовый смысл

Как именно проверяется: Проверяется, что SizeOfImage != 0, SizeOfHeaders != 0, SizeOfHeaders <= file_size, SectionAlignment != 0, FileAlignment != 0.

Для чего проверяется: Нулевой image или alignment ломает все сравнения RVA и выравниваний; headers за пределами файла нельзя читать.

Если не проходит: Hard stop: модель PE не строится.

Код: src/pe.c:109-120.

140.12. Data Directory entry должна быть доступна

Как именно проверяется: Проверяется NumberOfRvaAndSizes >= IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1 и dd_rel + 8 <= opt_size.

Для чего проверяется: Parser не имеет права читать DataDirectory[Exception], если Optional Header сам не содержит такую entry.

Если не проходит: Hard stop: сейчас это не finding, потому что Exception Directory entry недоступна на уровне header structure.

Код: src/pe.c:121-130, include/pe.h:22.

140.13. Section headers должны помещаться в файл

Как именно проверяется: Вычисляется secthdr + NumberOfSections * 40, проверяется integer overflow и попадание в file size.

Для чего проверяется: Section table нужна для всех последующих RVA -> section -> file offset переводов.

Если не проходит: Hard stop: если таблица секций не читается, проект не может безопасно анализировать .pdata и metadata placement.

Код: src/pe.c:139-148.

141. PE structural checks, которые дают PE findings

141.1. Raw range секции не должен выходить за файл

Как именно проверяется: Для каждой секции считается raw_end = raw_ptr + raw_size; если raw_size > 0 и raw_end > file_size, растет section_raw_oob.

Для чего проверяется: Raw range - это физические bytes на диске. Если они выходят за файл, чтение секции невозможно.

Если не проходит: Finding при --heuristics: pe_section_raw_oob, severity high, score +25.

Код: src/pe.c:198-204, src/heuristics.c:789-795, score src/heuristics.c:32.

141.2. Virtual range секции не должен выходить за SizeOfImage

Как именно проверяется: Считается virtual_span = max(VirtualSize, SizeOfRawData) и virtual_end = VirtualAddress + virtual_span; если конец больше SizeOfImage, растет section_virtual_oob.

Для чего проверяется: SizeOfImage задает виртуальную область загруженного image. Секция за этой границей нарушает карту RVA.

Если не проходит: Finding: pe_section_virtual_oob, severity high, score +20.

Код: src/pe.c:200-204, src/heuristics.c:796-802, score src/heuristics.c:33.

141.3. Raw ranges секций не должны пересекаться

Как именно проверяется: Каждая секция сравнивается с предыдущими через ranges_overlap_u64 по raw_ptr/raw_size.

Для чего проверяется: Если две секции указывают на одни bytes файла, file offset принадлежит сразу двум логическим секциям.

Если не проходит: Finding: pe_section_raw_overlap, severity medium, score +15.

Код: src/pe.c:28-34, src/pe.c:215-219, src/heuristics.c:803-809.

141.4. Virtual ranges секций не должны пересекаться

Как именно проверяется: Секции сравниваются по VirtualAddress и max(VirtualSize, SizeOfRawData).

Для чего проверяется: Один RVA не должен принадлежать двум секциям, иначе placement и права секции становятся неоднозначными.

Если не проходит: Finding: `pe_section_virtual_overlap`, severity high, score +20.

Код: `src/pe.c:220-226`, `src/heuristics.c:810-816`.

141.5. Alignment/header invariants должны быть согласованы

Как именно проверяется: Проверяются допустимость `FileAlignment`, отношение `SectionAlignment/FileAlignment`, кратность `SizeOfImage`, `SizeOfHeaders`, выравнивание `ImageBase`, секционных `VirtualAddress`, `PointerToRawData`, `SizeOfRawData`, а также порядок `VirtualAddress`.

Для чего проверяется: Эти условия нужны, чтобы file layout и memory layout совпадали с ожиданиями loader-а и parser-а.

Если не проходит: Finding: `pe_section_misaligned`, severity medium, score +8. Это менее жесткий сигнал, потому что часть нестандартных layout может быть benign.

Код: `src/pe.c:180-196`, `src/pe.c:205-213`, `src/heuristics.c:817-824`.

141.6. Exception Directory RVA/Size должны описывать область внутри image

Как именно проверяется: Если `exc_rva != 0 || exc_size != 0`, считается `exc_end = exc_rva + exc_size`; ошибка, если одно поле нулевое или `exc_end > SizeOfImage`.

Для чего проверяется: Exception Directory является входом в `.pdata`. Если entry противоречива или выходит за image, function table нельзя считать надежной.

Если не проходит: Finding: `pe_exception_directory_oob`, severity high, score +25. Если reader `.pdata` позже видит malformed/unreadable directory, `main` также ставит этот флаг.

Код: `src/pe.c:230-235`, `src/unwind.c:623-647`, `src/main.c:317-323`, `src/heuristics.c:825-832`.

142. `RVA -> file offset` checks и их последствия

142.1. RVA должен попадать в секцию или headers

Как именно проверяется: `pe_find_section_for_rva` ищет секцию, чей virtual range покрывает RVA; если секция не найдена, `pe_rva_to_fileoff` допускает только специальный случай `RVA < SizeOfHeaders`.

Для чего проверяется: RVA - это не file offset. Чтобы читать bytes из файла, надо доказать, где эта RVA-область физически лежит.

Если не проходит: Если mapping не найден: для Exception Directory это ведет к `pe_exception_directory_oob`; для `UNWIND_INFO` - к `unwind_parse_fail`; для placement checks может дать `unwind_unusual_section`.

Код: `src/pe.c:265-300`, `src/unwind.c:631-633`, `src/unwind.c:56-72`, `src/heuristics.c:693-696`.

142.2. RVA в virtual tail нельзя читать из файла

Как именно проверяется: Если секция найдена, считается `delta = rva - VirtualAddress`; если `delta >= SizeOfRawData`, mapping отклоняется.

Для чего проверяется: Virtual tail может существовать в памяти как zero-fill, но не иметь bytes на диске. Static parser не должен читать несуществующие bytes.

Если не проходит: Провал дает malformed Exception Directory или parse fail `UNWIND_INFO`, в зависимости от того, что пытались читать.

Код: `src/pe.c:281-285`, callers `src/unwind.c:59-72`, `src/unwind.c:631-637`.

142.3. File offset не должен переполняться и выходить за файл

Как именно проверяется: Для `raw_ptr + delta` используется `safe_add_u32`, затем проверяется `off < file_size`.

Для чего проверяется: Поврежденные значения section header могут вызвать arithmetic overflow и привести к чтению не той области.

Если не проходит: Mapping возвращает false; дальше это превращается в malformed directory, parse fail или невозможность подтвердить placement.

Код: `src/pe.c:21-26`, `src/pe.c:286-290`.

143. Exception Directory / `.pdata` reader checks

143.1. Directory отсутствует: `RVA=0` и `Size=0`

Как именно проверяется: `unwind_read_runtime_functions_ex` возвращает OK с `rf_count=0`.

Для чего проверяется: Zero/zero - нормальный способ PE сказать, что directory отсутствует.

Если не проходит: Если при этом есть executable sections и включены heuristics, появляется `no_runtime_functions`; если executable sections нет, добавляется нейтральная note.

Код: `src/unwind.c:623-625`, `src/heuristics.c:833-842`.

143.2. Одно поле нулевое, другое нет

Как именно проверяется: Reader проверяет `exception_rva == 0 || exception_size == 0` после absence-case.

Для чего проверяется: Адрес без размера и размер без адреса не задают валидную область таблицы.

Если не проходит: Reader возвращает malformed; `main` ставит `img.exception_directory_oob=true`; finding `pe_exception_directory_oob`.

Код: `src/unwind.c:626-629`, `src/main.c:317-323`, `src/heuristics.c:825-832`.

143.3. Directory RVA должен маппиться в file offset

Как именно проверяется: Reader вызывает `pe_rva_to_fileoff(img, exception_rva, &off)`.

Для чего проверяется: Даже если RVA внутри image, static чтение возможно только если bytes присутствуют в файле.

Если не проходит: Malformed directory -> `pe_exception_directory_oob`.

Код: `src/unwind.c:630-634`, `src/pe.c:277-300`.

143.4. Directory byte range должен помещаться в файл

Как именно проверяется: Проверяется `(size_t)off + exception_size <= file_size`.

Для чего проверяется: Начало таблицы может быть mappable, но заявленный размер может уходить за конец файла.

Если не проходит: Malformed directory -> `pe_exception_directory_oob`.

Код: `src/unwind.c:635-638`.

143.5. Directory size должен быть минимум 12 и кратен 12

Как именно проверяется: Проверяется `exception_size >= 12` и `exception_size % 12 == 0`.

Для чего проверяется: Одна x64 `RUNTIME_FUNCTION` - это 12 bytes: begin, end, unwind. Некратный размер означает, что это не корректный массив записей.

Если не проходит: Malformed directory -> общий finding `pe_exception_directory_oob`. Отдельного finding `size_not_multiple` сейчас нет.

Код: `src/unwind.c:639-647`, `src/heuristics.c:825-832`.

143.6. Каждая `RUNTIME_FUNCTION` запись должна читаться целиком

Как именно проверяется: Для каждой записи по `off + i*12` читаются три DWORD.

Для чего проверяется: Нельзя строить карту функций из частично прочитанной записи.

Если не проходит: Если чтение записи падает, reader возвращает malformed; дальше `pe_exception_directory_oob`.

Код: `src/unwind.c:654-667`, `src/main.c:317-323`.

144. Function table integrity checks

144.1. `EndAddress > BeginAddress`

Как именно проверяется: Для каждой записи проверяется `e <= b`.

Для чего проверяется: Функция должна иметь положительный диапазон. Иначе запись не описывает нормальную область кода.

Если не проходит: Finding: `invalid_ranges`; severity high, если таких записей больше 1% от таблицы, иначе medium.

Код: `src/heuristics.c:646-654`, `src/heuristics.c:844-856`, threshold `src/heuristics.c:80`.

144.2. Begin/End не должны выходить за `SizeOfImage`

Как именно проверяется: Проверяется `b >= SizeOfImage || e > SizeOfImage`.

Для чего проверяется: Function range должен быть частью загруженного image. Иначе функция ссылается за пределы модуля.

Если не проходит: Finding: `rf_oob`; high при >1%, иначе medium.

Код: `src/heuristics.c:655-658`, `src/heuristics.c:857-863`.

144.3. `UnwindInfoAddress` должен быть ненулевым и внутри image

Как именно проверяется: Проверяется `u >= SizeOfImage || u == 0`.

Для чего проверяется: Без валидного `UNWIND_INFO` невозможно корректно восстановить кадр этой функции.

Если не проходит: Finding: `unwind_rva_oob`; high при >1%, иначе medium.

Код: `src/heuristics.c:659-662`, `src/heuristics.c:864-870`.

144.4. Исходная таблица не должна идти назад по `BeginAddress`

Как именно проверяется: Проект сравнивает текущий `BeginAddress` с предыдущим.

Для чего проверяется: Runtime function table обычно ожидается отсортированной для корректного lookup.

Если не проходит: Finding: `rf_unsorted`, severity medium.

Код: `src/heuristics.c:664-670`, `src/heuristics.c:871-877`.

144.5. После сортировки ranges не должны пересекаться

Как именно проверяется: Проект сортирует ranges и проверяет `idx[i].begin < max_end`.

Для чего проверяется: Один RVA кода не должен одновременно принадлежать нескольким функциям.

Если не проходит: Finding: `rf_overlaps`, severity high.

Код: `src/heuristics.c:740-773`, `src/heuristics.c:878-884`.

144.6. После сортировки не должно быть duplicate begin

Как именно проверяется: Проверяется `idx[i].begin == sorted_prev_begin`.

Для чего проверяется: Две записи с одним началом конфликтуют при поиске функции по RIP/RVA.

Если не проходит: Finding: `duplicate_begin`, severity medium.

Код: `src/heuristics.c:754-762`, `src/heuristics.c:885-891`.

145. `UNWIND_INFO` parser checks

145.1. `UnwindInfoRVA` желательно 4-byte aligned

Как именно проверяется: Если `(unwind_rva & 3) != 0`, выставляется anomaly `UA_ALIGNMENT`.

Для чего проверяется: UNWIND_INFO обычно DWORD-aligned. Невыравненность может указывать на повреждение или намеренную нестандартность.

Если не проходит: Если parse дальше успешен, это попадет в `unwind_anomalies`; если parse упадет, приоритет будет у `unwind_parse_fail`.

Код: `src/unwind.c:187-195`, `include/unwind.h:63`, `src/heuristics.c:727-735`.

145.2. Header `UNWIND_INFO` должен читаться

Как именно проверяется: Parser переводит RVA в file offset, проверяет `off + 4 <= file_size` и читает 4 bytes header.

Для чего проверяется: Без header нельзя узнать version, flags и размер codes array.

Если не проходит: Parse fail -> finding `unwind_parse_fail`.

Код: `src/unwind.c:56-72`, `src/unwind.c:192-195`, `src/heuristics.c:893-899`.

145.3. Version должна быть 1 или 2

Как именно проверяется: Из первого byte берется `version = hdr[0] & 0x7`, затем проверяется `ver == 1 || ver == 2`.

Для чего проверяется: От версии зависит интерпретация opcode 6/7 и хвоста структуры.

Если не проходит: Unsupported version вызывает parse fail; итоговый finding `unwind_parse_fail`, a anomaly фиксируется в summary.

Код: `src/unwind.c:197-214`.

145.4. Flags не должны содержать неизвестные bits

Как именно проверяется: Проверяется `flags & ~(EHANDLER|UHANDLER|CHAININFO)`.

Для чего проверяется: Неизвестные flags меняют смысл хвоста структуры, но parser не знает, как их трактовать.

Если не проходит: Если parse не падает, это `unwind_anomalies`.

Код: `src/unwind.c:215-217`, `include/unwind.h:45-49`.

145.5. `CHAININFO` не должен конфликтовать с handler flags

Как именно проверяется: Проверяется одновременное наличие `CHAININFO` и `EHANDLER/UHANDLER`.

Для чего проверяется: В текущей модели хвост `UNWIND_INFO` не может одновременно быть и chained function, и handler RVA.

Если не проходит: Если parse продолжается, это `unwind_anomalies`.

Код: `src/unwind.c:218-220`.

145.6. Frame register / frame offset должны быть согласованы

Как именно проверяется: Проверяется `fr == 0 && foff != 0`, допустимость register и strict ABI-класс.

Для чего проверяется: Frame offset без frame register бессмыслен; недопустимый register ломает восстановление frame pointer.

Если не проходит: Обычные нарушения дают `unwind_anomalies`; strict ABI нарушения считаются `unwind_abi_warnings` и отдельного finding сейчас не создают.

Код: `src/unwind.c:221-229`, `src/heuristics.c:734-735`.

145.7. Codes array должен помещаться в файл

Как именно проверяется: Считается `slots_even = (count + 1) & ~1`, затем `codes_off + slots_even*2 <= file_size`.

Для чего проверяется: CountOfCodes может быть поврежден так, что parser ушел бы за конец файла.

Если не проходит: Parse fail -> `unwind_parse_fail`.

Код: `src/unwind.c:231-239`.

145.8. Каждый unwind slot должен читаться

Как именно проверяется: В цикле по `i < count` parser читает 16-bit slot.

Для чего проверяется: Каждая операция кодируется slot-ами; без чтения slot нельзя понять opcode/opinfo.

Если не проходит: Read failure -> parse fail -> `unwind_parse_fail`.

Код: `src/unwind.c:251-258`.

145.9. Opcode должен иметь известную длину в slots

Как именно проверяется: `slots_for_code` возвращает число slots для opcode/version/opinfo; неизвестный opcode выставляет `UA_BAD_OPCODE`, но parser безопасно продолжает как 1 slot.

Для чего проверяется: Нужно знать длину операции, чтобы не принять operand за новую операцию или наоборот.

Если не проходит: Если parser доходит до конца, это `unwind_anomalies`; если затем обнаруживается выход за count, это `unwind_parse_fail`.

Код: `src/unwind.c:24-54`, `src/unwind.c:265-274`.

145.10. Операция не должна выходить за `CountOfCodes`

Как именно проверяется: Проверяется `i + slots > count`.

Для чего проверяется: Opcode может требовать operand slots, которых уже нет в массиве.

Если не проходит: Parse fail -> `unwind_parse_fail`.

Код: `src/unwind.c:271-275`.

145.11. `CodeOffset` должен лежать в prolog и идти в ожидаемом порядке

Как именно проверяется: Для обычных descriptors проверяется `code_offset > prolog` и нарушение порядка относительно предыдущего offset.

Для чего проверяется: Unwind codes описывают действия в прологе. Offset за пределами prolog или странный порядок делает описание подозрительным.

Если не проходит: Если parse успешен, это `unwind_anomalies`.

Код: `src/unwind.c:276-286`, anomaly flags `include/unwind.h:57`, `include/unwind.h:61`.

145.12. Operand slots для `ALLOC_LARGE`, `SAVE_*` должны существовать

Как именно проверяется: Для операций с operands parser отдельно читает дополнительные slots; если их нет, возвращается parse fail.

Для чего проверяется: Без operand нельзя вычислить stack allocation или saved register offset.

Если не проходит: Parse fail -> `unwind_parse_fail`.

Код: `src/unwind.c:357-377`, `src/unwind.c:394-425`, `src/unwind.c:429-500`.

145.13. Operands должны быть логически разумными

Как именно проверяется: Проверяются suspicious sizes/offsets: zero/too-small/misaligned large alloc and far save offsets; проверяются допустимые register/opinfo.

Для чего проверяется: Структура может быть читаемой, но логически странной: например, far offset слишком мал для far form или register invalid.

Если не проходит: Если parse успешен, это `unwind_anomalies`; strict ABI-only случаи идут в `unwind_abi_warnings` без отдельного finding.

Код: `src/unwind.c:113-128`, `src/unwind.c:346-510`.

145.14. `SET_FPREG` должен соответствовать frame register

Как именно проверяется: Если `fr != 0`, но `SET_FPREG` не встречался и это не chain info, ставится anomaly.

Для чего проверяется: Frame register в header должен быть объяснен unwind operation.

Если не проходит: Если parse успешен, finding `unwind_anomalies`.

Код: `src/unwind.c:515-517`.

145.15. `CHAININFO` tail должен быть читаемым и валидным

Как именно проверяется: При `UNW_FLAG_CHAININFO` parser читает вложенную `RUNTIME_FUNCTION` и проверяет ее begin/end/unwind RVA относительно `SizeOfImage`.

Для чего проверяется: Chained unwind должен указывать на валидную функцию и валидную unwind-запись.

Если не проходит: Unreadable chain -> `unwind_parse_fail`; invalid chain fields -> `unwind_anomalies`.

Код: `src/unwind.c:522-538`.

145.16. Handler RVA должен читаться и быть внутри image

Как именно проверяется: При `EHANDLER/UHANDLER` parser читает DWORD handler RVA и проверяет `handler_rva < SizeOfImage`.

Для чего проверяется: Handler - это точка обработки исключения внутри image. Выход за image ломает handler layer.

Если не проходит: Unreadable handler -> `unwind_parse_fail`; handler OOB -> `unwind_anomalies`, a placement checks могут дополнительно дать handler findings.

Код: `src/unwind.c:540-552`.

145.17. Follow-chain не должен зациклиться или уйти слишком глубоко

Как именно проверяется: При `follow_chain` parser хранит `seen[]`, ограничивает depth через `max_chain_depth`, ловит loop и depth exceeded.

Для чего проверяется: Поврежденный chain может зациклить analyzer или скрыть реальную unwind chain.

Если не проходит: Loop/depth/secondary parse problem дают `unwind_anomalies`, не отдельный finding.

Код: `src/unwind.c:559-608`, CLI `src/main.c:212-219`.

146. Placement, rights, reuse, dominance

146.1. `UNWIND_INFO` должен лежать в понятной metadata section

Как именно проверяется: Для `UnwindInfoRVA` ищется section; имя сравнивается с `.xdata/.rdata/.pdata` и похожими вариантами.

Для чего проверяется: Unwind metadata обычно является служебными read-only данными, а не произвольной областью.

Если не проходит: Finding: `unwind_unusual_section`; если массово больше 5% (`n/20`), вес и severity выше.

Код: `src/heuristics.c:120-126`, `src/heuristics.c:690-704`, `src/heuristics.c:972-978`.

146.2. `UNWIND_INFO` не должен быть в executable section

Как именно проверяется: Проверяется `section->characteristics & IMAGE_SCN_MEM_EXECUTE`.

Для чего проверяется: Metadata не должна исполняться как код; execute placement подозрительно для unwind structures.

Если не проходит: Finding: `unwind_in_exec_section`, severity high, score +20.

Код: `src/heuristics.c:704-706`, `src/heuristics.c:965-971`.

146.3. `UNWIND_INFO` не должен быть в writable section без причины

Как именно проверяется: Проверяется `section->characteristics & IMAGE_SCN_MEM_WRITE`.

Для чего проверяется: Writable metadata может быть runtime-patched. Для обычного compiler-produced PE это нетипично.

Если не проходит: Finding: `unwind_writable_section`; при >5% affected entries вес выше.

Код: `src/heuristics.c:708-710`, `src/heuristics.c:980-986`.

146.4. Handler target должен быть executable

Как именно проверяется: Для `handler_rva` ищется section; если section отсутствует или не имеет execute bit, растет `handler_nonexec_section`.

Для чего проверяется: Handler должен быть исполняемой control-flow target.

Если не проходит: Finding: `handler_nonexec_section`, severity high, score +15.

Код: `src/heuristics.c:714-719`, `src/heuristics.c:988-994`.

146.5. Handler target не должен быть writable без причины

Как именно проверяется: Проверяется write bit handler section.

Для чего проверяется: Writable handler target может быть JIT/runtime case, но для обычного PE это требует проверки.

Если не проходит: Finding: `handler_writable_section`, severity medium, score +10.

Код: `src/heuristics.c:720-723`, `src/heuristics.c:995-1000`.

146.6. Массовый reuse `UNWIND_INFO`

Как именно проверяется: Hash map считает, сколько функций указывает на каждый `UnwindInfoRVA`; далее считается max_count/total.

Для чего проверяется: Слишком много функций на одну unwind-запись может быть centralized metadata/stub/dispatcher pattern.

Если не проходит: Finding: `unwind_reuse_strong` при ratio > 0.30 и n > 200; `unwind_reuse_moderate` при ratio > 0.10, n > 200 и наличии другого static signal.

Код: `src/heuristics.c:671-680`, `src/heuristics.c:775-780`, `src/heuristics.c:908-918`, `src/heuristics.c:946-963`.

146.7. Dominant handler

Как именно проверяется: Hash map считает, сколько `HandlerRVA` указывает на один target; ratio = max_handler_count / handler_count.

Для чего проверяется: Один handler на почти все функции может быть централизованной обработкой/dispatcher-ом.

Если не проходит: Finding: `handler_dominance`, если `handler_count > 50`, ratio > 0.80 и нет benign exemption.

Код: `src/heuristics.c:679-688`, `src/heuristics.c:782-785`, `src/heuristics.c:920-943`.

146.8. Combo placement/reuse findings

Как именно проверяется: Проверяются сочетания: executable unwind + nonexec handler; placement + integrity; reuse + dominance; writable unwind + writable handler.

Для чего проверяется: Комбинации снижают вероятность, что это одиночная benign-особенность.

Если не проходит: Findings: `placement_exec_nonexec_handler_combo`, `placement_integrity_combo`, `reuse_dominance_combo`, `writable_metadata_handler_combo`.

Код: `src/heuristics.c:1003-1022`.

147. Score, severity и отсутствие отдельного finding

147.1. Почему некоторые проверки не имеют собственного finding code

Как именно проверяется: Некоторые микропроверки складываются в общий counter или общий finding: например, разные причины unreadable Exception Directory дают `pe_exception_directory_oob`; разные проблемы `UNWIND_INFO` дают `unwind_parse_fail` или `unwind_anomalies`.

Для чего проверяется: Это сделано, чтобы 27 findings были уровня evidence families, а не сотней мелких parser error codes.

Если не проходит: При защите нужно честно говорить: проверка есть, но отдельного finding code для каждой микропричины нет. Деталь видна через error text, counters, affected_count, representative RVA и report evidence.

Код: `src/unwind.c:623-647`, `src/unwind.c:143-160`, `src/heuristics.c:893-906`.

147.2. Почему severity иногда зависит от процента

Как именно проверяется: Проект считает ratio для invalid ranges, rf OOB, unwind RVA OOB, parse fail, anomalies.

Для чего проверяется: Одна ошибка может быть единичным повреждением. Массовая доля показывает systemic issue.

Если не проходит: Если ratio > 1% или >5% для unwind anomalies, severity/score повышаются.

Код: `src/heuristics.c:80-84`, `src/heuristics.c:844-906`.

147.3. Почему `strict ABI` warnings не стали отдельными findings

Как именно проверяется: Strict ABI flags отмечают формальные нарушения Microsoft x64 ABI: register class, reserved opinfo, push order, chain restrictions, save offset.

Для чего проверяется: Некоторые реальные toolchains и hand-written assembly могут выглядеть нестандартно, поэтому эти warnings не смешиваются с основными 27 findings.

Если не проходит: При включенном `--strict-abi` они попадают в `unwind_abi_warnings`, но отдельного finding code сейчас не создают.

Код: `include/unwind.h:65-69`, `src/unwind.c:227-229`, `src/unwind.c:287-294`, `src/heuristics.c:734-735`.

148. Отчетная и динамическая проверка

148.1. Report metadata validation

Как именно проверяется: Перед JSON/HTML проверяются schema version, tool version, timestamp, SHA-256 length/hex, build mode, max chain depth.

Для чего проверяется: Отчет должен быть воспроизводимым доказательным артефактом, а не просто красивым HTML.

Если не проходит: Если metadata/model invalid, JSON/HTML не пишется; это ошибка записи отчета, не один из 27 findings.

Код: `src/main.c:450-480`, `src/report.c:556-620`.

148.2. Dynamic instrumentation completeness

Как именно проверяется: Dynamic mode считает module refresh failures, breakpoint failures, drops, pending/rearm failures, dropped events; complete session требует ran/ok/no timeout/target exited/instrumentation complete.

Для чего проверяется: Если отладочная сессия неполная, runtime evidence нельзя считать сильным подтверждением.

Если не проходит: Это влияет на dynamic evidence level/assessment, но не создает один из 27 static findings.

Код: `include/dynamic.h:41-90`, `src/dynamic_win.c:278-335`, `src/report.c:338-374`.

148.3. Dynamic events как runtime corroboration

Как именно проверяется: Проект ставит breakpoints на unwind handlers и exception APIs, ловит handler hits, VEH/VCH/UEF registrations, `RtlAddFunctionTable`, `RtlInstallFunctionTableCallback`, first/second chance exceptions.

Для чего проверяется: Статика показывает подозрительную структуру; динамика показывает, участвовали ли exception/unwind mechanisms в конкретном запуске.

Если не проходит: Dynamic signals повышают final confidence и могут дать correlated verdict, но сами не являются частью 27 static findings.

Код: `src/dynamic_win.c:561-599`, `src/dynamic_win.c:789-850`, `src/dynamic_win.c:902-984`, `src/report.c:326-440`.

149. Техническая детализация: как именно это проверяется в коде

149.1. Техническая схема безопасного чтения числа

Как именно проверяется: Любое число читается в два этапа: сначала `buf_read` проверяет границы и копирует bytes во временный массив, затем `le16/le32/le64` собирает little-endian значение. Например, `buf_read_u32` читает 4 bytes и превращает `tmp[0..3]` в `uint32_t`.

Для чего проверяется: PE хранит поля little-endian. Нельзя просто привести указатель к структуре: могут быть unaligned addresses, другой packing и выход за файл.

Если не проходит: Если `off > size` или `len > size - off`, чтение возвращает false. Выше по стеку это становится hard stop, malformed directory или `unwind_parse_fail` в зависимости от слоя.

Код: `src/util.c:42-70`, declarations `include/util.h:23-27`.

149.2. Техническая схема чтения DOS/NT headers

Как именно проверяется: Parser читает `MZ` по offset `0x00`, затем `e_lfanew` по offset `0x3c`. После этого проверяет, что по `e_lfanew` помещаются 4 bytes PE signature и 20 bytes COFF header. `filehdr = e_lfanew + 4`, `opthdr = filehdr + 20`.

Для чего проверяется: Это защищает переход от DOS header к NT headers. `e_lfanew` в поврежденном файле может указывать куда угодно.

Если не проходит: Если `MZ`, `e_lfanew`, PE signature или COFF не читаются, `pe_parse` возвращает false. Это hard stop, потому что parser еще не знает, что перед ним валидный PE.

Код: `src/pe.c:11-19`, `src/pe.c:40-65`.

149.3. Техническая схема offsets PE32+ Optional Header

Как именно проверяется: Для PE32+ проект читает поля по фиксированным offsets от начала Optional Header: `ImageBase` at `0x18`, `SectionAlignment` at `0x20`, `FileAlignment` at `0x24`, `SizeOfImage` at `0x38`, `SizeOfHeaders` at `0x3c`, `NumberOfRvaAndSizes` at `0x6c`. Magic читается по `0x00` и должен быть `0x20b`.

Для чего проверяется: Offsets PE32 и PE32+ различаются. Если перепутать magic, parser прочитает правильное количество bytes не из тех полей и построит ложную модель.

Если не проходит: Если magic не `0x20b`, проект не пытается адаптировать parser, а честно завершает анализ как unsupported format.

Код: `src/pe.c:82-108`, constants `include/pe.h:18-19`.

149.4. Техническая схема вычисления Exception Directory entry

Как именно проверяется: В PE32+ DataDirectory начинается на offset `0x70` от Optional Header. Exception entry имеет index `3`. Поэтому относительное смещение entry: `dd_rel = 0x70 + 3 * 8 = 0x88`. В entry первые 4 bytes - RVA, следующие 4 bytes - Size.

Для чего проверяется: Это точка входа в `.pdata`. Ошибка на 8 bytes или перепутанный index приведут к чтению другого directory, например import/export/resource.

Если не проходит: Если `NumberOfRvaAndSizes` меньше 4 или `dd_rel + 8 > SizeOfOptionalHeader`, parser делает hard stop. Если entry читается, но диапазон плохой, дальше это уже `pe_exception_directory_oob`.

Код: `src/pe.c:121-137`, `include/pe.h:22`.

149.5. Техническая схема чтения section header

Как именно проверяется: Каждый section header имеет размер 40 bytes. Для секции `i` адрес header: `secthdr + i * 40`. Проект читает name at `+0`, `VirtualSize` at `+8`, `VirtualAddress` at `+12`, `SizeOfRawData` at `+16`, `PointerToRawData` at `+20`, `Characteristics` at `+36`.

Для чего проверяется: Эти поля нужны для двух разных карт: физической карты файла и виртуальной карты image.

Если не проходит: Если заголовок секции не помещается или одно поле не читается, `pe_parse` прекращается. Если читается, но значения странные, это уже structural counters и PE findings.

Код: `src/pe.c:139-172`.

149.6. Техническая схема проверки alignments

Как именно проверяется: Проект вычисляет `compact_alignment = SectionAlignment < 0x1000 && FileAlignment == SectionAlignment`. `FileAlignment` считается валидным, если это power-of-two и лежит в `0x200..0x10000`, либо compact case. Затем проверяет `SectionAlignment < FileAlignment`, special compact rule, `SizeOfImage % SectionAlignment`, `SizeOfHeaders % FileAlignment`, `SizeOfHeaders >= needed`, `ImageBase & 0xffff`.

Для чего проверяется: Это набор loader/layout invariants: размеры и адреса должны ложиться на ожидаемые границы, иначе mapping из файла в память становится нестандартным.

Если не проходит: Каждое нарушение увеличивает `misaligned`; позже это один finding `pe_section_misaligned`, потому что это семейство нарушений согласованности PE layout.

Код: `src/pe.c:180-196`, `src/heuristics.c:817-824`.

149.7. Техническая формула пересечения диапазонов

Как именно проверяется: Функция overlap возвращает false, если один из размеров нулевой. Иначе используется стандартная полуоткрытая формула: `a_begin < b_begin + b_size && b_begin < a_begin + a_size`.

Для чего проверяется: Полуоткрытые интервалы `[begin, begin+size)` позволяют корректно считать соседние секции непересекающимися, если конец одной равен началу другой.

Если не проходит: Для raw ranges провал дает `pe_section_raw_overlap`; для virtual ranges - `pe_section_virtual_overlap`.

Код: `src/pe.c:28-34`, raw use `src/pe.c:217-219`, virtual use `src/pe.c:220-226`.

149.8. Техническая схема `RVA -> section`

Как именно проверяется: Для каждой секции считается `size = max(VirtualSize, SizeOfRawData)`, `start = VirtualAddress`, `end = start + size`. RVA принадлежит секции, если `rva >= start && rva < end`.

Для чего проверяется: `VirtualSize` важен для memory layout, `SizeOfRawData` важен для file layout. Использование max делает parser терпимым к типичным PE-особенностям, где одно поле меньше другого.

Если не проходит: Если секция не найдена, caller либо попробует headers fallback, либо получит failure. Это может стать malformed directory, parse fail или unusual placement.

Код: `src/pe.c:265-274`.

149.9. Техническая схема `RVA -> file offset`

Как именно проверяется: Если section найдена: `delta = rva - Section.VirtualAddress`. Если `delta >= SizeOfRawData`, возврат false. Иначе `off = PointerToRawData + delta`, причем сложение идет через `safe_add_u32`, затем проверяется `off < file_size`.

Для чего проверяется: Это отделяет memory-only zero-fill область от bytes, реально существующих на диске. Static parser не моделирует loader-zero-fill как реальные file bytes.

Если не проходит: Провал при чтении Exception Directory дает `pe_exception_directory_oob`; провал при чтении `UNWIND_INFO` дает `unwind_parse_fail`.

Код: `src/pe.c:277-300`, `src/unwind.c:59-72`, `src/unwind.c:631-637`.

149.10. Техническая схема headers fallback

Как именно проверяется: Если RVA не попал ни в одну секцию, проект проверяет `SizeOfHeaders > 0`, `rva < SizeOfHeaders`, и `rva < file_size`. Тогда file offset считается равным RVA.

Для чего проверяется: PE headers тоже занимают виртуальную область в начале image. Некоторые RVA могут законно указывать в headers, а не в обычную секцию.

Если не проходит: Если и headers fallback не подходит, mapping возвращает false. Это важно: проект не притворяется, что любой RVA можно прочитать.

Код: `src/pe.c:293-299`.

149.11. Техническая схема чтения `.pdata`

Как именно проверяется: Reader сначала обрабатывает absence `RVA=0 && Size=0`. Затем reject malformed pair `RVA=0 xor Size=0`. Потом переводит directory RVA в file offset, проверяет `off + Size <= file_size`, `Size >= 12`, `Size % 12 == 0`, затем `n = Size / 12`.

Для чего проверяется: Это превращает directory bytes в строго типизированный массив `RuntimeFunction[n]`.

Если не проходит: Любой провал, кроме absence, дает malformed read. `main` сохраняет это как `img.exception_directory_oob=true`, а heuristics делает `pe_exception_directory_oob`.

Код: `src/unwind.c:623-647`, `src/main.c:306-323`, `src/heuristics.c:825-832`.

149.12. Техническая схема чтения одной `RUNTIME_FUNCTION`

Как именно проверяется: Для записи `i` смещение `e = off + i*12`. По `e+0` читается `BeginAddress`, по `e+4` - `EndAddress`, по `e+8` - `UnwindInfoAddress`. Все три поля являются RVA, не VA и не file offset.

Для чего проверяется: Эта запись задает связь: range функции -> unwind metadata. Все последующие проверки используют именно эти RVA.

Если не проходит: Если одно из трех DWORD не читается, вся directory считается malformed, потому что массив записей оборван.

Код: `src/unwind.c:654-667`, struct `include/unwind.h:16-20`.

149.13. Техническая схема проверки function ranges

Как именно проверяется: В heuristics loop для каждой записи берутся `b=begin`, `e=end`, `u=unwind`. Проверки: `e <= b`, `b >= SizeOfImage || e > SizeOfImage`, `u >= SizeOfImage || u == 0`. Отдельно проверяется order: `prev_valid && b < prev_begin`.

Для чего проверяется: Это базовая валидация карты функций: положительная длина, попадание в image и наличие указателя на unwind data.

Если не проходит: Провалы дают `invalid_ranges`, `rf_oob`, `unwind_rva_oob`, `rf_unsorted`.

Код: `src/heuristics.c:646-670`, findings `src/heuristics.c:850-877`.

149.14. Техническая схема overlap/duplicate после сортировки

Как именно проверяется: Создается массив `RfIdx` с begin/end, сортируется `qsort` по begin, затем end. В проходе по sorted array: если `begin == sorted_prev_begin`, duplicate; если `begin < max_end`, overlap; `max_end` обновляется максимальным end.

Для чего проверяется: Так проект ловит пересечения даже если исходная таблица не отсортирована.

Если не проходит: Провалы дают `duplicate_begin` и `rf_overlaps`.

Код: `src/heuristics.c:343-350`, `src/heuristics.c:740-773`, findings `src/heuristics.c:878-891`.

149.15. Техническая схема bit layout `UNWIND_INFO` header

Как именно проверяется: После чтения 4 bytes header проект извлекает: `version = hdr[0] & 0x7`, `flags = hdr[0] >> 3`, `prolog = hdr[1]`, `count = hdr[2]`, `frame_reg = hdr[3] & 0x0f`, `frame_off = hdr[3] >> 4`.

Для чего проверяется: Это packed bit-field формат Windows x64 unwind. Нельзя читать его как обычную C-структуру без понимания bit layout.

Если не проходит: Unknown version дает parse fail. Bad flags/frame combinations дают `unwind_anomalies`, если parse доходит до конца.

Код: `src/unwind.c:187-229`.

149.16. Техническая схема размера unwind code array

Как именно проверяется: Количество slots на диске выравнивается до четного: `slots_even = (count + 1) & ~1`. Начало codes: `codes_off = uoff + 4`. Размер bytes: `codes_bytes = slots_even * 2`. Проверяется `codes_off + codes_bytes <= file_size`.

Для чего проверяется: Windows хранит array из 2-byte slots и padding до even count перед handler/chain tail. Без этого parser сдвинет хвост и прочитает handler/chain не там.

Если не проходит: Если array не помещается, parse fail -> `unwind_parse_fail`.

Код: `src/unwind.c:231-239`.

149.17. Техническая схема decode одного unwind slot

Как именно проверяется: Slot читается как `uint16_t`. `code_offset = slot & 0xff`; `op_byte = slot >> 8`; `op = op_byte & 0x0f`; `opinfo = op_byte >> 4`. Затем `slots_for_code(version, op, opinfo, v2_seen_epilog)` сообщает, сколько slots занимает операция.

Для чего проверяется: Так parser понимает, где начинается следующая операция и какие operands надо прочитать.

Если не проходит: Unknown opcode ставит `UA_BAD_OPCODE`; если заявленная длина выходит за `CountOfCodes`, это parse fail.

Код: `src/unwind.c:24-54`, `src/unwind.c:251-275`.

149.18. Техническая схема проверки `CodeOffset`

Как именно проверяется: Для не-v2-epilog descriptors проверяется `code_offset > prolog` и нарушение порядка `have_prev_code_offset && code_offset > prev_code_offset`.

Для чего проверяется: `CodeOffset` должен описывать instruction offset внутри prolog. Порядок важен, потому что unwind codes должны соответствовать обратному восстановлению действий пролога.

Если не проходит: Провал не всегда ломает чтение, поэтому это anomaly -> `unwind_anomalies`, а не parse fail.

Код: `src/unwind.c:276-286`.

149.19. Техническая схема operands `UWOP_ALLOC_*`

Как именно проверяется: `UWOP_ALLOC_SMALL` добавляет `opinfo*8+8` stack bytes. `UWOP_ALLOC_LARGE mode 0` читает один 16-bit slot и умножает на 8. `mode 1` читает два slots как 32-bit size. Нечитаемые operands дают parse fail; подозрительные размеры ставят `UA_OPERAND_SANITY`.

Для чего проверяется: Эти операции задают размер fixed stack frame. Ошибка здесь означает неверный расчет восстановления RSP.

Если не проходит: Truncated operands -> `unwind_parse_fail`; suspicious operands -> `unwind_anomalies`.

Код: `src/unwind.c:357-384`, sanity `src/unwind.c:113-120`.

149.20. Техническая схема operands `UWOP_SAVE_*`

Как именно проверяется: `SAVE_NONVOL` читает один scaled offset slot; `SAVE_NONVOL_FAR` читает два slots как 32-bit offset. Аналогично XMM/XMM128 variants читают required operands. Проверяются register classes, zero offsets under strict ABI, far offset sanity and alignment.

Для чего проверяется: Эти операции показывают, где в stack frame сохранены registers. Если offset/operand неверный, virtual unwind восстановит неверные registers.

Если не проходит: Truncated operands -> `unwind_parse_fail`; invalid/suspicious operands -> `unwind_anomalies` или strict ABI warning.

Код: `src/unwind.c:394-500`, sanity `src/unwind.c:122-128`.

149.21. Техническая схема tail: handler или chain

Как именно проверяется: После codes array хвост начинается по `after_codes = codes_off + codes_bytes`. Если `CHAININFO`, parser читает 12 bytes nested `RuntimeFunction`. Если `EHANDLER/UHANDLER`, parser читает 4-byte `handler_rva`.

Для чего проверяется: Even-padding важен: если считать tail сразу после raw `count*2`, handler/chain будет прочитан со сдвигом.

Если не проходит: Truncated tail -> `unwind_parse_fail`. Invalid chain/handler OOB -> `unwind_anomalies`, а placement layer может дать handler findings.

Код: `src/unwind.c:519-552`.

149.22. Техническая схема follow-chain

Как именно проверяется: Если chain following включен, parser идет от `out->chain.unwind_rva`, хранит до 16 seen RVA, на каждом depth вызывает `unwind_parse` повторно с `follow_chain=false`, суммирует stack effects и masks, проверяет mismatch frame reg/off, loop и max depth.

Для чего проверяется: Это дает ограниченную проверку chained metadata без риска бесконечной рекурсии.

Если не проходит: Loop/depth/mismatch становятся anomalies. Невозможность разобрать chained target тоже добавляет anomaly и прекращает chain walk.

Код: `src/unwind.c:559-608`.

149.23. Техническая схема section rights для placement

Как именно проверяется: Для `UnwindInfoRVA` и `HandlerRVA` проект снова использует `pe_find_section_for_rva`. Затем смотрит bits `IMAGE_SCN_MEM_EXECUTE = 0x20000000` и `IMAGE_SCN_MEM_WRITE = 0x80000000` в `Characteristics`.

Для чего проверяется: Это технически связывает PE section table с semantic role: metadata должна быть data, handler должен быть code.

Если не проходит: Провалы дают `unwind_in_exec_section`, `unwind_writable_section`, `handler_nonexec_section`, `handler_writable_section`; unknown/unusual section дает `unwind_unusual_section`.

Код: `src/heuristics.c:7-12`, `src/heuristics.c:690-723`, findings `src/heuristics.c:965-1000`.

149.24. Техническая схема hash maps для reuse/dominance

Как именно проверяется: Для каждого `UnwindInfoRVA` вызывается `hmap_inc(&unwind_counts, u)`. Для каждого handler вызывается `hmap_inc(&handler_counts, handler_rva)`. Потом берется `hmap_max_val`, `hmap_unique`, ratio = max/total или max/handler_count.

Для чего проверяется: Это не сравнение всех со всеми O(n^2), а счетчик частот. Он показывает концентрацию ссылок на одну metadata/handler точку.

Если не проходит: Ratios выше thresholds дают `unwind_reuse_strong`, `unwind_reuse_moderate`, `handler_dominance`.

Код: `src/heuristics.c:146-170`, `src/heuristics.c:671-688`, `src/heuristics.c:775-785`, findings `src/heuristics.c:908-963`.

149.25. Техническая схема score/confidence

Как именно проверяется: Каждый finding вызывает `add_score`, который суммирует вес и clamp-ит score в диапазон 0..100. Затем `finalize_verdict` считает strong/medium/weak findings, family count, bonuses/penalties и verdict thresholds.

Для чего проверяется: Это отделяет факт нарушения от итоговой оценки: несколько слабых признаков и один hard integrity finding дают разный уровень уверенности.

Если не проходит: Score thresholds дают `high/moderate/low/no_strong_static_anomalies`; mitigation rules могут снизить confidence без удаления findings.

Код: `src/heuristics.c:359-364`, `src/heuristics.c:514-588`.

149.26. Техническая схема dynamic breakpoints

Как именно проверяется: Dynamic layer ставит software breakpoints: сохраняет original byte, пишет `0xCC`, при breakpoint hit чистит breakpoint, откатывает RIP на address INT3, ставит Trap Flag для single-step, после single-step возвращает breakpoint обратно.

Для чего проверяется: Так можно наблюдать handler/API hits и при этом дать debuggee продолжить выполнение оригинальной инструкции.

Если не проходит: Если breakpoint set/rearm fails, `instrumentation_complete=false`; dynamic evidence ослабляется и не считается полноценным подтверждением.

Код: `src/dynamic_win.c:278-335`, hit/rearm flow `src/dynamic_win.c:852-1108`.

149.27. Техническая схема чтения аргументов Windows x64 API в dynamic mode

Как именно проверяется: При API breakpoint project читает `CONTEXT` thread. Для Windows x64 первые аргументы находятся в `RCX`, `RDX`, `R8`, `R9`: VEH/VCH handler берется из `RDX`, UEF handler из `RCX`, `RtlAddFunctionTable` table/count/base из `RCX/RDX/R8`, callback variant из `RCX/RDX/R8/R9`.

Для чего проверяется: Так dynamic layer превращает runtime API calls в evidence: кто регистрирует handlers и runtime function tables.

Если не проходит: Эти события не создают 27 static findings, но попадают в `DynamicReport` и могут усилить final assessment/correlation.

Код: `src/dynamic_win.c:902-984`, assessment `src/report.c:326-440`.

150. Как это связано с 27 findings технически

Теперь связь такая: десятки микропроверок не равны десяткам finding codes. Они агрегируются. Например, `pe_exception_directory_oob` покрывает несколько технических провалов: противоречивая пара RVA/Size, directory за `SizeOfImage`, unmappable RVA, directory за концом файла, size < 12, size % 12 != 0, обрыв записи. `unwind_parse_fail` покрывает unmappable `UnwindInfoRVA`, header OOB, codes array OOB, opcode spans beyond count, truncated operands, truncated handler/chain tail. `unwind_anomalies` покрывает те случаи, когда parse технически возможен, но значения подозрительны: bad flags, bad operands, code offsets, handler OOB, chain invalid, loop/depth.

151. Ответ комиссии по этой теме

Если спрашивают 'какие признаки вытекают, если проверка не проходит', ответ такой: ранние parser gates дают hard stop без findings, потому что модель еще не построена; PE structural checks дают PE/table findings; Exception Directory reader problems дают `pe_exception_directory_oob`; function table checks дают `invalid_ranges`, `rf_oob`, `unwind_rva_oob`, `rf_unsorted`, `rf_overlaps`, `duplicate_begin`; `UNWIND_INFO` parser failures дают `unwind_parse_fail`, читаемые странности дают `unwind_anomalies`; placement checks дают `unwind_*_section` и `handler_*_section`; reuse/dominance checks дают `unwind_reuse_*` и `handler_dominance`; сочетания слоев дают combo findings. Отдельные микропричины не всегда имеют отдельный код finding, но они входят в общий evidence-family finding и показываются через counters, messages, affected_count и representative RVA.

Самая точная формулировка: 'Проект проверяет бинарник слоями. На каждом слое провал имеет разный смысл: до PE-модели это hard stop; после PE-модели это structural finding; после function/unwind model это heuristic finding; после dynamic run это runtime evidence. Поэтому 27 findings - не все if-ы программы, а финальная evidence-модель поверх десятков микропроверок.'

РАСШИРЕННОЕ ТЕХНИЧЕСКОЕ ОБЪЯСНЕНИЕ ПРОВЕРОК

Этот раздел добавлен как подробное исправление к предыдущей краткой матрице. Здесь каждая проверка описывается не одним предложением, а как реальная операция анализатора: откуда берутся данные, какие условия проверяются, зачем они нужны, как трактуется нормальный и ненормальный результат, какой счетчик или finding появляется и где это видно в коде.

Главная мысль: проект не просто проходит по структурам PE набором if-ов. Он строит координатную модель бинарника, проверяет согласованность file layout и memory layout, читает таблицу RUNTIME_FUNCTION, разбирает UNWIND_INFO как маленький байтовый язык, затем превращает десятки микропроверок в 27 понятных признаков, score, confidence, verdict и отчет. Поэтому сложность лежит не в количестве синтаксических операторов, а в корректном сохранении смысла между уровнями: file offset, RVA, VA, секция, таблица функций, unwind-коды, handler, runtime evidence.

152. Как читать дальнейшее описание

Для каждого блока используется одна и та же схема. Сначала указано, какие данные пришли на вход. Потом описано, как именно код проверяет данные технически. Затем объясняется, зачем эта проверка вообще нужна с точки зрения PE/unwind. После этого дано толкование результата: что значит успешное прохождение, что значит провал, почему провал не всегда равен вредоносности, но является доказательным сигналом.

Если проверка не может честно породить finding, это тоже прямо указано. Например, неверная MZ-сигнатура не становится одним из 27 findings, потому что анализатор еще не построил PE-модель. Это hard stop: программа говорит, что вход не является поддерживаемым PE32+ x64 образом, и останавливается.

Если проверка выполняется уже после построения PeImage, она либо записывает структурный счетчик в PeImage, либо позже превращается в finding в heuristics_analyze. Именно этот переход важен для защиты: finding - это не случайный текст, а итог наблюдаемого счетчика, порога и семейства доказательств.

153. Полная цепочка анализа от файла до вывода

Шаг 1. main.c читает аргументы командной строки и включает нужные режимы: обычный анализ, --dump, --ops, --heuristics, --dynamic, --json, --html. Это задает, какие блоки будут обязательными, а какие опциональными.

Шаг 2. util.c загружает файл целиком в память как массив байтов Buf. Дальше все чтение идет через безопасные функции buf_read, buf_read_u16, buf_read_u32, buf_read_u64. Это принципиально: парсер не читает указателями в произвольное место, а каждый раз проверяет, что offset + length помещается в размер файла.

Шаг 3. pe.c строит PeImage: проверяет MZ, e_lfanew, PE signature, COFF Machine, OptionalHeader Magic 0x20b, SizeOfImage, SizeOfHeaders, alignment, DataDirectory[Exception], section headers. На этом этапе создается карта, без которой нельзя честно читать .pdata и .xdata.

Шаг 4. unwind.c читает Exception Directory как массив RUNTIME_FUNCTION. Для x64 каждая запись 12 байт: BeginAddress, EndAddress, UnwindInfoAddress. Размер каталога должен быть кратен 12, иначе невозможно понять, где заканчивается одна запись и начинается следующая.

Шаг 5. main.c проходит по каждой RUNTIME_FUNCTION и вызывает unwind_parse для UnwindInfoAddress. На этом этапе каждая UNWIND_INFO превращается в UnwindSummary: version, flags, prolog size, count of codes, frame register, handler, chained info, stack effect, anomaly bits.

Шаг 6. heuristics.c анализирует массив RuntimeFunction плюс массив UnwindSummary. Он считает структурные нарушения, проценты, reuse-ratios, dominance-ratios, расположение metadata и handler-ов по секциям, затем создает findings, score, suspicion, confidence, evidence_level и verdict.

Шаг 7. dynamic_win.c запускается только при --dynamic и только на Windows. Он стартует цель под Debug API, ставит breakpoints на runtime exception APIs и найденные handler addresses, собирает события и потом report.c сопоставляет runtime evidence co static evidence.

Шаг 8. report.c выводит результат: brief console report, CSV/Ghidra CSV, JSON и HTML. В JSON/HTML сохраняются не только итоговые слова, но и поля доказательств: sections, validation counters, findings, representative_rva, affected_count, dynamic events, trace.

Ключевые файлы цепочки:
src/main.c:270-526 - общий pipeline.
src/util.c:13-35, 42-70 - загрузка файла и безопасное чтение байтов.
src/pe.c:36-256 - построение PE-модели и счетчики структурных проблем.
src/pe.c:265-300 - поиск секции по RVA и перевод RVA в file offset.
src/unwind.c:611-681 - чтение Exception Directory / RUNTIME_FUNCTION.
src/unwind.c:165-608 - разбор UNWIND_INFO.
src/heuristics.c:590-1032 - превращение счетчиков и summary в findings.
src/dynamic_win.c:657-1108 - динамическая проверка через Windows Debug API.

src/report.c:326-440, 622-920, 1329-1501 - assessment, JSON и HTML отчет.

154. Блок безопасного чтения: почему анализ начинается не с PE, а с границ массива байтов

Файл на диске для программы - это не структура PE, а просто массив байтов. Любое поле PE сначала является числом offset внутри этого массива. Поэтому технически первая защита анализатора - не MZ и не PE signature, а запрет чтения за пределами Buf.

Функция buf_read получает offset и length. Она проверяет два условия: off <= b->size и len <= b->size - off. Второе условие специально записано через вычитание после проверки off <= size, чтобы не получить переполнение при off + len. Только после этого memcpy копирует байты.

buf_read_u16/u32/u64 читают 2/4/8 байт и преобразуют их из little-endian. Это важно, потому что PE хранит числовые поля в little-endian порядке. Например байты 0B 02 в OptionalHeader.Magic означают 0x020B, то есть PE32+.

154.1. Техническая проверка чтения поля

Что читает: Любое поле PE: WORD, DWORD, QWORD, имя секции, массив unwind slots.

Как проверяет: Перед копированием проверяется, что запрошенный диапазон [off, off + len) целиком помещается в [0, file_size). Формально код избегает выражения off + len как основного условия и использует len > b->size - off, чтобы не поймать integer overflow.

Для чего: Поврежденный или специально созданный файл может поставить e_lfanew, section header pointer, Exception Directory RVA или UnwindInfoAddress так, чтобы наивный парсер ушел за конец файла. Без безопасного чтения анализатор сам станет нестабильным.

Интерпретация провала: Если раннее поле не читается, анализ не может продолжаться. Это не один из 27 findings, потому что PE-модель еще не доказана. Если чтение ломается уже внутри Exception Directory или UNWIND_INFO, тогда это преобразуется в pe_exception_directory_oob или unwind_parse_fail.

Код: src/util.c:42-70; использование по всему src/pe.c и src/unwind.c.

155. PE-gates: проверки, без которых нельзя говорить о признаках

PE-gates - это ранние входные ворота. Они отвечают на вопрос: можно ли вообще применять к этому файлу логику x64 Exception Directory и Windows UNWIND_INFO? Если ответ нет, программа не должна создавать псевдо-findings, потому что это было бы ложной аналитикой.

Сначала читается DOS magic MZ по offset 0. Потом из offset 0x3c читается e_lfanew - смещение до NT Headers. Затем проверяется, что по e_lfanew действительно есть 4 байта PE\0\0 и после них 20 байт COFF header. После этого проверяется Machine == 0x8664, потому что проект реализует x64 unwind, а не x86 SEH и не ARM64 unwind.

Дальше проверяется OptionalHeader.Magic == 0x20b. Это PE32+, то есть 64-битный PE. Если там 0x10b, это PE32, и формат exception/unwind для цели проекта другой. Поэтому проект честно останавливается, а не пытается интерпретировать x86 как x64.

155.1. Почему Machine и Magic не являются обычными findings

Технически: Machine берется из COFF header offset filehdr + 0, OptionalHeader.Magic берется из opthdr + 0.

Смысл: Эти поля выбирают саму грамматику дальнейших структур. RUNTIME_FUNCTION и UNWIND_INFO в текущей реализации имеют смысл только для AMD64 PE32+.

Если провал: Анализатор завершает pe_parse с ошибкой. Это не 'подозрительный unwind', а неподдерживаемый вход. Для комиссии это нужно объяснять так: findings оценивают x64 EH/unwind модель, а не классифицируют любой бинарный файл на свете.

Код: src/pe.c:40-90; include/pe.h:15-20.

156. Optional Header и Data Directory: где проект берет координаты Exception Directory

Optional Header в PE32+ содержит поля, которые задают координатную систему образа: ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders, NumberOfRvaAndSizes. Без них нельзя понять, какие RVA допустимы, где заканчивается image и можно ли переводить виртуальные адреса в смещения файла.

Exception Directory находится в DataDirectory с индексом 3. В PE32+ массив DataDirectory начинается относительно Optional Header на offset 0x70. Каждая запись занимает 8 байт: VirtualAddress и Size. Поэтому offset нужной записи вычисляется как 0x70 + 3 * 8 = 0x88. Проект именно так вычисляет dd_rel.

Важно: VirtualAddress из DataDirectory - это RVA, а не VA и не file offset. RVA нужно сначала сопоставить с секцией, и только потом можно получить file offset для чтения с диска. Size задает длину области Exception Directory в image coordinate system.

```
Формула Exception Directory entry:
dd_rel = 0x70 + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8
dd = opthdr + dd_rel
ExceptionRVA = read_u32(dd + 0)
ExceptionSize = read_u32(dd + 4)
Код: src/pe.c:99-137.
```

156.1. Проверка доступности DataDirectory[Exception]

Что проверяется: NumberOfRvaAndSizes должен быть больше индекса Exception Directory, а dd_rel + 8 должен помещаться в SizeOfOptionalHeader.

Для чего: Нельзя читать DataDirectory[3], если сам Optional Header заявил меньше записей или если SizeOfOptionalHeader обрезан. Иначе анализатор прочитает чужие байты и построит ложную модель.

Интерпретация: Если запись недоступна, это hard stop на PE parser stage. Если запись доступна, но пара RVA/Size странная, это уже может стать структурным evidence через exception_directory_oob или ошибку чтения runtime functions.

Код: src/pe.c:121-137.

157. Section table: как проект проверяет согласованность file layout и memory layout

Каждая секция имеет две координатные природы. На диске она описана PointerToRawData и SizeOfRawData. В памяти она описана VirtualAddress и VirtualSize. Эти диапазоны похожи по смыслу, но не обязаны быть одинаковыми. Именно поэтому проект проверяет raw-диапазоны и virtual-диапазоны отдельно.

Raw range отвечает на вопрос: какие байты реально лежат в файле. Virtual range отвечает на вопрос: какая область существует после загрузки image в память. Если VirtualSize больше SizeOfRawData, хвост секции в памяти будет заполнен нулями loader-ом. Статический анализатор не может читать этот хвост из файла, потому что физических байтов там нет.

Проект также проверяет alignment. FileAlignment задает выравнивание raw layout, SectionAlignment - выравнивание memory layout. Если секции не выровнены, пересекаются или выходят за SizeOfImage/file_size, то дальнейшие RVA mappings могут стать недостоверными.

157.1. Raw OOB

Формула: raw_end = section.raw_ptr + section.raw_size; если raw_size > 0 и raw_end > file_size, счетчик section_raw_oob увеличивается.

Зачем: Секция заявляет байты, которых нет в файле. Любое чтение данных из такой области будет либо невозможно, либо прочитает мусор.

Finding: pe_section_raw_oob, high, +25. Код счетчика: src/pe.c:198-204. Код finding: src/heuristics.c:789-795.

157.2. Virtual OOB

Формула: virtual_span = max(VirtualSize, SizeOfRawData); virtual_end = VirtualAddress + virtual_span; если virtual_end > SizeOfImage, section_virtual_oob увеличивается.

Зачем: SizeOfImage задает верхнюю границу загруженного image. Если секция выходит за нее, координатная модель образа противоречива.

Finding: pe_section_virtual_oob, high, +20. Код: src/pe.c:200-204; src/heuristics.c:796-802.

157.3. Raw/virtual overlaps

Формула: Диапазоны пересекаются, если a_begin < b_begin + b_size и b_begin < a_begin + a_size. Нулевые размеры не считаются пересечением.

Зачем: Две секции не должны одновременно владеть одними и теми же байтами файла или одной и той же областью image. При пересечении неясно, какая секция является владельцем RVA или file offset.

Findings: pe_section_raw_overlap, medium, +15; pe_section_virtual_overlap, high, +20. Код: src/pe.c:28-34, 215-226; src/heuristics.c:803-816.

157.4. Misalignment

Что входит: Проверяется power-of-two FileAlignment, допустимый диапазон 0x200..0x10000 или compact case, SectionAlignment >= FileAlignment, SizeOfImage кратен SectionAlignment, SizeOfHeaders кратен FileAlignment и не меньше section table, ImageBase выровнен на 0x10000, адреса и размеры секций согласованы с alignment, порядок VirtualAddress возрастает.

Зачем: Это инварианты loader-а и линкера. Если они нарушены, файл может быть специально искажен или нестандартно упакован, а вычисление layout становится менее надежным.

Finding: pe_section_misaligned, medium, +8. Низкий вес потому, что alignment может нарушаться у нестандартных packed/handmade PE и сам по себе не доказывает проблему unwind. Код: src/pe.c:180-196, 205-213; src/heuristics.c:817-824.

158. RVA -> file offset: самая важная техническая развилка проекта

RVA - это смещение относительно ImageBase в памяти. File offset - это смещение в файле на диске. VA - это реальный адрес в процессе после загрузки: VA = loaded_base + RVA. Статический парсер читает файл, поэтому ему нужен file offset, но DataDirectory и RUNTIME_FUNCTION хранят RVA. Поэтому проект постоянно переводит RVA в file offset.

Алгоритм pe_find_section_for_rva ищет секцию, у которой rva находится в [VirtualAddress, VirtualAddress + max(VirtualSize, SizeOfRawData)). Использование max делает mapping терпимым к типичным PE, где raw size и virtual size отличаются. Но pe_rva_to_fileoff затем дополнительно проверяет delta < raw_size. Если RVA попал только в virtual zero-fill хвост, функция возвращает false, потому что байтов на диске нет.

Если RVA не попал ни в одну секцию, проект отдельно проверяет headers fallback: rva < SizeOfHeaders и rva < file_size. Это нужно, потому что некоторые RVA законно указывают в область заголовков, а не в обычную секцию.

```
Алгоритм mapping:
s = pe_find_section_for_rva(img, rva)
if s exists:
delta = rva - s.VirtualAddress
if delta >= s.SizeOfRawData: fail, потому что это memory-only zero-fill
off = s.PointerToRawData + delta
if off >= file_size: fail
else if rva < SizeOfHeaders and rva < file_size:
off = rva
else:
fail
Код: src/pe.c:265-300.
```

158.1. Как интерпретировать fail в RVA mapping

Для Exception Directory: Если Exception Directory RVA не переводится в file offset, таблицу RUNTIME_FUNCTION нельзя прочитать из файла. Это приводит к malformed read и далее к pe_exception_directory_oob как совместимому finding-коду.

Для UNWIND_INFO: Если UnwindInfoRVA не переводится, конкретная функция получает parse fail. Потом heuristics_analyze считает unwind_parse_fail и создает finding unwind_parse_fail.

Почему это не просто 'файл плохой': RVA может быть допустимым в памяти, но не иметь raw-байтов в файле. Для статического анализа это значит: loader мог бы заполнить область нулями, но parser не имеет права выдумывать байты.

159. Exception Directory: как проект понимает, есть ли таблица функций

На x64 Exception Directory обычно является массивом RUNTIME_FUNCTION. Каждая запись занимает 12 байт: BeginAddress, EndAddress, UnwindInfoAddress. Эти поля являются RVA. Begin/End описывают диапазон функции, UnwindInfoAddress указывает на UNWIND_INFO.

Проект отдельно обрабатывает отсутствие каталога: если ExceptionRVA == 0 и ExceptionSize == 0, unwind_read_runtime_functions_ex возвращает OK с count = 0. Это не ошибка чтения. Позже heuristics решает, подозрительно ли отсутствие runtime functions: если есть executable section, создается no_runtime_functions, а если исполняемых секций нет, добавляется note о resource-only/forwarding image.

Если одно из полей нулевое, а другое нет, это malformed. Такая половинчатая запись не дает понять, то ли таблица отсутствует, то ли ее повредили. Поэтому чтение RUNTIME_FUNCTION прекращается как malformed, main.c ставит img.exception_directory_oob = true и продолжает частичный структурный отчет.

159.1. Проверки Exception Directory перед чтением entries

Пара RVA/Size: Оба нуля - таблицы нет. Один ноль - malformed. Оба ненулевые - нужно читать.

Mapping: ExceptionRVA переводится через pe_rva_to_fileoff. Если не переводится, таблица не читается.

File range: off + ExceptionSize должен помещаться в file_size. Иначе каталог заявляет больше байтов, чем реально есть в файле.

Минимальный размер: ExceptionSize должен быть хотя бы 12 байт, иначе не помещается даже одна RUNTIME_FUNCTION.

Кратность 12: ExceptionSize % 12 == 0. Если остаток не ноль, невозможно разбить каталог на целые записи.

Код: src/unwind.c:623-647; реакция main.c:306-323; finding src/heuristics.c:825-832.

> Пример:
> ExceptionRVA = 0x5000, ExceptionSize = 0x78.
> 0x78 = 120 байт. 120 / 12 = 10 RUNTIME_FUNCTION entries.
> Если Size был бы 0x7A, остаток от деления на 12 был бы 2 байта. Это не может быть корректным массивом записей.

160. RUNTIME_FUNCTION: как проверяется карта функций

После чтения Exception Directory проект получает массив RuntimeFunction. Сама возможность прочитать массив еще не означает, что записи корректны. Каждая запись должна описывать непустой диапазон функции внутри SizeOfImage, а UnwindInfoAddress должен указывать на существующую UNWIND_INFO.

Первый набор проверок делается за один проход по исходному порядку таблицы: EndAddress должен быть больше BeginAddress; Begin/End должны лежать в SizeOfImage; UnwindInfoAddress должен быть ненулевым и меньше SizeOfImage; BeginAddress в таблице должен идти неубывающе.

Второй набор проверок делается после сортировки копии таблицы по BeginAddress. Так проект отделяет две проблемы: 'таблица в файле не отсортирована' и 'диапазоны реально перекрываются'. Для overlap он хранит max_end предыдущих диапазонов и проверяет, начинается ли текущий диапазон раньше max_end.

160.1. Invalid range

Формула: Если EndAddress <= BeginAddress, диапазон пустой или обратный.

Для чего: Функция не может занимать отрицательное или нулевое пространство. Virtual unwind не сможет понять, какой код покрыт записью.

Finding: invalid_ranges. Если повреждено больше 1% записей, severity high и +20; иначе medium и +10. Код: src/heuristics.c:646-654, 850-856.

160.2. Begin/End out of SizeOfImage

Формула: b >= SizeOfImage или e > SizeOfImage.

Для чего: RUNTIME_FUNCTION описывает диапазон внутри image. Если функция выходит за SizeOfImage, Windows loader не должен считать это нормальным кодовым диапазоном модуля.

Finding: rf_oob. Больше 1% - high +25, иначе medium +10. Код: src/heuristics.c:655-658, 857-863.

160.3. UnwindInfoAddress out of bounds

Формула: u == 0 или u >= SizeOfImage.

Для чего: Запись функции без валидного UnwindInfoAddress не дает Windows x64 корректно выйти из кадра при исключении/stack walking.

Finding: unwind_rva_oob. Больше 1% - high +25, иначе medium +10. Код: src/heuristics.c:659-662, 864-870.

160.4. Sorting, duplicates, overlaps

Unsorted: В исходном порядке, если текущий BeginAddress меньше предыдущего BeginAddress, увеличивается rf_unsorted.

Duplicate begin: После сортировки, если два диапазона имеют один BeginAddress, увеличивается duplicate_begin.

Overlap: После сортировки, если текущий BeginAddress меньше max_end предыдущих диапазонов, увеличивается rf_overlaps.

Смысл: Windows lookup по function table ожидает упорядоченную и недвусмысленную карту функций. Перекрытия и дубликаты делают принадлежность RIP функции неоднозначной.

Findings: rf_unsorted medium +15; duplicate_begin medium +10; rf_overlaps high +25. Код: src/heuristics.c:664-773, 871-891.

161. UNWIND_INFO header: как 4 байта превращаются в модель пролога

UNWIND_INFO начинается с 4 байт. Первый байт хранит сразу два смысла: младшие 3 бита - version, старшие 5 бит - flags. Второй байт - SizeOfProlog. Третий - CountOfCodes. Четвертый тоже разделен: младшие 4 бита FrameRegister, старшие 4 бита FrameOffset, причем FrameOffset масштабируется на 16.

Проект сначала проверяет, что UnwindInfoRVA выровнен по 4 байтам. Невыровненность не останавливает разбор сама по себе, но ставит anomaly UA_ALIGNMENT. Затем RVA переводится в file offset. Если не переводится или не хватает 4 байт header-a, unwind_parse возвращает false, а конкретная запись позже попадет в unwind_parse_fail.

Версия должна быть 1 или 2. Неизвестная версия - это parse fail, потому что дальше неизвестна грамматика unwind codes. Flags не должны содержать биты кроме EHANDLER, UHANDLER, CHAININFO. CHAININFO не должен сочетаться с EHANDLER/UHANDLER, потому что хвост структуры имеет либо chained RUNTIME_FUNCTION, либо handler RVA, но не оба смысла сразу.

```
Bit layout UNWIND_INFO header:
byte0: bits 0..2 = Version, bits 3..7 = Flags
byte1: SizeOfProlog
byte2: CountOfCodes
byte3: bits 0..3 = FrameRegister, bits 4..7 = FrameOffset
Код: src/unwind.c:187-229.
```

161.1. Как трактуются ошибки header-a

Parse fail: Нечитаемый header, неизвестная version, обрезанный массив codes, обрезанный handler/chain - это unwind_parse_fail.

Anomaly: Незнакомые flags, конфликт flags, странный frame register/frame offset, нарушение ABI - это anomaly bits. Если parser все равно может безопасно продолжить, он не падает, а помечает запись.

Finding: unwind_parse_fail создается по числу записей us_ok == false; unwind_anomalies создается по числу записей с anomaly bits вне ABI-only mask. Код: src/heuristics.c:727-735, 893-905.

162. UNWIND_CODE array: как проект декодирует маленький байтовый язык

После header-a идет массив UNWIND_CODE. В файле каждый slot занимает 2 байта. Первый байт slot-a - CodeOffset, второй байт делится на UnwindOp и OpInfo. Но не каждая операция занимает один slot. Например UWOP_ALLOC_LARGE может занимать 2 или 3 slots, UWOP_SAVE_NONVOL - 2 slots, UWOP_SAVE_NONVOL_FAR - 3 slots. Поэтому проект сначала определяет slots_for_code, а затем проверяет, что операция не выходит за CountOfCodes.

CountOfCodes хранит число значимых slots, но на диске массив выравнивается до четного количества slots. Поэтому проект считает slots_even = (count + 1) & ~1 и codes_bytes = slots_even * 2. Это защищает чтение tail-a: handler RVA или chained RUNTIME_FUNCTION начинается не сразу после count slots, а после выровненного массива.

Проект проверяет CodeOffset относительно SizeOfProlog. Для обычных v1 operations CodeOffset не должен быть больше prolog size, иначе unwind code ссылается на действие за пределами пролога. Также проверяется порядок CodeOffset: если текущий offset больше предыдущего, ставится UA_CODEOFFSET_ORDER.

```
Slot decode:
slot = read_u16(codes_off + i * 2)
CodeOffset = slot & 0xff
op_byte = slot >> 8
UnwindOp = op_byte & 0x0f
OpInfo = op_byte >> 4
slots = slots_for_code(version, UnwindOp, OpInfo, ...)
if i + slots > CountOfCodes: parse fail
Код: src/unwind.c:231-285.
```

162.1. Как проект считает смысл unwind operations

Stack effect: PUSH_NONVOL добавляет 8 байт к fixed_stack, ALLOC_SMALL добавляет OpInfo * 8 + 8, ALLOC_LARGE читает operand и добавляет большой размер, PUSH_MACHFRAME добавляет 40 или 48 байт.

Saved registers: Операции save/push выставляют маски saved_nonvol_mask и saved_xmm_mask. Это не finding само по себе, а диагностическая модель: отчет может показать, какие регистры описывает unwind.

Operand sanity: Слишком странные размеры alloc/save помечаются UA_OPERAND_SANITY. Неверный register/opinfo - UA_BAD_OPERAND. Unknown opcode - UA_BAD_OPCODE, но parser пытается безопасно продолжить как 1 slot, если это возможно.

Код: src/unwind.c:24-54, 346-510.

163. Tail UNWIND_INFO: handler или chained info

После выровненного массива unwind codes находится хвост. Его смысл зависит от flags. Если установлен CHAININFO, хвост содержит еще одну структуру RUNTIME_FUNCTION размером 12 байт. Если установлен EHANDLER или UHANDLER, хвост содержит Handler RVA, а за ним могут идти language-specific data, которые проект не обязан полностью интерпретировать для текущей цели.

Проект не смешивает эти смыслы: CHAININFO вместе с EHANDLER/UHANDLER помечается как bad operand. Это важно, потому что иначе один и тот же набор байтов одновременно трактовался бы как chain и как handler.

При CHAININFO проект проверяет Begin/End/UnwindRVA chained записи на базовую валидность и при включенном follow_chain переходит по цепочке до max depth. Он хранит seen RVA, чтобы поймать loop, и ограничивает глубину, чтобы не уйти в бесконечный разбор.

163.1. Handler RVA

Как читается: Если flags содержит EHANDLER или UHANDLER, проект читает uint32_t handler_rva из after_codes.

Что проверяется: Если handler_rva >= SizeOfImage, ставится UA_HANDLER_OOB.

Как потом используется: heuristics проверяет секцию handler_rva. Handler должен указывать на исполняемый код; если секция неизвестна или не executable, это handler_nonexec_section.

Код: src/unwind.c:540-551; src/heuristics.c:714-723, 988-1000.

163.2. Chained info

Как читается: Если flags содержит CHAININFO, после codes читаются три DWORD как begin/end/unwind_rva chained RUNTIME_FUNCTION.

Что проверяется: End > Begin, Begin < SizeOfImage, End <= SizeOfImage, chain.unwind_rva != 0, chain.unwind_rva < SizeOfImage.

Интерпретация: CHAININFO нужен для описания chained unwind metadata. Ошибка здесь может ломать stack walking не только одной функции, но и цепочки восстановления.

Код: src/unwind.c:522-537, 559-604.

164. Placement checks: почему важно, в какой секции лежит metadata и handler

UNWIND_INFO - это metadata, а не исполняемый код. Обычно он лежит в .xdata, .rdata или иногда рядом с .pdata. Поэтому проект проверяет не только числовую валидность RVA, но и смысловую роль секции: имя секции и права памяти.

Если UNWIND_INFO лежит в executable section, это сильный сигнал: metadata находится там, где обычно лежат инструкции. Это может быть нормальным для некоторых ручных/упакованных layout, но для компиляторного PE необычно. Если UNWIND_INFO лежит в writable section, его можно менять во время выполнения, что повышает риск подмены unwind semantics.

Для handler-а логика обратная: handler - это код, значит он должен находиться в executable section. Если handler RVA указывает в non-executable или unknown section, это гораздо страннее, потому что обработчик должен выполняться процессором. Writable handler section тоже подозрительна, но менее жестко, потому что JIT/пакеры могут создавать writable/executable переходные области.

164.1. Как технически проверяются права

Флаги: IMAGE_SCN_MEM_EXECUTE = 0x20000000, IMAGE_SCN_MEM_WRITE = 0x80000000. Проект проверяет эти биты в section.characteristics.

UNWIND_INFO: pe_find_section_for_rva(img, unwind_rva). Если секции нет, или имя не похоже на .xdata/.rdata/.pdata, увеличивается unwind_unusual_section. Если execute bit есть - unwind_exec_section. Если write bit есть - unwind_writable_section.

Handler: pe_find_section_for_rva(img, handler_rva). Если секции нет или execute bit не установлен - handler_nonexec_section. Если write bit установлен - handler_writable_section.

Код: src/heuristics.c:690-723, 965-1000.

165. Reuse и dominance: как проект ищет слишком централизованные схемы

Reuse означает, что много RUNTIME_FUNCTION entries указывают на один и тот же UnwindInfoRVA. Само повторное использование не запрещено: leaf-like функции или одинаковые прологи могут иметь похожую metadata. Поэтому проект не делает finding от одного маленького повтора.

Проект считает hash map: ключ - UnwindInfoRVA, значение - сколько записей на него ссылаются. Затем вычисляет unwind_reuse_ratio = max_count / total и unwind_unique_ratio = unique / total. Для сильного reuse нужно больше 200 функций и ratio > 0.30. Для умеренного reuse нужно больше 200 функций, ratio > 0.10 и обязательно другой статический сигнал.

Handler dominance работает похоже, но только для записей, где UNWIND_INFO имеет handler. Считается handler_reuse_ratio = max_handler_count / handler_count. Finding появляется, если handler_count > 50 и ratio > 0.80, но есть смягчение: если доминирующий handler лежит в обычной code-like executable section и нет другого suspicious static context, finding не добавляется.

165.1. Почему стоят пороги 200, 10%, 30%, 50, 80%

n > 200: На малых выборках ratio легко становится большим случайно. Например 3 функции и 1 общий UnwindInfoRVA уже дают 33%, но это статистически слабый вывод.

10%: Это граница 'заметный паттерн', но он не используется отдельно: нужен другой сигнал. Поэтому moderate reuse не превращает обычный файл в подозрительный без корреляции.

30%: Это уже сильная концентрация: почти треть или больше функций смотрят на одну metadata-точку. Для нормального большого бинарника это требует объяснения.

handler_count > 50 и 80%: Handler dominance оценивается только если обработчиков достаточно много. 80% означает, что почти вся обработка сведена к одному dispatcher-like адресу.

Код: src/heuristics.c:630-685, 775-785, 908-962.

166. Scoring, severity, confidence и verdict: как технические счетчики становятся выводом

Finding создается функцией add_finding. У него есть code, family, severity, message, representative_rva и affected_count. representative_rva нужен, чтобы пользователь мог открыть конкретное место в PE-viewer/IDA/Ghidra и проверить не абстрактный вывод, а реальную точку.

Score ограничен диапазоном 0..100. Добавление веса идет через add_score, который использует clamp. Веса не являются математической вероятностью вредоносности. Это инженерная шкала важности: нарушения целостности таблиц и unreadable metadata получают большие веса, placement/reuse - средние, isolated alignment - малый.

Verdict в статике зависит от score: >=70 high_suspicion, >=40 moderate_suspicion, >=15 low_suspicion, иначе no_strong_static_evidence. Evidence level зависит от количества weak/medium/strong findings и score. Confidence строится отдельно: базовое значение, надбавки за число evidence families и сильные признаки, бонусы за hard table/unwind issues, штрафы за одиночные/малые/только placement сигналы.

```
Статика:
score >= 70 -> high_suspicion
score >= 40 -> moderate_suspicion
score >= 15 -> low_suspicion
score < 15 -> no_strong_static_evidence

Confidence:
base 0.15, zero-score base 0.20, +0.11 per family, +0.08 per strong, +0.04 per medium,
+0.08 за hard table issues, +0.08 за hard unwind issues,
-0.12 single non-strong signal, -0.06 small sample, -0.10 placement-only, -0.08 single family,
clamp 0.05..0.90.
Код: src/heuristics.c:31-84, 359-430, 504-588.
```

167. Dynamic checks: как статическая гипотеза проверяется выполнением

Динамический режим не заменяет статический анализ. Он отвечает на другой вопрос: проявляется ли подозрительная exception/unwind логика во время выполнения? Поэтому он запускается опционально через --dynamic и собирает runtime evidence.

На Windows проект вызывает CreateProcessA с DEBUG_ONLY_THIS_PROCESS. Далее он обрабатывает Debug API events: CREATE_PROCESS_DEBUG_EVENT, LOAD_DLL_DEBUG_EVENT, EXIT_PROCESS_DEBUG_EVENT, EXCEPTION_DEBUG_EVENT. На CREATE_PROCESS он узнает base_main - реальную базу загрузки главного image, и может перевести handler RVA в VA как base_main + handler_rva.

Проект ставит software breakpoints на API регистрации обработчиков: AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback. Когда breakpoint срабатывает, код читает CONTEXT потока. По Windows x64 calling convention первые аргументы лежат в RCX, RDX, R8, R9. Поэтому dynamic_win.c может записать handler pointer, table pointer, count, base address и callback.

Также проект ставит breakpoints на handler_rva, найденные из UNWIND_INFO. Если срабатывает handler breakpoint, это runtime corroboration: статически найденный handler реально исполнялся. После попадания проект может включить single-step trace на ограниченное число шагов, чтобы сохранить цепочку RIP.

167.1. Как dynamic evidence влияет на итог

Runtime levels: Handler hits считаются strong runtime signal, RtlAddFunctionTable/RtlInstallFunctionTableCallback и second-chance exceptions - medium, registrations и first-chance exceptions - weak.

Correlation: Если статически есть suspicious handler context и динамически есть handler hit/VEH/VCH/UEF hit, это корреляция. Если статически есть placement issues и динамически видны runtime function table calls, это тоже корреляция.

Final assessment: report.c добавляет confidence за complete dynamic session и за correlation, а final_verdict может стать high_suspicion_correlated/moderate_suspicion_correlated/low_suspicion_correlated.

Код: src/dynamic_win.c:657-715, 768-850, 852-1108; src/report.c:326-440.

168. Report: что именно видит пользователь как доказательство

В JSON отчет попадает не только verdict. Там есть report_metadata, input hash, image_base, size_of_image, size_of_headers, alignments, exception_directory, pe_validation counters, sections with rights, heuristic_analysis with all counters and findings, dynamic_validation with event list, trace, assessment и functions table.

В HTML отчет выводит итоговую оценку, план ручной проверки, статические признаки, правила снижения уверенности, журнал динамической проверки и таблицу RUNTIME_FUNCTION. Для каждой функции пользователь видит Begin, End, section, Unwind RVA, unwind section, status, version/flags, stack, handler и anomalies/error.

Это важно для защиты: проект не является 'черным ящиком'. Он не просто говорит suspicious. Он показывает путь доказательства: какое поле прочитано, какой счетчик вырос, какой finding создан, какой RVA является representative, в какой секции это лежит и какие runtime events это подтвердили или не подтвердили.

169. Паспорта всех 27 признаков: как проверяется, зачем нужно, как интерпретируется

Ниже каждый finding описан одинаково подробно. Это можно использовать как готовые ответы комиссии: где признак рождается технически, почему он важен, что он значит, какой у него вес и почему он не является безусловным обвинением сам по себе.

169.1. pe_section_raw_oob

Техническая проверка: Для каждой секции берутся PointerToRawData и SizeOfRawData. Считается raw_end = raw_ptr + raw_size в 64-битной арифметике. Если raw_size > 0 и raw_end > file_size, секция заявляет байты за пределами файла.

Для чего: Статический анализатор читает байты из файла. Если секция выходит за file_size, парсер не может честно читать данные, которые секция обещает. Это может ломать чтение таблиц, строк, кода или metadata.

Интерпретация: Один такой случай уже серьезен, потому что это нарушение физической раскладки файла. Но это не автоматическое 'вирус': иногда packed/handmade PE бывают нестандартными. Поэтому finding говорит о нарушении целостности формата.

Finding: family table_integrity, severity high, weight +25, affected_count = число секций. Код: src/pe.c:198-204; src/heuristics.c:789-795.

169.2. pe_section_virtual_oob

Техническая проверка: Для каждой секции считается virtual_span = max(VirtualSize, SizeOfRawData), virtual_end = VirtualAddress + virtual_span. Если virtual_span > 0 и virtual_end > SizeOfImage, секция выходит за границу image.

Для чего: SizeOfImage - верхняя граница виртуального образа, который loader должен создать в памяти. Если секция выходит за нее, любые RVA внутри хвоста оказываются вне заявленного image.

Интерпретация: Это нарушение memory layout. Оно особенно важно для unwind, потому что Begin/End/Unwind RVA сравниваются именно с SizeOfImage.

Finding: table_integrity, high, +20. Код: src/pe.c:200-204; src/heuristics.c:796-802.

169.3. pe_section_raw_overlap

Техническая проверка: Каждая секция сравнивается со всеми предыдущими по raw range. Пересечение: a_begin < b_begin + b_size и b_begin < a_begin + a_size.

Для чего: Raw overlap означает, что два section headers претендуют на одни и те же байты файла. Тогда одно и то же file offset может иметь две разные интерпретации.

Интерпретация: Это менее жестко, чем выход за файл, поэтому severity medium: некоторые packer-ы могут играть raw layout. Но для доказательной базы это важный сигнал неканоничности.

Finding: table_integrity, medium, +15. Код: src/pe.c:215-219; src/heuristics.c:803-808.

169.4. pe_section_virtual_overlap

Техническая проверка: Сравниваются virtual ranges секций: [VirtualAddress, VirtualAddress + max(VirtualSize, SizeOfRawData)). Если они пересекаются, увеличивается section_virtual_overlaps.

Для чего: Virtual overlap делает ownership RVA неоднозначным. Один RIP/RVA может попадать в две секции, а значит права и роль области становятся спорными.

Интерпретация: Для unwind это особенно плохо: проект должен понять, в какой секции лежит UnwindInfoAddress или handler. Если секции пересекаются, интерпретация прав может быть ненадежной.

Finding: table_integrity, high, +20. Код: src/pe.c:220-226; src/heuristics.c:810-816.

169.5. pe_section_misaligned

Техническая проверка: Счетчик увеличивается за нарушения FileAlignment/SectionAlignment, SizeOfImage % SectionAlignment, SizeOfHeaders % FileAlignment, слишком маленький SizeOfHeaders, невыровненный ImageBase, невыровненные raw/virtual поля секций, невозрастающий порядок VirtualAddress.

Для чего: Alignment - это договор между linker, loader и parser. Он говорит, где должны начинаться секции и как переводить layout с диска в память.

Интерпретация: Вес +8, потому что misalignment сам по себе часто слабее прямого OOB. Он говорит 'координатная система странная', но не обязательно 'unwind поврежден'.

Finding: table_integrity, medium, +8. Код: src/pe.c:180-196, 205-213; src/heuristics.c:817-824.

169.6. pe_exception_directory_oob

Техническая проверка: Сначала pe.c проверяет пару ExceptionRVA/ExceptionSize относительно SizeOfImage: если одно поле нулевое, другое нет, или rva + size > SizeOfImage, выставляется exception_directory_oob. Потом unwind_read_runtime_functions_ex дополнительно проверяет mappability, file range, minimum size и кратность 12. При malformed main.c также ставит img.exception_directory_oob = true.

Для чего: Exception Directory - вход в RUNTIME_FUNCTION table. Если он битый, нельзя надежно построить карту функций и дальше разбирать UNWIND_INFO.

Интерпретация: Это high +25, потому что поврежден сам каталог, от которого зависит весь x64 exception/unwind механизм. Но если каталог полностью отсутствует двумя нулями, это не этот finding, а отдельная логика no_runtime_functions/note.

Finding: table_integrity, high, +25. Код: src/pe.c:230-235; src/unwind.c:623-647; src/main.c:306-323; src/heuristics.c:825-832.

169.7. no_runtime_functions

Техническая проверка: Если rf_count == 0, ExceptionRVA == 0, ExceptionSize == 0 и в image есть executable section, heuristics добавляет finding.

Для чего: В x64 PE non-leaf функции обычно требуют unwind metadata. Полное отсутствие таблицы при наличии исполняемого кода может означать leaf-only image, ручную сборку, packer или неполный анализ.

Интерпретация: Severity medium и +10, потому что это не жесткая ошибка. Комментарий в коде прямо говорит: это может быть valid for leaf-only image, нужно confirm code complexity manually.

Finding: table_integrity, medium, +10. Код: src/heuristics.c:833-842.

169.8. invalid_ranges

Техническая проверка: Для каждой RUNTIME_FUNCTION проверяется EndAddress <= BeginAddress. Если да, rf_invalid_ranges увеличивается.

Для чего: Функция должна занимать непустой диапазон кода. Нулевой или обратный диапазон ломает поиск функции по RIP.

Интерпретация: Если таких записей больше 1% от таблицы, это уже не единичный сбой, severity high и +20. Иначе medium и +10.

Finding: table_integrity. Код: src/heuristics.c:651-654, 844-856.

169.9. rf_oob

Техническая проверка: Проверяется b >= SizeOfImage или e > SizeOfImage для BeginAddress/EndAddress.

Для чего: Диапазон функции должен находиться внутри загруженного image. Иначе таблица описывает код, которого нет в координатной системе модуля.

Интерпретация: Больше 1% - high +25; единичные - medium +10. Это сильнее invalid_ranges, потому что запись может указывать вне image, а значит на невозможный адрес модуля.

Finding: table_integrity. Код: src/heuristics.c:655-658, 857-863.

169.10. unwind_rva_oob

Техническая проверка: Для каждой записи проверяется u == 0 или u >= SizeOfImage.

Для чего: UnwindInfoAddress должен указывать на UNWIND_INFO внутри image. Без него Windows не сможет выполнить virtual unwind для этой функции.

Интерпретация: Больше 1% - high +25; иначе medium +10. Это прямой сигнал повреждения карты unwind metadata.

Finding: table_integrity. Код: src/heuristics.c:659-662, 864-870.

169.11. rf_unsorted

Техническая проверка: В исходном порядке таблицы проект сравнивает текущий BeginAddress с предыдущим. Если b < prev_begin, счетчик unsorted увеличивается.

Для чего: Function table обычно должна быть отсортирована по BeginAddress, чтобы lookup работал быстро и однозначно.

Интерпретация: Medium +15: нарушение важно, но проект затем сортирует копию и отдельно проверяет реальные overlaps/deduplicates, поэтому unsorted не смешивается с более тяжелыми проблемами.

Finding: table_integrity. Код: src/heuristics.c:664-669, 871-877.

169.12. rf_overlaps

Техническая проверка: После сортировки по BeginAddress проект ведет max_end предыдущих диапазонов. Если текущий begin < max_end, диапазоны перекрываются.

Для чего: Один RIP не должен принадлежать двум функциям одновременно. Overlap делает virtual unwind неоднозначным.

Интерпретация: High +25, потому что это уже не просто порядок таблицы, а конфликт смысла функции.

Finding: table_integrity. Код: src/heuristics.c:740-773, 878-884.

169.13. duplicate_begin

Техническая проверка: После сортировки проверяется, равен ли BeginAddress текущей записи BeginAddress предыдущей.

Для чего: Две функции с одинаковым началом не дают однозначного выбора RUNTIME_FUNCTION для RIP.

Интерпретация: Medium +10: дубликат начала подозрителен, но иногда может возникать из-за оберток/нестандартных таблиц, поэтому вес ниже overlap.

Finding: table_integrity. Код: src/heuristics.c:758-762, 885-891.

169.14. unwind_parse_fail

Техническая проверка: main.c вызывает unwind_parse для каждого UnwindInfoRVA. Если функция возвращает false, us_ok[i] = false. heuristics считает такие случаи как unwind_parse_fail.

Для чего: Если UNWIND_INFO не читается или не разбирается, таблица функций может существовать, но выйти из кадра корректно нельзя.

Причины: RVA не переводится в file offset, header обрезан, version неизвестна, codes array выходит за файл, opcode требует operands за CountOfCodes, handler/chain tail обрезан.

Интерпретация: Больше 1% - high +25; иначе medium +10. Код: src/main.c:351-356; src/unwind.c:189-239, 251-275, 522-551; src/heuristics.c:727-729, 893-899.

169.15. unwind_anomalies

Техническая проверка: Если unwind_parse завершился успешно, но выставил anomaly bits вне ABI-only mask, heuristics увеличивает unwind_anomalies.

Для чего: Parser может безопасно прочитать структуру, но заметить странности: неизвестные flags, конфликт CHAININFO с handler flags, CodeOffset вне prolog, плохой operand, chain loop, handler OOB, плохой порядок CodeOffset.

Интерпретация: Если аномалий больше 5% записей, severity high и +20; иначе medium +10. 5% означает широкий паттерн, а не единичную экзотику компилятора.

Finding: unwind_integrity. Код anomaly bits: include/unwind.h:52-70; src/unwind.c:211-229, 276-295, 346-551, 559-604; src/heuristics.c:730-735, 900-906.

169.16. unwind_reuse_strong

Техническая проверка: Hash map считает, сколько RUNTIME_FUNCTION entries ссылаются на каждый UnwindInfoRVA. Берется max_count и ratio = max_count / total. Finding появляется при total > 200 и ratio > 0.30.

Для чего: Слишком много функций, смотрящих в одну metadata-точку, может означать искусственную централизацию unwind описаний или dispatcher-like схему.

Интерпретация: High +20. Порог 30% выбран как сильная концентрация в большой таблице. Малые таблицы исключены условием n > 200.

Finding: unwind_reuse. Код: src/heuristics.c:630-685, 775-780, 908-918.

169.17. handler_dominance

Техническая проверка: Для всех parsed UNWIND_INFO с has_handler считается hash map HandlerRVA. ratio = max_handler_count / handler_count. Finding возможен при handler_count > 50 и ratio > 0.80.

Для чего: Если почти все обработчики указывают на один адрес, это похоже на централизованный обработчик/dispatcher, который управляет exception flow.

Смягчение: Если доминирующий handler находится в нормальной executable code-like секции и нет другого suspicious static context, finding не создается. Это защищает от ложных выводов по обычным runtime библиотекам.

Finding: handler_dominance, medium, +15. Код: src/heuristics.c:679-685, 782-785, 920-944.

169.18. unwind_reuse_moderate

Техническая проверка: moderate_reuse_candidate = unwind_reuse_ratio > 0.10 и n > 200. Но finding создается только если нет strong reuse и есть другой статический сигнал: handler_dominance, table integrity issue, parse fail, placement issue и т.п.

Для чего: 10% reuse может быть нормальным, если больше ничего странного нет. Поэтому проект требует корреляцию.

Интерпретация: Medium +10. Это пример аккуратной эвристики: сам по себе moderate reuse не обвиняет файл.

Finding: unwind_reuse. Код: src/heuristics.c:908-909, 946-962.

169.19. unwind_in_exec_section

Техническая проверка: Для каждого parsed UnwindInfoRVA находится секция. Если section.characteristics содержит IMAGE_SCN_MEM_EXECUTE, увеличивается unwind_exec_section.

Для чего: UNWIND_INFO - metadata. Обычно она должна лежать в read-only data, а не в исполняемом коде.

Интерпретация: High +20. Это не прямое доказательство вредоносности, но сильный placement anomaly: metadata размещена в области, назначение которой - инструкции.

Finding: metadata_placement. Код: src/heuristics.c:692-710, 965-971.

169.20. unwind_unusual_section

Техническая проверка: Если секция для UnwindInfoRVA не найдена, или имя секции не разрешено/не похоже на .xdata/.rdata/.pdata, увеличивается unwind_unusual_section.

Для чего: Имя секции не является абсолютной истиной, но оно дает semantic hint. Unwind metadata обычно лежит в специализированных или read-only data секциях.

Интерпретация: Если больше 5% записей - high +15, иначе medium +8. Низкий/средний вес защищает от ложных срабатываний у нестандартных линкеров.

Finding: metadata_placement. Код: src/heuristics.c:125-144, 692-703, 972-979.

169.21. unwind_writable_section

Техническая проверка: Если секция UnwindInfoRVA имеет IMAGE_SCN_MEM_WRITE, увеличивается unwind_writable_section.

Для чего: Writable metadata можно изменить во время выполнения. Для unwind это важно: изменение UNWIND_INFO меняет то, как Windows восстанавливает стек и находит handlers.

Интерпретация: Если больше 5% записей - high +18, иначе medium +10. Это сильнее unusual name, потому что право записи влияет на возможность подмены.

Finding: metadata_placement. Код: src/heuristics.c:704-710, 980-987.

169.22. handler_nonexec_section

Техническая проверка: Если UNWIND_INFO имеет handler, проект ищет секцию handler_rva. Если секции нет или нет IMAGE_SCN_MEM_EXECUTE, увеличивается handler_nonexec_section.

Для чего: Handler - это адрес кода обработчика. Код должен быть исполняемым. Non-executable handler target означает, что указатель не похож на нормальную функцию.

Интерпретация: High +15. Это placement issue, но с сильным смыслом: обработчик исключения находится не там, где должен находиться код.

Finding: metadata_placement. Код: src/heuristics.c:714-719, 988-994.

169.23. handler_writable_section

Техническая проверка: Если handler_rva лежит в секции с IMAGE_SCN_MEM_WRITE, увеличивается handler_writable_section.

Для чего: Обработчик исключений в writable section может быть легитимен для JIT/packer, но для обычного native PE это требует внимания.

Интерпретация: Medium +10. Вес ниже, чем handler_nonexec, потому что writable не запрещает исполнение, но повышает риск изменяемого control flow.

Finding: metadata_placement. Код: src/heuristics.c:720-723, 995-1000.

169.24. placement_exec_nonexec_handler_combo

Техническая проверка: Если одновременно unwind_exec_section > 0 и handler_nonexec_section > 0, добавляется combo finding.

Для чего: По отдельности каждое размещение уже странное. Вместе они образуют перевернутую картину: metadata лежит в executable области, а handler указывает в non-executable/unknown область.

Интерпретация: High +10 поверх отдельных сигналов. Combo показывает корреляцию внутри одного семейства evidence.

Finding: metadata_placement. Код: src/heuristics.c:1003-1007.

169.25. placement_integrity_combo

Техническая проверка: Если есть placement issue (unwind_exec_section или handler_nonexec_section) и одновременно integrity issue (unwind_parse_fail, unwind_anomalies или rf_unwind_oob), создается combo.

Для чего: Размещение может быть нестандартным, а integrity issue может быть случайной порчей. Вместе они усиливают друг друга: metadata не только лежит странно, но и плохо читается/нарушает структуру.

Интерпретация: Medium +8. Это не новый байтовый факт, а корреляционное правило поверх уже найденных фактов.

Finding: unwind_integrity. Код: src/heuristics.c:1008-1013.

169.26. reuse_dominance_combo

Техническая проверка: Если unwind_reuse_ratio > 0.10 и уже есть handler_dominance, создается combo finding.

Для чего: Один общий UnwindInfoRVA и один доминирующий handler вместе похожи на централизованную exception-dispatch схему.

Интерпретация: Medium +8. По смыслу это 'два независимых паттерна централизации совпали'.

Finding: unwind_reuse. Код: src/heuristics.c:1014-1018.

169.27. writable_metadata_handler_combo

Техническая проверка: Если одновременно unwind_writable_section > 0 и handler_writable_section > 0, создается combo.

Для чего: И metadata, и handler targets связаны с writable областями. Это необычно для обычного компиляторного PE и говорит о потенциально изменяемой exception/unwind логике.

Интерпретация: High +10. Combo не заменяет отдельные findings, а показывает, что writable pattern затрагивает оба уровня: описание unwind и обработчик.

Finding: metadata_placement. Код: src/heuristics.c:1019-1023.

170. Как объяснять комиссии, что проверка не сводится к 'нескольким if'

Формально каждая проверка действительно выражается условием: if raw_end > file_size, if End <= Begin, if UnwindInfoRVA >= SizeOfImage. Но сложность проекта не в наличии ключевого слова if. Сложность в том, что каждое условие стоит на правильном уровне модели и сравнивает значения в правильной координатной системе.

Например, сравнить ExceptionRVA напрямую с file_size было бы ошибкой, потому что ExceptionRVA - это RVA, а не file offset. Сначала нужно проверить SizeOfImage, затем найти секцию, затем проверить raw_size, затем получить file offset. Только после этого можно читать байты. Эта цепочка уже не 'просто if', а корректная модель PE loader semantics.

То же самое с UNWIND_INFO. CountOfCodes - это не просто количество байтов. Это количество 2-байтовых slots, причем физический массив выравнивается до четного числа slots, а каждая операция может занимать 1, 2 или 3 slots. Если это не учитывать, parser будет читать tail handler-а не с того места и сделает неверный вывод.

Еще один пример - reuse. Наивный анализ сказал бы: один UnwindInfoRVA повторился, значит подозрительно. В проекте сделано аккуратнее: считается ratio, вводится минимальная выборка n > 200, есть разные пороги 10% и 30%, moderate reuse требует корреляции, handler dominance имеет смягчение для нормальной executable code-like секции. Это уже evidence model, а не одиночный if.

Поэтому правильная формулировка защиты такая: проект реализует статико-динамический анализ EH/unwind слоя PE32+ x64. Его сложность заключается в сохранении корректной координатной модели PE, безопасном чтении недоверенного бинарного формата, разборе переменной структуры UNWIND_INFO, классификации аномалий по семействам evidence и корреляции статических признаков с runtime событиями.

171. Что считать нормальным, подозрительным и критичным результатом

Нормальный результат - это не обязательно нулевой отчет. У реального бинарника могут быть особенности компилятора, нестандартные секции или ABI warnings. Нормальность означает, что нет сильных нарушений целостности таблицы, UNWIND_INFO читается, RUNTIME_FUNCTION ranges лежат внутри image, handler targets исполняемы, а необычные placement/reuse сигналы не коррелируют друг с другом.

Подозрительный результат - это когда есть не один изолированный слабый признак, а связанная цепочка. Например: UnwindInfoAddress указывает за SizeOfImage, часть UNWIND_INFO не парсится, а metadata лежит в writable/executable section. В этом случае проект не просто суммирует числа: evidence families и confidence показывают, что разные уровни подтверждают проблему.

Критичный для ручной проверки результат - это high score или коррелированный static+dynamic case. Например, статически найден handler_nonexec_section, а динамически зафиксирован handler hit или регистрация runtime function table. Тогда аналитик получает не только предположение по файлу, но и runtime наблюдение, которое стоит проверить в отладчике/дизассемблере.

172. Короткая шпаргалка по технической интерпретации

Если сломаны MZ/PE/Machine/Magic - это не finding, а неподдерживаемый или невалидный вход. Проект не должен притворяться, что нашел unwind признаки там, где нет корректной x64 PE модели.

Если сломаны секции - это table_integrity evidence, потому что карта файла/образа ненадежна. Эти признаки отвечают на вопрос: можно ли доверять координатной системе?

Если сломана Exception Directory или RUNTIME_FUNCTION table - это table_integrity evidence более близкого к unwind уровня. Эти признаки отвечают на вопрос: можно ли доверять карте функций?

Если UNWIND_INFO не читается или содержит anomaly bits - это unwind_integrity evidence. Эти признаки отвечают на вопрос: можно ли корректно восстановить стек и обработку исключений?

Если metadata/handler лежат в странных секциях - это metadata_placement evidence. Эти признаки отвечают на вопрос: соответствует ли роль данных правам и секции?

Если много функций используют один UnwindInfoRVA или один handler - это reuse/dominance evidence. Эти признаки отвечают на вопрос: нет ли искусственной централизации exception/unwind flow?

Если static findings подтверждаются dynamic events - это уже не просто статическая гипотеза, а коррелированное наблюдение выполнения.

173. Подробный пошаговый алгоритм: как анализатор проверяет бинарник

Этот блок написан как учебный проход по одному запуску программы. Его смысл - показать не только список checks, а ход мысли анализатора: какая модель уже построена, чему можно доверять, что еще нужно доказать и где возникает каждый тип результата.

173.1. Получить путь к файлу

Что делает: main.c сначала разбирает аргументы. На этом уровне проект еще ничего не знает о PE. Он только понимает, какие режимы включены: надо ли печатать dump, писать JSON/HTML, запускать эвристики, включать динамику. Если опция неизвестна или число вне диапазона, это ошибка запуска, а не finding.

173.2. Прочитать файл полностью

Что делает: buf_init_from_file открывает файл в binary mode, получает размер через fseek/ftell, выделяет память и читает ровно size байт. Если файл не прочитан полностью, анализ прекращается: нельзя проверять частично прочитанный бинарник как будто он полный.

173.3. Включить режим недоверенного ввода

Что делает: После чтения каждый offset считается потенциально опасным. Даже если поле PE выглядит корректно, проект не использует прямое приведение указателей к структурам Windows. Он читает поля через buf_read_uXX, потому что файл может быть коротким, обрезанным или злонамеренно размеченным.

173.4. Доказать MZ

Что делает: Первые два байта должны быть 0x4D 0x5A в файле, что в little-endian читается как 0x5A4D. Это не 'признак подозрительности', а входное условие формата. Без MZ нет основания читать e_lfanew как PE offset.

173.5. Прочитать e_lfanew

Что делает: e_lfanew лежит по offset 0x3c. Он говорит, где начинается NT Headers. Проект проверяет, что e_lfanew + 4 + 20 помещается в файл: 4 байта PE signature и 20 байт COFF header. Так предотвращается чтение заголовка за пределами файла.

173.6. Доказать PE signature

Что делает: По e_lfanew читается DWORD и сравнивается с 0x00004550. Если там не PE\0\0, дальше нельзя читать COFF/Optional Header, потому что координатная модель не доказана.

173.7. Проверить архитектуру

Что делает: Из COFF header читается Machine. Проект требует IMAGE_FILE_MACHINE_AMD64 = 0x8664. Это важно: текущий анализатор проверяет x64 RUNTIME_FUNCTION/UNWIND_INFO, а не x86 SEH.

173.8. Проверить PE32+

Что делает: OptionalHeader.Magic должен быть 0x20b. Это означает 64-битный Optional Header. Если там 0x10b, это PE32, и offset-ы Optional Header отличаются. Нельзя просто продолжить с теми же смещениями.

173.9. Прочитать координаты image

Что делает: Из Optional Header читаются ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders, NumberOfRvaAndSizes. Эти поля становятся базой всех дальнейших сравнений. Например, SizeOfImage - это граница для RVA, а FileAlignment - граница для raw layout.

173.10. Проверить доступность Exception Directory entry

Что делает: Проект проверяет, что NumberOfRvaAndSizes позволяет читать index 3 и что 8 байт записи помещаются в Optional Header. Это защищает от ситуации, когда DataDirectory[3] читается из чужих байтов.

173.11. Прочитать section headers

Что делает: Количество секций берется из COFF header. Каждая секция занимает 40 байт. Проект проверяет переполнение при secthdr + nsects * 40 и сам выход за file_size. Затем читает имя, VirtualSize, VirtualAddress, RawSize, RawPtr, Characteristics.

173.12. Проверить секции как raw ranges

Что делает: Для каждой секции проверяется, не выходит ли raw range за файл. Это отвечает на вопрос: можно ли читать заявленные этой секцией байты с диска.

173.13. Проверить секции как virtual ranges

Что делает: Для каждой секции проверяется, не выходит ли virtual range за SizeOfImage. Это отвечает на вопрос: может ли loader разместить эту секцию внутри заявленного image.

173.14. Проверить пересечения секций

Что делает: Проект отдельно сравнивает raw ranges и virtual ranges. Raw overlap ломает файловую интерпретацию, virtual overlap ломает memory/RVA интерпретацию.

173.15. Проверить alignment и порядок

Что делает: Здесь проект проверяет не один факт, а группу инвариантов loader-а: кратности, power-of-two, compact alignment, порядок VirtualAddress, выравнивание ImageBase. Это нужно, чтобы дальнейший mapping не строился на противоречивой геометрии.

173.16. Оценить Exception Directory на уровне PeImage

Что делает: Если ExceptionRVA/Size не оба нулевые, проект считает exc_end = rva + size и сравнивает с SizeOfImage. Если rva или size нулевой по отдельности, либо конец за image, ставится exception_directory_oob.

173.17. Перевести ExceptionRVA в file offset

Что делает: unwind_read_runtime_functions_ex вызывает pe_rva_to_fileoff. Здесь проверяется, в какой секции лежит RVA, есть ли соответствующие raw bytes, не попадает ли RVA только в zero-fill хвост, или не находится ли RVA в headers.

173.18. Проверить размер Exception Directory

Что делает: После mapping проверяется off + size <= file_size, size >= 12 и size % 12 == 0. Только после этого можно вычислить count = size / 12.

173.19. Прочитать RUNTIME_FUNCTION entries

Что делает: Каждая запись читается как три DWORD. Проект не валидирует смысл прямо в reader-е, а только безопасно извлекает begin/end/unwind_rva. Смысловая проверка выполняется в heuristics, где уже есть вся таблица.

173.20. Разобрать каждую UNWIND_INFO

Что делает: main.c идет по rf_count и вызывает unwind_parse для каждого UnwindInfoRVA. Результат сохраняется в us[i], а успех/провал в us_ok[i]. Это дает параллельные массивы: таблица функций и summary unwind metadata.

173.21. Проверить header UNWIND_INFO

Что делает: unwind_parse проверяет alignment RVA, mapping, 4 байта header-а, version, flags, frame register/frame offset. Здесь parser решает, можно ли читать дальше, и какие anomaly bits нужно поставить.

173.22. Проверить массив unwind codes

Что делает: Считается выровненный размер массива slots_even. Для каждой операции вычисляется количество slots. Если операция требует operand slots за CountOfCodes, это parse fail. Если операция известна, но operand странный, это anomaly.

173.23. Проверить handler или chain tail

Что делает: После codes parser читает хвост. Если CHAININFO, хвост - RUNTIME_FUNCTION; если EHANDLER/UHANDLER, хвост - HandlerRVA. Проверяются границы, OOB и цепочки.

173.24. Посчитать счетчики heuristics

Что делает: heuristics_analyze за один проход считает invalid ranges, OOB, unsorted, parse failures, anomalies, placement, handler counts и hash maps reuse. Это промежуточные факты, еще не итоговый verdict.

173.25. Отсортировать копию function table

Что делает: Для overlap/deduplicate проверок проект создает отдельный массив begin/end и сортирует его. Исходную таблицу он не переписывает, потому что порядок в файле тоже evidence.

173.26. Превратить счетчики в findings

Что делает: После подсчета ratios и percentages код идет по правилам scoring. Если счетчик ненулевой и условия порога выполнены, создается finding с code, family, severity, message, affected_count и representative_rva.

173.27. Применить корреляционные правила

Что делает: Combo findings создаются не из новых байтов, а из совпадения уже найденных сигналов. Например placement_integrity_combo означает, что странное размещение совпало с проблемой читаемости/валидности unwind.

173.28. Рассчитать score, suspicion, confidence

Что делает: Score - сумма весов с ограничением 0..100. Suspicion = score / 100. Confidence считается отдельно: она растет от разных evidence families и сильных признаков, но уменьшается, если сигнал одиночный, маленькая выборка или только placement.

173.29. При необходимости запустить динамику

Что делает: Если включен --dynamic, программа запускает цель под Windows debugger, ставит breakpoints, собирает exception events, handler hits, VEH/VCH/UEF registrations, runtime function table calls и trace.

173.30. Сформировать отчет

Что делает: report.c выводит не только verdict, но и все данные для перепроверки: counters, sections, findings, representative RVA, dynamic events, functions table и manual validation steps.

174. Числовые примеры: как именно сравниваются адреса и размеры

174.1. Пример правильного RVA -> file offset

Пусть секция .pdata имеет VirtualAddress = 0x5000, VirtualSize = 0x1000, PointerToRawData = 0x2400, SizeOfRawData = 0x600. ExceptionRVA = 0x5120. Проект находит, что 0x5120 попадает в virtual range секции: 0x5000 <= 0x5120 < 0x6000.

Затем считается delta = 0x5120 - 0x5000 = 0x120. Так как delta < SizeOfRawData, то нужные байты реально существуют в файле. File offset = 0x2400 + 0x120 = 0x2520. Именно с offset 0x2520 можно читать массив RUNTIME_FUNCTION.

Если бы проект просто пошел в файл на offset 0x5120, он прочитал бы не ту область. Если бы он прибавил ImageBase, он получил бы VA, который нужен для runtime, но не для чтения файла.

```
VirtualAddress(.pdata) = 0x5000
PointerToRawData(.pdata) = 0x2400
RVA = 0x5120
delta = RVA - VirtualAddress = 0x120
file_offset = PointerToRawData + delta = 0x2520
```

174.2. Пример zero-fill хвоста

Пусть та же секция имеет VirtualSize = 0x1000, но SizeOfRawData = 0x600. RVA = 0x5900 попадает в virtual range: 0x5000..0x6000. Но delta = 0x900, а это больше или равно raw_size 0x600.

Это означает: область существует в памяти после загрузки, но соответствующих байтов в файле нет. Loader заполнит этот хвост нулями. Статический parser не имеет права читать файл по PointerToRawData + 0x900, потому что это уже не часть raw data секции.

Именно поэтому pe_rva_to_fileoff возвращает false. Если это был UnwindInfoRVA, конкретная UNWIND_INFO не читается и затем может стать unwind_parse_fail.

```
Virtual range: 0x5000..0x6000
Raw-backed part: 0x5000..0x5600
RVA 0x5900 находится в memory-only части.
Результат: mapping fail, потому что байтов на диске нет.
```

174.3. Пример Exception Directory size

RUNTIME_FUNCTION для x64 занимает 12 байт: DWORD BeginAddress, DWORD EndAddress, DWORD UnwindInfoAddress. Если ExceptionSize = 0x78, то это 120 байт, то есть 10 записей. Если ExceptionSize = 0x7A, то 122 байта, остаток от деления на 12 равен 2. Эти 2 байта не могут быть полноценной записью.

Проект не пытается угадывать, что означают лишние байты. Он фиксирует malformed read, main.c превращает это в structural report, а heuristics выдает pe_exception_directory_oob. Это честнее, чем молча отбросить остаток и скрыть повреждение.

174.4. Пример overlap functions

Пусть после сортировки есть функции [0x1000, 0x1100) и [0x1080, 0x1200). max_end после первой записи равен 0x1100. У второй begin = 0x1080, а 0x1080 < 0x1100. Значит диапазоны перекрываются.

Это плохо не потому, что числа 'некрасивые'. Это плохо потому, что RIP = 0x1090 одновременно попадает в две RUNTIME_FUNCTION записи. Virtual unwind должен выбрать одну запись, но таблица дает две конкурирующие интерпретации.

175. Как читать findings в отчете: не просто код, а доказательная цепочка

175.1. Что означает affected_count

affected_count - это количество затронутых объектов. Для section findings это число секций. Для rf_* это число RUNTIME_FUNCTION entries. Для unwind_* это число UNWIND_INFO или функций, у которых возникла проблема. Для reuse/dominance это max_count доминирующего RVA.

Важно не путать affected_count с score. Например affected_count = 1 может дать high severity, если это критическое нарушение формата, а affected_count = 20 может дать medium, если это мягкий placement pattern. Score зависит не только от количества, но и от смысла нарушения.

175.2. Что означает representative_rva

representative_rva - это не все адреса, а первый или характерный адрес, который можно открыть руками. Он нужен, чтобы пользователь не получил абстрактное 'есть проблема', а мог перейти к конкретной записи: секции, BeginAddress, UnwindInfoRVA или HandlerRVA.

Например для unwind_rva_oob representative_rva будет проблемным UnwindInfoAddress. Для handler_nonexec_section это будет handler_rva. Для section issues это будет VirtualAddress проблемной секции. Так отчет связывает finding с реальным местом в бинарнике.

175.3. Что означает family

family показывает слой доказательства. table_integrity - проблема карты PE или RUNTIME_FUNCTION table. unwind_integrity - проблема разбора UNWIND_INFO. metadata_placement - проблема секции/прав размещения metadata или handler. unwind_reuse - статистическая концентрация UnwindInfoRVA. handler_dominance - концентрация handler targets.

Это нужно, чтобы не складывать все признаки в одну кучу. Один finding из одного family слабее, чем несколько findings из разных families, потому что разные families подтверждают проблему с разных сторон.

175.4. Что означает severity

severity - это качественная важность finding. High обычно означает нарушение, которое сильно влияет на корректность PE/unwind модели: выходы за границы, overlaps, parse failures, исполняемая секция для metadata, non-executable handler. Medium означает важный, но более интерпретируемый сигнал: unsorted, duplicate, moderate reuse, writable handler.

Severity не равна verdict. Verdict считается из score, evidence families, confidence и динамической корреляции. Поэтому high finding может требовать ручной проверки, но не обязан автоматически означать 'вредоносный файл'.

176. Почему разные проверки имеют разные веса

Вес +25 получают признаки, которые ломают базовую возможность доверять структуре: raw OOB, Exception Directory OOB, RF OOB при массовости, unwind_rva_oob при массовости, RF overlaps, массовый unwind_parse_fail. Эти проблемы мешают корректно читать или применять таблицы.

Вес около +20 получают сильные нарушения, которые важны, но могут быть чуть менее абсолютными: virtual OOB, virtual overlap, invalid_ranges при массовости, unwind_anomalies при массовости, strong unwind reuse, unwind_in_exec_section.

Вес +8..+15 получают признаки, которые часто требуют контекста: misalignment, raw overlap, duplicate begin, handler dominance, unusual section, writable section, combo medium. Они важны как часть цепочки, но не должны одиночно превращать любой нестандартный PE в сильный verdict.

Эти числа назначает разработчик как инженерную calibration model. Они не претендуют на ML-вероятность. Их задача - сделать вывод воспроизводимым, прозрачным и защищаемым: одинаковый файл при одинаковой версии инструмента получает одинаковую оценку.

177. Какие ошибки начинающего разработчика предотвращает проект

Ошибка 1: читать PE структуры прямым cast-ом. Проект этого не делает, потому что cast не проверяет границы файла, alignment и endian conversion.

Ошибка 2: путать RVA и file offset. Проект переводит RVA через section table и отдельно учитывает headers fallback и zero-fill хвост.

Ошибка 3: считать, что DataDirectory.Size гарантирует валидность содержимого. Проект сначала проверяет размер области, потом каждую RUNTIME_FUNCTION, потом каждую UNWIND_INFO.

Ошибка 4: считать VirtualSize и SizeOfRawData одним и тем же. Проект использует virtual range для принадлежности RVA секции, но raw_size для разрешения чтения байтов из файла.

Ошибка 5: считать любое unusual placement вредоносным. Проект вводит веса, пороги, mitigation и correlation, чтобы отделить слабый одиночный сигнал от сильной цепочки.

Ошибка 6: игнорировать dynamic behavior. Проект умеет подтвердить часть статических гипотез runtime событиями, но не делает динамику обязательной, потому что не каждый sample безопасно/возможно запускать.

178. Как вручную перепроверить вывод проекта

Если finding про секции, открой PE viewer и проверь section table: RawPtr, RawSize, VirtualAddress, VirtualSize, Characteristics, SizeOfImage, SizeOfHeaders, FileAlignment, SectionAlignment.

Если finding про Exception Directory, открой Optional Header -> DataDirectory[Exception], запиши RVA и Size, переведи RVA в file offset через секцию .pdata и проверь, что Size кратен 12.

Если finding про RUNTIME_FUNCTION, отсортируй BeginAddress, проверь EndAddress > BeginAddress, отсутствие overlaps/deduplicates, попадание Begin/End/UnwindInfoAddress внутрь SizeOfImage.

Если finding про UNWIND_INFO, перейди по UnwindInfoAddress, проверь первые 4 байта header-a, CountOfCodes, выровненный массив slots, tail handler/chain, flags и operands.

Если finding про placement, посмотри секцию representative_rva и ее права. Для UnwindInfo ожидай data/read-only секцию, для handler ожидай executable code section.

Если finding про reuse/dominance, выгрузи таблицу в CSV и сгруппируй по UnwindInfoRVA или HandlerRVA. Сравни max_count с total/handler_count и проверь, есть ли другие признаки, подтверждающие подозрение.

Если есть dynamic events, проверь event list: какие API были вызваны, какие handler hits были, какие адреса попали в trace, завершилась ли instrumentation complete.

Итог для защиты: каждый finding нужно объяснять не как 'программа нашла подозрительность', а как 'программа проверила конкретный инвариант формата или поведения, получила конкретное нарушение, записала его в счетчик, присвоила вес по силе доказательства, показала representative RVA и дала пользователю возможность перепроверить это вручную'.

КАРТА ПРОВЕРОК ПО СТРОКАМ КОДА

Этот раздел специально добавлен после твоего замечания. Здесь каждая важная проверка описана предельно прямым языком и с конкретными строками исходников: где поле читается, где условие проверяется, где счетчик сохраняется, где создается finding, где это попадает в отчет. Это не замена предыдущих разделов, а более прикладная карта для защиты и самостоятельного чтения кода.

Главное правило чтения этого раздела: finding появляется не в момент чтения байтов, а позже. Обычно путь такой: pe.c/unwind.c читает и проверяет структуру, heuristics.c считает счетчики и создает finding, report.c выводит это в JSON/HTML. Поэтому у многих признаков несколько ссылок на код, а не одна.

179. Как читать ссылки на строки

179.1. Четыре места одной проверки

1. Константа или структура: Например IMAGE_NT_OPTIONAL_HDR64_MAGIC = 0x20b лежит в include/pe.h:19, а поля PeImage со счетчиками лежат в include/pe.h:48-53.

2. Чтение и первичная проверка: Например section header fields читаются в src/pe.c:156-172, а raw/virtual ranges проверяются в src/pe.c:198-226.

3. Превращение в finding: Например pe_section_raw_oob создается не в pe.c, а в src/heuristics.c:789-795, потому что именно heuristics layer решает, какой код finding, severity и вес дать.

4. Вывод в отчет: JSON-поля PE validation выводятся в src/report.c:673-680, heuristic counters и findings - в src/report.c:701-778, функции и unwind details - в src/report.c:864-914.

```
Пример цепочки pe_section_raw_oob:
include/pe.h:48 - поле section_raw_oob в PeImage
src/pe.c:163-167 - чтение RawSize/RawPtr из section header
src/pe.c:198-204 - raw_end > file_size, счетчик raw_oob++
src/pe.c:250 - out->section_raw_oob = raw_oob
src/heuristics.c:605 - копирование в HeuristicReport
src/heuristics.c:789-795 - add_score + add_finding('pe_section_raw_oob')
src/report.c:673-680, 714-719 - вывод счетчиков в JSON
```

180. Ранние проверки: они не findings, но без них нельзя доверять анализу

180.1. Файл должен быть полностью прочитан

Где в коде: src/main.c:283-288 вызывает buf_init_from_file. src/util.c:13-35 открывает файл, измеряет размер, выделяет память и читает ровно столько байт. Простыми словами: проект сначала кладет весь файл в память. Если файл не прочитан полностью, дальше нельзя честно проверять PE: половина заголовков может просто отсутствовать.

Как трактуется: Это не finding из 27, потому что finding относится к PE/unwind модели, а здесь модель еще не создана. При провале программа завершает анализ входа.

180.2. Любое чтение байтов проверяется на границы

Где в коде: src/util.c:42-70: buf_read проверяет off <= size и len <= size - off, затем buf_read_u16/u32/u64 читают little-endian числа. Простыми словами: каждый раз, когда код хочет взять поле из файла, он сначала спрашивает: эти байты вообще есть в файле?

Как трактуется: Если чтение ломается на раннем PE header, это hard stop. Если ломается внутри Exception Directory или UNWIND_INFO, это уже может привести к pe_exception_directory_oob или unwind_parse_fail.

180.3. MZ и e_lfanew

Где в коде: include/pe.h:15 задает PE_DOS_MAGIC. src/pe.c:11-18 читает MZ и e_lfanew. src/pe.c:40-48 вызывает это из pe_parse и проверяет, что PE header offset не выходит за файл. Простыми словами: e_lfanew говорит, где искать PE-заголовок; если он указывает за конец файла, дальше читать нечего.

Как трактуется: Не finding. Это проверка 'перед нами вообще PE-подобный файл или нет'.

180.4. PE signature

Где в коде: include/pe.h:16 задает PE_NT_MAGIC. src/pe.c:50-54 читает DWORD по e_lfanew и сравнивает с PE\0\0. Простыми словами: после DOS-заголовка проект проверяет, что по указанному адресу действительно начинается PE-заголовок.

Как трактуется: Не finding. Если подписи нет, нельзя читать COFF/Optional Header как PE.

180.5. AMD64 machine

Где в коде: include/pe.h:18 задает IMAGE_FILE_MACHINE_AMD64 = 0x8664. src/pe.c:56-70 читает Machine из COFF header и требует AMD64. Простыми словами: проект сделан для x64 unwind. У x86 другой механизм исключений, поэтому

продолжать тем же алгоритмом было бы ошибкой.

Как трактуется: Не finding. Это ограничение области применения проекта.

180.6. PE32+ magic 0x20b

Где в коде: include/pe.h:19 задает IMAGE_NT_OPTIONAL_HDR64_MAGIC = 0x20b. src/pe.c:72-90 проверяет Optional Header bounds и Magic. Простыми словами: 0x20b означает 64-битный Optional Header. Если там 0x10b, смещения полей другие, и x64 RUNTIME_FUNCTION/UNWIND_INFO логика неприменима.

Как трактуется: Не finding. Это фильтр поддерживаемого формата.

180.7. Ключевые поля Optional Header

Где в коде: src/pe.c:92-124 читает ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders, NumberOfRvaAndSizes и проверяет нули/границы. Простыми словами: проект получает линейку, по которой дальше будет мерить все RVA и секции.

Как трактуется: Если эти базовые поля невозможны, анализ останавливается: без SizeOfImage и alignments нельзя честно проверять границы.

180.8. DataDirectory[Exception]

Где в коде: src/pe.c:121-137 проверяет, что DataDirectory[3] доступна, вычисляет dd_rel = 0x70 + 3 * 8, читает ExceptionRVA и ExceptionSize. Простыми словами: проект не угадывает, где .pdata, а берет официальный указатель из Optional Header.

Как трактуется: Если entry недоступна, это hard stop. Если entry доступна, но ее значения противоречивы, это позже может стать pe_exception_directory_oob.

180.9. Section headers

Где в коде: src/pe.c:139-172 проверяет размер таблицы секций и читает поля каждой секции: name, VirtualSize, VirtualAddress, RawSize, RawPtr, Characteristics. Простыми словами: проект строит карту 'что где лежит на диске' и 'что где будет в памяти'.

Как трактуется: Если section headers не читаются, невозможно переводить RVA в file offset и невозможно надежно оценивать права секций.

181. PE structural findings: где именно они появляются в коде

181.1. pe_section_raw_oob

Где в коде: src/pe.c:163-167 читает raw_size и raw_ptr. src/pe.c:198-204 считает raw_end = raw_ptr + raw_size и проверяет raw_end > file_size. src/pe.c:250 сохраняет счетчик в PeImage. src/heuristics.c:605 копирует счетчик в HeuristicReport. src/heuristics.c:789-795 создает finding pe_section_raw_oob. Вес задан в src/heuristics.c:32. JSON вывод: src/report.c:673-680 и 714.

Понятно по смыслу: Простыми словами: секция обещает, что ее байты лежат в файле, но конец этих байтов находится дальше конца файла. Это как если в оглавлении книги написано 'глава на страницах 900-950', а книга заканчивается на странице 300.

Как интерпретировать: Это high finding, потому что анализатор не может читать байты, которых нет. Это нарушение физической раскладки файла.

181.2. pe_section_virtual_oob

Где в коде: src/pe.c:200-204 считает virtual_span = max(VirtualSize, RawSize), virtual_end = VirtualAddress + virtual_span и сравнивает с SizeOfImage. src/pe.c:251 сохраняет счетчик. src/heuristics.c:606 копирует его. src/heuristics.c:796-802 создает finding. Вес: src/heuristics.c:33.

Понятно по смыслу: Простыми словами: секция говорит 'после загрузки я буду занимать такую-то область памяти', но эта область вылезает за общий размер образа SizeOfImage.

Как интерпретировать: Это high finding, потому что SizeOfImage - главная граница загруженного PE-образа.

181.3. pe_section_raw_overlap

Где в коде: src/pe.c:28-34 содержит формулу пересечения диапазонов. src/pe.c:215-219 сравнивает raw ranges секций. src/pe.c:252 сохраняет счетчик. src/heuristics.c:607 копирует. src/heuristics.c:803-808 создает finding. Вес: src/heuristics.c:34.

Понятно по смыслу: Простыми словами: две секции указывают на одни и те же байты файла. Получается, один кусок файла одновременно считается содержимым двух секций.

Как интерпретировать: Severity medium: это странно и важно, но некоторые packer-ы могут специально играть raw layout, поэтому вес ниже, чем у выхода за файл.

181.4. pe_section_virtual_overlap

Где в коде: src/pe.c:28-34 - формула overlap. src/pe.c:220-226 сравнивает virtual ranges. src/pe.c:253 сохраняет счетчик. src/heuristics.c:608 копирует. src/heuristics.c:810-816 создает finding. Вес: src/heuristics.c:35.

Понятно по смыслу: Простыми словами: две секции претендуют на одну и ту же область памяти после загрузки. Тогда один RVA может принадлежать сразу двум секциям.

Как интерпретировать: Это high finding, потому что права секции и роль данных становятся неоднозначными.

181.5. pe_section_misaligned

Где в коде: src/pe.c:180-196 проверяет FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders, ImageBase. src/pe.c:205-213 проверяет выравнивание каждой секции и порядок VirtualAddress. src/pe.c:254 сохраняет счетчик. src/heuristics.c:609 копирует. src/heuristics.c:817-824 создает finding. Вес: src/heuristics.c:36.

Понятно по смыслу: Простыми словами: секции должны начинаться и иметь размеры по правилам выравнивания. Если эти правила нарушены, карта файла похожа на криво размеченную.

Как интерпретировать: Severity medium и вес +8, потому что это часто вспомогательный сигнал. Он говорит 'координатная система подозрительная', но сам по себе слабее прямого OOB.

181.6. pe_exception_directory_oob

Где в коде: src/pe.c:230-235 проверяет ExceptionRVA/ExceptionSize против SizeOfImage. src/pe.c:255 сохраняет флаг. src/unwind.c:623-647 дополнительно проверяет отсутствие/половинчатость каталога, mapping RVA, file range, size >= 12, size % 12 == 0. src/main.c:306-323 при malformed read ставит img.exception_directory_oob = true. src/heuristics.c:610 копирует, src/heuristics.c:825-832 создает finding. Вес: src/heuristics.c:37.

Понятно по смыслу: Простыми словами: Exception Directory - это вход в таблицу функций .pdata. Если вход указывает не туда, обрезан или размер не делится на 12, проект не может честно читать RUNTIME_FUNCTION.

Как интерпретировать: Это high finding, потому что ломается сам вход в x64 unwind-карту функций.

181.7. no_runtime_functions

Где в коде: src/unwind.c:623-624 возвращает OK с count=0, если ExceptionRVA=0 и ExceptionSize=0. src/heuristics.c:833-842 проверяет: если rf_count=0, каталог отсутствует и есть executable section, создается no_runtime_functions. Вес: src/heuristics.c:31.

Понятно по смыслу: Простыми словами: у файла есть исполняемый код, но статической x64 function table нет. Это может быть нормально для leaf-only/особого файла, но для обычного сложного x64 EXE требует ручной проверки.

Как интерпретировать: Severity medium: это не поломка таблицы, а отсутствие ожидаемой карты функций.

182. RUNTIME_FUNCTION findings: где проверяется карта функций

RUNTIME_FUNCTION entries читаются в src/unwind.c:654-667. Но смысл этих записей проверяется позже в src/heuristics.c:646-773. Это сделано специально: reader только безопасно извлекает массив, а heuristics layer уже оценивает таблицу целиком.

182.1. invalid_ranges

Где в коде: src/heuristics.c:646-654 берет b=BeginAddress, e=EndAddress и проверяет e <= b. src/heuristics.c:844 считает процент invalid_ranges_pct. src/heuristics.c:850-856 создает finding invalid_ranges. Порог >1% задан в src/heuristics.c:80. Веса: src/heuristics.c:38-39.

Понятно по смыслу: Простыми словами: конец функции должен быть дальше начала. Если конец равен началу или меньше, такой диапазон не описывает нормальную функцию.

Как интерпретировать: Если таких записей больше 1%, это уже массовая проблема таблицы, severity high. Если единично - medium.

182.2. rf_oob

Где в коде: src/heuristics.c:655-658 проверяет b >= SizeOfImage или e > SizeOfImage. src/heuristics.c:845 считает процент. src/heuristics.c:857-863 создает rf_oob. Порог >1%: src/heuristics.c:80. Веса: src/heuristics.c:40-41.

Понятно по смыслу: Простыми словами: таблица функций говорит, что функция лежит за пределами загруженного PE-образа. Такого кода внутри модуля быть не должно.

Как интерпретировать: Это сильный сигнал, потому что функция указывает за главную границу image.

182.3. unwind_rva_oob

Где в коде: src/heuristics.c:659-662 проверяет u == 0 или u >= SizeOfImage. src/heuristics.c:846 считает процент. src/heuristics.c:864-870 создает finding. Веса: src/heuristics.c:42-43.

Понятно по смыслу: Простыми словами: RUNTIME_FUNCTION нашла функцию, но адрес ее UNWIND_INFO пустой или за пределами image. Значит Windows не сможет взять нормальную инструкцию 'как выйти из кадра'.

Как интерпретировать: Больше 1% - high; единично - medium. Это проверка связки function table -> unwind metadata.

182.4. rf_unsorted

Где в коде: src/heuristics.c:664-669 сравнивает текущий BeginAddress с предыдущим в исходном порядке таблицы. src/heuristics.c:738 сохраняет rf_unsorted. src/heuristics.c:871-877 создает finding. Вес: src/heuristics.c:44.

Понятно по смыслу: Простыми словами: таблица функций обычно должна идти по возрастанию адресов начала. Если порядок скачет назад, поиск функции по RIP может быть менее надежным или таблица сделана нестандартно.

Как интерпретировать: Medium: это нарушение порядка, но проект отдельно проверяет более серьезные overlaps после сортировки.

182.5. rf_overlaps

Где в коде: src/heuristics.c:740-773 создает отсортированную копию begin/end и проверяет begin < max_end. src/heuristics.c:878-884 создает rf_overlaps. Вес: src/heuristics.c:45.

Понятно по смыслу: Простыми словами: две функции перекрываются по адресам. Один RIP может попасть сразу в две записи, и непонятно, какую UNWIND_INFO применять.

Как интерпретировать: High, потому что это конфликт смысла function table, а не просто порядок.

182.6. duplicate_begin

Где в коде: src/heuristics.c:758-762 после сортировки проверяет одинаковые BeginAddress. src/heuristics.c:885-891 создает finding duplicate_begin. Вес: src/heuristics.c:46.

Понятно по смыслу: Простыми словами: две записи говорят, что начинаются в одной и той же точке. Это делает карту функций неоднозначной.

Как интерпретировать: Medium: проблема важная, но обычно слабее настоящего overlap.

183. UNWIND_INFO findings: где проверяется сама инструкция раскрутки стека

183.1. unwind_parse_fail

Где в коде: src/main.c:351-356 вызывает unwind_parse для каждой rf[i].unwind_rva и пишет результат в us_ok[i]. src/unwind.c:56-72 переводит UnwindInfoRVA в file offset и читает 4 байта header. src/unwind.c:187-239 проверяет alignment, version, flags, frame fields и размер codes array. src/unwind.c:251-285 читает slots и проверяет выход op за CountOfCodes. src/unwind.c:346-510 читает operands операций. src/unwind.c:522-551 читает CHAININFO или handler tail. src/heuristics.c:727-729 считает failures. src/heuristics.c:893-899 создает finding. Веса: src/heuristics.c:47-48.

Понятно по смыслу: Простыми словами: RUNTIME_FUNCTION указала на UNWIND_INFO, но проект не смог безопасно разобрать эту структуру. Либо адрес не переводится в файл, либо структура обрезана, либо операция требует байты, которых нет.

Как интерпретировать: Это не 'косметика'. Если UNWIND_INFO не читается, невозможно надежно понять, как восстановить стек для этой функции.

183.2. unwind_anomalies

Где в коде: include/unwind.h:52-70 перечисляет anomaly bits. src/unwind.c:189-229 ставит alignment/version/flags/frame anomalies. src/unwind.c:276-295 проверяет CodeOffset/prolog/order/ABI restrictions. src/unwind.c:346-510 проверяет operands и stack effects. src/unwind.c:522-604 проверяет chain, handler, loops, depth. src/heuristics.c:730-735 считает anomaly entries. src/heuristics.c:900-906 создает finding. Порог >5%: src/heuristics.c:81. Веса: src/heuristics.c:49-50.

Понятно по смыслу: Простыми словами: структура вроде читается, но внутри есть странные комбинации. Например флаги противоречат друг другу, CodeOffset выходит за размер пролога, handler указывает за image, chain зацикливается.

Как интерпретировать: Если аномалий больше 5%, это уже широкая проблема, high. Если меньше - medium, потому что единичные необычности могут быть от компилятора или ручной сборки.

184. Placement findings: где проверяется роль секции и права памяти

Здесь проект задает простой вопрос: соответствует ли адрес своей роли? UNWIND_INFO - это metadata, поэтому ожидается data/read-only область. Handler - это код, поэтому ожидается executable область. Технически это проверяется через pe_find_section_for_rva и биты IMAGE_SCN_MEM_EXECUTE/WRITE.

184.1. Общая проверка секции по RVA

Где в коде: src/pe.c:265-274 ищет секцию, покрывающую RVA. src/heuristics.c:7-12 задает IMAGE_SCN_MEM_EXECUTE и IMAGE_SCN_MEM_WRITE. src/heuristics.c:690-723 выполняет все placement counters.

Понятно по смыслу: Простыми словами: проект берет адрес metadata или handler-а, ищет, в какую секцию он попал, и смотрит, какие у секции права.

184.2. unwind_in_exec_section

Где в коде: src/heuristics.c:692-710 находит секцию UnwindInfoRVA и если стоит IMAGE_SCN_MEM_EXECUTE, увеличивает unwind_exec_section. src/heuristics.c:965-971 создает finding. Вес: src/heuristics.c:54.

Понятно по смыслу: UNWIND_INFO - не код, а служебное описание. Если оно лежит в исполняемой секции, это странно: секция предназначена для инструкций, а не для metadata.

184.3. unwind_unusual_section

Где в коде: src/heuristics.c:125-144 задает логику допустимых/похожих имен секций. src/heuristics.c:692-703 увеличивает unwind_unusual_section, если секция не найдена или имя не похоже на .xdata/.rdata/.pdata. src/heuristics.c:972-979 создает finding. Веса: src/heuristics.c:55-56.

Понятно по смыслу: Простыми словами: если UNWIND_INFO лежит не там, где обычно лежит unwind metadata, это повод проверить. Вес мягче, потому что имя секции не является строгим правилом.

184.4. unwind_writable_section

Где в коде: src/heuristics.c:704-710 проверяет IMAGE_SCN_MEM_WRITE у секции UnwindInfoRVA. src/heuristics.c:980-987 создает finding. Веса: src/heuristics.c:57-58.

Понятно по смыслу: Простыми словами: если metadata можно менять во время работы, то можно менять и то, как программа описывает раскрутку стека. Это подозрительнее, чем просто необычное имя секции.

184.5. handler_nonexec_section

Где в коде: src/heuristics.c:714-719 проверяет handler_rva: если секции нет или нет execute bit, увеличивает handler_nonexec_section. src/heuristics.c:988-994 создает finding. Вес: src/heuristics.c:59.

Понятно по смыслу: Handler - это функция, которая должна выполняться. Если адрес обработчика указывает в неисполняемую область, это сильная странность.

184.6. handler_writable_section

Где в коде: src/heuristics.c:720-723 проверяет write bit у секции handler_rva. src/heuristics.c:995-1000 создает finding. Вес: src/heuristics.c:60.

Понятно по смыслу: Простыми словами: обработчик лежит в области, которую можно менять. Это не всегда ошибка, но для обычного PE требует внимания.

185. Reuse и dominance: где считаются проценты и почему это не магия

185.1. unwind_reuse_strong

Где в коде: src/heuristics.c:630-685 создает hash map и считает частоты UnwindInfoRVA. src/heuristics.c:775-780 берет max_count, dominant_unwind_rva, unique и считает unwind_reuse_ratio = max_count / total. src/heuristics.c:82-83 задает пороги 10% и 30%. src/heuristics.c:910-918 создает unwind_reuse_strong при ratio > 0.30 и n > 200. Вес: src/heuristics.c:52.

Понятно по смыслу: Простыми словами: если очень много функций указывают на один и тот же UNWIND_INFO, проект считает это централизацией metadata. Условие n > 200 нужно, чтобы маленькие файлы не давали ложные большие проценты.

185.2. handler_dominance

Где в коде: src/heuristics.c:679-685 считает частоты handler_rva. src/heuristics.c:782-785 считает handler_reuse_ratio. Порог 80% задан в src/heuristics.c:84. src/heuristics.c:920-944 создает handler_dominance, но с защитой: если доминирующий handler выглядит как нормальный code-like executable target и нет другого suspicious context, finding не создается. Вес: src/heuristics.c:53.

Понятно по смыслу: Простыми словами: если почти все обработчики исключений ведут в один адрес, это похоже на единый dispatcher. Но проект не обвиняет такой случай вслепую, если он похож на нормальный runtime handler.

185.3. unwind_reuse_moderate

Где в коде: src/heuristics.c:908-909 создает candidate при ratio > 0.10 и n > 200. src/heuristics.c:946-962 создает finding только если есть другой статический сигнал. Вес: src/heuristics.c:51.

Понятно по смыслу: Простыми словами: 10% reuse сам по себе слабоват. Поэтому проект требует подтверждение другим признаком: table issue, parse fail, placement issue или handler dominance.

186. Combo findings: где проект говорит 'два слабых/средних факта вместе сильнее'

186.1. placement_exec_nonexec_handler_combo

Где в коде: src/heuristics.c:1003-1007: если unwind_exec_section > 0 и handler_nonexec_section > 0, создается combo. Вес: src/heuristics.c:61.

Понятно по смыслу: Простыми словами: metadata лежит в executable секции, а handler указывает в non-executable/unknown секцию. По отдельности странно, вместе еще страннее.

186.2. placement_integrity_combo

Где в коде: src/heuristics.c:1008-1013: если есть placement issue и одновременно unwind_parse_fail/unwind_anomalies/rf_unwind_oob, создается combo. Вес: src/heuristics.c:62.

Понятно по смыслу: Простыми словами: мало того, что metadata лежит странно, она еще и плохо читается или указывает не туда.

186.3. reuse_dominance_combo

Где в коде: src/heuristics.c:1014-1018: если unwind_reuse_ratio > 0.10 и есть handler_dominance, создается combo. Вес: src/heuristics.c:63.

Понятно по смыслу: Простыми словами: один общий UNWIND_INFO и один доминирующий handler вместе похожи на централизованную exception-dispatch схему.

186.4. writable_metadata_handler_combo

Где в коде: src/heuristics.c:1019-1023: если metadata writable и handler writable, создается combo. Вес: src/heuristics.c:64.

Понятно по смыслу: Простыми словами: и описание unwind, и обработчик связаны с изменяемыми областями. Для обычного компиляторного PE это выглядит необычно.

187. Score, confidence и verdict: точные строки

187.1. Где заданы веса и пороги

Веса: src/heuristics.c:31-64. Здесь лежат все SCORE_*. Это таблица инженерной важности признаков.

Пороги: src/heuristics.c:80-84. >1% для массовых table issues, >5% для широких unwind anomalies, 10/30% для reuse, 80% для handler dominance.

Простыми словами: Цифры не 'из воздуха в момент запуска'. Они зашиты разработчиком как воспроизводимая шкала: один и тот же бинарник на этой версии инструмента получит один и тот же результат.

187.2. Где создается итоговый verdict

Статика: src/heuristics.c:514-588: считается evidence_level, confidence, mitigation penalties, requires_manual_review и verdict по score: >=70 high, >=40 moderate, >=15 low.

Динамика и финальная оценка: src/report.c:326-440: dynamic_signal_count, has_static_dynamic_correlation, runtime_evidence_level, assessment_summary и final_verdict.

Простыми словами: Score отвечает 'сколько и насколько сильных статических сигналов'. Confidence отвечает 'насколько этому выводу можно доверять'. Dynamic correlation отвечает 'подтвердилось ли это поведением при запуске'.

188. Dynamic checks: где проект проверяет поведение при запуске

188.1. Включение динамики из main

Где в коде: src/main.c:401-424 вызывает dynamic_collect_windows, печатает exceptions_seen, unwind_handler_hits, veh_hits, dyn_ftable_calls. Простыми словами: динамика запускается только если пользователь включил --dynamic.

188.2. Запуск процесса под Debug API

Где в коде: src/dynamic_win.c:657-715 строит command line и вызывает CreateProcessA с DEBUG_ONLY_THIS_PROCESS. Простыми словами: проект запускает цель как отладчик, чтобы видеть события исключений и загрузки модулей.

188.3. Модули и breakpoints

Где в коде: src/dynamic_win.c:789-817 на CREATE_PROCESS/LOAD_DLL обновляет список модулей и ставит API breakpoints; src/dynamic_win.c:798-808 ставит breakpoints на handler_rva из UNWIND_INFO. Простыми словами: статически найденные handler-ы превращаются в runtime точки наблюдения.

188.4. События исключений

Где в коде: src/dynamic_win.c:832-850 записывает обычные exceptions, first/second chance и адрес. Простыми словами: проект видит, где реально произошли исключения во время выполнения.

188.5. Попадания в breakpoints и чтение регистров

Где в коде: src/dynamic_win.c:852-1108 обрабатывает EXCEPTION_BREAKPOINT/SINGLE_STEP, читает CONTEXT, берет RCX/RDX/R8/R9 для Windows x64 API arguments, фиксирует VEH/VCH/UEF/RtlAddFunctionTable и handler hits. Простыми словами: проект не угадывает аргументы API, а берет их из регистров по calling convention.

189. Отчет: где найденные проверки становятся видимыми пользователю

189.1. JSON PE validation

Где в коде: src/report.c:622-680 пишет metadata, image_base, SizeOfImage, alignments, Exception Directory и pe_validation counters. Простыми словами: пользователь видит не только verdict, но и исходные координаты анализа.

189.2. JSON heuristic analysis

Где в коде: src/report.c:701-778 пишет score, suspicion, confidence, requires_manual_review, verdict, evidence_level, все counters, findings, affected_count, representative_rva, representative_section и weakening rules. Простыми словами: каждый finding можно проследить до числа и RVA.

189.3. JSON dynamic validation

Где в коде: src/report.c:781-845 пишет dynamic counters, events и trace. Простыми словами: если динамика запускалась, отчет хранит не только итог, но и события выполнения.

189.4. JSON assessment и functions

Где в коде: src/report.c:847-914 пишет final assessment и таблицу functions с begin/end/unwind/handler/anomalies. Простыми словами: можно открыть каждую функцию и увидеть ее UNWIND_INFO summary.

189.5. HTML report

Где в коде: src/report.c:1151-1501 формирует HTML: итоговая оценка, статические признаки, mitigation, dynamic block, таблица RUNTIME_FUNCTION. Простыми словами: это человекочитаемая оболочка над теми же evidence, что есть в JSON.

190. Очень простая формула объяснения комиссии

Если тебя спрашивают 'где это в коде?', отвечай по цепочке: 'Поле читается в pe.c/unwind.c, счетчик сохраняется в PeImage или UnwindSummary, finding создается в heuristics.c, а в отчет попадает через report.c'. Это честнее, чем называть одну строку, потому что проверка в проекте реально распределена по слоям.

Шаблон ответа:
1. Сначала проект читает поле через безопасный buf_read, чтобы не выйти за файл.
2. Потом сравнивает это поле с границей: file_size, SizeOfImage, SizeOfHeaders, raw_size, CountOfCodes или правами секции.
3. Если условие нарушено, увеличивается конкретный счетчик.
4. heuristics.c смотрит на счетчик, пороги и контекст и создает finding с code/severity/weight.
5. report.c выводит finding, affected_count и representative_rva, чтобы аналитик мог перепроверить руками.

191. Все микропроверки UNWIND_INFO: что именно внутри считается ошибкой

UNWIND_INFO - самая сложная часть проекта, потому что это не одна структура фиксированного размера. Это заголовок, массив 2-байтовых slots переменной длины, operands разной длины и хвост, который меняет смысл в зависимости от flags. Поэтому ниже перечислены именно микропроверки parser-а: какие дают parse fail, какие дают anomaly, а какие потом превращаются в findings.

191.1. UnwindInfoRVA должен переводиться в file offset

Где в коде: src/unwind.c:56-72 вызывает pe_rva_to_fileoff. Если RVA не попал в raw-backed область файла, parser не может прочитать 4 байта header.

Какой вывод делает проект: Результат: unwind_parse возвращает false, us_ok[i]=false, затем heuristics.c:727-729 считает unwind_parse_fail, а heuristics.c:893-899 создает finding unwind_parse_fail.

191.2. UnwindInfoRVA должен быть выровнен на 4

Где в коде: src/unwind.c:187-191 проверяет (unwind_rva & 3u) != 0 и ставит UA_ALIGNMENT.

Какой вывод делает проект: Результат: это anomaly, а не немедленный parse fail. Простыми словами: структура читается, но адрес выглядит неправильно выровненным.

191.3. Header должен иметь 4 байта в файле

Где в коде: src/unwind.c:63-70 проверяет off + 4 <= file_size и buf_read header bytes.

Какой вывод делает проект: Результат: parse fail, потому что без первых 4 байт невозможно узнать version, flags, prolog, count и frame fields.

191.4. Version должна быть 1 или 2

Где в коде: src/unwind.c:197-214 извлекает version из младших 3 бит первого байта и проверяет ver == 1 || ver == 2.

Какой вывод делает проект: Результат: неизвестная version дает UA_VERSION_UNKNOWN и parse fail. Простыми словами: parser не знает грамматику такой версии.

191.5. Flags не должны иметь неизвестные биты

Где в коде: src/unwind.c:197-219 извлекает flags и проверяет маску разрешенных флагов EHANDLER, UHANDLER, CHAININFO.

Какой вывод делает проект: Результат: неизвестные биты дают UA_BAD_OPERAND. Parser может продолжить, потому что базовый layout еще понятен, но структура отмечается как подозрительная.

191.6. CHAININFO нельзя смешивать с EHANDLER/UHANDLER

Где в коде: src/unwind.c:218-220 проверяет конфликт flags.

Какой вывод делает проект: Результат: UA_BAD_OPERAND. Простыми словами: хвост UNWIND_INFO не может одновременно означать chained RUNTIME_FUNCTION и handler RVA.

191.7. FrameOffset без FrameRegister бессмысленен

Где в коде: src/unwind.c:221-223 проверяет fr == 0 && foff != 0.

Какой вывод делает проект: Результат: UA_BAD_OPERAND. Простыми словами: offset кадра указан, но не указан регистр, от которого этот offset считать.

191.8. FrameRegister должен быть допустимым регистром

Где в коде: src/unwind.c:224-229 проверяет fr через is_unwindable_gpr и при strict_abi через is_abi_nonvolatile_gpr.

Какой вывод делает проект: Результат: обычное нарушение дает UA_BAD_OPERAND, strict ABI нарушение дает UA_ABI_REGISTER_CLASS. ABI-only warnings отдельно считаются в heuristics.c:734-735.

191.9. Массив unwind codes должен помещаться в файл

Где в коде: src/unwind.c:231-239 считает slots_even = (count + 1) & ~1, codes_bytes = slots_even * 2 и проверяет codes_off + codes_bytes <= file_size.

Какой вывод делает проект: Результат: UA_TRUNCATED и parse fail. Простыми словами: CountOfCodes заявил массив, но физически этих байтов в файле нет.

191.10. Каждый 2-байтовый slot должен читаться

Где в коде: src/unwind.c:251-258 читает slot через read_u16_slot.

Какой вывод делает проект: Результат: если slot не читается, UA_TRUNCATED и parse fail.

191.11. Opcode должен иметь известную длину в slots

Где в коде: src/unwind.c:24-54 задает slots_for_code для UWOP_*. src/unwind.c:265-270 вызывает slots_for_code.

Какой вывод делает проект: Результат: неизвестный opcode ставит UA_BAD_OPCODE. Parser пытается безопасно продолжить как 1 slot, чтобы не падать на первом странном байте.

191.12. Операция не должна выходить за CountOfCodes

Где в коде: src/unwind.c:271-275 проверяет i + slots > count.

Какой вывод делает проект: Результат: UA_TRUNCATED и parse fail. Простыми словами: операция говорит 'мне нужны дополнительные operand slots', но массив уже закончился.

191.13. CodeOffset не должен выходить за prolog

Где в коде: src/unwind.c:276-285 для обычных не-v2 descriptor операций проверяет code_offset > prolog.

Какой вывод делает проект: Результат: UA_CODEOFFSET_OUTSIDE. Это anomaly: запись читается, но говорит о действии за пределами пролога.

191.14. CodeOffset должен идти в ожидаемом порядке

Где в коде: src/unwind.c:281-285 проверяет, не стал ли текущий code_offset больше предыдущего.

Какой вывод делает проект: Результат: UA_CODEOFFSET_ORDER. Простыми словами: порядок unwind actions выглядит не таким, как ожидается для корректного описания пролога.

191.15. Strict ABI push order

Где в коде: src/unwind.c:287-294 при strict_abi проверяет порядок PUSH_NONVOL и ограничения CHAININFO.

Какой вывод делает проект: Результат: UA_ABI_PUSH_ORDER или UA_ABI_CHAIN_RESTRICTION. Эти биты относятся к ABI warnings, а не всегда к основным suspicious anomalies.

191.16. UWOP_ALLOC_LARGE должен иметь operands

Где в коде: src/unwind.c:357-380 читает operands для mode 0 и mode 1; при нехватке bytes возвращает parse fail; при странном размере ставит UA_OPERAND_SANITY.

Какой вывод делает проект: Результат: parse fail при обрезке, anomaly при подозрительном размере allocation.

191.17. UWOP_SAVE_NONVOL / FAR должны иметь operands и допустимый регистр

Где в коде: src/unwind.c:394-428 читает save offsets, проверяет register class и zero offsets при strict_abi.

Какой вывод делает проект: Результат: parse fail при обрезке operand, UA_BAD_OPERAND/UA_ABI_* при странном регистре или offset.

191.18. XMM save operations должны иметь operands и допустимый XMM register

Где в коде: src/unwind.c:429-501 обрабатывает SAVE_XMM, SAVE_XMM_FAR, SAVE_XMM128, SAVE_XMM128_FAR.

Какой вывод делает проект: Результат: parse fail при обрезке, anomaly при плохом XMM/opinfo/offset.

191.19. PUSH_MACHFRAME opinfo должен быть 0 или 1

Где в коде: src/unwind.c:503-506 проверяет opinfo > 1.

Какой вывод делает проект: Результат: UA_BAD_OPERAND. Простыми словами: операция машинного кадра имеет только два документированных варианта размера.

191.20. FrameRegister требует SET_FPREG

Где в коде: src/unwind.c:515-517 проверяет fr != 0 && !saw_set_fpreg && !CHAININFO.

Какой вывод делает проект: Результат: UA_BAD_OPERAND. Простыми словами: header говорит 'есть frame register', но unwind codes не описали установку этого регистра.

191.21. CHAININFO tail должен читаться как RUNTIME_FUNCTION

Где в коде: src/unwind.c:519-537 читает chained RuntimeFunction и проверяет Begin/End/UnwindRVA против SizeOfImage.

Какой вывод делает проект: Результат: parse fail при обрезке хвоста, UA_CHAIN_INVALID при плохих значениях.

191.22. Handler tail должен иметь HandlerRVA

Где в коде: src/unwind.c:540-551 читает handler_rva и проверяет handler_rva >= SizeOfImage.

Какой вывод делает проект: Результат: parse fail при обрезке handler RVA, UA_HANDLER_OOB при выходе handler за image.

191.23. Follow-chain не должен зациклиться

Где в коде: src/unwind.c:559-604 хранит seen chain unwind RVA, ограничивает depth max_depth и вызывает unwind_parse для следующего chain элемента.

Какой вывод делает проект: Результат: UA_CHAIN_LOOP при повторе RVA, UA_CHAIN_DEPTH_EXCEEDED при превышении глубины, UA_TRUNCATED если chained unwind не парсится.

191.24. Как микропроверки становятся двумя findings

Где в коде: src/heuristics.c:727-735 разделяет результат: если us_ok[i] == false, это unwind_parse_fail; если parse OK, но есть anomaly bits вне UA_ABI_MASK, это unwind_anomalies.

Какой вывод делает проект: Результат: десятки микропроверок не превращаются в десятки отдельных findings, чтобы отчет не стал шумным. Они агрегируются в два понятных признака: не смогли разобрать или разобрали, но нашли аномалии.

192. Полная логика интерпретации: от проверки к выводу

192.1. Hard stop

Когда бывает: Файл не читается, нет MZ, e_lfanew за файлом, нет PE signature, Machine не AMD64, Magic не 0x20b, Optional Header/section headers не помещаются.

Как проект это трактует: Это не finding. Проект говорит: вход не является поддерживаемой PE32+ x64 моделью. Нельзя честно делать выводы про EH/unwind, если сам формат не доказан.

Где в коде: src/main.c:283-299; src/pe.c:36-148.

192.2. Structural counter

Когда бывает: Секция вышла за file_size/SizeOfImage, секции пересеклись, alignment нарушен, Exception Directory противоречива.

Как проект это трактует: pe.c строит PeImage и кладет счетчики в поля PeImage. Это еще не финальный вывод, но уже evidence о качестве координатной модели.

Где в коде: include/pe.h:48-53; src/pe.c:174-255.

192.3. Runtime-function counter

Когда бывает: Begin/End плохие, функция выходит за SizeOfImage, UnwindInfoAddress пустой/вне image, таблица не отсортирована, диапазоны пересекаются, begin дублируется.

Как проект это трактует: Это evidence о карте функций. Если карта функций плохая, stack walking и virtual unwind становятся недостоверными.

Где в коде: src/heuristics.c:646-773.

192.4. Unwind parser result

Когда бывает: UNWIND_INFO либо не разобралась, либо разобралась с anomaly bits.

Как проект это трактует: parse fail - сильнее, потому что структура не читается. anomaly - мягче, потому что структура читается, но внутри есть странности.

Где в коде: src/unwind.c:165-608; src/heuristics.c:727-735, 893-906.

192.5. Placement result

Когда бывает: UNWIND_INFO в executable/writable/unusual section; handler в non-executable/writable section.

Как проект это трактует: Это не всегда ошибка формата, а смысловая странность: роль адреса не совпадает с ролью секции. Поэтому placement findings часто требуют ручной проверки и корреляции.

Где в коде: src/heuristics.c:690-723, 965-1000.

192.6. Statistical pattern

Когда бывает: Много функций ссылаются на один UnwindInfoRVA или много handler entries ведут в один HandlerRVA.

Как проект это трактует: Это не OOB и не parse error, а паттерн концентрации. Поэтому есть пороги n > 200, 10%, 30%, handler_count > 50, 80%, а moderate reuse требует другой сигнал.

Где в коде: src/heuristics.c:630-685, 775-785, 908-962.

192.7. Final verdict

Когда бывает: После всех counters/findings score и confidence собираются в итоговую оценку.

Как проект это трактует: Вердикт не говорит 'это точно вирус'. Он говорит, насколько сильна evidence-модель по EH/unwind слою и нужна ли ручная проверка.

Где в коде: src/heuristics.c:514-588; src/report.c:405-440.

МАТРИЦА РЕШЕНИЙ И ДЕЙСТВИЙ ПОСЛЕ ПРОВЕРОК

Этот раздел отвечает именно на вопрос: что проект делает после результата проверки. Проверка сама по себе - это только сравнение. Важнее, какое решение принимает программа: остановиться, продолжить, увеличить счетчик, создать finding, снизить или повысить confidence, потребовать ручную проверку, добавить dynamic evidence или вывести это в отчет.

В проекте есть несколько типов результата. Не каждый плохой результат сразу становится finding. Ранние ошибки формата дают hard stop. Структурные нарушения PE становятся счетчиками. Ошибки UNWIND_INFO становятся parse status или anomaly bits. Findings создаются позже, в heuristics.c, когда уже есть контекст и можно оценить силу сигнала.

193. Типы решений после проверки

193.1. Hard stop

Когда возникает: Проверка провалилась до построения PE-модели: файл не прочитан, нет MZ/PE, неподдерживаемая архитектура, неподдерживаемый Magic, заголовки не помещаются в файл.

Что делает программа: Программа завершает анализ входа и выводит ошибку. Findings не создаются, потому что еще нет корректной x64 PE/unwind модели. Код: src/main.c:283-299; src/pe.c:36-148.

Какой вывод делает проект: Вывод: объект не подходит для данного анализатора или поврежден до уровня, где x64 EH/unwind анализ невозможен.

Что делать человеку: Действие аналитика: проверить, тот ли файл выбран, PE ли это вообще, x64 ли это, не обрезан ли файл.

193.2. Continue with structural counter

Когда возникает: PE-модель построена, но обнаружено нарушение секций или Exception Directory: OOB, overlap, misalignment, malformed directory.

Что делает программа: Программа не падает. Она сохраняет счетчики в PeImage, затем heuristics.c превращает их в findings и score. Код: src/pe.c:174-255; src/heuristics.c:605-610, 789-832.

Какой вывод делает проект: Вывод: координатная система файла сомнительна, но анализ можно продолжить настолько, насколько данные доступны.

Что делать человеку: Действие аналитика: открыть PE headers/sections и вручную проверить raw/virtual ranges, SizeOfImage, DataDirectory[Exception].

193.3. Continue with partial structural report

Когда возникает: Exception Directory есть, но ее нельзя прочитать как корректный массив RUNTIME_FUNCTION: RVA не мапится, file range выходит за файл, size < 12, size не кратен 12.

Что делает программа: unwind_read_runtime_functions_ex возвращает malformed; main.c ставит img.exception_directory_oob = true, пишет 'Continuing with a partial structural report' и продолжает с rf_count=0. Код: src/unwind.c:623-647; src/main.c:306-323.

Какой вывод делает проект: Вывод: таблица функций недоступна или повреждена. Проект все равно сохраняет PE-level evidence, чтобы отчет не был пустым.

Что делать человеку: Действие аналитика: проверить DataDirectory[Exception].RVA/Size и перевод RVA в file offset.

193.4. Continue with per-function parse status

Когда возникает: RUNTIME_FUNCTION прочитаны, но отдельная UNWIND_INFO не разобралась.

Что делает программа: main.c сохраняет us_ok[i] = false. heuristics.c считает unwind_parse_fail и создает finding. Остальные записи продолжают анализироваться. Код: src/main.c:351-356; src/heuristics.c:727-729, 893-899.

Какой вывод делает проект: Вывод: не вся таблица сломана, но конкретные функции имеют неразборчивую unwind metadata.

Что делать человеку: Действие аналитика: смотреть representative UnwindInfoRVA, parse_error и соответствующую запись RUNTIME_FUNCTION.

193.5. Continue with anomaly bits

Когда возникает: UNWIND_INFO читается, но внутри странные поля: flags, CodeOffset, chain, handler, operands.

Что делает программа: unwind_parse выставляет anomaly bits, но возвращает true. heuristics.c считает unwind_anomalies, если anomaly не только ABI warning. Код: src/unwind.c:187-608; src/heuristics.c:730-735, 900-906.

Какой вывод делает проект: Вывод: структура физически читается, но ее смысл нетипичен или противоречив.

Что делать человеку: Действие аналитика: сравнить anomaly text с дизассемблированным прологом и секцией handler-a.

193.6. Create finding and score

Когда возникает: Счетчик/условие достаточно значимы: section issue, table issue, parse fail, placement issue, reuse/dominance, combo.

Что делает программа: heuristics.c вызывает add_score и add_finding, сохраняет code, family, severity, message, affected_count, representative_rva. Код: src/heuristics.c:408-430, 789-1023.

Какой вывод делает проект: Вывод: это уже не просто внутреннее наблюдение, а признак для отчета и итоговой оценки.

Что делать человеку: Действие аналитика: читать finding вместе с family, severity, affected_count и representative_rva.

193.7. Apply mitigation

Когда возникает: Сигнал найден, но он одиночный, слабый, placement-only или выборка мала.

Что делает программа: finalize_verdict снижает confidence и добавляет mitigation message. Код: src/heuristics.c:531-573.

Какой вывод делает проект: Вывод: проект не хочет преувеличивать слабый сигнал. Подозрительность есть, но уверенность ниже.

Что делать человеку: Действие аналитика: не делать сильный вывод по одному слабому finding, искать корреляцию.

193.8. Dynamic correlation

Когда возникает: При --dynamic runtime events подтверждают или не подтверждают статические подозрения.

Что делает программа: dynamic_win.c собирает events; report.c считает dynamic_signal_count, static_dynamic_correlation и final assessment. Код: src/dynamic_win.c:657-1108; src/report.c:326-440.

Какой вывод делает проект: Вывод: если статический handler/placement/reuse совпадает с runtime handler hit или runtime function table calls, вывод становится сильнее.

Что делать человеку: Действие аналитика: смотреть events, trace, instrumentation_complete и correlation status.

194. Решения по ранним проверкам формата

194.1. Файл прочитан полностью

Если проверка прошла: Создается Buf, дальше все проверки работают с полным массивом байтов.

Если проверка не прошла: main.c выводит Failed to read file и завершает работу с кодом ошибки. Findings нет.

Вывод и действие: Без полного файла нельзя отличить поврежденный PE от просто недочитанного файла.

194.2. Безопасное чтение поля

Если проверка прошла: Поле можно интерпретировать как little-endian WORD/DWORD/QWORD.

Если проверка не прошла: Caller получает false. На раннем PE этапе это hard stop; внутри Exception Directory/UNWIND_INFO это malformed или parse fail.

Вывод и действие: Проект защищает себя от выхода за границы и не строит выводы на несуществующих байтах.

194.3. MZ, e_lfanew, PE signature

Если проверка прошла: Проект переходит к COFF и Optional Header.

Если проверка не прошла: pe_parse возвращает false, main.c печатает PE parse error и прекращает анализ.

Вывод и действие: Файл не доказан как PE, поэтому EH/unwind findings создавать нельзя.

194.4. Machine == AMD64

Если проверка прошла: Можно применять x64 RUNTIME_FUNCTION/UNWIND_INFO модель.

Если проверка не прошла: pe_parse возвращает unsupported Machine.

Вывод и действие: Это не подозрительность, а неподдерживаемая архитектура для текущей ВКР.

194.5. OptionalHeader Magic == 0x20b

Если проверка прошла: Проект читает PE32+ offsets: ImageBase, alignments, SizeOfImage, DataDirectory.

Если проверка не прошла: pe_parse возвращает unsupported OptionalHeader magic.

Вывод и действие: PE32 и PE32+ имеют разные layout и exception model; продолжать нельзя.

194.6. SizeOfImage/Headers/Alignments валидны

Если проверка прошла: Можно проверять RVA и секции.

Если проверка не прошла: pe_parse возвращает ошибку и останавливает анализ.

Вывод и действие: Без этих границ невозможно честно сказать, вышел ли RVA за image или нет.

194.7. DataDirectory[Exception] доступна

Если проверка прошла: ExceptionRVA/ExceptionSize читаются и сохраняются в PeImage.

Если проверка не прошла: pe_parse останавливается с DataDirectory too small или entry outside optional header.

Вывод и действие: Если каталог даже нельзя прочитать из Optional Header, таблицу функций искать по нему нельзя.

195. Решения по PE structural findings

195.1. pe_section_raw_oob

Если проверка прошла: Секция не выходит за file_size: счетчик не растет, finding не создается.

Если проверка не прошла: raw_oob++ в pe.c; finding pe_section_raw_oob; score +25; severity high.

Какой вывод делает проект: Файл заявляет секционные байты, которых нет. Дальше анализ возможен, но доверие к raw layout ниже.

Что делать человеку: Проверить RawPtr/RawSize секции и фактический размер файла.

195.2. pe_section_virtual_oob

Если проверка прошла: Секция помещается в SizeOfImage: счетчик не растет, finding не создается.

Если проверка не прошла: virtual_oob++ ; finding pe_section_virtual_oob; score +20; severity high.

Какой вывод делает проект: Секция выходит за виртуальный образ. RVA-модель подозрительна.

Что делать человеку: Проверить VirtualAddress/VirtualSize/SizeOfImage.

195.3. pe_section_raw_overlap

Если проверка прошла: Raw ranges не пересекаются: счетчик не растет, finding не создается.

Если проверка не прошла: raw_overlaps++ ; finding pe_section_raw_overlap; score +15; severity medium.

Какой вывод делает проект: Две секции делят одни file bytes. Это может быть packer/нестандартность.

Что делать человеку: Сравнить raw ranges секций в PE viewer.

195.4. pe_section_virtual_overlap

Если проверка прошла: Virtual ranges не пересекаются: счетчик не растет, finding не создается.

Если проверка не прошла: virtual_overlaps++ ; finding pe_section_virtual_overlap; score +20; severity high.

Какой вывод делает проект: Один RVA может относиться к двум секциям. Права и роль области неоднозначны.

Что делать человеку: Проверить section table и owner RVA.

195.5. pe_section_misaligned

Если проверка прошла: Alignment и порядок секций выглядят согласованно: счетчик не растет, finding не создается.

Если проверка не прошла: misaligned++ ; finding pe_section_misaligned; score +8; severity medium.

Какой вывод делает проект: Координатная система странная, но это вспомогательный сигнал.

Что делать человеку: Проверить FileAlignment/SectionAlignment/SizeOfHeaders/порядок секций.

195.6. pe_exception_directory_oob

Если проверка прошла: Exception Directory отсутствует корректно или лежит в пределах image: счетчик не растет, finding не создается.

Если проверка не прошла: exception_directory_oob=true; finding pe_exception_directory_oob; score +25; severity high.

Какой вывод делает проект: Вход в .pdata/RUNTIME_FUNCTION недостоверен. Может быть частичный отчет.

Что делать человеку: Проверить DataDirectory[Exception].RVA/Size, кратность 12 и RVA mapping.

195.7. no_runtime_functions

Если проверка прошла: Есть RUNTIME_FUNCTION entries или нет executable code: счетчик не растет, finding не создается.

Если проверка не прошла: finding no_runtime_functions; score +10; severity medium.

Какой вывод делает проект: Код есть, но таблицы функций нет. Может быть leaf-only, но требует проверки сложности кода.

Что делать человеку: Проверить наличие исполняемых секций и тип бинарника.

196. Решения по RUNTIME_FUNCTION table

196.1. invalid_ranges

Если проверка прошла: EndAddress > BeginAddress: запись считается нормальной по этому критерию.

Если проверка не прошла: rf_invalid_ranges++; finding invalid_ranges; >1% дает high и больший вес. Анализ остальных записей продолжается.

Какой вывод делает проект: Функция описана пустым или обратным диапазоном. Такая запись не описывает нормальный кодовый интервал.

Что делать человеку: Открыть запись .pdata и проверить Begin/End.

196.2. rf_oob

Если проверка прошла: Begin/End внутри SizeOfImage: запись считается нормальной по этому критерию.

Если проверка не прошла: rf_oob++; finding rf_oob; >1% дает high. Анализ остальных записей продолжается.

Какой вывод делает проект: Таблица функций указывает за пределы образа. Stack walking по такой записи недостоверен.

Что делать человеку: Проверить SizeOfImage и диапазоны функций.

196.3. unwind_rva_oob

Если проверка прошла: UnwindInfoRVA ненулевой и внутри SizeOfImage: запись считается нормальной по этому критерию.

Если проверка не прошла: rf_unwind_oob++; finding unwind_rva_oob; >1% дает high. Анализ остальных записей продолжается.

Какой вывод делает проект: Функция есть, но ее unwind metadata отсутствует или за image.

Что делать человеку: Проверить UnwindInfoAddress конкретной записи.

196.4. rf_unsorted

Если проверка прошла: BeginAddress в исходной таблице не убывает: запись считается нормальной по этому критерию.

Если проверка не прошла: rf_unsorted++; finding rf_unsorted; score +15. Анализ остальных записей продолжается.

Какой вывод делает проект: Таблица не отсортирована. Это может мешать lookup или говорить о нестандартной генерации.

Что делать человеку: Сортировать таблицу вручную и сравнить порядок.

196.5. rf_overlaps

Если проверка прошла: После сортировки диапазоны не перекрываются: запись считается нормальной по этому критерию.

Если проверка не прошла: rf_overlaps++; finding rf_overlaps; score +25; high. Анализ остальных записей продолжается.

Какой вывод делает проект: Один RIP может принадлежать двум функциям. Это сильный конфликт function map.

Что делать человеку: Проверить пересекающиеся ranges в Ghidra/IDA.

196.6. duplicate_begin

Если проверка прошла: После сортировки нет одинаковых BeginAddress: запись считается нормальной по этому критерию.

Если проверка не прошла: rf_duplicate_begins++; finding duplicate_begin; score +10. Анализ остальных записей продолжается.

Какой вывод делает проект: Две записи стартуют в одной точке, карта функций неоднозначна.

Что делать человеку: Проверить, не является ли это alias/thunk/ошибка таблицы.

197. Решения по UNWIND_INFO parser

197.1. Header/mapping/codes/tail читаются

Если проверка прошла: us_ok[i]=true, UnwindSummary заполняется: version, flags, prolog, stack, handler, chain, anomalies.

Если проверка не прошла: us_ok[i]=false, parse_error сохраняется, later finding unwind_parse_fail.

Какой вывод делает проект: Проект отличает 'не смог прочитать структуру' от 'прочитал, но структура странная'.

Что делать человеку: Если parse fail - идти к UnwindInfoRVA и смотреть, где parser остановился.

197.2. Anomaly bits отсутствуют

Если проверка прошла: Запись считается валидной по semantic checks.

Если проверка не прошла: anomalies выставлены; later finding unwind_anomalies, если это не только ABI warning.

Какой вывод делает проект: Проект говорит: структура есть, но внутри есть подозрительные комбинации.

Что делать человеку: Сравнить anomaly с реальным прологом функции.

197.3. HandlerRVA внутри image

Если проверка прошла: handler_rva сохраняется и потом проверяется placement.

Если проверка не прошла: UA_HANDLER_OOB; затем может попасть в unwind_anomalies.

Какой вывод делает проект: Handler target не должен быть за image.

Что делать человеку: Проверить, куда указывает handler и существует ли такой код.

197.4. CHAININFO корректен

Если проверка прошла: chain сохраняется, при follow_chain parser проверяет следующий unwind.

Если проверка не прошла: UA_CHAIN_INVALID/LOOP/DEPTH/TRUNCATED.

Какой вывод делает проект: Цепочка unwind metadata может ломать восстановление стека.

Что делать человеку: Проверить chained records и max depth.

197.5. ABI warnings

Если проверка прошла: Если strict_abi выключен или нарушений нет, это не влияет.

Если проверка не прошла: unwind_abi_warnings++ при UA_ABI_MASK.

Какой вывод делает проект: Это формальные ABI нарушения. Они отдельно считаются, но не всегда превращаются в основной suspicious finding.

Что делать человеку: Использовать как дополнительный диагностический слой, не как единственное обвинение.

198. Решения по placement, reuse, combo

198.1. unwind_in_exec_section

Если проверка прошла: UNWIND_INFO не в executable секции: отдельный finding не создается.

Если проверка не прошла: finding unwind_in_exec_section; score +20; high.

Какой вывод делает проект: Metadata лежит там, где обычно лежит код. Это сильная смысловая странность.

Что делать человеку: Проверить секцию UnwindInfoRVA и права execute/write.

198.2. unwind_unusual_section

Если проверка прошла: UNWIND_INFO в .xdata/.rdata/.pdata или похожей секции: отдельный finding не создается.

Если проверка не прошла: finding unwind_unusual_section; >5% повышает вес/severity.

Какой вывод делает проект: Metadata находится в необычном месте. Само по себе требует осторожной интерпретации.

Что делать человеку: Проверить имя секции, linker/packer, долю affected_count.

198.3. unwind_writable_section

Если проверка прошла: UNWIND_INFO не writable: отдельный finding не создается.

Если проверка не прошла: finding unwind_writable_section; >5% дает больший вес.

Какой вывод делает проект: Metadata можно менять во время работы, значит можно менять unwind semantics.

Что делать человеку: Проверить, есть ли JIT/packer/самомодификация.

198.4. handler_nonexec_section

Если проверка прошла: Handler в executable секции: отдельный finding не создается.

Если проверка не прошла: finding handler_nonexec_section; high.

Какой вывод делает проект: Handler должен быть кодом. Non-executable target выглядит противоречиво.

Что делать человеку: Проверить HandlerRVA и memory rights.

198.5. handler_writable_section

Если проверка прошла: Handler не writable: отдельный finding не создается.

Если проверка не прошла: finding handler_writable_section; medium.

Какой вывод делает проект: Handler лежит в изменяемой области. Это не всегда ошибка, но требует внимания.

Что делать человеку: Проверить, не runtime/JIT ли это.

198.6. unwind_reuse_strong

Если проверка прошла: Reuse ratio не превышает 30% или n <= 200: отдельный finding не создается.

Если проверка не прошла: finding unwind_reuse_strong; high.

Какой вывод делает проект: Очень много функций указывают на один UNWIND_INFO. Это похоже на централизацию metadata.

Что делать человеку: Сгруппировать CSV по UnwindInfoRVA.

198.7. handler_dominance

Если проверка прошла: Нет dominance или есть benign code-like context: отдельный finding не создается.

Если проверка не прошла: finding handler_dominance; medium.

Какой вывод делает проект: Один handler обслуживает большую часть функций. Возможно dispatcher-style обработка.

Что делать человеку: Проверить доминирующий handler в дизассемблере.

198.8. unwind_reuse_moderate

Если проверка прошла: Reuse <=10%, n<=200 или нет второго сигнала: отдельный finding не создается.

Если проверка не прошла: finding только при reuse >10% + другой static signal.

Какой вывод делает проект: Проект не усиливает умеренный reuse без корреляции.

Что делать человеку: Искать сопутствующие table/placement/handler признаки.

198.9. Combo findings

Если проверка прошла: Комбинации условий отсутствуют: отдельный finding не создается.

Если проверка не прошла: создаются placement_exec_nonexec_handler_combo, placement_integrity_combo, reuse_dominance_combo, writable_metadata_handler_combo.

Какой вывод делает проект: Комбо значит: несколько независимых наблюдений усилили друг друга.

Что делать человеку: Проверять связанные findings вместе, а не по отдельности.

199. Решения по score, confidence, report

199.1. Score

Что делает программа: Если findings нет, score остается 0. Если findings есть, add_score суммирует веса с ограничением 0..100.

Какой вывод делает проект: Проект делает вывод о статической подозрительности: >=70 high, >=40 moderate, >=15 low, ниже - no strong static evidence.

Что делать человеку: Человеку нельзя читать score как вероятность вируса. Это шкала силы evidence по EH/unwind слою.

199.2. Confidence

Что делает программа: Если много evidence families и сильных findings, confidence растет. Если сигнал одиночный, маленькая выборка или только placement, confidence снижается.

Какой вывод делает проект: Проект решает, насколько уверенно можно доверять статическому verdict.

Что делать человеку: Если confidence снижен, искать дополнительное подтверждение: dynamic run, manual validation, disassembly.

199.3. requires_manual_review

Что делает программа: Включается при score >=20 или при важных placement/parse/dynamic условиях.

Какой вывод делает проект: Проект говорит: автоматический вывод недостаточен, нужно руками проверить representative RVA/sections/handlers.

Что делать человеку: Открыть report manual steps, PE viewer, Ghidra/IDA.

199.4. JSON/HTML report

Что делает программа: Все findings, counters, sections, functions, anomalies, dynamic events выводятся в отчет.

Какой вывод делает проект: Проект дает evidence trail, а не черный ящик.

Что делать человеку: Использовать report как карту ручной проверки: finding -> representative_rva -> section -> source reason.

199.5. Dynamic final assessment

Что делает программа: Если dynamic session complete и есть events, confidence может вырасти. Если static/dynamic коррелируют, final_verdict становится correlated.

Какой вывод делает проект: Проект отделяет 'статически похоже' от 'наблюдалось во время выполнения'.

Что делать человеку: Проверить instrumentation_complete, dropped_events, handler hits, runtime function table calls.

200. Короткая таблица: finding - действие программы - смысл вывода

Это сжатая шпаргалка по всем 27 findings. Подробные строки кода уже указаны в предыдущем разделе, а здесь главное - что происходит после результата проверки.

pe_section_raw_oob

Действие программы: Увеличивает score на 25, high

Вывод проекта: Файл обещает raw bytes за концом файла

Действие аналитика: Проверить raw range секции

pe_section_virtual_oob

Действие программы: Score +20, high

Вывод проекта: Секция выходит за SizeOfImage

Действие аналитика: Проверить virtual range

pe_section_raw_overlap

Действие программы: Score +15, medium

Вывод проекта: Секции делят file bytes

Действие аналитика: Проверить raw overlap

pe_section_virtual_overlap

Действие программы: Score +20, high

Вывод проекта: RVA ownership неоднозначен

Действие аналитика: Проверить virtual overlap

pe_section_misaligned

Действие программы: Score +8, medium

Вывод проекта: Нарушены alignment/header invariants

Действие аналитика: Проверить alignments

pe_exception_directory_oob

Действие программы: Score +25, high

Вывод проекта: Вход в .pdata поврежден/нечитаем

Действие аналитика: Проверить DataDirectory[Exception]

no_runtime_functions

Действие программы: Score +10, medium

Вывод проекта: Код есть, function table отсутствует

Действие аналитика: Проверить leaf-only/resource-only

invalid_ranges

Действие программы: +10/+20, medium/high

Вывод проекта: End <= Begin у функций

Действие аналитика: Проверить .pdata ranges

rf_oob

Действие программы: +10/+25, medium/high

Вывод проекта: Функции за SizeOfImage

Действие аналитика: Проверить Begin/End

unwind_rva_oob

Действие программы: +10/+25, medium/high

Вывод проекта: UnwindInfoRVA пустой/за image

Действие аналитика: Проверить UnwindInfoAddress

rf_unsorted

Действие программы: +15, medium

Вывод проекта: Таблица не отсортирована

Действие аналитика: Сравнить порядок BeginAddress

rf_overlaps

Действие программы: +25, high

Вывод проекта: Функции перекрываются

Действие аналитика: Проверить конфликт ranges

duplicate_begin

Действие программы: +10, medium

Вывод проекта: Две функции с одним start

Действие аналитика: Проверить дубликаты BeginAddress

unwind_parse_fail

Действие программы: +10/+25, medium/high

Вывод проекта: UNWIND_INFO не читается

Действие аналитика: Открыть parse_error и RVA

unwind_anomalies

Действие программы: +10/+20, medium/high

Вывод проекта: UNWIND_INFO читается, но странная

Действие аналитика: Сравнить с прологом

unwind_reuse_strong

Действие программы: +20, high

Вывод проекта: Много функций делят один UNWIND_INFO

Действие аналитика: Группировать по UnwindInfoRVA

handler_dominance

Действие программы: +15, medium

Вывод проекта: Один handler доминирует

Действие аналитика: Дизассемблировать handler

unwind_reuse_moderate

Действие программы: +10, medium

Вывод проекта: Умеренный reuse + второй сигнал

Действие аналитика: Искать корреляцию

unwind_in_exec_section

Действие программы: +20, high

Вывод проекта: Metadata лежит в code section

Действие аналитика: Проверить права секции

unwind_unusual_section

Действие программы: +8/+15, medium/high

Вывод проекта: Metadata в необычной секции

Действие аналитика: Проверить имя/роль секции

unwind_writable_section

Действие программы: +10/+18, medium/high

Вывод проекта: Metadata writable

Действие аналитика: Проверить возможность подмены

handler_nonexec_section

Действие программы: +15, high

Вывод проекта: Handler не executable

Действие аналитика: Проверить handler target

handler_writable_section

Действие программы: +10, medium

Вывод проекта: Handler writable

Действие аналитика: Проверить runtime/JIT/packer

placement_exec_nonexec_handler_combo

Действие программы: +10, high

Вывод проекта: Metadata/code roles противоречат вместе

Действие аналитика: Проверять оба placement findings

placement_integrity_combo

Действие программы: +8, medium

Вывод проекта: Placement issue + integrity issue

Действие аналитика: Проверять связанные признаки вместе

reuse_dominance_combo

Действие программы: +8, medium

Вывод проекта: Reuse совпал с handler dominance

Действие аналитика: Искать dispatcher pattern

writable_metadata_handler_combo

Действие программы: +10, high

Вывод проекта: Metadata и handler writable

Действие аналитика: Проверить изменяемую EH/unwind логику

Итоговая логика проекта такая: проверка дает наблюдение, наблюдение превращается в счетчик или status, heuristics превращает значимые счетчики в findings, score показывает силу статической evidence, confidence показывает надежность вывода, dynamic mode может подтвердить или ослабить интерпретацию, а report дает адреса для ручной перепроверки.

СКВОЗНОЙ СИСТЕМНЫЙ РАЗБОР ПРОВЕРОК ПРОЕКТА

Этот раздел добавлен в конец документа после страницы 172. Его задача - не пересказать проект кусками, а показать один непрерывный маршрут: от запуска инструмента и чтения бинарника до формирования findings, score, confidence, dynamic evidence и итогового отчета.

Главная идея раздела: каждая проверка описана как инженерное решение. Для нее указано, где она находится в коде, что именно сравнивается, почему это сравнение имеет смысл, какой finding или счетчик возникает, что программа делает после результата и как pipeline идет дальше. Там, где проверка не порождает finding, это прямо указано: такие проверки являются воротами безопасности, ограничением области применимости или проверкой вывода.

Как читать карточку проверки

Режим выполнения: Показывает, выполняется ли шаг всегда или только при включенных режимах heuristics, dynamic, dump, JSON/HTML/CSV.

Где в коде: Файл и диапазон строк, где находится фактическая логика. Если finding создается позже, указаны оба места: где факт измеряется и где он превращается в finding.

Что проверяется: Смысловая формулировка проверки без потери технической точности.

Как проверяется технически: Конкретные поля, сравнения, формулы, counters, flags, offsets, RVA/file offset mapping.

Какой вывод делает проект: Что значит нормальный результат и что значит провал проверки.

Решение программы: Останавливает ли программа анализ, продолжает ли частично, создает ли counter/finding, меняет ли exit_code, пишет ли отчет.

Как продолжается выполнение: Следующий шаг pipeline, чтобы не терялась сквозная логика.

Важное разделение: hard parse error останавливает анализ, потому что формат еще не распознан. Finding создается только тогда, когда проект уже построил достаточно надежную модель PE и может доказательно сказать, что конкретная структура нарушает ожидаемый контракт. Поэтому не все if-проверки обязаны быть findings.

Блок A. Запуск и входные данные

Этот блок отвечает за то, чтобы инструмент начал анализ только с корректной командой и реальным набором bytes. Здесь еще нет PE findings, потому что бинарник еще не распознан как PE.

Шаг 1. Разбор аргументов командной строки

Режим выполнения: Всегда выполняется при запуске.

Где в коде: src/main.c:270-281; вспомогательная логика аргументов - src/main.c:176-243

Что проверяется: Проект проверяет, что пользователь передал понятную команду: входной файл, опции вывода, флаги эвристик, динамики, дампа, глубины цепочек, строгого ABI и путей отчетов.

Как проверяется технически: Функция parse_args заполняет структуру Args. Если пользователь просит --help или --version, программа не анализирует бинарник, а корректно печатает справочную информацию или версию. Если аргументы противоречат формату запуска, main показывает usage и возвращает код 2.

Зачем это нужно: Это первый слой защиты от ложного анализа. Нельзя начинать PE-разбор, если непонятно, какой файл анализировать и какие режимы включены. Для комиссии важно: проект не является 'скриптом на один случай', у него есть управляемый CLI-жизненный цикл.

Какой вывод делает проект: При корректных аргументах создается конфигурация анализа. При ошибке анализа бинарника нет, findings не создаются, потому что еще нет объекта PE и нет доказательной базы.

Finding / счетчик / статус: Нет finding. Это входная валидация CLI, а не признак бинарника.

Решение программы: Если аргументы нормальные - перейти к чтению файла. Если --help/--version - завершить с 0. Если аргументы неверные - завершить с 2.

Как продолжается выполнение: Дальше программа открывает файл и формирует Buf, из которого все модули читают байты.

Шаг 2. Чтение файла целиком в безопасный буфер

Режим выполнения: Всегда, если CLI разобран.

Где в коде: src/main.c:283-288; src/util.c:13-35; include/util.h

Что проверяется: Проверяется, можно ли открыть входной путь, узнать размер файла, выделить память и считать все байты без усечения.

Как проверяется технически: buf_init_from_file возвращает Buf с указателем data и размером size. Все дальнейшие чтения идут через Buf, а не через произвольные указатели. Если файл не читается, main печатает Failed to read file и сразу возвращает 1.

Зачем это нужно: PE - бинарный формат со смещениями. Любая ошибка чтения файла ломает координатную систему: offset, RVA и таблицы нельзя интерпретировать надежно. Поэтому это жесткая проверка перед всей аналитикой.

Какой вывод делает проект: Успех означает: есть единый неизменяемый массив байтов. Провал означает: анализ невозможен, потому что нет исходных данных.

Finding / счетчик / статус: Нет finding. Это ошибка ввода-вывода, не свойство PE-файла.

Решение программы: При успехе перейти к pe_parse. При провале завершиться, не создавая отчет о признаках.

Как продолжается выполнение: Дальше parser начинает читать DOS Header и NT Headers.

Шаг 3. Безопасные чтения primitive-типов

Режим выполнения: Используется на всех этапах парсинга.

Где в коде: src/util.c:42-70; вызовы buf_read_u16/u32/u64 в src/pe.c и src/unwind.c

Что проверяется: Проверяется, что перед чтением 2, 4, 8 или N байт запрошенный диапазон помещается в b->size.

Как проверяется технически: Вместо прямого приведения указателя к структуре проект вызывает buf_read_*. Эти функции проверяют границы и только потом копируют байты. Поэтому поврежденный файл не заставляет parser читать память за пределами файла.

Зачем это нужно: Это основа надежности проекта. Многие 'несколько if else' проекты ломаются именно здесь: они доверяют offset-ам из файла. ВКР проверяет границу перед каждым смысловым чтением.

Какой вывод делает проект: Успех дает конкретное число из файла. Провал передается вызывающему коду как false и превращается либо в hard parse error, либо в malformed finding на более высоком уровне.

Finding / счетчик / статус: Прямого finding нет. Но провалы чтения могут привести к pe_exception_directory_oob, unwind_parse_fail или полной ошибке PE parse.

Решение программы: Решение зависит от уровня: базовые PE-заголовки - остановка; Exception Directory/UNWIND_INFO - фиксация проблемы и частичный анализ, если возможно.

Как продолжается выполнение: Дальше каждая структура PE читается через эти безопасные функции.

Блок B. PE parser: вход в формат

Здесь проект доказывает, что файл соответствует базовому PE-контракту и именно x64/PE32+, а не произвольному файлу или PE32 другой архитектуры.

Шаг 4. Проверка DOS magic MZ и поля e_lfanew

Режим выполнения: Всегда.

Где в коде: src/pe.c:40-48; helper read_dos_lfanew - src/pe.c:11-18; constants - include/pe.h:15

Что проверяется: Проверяется, что файл начинается как PE-совместимый образ: есть DOS-заголовок MZ и поле e_lfanew указывает на будущий NT Header.

Как проверяется технически: read_dos_lfanew читает сигнатуру 0x5A4D и смещение e_lfanew. Затем pe_parse проверяет, что по этому смещению хотя бы помещаются PE signature и COFF File Header.

Зачем это нужно: e_lfanew - мост от старого DOS Header к реальному PE Header. Если он испорчен или указывает за файл, любые дальнейшие поля будут прочитаны из мусора.

Какой вывод делает проект: Если MZ/e_lfanew корректны, parser знает, где искать PE Signature. Если нет - это не анализируемый PE или сильно поврежденный файл.

Finding / счетчик / статус: Нет finding, потому что это не 'подозрительность unwind', а невозможность распознать формат.

Решение программы: При провале pe_parse возвращает false, main печатает PE parse error и завершает работу с кодом 1.

Как продолжается выполнение: При успехе parser переходит к PE signature.

Шаг 5. Проверка PE signature

Режим выполнения: Всегда после e_lfanew.

Где в коде: src/pe.c:50-54; constant PE_NT_MAGIC - include/pe.h:16

Что проверяется: Проверяется, что по адресу e_lfanew лежит сигнатура PE\0\0.

Как проверяется технически: buf_read_u32 читает 4 байта, затем значение сравнивается с PE_NT_MAGIC. Это отделяет настоящий NT Header от произвольных данных.

Зачем это нужно: Без этой проверки нельзя доверять Machine, NumberOfSections, OptionalHeader и DataDirectory. Комиссии можно объяснить: проект не 'ищет .pdata по строке', он идет по контракту загрузчика Windows.

Какой вывод делает проект: Успех означает: можно читать COFF File Header. Провал означает: координатная система PE не подтверждена.

Finding / счетчик / статус: Нет finding. Это hard format validation.

Решение программы: При провале pe_parse останавливается и возвращает ошибку Invalid PE signature.

Как продолжается выполнение: Дальше читается COFF File Header.

Шаг 6. Чтение COFF File Header и проверка Machine

Режим выполнения: Всегда.

Где в коде: src/pe.c:56-70; IMAGE_FILE_MACHINE_AMD64 - include/pe.h:18

Что проверяется: Проверяются Machine, NumberOfSections и SizeOfOptionalHeader. Главное ограничение проекта: поддерживается x64/AMD64.

Как проверяется технически: Parser читает machine, nsects и opt_size. Если machine не равен 0x8664, pe_parse возвращает ошибку Unsupported Machine.

Зачем это нужно: Проект работает с x64 unwind. На x86 другая модель исключений и нет той же обязательной таблицы RUNTIME_FUNCTION. Поэтому анализ PE32 в этой версии сознательно не смешан с PE32+.

Какой вывод делает проект: Если Machine AMD64 - можно интерпретировать Exception Directory как x64 RUNTIME_FUNCTION. Если нет - проект не делает ложный вывод по другой архитектуре.

Finding / счетчик / статус: Нет finding. Это ограничение области применимости, а не подозрительный признак файла.

Решение программы: При unsupported machine программа останавливает PE parse. Это лучше, чем неправильно применить x64 правила к x86.

Как продолжается выполнение: Дальше проверяется Optional Header.

Шаг 7. Проверка границ и минимального размера Optional Header

Режим выполнения: Всегда.

Где в коде: src/pe.c:72-80

Что проверяется: Проверяется, что Optional Header физически лежит внутри файла и имеет минимум байтов для обязательных PE32+ полей до DataDirectory.

Как проверяется технически: Сначала вычисляется opthdr = filehdr + 20. Затем проверяется opthdr + opt_size <= file size и opt_size >= 0x70.

Зачем это нужно: Optional Header содержит ImageBase, SizeOfImage, SizeOfHeaders, Alignment и DataDirectory. Если он обрезан, проект не должен читать дальше.

Какой вывод делает проект: Успех позволяет перейти к Magic. Провал означает: заголовок поврежден или файл усечен.

Finding / счетчик / статус: Нет finding. Это hard parse error на фундаментальном уровне.

Решение программы: При провале pe_parse возвращает false. Отчет по findings не строится, потому что нет валидной модели image.

Как продолжается выполнение: Дальше проверяется Magic 0x20b.

Шаг 8. Проверка OptionalHeader.Magic = PE32+

Режим выполнения: Всегда.

Где в коде: src/pe.c:82-90; IMAGE_NT_OPTIONAL_HDR64_MAGIC - include/pe.h:19

Что проверяется: Проверяется, что Optional Header относится к PE32+ и имеет magic 0x20b.

Как проверяется технически: buf_read_u16 читает первые два байта Optional Header. Значение сравнивается с 0x20b. Если пришло 0x10b, это PE32, и проект не продолжает x64-unwind анализ.

Зачем это нужно: 0x20b - это не 'магическое число разработчика', а значение формата PE для 64-битного образа. Именно у PE32+ на Windows x64 используется таблица .pdata/RUNTIME_FUNCTION для unwinding.

Какой вывод делает проект: Успех фиксирует img.is_pe64 = true в дальнейшем. Провал защищает от неправильной трактовки размеров и смещений Optional Header.

Finding / счетчик / статус: Нет finding. Это проверка применимости модели.

Решение программы: При провале pe_parse возвращает Unsupported OptionalHeader magic.

Как продолжается выполнение: Дальше parser читает ключевые поля Optional Header.

Блок B. PE parser: Optional Header

Этот блок извлекает поля, которые задают виртуальную координатную систему image: SizeOfImage, SizeOfHeaders, ImageBase и alignment.

Шаг 9. Чтение ImageBase, SizeOfImage, SizeOfHeaders и alignment

Режим выполнения: Всегда для PE32+.

Где в коде: src/pe.c:92-108; include/pe.h:33-54

Что проверяется: Проверяются ключевые поля виртуальной модели образа: ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders, NumberOfRvaAndSizes.

Как проверяется технически: Parser читает эти поля по PE32+ offset-ам: ImageBase +0x18, SectionAlignment +0x20, FileAlignment +0x24, SizeOfImage +0x38, SizeOfHeaders +0x3c, NumberOfRvaAndSizes +0x6c.

Зачем это нужно: Эти поля задают правила перевода между файлом и памятью. SizeOfImage ограничивает виртуальный image, SizeOfHeaders отделяет headers-RVA, alignment задает легальность секций.

Какой вывод делает проект: Если чтение прошло, эти значения становятся основой всех дальнейших boundary checks. Если не прошло, parser не знает координатную систему.

Finding / счетчик / статус: Нет finding напрямую. Но эти поля питают будущие findings pe_section_virtual_oob, pe_section_misaligned, pe_exception_directory_oob, rf_oob и unwind_rva_oob.

Решение программы: При невозможности прочитать поля pe_parse останавливается.

Как продолжается выполнение: Дальше проверяются нулевые и физически невозможные значения.

Шаг 10. Жесткая проверка SizeOfImage

Режим выполнения: Всегда.

Где в коде: src/pe.c:109-112

Что проверяется: Проверяется, что SizeOfImage не равен нулю.

Как проверяется технически: Простое условие size_of_image == 0 запрещает image без виртуального размера. Такой образ нельзя корректно проверить на выходы RVA за границы.

Зачем это нужно: SizeOfImage - верхняя граница виртуального PE-образа в памяти. Почти все later checks сравнивают RVA или диапазоны с этим значением.

Какой вывод делает проект: Успех означает: есть осмысленная верхняя граница image. Провал означает: таблицы, функции и unwind RVA не с чем сравнивать.

Finding / счетчик / статус: Не finding, a hard parse error. Позже уже внутри валидного PE выходы за SizeOfImage станут findings.

Решение программы: При нуле pe_parse прекращает работу.

Как продолжается выполнение: Дальше проверяется SizeOfHeaders.

Шаг 11. Жесткая проверка SizeOfHeaders

Режим выполнения: Всегда.

Где в коде: src/pe.c:113-116

Что проверяется: Проверяется, что SizeOfHeaders не равен нулю и не превышает размер файла.

Как проверяется технически: Условие size_of_headers == 0 или size_of_headers > b->size запрещает невозможный headers-region. Именно этот region позже учитывается в pe_rva_to_fileoff для RVA, которые лежат не в секциях, а в headers.

Зачем это нужно: Headers тоже являются частью image. Надежный parser не обязан считать, что every RVA лежит в секции. Но headers можно использовать только если они реально есть в файле.

Какой вывод делает проект: Успех позволяет легально маппить headers-RVA. Провал означает: даже заголовочная область описана неправильно.

Finding / счетчик / статус: Не finding, a hard parse error.

Решение программы: При провале pe_parse возвращает ошибку SizeOfHeaders is zero or exceeds the file size.

Как продолжается выполнение: Дальше проверяются SectionAlignment и FileAlignment.

Шаг 12. Жесткая проверка ненулевых alignments

Режим выполнения: Всегда.

Где в коде: src/pe.c:117-120

Что проверяется: Проверяется, что SectionAlignment и FileAlignment не равны нулю.

Как проверяется технически: Это базовая защита от деления по нулю и бессмысленных проверок кратности. Позже эти поля участвуют в подсчете pe_section_misaligned.

Зачем это нужно: Alignment задает, как секции должны лежать в памяти и на диске. Нулевое выравнивание делает весь section table неинтерпретируемым.

Какой вывод делает проект: Успех позволяет выполнять согласовательные проверки. Провал останавливает parsing.

Finding / счетчик / статус: Не finding, потому что без этих чисел нельзя даже корректно измерить нарушение.

Решение программы: При нуле pe_parse возвращает SectionAlignment/FileAlignment must be non-zero.

Как продолжается выполнение: Дальше проверяется доступность DataDirectory.

Блок B. PE parser: Data Directory

Здесь проект получает адрес и размер Exception Directory, но пока не доверяет им как читаемой таблице.

Шаг 13. Проверка NumberOfRvaAndSizes

Режим выполнения: Всегда.

Где в коде: src/pe.c:121-130; IMAGE_DIRECTORY_ENTRY_EXCEPTION - include/pe.h:22

Что проверяется: Проверяется, что массив DataDirectory достаточно длинный, чтобы законно читать entry 3 - Exception Directory.

Как проверяется технически: Parser сравнивает NumberOfRvaAndSizes с IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1 и отдельно проверяет, что относительный offset dd_rel + 8 помещается в Optional Header.

Зачем это нужно: Это защита от чтения поля, которого в Optional Header формально нет. Комиссии можно объяснить: проект не доверяет индексу каталога, пока сам Optional Header не подтвердил наличие этой записи.

Какой вывод делает проект: Успех позволяет читать VirtualAddress/Size Exception Directory. Провал означает, что x64 unwind table невозможно получить стандартным путем.

Finding / счетчик / статус: Не finding. Это hard parse error, потому что отсутствует сама directory entry, из которой строится исследование.

Решение программы: При провале pe_parse останавливается с DataDirectory too small или directory outside optional header.

Как продолжается выполнение: Дальше читается пара ExceptionDirectory.VirtualAddress/Size.

Шаг 14. Чтение Exception Directory RVA и Size

Режим выполнения: Всегда после проверки DataDirectory.

Где в коде: src/pe.c:132-137

Что проверяется: Проверяется, что два DWORD поля Exception Directory можно прочитать: VirtualAddress и Size.

Как проверяется технически: VirtualAddress сохраняется как exc_rva, Size как exc_sz. На этом этапе parser еще не читает RUNTIME_FUNCTION entries, а только фиксирует адрес и размер каталога.

Зачем это нужно: Важно разделять этапы: DataDirectory говорит 'где должна быть таблица', а отдельный reader позже проверяет, можно ли эту область реально прочитать из файла.

Какой вывод делает проект: Успех дает исходные evidence-числа для .pdata. Провал означает поврежденный Optional Header.

Finding / счетчик / статус: Прямого finding нет. Но exc_rva/exc_sz позже могут привести к pe_exception_directory_oob или no_runtime_functions.

Решение программы: При невозможности прочитать поля pe_parse останавливается.

Как продолжается выполнение: Дальше читается таблица секций.

Блок C. PE parser: Section Table

Секция - это связь между виртуальным RVA-пространством и физическим файлом. Здесь рождаются первые structural counters для будущих findings.

Шаг 15. Проверка границ таблицы секций

Режим выполнения: Всегда.

Где в коде: src/pe.c:139-148

Что проверяется: Проверяется, что NumberOfSections не вызывает переполнение и что все section headers по 40 байт помещаются в файле.

Как проверяется технически: Parser вычисляет secthdr = opthdr + opt_size и needed = secthdr + nsects * 40. Перед умножением есть overflow-check через SIZE_MAX.

Зачем это нужно: Section Table - карта перевода RVA в file offset и источник прав секций. Если она обрезана, нельзя надежно говорить, где лежит .pdata/.xdata или handler.

Какой вывод делает проект: Успех позволяет выделить массив PeSection. Провал означает, что section metadata недоступна.

Finding / счетчик / статус: Не finding, hard parse error. Findings о секциях начинаются только когда секции хотя бы прочитаны и их можно сравнивать.

Решение программы: При переполнении или выходе за файл pe_parse возвращает ошибку.

Как продолжается выполнение: Дальше читаются отдельные section headers.

Шаг 16. Чтение каждой секции

Режим выполнения: Всегда.

Где в коде: src/pe.c:150-172; PeSection - include/pe.h:24-31

Что проверяется: Для каждой секции читаются имя, VirtualSize, VirtualAddress, SizeOfRawData, PointerToRawData и Characteristics.

Как проверяется технически: Цикл идет по nsects. На каждой итерации offset = secthdr + i * 40. Данные складываются в PeSection, где name завершается нулем, чтобы его можно было сравнивать как строку.

Зачем это нужно: Эти поля нужны не только для красоты. VirtualAddress/VirtualSize описывают память, raw_ptr/raw_size - файл, characteristics - права R/W/X. На их основе делаются findings по границам, пересечениям и placement.

Какой вывод делает проект: Успех дает полную карту секций. Провал чтения конкретного поля означает усеченный или поврежденный PE.

Finding / счетчик / статус: Не finding на этом уровне. Позже значения этих полей порождают pe_section_raw_oob, pe_section_virtual_oob, pe_section_misaligned и placement findings.

Решение программы: При невозможности прочитать секцию parser освобождает secs и останавливается.

Как продолжается выполнение: Дальше parser начинает считать structural counters.

Шаг 17. Проверка глобальных правил выравнивания

Режим выполнения: Всегда после чтения секций.

Где в коде: src/pe.c:174-196

Что проверяется: Проверяется, согласованы ли FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders, размер заголовков и ImageBase.

Как проверяется технически: Код считает compact_alignment, проверяет file_alignment как степень двойки и допустимый диапазон 0x200..0x10000, сравнивает section_alignment с file_alignment, проверяет кратность SizeOfImage, кратность SizeOfHeaders и 64K alignment ImageBase.

Зачем это нужно: Это не поиск вируса, а проверка PE-инвариантов. Если loader/линкер обычно строит образ по этим правилам, нарушение значит: файл поврежден, вручную модифицирован, упакован нестандартно или parser читает не тот формат.

Какой вывод делает проект: Каждое нарушение увеличивает counter misaligned. Пока это еще не finding, а накопленный факт для эвристического слоя.

Finding / счетчик / статус: finding pe_section_misaligned создается позже в src/heuristics.c:817-823. Смысл: нарушены правила согласованности заголовков/секций.

Решение программы: Программа не останавливается, потому что некоторые нестандартные PE еще можно частично анализировать. Она сохраняет счетчик в PeImage.

Как продолжается выполнение: Дальше проверяются raw и virtual диапазоны каждой секции.

Шаг 18. Проверка raw-диапазона секции на выход за файл

Режим выполнения: Всегда для каждой секции.

Где в коде: src/pe.c:198-204; finding emission - src/heuristics.c:789-795

Что проверяется: Проверяется, что PointerToRawData + SizeOfRawData не выходит за b->size.

Как проверяется технически: Для секции вычисляется raw_end = raw_ptr + raw_size в uint64_t. Если raw_size > 0 и raw_end > file size, увеличивается raw_oob.

Зачем это нужно: Raw-диапазон - это место секции на диске. Если он выходит за файл, parser не может безопасно читать обещанные байты. Это может быть повреждение, искусственная ловушка или несогласованность упаковщика.

Какой вывод делает проект: Если нарушений нет - raw layout секций физически читаем. Если есть - позже создается finding pe_section_raw_oob.

Finding / счетчик / статус: pe_section_raw_oob, family table_integrity, high, score +25. Значит: section table обещает байты, которых в файле нет.

Решение программы: Программа не падает, а сохраняет counter img.section_raw_oob. Heuristics превратит его в finding и score.

Как продолжается выполнение: Дальше проверяется virtual-диапазон секции.

Шаг 19. Проверка virtual-диапазона секции на выход за SizeOfImage

Режим выполнения: Всегда для каждой секции.

Где в коде: src/pe.c:200-204; finding emission - src/heuristics.c:796-802

Что проверяется: Проверяется, что VirtualAddress + max(VirtualSize, SizeOfRawData) не выходит за SizeOfImage.

Как проверяется технически: Код берет virtual_span = max(virtual_size, raw_size), потому что фактический виртуальный охват секции для анализа должен учитывать и заявленный virtual размер, и raw payload. Затем virtual_end сравнивается с SizeOfImage.

Зачем это нужно: SizeOfImage - верхняя граница образа в памяти. Если секция выходит за нее, loader-модель нарушена: таблицы или код могут ссылаться на область вне image.

Какой вывод делает проект: Если все секции внутри image - virtual layout согласован. Нарушение увеличивает img.section_virtual_oob.

Finding / счетчик / статус: pe_section_virtual_oob, family table_integrity, high, score +20. Значит: секция описывает память за пределами image.

Решение программы: Программа продолжает анализ, потому что другие секции и таблицы могут быть полезны как evidence.

Как продолжается выполнение: Дальше проверяется выравнивание и порядок секций.

Шаг 20. Проверка выравнивания и порядка секций

Режим выполнения: Всегда для каждой секции.

Где в коде: src/pe.c:205-213; finding emission - src/heuristics.c:817-823

Что проверяется: Проверяются кратность VirtualAddress к SectionAlignment, кратность raw_ptr/raw_size к FileAlignment и монотонный порядок VirtualAddress.

Как проверяется технически: Если VirtualAddress не кратен SectionAlignment, raw_ptr/raw_size не кратны FileAlignment или текущая секция идет не после предыдущей по VirtualAddress, counter misaligned увеличивается.

Зачем это нужно: Loader ожидает согласованную раскладку. Нарушения не всегда вредоносны, но они ломают доверие к section table как к карте файла.

Какой вывод делает проект: Нормальный результат - counter остается нулевым. Нарушения попадают в общий pe_section_misaligned.

Finding / счетчик / статус: pe_section_misaligned, medium, score +8. Это более мягкий сигнал, потому что некоторые toolchain/packer варианты могут быть нестандартными без вредоносности.

Решение программы: Программа не останавливается. Она сохраняет counter, а дальнейшие проверки покажут, коррелирует ли он с unwind проблемами.

Как продолжается выполнение: Дальше ищутся пересечения raw/virtual диапазонов.

Шаг 21. Проверка raw-пересечений секций

Режим выполнения: Всегда попарно для секций.

Где в коде: src/pe.c:215-219; helper ranges_overlap_u64 - src/pe.c:28-34; finding emission - src/heuristics.c:803-809

Что проверяется: Проверяется, пересекаются ли физические диапазоны секций на диске.

Как проверяется технически: Для каждой пары текущая/предыдущая секция вызывается ranges_overlap_u64(raw_ptr, raw_size, ...). Нулевые диапазоны не считаются пересечениями.

Зачем это нужно: Две секции обычно не должны претендовать на одни и те же raw bytes. Такое пересечение может быть упаковкой, overlay-трюком, ошибкой линковки или попыткой запутать parser.

Какой вывод делает проект: Если пересечений нет - raw-карта однозначна. Если есть - увеличивается raw_overlaps.

Finding / счетчик / статус: pe_section_raw_overlap, family table_integrity, medium, score +15. Medium, потому что физическое пересечение подозрительно, но иногда встречается в нестандартных бинарях.

Решение программы: Программа продолжает, но downstream-переводы RVA в file offset становятся менее доверенными.

Как продолжается выполнение: Дальше проверяются virtual-пересечения секций.

Шаг 22. Проверка virtual-пересечений секций

Режим выполнения: Всегда попарно для секций.

Где в коде: src/pe.c:220-226; finding emission - src/heuristics.c:810-816

Что проверяется: Проверяется, пересекаются ли виртуальные ranges секций в загруженном image.

Как проверяется технически: Для каждой пары сравниваются [VirtualAddress, VirtualAddress + max(VirtualSize, RawSize)). Если ranges пересекаются, virtual_overlaps увеличивается.

Зачем это нужно: В памяти две секции не должны неоднозначно покрывать один RVA. Иначе непонятно, к какой секции относится UnwindInfo, handler или код.

Какой вывод делает проект: Нормальный результат - нет пересечений. Нарушение означает, что RVA-to-section mapping неоднозначен.

Finding / счетчик / статус: pe_section_virtual_overlap, family table_integrity, high, score +20. High, потому что это ломает виртуальную координатную систему.

Решение программы: Программа продолжает, но findings и representative locations показывают пользователю, что секционная карта сомнительна.

Как продолжается выполнение: Дальше проверяется сам Exception Directory на границы image.

Блок C. PE parser: Data Directory boundary

На этом уровне Exception Directory проверяется как виртуальный диапазон внутри image.

Шаг 23. Проверка пары Exception RVA/Size на согласованность и SizeOfImage

Режим выполнения: Всегда после секций.

Где в коде: src/pe.c:230-235; finding emission - src/heuristics.c:825-832

Что проверяется: Проверяется, не является ли Exception Directory полунулевой и не выходит ли [exception_rva, exception_rva + exception_size) за SizeOfImage.

Как проверяется технически: Если оба поля нулевые, каталог считается отсутствующим и exception_directory_oob не ставится. Если одно поле нулевое, другое нет, или конец диапазона больше SizeOfImage, выставляется exception_directory_oob.

Зачем это нужно: DataDirectory хранит RVA и Size, а не file offset. Поэтому первая смысловая граница - SizeOfImage: каталог должен описывать область внутри загруженного image.

Какой вывод делает проект: Успех означает: directory entry выглядит виртуально допустимой или отсутствует. Провал означает: источник таблицы .pdata поврежден или непригоден.

Finding / счетчик / статус: pe_exception_directory_oob, table_integrity, high, score +25. Значит: Exception Directory malformed, unreadable, incomplete или out of bounds.

Решение программы: Программа сохраняет флаг в PeImage и не обязана падать. Позже runtime-function reader сделает более физическую проверку чтения.

Как продолжается выполнение: Дальше PeImage заполняется итоговыми полями и counters.

Блок C. PE parser: модель PeImage

После этих шагов parser передает остальным модулям единую модель PE, а не разрозненные bytes.

Шаг 24. Фиксация результата PE parser

Режим выполнения: Всегда при успешном pe_parse.

Где в коде: src/pe.c:237-256; PeImage - include/pe.h:33-54

Что проверяется: Проект сохраняет подтвержденные поля image и счетчики нарушений в структуру PeImage.

Как проверяется технически: В out попадают machine, image_base, size_of_image, size_of_headers, alignments, exception_rva/size, file_size, sections, num_sections и counters section_* / exception_directory_oob.

Зачем это нужно: Это момент перехода от низкоуровневого чтения байтов к общей модели анализа. Дальше unwind, heuristics, dynamic и report работают не с сырыми заголовками напрямую, а с PeImage.

Какой вывод делает проект: Успех pe_parse означает: формат базово распознан и можно пробовать читать .pdata. Провал до этой точки останавливал программу.

Finding / счетчик / статус: Сами findings еще не созданы, но данные для первых 6 findings уже накоплены.

Решение программы: main получает img и продолжает на unwind_read_runtime_functions_ex.

Как продолжается выполнение: Дальше начинается чтение Exception Directory как массива RUNTIME_FUNCTION.

Блок D. RVA mapping

Этот блок объясняет самое важное для защиты: RVA, VA и file offset - разные координаты, и проект не смешивает их.

Шаг 25. Поиск секции для RVA

Режим выполнения: Используется при чтении Exception Directory, UNWIND_INFO, placement и handler checks.

Где в коде: src/pe.c:265-274

Что проверяется: Проверяется, попадает ли RVA в виртуальный диапазон какой-либо секции.

Как проверяется технически: pe_find_section_for_rva проходит по sections и сравнивает rva с [VirtualAddress, VirtualAddress + max(VirtualSize, RawSize)). Если попал, возвращается PeSection.

Зачем это нужно: RVA - это не file offset. Без section table нельзя понять, где байты лежат на диске и какие права имеет область.

Какой вывод делает проект: Если секция найдена - можно использовать ее raw_ptr/raw_size/characteristics. Если нет - это может быть headers-RVA, отсутствие данных или поврежденный указатель.

Finding / счетчик / статус: Сам по себе не создает finding, но питает unwind_unusual_section, handler_nonexec_section, handler_writable_section и физическое чтение.

Решение программы: Функция возвращает указатель на секцию или NULL; решение принимает вызывающий код.

Как продолжается выполнение: Дальше при необходимости вызывается pe_rva_to_fileoff.

Шаг 26. Перевод RVA в file offset с учетом zero-fill и headers

Режим выполнения: Используется при чтении из файла.

Где в коде: src/pe.c:277-300

Что проверяется: Проверяется, можно ли не просто найти секцию, а реально прочитать bytes по этому RVA из файла.

Как проверяется технически: Если RVA попал в секцию, вычисляется delta = rva - section.VirtualAddress. Если delta >= raw_size, функция возвращает false: это виртуальный хвост, который loader заполнит нулями, но в файле этих байтов нет. Если секция не найдена, но rva < SizeOfHeaders и rva < file_size, headers-RVA маппится напрямую.

Зачем это нужно: Это один из самых важных нюансов проекта: virtual layout и file layout не равны. Static parser читает только те байты, которые физически есть на диске.

Какой вывод делает проект: Успех дает file offset. Провал означает: RVA не читается из файла, даже если теоретически существует в памяти.

Finding / счетчик / статус: Провал может привести к pe_exception_directory_oob при Exception Directory или unwind_parse_fail при UNWIND_INFO.

Решение программы: Функция не падает и не придумывает bytes. Она возвращает false, чтобы верхний слой принял честное решение.

Как продолжается выполнение: Дальше RuntimeFunction reader пытается прочитать .pdata.

Блок E. Чтение Exception Directory

Здесь проект проверяет, можно ли превратить Exception Directory в массив RUNTIME_FUNCTION entries.

Шаг 27. Случай полного отсутствия Exception Directory

Режим выполнения: Всегда после pe_parse.

Где в коде: src/main.c:306-323; src/unwind.c:611-625

Что проверяется: Проверяется пара img.exception_rva == 0 и img.exception_size == 0.

Как проверяется технически: unwind_read_runtime_functions_ex при обоих нулях возвращает RUNTIME_FUNCTION_READ_OK с out_count = 0 и out_rf = NULL. Это не ошибка чтения: каталог отсутствует по PE-модели.

Зачем это нужно: На x64 отсутствие static .pdata может быть нормальным только для очень простого leaf-only/resource/forwarding образа. Поэтому это не hard error, но может стать эвристическим finding, если есть executable code.

Какой вывод делает проект: Если нет executable sections - heuristics добавит note, а не finding. Если executable sections есть - позже создается no_runtime_functions.

Finding / счетчик / статус: finding no_runtime_functions создается в src/heuristics.c:833-842, score +10, medium.

Решение программы: Программа продолжает: rf_count = 0, массивы us/us_ok выделяются размером 1 для безопасности, дальше отчет может быть создан.

Как продолжается выполнение: Дальше обрабатывается полунулевой или ненулевой Exception Directory.

Шаг 28. Полунулевая пара RVA/Size

Режим выполнения: Всегда, если каталог не полностью отсутствует.

Где в коде: src/unwind.c:626-628; src/main.c:308-323

Что проверяется: Проверяется ситуация, когда exception_rva == 0, а size != 0, или наоборот.

Как проверяется технически: Reader возвращает RUNTIME_FUNCTION_READ_MALFORMED с сообщением 'RVA and size must both be zero or non-zero'. main не завершает процесс полностью, а выставляет img.exception_directory_oob = true, пишет partial structural report и ставит exit_code = 1.

Зачем это нужно: Такая комбинация говорит, что directory entry не описывает нормальную область. Нельзя безопасно читать .pdata, но можно показать пользователю структурную проблему.

Какой вывод делает проект: Вывод: Exception Directory malformed. Дальнейшие RF/UNWIND проверки для записей не выполняются, потому что записей нет.

Finding / счетчик / статус: pe_exception_directory_oob, high, score +25, если включены heuristics.

Решение программы: Программа продолжает частично: выводит brief/report, но финальный exit code будет 1.

Как продолжается выполнение: Дальше, если каталог был нормальным, reader пытается перевести RVA в file offset.

Шаг 29. Маппинг Exception Directory RVA в file offset

Режим выполнения: Всегда для ненулевого каталога.

Где в коде: src/unwind.c:630-633; src/pe.c:277-300

Что проверяется: Проверяется, можно ли VirtualAddress из DataDirectory превратить в offset файла.

Как проверяется технически: unwind_read_runtime_functions_ex вызывает pe_rva_to_fileoff(img, img->exception_rva, &off). Если false, значит каталог не лежит в читаемой raw-области или headers.

Зачем это нужно: DataDirectory указывает RVA. Если parser пошел бы сразу на offset = RVA, он мог бы прочитать не те bytes. Здесь проект явно соблюдает PE-координаты.

Какой вывод делает проект: Успех дает off - старт таблицы RUNTIME_FUNCTION на диске. Провал означает: каталог виртуально заявлен, но статически не читается.

Finding / счетчик / статус: pe_exception_directory_oob через main:321 и heuristics:825-832.

Решение программы: При malformed reader main продолжает частичный отчет, но rf_count остается 0 и exit_code становится 1.

Как продолжается выполнение: Если offset получен, проверяется физический диапазон off + size.

Шаг 30. Проверка физического диапазона Exception Directory

Режим выполнения: Всегда после successful RVA mapping.

Где в коде: src/unwind.c:635-637

Что проверяется: Проверяется, что off + exception_size не выходит за размер файла.

Как проверяется технически: Даже если начало таблицы маппится, заявленный Size может увести чтение за EOF. Код сравнивает (size_t)off + img->exception_size с b->size.

Зачем это нужно: Это вторая граница после SizeOfImage: сначала каталог должен быть допустим в memory image, затем реально читаться из файла.

Какой вывод делает проект: Успех позволяет проверить размер записи. Провал означает: таблица физически обрезана или Size ложный.

Finding / счетчик / статус: pe_exception_directory_oob. Смысл: directory unreadable/out of file range.

Решение программы: При провале main сохраняет частичный структурный отчет и выставляет exit_code = 1.

Как продолжается выполнение: Дальше проверяется минимальный размер и кратность RUNTIME_FUNCTION.

Шаг 31. Проверка минимального размера RUNTIME_FUNCTION table

Режим выполнения: Всегда для физически читаемого каталога.

Где в коде: src/unwind.c:639-641; RuntimeFunction - include/unwind.h:16-20

Что проверяется: Проверяется, что Exception Directory size не меньше 12 байт.

Как проверяется технически: На x64 одна запись RUNTIME_FUNCTION состоит из трех DWORD: BeginAddress, EndAddress, UnwindInfoAddress. Это 12 байт. Если size < 12, ни одной полноценной записи нет.

Зачем это нужно: Каталог может быть ненулевым, но бесполезным: указал область, которая меньше одной структуры. Это сильный структурный дефект.

Какой вывод делает проект: Успех означает, что теоретически есть хотя бы одна запись. Провал - malformed.

Finding / счетчик / статус: pe_exception_directory_oob через частичный report.

Решение программы: Программа не читает неполную запись, чтобы не создать ложные Begin/End/UnwindRVA.

Как продолжается выполнение: Дальше проверяется кратность size к 12.

Шаг 32. Проверка кратности Size размеру RUNTIME_FUNCTION

Режим выполнения: Всегда для физически читаемого каталога.

Где в коде: src/unwind.c:643-647

Что проверяется: Проверяется, что exception_size % 12 == 0.

Как проверяется технически: Если остаток не ноль, каталог не является чистым массивом RUNTIME_FUNCTION. Значит, parser либо читает не ту область, либо файл поврежден/обфусцирован.

Зачем это нужно: Это пример проверки, где число 12 не придумано разработчиком, а следует из структуры Windows x64 RUNTIME_FUNCTION: 3 поля DWORD.

Какой вывод делает проект: Успех позволяет вычислить n = size / 12. Провал значит, что безопасно разбить каталог на записи невозможно.

Finding / счетчик / статус: pe_exception_directory_oob. В сообщении reader прямо пишет 'not a multiple of RUNTIME_FUNCTION size'.

Решение программы: При провале main продолжает только частичный structural report, без per-entry анализа.

Как продолжается выполнение: Дальше выделяется массив RuntimeFunction.

Шаг 33. Выделение массива RUNTIME_FUNCTION

Режим выполнения: Всегда, если size кратен 12.

Где в коде: src/unwind.c:647-652

Что проверяется: Проверяется, может ли программа выделить память под n записей RUNTIME_FUNCTION.

Как проверяется технически: n вычисляется как exception_size / 12, затем calloc(n, sizeof(RuntimeFunction)). При OOM reader возвращает RUNTIME_FUNCTION_READ_OOM.

Зачем это нужно: Это не свойство бинарника, но важная robustness-проверка. Большой size может быть злоупотреблением против анализатора.

Какой вывод делает проект: Успех дает массив для записей. Провал означает проблему ресурсов.

Finding / счетчик / статус: Нет finding. OOM - ошибка выполнения инструмента, а не доказательство PE-аномалии.

Решение программы: main при RUNTIME_FUNCTION_READ_OOM завершает работу с кодом 1 и освобождает ресурсы.

Как продолжается выполнение: Дальше каждая запись читается из файла.

Шаг 34. Чтение каждой записи RUNTIME_FUNCTION

Режим выполнения: Всегда для n > 0.

Где в коде: src/unwind.c:654-667; include/unwind.h:16-20

Что проверяется: Для каждой записи читаются BeginAddress, EndAddress и UnwindInfoAddress.

Как проверяется технически: Для i-й записи offset = off + i * 12. Три buf_read_u32 читают bva, eva, uva, затем они сохраняются как begin_rva, end_rva, unwind_rva.

Зачем это нужно: На этом этапе проект еще не решает, нормальные ли диапазоны. Он только честно переносит таблицу из файла в структурированный массив.

Какой вывод делает проект: Если чтение всех записей прошло, rf_count = n. Если конкретная запись не читается, весь read считается malformed.

Finding / счетчик / статус: Провал чтения ведет к pe_exception_directory_oob; дальнейшие RF findings невозможны без массива.

Решение программы: При успехе main печатает brief и продолжает к UNWIND_INFO. При malformed - частичный report.

Как продолжается выполнение: Дальше создаются массивы UnwindSummary/us_ok.

Блок F. Подготовка к UNWIND_INFO

Этот блок готовит место для результатов decode и опции разбора chain/ABI.

Шаг 35. Выделение us/us_ok и настройка unwind parse options

Режим выполнения: Всегда после чтения RF table, даже если rf_count = 0.

Где в коде: src/main.c:328-346; UnwindSummary - include/unwind.h:22-43

Что проверяется: Проверяется, можно ли выделить массив результатов parse для каждой записи и настроить follow_chain/strict_abi/max_chain_depth.

Как проверяется технически: calloc выделяет UnwindSummary и bool us_ok. Если rf_count ноль, выделяется один элемент, чтобы не работать с NULL в дальнейшем. Затем main заполняет UnwindParseOpts из аргументов.

Зачем это нужно: us хранит decoded факты: version, flags, prolog_size, count_codes, stack bytes, handler, chain и anomaly bits. us_ok отделяет успешно разобранные UNWIND_INFO от провалившихся.

Какой вывод делает проект: Успех позволяет перейти к per-entry decode. OOM останавливает программу.

Finding / счетчик / статус: Нет finding на выделение памяти. Но us_ok позже порождает unwind_parse_fail.

Решение программы: При OOM main освобождает уже выделенное и возвращает 1.

Как продолжается выполнение: Дальше main запускает цикл по RUNTIME_FUNCTION entries.

Блок G. UNWIND_INFO decoder

Здесь проект разбирает .xdata: структуру UNWIND_INFO, unwind codes, operands, chain и handler layer.

Шаг 36. Вызов unwind_parse для каждой записи

Режим выполнения: Всегда для каждой RUNTIME_FUNCTION.

Где в коде: src/main.c:351-360; src/unwind.c:165-608

Что проверяется: Для каждого rf[i] проверяется и декодируется UNWIND_INFO по адресу rf[i].unwind_rva.

Как проверяется технически: main создает perr, вызывает unwind_parse, сохраняет результат bool в us_ok[i], а сам summary кладет в us[i]. Даже если parse не прошел, anomalies и часть контекста могут быть сохранены.

Зачем это нужно: Это отделяет чтение таблицы .pdata от проверки .xdata. Таблица может существовать, но ее UnwindInfoAddress может указывать на мусор.

Какой вывод делает проект: Если ok=true, запись идет в placement/reuse/handler analysis. Если ok=false, heuristics позже увеличит unwind_parse_fail.

Finding / счетчик / статус: unwind_parse_fail создается в src/heuristics.c:893-899; related anomalies - src/heuristics.c:900-906.

Решение программы: Программа не останавливается на одной плохой функции. Она анализирует все записи, чтобы увидеть масштаб проблемы.

Как продолжается выполнение: Дальше внутри unwind_parse проверяется alignment и header.

Шаг 37. Проверка 4-byte alignment UnwindInfoRVA

Режим выполнения: Для каждой записи.

Где в коде: src/unwind.c:187-195

Что проверяется: Проверяется, кратен ли unwind_rva четырем.

Как проверяется технически: Если (unwind_rva & 3) != 0, выставляется anomaly UA_ALIGNMENT, но parser все равно пытается прочитать header.

Зачем это нужно: UNWIND_INFO обычно выровнен, потому что tail и chained info содержат DWORD-aligned структуры. Некратность - сигнал, но не всегда причина немедленно бросать запись.

Какой вывод делает проект: Успех - anomaly не ставится. Провал - запись остается анализируемой, но позже может попасть в unwind_anomalies.

Finding / счетчик / статус: unwind_anomalies, если parse завершился ok и anomaly не относится только к ABI warning.

Решение программы: Решение мягкое: не останавливать decode, потому что иногда можно прочитать и получить больше evidence.

Как продолжается выполнение: Дальше parser читает UNWIND_INFO header.

Шаг 38. Маппинг и чтение UNWIND_INFO header

Режим выполнения: Для каждой записи.

Где в коде: src/unwind.c:56-72; вызов - src/unwind.c:192-195; pe_rva_to_fileoff - src/pe.c:277-300

Что проверяется: Проверяется, можно ли перевести unwind_rva в file offset и прочитать первые 4 байта UNWIND_INFO.

Как проверяется технически: parse_unwindinfo_header вызывает pe_rva_to_fileoff, затем buf_read 4 байта. Если RVA попал в zero-fill или за файл, функция возвращает false.

Зачем это нужно: Это техническая проверка доступности .xdata. RUNTIME_FUNCTION может указывать на адрес в image, но статический parser читает только физические bytes.

Какой вывод делает проект: Успех дает hdr[0..3]. Провал означает, что UNWIND_INFO не читается.

Finding / счетчик / статус: unwind_parse_fail, обычно с UA_TRUNCATED.

Решение программы: При провале unwind_parse возвращает false; main продолжает следующие записи.

Как продолжается выполнение: Дальше декодируются version, flags, prolog, count, frame register.

Шаг 39. Проверка версии UNWIND_INFO

Режим выполнения: Для каждой успешно прочитанной header.

Где в коде: src/unwind.c:197-214

Что проверяется: Проверяется, что version равна 1 или 2.

Как проверяется технически: Нижние 3 бита hdr[0] дают ver. Если ver не 1 и не 2, ставится UA_VERSION_UNKNOWN и parse завершается failure.

Зачем это нужно: Версия определяет смысл unwind opcodes, особенно v2 epilog descriptors. Неизвестную версию нельзя безопасно декодировать.

Какой вывод делает проект: Успех позволяет проверять flags и slots. Провал делает запись unreadable для проекта.

Finding / счетчик / статус: unwind_parse_fail. Это сильнее, чем просто anomaly, потому что decoder не знает формат.

Решение программы: Решение: не продолжать по неизвестному формату, чтобы не выдумывать структуру.

Как продолжается выполнение: Дальше проверяются flags и frame fields.

Шаг 40. Проверка flags и несовместимых комбинаций

Режим выполнения: Для каждой записи.

Где в коде: src/unwind.c:215-220; flags - include/unwind.h:45-49

Что проверяется: Проверяется, что установлены только известные флаги EHANDLER, UHANDLER, CHAININFO, и что CHAININFO не смешан с EHANDLER/UHANDLER.

Как проверяется технически: Код маскирует flags неизвестными битами и отдельно проверяет конфликт chain+handler. При нарушении ставится UA_BAD_OPERAND, но parse может продолжаться.

Зачем это нужно: Flags определяют tail после unwind codes: либо chained RUNTIME_FUNCTION, либо handler RVA. Неверная комбинация делает tail неоднозначным.

Какой вывод делает проект: Успех - tail имеет понятный тип. Нарушение - потенциальная логическая порча UNWIND_INFO.

Finding / счетчик / статус: unwind_anomalies, если остальной parse прошел.

Решение программы: Решение мягкое: продолжить decode, чтобы увидеть handler/chain и собрать больше фактов.

Как продолжается выполнение: Дальше проверяются frame register и ABI warnings.

Шаг 41. Проверка frame register / frame offset

Режим выполнения: Для каждой записи.

Где в коде: src/unwind.c:221-229; src/unwind.c:515-517

Что проверяется: Проверяется согласованность frame register, frame offset и наличия UWOP_SET_FPREG.

Как проверяется технически: Если fr == 0, но foff != 0, это bad operand. Если fr не является допустимым unwindable GPR - bad operand. В strict ABI mode дополнительно проверяется, что register относится к nonvolatile. После обработки codes, если fr != 0, но SET_FPREG не встретился и нет CHAININFO, тоже ставится bad operand.

Зачем это нужно: Frame register описывает базу кадра. Если header говорит, что frame register есть, но codes не устанавливают его, virtual unwind может восстановить frame неверно.

Какой вывод делает проект: Успех подтверждает внутреннюю согласованность header/codes. Нарушение повышает anomaly count.

Finding / счетчик / статус: unwind_anomalies или ABI warning в зависимости от флага.

Решение программы: Программа продолжает, потому что другие поля записи все еще могут быть полезны для placement/reuse.

Как продолжается выполнение: Дальше проверяется массив unwind codes.

Шаг 42. Проверка длины массива unwind codes

Режим выполнения: Для каждой записи.

Где в коде: src/unwind.c:231-239

Что проверяется: Проверяется, что CountOfCodes с учетом выравнивания до even slots помещается в файле.

Как проверяется технически: slots_even = (count + 1) & ~1. codes_off = uoff + 4. codes_bytes = slots_even * 2. Если codes_off + codes_bytes > file size, ставится UA_TRUNCATED и parse fails.

Зачем это нужно: CountOfCodes задает количество 2-byte slots. Tail handler/chain начинается после even-aligned массива. Если массив выходит за файл, весь tail тоже ненадежен.

Какой вывод делает проект: Успех позволяет читать opcodes. Провал - UNWIND_INFO truncated.

Finding / счетчик / статус: unwind_parse_fail.

Решение программы: Решение: остановить decode этой записи, потому что дальнейшие reads будут за пределами файла.

Как продолжается выполнение: Дальше parser идет по slots 0..count-1.

Шаг 43. Проверка opcode и количества slots на операцию

Режим выполнения: Для каждой unwind operation.

Где в коде: src/unwind.c:251-275; slots_for_code - src/unwind.c:24-54

Что проверяется: Проверяется, известен ли opcode для данной версии и не выходит ли операция за CountOfCodes.

Как проверяется технически: Parser читает 16-bit slot, выделяет code_offset, op, opinfo. slots_for_code возвращает, сколько slots занимает операция. Если opcode неизвестен, ставится UA_BAD_OPCODE и parser пытается продолжить как 1 slot. Если i + slots > count - UA_TRUNCATED и failure.

Зачем это нужно: UNWIND_CODE - переменной длины: например ALLOC_LARGE может занимать 2 или 3 slots. Нельзя просто читать по одному slot без знания opcode.

Какой вывод делает проект: Успех означает, что текущая операция структурно помещается. Провал по length - запись unreadable.

Finding / счетчик / статус: unwind_anomalies для unknown opcode, unwind_parse_fail для выхода за CountOfCodes.

Решение программы: Решение: unknown opcode - продолжить осторожно; truncated opcode - остановить запись.

Как продолжается выполнение: Дальше проверяются code offsets и порядок операций.

Шаг 44. Проверка CodeOffset относительно prolog и порядка

Режим выполнения: Для каждой unwind operation, кроме v2 epilog descriptors.

Где в коде: src/unwind.c:276-285

Что проверяется: Проверяется, что code_offset не больше prolog_size и что offsets идут в ожидаемом порядке.

Как проверяется технически: Если code_offset > prolog, ставится UA_CODEOFFSET_OUTSIDE. Если текущий code_offset больше предыдущего, ставится UA_CODEOFFSET_ORDER.

Зачем это нужно: Unwind codes описывают действия в прологе функции. Если код говорит об offset за пределами prolog или странном порядке, карта восстановления кадра выглядит подозрительно.

Какой вывод делает проект: Успех - prolog layout согласован. Нарушение - anomaly, но не обязательно full parse failure.

Finding / счетчик / статус: unwind_anomalies.

Решение программы: Решение: продолжить decode, потому что stack effects и handler still matter.

Как продолжается выполнение: Дальше применяются strict ABI правила, если включены.

Шаг 45. Опциональная strict ABI проверка порядка prolog действий

Режим выполнения: Опционально: только если включен strict_abi.

Где в коде: src/unwind.c:287-295

Что проверяется: Проверяется, что после PUSH_NONVOL не идут неожиданные операции, и что CHAININFO ограничивает допустимые ops.

Как проверяется технически: Если saw_push_nonvol уже true, а следующая операция не PUSH_NONVOL и не PUSH_MACHFRAME, ставится UA_ABI_PUSH_ORDER. Если CHAININFO и op != SAVE_NONVOL, ставится UA_ABI_CHAIN_RESTRICTION.

Зачем это нужно: Это строгая проверка ABI-канона. Она полезна для ручной валидации, но не должна давать слишком жесткий verdict сама по себе.

Какой вывод делает проект: Успех - ABI-порядок выглядит нормальным. Нарушение увеличивает ABI warning.

Finding / счетчик / статус: Не основной finding, а unwind_abi_warnings. В src/heuristics.c:734-735 ABI mask не идет в unwind_anomalies как hard anomaly.

Решение программы: Программа продолжает. Эти предупреждения помогают аналитикам, но не доминируют score.

Как продолжается выполнение: Дальше decoder вычисляет effects операций.

Шаг 46. Вычисление stack effects и проверка operands

Режим выполнения: Для каждой операции.

Где в коде: src/unwind.c:346-510

Что проверяется: Проверяются operands для PUSH_NONVOL, ALLOC_LARGE, ALLOC_SMALL, SET_FPREG, SAVE_NONVOL, SAVE_XMM, PUSH_MACHFRAME и др.; параллельно считается fixed_stack_bytes и сохраненные register masks.

Как проверяется технически: Для операций с дополнительными slots reader читает operands через read_u16_slot/read_u32_from_slots. Если operand не помещается, parse fails. Если register/opinfo невозможен или размер выглядит sanity-suspicious, ставятся anomaly bits.

Зачем это нужно: Это уже не простая синтаксическая проверка. Проект строит семантическое резюме: сколько стековых байт резервируется, какие nonvolatile registers сохранены, есть ли странно огромные offsets.

Какой вывод делает проект: Успех дает полезный UnwindSummary. Нарушения operands дают unwind_anomalies или unwind_parse_fail.

Finding / счетчик / статус: unwind_parse_fail при truncation; unwind_anomalies при bad operands/sanity.

Решение программы: Программа продолжает по всем функциям, чтобы найти процент и корреляции, а не один isolated byte.

Как продолжается выполнение: Дальше читается tail: chain или handler.

Шаг 47. Проверка CHAININFO tail

Режим выполнения: Только если UNW_FLAG_CHAININFO установлен.

Где в коде: src/unwind.c:519-538

Что проверяется: Проверяется, что после unwind codes лежит chained RUNTIME_FUNCTION и что его Begin/End/UnwindRVA выглядят валидно.

Как проверяется технически: parse_chain_rf читает 12 байт tail. Затем проверяется end > begin, begin/end внутри SizeOfImage, unwind_rva не нулевой и меньше SizeOfImage.

Зачем это нужно: CHAININFO означает, что текущая запись переиспользует или связывает unwind metadata другой функции. Это легитимно, но ломается при неверных границах.

Какой вывод делает проект: Успех: out->has_chain = true и chain сохранен. Нарушение - UA_CHAIN_INVALID или parse failure при truncation.

Finding / счетчик / статус: unwind_parse_fail при truncated chain; unwind_anomalies при invalid chain.

Решение программы: Решение: truncated tail останавливает запись; invalid values сохраняются как anomaly.

Как продолжается выполнение: Дальше проверяется handler tail, если установлен EHANDLER/UHANDLER.

Шаг 48. Проверка EHANDLER/UHANDLER tail

Режим выполнения: Только если установлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER.

Где в коде: src/unwind.c:540-551

Что проверяется: Проверяется, что после unwind codes можно прочитать Handler RVA и что он не выходит за SizeOfImage.

Как проверяется технически: buf_read_u32 читает handler_rva из tail. Если чтение не удалось - UA_TRUNCATED и parse failure. Если handler_rva >= SizeOfImage - UA_HANDLER_OOB.

Зачем это нужно: Handler layer отвечает за переход к language-specific обработчику. Если handler указывает вне image, Windows не сможет нормально применить этот обработчик или это будет признак вмешательства.

Какой вывод делает проект: Успех: out->has_handler = true, handler_rva сохранен. Нарушение дает anomaly или failure.

Finding / счетчик / статус: unwind_parse_fail при truncated handler; unwind_anomalies при handler OOB. Placement findings появятся позже по section rights.

Решение программы: Решение: сохранить handler для корреляции, если он хотя бы прочитан.

Как продолжается выполнение: Дальше может выполняться follow_chain.

Шаг 49. Опциональное следование chained unwind info

Режим выполнения: Опционально: follow_chain включен по умолчанию, ограничен max_chain_depth.

Где в коде: src/unwind.c:559-604; options - src/main.c:342-346

Что проверяется: Проверяется, не образуют ли chained UNWIND_INFO петлю, не превышают ли глубину и можно ли разобрать следующие chain entries.

Как проверяется технически: Decoder хранит seen[16], идет по chain.unwind_rva, на каждом шаге вызывает unwind_parse с follow_chain=false, суммирует fixed_stack_bytes/register masks и проверяет consistency frame_reg/frame_off.

Зачем это нужно: Chained unwind может быть нормальной оптимизацией, но циклы, слишком глубокие цепочки и несовместимые frame fields делают virtual unwind ненадежным.

Какой вывод делает проект: Успех расширяет summary текущей записи. Нарушения ставят UA_CHAIN_LOOP, UA_CHAIN_DEPTH_EXCEEDED, UA_TRUNCATED или UA_BAD_OPERAND.

Finding / счетчик / статус: unwind_anomalies или unwind_parse_fail в зависимости от того, где сломалось.

Решение программы: Программа не зацикливается: depth ограничивается максимумом 16.

Как продолжается выполнение: Дальше unwind_parse возвращает valid=true или false.

Блок H. Статические эвристики: вход

С этого момента измеренные факты превращаются в evidence report.

Шаг 50. Запуск heuristics_analyze

Режим выполнения: Опционально: только если пользователь включил heuristics.

Где в коде: src/main.c:382-399; src/heuristics.c:590-604

Что проверяется: Проверяется, можно ли создать HeuristicReport и связать PeImage, RUNTIME_FUNCTION, UnwindSummary и us_ok.

Как проверяется технически: heuristics_analyze проверяет входные указатели, инициализирует details buffer и сохраняет total = n.

Зачем это нужно: Это переход от 'сырых проверок' к evidence engine: теперь отдельные counters становятся findings, score, confidence и verdict.

Какой вывод делает проект: Успех дает report hr. Провал обычно связан с memory allocation, а не с бинарником.

Finding / счетчик / статус: Нет finding при ошибке самого анализатора.

Решение программы: При ошибке heuristics main печатает Heuristic analysis failed и ставит exit_code = 1, но остальные включенные outputs могут продолжить.

Как продолжается выполнение: Дальше копируются PE counters.

Блок H. Статические эвристики: PE counters

Этот блок связывает результаты parser-a с scoring engine.

Шаг 51. Перенос structural counters из PeImage

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:604-610

Что проверяется: Проверяется и переносится все, что PE parser уже накопил: raw_oob, virtual_oob, raw_overlaps, virtual_overlaps, misaligned, exception_directory_oob.

Как проверяется технически: Эвристика не перечитывает PE заново. Она берет проверенные counters из PeImage, чтобы later scoring был привязан к одному источнику истины.

Зачем это нужно: Это важно архитектурно: parser отвечает за корректное чтение и структурную валидацию, heuristics - за интерпретацию и вес.

Какой вывод делает проект: Если counters нулевые - первые structural findings не появятся. Если ненулевые - будут созданы findings 1-6.

Finding / счетчик / статус: pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob.

Решение программы: Решение программы: не менять counters, а превратить их в findings на этапе scoring.

Как продолжается выполнение: Дальше анализируются поля каждой RUNTIME_FUNCTION.

Блок H. Статические эвристики: RUNTIME_FUNCTION

Здесь проверяется карта функций: границы, сортировка, выходы за image и ссылки на unwind metadata.

Шаг 52. Проверка Begin/End диапазона функции

Режим выполнения: Если heuristics включены, для каждой RF entry.

Где в коде: src/heuristics.c:646-654; finding emission - src/heuristics.c:850-856

Что проверяется: Проверяется, что EndAddress больше BeginAddress.

Как проверяется технически: Для каждой записи b = begin_rva, e = end_rva. Если e <= b, увеличивается rf_invalid_ranges и запоминается representative location.

Зачем это нужно: RUNTIME_FUNCTION описывает полуоткрытый диапазон функции [Begin, End). Нулевая или обратная длина означает, что карта функции математически неверна.

Какой вывод делает проект: Успех: функция имеет положительный диапазон. Нарушение: invalid_ranges.

Finding / счетчик / статус: invalid_ranges, table_integrity, medium/high. Score +10 или +20, если доля >1%.

Решение программы: Программа не удаляет запись, а считает масштаб нарушения. Это позволяет увидеть: один битый элемент или системная подмена таблицы.

Как продолжается выполнение: Дальше проверяется попадание Begin/End в SizeOfImage.

Шаг 53. Проверка Begin/End против SizeOfImage

Режим выполнения: Если heuristics включены, для каждой RF entry.

Где в коде: src/heuristics.c:655-658; finding emission - src/heuristics.c:857-863

Что проверяется: Проверяется, что BeginAddress < SizeOfImage и EndAddress <= SizeOfImage.

Как проверяется технически: Если begin лежит за image или end выходит за image, увеличивается rf_oob.

Зачем это нужно: Begin/End - RVA внутри image. Если функция выходит за SizeOfImage, Windows unwind table ссылается на область, которая не является частью загруженного образа.

Какой вывод делает проект: Успех: function range внутри image. Нарушение: карта функций указывает за виртуальную границу.

Finding / счетчик / статус: rf_oob, table_integrity, medium/high. Score +10 или +25, если доля >1%.

Решение программы: Программа продолжает и сравнивает проценты, потому что единичный outlier и массовая порча имеют разную силу.

Как продолжается выполнение: Дальше проверяется UnwindInfoRVA.

Шаг 54. Проверка UnwindInfoRVA против SizeOfImage и нуля

Режим выполнения: Если heuristics включены, для каждой RF entry.

Где в коде: src/heuristics.c:659-662; finding emission - src/heuristics.c:864-870

Что проверяется: Проверяется, что UnwindInfoAddress не равен нулю и меньше SizeOfImage.

Как проверяется технически: Если u >= SizeOfImage или u == 0, увеличивается rf_unwind_oob.

Зачем это нужно: UnwindInfoRVA - ссылка на UNWIND_INFO. Ноль или выход за image означает, что запись не может указать на нормальные unwind metadata.

Какой вывод делает проект: Успех: ссылка виртуально допустима. Нарушение: запись .pdata не ведет к нормальной .xdata.

Finding / счетчик / статус: unwind_rva_oob, table_integrity, medium/high. Score +10 или +25 при доле >1%.

Решение программы: Даже если unwind_parse уже пытался читать, эта проверка фиксирует именно table-level дефект адреса.

Как продолжается выполнение: Дальше проверяется сортировка таблицы.

Шаг 55. Проверка сортировки по BeginAddress

Режим выполнения: Если heuristics включены, один проход по RF table.

Где в коде: src/heuristics.c:612-669; finding emission - src/heuristics.c:871-877

Что проверяется: Проверяется, что записи идут в неубывающем порядке BeginAddress.

Как проверяется технически: Код хранит prev_begin. Если текущий b < prev_begin, увеличивается unsorted и запоминается location.

Зачем это нужно: Windows lookup по function table опирается на упорядоченность. Нестандартный порядок может ломать поиск функции или указывать на ручную сборку таблицы.

Какой вывод делает проект: Успех: порядок нормальный. Нарушение: rf_unsorted.

Finding / счетчик / статус: rf_unsorted, table_integrity, medium, score +15.

Решение программы: Программа не переставляет исходную таблицу. Она фиксирует факт, а отдельную sorted copy использует только для проверки overlaps.

Как продолжается выполнение: Дальше считаются reuse hash maps.

Блок H. Статические эвристики: reuse counters

Здесь проект измеряет массовые паттерны: reuse metadata и доминирование handler-ов.

Шаг 56. Подсчет повторов UnwindInfoRVA и HandlerRVA

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:630-688; ratios - src/heuristics.c:775-785

Что проверяется: Проверяется, сколько разных функций ссылаются на один и тот же UnwindInfoRVA или один handler.

Как проверяется технически: Создаются hash maps unwind_counts и handler_counts. Для каждой RF увеличивается счетчик по u. Если us_ok[i] и has_handler, увеличивается counter по handler_rva. Позже берутся max value, unique count и ratios.

Зачем это нужно: Переиспользование unwind info бывает нормальным для одинаковых prologs, но массовое reuse вместе с handler dominance похоже на centralized dispatcher или protected binary pattern.

Какой вывод делает проект: Успех дает ratios. Нарушение памяти приводит к ошибке analyzer, не к finding.

Finding / счетчик / статус: unwind_reuse_strong, unwind_reuse_moderate, handler_dominance создаются позже, когда ratios сравниваются с thresholds.

Решение программы: Программа сначала считает факты, а не выдает вывод сразу: reuse без контекста может быть benign.

Как продолжается выполнение: Дальше проверяется placement UnwindInfo и handler.

Блок H. Статические эвристики: placement

Здесь проверяется соответствие роли данных их секции и правам памяти.

Шаг 57. Проверка секции, где лежит UNWIND_INFO

Режим выполнения: Если heuristics включены и us_ok[i] == true.

Где в коде: src/heuristics.c:690-712; section-name helpers - src/heuristics.c:86-144; finding emission - src/heuristics.c:964-987

Что проверяется: Проверяется, в какой секции лежит UnwindInfoRVA, соответствует ли имя ожидаемым .xdata/.rdata/.pdata-like секциям и нет ли прав EXECUTE/WRITE.

Как проверяется технически: pe_find_section_for_rva ищет секцию. Если секция не найдена - unwind_unusual_section. Если имя не allowed/plausible - тоже unusual. Если characteristics содержит IMAGE_SCN_MEM_EXECUTE - unwind_exec_section. Если IMAGE_SCN_MEM_WRITE - unwind_writable_section.

Зачем это нужно: UNWIND_INFO - metadata. Обычно она должна быть read-only data, а не executable code и не writable area. Имена секций не являются абсолютным законом, поэтому необычное имя мягче, чем executable/writable.

Какой вывод делает проект: Нормальный результат: .xdata/.rdata/.pdata-like, readable, not executable, not writable. Нарушения дают placement findings.

Finding / счетчик / статус: unwind_in_exec_section (+20 high), unwind_unusual_section (+8/+15), unwind_writable_section (+10/+18).

Решение программы: Программа считает количество таких записей и representative RVA, а затем решает severity по распространенности.

Как продолжается выполнение: Дальше проверяется placement handler.

Шаг 58. Проверка секции, где лежит exception handler

Режим выполнения: Если heuristics включены, us_ok[i] и has_handler.

Где в коде: src/heuristics.c:714-723; finding emission - src/heuristics.c:988-1000

Что проверяется: Проверяется, попадает ли handler_rva в executable section и не находится ли он в writable section.

Как проверяется технически: pe_find_section_for_rva ищет секцию handler. Если секции нет или нет IMAGE_SCN_MEM_EXECUTE, увеличивается handler_nonexec_section. Если секция есть и IMAGE_SCN_MEM_WRITE выставлен, увеличивается handler_writable_section.

Зачем это нужно: Handler - это код, которому Windows exception machinery передает управление. Он должен исполняться. Handler в non-executable области противоречит роли поля; writable handler повышает риск runtime-подмены.

Какой вывод делает проект: Успех: handler выглядит как исполняемый код. Нарушение: placement handler findings.

Finding / счетчик / статус: handler_nonexec_section (+15 high), handler_writable_section (+10 medium).

Решение программы: Программа не исполняет handler в static mode; она только проверяет координату и права секции.

Как продолжается выполнение: Дальше учитываются parse failures и anomaly counters.

Блок H. Статические эвристики: parse/anomaly counters

Здесь decoder results превращаются в количественные integrity-сигналы.

Шаг 59. Подсчет unwind_parse_fail и unwind_anomalies

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:727-735; emission - src/heuristics.c:893-906

Что проверяется: Проверяется, какие записи не удалось разобрать, а какие разобрались, но содержат anomaly bits.

Как проверяется технически: Если us_ok[i] false - unwind_parse_fail++. Если parse ok, но anomalies содержит не-ABI bits, увеличивается unwind_anomalies. ABI-only warnings считаются отдельно.

Зачем это нужно: Так проект разделяет 'не смог прочитать структуру' и 'прочитал, но внутри странные поля/комбинации'. Это важно для точности защиты.

Какой вывод делает проект: Успех: запись чистая или только ABI warning. Нарушение: hard unwind integrity signal.

Finding / счетчик / статус: unwind_parse_fail (+10/+25), unwind_anomalies (+10/+20). Threshold high: >1% parse fails, >5% anomalies.

Решение программы: Программа считает долю от общего n, потому что один необычный runtime helper и массовая поломка имеют разную доказательную силу.

Как продолжается выполнение: Дальше проверяются overlaps и duplicate begins на отсортированной копии.

Блок H. Статические эвристики: sorted integrity

Здесь проект проверяет математическую непротиворечивость function ranges независимо от исходного порядка таблицы.

Шаг 60. Проверка overlaps и duplicate BeginAddress

Режим выполнения: Если heuristics включены и n > 0.

Где в коде: src/heuristics.c:740-773; emission - src/heuristics.c:878-891

Что проверяется: Проверяется, пересекаются ли function ranges после сортировки и есть ли одинаковые BeginAddress.

Как проверяется технически: Создается массив RfIdx, копируются begin/end, qsort сортирует по begin. Затем код хранит max_end и sorted_prev_begin. Если begin == previous begin - duplicate. Если begin < max_end - overlap.

Зачем это нужно: Даже если исходная таблица была не отсортирована, overlap надо проверять по математическому порядку диапазонов. Это отличает сортировку как property таблицы от пересечений как property карты функций.

Какой вывод делает проект: Успех: функции не перекрываются. Нарушение: две записи претендуют на один кодовый range или один старт.

Finding / счетчик / статус: rf_overlaps (+25 high), duplicate_begin (+10 medium).

Решение программы: Программа не исправляет таблицу, а фиксирует конфликт как evidence.

Как продолжается выполнение: Дальше ratios сравниваются с thresholds и создаются reuse/dominance findings.

Блок H. Статические эвристики: thresholds

Здесь единичные события отделяются от массовых закономерностей.

Шаг 61. Оценка процентов и порогов

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:80-84; percentages - src/heuristics.c:844-848

Что проверяется: Проверяется масштаб проблемы: доли invalid/oob/parse/anomaly/reuse/handler dominance.

Как проверяется технически: safe_ratio делит count на total n. High percent для table defects - >1%, anomalies high - >5%, reuse moderate - >10%, reuse high - >30%, handler dominance - >80%, reuse рассматривается только при n > 200.

Зачем это нужно: Пороги не являются 'истиной природы'. Это инженерная калибровка: единичные артефакты не должны давить verdict так же, как массовая системная схема.

Какой вывод делает проект: Результат влияет на severity и score. Например rf_oob при >1% получает high-score, иначе low-score.

Finding / счетчик / статус: Связано с findings invalid_ranges, rf_oob, unwind_rva_oob, unwind_parse_fail, unwind_anomalies, unwind_reuse_*.

Решение программы: Программа принимает решение не бинарно, а через вес и confidence.

Как продолжается выполнение: Дальше создаются конкретные findings.

Блок H. Статические эвристики: emission

Здесь counters становятся формальными findings с кодами, весами, severity и locations.

Шаг 62. Создание первых 6 PE findings

Режим выполнения: Если heuristics включены и counters ненулевые.

Где в коде: src/heuristics.c:789-832

Что проверяется: Проверяются накопленные counters PeImage и добавляются findings 1-6.

Как проверяется технически: Для каждого counter add_score добавляет вес, add_finding создает code/family/severity/message, annotate_finding добавляет representative_rva и affected_count, когда возможно.

Зачем это нужно: Это момент, где structural facts становятся defense-ready evidence: код finding, семейство, severity, счет, место и сообщение.

Какой вывод делает проект: Если counter ноль - finding не создается. Если counter >0 - finding появляется в JSON/HTML и влияет на verdict.

Finding / счетчик / статус: pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob.

Решение программы: Программа продолжает к отсутствию .pdata и RF findings.

Как продолжается выполнение: Дальше создается no_runtime_functions или table findings.

Шаг 63. Finding no_runtime_functions

Режим выполнения: Если heuristics включены и rf_count == 0.

Где в коде: src/heuristics.c:833-842

Что проверяется: Проверяется, отсутствует ли Exception Directory при наличии executable section.

Как проверяется технически: Если n == 0, exception_rva == 0, exception_size == 0 и image_has_executable_section(img), добавляется score +10 и finding no_runtime_functions. Если executable sections нет, добавляется только note.

Зачем это нужно: Для обычного x64 кода отсутствие .pdata странно, но leaf-only или resource-only образы возможны. Поэтому finding medium и с ручным подтверждением.

Какой вывод делает проект: Успех/норма: resource-like image без executable section не подозрителен. Нарушение: executable code есть, runtime functions нет.

Finding / счетчик / статус: no_runtime_functions, table_integrity, medium, score +10.

Решение программы: Программа продолжает формировать отчет даже с rf_count 0.

Как продолжается выполнение: Дальше создаются findings по RF ranges.

Шаг 64. Создание findings по таблице функций

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:850-891

Что проверяется: Проверяются counters invalid_ranges, rf_oob, rf_unwind_oob, rf_unsorted, rf_overlaps, rf_duplicate_begins.

Как проверяется технически: Каждый counter порождает finding с severity и score. invalid/rf_oob/unwind_oob используют проценты для low/high веса, overlaps всегда high, duplicate/sort medium.

Зачем это нужно: Эти findings отвечают за карту функций: где начинаются/заканчиваются функции, не пересекаются ли они, ведут ли к существующей unwind metadata.

Какой вывод делает проект: Если counter ноль - нет finding. Если counter есть - report получает конкретный code, affected_count и representative location.

Finding / счетчик / статус: invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin.

Решение программы: Программа не отбрасывает плохие entries; она объясняет их и продолжает к UNWIND_INFO integrity.

Как продолжается выполнение: Дальше создаются findings unwind_parse_fail/unwind_anomalies.

Шаг 65. Создание findings по UNWIND_INFO integrity

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:893-906

Что проверяется: Проверяются counters unwind_parse_fail и unwind_anomalies.

Как проверяется технически: unwind_parse_fail получает score +10/+25 по доле >1%. unwind_anomalies получает +10/+20 по доле >5%.

Зачем это нужно: Эти findings показывают не просто плохую таблицу .pdata, а плохое содержимое .xdata: version/opcode/chain/handler/tail/operand problems.

Какой вывод делает проект: Если decoder стабильно читает все records без non-ABI anomalies, findings нет. Если ломается много записей, severity растет.

Finding / счетчик / статус: unwind_parse_fail, unwind_anomalies.

Решение программы: Программа сохраняет representative RVA, чтобы аналитик открыл конкретный UNWIND_INFO blob.

Как продолжается выполнение: Дальше создаются reuse/dominance findings.

Шаг 66. Strong UnwindInfo reuse

Режим выполнения: Если heuristics включены и n > 200.

Где в коде: src/heuristics.c:908-918; thresholds - src/heuristics.c:82-83

Что проверяется: Проверяется, не ссылается ли более 30% функций на один и тот же UnwindInfoRVA.

Как проверяется технически: Если unwind_reuse_ratio > 0.30 и n > 200, добавляется finding unwind_reuse_strong. dominant_unwind_rva становится representative location.

Зачем это нужно: Массовое reuse одной UNWIND_INFO записи может быть признаком централизованной подмены/обфускации. Ограничение n > 200 защищает от ложных процентов на маленьких файлах.

Какой вывод делает проект: Успех: reuse либо не массовый, либо sample маленький. Нарушение: один UnwindInfoRVA доминирует.

Finding / счетчик / статус: unwind_reuse_strong, unwind_reuse, high, score +20.

Решение программы: Программа не делает вывод 'вирус', а добавляет сильный статический сигнал.

Как продолжается выполнение: Дальше проверяется handler dominance.

Шаг 67. Handler dominance

Режим выполнения: Если heuristics включены и handler_count > 50.

Где в коде: src/heuristics.c:920-944; threshold - src/heuristics.c:84

Что проверяется: Проверяется, не ведет ли более 80% handler-ссылок к одному HandlerRVA.

Как проверяется технически: Код считает handler_reuse_ratio. Если ratio > 0.80 и handler_count > 50, дополнительно проверяется static context и dominant_runtime_like. Если доминирующий handler похож на нормальный runtime helper без других сигналов, finding может не добавляться.

Зачем это нужно: Это пример зрелой эвристики: доминирование handler само по себе может быть нормальным для runtime, поэтому проект смотрит контекст.

Какой вывод делает проект: Нарушение означает: многие функции централизуют exception handling в одной точке, особенно подозрительно вместе с table/unwind/placement signals.

Finding / счетчик / статус: handler_dominance, handler_dominance family, medium, score +15.

Решение программы: Программа добавляет finding только если benign runtime explanation не выглядит достаточной.

Как продолжается выполнение: Дальше проверяется moderate reuse с корреляцией.

Шаг 68. Moderate UnwindInfo reuse только при корреляции

Режим выполнения: Если heuristics включены и n > 200.

Где в коде: src/heuristics.c:946-962

Что проверяется: Проверяется reuse >10%, но только если strong reuse не сработал и есть другой статический сигнал.

Как проверяется технически: moderate_reuse_candidate true при ratio > 0.10 и n > 200. Finding добавляется только если уже есть handler_dominance или table/unwind/placement counters.

Зачем это нужно: Это защищает от ложных срабатываний: умеренное reuse может быть нормальным, если несколько функций имеют одинаковый пролог.

Какой вывод делает проект: Нарушение означает: reuse не одиночный факт, а коррелирует с другими проблемами.

Finding / счетчик / статус: unwind_reuse_moderate, unwind_reuse, medium, score +10.

Решение программы: Программа использует correlation rule, а не голый threshold.

Как продолжается выполнение: Дальше создаются placement findings.

Шаг 69. Placement findings по metadata и handler

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:964-1000

Что проверяется: Проверяются накопленные counters unwind_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section.

Как проверяется технически: Каждый counter добавляет score и finding. Для unusual/writable unwind вес зависит от распространенности: больше 5% записей - высокий вариант.

Зачем это нужно: Роль секции должна совпадать с правами: metadata не должна исполняться или быть writable без причины, handler должен быть executable.

Какой вывод делает проект: Нормальный результат - no placement findings. Нарушение - отчет показывает адрес, секцию и тип несоответствия.

Finding / счетчик / статус: unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section.

Решение программы: Программа не блокирует анализ, потому что placement - эвристический слой, требующий ручной проверки.

Как продолжается выполнение: Дальше создаются combo findings.

Шаг 70. Комбинационные findings

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:1003-1023

Что проверяется: Проверяется, усиливают ли друг друга разные группы сигналов.

Как проверяется технически: Если unwind metadata в executable section и handler в non-exec/unknown - placement_exec_nonexec_handler_combo. Если suspicious placement коррелирует с parse/anomaly/unwind_oob - placement_integrity_combo. Если reuse >10% и handler_dominance - reuse_dominance_combo. Если metadata writable и handler writable - writable_metadata_handler_combo.

Зачем это нужно: Комбинация важнее суммы отдельных facts. Один необычный placement может быть toolchain artifact, но placement + integrity + dominance уже сложнее объяснить случайностью.

Какой вывод делает проект: Если условий нет - combo не создается. Если есть - score получает дополнительный вес.

Finding / счетчик / статус: placement_exec_nonexec_handler_combo (+10 high), placement_integrity_combo (+8 medium), reuse_dominance_combo (+8 medium), writable_metadata_handler_combo (+10 high).

Решение программы: Программа поднимает доказательную силу только при корреляции разных evidence families.

Как продолжается выполнение: Дальше финализируется score/confidence/verdict.

Блок H. Статические эвристики: финал

Здесь проект превращает набор findings в score, confidence, mitigations и verdict.

Шаг 71. Финализация score, confidence и verdict

Режим выполнения: Если heuristics включены.

Где в коде: src/heuristics.c:1025-1032; finalize_verdict - src/heuristics.c:514-588; score constants - src/heuristics.c:31-64

Что проверяется: Проверяется итоговая сумма evidence и вычисляется подозрительность, confidence, evidence level и verdict.

Как проверяется технически: Если score == 0, добавляется note 'No strong anomalies'. finalize_verdict считает confidence от базы, количества evidence families, strong/medium findings и hard bonuses. Затем применяет penalties: single signal, small sample, placement-only, single-family. Verdict: score >=70 high, >=40 moderate, >=15 low, иначе no_strong_static_anomalies.

Зачем это нужно: Это не машинное обучение и не антивирусный verdict. Это прозрачная экспертная scoring model: каждая цифра видна в коде и связана с finding.

Какой вывод делает проект: Успех: hr содержит score, suspicion, confidence, verdict, mitigations, findings. Если findings нет - отчет прямо говорит об отсутствии сильных static anomalies.

Finding / счетчик / статус: Все findings влияют на score; mitigations влияют на confidence.

Решение программы: Программа печатает heuristics report и передает hr в JSON/HTML, если включены.

Как продолжается выполнение: Дальше опционально запускается dynamic collector.

Блок I. Динамический анализ

Динамика не заменяет статику. Она добавляет runtime evidence: что реально регистрируется и достигается при выполнении.

Шаг 72. Запуск процесса под Windows debugger

Режим выполнения: Опционально: только если включен dynamic; на Windows.

Где в коде: src/main.c:401-424; src/dynamic_win.c:657-715

Что проверяется: Проверяется, можно ли запустить анализируемый PE как child process под DEBUG_ONLY_THIS_PROCESS.

Как проверяется технически: dynamic_collect_windows строит command line, вызывает CreateProcessA с DEBUG_ONLY_THIS_PROCESS и инициализирует DynamicReport. При ошибке возвращает false.

Зачем это нужно: Static анализ говорит, что записано в файле. Dynamic показывает, что реально происходит при запуске: исключения, регистрации handler-ов, runtime function table calls.

Какой вывод делает проект: Успех: начинается debug loop. Провал: dynamic evidence отсутствует, но static report остается.

Finding / счетчик / статус: Динамика не создает 27 static findings. Она дает runtime evidence для assessment и correlation.

Решение программы: main при failed dynamic ставит exit_code = 1, но не уничтожает static результаты.

Как продолжается выполнение: Дальше debug loop собирает модули, breakpoints и events.

Шаг 73. Сбор handler RVAs из static UNWIND_INFO и установка breakpoints

Режим выполнения: Опционально dynamic.

Где в коде: src/dynamic_win.c:726-746; breakpoint helpers - src/dynamic_win.c:244-346; set on process/load - src/dynamic_win.c:789-817

Что проверяется: Проверяется, какие handlers уже обнаружены статическим decoder-ом, и можно ли поставить breakpoint на base_main + handler_rva.

Как проверяется технически: Collector проходит us/us_ok, собирает уникальные handler_rvas, после CREATE_PROCESS_DEBUG_EVENT получает base_main и пытается поставить breakpoints на unwind handlers и API breakpoints.

Зачем это нужно: Это связь статики и динамики: static адрес handler превращается в runtime VA через module base. Так проект проверяет, реально ли этот handler достигается.

Какой вывод делает проект: Успех: увеличиваются counters breakpoint_set_attempts/unwind_handler_bp_set. Провалы фиксируются в details, но не рушат весь session.

Finding / счетчик / статус: Runtime evidence: unwind_handler_hits, breakpoint failures, details. Не static finding.

Решение программы: Программа best-effort ставит точки и продолжает debug loop.

Как продолжается выполнение: Дальше обрабатываются DEBUG_EVENT-ы.

Шаг 74. Debug event loop, timeout и exception events

Режим выполнения: Опционально dynamic.

Где в коде: src/dynamic_win.c:768-850

Что проверяется: Проверяется, приходят ли события от отлаживаемого процесса и не превышен ли timeout.

Как проверяется технически: Loop вызывает WaitForDebugEvent с коротким ожиданием. При timeout добавляется dynamic event 'timeout'. LOAD_DLL/UNLOAD_DLL обновляют module list. EXCEPTION_DEBUG_EVENT увеличивает exception counters и запускает обработку breakpoints/exception flow.

Зачем это нужно: Динамический анализ опасен зависаниями. Timeout делает session ограниченной и воспроизводимой.

Какой вывод делает проект: Успех: events собираются до завершения/timeout. Ошибка WaitForDebugEvent фиксируется как dynamic failure.

Finding / счетчик / статус: Runtime evidence fields, не static findings.

Решение программы: Программа продолжает или завершает dynamic session в зависимости от события/timeout.

Как продолжается выполнение: Дальше при hit breakpoints снимаются регистры и адреса handler-ов.

Шаг 75. Захват регистраций VEH/VCH/UEF и hits handler-ов

Режим выполнения: Опционально dynamic.

Где в коде: src/dynamic_win.c:852-1108

Что проверяется: Проверяется, вызывает ли процесс AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable/RtlInstallFunctionTableCallback и попадает ли execution в unwind/VEH/VCH/UEF handlers.

Как проверяется технически: При breakpoint hit collector читает CONTEXT. На Windows x64 аргументы API берутся из RCX/RDX/R8/R9. Например handler для AddVectoredExceptionHandler берется из RDX, для SetUnhandledExceptionFilter - из RCX. Затем ставятся breakpoints на зарегистрированные handlers и добавляются DynamicEvent records.

Зачем это нужно: Это runtime evidence, которое static parser не может получить из файла: динамическая регистрация обработчиков и runtime function tables может появиться только при выполнении.

Какой вывод делает проект: Успех: report содержит registrations/hits/calls. Отсутствие hits не отменяет static findings, но снижает runtime evidence.

Finding / счетчик / статус: Runtime signals: veh_registrations, vch_registrations, uef_set_calls, unwind_handler_hits, veh_hits, vch_hits, uef_hits, rtl_add_function_table_calls.

Решение программы: Программа re-arm-ит breakpoints через single-step и продолжает до завершения/timeout.

Как продолжается выполнение: Дальше report.c коррелирует static и dynamic evidence.

Блок J. Корреляция и отчет

Этот блок публикует evidence и делает итоговую человеко- и машинно-читаемую интерпретацию.

Шаг 76. Assessment: объединение static и dynamic evidence

Режим выполнения: При JSON/HTML report, если есть hr/dr.

Где в коде: src/report.c:326-440; manual validation - src/report.c:465-515

Что проверяется: Проверяется, есть ли runtime signals и коррелируют ли они со статическими findings.

Как проверяется технически: assessment_summary считает dynamic_signal_count, has_static_dynamic_correlation, runtime_evidence_level, strength. Если dynamic session complete и есть signals, confidence повышается. Если static+dynamic коррелируют, confidence повышается еще. Verdict получает суффиксы correlated/static/runtime_signals_only.

Зачем это нужно: Это финальный смысловой слой: static findings показывают подозрительную структуру, dynamic events показывают поведение. Вместе они сильнее, чем по отдельности.

Какой вывод делает проект: Успех: отчет объясняет final_verdict/final_confidence и manual steps. Если динамики нет, assessment не притворяется, что она была.

Finding / счетчик / статус: Не новый finding, а финальная интерпретация evidence.

Решение программы: Программа не меняет исходные findings, а формирует итоговую оценку над ними.

Как продолжается выполнение: Дальше данные записываются в CSV/JSON/HTML.

Шаг 77. CSV и Ghidra CSV output

Режим выполнения: Опционально по аргументам.

Где в коде: src/main.c:427-448; report_write_csv - src/report.c:124 и далее

Что проверяется: Проверяется, можно ли записать табличный отчет по функциям и unwind summary.

Как проверяется технически: main вызывает report_write_csv обычный или ghidra mode. При ошибке записи печатает сообщение и ставит exit_code = 1.

Зачем это нужно: CSV нужен для внешней проверки и импорта в Ghidra. Это не новый анализ, а перенос уже вычисленных RF/UNWIND facts в удобный формат.

Какой вывод делает проект: Успех: файл создан. Провал: analysis уже сделан, но output не записан.

Finding / счетчик / статус: Нет finding. Это ошибка вывода.

Решение программы: Программа продолжает к JSON/HTML, если они включены.

Как продолжается выполнение: Дальше собирается metadata для JSON/HTML.

Шаг 78. Metadata: timestamp, SHA256, invocation, build info

Режим выполнения: Если включен JSON или HTML.

Где в коде: src/main.c:450-481

Что проверяется: Проверяется, можно ли подготовить метаданные отчета.

Как проверяется технически: main строит UTF-8 input_display, command_line, UTC timestamp, SHA256 входного файла, version/schema/compiler/platform/build_mode и параметры strict_abi/follow_chain/max_chain_depth.

Зачем это нужно: Метаданные делают отчет воспроизводимым: можно понять, какой файл, какой командой и какой сборкой анализировали.

Какой вывод делает проект: Успех: metadata_ptr передается в report_write_json_ex/report_write_html. Провал: output может быть пропущен, exit_code = 1.

Finding / счетчик / статус: Нет finding. Это provenance layer.

Решение программы: Программа не искажает findings при ошибке metadata, но не пишет неполный JSON/HTML как будто все хорошо.

Как продолжается выполнение: Дальше пишутся JSON и HTML.

Шаг 79. JSON report

Режим выполнения: Опционально.

Где в коде: src/main.c:484-497; report_write_json_ex - src/report.c:622-920

Что проверяется: Проверяется, можно ли сериализовать PE validation, functions, heuristic analysis, findings, dynamic report и assessment в JSON.

Как проверяется технически: report_write_json_ex пишет структурированный report: PE поля и validation counters, runtime functions/unwind summaries, heuristic score/confidence/verdict/findings, dynamic counters/events, final assessment.

Зачем это нужно: JSON - машинно-читаемая доказательная база. Комиссии можно показать, что каждый finding не просто напечатан строкой, а имеет code, family, severity, affected_count и location.

Какой вывод делает проект: Успех: JSON создан. Провал: ошибка output, exit_code = 1.

Finding / счетчик / статус: Findings уже созданы в heuristics; JSON их только публикует.

Решение программы: Программа продолжает к HTML, если включен.

Как продолжается выполнение: Дальше пишется HTML report.

Шаг 80. HTML report и русское объяснение findings

Режим выполнения: Опционально.

Где в коде: src/main.c:498-510; report_write_html - src/report.c:1151-1520; finding messages - src/report.c:1072-1131

Что проверяется: Проверяется, можно ли создать человеко-читаемый HTML отчет.

Как проверяется технически: report_write_html оформляет summary, PE validation, heuristic findings, dynamic evidence и assessment. Для findings используются русские пояснения, severity, family, affected_count, representative RVA и section.

Зачем это нужно: HTML нужен не машине, а человеку: reverse engineer или комиссия должны увидеть не только score, но и почему он возник.

Какой вывод делает проект: Успех: HTML создан. Провал: ошибка output.

Finding / счетчик / статус: Findings не создаются здесь, но становятся понятными пользователю.

Решение программы: main выставляет exit_code = 1 при ошибке записи, но cleanup все равно выполняется.

Как продолжается выполнение: Дальше освобождаются ресурсы и возвращается итоговый код.

Блок K. Завершение

Последний блок корректно освобождает память и возвращает итоговый exit code.

Шаг 81. Освобождение ресурсов и итоговый exit code

Режим выполнения: Всегда.

Где в коде: src/main.c:512-526; pe_free - src/pe.c:259-263

Что проверяется: Проверяется, что все выделенные буферы и структуры освобождены независимо от того, были ли warnings или частичный report.

Как проверяется технически: main освобождает invocation, input_display, heuristics details, dynamic report, ops, us_ok, us, rf, img sections, err buffers и owned file buffer. Возвращает exit_code, который мог стать 1 при malformed directory, failed heuristics/dynamic/output.

Зачем это нужно: Это завершает жизненный цикл инструмента. Важно, что программа различает 'analysis completed with findings' и 'tool/output/dynamic problem'.

Какой вывод делает проект: Успех без runtime/output проблем возвращает 0, даже если findings есть. Некоторые structural malformed состояния могут вернуть 1, но report все равно может быть частично создан.

Finding / счетчик / статус: Нет new finding.

Решение программы: Программа завершает процесс.

Как продолжается выполнение: Сквозной pipeline закончен: от bytes файла до evidence report.

Индекс 27 findings по шагам

Ниже - контрольный индекс: каждый finding из набора 27 привязан к шагам сквозного pipeline. Это помогает быстро ответить комиссии, где именно finding рождается, из какой проверки он вытекает и какое решение принимает программа.

1. pe_section_raw_oob

Шаги pipeline: 18, 62

Код: src/pe.c:198-204; src/heuristics.c:789-795

Что значит: Секция обещает raw bytes за пределами файла.

Вес / severity: +25 high

Решение программы: Сохранить counter, добавить finding, продолжить анализ.

2. pe_section_virtual_oob

Шаги pipeline: 19, 62

Код: src/pe.c:200-204; src/heuristics.c:796-802

Что значит: Секция выходит за SizeOfImage.

Вес / severity: +20 high

Решение программы: Добавить finding; считать виртуальную карту подозрительной.

3. pe_section_raw_overlap

Шаги pipeline: 21, 62

Код: src/pe.c:215-219; src/heuristics.c:803-809

Что значит: Raw ranges секций пересекаются.

Вес / severity: +15 medium

Решение программы: Добавить finding; отметить неоднозначность file layout.

4. pe_section_virtual_overlap

Шаги pipeline: 22, 62

Код: src/pe.c:220-226; src/heuristics.c:810-816

Что значит: Virtual ranges секций пересекаются.

Вес / severity: +20 high

Решение программы: Добавить finding; отметить неоднозначность RVA mapping.

5. pe_section_misaligned

Шаги pipeline: 17, 20, 62

Код: src/pe.c:180-196,205-213; src/heuristics.c:817-823

Что значит: Нарушены alignment/header/ordering инварианты.

Вес / severity: +8 medium

Решение программы: Добавить мягкий structural signal.

6. pe_exception_directory_oob

Шаги pipeline: 23, 28-32, 62

Код: src/pe.c:230-235; src/unwind.c:626-645; src/main.c:321; src/heuristics.c:825-832

Что значит: Exception Directory malformed/unreadable/out of bounds.

Вес / severity: +25 high

Решение программы: Продолжить partial report, если возможно; exit_code может стать 1.

7. no_runtime_functions

Шаги pipeline: 27, 63

Код: src/unwind.c:623-625; src/heuristics.c:833-842

Что значит: Нет RUNTIME_FUNCTION при наличии executable code.

Вес / severity: +10 medium

Решение программы: Добавить finding с ручной проверкой; для resource-only - только note.

8. invalid_ranges

Шаги pipeline: 52, 64

Код: src/heuristics.c:651-654,850-856

Что значит: EndAddress <= BeginAddress.

Вес / severity: +10/+20

Решение программы: Считать долю; добавить medium/high finding.

9. rf_oob

Шаги pipeline: 53, 64

Код: src/heuristics.c:655-658,857-863

Что значит: Begin/End function range выходит за SizeOfImage.

Вес / severity: +10/+25

Решение программы: Считать долю; добавить table integrity finding.

10. unwind_rva_oob

Шаги pipeline: 54, 64

Код: src/heuristics.c:659-662,864-870

Что значит: UnwindInfoRVA нулевой или вне image.

Вес / severity: +10/+25

Решение программы: Добавить finding; связать с невозможной ссылкой на .xdata.

11. rf_unsorted

Шаги pipeline: 55, 64

Код: src/heuristics.c:664-669,871-877

Что значит: Таблица не отсортирована по BeginAddress.

Вес / severity: +15 medium

Решение программы: Добавить finding; дальше использовать sorted copy для overlaps.

12. rf_overlaps

Шаги pipeline: 60, 64

Код: src/heuristics.c:740-773,878-884

Что значит: Function ranges перекрываются.

Вес / severity: +25 high

Решение программы: Добавить high finding, потому что карта функций неоднозначна.

13. duplicate_begin

Шаги pipeline: 60, 64

Код: src/heuristics.c:759-762,885-891

Что значит: Несколько функций имеют одинаковый BeginAddress.

Вес / severity: +10 medium

Решение программы: Добавить finding; проверить вручную, thunk это или конфликт.

14. unwind_parse_fail

Шаги pipeline: 36-49, 59, 65

Код: src/unwind.c:165-608; src/heuristics.c:727-729,893-899

Что значит: UNWIND_INFO не удалось безопасно разобрать.

Вес / severity: +10/+25

Решение программы: Считать долю; не останавливать весь анализ.

15. unwind_anomalies

Шаги pipeline: 37, 40-49, 59, 65

Код: src/unwind.c:187-604; src/heuristics.c:730-733,900-906

Что значит: UNWIND_INFO разобран, но содержит странные version/opcode/chain/handler/operand признаки.

Вес / severity: +10/+20

Решение программы: Считать non-ABI anomalies; ABI warnings отделить.

16. unwind_reuse_strong

Шаги pipeline: 56, 66

Код: src/heuristics.c:775-780,908-918

Что значит: Более 30% функций разделяют один UnwindInfoRVA при n > 200.

Вес / severity: +20 high

Решение программы: Добавить strong reuse finding.

17. handler_dominance

Шаги pipeline: 56, 67

Код: src/heuristics.c:782-785,920-944

Что значит: Более 80% handlers ведут в одну точку при handler_count > 50, если benign context не объясняет это.

Вес / severity: +15 medium

Решение программы: Добавить finding только после context check.

18. unwind_reuse_moderate

Шаги pipeline: 56, 68

Код: src/heuristics.c:946-962

Что значит: Более 10% reuse при n > 200 и наличии другого статического сигнала.

Вес / severity: +10 medium

Решение программы: Добавить только при корреляции, не по одному threshold.

19. unwind_in_exec_section

Шаги pipeline: 57, 69

Код: src/heuristics.c:704-706,965-970

Что значит: UNWIND_INFO лежит в executable section.

Вес / severity: +20 high

Решение программы: Добавить placement finding.

20. unwind_unusual_section

Шаги pipeline: 57, 69

Код: src/heuristics.c:693-703,972-978

Что значит: UNWIND_INFO не в ожидаемой .xdata/.rdata/.pdata-like области.

Вес / severity: +8/+15

Решение программы: Вес зависит от массовости >5%.

21. unwind_writable_section

Шаги pipeline: 57, 69

Код: src/heuristics.c:708-710,980-987

Что значит: UNWIND_INFO лежит в writable section.

Вес / severity: +10/+18

Решение программы: Добавить finding; проверить JIT/runtime объяснение.

22. handler_nonexec_section

Шаги pipeline: 58, 69

Код: src/heuristics.c:714-719,988-993

Что значит: Exception handler указывает в non-executable или unknown section.

Вес / severity: +15 high

Решение программы: Добавить finding; handler должен быть кодом.

23. handler_writable_section

Шаги pipeline: 58, 69

Код: src/heuristics.c:720-723,995-1000

Что значит: Handler находится в writable section.

Вес / severity: +10 medium

Решение программы: Добавить finding; проверить возможность runtime подмены.

24. placement_exec_nonexec_handler_combo

Шаги pipeline: 70

Код: src/heuristics.c:1003-1007

Что значит: UNWIND_INFO executable + handler nonexec/unknown.

Вес / severity: +10 high

Решение программы: Усилить вывод, потому что роли metadata/handler противоречат секциям одновременно.

25. placement_integrity_combo

Шаги pipeline: 70

Код: src/heuristics.c:1008-1012

Что значит: Suspicious placement подтверждается parse/anomaly/unwind_oob.

Вес / severity: +8 medium

Решение программы: Усилить вывод по корреляции placement и integrity.

26. reuse_dominance_combo

Шаги pipeline: 70

Код: src/heuristics.c:1014-1017

Что значит: Reuse >10% совпадает с handler dominance.

Вес / severity: +8 medium

Решение программы: Отметить centralized dispatcher-like pattern.

27. writable_metadata_handler_combo

Шаги pipeline: 70

Код: src/heuristics.c:1019-1022

Что значит: Writable metadata совпадает с writable handler.

Вес / severity: +10 high

Решение программы: Усилить вывод по writable control data/code target.

Системный ответ на вопрос: что делает проект проверочно

Проект делает не одну проверку и не набор случайных условий. Он последовательно строит доказательную цепочку.

1. Сначала он доказывает, что файл вообще можно читать как x64 PE32+. Без MZ, PE signature, AMD64 Machine, PE32+ Magic, SizeOfImage и DataDirectory дальнейший unwind-анализ был бы применением неправильной модели.

2. Затем он строит карту image: headers, sections, raw layout, virtual layout, права секций. Эта карта нужна для любого RVA: понять, лежит ли он в image, в какой секции, можно ли прочитать bytes из файла, какие memory permissions имеет область.

3. Затем он читает Exception Directory как источник RUNTIME_FUNCTION table. Он не путает RVA, VA и file offset: directory entry дает RVA, section table переводит RVA в file offset, ImageBase нужен только для runtime VA.

4. Затем он проверяет каждую RUNTIME_FUNCTION entry как карту функции. Begin/End должны образовывать нормальный диапазон внутри SizeOfImage, таблица должна быть отсортирована, ranges не должны конфликтовать, UnwindInfoRVA должен вести к доступной metadata.

5. Затем он декодирует UNWIND_INFO как инструкцию по выходу из кадра. Version, flags, CountOfCodes, opcodes, operands, chain tail и handler tail должны быть структурно читаемыми и логически согласованными.

6. Затем он превращает измеренные факты в findings. Parser и decoder собирают counters/anomalies, a heuristics слой назначает code, family, severity, affected_count, score и confidence.

7. Затем при включенной динамике он проверяет runtime-поведение. Debug collector ищет exception events, регистрации VEH/VCH/UEF, runtime function table API calls и реальные hits handler-ов.

8. В конце он формирует отчет. CSV дает табличный экспорт, JSON дает машинно-читаемую evidence базу, HTML дает человеко-читаемое объяснение findings и assessment.

Именно поэтому сложность проекта не в количестве строк if, а в сохранении корректной координатной системы PE/unwind: file offset, RVA, VA, section ranges, SizeOfImage, SizeOfHeaders, raw/virtual layout, x64 ABI, UNWIND_CODE variable-length encoding, handler/chained tails, scoring, confidence и static-dynamic correlation должны согласованно работать в одном pipeline.

ФИНАЛЬНЫЙ АУДИТ ПОЛНОТЫ И СНЯТИЕ СЕРЫХ ЗОН

Этот раздел добавлен после страницы 219 как финальная редакционная проверка документа. Его задача - не повторить уже описанные 27 findings, а зафиксировать, что именно проект проверяет прямо, что ловит косвенно, а что честно находится вне области текущей версии.

По технической сверке документа с исходниками в PDF присутствуют все 27 finding codes, основные модули src/main.c, src/pe.c, src/unwind.c, src/heuristics.c, src/dynamic_win.c, src/report.c, src/util.c, ключевые функции pe_parse, pe_rva_to_fileoff, unwind_read_runtime_functions_ex, unwind_parse, heuristics_analyze, dynamic_collect_windows, report_write_json_ex/report_write_html/report_write_csv, а также основные понятия PE32+, AMD64, SizeOfImage, SizeOfHeaders, RUNTIME_FUNCTION, UNWIND_INFO, Exception Directory, score, confidence, verdict, JSON/HTML/CSV/Ghidra.

Но при защите опасны не только пропуски, а и преувеличения. Поэтому ниже явно отделены прямые проверки от косвенных и от того, что проект не заявляет как реализованную функциональность.

1. Карта прямых, косвенных и внеобластных проверок

Exception Directory size не кратен 12

Статус: Проверяется прямо.

Где в коде: src/unwind.c:643-645; main.c:308-323; heuristics.c:825-832

Как работает: Reader не может разбить область на целые RUNTIME_FUNCTION entries и возвращает RUNTIME_FUNCTION_READ_MALFORMED. main помечает img.exception_directory_oob, продолжает частичный отчет и ставит exit_code = 1.

Как защищать формулировку: Это не отдельный код finding, а часть pe_exception_directory_oob. Так сделано осознанно: все варианты структурно нечитаемого Exception Directory сведены в один совместимый finding.

EndAddress <= BeginAddress

Статус: Проверяется прямо.

Где в коде: src/heuristics.c:651-654; emission: 850-856

Как работает: Каждая RUNTIME_FUNCTION entry сравнивается как полуоткрытый диапазон [BeginAddress, EndAddress). Если конец не больше начала, counter rf_invalid_ranges увеличивается.

Как защищать формулировку: Finding invalid_ranges. Это ошибка карты функции, а не ошибка чтения файла.

RUNTIME_FUNCTION table не отсортирована или пересекается

Статус: Проверяется прямо.

Где в коде: unsorted: src/heuristics.c:664-669; overlaps/duplicates: 740-773; emission: 871-891

Как работает: Сначала проверяется исходный порядок BeginAddress. Затем создается отсортированная копия, где проверяются duplicate begins и пересечения через max_end.

Как защищать формулировку: Findings rf_unsorted, rf_overlaps, duplicate_begin. Документ теперь явно разделяет порядок таблицы и математический конфликт диапазонов.

UnwindInfoAddress за пределами image или равен нулю

Статус: Проверяется прямо.

Где в коде: src/heuristics.c:659-662; emission: 864-870

Как работает: Если UnwindInfoRVA == 0 или >= SizeOfImage, counter rf_unwind_oob увеличивается.

Как защищать формулировку: Finding unwind_rva_oob. Это table-level дефект ссылки из .pdata в .xdata.

UnwindInfoAddress указывает в область, которую нельзя прочитать из файла

Статус: Проверяется прямо на уровне чтения, но вывод идет через parse failure.

Где в коде: src/pe.c:277-300; src/unwind.c:56-72,187-195; heuristics.c:727-729,893-899

Как работает: pe_rva_to_fileoff запрещает чтение virtual zero-fill хвоста и несуществующих raw bytes. Если header UNWIND_INFO не читается, unwind_parse возвращает false.

Как защищать формулировку: Finding unwind_parse_fail. Это честнее, чем придумывать отдельный адресный finding для каждого варианта нечитаемости.

UnwindInfoAddress указывает в середину другой структуры

Статус: Не доказывается как отдельный факт, но ловится косвенно.

Где в коде: src/unwind.c:197-551; heuristics.c:893-906

Как работает: Статически без полной карты всех возможных language/runtime structures нельзя надежно доказать, что адрес именно в середине другой структуры. Проект проверяет, читается ли по этому адресу корректный UNWIND_INFO: version, flags, CountOfCodes, opcodes, operands, chain, handler tail.

Как защищать формулировку: Если bytes не образуют корректную структуру - unwind_parse_fail или unwind_anomalies. Если случайно образуют синтаксически валидный UNWIND_INFO, отдельного finding 'middle_of_other_structure' нет.

UNWIND_INFO лежит в executable/writable/unusual section

Статус: Проверяется прямо по section table.

Где в коде: src/heuristics.c:690-712; emission: 964-987

Как работает: Для каждого успешно разобранного UNWIND_INFO ищется секция. Проверяется имя секции и bits IMAGE_SCN_MEM_EXECUTE / IMAGE_SCN_MEM_WRITE.

Как защищать формулировку: Findings unwind_in_exec_section, unwind_unusual_section, unwind_writable_section. Это static section characteristics, а не runtime VirtualProtect snapshot.

Exception handler указывает в non-executable или writable section

Статус: Проверяется прямо.

Где в коде: handler read: src/unwind.c:540-551; placement: src/heuristics.c:714-723; emission: 988-1000

Как работает: Decoder читает HandlerRVA из tail UNWIND_INFO, затем heuristics ищет секцию handler-а и проверяет executable/write bits.

Как защищать формулировку: Findings handler_nonexec_section и handler_writable_section.

Language-specific данные после handler RVA

Статус: Полностью не парсятся как отдельный формат.

Где в коде: src/unwind.c:540-551; report.c:465-515,1064

Как работает: Проект фиксирует сам факт EHANDLER/UHANDLER, читает HandlerRVA и проверяет его границы/placement. Но он не разбирает C++ scope table, SEH scope table или compiler-specific LSDA как отдельную структуру.

Как защищать формулировку: Это ограничение надо защищать честно: цель BKP - PE/x64 unwind/EH metadata integrity и handler target evidence, а не полноценный декодер всех language-specific exception formats.

Полный RtlVirtualUnwind / stack walk

Статус: Не реализован как отдельный runtime stack walking engine.

Где в коде: dynamic loop: src/dynamic_win.c:657-1108; static decode: src/unwind.c:165-608

Как работает: Проект декодирует UNWIND_INFO и проверяет, можно ли доверять данным, из которых Windows делала бы virtual unwind. Динамика собирает debug events, handler hits и регистрации обработчиков. Но вызова RtlVirtualUnwind и построения полного stack trace в коде нет.

Как защищать формулировку: Это не надо обещать на защите. Правильная формулировка: проект проверяет корректность и подозрительность unwind metadata, а не заменяет WinDbg stack walker.

Сравнение unwind codes с реальными байтами пролога

Статус: Не реализовано как disassembler-level проверка.

Где в коде: unwind code semantics: src/unwind.c:251-510

Как работает: Проект проверяет структуру UNWIND_INFO, operands, порядок CodeOffset и ABI-warning режим. Но он не дизассемблирует .text и не доказывает, что каждая UWOP операция совпадает с конкретной машинной инструкцией пролога.

Как защищать формулировку: Это честное ограничение: для такой проверки нужен disassembler/Capstone/LLVM MC или собственный x86-64 decoder, что выходит за текущую архитектуру.

PE32/x86

Статус: Сознательно вне области проекта.

Где в коде: src/pe.c:67-90; include/pe.h:18-19

Как работает: Machine должен быть AMD64 0x8664, OptionalHeader.Magic должен быть PE32+ 0x20b. PE32 0x10b не анализируется.

Как защищать формулировку: Это не недостаток, а граница предметной области: x86 exception handling устроен иначе, а проект посвящен x64 .pdata/.xdata unwind model.

Все PE Data Directories, imports, relocations, resources

Статус: Не являются целью полной валидации.

Где в коде: Exception Directory: src/pe.c:121-137; sections: src/pe.c:139-256

Как работает: Проект глубоко проверяет то, что нужно для EH/unwind: headers, section table, SizeOfImage/SizeOfHeaders, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handler placement. Он не является универсальным PE-linter для всех каталогов.

Как защищать формулировку: На защите формулировать так: это специализированный анализатор unwind/EH области PE32+, а не замена PE-bear/Detect It Easy/полного AV движка.

Runtime page protections после загрузчика

Статус: Частично проверяется статически, но не как полный runtime memory map.

Где в коде: static placement: src/heuristics.c:690-723; report permissions: src/report.c:694-696,1313-1315; dynamic: src/dynamic_win.c:657-1108

Как работает: Статика читает Characteristics секций и выводит R/W/X. Динамика ставит breakpoints и собирает события, но не строит полный снимок VirtualQueryEx по всем страницам.

Как защищать формулировку: Правильная формулировка: проект проверяет заявленные section permissions и runtime evidence handlers/API, но не является полноценным монитором memory protection changes.

2. Что теперь считать окончательной границей проекта

Ядро проекта

Итоговая формулировка: Статический анализ PE32+ x64 EH/unwind области: DOS/NT headers, Optional Header, section table, Exception Directory, RUNTIME_FUNCTION table, UNWIND_INFO, chained unwind, handler RVA, placement и scoring.

Сильная сторона

Итоговая формулировка: Проект не просто читает поля, а сохраняет координатную систему: RVA != VA != file offset, SizeOfImage != размер файла, VirtualSize != SizeOfRawData, headers тоже могут быть RVA-областью, zero-fill bytes нельзя читать из файла.

Эвристическая часть

Итоговая формулировка: Findings не являются 'истиной антивируса'. Это прозрачные evidence units: code, family, severity, affected_count, representative location, score, confidence и mitigation rules.

Динамическая часть

Итоговая формулировка: Dynamic collector подтверждает runtime events: debugging session, API breakpoints, handler registrations, handler hits, exception events и correlations. Он не является полноценным трассировщиком всех инструкций процесса.

Не область проекта

Итоговая формулировка: Полный PE-linter всех directories, disassembler-level проверка машинного пролога, C++/SEH scope table decoder, RtlVirtualUnwind stack walker, memory forensics engine и антивирусный malware classifier.

3. Безопасные ответы комиссии

Если спросят 'проект делает stack walking?'

Ответ: Нет, в текущей версии он не строит полный stack trace через RtlVirtualUnwind. Он проверяет статические unwind metadata и собирает runtime evidence по handler/API events. Это осознанная граница: проект отвечает на вопрос 'можно ли доверять EH/unwind структурам и где они подозрительны', а не заменяет WinDbg.

Если спросят 'проект разбирает C++ exception scope tables?'

Ответ: Нет, он не является C++ ABI/SEH scope table decoder. Он читает handler RVA из UNWIND_INFO, проверяет его границы, секцию, права и runtime hits. Language-specific данные после handler RVA оставлены для ручной проверки и будущего расширения.

Если спросят 'почему нет отдельного finding для Size % 12?'

Ответ: Потому что это один из вариантов structurally unreadable Exception Directory. Он объединен в pe_exception_directory_oob, чтобы не плодить мелкие коды для одного класса отказа чтения.

Если спросят 'почему не проверяются все PE directories?'

Ответ: Потому что предмет ВКР - x64 EH/unwind область: Exception Directory, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO и handler placement. Imports/resources/relocations относятся к другому классу PE-audit.

Если спросят 'а права памяти реальные или из файла?'

Ответ: В статике это Characteristics секций из PE. Это заявленные loader permissions, а не снимок всех runtime страниц. Runtime часть добавляет события и hits, но полный VirtualQueryEx memory map не строит.

Если спросят 'это антивирусный verdict?'

Ответ: Нет. Это прозрачная экспертная evidence/scoring модель для reverse engineer / malware analyst. Она показывает причины, веса, families, confidence и manual validation steps, а не прячет решение в черном ящике.

4. Финальная оценка понятности документа

После добавления этого раздела документ можно считать подготовленным к защите по трем уровням.

Уровень 1 - для непогруженного человека

Что понятно: Документ объясняет, что проект берет PE-файл, строит карту его x64 unwind/EH структур, проверяет согласованность .pdata/.xdata/handler layer и выдает читаемый отчет.

Почему это важно: Даже если человек не знает PE, он видит цепочку: файл -> заголовки -> секции -> Exception Directory -> RUNTIME_FUNCTION -> UNWIND_INFO -> findings -> score/verdict/report.

Уровень 2 - для комиссии

Что понятно: По каждому finding есть объяснение: почему такая проверка нужна, где она в коде, какой результат считается нормой, что значит нарушение и как программа продолжает выполнение.

Почему это защищает: Комиссия может спросить про 'несколько if else', а ответ будет про целостную координатную модель PE/unwind, безопасные границы чтения и доказательную матрицу.

Уровень 3 - для технического специалиста

Что понятно: Указаны ограничения и точные границы: нет PE32, нет full RtlVirtualUnwind stack walker, нет глубокого C++ scope table parse, нет disassembler-level prolog matching. Зато есть безопасный static parser, decoder, placement analysis, scoring и dynamic evidence collector.

Почему это честно: Документ не обещает невозможное. Он показывает, что проект специализированный и законченный в своей области, а не универсальный анализатор всего PE и всего runtime.

Финальная редакционная рекомендация для защиты: говорить не 'проект полностью анализирует все исключения Windows', а 'проект анализирует и валидирует x64 PE EH/unwind metadata, строит evidence по .pdata/.xdata/handler placement, выявляет 27 классов структурных и эвристических отклонений и формирует прозрачный отчет для reverse engineer / malware analyst'. Эта формулировка точная, сильная и совпадает с кодом.

Дополнение к глубокому описанию проекта

Три режима запуска и практическое использование EH/Unwind Analyzer

Этот раздел объясняет, как пользоваться проектом на практике: как запускать только статическую проверку, только динамическую проверку и смешанную проверку одного и того же бинарника. Важно понимать не только команды, но и смысл режимов: инструмент всегда начинает с чтения PE-файла и построения координатной карты, а уже затем, в зависимости от флагов, включает статические эвристики, динамический сбор runtime evidence или оба слоя одновременно.

> Ключевая мысль: проект не является тремя разными программами. Это один анализатор с переключаемыми слоями проверки. Базовый слой PE/unwind-разбора нужен всегда, потому что без него невозможно понять, где лежат .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO и handler RVA. Флаг --heuristics добавляет статическую оценку признаков. Флаг --dynamic запускает проверяемый EXE под Windows debugger и собирает события выполнения. Если указаны оба флага, отчет связывает статические и динамические наблюдения.

1. Что выполняется всегда, даже без специальных режимов

На уровне исходного кода это видно в src/main.c. Сначала разбираются аргументы командной строки (строки 180-236), затем файл читается в память (строки 283-288), после этого вызывается PE parser (строки 293-299), читается Exception Directory и массив RUNTIME_FUNCTION (строки 306-324), а затем для каждой записи разбирается UNWIND_INFO (строки 351-356). Эти действия являются общей основой всех режимов.

- Программа получает путь к PE-файлу. Это первый аргумент после имени analyzer.
- Файл читается как набор байтов. На этом этапе программа еще не делает выводов о подозрительности.
- PE parser проверяет DOS header, NT headers, Optional Header, секции, SizeOfImage, SizeOfHeaders, Data Directory и базовые границы.
- Из DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION] берутся RVA и Size Exception Directory.
- RVA каталога переводится в file offset через таблицу секций. Если это невозможно, фиксируется структурная проблема.
- Массив RUNTIME_FUNCTION читается как набор записей по 12 байт. Каждая запись связывает диапазон функции и RVA unwind metadata.
- Для каждой записи программа пытается разобрать UNWIND_INFO: версию, флаги, размер пролога, unwind-коды, handler или chained info.

Поэтому даже при динамическом запуске в отчете могут быть видны статические служебные данные: количество RUNTIME_FUNCTION, ошибки разбора UNWIND_INFO, секции, ImageBase, SizeOfImage и прочее. Но это еще не значит, что была включена статическая оценка. Оценка по признакам включается отдельно флагом --heuristics.

2. Общая схема режимов

Режим	Комбинация флагов	Что делает	Когда использовать
Статический	--heuristics	Разбирает PE/unwind-структуры, применяет правила и 27 признаков, считает score, confidence и статический verdict. EXE не запускается.	Для безопасной первичной проверки, для подозрительных файлов, для демонстрации структурных ошибок .pdata/.xdata/handler placement.
Динамический	--dynamic	Запускает target под Windows Debug API, ставит breakpoints на интересующие API/handler, собирает события выполнения. Статическая шкала признаков не считается.	Для учебных или доверенных тестовых бинарников, когда нужно показать runtime-поведение: VEH/VCH/UEF, handler hits, RtlAddFunctionTable.
Смешанный	--heuristics --dynamic	В одном отчете объединяет статические findings и динамические runtime-события. Это самый полный режим.	Для финальной демонстрации ВКР: видно и что подозрительно в структуре файла, и что реально проявилось при запуске.

В HTML-отчете верхняя панель специально сделана одинаковой для всех трех режимов. Если слой не запускался, он не исчезает, а явно помечается как не запускалась или не запрашивалась. Это важно для защиты: комиссия видит не пустоту, а осознанное состояние анализа.

3. Подготовка к запуску в PowerShell

Команды ниже рассчитаны на Windows PowerShell или Developer PowerShell. Они специально написаны отдельными строками, без склеивания через сложные переносы. Так меньше шансов получить ошибку разбора PowerShell. Первым делом нужно перейти в корень проекта:

> Важно: если в PDF длинная команда визуально перенеслась на следующую строку из-за ширины страницы, это обычный перенос отображения. В терминале лучше вводить каждую команду как одну логическую строку. Именно поэтому ниже используются переменные $tool и $sample: они сокращают команды и делают копирование надежнее.

```
cd "C:\Users\Templar\Desktop\вкр\ВКР ПРОЕКТ ОКОНЧЕННЫЙ\ip_full"
```

После перехода удобно один раз задать две переменные: путь к analyzer и путь к тестовому бинарнику. Так команды становятся короче, а риск ошибиться в длинном русском пути сильно ниже.

```
$tool = ".\artifacts\windows-x64\eh_unwind_analyzer.exe"
$sample = ".\бинарники на тестирование\бинарники\mixed\20_mixed_static_unwind_oob_dynamic_uef.exe"
```

Теперь проверяем, что оба файла существуют:

```
Test-Path $tool
Test-Path $sample
```

Если PowerShell два раза выводит True, analyzer и тестовый бинарник найдены. Если где-то выводится False, значит либо проект открыт не в той папке, либо artifact еще не собран/не скопирован в artifacts/windows-x64, либо тестовый файл лежит в другой подпапке.

Дальше создадим три папки для отчетов. Это не обязательное требование программы, но хороший порядок: отчеты по разным режимам не перетирают друг друга.

```
New-Item -ItemType Directory -Force -Path ".\reports\demo_static_only"
New-Item -ItemType Directory -Force -Path ".\reports\demo_dynamic_only"
New-Item -ItemType Directory -Force -Path ".\reports\demo_mixed"
```

4. Почему в командах используется символ &

В PowerShell путь к программе часто берется в кавычки, потому что в нем могут быть пробелы, русские буквы или специальные символы. Но строка в кавычках сама по себе является просто текстом. Чтобы PowerShell выполнил этот текст как команду, перед путем ставится оператор вызова &.

Запись	Смысл
"...\eh_unwind_analyzer.exe"	Просто строка с путем. PowerShell не обязан ее запускать.
& "...\eh_unwind_analyzer.exe"	Запустить файл по указанному пути как программу.
Start-Process ".\reports\demo_mixed\mixed_report.html"	Открыть готовый HTML-файл в браузере через системную ассоциацию.

Именно из-за этого часто возникает ошибка вида Unexpected token или Непредвиденная лексема: PowerShell начинает воспринимать часть пути или HTML-файла как кусок выражения, а не как путь. Безопасное правило простое: analyzer запускаем через &, HTML открываем через Start-Process, все пути с пробелами и русскими буквами берем в кавычки.

5. Режим 1: только статическая проверка

Статический режим нужен, когда мы хотим изучить PE/unwind-структуры без запуска проверяемого файла. Это самый безопасный режим для подозрительных EXE. Он отвечает на вопросы: корректна ли Exception Directory, можно ли прочитать RUNTIME_FUNCTION, куда указывают UnwindInfoAddress, в каких секциях лежит unwind metadata, есть ли пересечения диапазонов функций, есть ли ошибки разбора UNWIND_INFO, какие findings сработали и какой итоговый score получился.

```
& $tool $sample --heuristics --json ".\reports\demo_static_only\static_report.json" --html
".\reports\demo_static_only\static_report.html"
```

Разберем команду по частям:

- & $tool - запускает analyzer из папки artifacts/windows-x64. Символ & нужен PowerShell, чтобы выполнить путь как команду.
- $sample - это анализируемый PE-файл. В примере специально выбран смешанный тестовый бинарник, чтобы один и тот же файл можно было проверить всеми тремя режимами.
- --heuristics - включает статическую оценку признаков. Без этого флага программа все равно разберет PE/unwind, но не посчитает статический score по findings.
- --json ".\reports\demo_static_only\static_report.json" - сохраняет машинно-читаемый JSON-отчет.
- --html ".\reports\demo_static_only\static_report.html" - сохраняет удобный HTML-отчет для просмотра в браузере.

После выполнения открой HTML:

```
Start-Process ".\reports\demo_static_only\static_report.html"
```

В этом отчете нужно смотреть прежде всего блоки Статическая оценка, Итоговый вывод, Структурная проверка, Статические findings и Таблица RUNTIME_FUNCTION. Карточка Динамическая проверка должна показывать, что она не запрашивалась. Это нормально: файл не запускался.

> Практический смысл статического режима: он показывает, что подозрительность может быть доказана по самому формату файла. Например, если UnwindInfoAddress указывает за пределы image или UNWIND_INFO невозможно разобрать, это видно без исполнения EXE. Для malware analysis это критично, потому что запускать неизвестный файл на рабочей машине опасно.

6. Режим 2: только динамическая проверка

Динамический режим нужен, когда мы хотим увидеть поведение файла во время выполнения. В этом режиме analyzer запускает target как отлаживаемый процесс, наблюдает за exception-related API и handler-событиями, фиксирует вызовы AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback, попадания в handler и другие runtime-наблюдения. В исходном коде включение динамики находится в src/main.c, строки 401-424.

```
& $tool $sample --dynamic --timeout-ms 5000 --json ".\reports\demo_dynamic_only\dynamic_report.json" --html
".\reports\demo_dynamic_only\dynamic_report.html"
```

Разберем команду:

- --dynamic - включает динамический слой. Analyzer создаёт процесс target и работает с ним через Windows Debug API.
- --timeout-ms 5000 - ограничивает динамический запуск пятью секундами. Это защита от зависания, бесконечного ожидания или слишком долгой работы test binary.
- --json и --html - сохраняют отчет так же, как в статическом режиме.
- --heuristics здесь отсутствует, поэтому статический score не считается. При этом базовый PE/unwind-разбор все равно выполняется, потому что динамике нужны координаты модулей, секций, функций и unwind metadata.

```
Start-Process ".\reports\demo_dynamic_only\dynamic_report.html"
```

В динамическом отчете нужно смотреть блок Динамическая проверка, таблицу runtime-событий, журнал выполнения, counters по исключениям, handler hits, вызовам runtime-таблиц и потерянным событиям. Если вверху написано Статическая оценка: не запускалась, это не ошибка. Это честное указание, что флаг --heuristics не был задан.

> Динамический режим опаснее статического, потому что он реально запускает EXE. Его нужно использовать только на учебных тестовых бинарниках или в контролируемой лабораторной среде. Для реального подозрительного файла обычный порядок такой: сначала статический анализ, потом, если нужно, динамика в изолированной среде.

7. Режим 3: смешанная проверка одного бинарника

Смешанный режим - главный демонстрационный режим проекта. Он показывает, что программа умеет не просто проверять файл по структурам и не просто смотреть runtime-события, а соединять оба слоя evidence. Это важно для защиты ВКР: статическая часть отвечает на вопрос 'что подозрительно в PE/unwind metadata', а динамическая часть отвечает на вопрос 'что из exception-handling поведения проявилось при запуске'.

```
& $tool $sample --heuristics --dynamic --timeout-ms 5000 --json ".\reports\demo_mixed\mixed_report.json" --html
".\reports\demo_mixed\mixed_report.html"
```

Открываем итоговый HTML:

```
Start-Process ".\reports\demo_mixed\mixed_report.html"
```

В этом режиме в верхней панели должны быть заполнены обе стороны: статическая оценка и динамические сигналы. Если статические findings есть, они будут перечислены в таблице findings с правилом, наблюдением, ожидаемым состоянием и влиянием. Если dynamic collector увидел runtime-события, они попадут в таблицу динамической проверки и в журнал выполнения.

> Смешанный режим не означает, что динамика обязана подтвердить каждый статический finding. Это принципиально важно. Статика видит структуру файла полностью, а динамика видит только тот путь выполнения, который реально произошел за время запуска. Если подозрительный handler существует в файле, но программа во время теста до него не дошла, статический finding останется, а динамический handler hit может быть нулевым. Это не ошибка проекта, а нормальное различие между static analysis и runtime observation.

8. Что именно означает каждый основной флаг

Флаг	Техническое значение	Как объяснять на защите
--heuristics	Вызывает heuristics_analyze после PE/unwind-разбора.	Включает статическую доказательную базу: findings, score, verdict, confidence.
--dynamic	Вызывает dynamic_collect_windows и запускает target через Windows debugger.	Включает runtime evidence: исключения, handler hits, VEH/VCH/UEF, runtime function table events.
--timeout-ms N	Ограничивает время динамического запуска.	Нужен, чтобы анализ не зависал и чтобы демонстрация была управляемой.
--trace-steps N	Настраивает число single-step samples после handler hit.	Позволяет тоньше наблюдать поведение после попадания в handler, но может замедлить динамику.
--json <path>	Пишет структурированный JSON-отчет.	Нужен для автоматической проверки, регрессии, интеграции и сравнения результатов.
--html <path>	Пишет автономный HTML-отчет.	Нужен человеку: reverse engineer, malware analyst или комиссии проще читать dashboard.
--strict-abi	Добавляет формальные проверки Microsoft x64 unwind ABI.	Полезно, когда нужно строже отличать допустимые нестандартности от нарушений ABI.
--no-follow-chain	Отключает рекурсивный разбор chained unwind records.	Нужно для диагностических сравнений, когда важно увидеть поведение без follow-chain.
--max-chain-depth N	Ограничивает глубину цепочек chained unwind.	Защищает анализатор от зацикливания и слишком глубокой рекурсии на поврежденных структурах.
--	Все аргументы после этого передаются target-программе.	Нужно, если анализируемому EXE требуются собственные параметры командной строки.

9. Как передать аргументы анализируемому EXE

Иногда target-программа сама требует параметры: путь к входному файлу, режим работы, ключи командной строки. Чтобы analyzer не пытался воспринимать эти параметры как свои собственные флаги, используется разделитель --. Все, что стоит после него, передается уже target-программе.

```
& $tool ".\some_target.exe" --dynamic --timeout-ms 7000 --json ".\reports\target_dynamic.json" --html
".\reports\target_dynamic.html" -- --input ".\sample.dat" --mode test
```

В этой команде --dynamic, --timeout-ms, --json и --html относятся к analyzer. А --input .\sample.dat --mode test относятся уже к проверяемой программе. Это стандартный прием для инструментов, которые сами запускают другую программу.

10. Как читать верхнюю панель HTML в трех режимах

После доработки HTML-отчета верхняя панель имеет единый стиль во всех режимах. Это сделано специально: пользователь не должен гадать, какой слой анализа был выполнен, а какой нет. Карточки не исчезают; они показывают состояние слоя.

Карточка	В статике	В динамике	В смешанном режиме
Записи RUNTIME_FUNCTION	Показывает число записей .pdata.	Тоже показывает, потому что базовый parse выполняется всегда.	Показывает то же число для общей координатной базы.
Ошибки разбора	Показывает ошибки чтения/разбора UNWIND_INFO.	Может показывать ошибки базового parse, даже если score не считался.	Используется и для статической оценки, и для объяснения evidence.
Эвристические признаки	Заполнены числом findings.	Показывает 'не запускалась'.	Заполнены числом findings.
Статическая оценка	Показывает score, например 50 / 100.	Показывает 'не запускалась'.	Показывает score и участвует в итоговой интерпретации.
Статика / динамика	Например 'сильный / нет'.	Например 'нет / слабый'.	Показывает силу обоих слоев рядом.
Динамические сигналы	Обычно 0, если dynamic не запускался.	Показывает число runtime-сигналов.	Показывает число runtime-сигналов рядом со статической оценкой.
Динамическая проверка	Пишет 'не запрашивалась'.	Пишет 'завершена' или состояние ошибки/усечения.	Пишет итог выполнения dynamic collector.

> Если в динамическом режиме наверху видна низкая итоговая уверенность, это не обязательно значит, что динамика бесполезна. Итоговая уверенность осторожна, потому что без --heuristics отсутствует статическая доказательная база. Динамические события при этом остаются в таблицах и журнале выполнения.

11. Почему один и тот же бинарник можно запускать в трех режимах

Задача проекта действительно формулируется так: взять один PE-файл, построить его статическую картину и, если пользователь разрешил, дополнительно запустить его и собрать динамические события. Поэтому корректная демонстрация может использовать один и тот же файл 20_mixed_static_unwind_oob_dynamic_uef.exe трижды: сначала как static-only, затем как dynamic-only, затем как mixed. Меняется не бинарник, а включенные слои анализа.

- Static-only показывает, что в файле есть структурные признаки: например, плохой UnwindInfoAddress или parse fail.

- Dynamic-only показывает, какие exception-related события реально возникли при запуске.
- Mixed показывает обе картины вместе и помогает объяснить, совпадают ли статические подозрения с runtime-поведением.

На защите это можно сформулировать так: инструмент поддерживает независимое включение слоев анализа. Базовый PE/unwind parser нужен всегда; статический режим добавляет оценку признаков; динамический режим добавляет runtime evidence; смешанный режим объединяет обе доказательные базы в одном отчете.

12. Что писать в отчете ВКР про безопасность запуска

Статический режим не исполняет проверяемый файл, поэтому он безопаснее и подходит для первичного анализа подозрительных PE. Динамический режим исполняет target. Это принципиально другая модель риска: код target получает возможность выполняться в системе. Поэтому динамику следует проводить только на тестовых бинарниках или в изолированной среде. Для реальных вредоносных образцов нужна виртуальная машина, снапшот, отключенные личные учетные записи, контролируемая сеть и понимание последствий запуска.

Также важно помнить, что динамика в проекте Windows-only. Причина не в ограничении языка C, а в предметной области: динамический collector использует Windows Debug API и наблюдает Windows exception/unwind behavior. На macOS такой runtime-слой напрямую не повторяется, потому что там другой loader, другой формат исполняемых файлов и другая модель исключений. На MacBook проект можно изучать, читать, переносить статическую часть или запускать Windows-сборку в Windows VM/Parallels, но динамический режим требует Windows-среды.

13. Типовые ошибки пользователя и как их исправлять

Симптом	Причина	Что делать
PowerShell пишет 'Непредвиденная лексема html'.	Путь к HTML вставлен без Start-Process или команда была склеена неправильно.	Открывать отчет так: Start-Process ".\reports\demo_mixed\mixed_report.html".
Start-Process пишет, что файл не найден.	Отчет еще не создан или путь отличается.	Сначала выполнить команду analyzer, затем проверить Test-Path ".\reports\demo_mixed\mixed_report.html".
В dynamic-only отчете написано 'Статическая оценка: не запускалась'.	Запуск был без --heuristics.	Это нормально. Для полной оценки использовать смешанный режим --heuristics --dynamic.
В static-only отчете написано 'Динамическая проверка: не запрашивалась'.	Запуск был без --dynamic.	Это нормально. Файл не исполнялся.
В журнале динамики есть строки 'module not present in target yet'.	Debugger пытался разрешить API до загрузки нужного модуля в target.	Это диагностическая строка, не обязательно ошибка анализа.
Динамический запуск не видит ожидаемый handler hit.	Путь выполнения target не дошел до handler за timeout или событие не произошло.	Увеличить --timeout-ms, проверить тестовый бинарник, передать нужные аргументы после --.
Команда не запускается из-за русских путей или пробелов.	Пути не заключены в кавычки или отсутствует оператор & перед analyzer.	Все пути брать в кавычки; analyzer запускать через &.

14. Практический сценарий проверки перед защитой

Перед демонстрацией удобно выполнить короткий ритуал проверки, чтобы не зависеть от памяти и не путаться в путях. Его смысл: перейти в корень проекта, проверить наличие analyzer, создать папки reports, выполнить три режима, открыть три HTML и сравнить верхние панели.

```
cd "C:\Users\Templar\Desktop\вкр\ВКР ПРОЕКТ ОКОНЧЕННЫЙ\ip_full"
$tool = ".\artifacts\windows-x64\eh_unwind_analyzer.exe"
$sample = ".\бинарники на тестирование\бинарники\mixed\20_mixed_static_unwind_oob_dynamic_uef.exe"
Test-Path $tool
Test-Path $sample
```

```
New-Item -ItemType Directory -Force -Path ".\reports\demo_static_only"
New-Item -ItemType Directory -Force -Path ".\reports\demo_dynamic_only"
New-Item -ItemType Directory -Force -Path ".\reports\demo_mixed"
```

```
& $tool $sample --heuristics --json ".\reports\demo_static_only\static_report.json" --html
".\reports\demo_static_only\static_report.html"
& $tool $sample --dynamic --timeout-ms 5000 --json ".\reports\demo_dynamic_only\dynamic_report.json" --html
".\reports\demo_dynamic_only\dynamic_report.html"
& $tool $sample --heuristics --dynamic --timeout-ms 5000 --json ".\reports\demo_mixed\mixed_report.json" --html
".\reports\demo_mixed\mixed_report.html"
```

```
Start-Process ".\reports\demo_static_only\static_report.html"
Start-Process ".\reports\demo_dynamic_only\dynamic_report.html"
Start-Process ".\reports\demo_mixed\mixed_report.html"
```

Если нужно показать комиссии именно отличие режимов, открываются три вкладки браузера. В первой видно статическую доказательную базу без запуска EXE. Во второй видно runtime evidence без статической шкалы. В третьей видно объединение двух слоев. Это лучший способ объяснить архитектуру проекта без ухода в одни только исходники.

15. Как коротко объяснить архитектуру режимов комиссии

Можно использовать такую формулировку:

> Анализатор имеет общий обязательный pipeline: чтение PE, проверка заголовков, построение карты секций, чтение Exception Directory, чтение RUNTIME_FUNCTION и разбор UNWIND_INFO. После этого пользователь выбирает слой оценки. Флаг --heuristics включает статические признаки и скоринг. Флаг --dynamic включает запуск под debugger и сбор runtime-событий. При совместном использовании флагов программа формирует единый HTML/JSON, где статические findings и динамические сигналы видны вместе. Поэтому проект не просто делает несколько if, а строит доказательную цепочку от PE-структур до итогового отчета.

С точки зрения исходников это подтверждается маршрутом в src/main.c: help и список флагов находятся в строках 95-115; разбор аргументов - в строках 180-236; обязательное чтение файла и PE parse - в строках 283-299; чтение Exception Directory и RUNTIME_FUNCTION - в строках 306-324; разбор UNWIND_INFO - в строках 351-356; статические эвристики включаются только при --heuristics в строках 388-399; динамический collector включается только при --dynamic в строках 401-424; JSON/HTML пишутся в строках 484-509.

Главный вывод: три режима - это не косметика командной строки, а отражение трех разных уровней доказательности. Статика отвечает за структуру файла. Динамика отвечает за наблюдаемое поведение процесса. Смешанный режим показывает, как эти две картины соотносятся друг с другом.

16. Защитное пояснение: почему статика имеет 50 / 100, а динамика показана как слабая и завершена

В HTML-отчете рядом находятся несколько карточек, которые на первый взгляд могут выглядеть как одна общая оценочная шкала. На самом деле это разные типы информации. Комиссия может спросить: если у статической проверки есть число 50 / 100, почему у динамической проверки не написано, например, 30 / 100? Ответ: потому что статическая часть и динамическая часть имеют разную природу доказательности. Статика оценивает полную структуру файла, а динамика фиксирует события, которые реально произошли в одном конкретном запуске.

> Короткая формулировка для защиты: в проекте числовой score относится к статической эвристической оценке PE/unwind-структур. Динамика оценивается не отдельной шкалой 0-100, а уровнем runtime evidence: нет, слабый, средний, сильный. Карточка Динамическая проверка показывает не оценку подозрительности, а технический статус запуска dynamic collector: не запрашивалась, не запускалась, завершена, тайм-аут или ошибка.

16.1. Какие карточки видит пользователь и что они означают

Карточка в HTML	Что это за сущность	Как правильно объяснять
Статическая оценка: 50 / 100	Числовой score статических признаков. Он появляется только если запуск был с --heuristics.	Это оценка структуры PE/unwind metadata: findings, веса, пороги, severity, confidence. Она не означает, что динамика тоже должна иметь такую же шкалу.
Статика / динамика: сильный / слабый	Сравнение силы доказательств двух слоев. Левая часть относится к статике, правая - к динамике.	Вот здесь динамика как раз оценена: слабый - это уровень runtime evidence, а не статус запуска.
Динамические сигналы: 3	Количество категорий runtime-событий, которые analyzer заметил в динамическом запуске.	Это счетчик наблюдаемых фактов, а не итоговая оценка 3 из 100 или 3 балла. Один сигнал - это семейство событий, например handler hit или UEF.
Динамическая проверка: завершена	Технический статус dynamic collector.	Это значит, что динамический запуск был выполнен без общей ошибки collector. Это не значит, что динамика дала высокий уровень подозрительности.
Итоговая уверенность: 69%	Объединенная уверенность summary-слоя.	Она учитывает статическую confidence и может усиливаться динамическими сигналами или корреляцией, но не заменяет подробные findings.

Главная опасность неправильного чтения отчета - перепутать статус и оценку. Слово завершена отвечает на вопрос: выполнилась ли динамическая процедура? Слово слабый отвечает на другой вопрос: насколько сильными оказались runtime-доказательства в этом запуске? Число 50 / 100 отвечает на третий вопрос: насколько подозрительна статическая картина файла по выбранным признакам?

16.2. Почему статике можно дать 0-100, а динамике опасно давать такую же шкалу

Статический анализ читает файл как объект. Если PE-файл лежит на диске, analyzer может последовательно прочитать заголовки, секции, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handler RVA и проверить инварианты формата. Эта область конечна: у файла есть конкретные байты, конкретные RVA, конкретные границы SizeOfImage, конкретные section characteristics. Поэтому findings можно взвесить и привести к числовому score. Score 50 / 100 означает не вероятность вируса, а суммарную выраженность статических нарушений и подозрительных сочетаний.

Динамический анализ устроен иначе. Он не видит всю возможную жизнь программы, а наблюдает один запуск: с данным timeout, с данными аргументами, в данной ОС, с данной загрузкой DLL, по тому пути выполнения, который реально произошел. Если событие не произошло за пять секунд, это не доказывает, что такого события в программе нет. Возможно, нужен другой входной файл, другой аргумент, другой branch, другая ошибка или другое окружение. Поэтому механически переводить динамику в 0-100 было бы ложной точностью.

- Статика полнее по покрытию файла: она может проверить таблицы и адреса, даже если код никогда не был выполнен.
- Динамика сильнее по факту поведения: если handler реально вызвался, это важное evidence, потому что оно произошло в runtime.
- Но динамика неполна по пространству путей: один запуск не обязан пройти все ветки программы.
- Отсутствие динамического события не равно отсутствию механизма: handler может существовать, но не сработать в данном запуске.
- Поэтому проект использует уровни evidence: none, weak, medium, strong, а не притворяется, что динамический запуск дал абсолютный процент.

16.3. Как именно проект оценивает динамику технически

Динамика в проекте не остается без оценки. Она просто оценивается другой шкалой. В src/report.c есть функция dynamic_signal_count в строках 486-495. Она считает не каждое событие подряд, а категории динамических сигналов. Это сделано правильно: если одна и та же API вызвалась несколько раз, нельзя бесконечно накручивать score только повторением одного типа события.

```
dynamic_signal_count(dr):
  +1 если были попадания в unwind/VEH/VCH/UEF handler
  +1 если были регистрации VEH
  +1 если были регистрации VCH
  +1 если был SetUnhandledExceptionFilter или UEF hit
  +1 если были RtlAddFunctionTable или RtlInstallFunctionTableCallback
  +1 если было second-chance exception
```

После этого работает runtime_evidence_level в src/report.c, строки 522-534. Она переводит runtime-наблюдения в качественный уровень: none, weak, medium или strong. Логика примерно такая:

```
strong evidence:
  handler реально сработал: unwind_handler_hits / veh_hits / vch_hits / uef_hits

medium evidence:
  runtime function table API: RtlAddFunctionTable / RtlInstallFunctionTableCallback
  или second-chance exception

weak evidence:
  регистрации обработчиков или first-chance exception

итоговый уровень:
  strong + medium -> strong
  strong или два medium -> medium
  один medium или один weak -> weak
  если dynamic session не полностью complete, ненулевой уровень понижается до weak
```

Отсюда важный вывод: слабый не означает, что динамика не работает. Это означает, что в данном запуске runtime evidence есть, но оно не достаточно сильное или не достаточно полное, чтобы называть его средним или сильным. Например, регистрации обработчиков и отдельные runtime-события являются сигналами, но они слабее, чем реальное попадание в suspicious handler в сочетании с runtime function table activity.

16.4. Почему завершена и слабый могут стоять одновременно

Это нормальная ситуация, потому что эти слова отвечают на разные вопросы. В src/report.c, строки 1200-1205, функция ru_status_text формирует текст статуса динамического запуска. Если dynamic report есть, запуск не ушел в timeout, и collector вернул ok, в карточке показывается завершена. Это техническое состояние: процедура динамического наблюдения отработала.

Но уровень слабый берется не из ru_status_text, а из runtime_evidence_level. Он зависит от того, какие именно runtime-сигналы были найдены и была ли динамическая сессия полной по строгим критериям. Строгая полнота dynamic session проверяется в dynamic_session_complete, строки 498-505: ran, ok, не timeout, target_exited и instrumentation_complete. Поэтому возможна ситуация: collector в целом завершился нормально, но evidence остается слабым, потому что события были ограниченными, не было сильной корреляции или instrumentation не дала полной картины.

Вопрос	Где ответ в отчете	Что значит
Запускалась ли динамика и не упал ли collector?	Динамическая проверка: завершена	Да, dynamic collector отработал как процедура наблюдения.
Насколько сильны runtime-доказательства?	Статика / динамика: сильный / слабый	Статика дала сильный слой evidence, динамика дала слабый слой evidence.
Сколько категорий runtime-сигналов было замечено?	Динамические сигналы: 3	Найдены три семейства динамических наблюдений, но это не число подозрительности из 100.
Каков общий вывод по файлу?	Итоговый вывод и Итоговая уверенность	Объединенный summary учитывает статический score, runtime evidence и корреляцию слоев.

16.5. Как объяснить конкретный пример со скрина

На скрине показано: Статическая оценка 50 / 100, Итоговый вывод: умеренная статическая подозрительность, Статика / динамика: сильный / слабый, Динамические сигналы: 3, Динамическая проверка: завершена. Правильное чтение такое:

- 50 / 100 - это score статических findings. Он говорит, что в структуре PE/unwind есть достаточно заметные проблемы, но они не дошли до high threshold.

- Умеренная статическая подозрительность - итоговый verdict по статике, потому что score попал в умеренный диапазон.
- Сильный / слабый - сравнение слоев evidence. Статика сильная, потому что findings структурно выражены. Динамика слабая, потому что runtime-события есть, но они не дают такой же силы доказательства.
- Динамические сигналы: 3 - обнаружены три семейства динамических наблюдений. Это помогает анализу, но не превращается автоматически в 70 / 100.
- Завершена - dynamic collector выполнил запуск и не сообщил общий failure. Это технический статус, а не verdict.

Если комиссия спросит, почему при трех динамических сигналах уровень динамики слабый, ответ такой: потому что количество сигналов и сила evidence - разные вещи. Три слабых или неполных наблюдения не равны одному сильному подтверждению. Сильным runtime evidence будет, например, реальное попадание в suspicious handler вместе с activity runtime function table или другой корреляцией со статическими findings. Проект специально не завышает динамический вывод, чтобы не выдавать ограниченный запуск за полное доказательство.

16.6. Почему это архитектурно правильно

Если бы проект показывал Динамическая оценка: 30 / 100, у комиссии сразу возник бы более тяжелый вопрос: откуда взялись эти 30, почему не 20 и не 60, и почему один запуск программы можно считать полным измерением поведения? Поэтому выбран более честный подход: статике дается числовой score, потому что она проверяет конечный набор структурных признаков; динамике дается уровень evidence, потому что она наблюдает ограниченный runtime-сценарий.

Такой подход лучше для reverse engineering и malware analysis. Аналитик не должен воспринимать динамический результат как абсолютный процент опасности. Он должен видеть, что именно произошло: какие handlers были зарегистрированы, были ли hits, были ли runtime function table calls, были ли exceptions, были ли ошибки breakpoint, был ли timeout, была ли instrumentation complete. Поэтому подробные динамические counters оставлены в блоке Динамическая проверка, а верхняя карточка показывает только краткое summary.

Итоговая защита решения: проект не игнорирует динамику. Он разделяет три разных понятия: статический score, динамический уровень evidence и технический статус dynamic collector. Это делает отчет честнее, потому что не смешивает структурную оценку файла с ограниченным наблюдением одного запуска.

16.7. Готовый ответ комиссии

В отчете число 50 / 100 относится только к статической оценке, потому что статическая часть проверяет конечный и воспроизводимый набор PE/unwind-признаков. Динамическая часть устроена иначе: она наблюдает один конкретный запуск и поэтому не должна изображать абсолютный процент покрытия поведения. В проекте динамика оценивается качественным уровнем evidence - на данном скрине это слабый. Отдельная карточка Динамическая проверка: завершена показывает не оценку подозрительности, а статус того, что dynamic collector отработал. Динамические сигналы: 3 показывает число семейств runtime-наблюдений. Итоговая уверенность объединяет статическую confidence, runtime evidence и корреляцию слоев. Поэтому статика и динамика в проекте оцениваются обе, но разными шкалами, потому что у них разная доказательная природа.

17. Подробный туториал: как правильно читать HTML-отчет

HTML-отчет нужно читать не как один большой список чисел, а как доказательную цепочку. Отчет построен так, чтобы пользователь сначала увидел краткий итог, затем проверил исходные координаты PE-файла, затем посмотрел структурные нарушения, затем разобрал статические findings, динамические события и только после этого сделал вывод. Правильный порядок чтения особенно важен на защите: если сразу смотреть только score, можно потерять смысл evidence.

> Короткая схема чтения: сначала режим запуска и верхние карточки, затем метаданные, затем PE-образ и структурная проверка, затем секции, затем итоговая оценка, затем статические признаки, затем динамическая проверка, затем таблица RUNTIME_FUNCTION. В конце связываем все обратно в один вывод.

17.1. Верхний блок отчета - быстрый dashboard

Верхний блок нужен для первого взгляда. Он отвечает на вопрос: что произошло в целом? Но он не заменяет подробные таблицы ниже. Если комиссия показывает на одну карточку и спрашивает, что она значит, нужно объяснять ее не отдельно, а в связке с остальными карточками.

Поле	Что означает	Как читать
Записи RUNTIME_FUNCTION	Сколько записей function table найдено в Exception Directory.	Если 0, то либо таблица отсутствует, либо parser не смог ее прочитать. Для x64 PE наличие записей важно для unwind.
Ошибки разбора	Сколько UNWIND_INFO не удалось корректно разобрать.	Ненулевое значение важно: таблица функций есть, но часть unwind metadata повреждена, недоступна или указывает не туда.
Аномальные записи	Сколько записей имеют признаки необычности на уровне unwind-разбора.	Это не всегда критическая ошибка, но повод перейти в таблицу RUNTIME_FUNCTION и посмотреть конкретные строки.
Эвристические признаки	Сколько статических findings сработало при --heuristics.	Это количество найденных признаков, а не score. Один сильный finding может быть важнее нескольких слабых.
Статическая оценка	Числовой score по статическим признакам.	Это основная шкала статики. Она показывает суммарную выраженность статических нарушений, но не является процентом вирусности.
Итоговый вывод	Краткий verdict summary-слоя.	Читать вместе со score, confidence и подробными findings. Например, умеренная статическая подозрительность требует просмотра причин ниже.
Статика / динамика	Сила evidence по двум слоям.	Левая часть - статика, правая - runtime evidence. Это место, где видно, насколько динамика усиливает или не усиливает статический вывод.
Динамические сигналы	Количество семейств runtime-событий.	Это факты поведения, а не score. Счетчик помогает понять, была ли динамика вообще информативной.
Итоговая уверенность	Объединенная confidence summary-слоя.	Это осторожная уверенность в выводе, а не абсолютная вероятность вредоносности.

Поле	Что означает	Как читать
Динамическая проверка	Статус dynamic collector.	Показывает, была ли динамика запрошена и чем завершился запуск: не запрашивалась, завершена, ошибка или тайм-аут.

17.2. Метаданные отчета - как доказать, что анализ воспроизводим

Сразу под верхними карточками находится блок метаданных: время формирования, версия инструмента и schema, SHA-256 файла, сборка, параметры анализа, размер файла и командная строка. Это не декоративный блок. Он нужен, чтобы отчет можно было воспроизвести и проверить.

- Сформировано - время создания отчета. Помогает понять, какой отчет свежий.
- Инструмент / schema - версия analyzer и версия JSON/HTML schema. Это важно, если формат отчета менялся.
- SHA-256 - криптографический отпечаток анализируемого файла. Если файл изменится, hash будет другим.
- Сборка - платформа, компилятор и режим сборки. Это помогает объяснить, какой artifact использовался.
- Параметры анализа - strict ABI, follow chain, max depth. Эти параметры влияют на строгость разбора chained unwind.
- Размер файла - базовая проверка, что анализировался ожидаемый файл.
- Команда - точная командная строка запуска. По ней видно, была ли статика, динамика или смешанный режим.

Если комиссия спрашивает, как понять, что отчет относится именно к этому бинарнику, ответ: по SHA-256, пути входного файла и сохраненной командной строке. Если спрашивает, были ли включены эвристики и динамика, ответ: это видно в командной строке по флагам --heuristics и --dynamic.

17.3. Блок Образ PE - координатная система файла

Блок Образ PE объясняет, в какой координатной системе работает parser. Здесь важны ImageBase, SizeOfImage, SizeOfHeaders, Exception directory и alignment. Без этих полей нельзя корректно интерпретировать RVA, VA и file offset.

Поле	Смысл	Почему важно
ImageBase	Предпочтительная база загрузки образа.	VA = ImageBase + RVA при загрузке по preferred base. Для статического чтения это не file offset.
SizeOfImage	Размер образа в памяти после загрузки.	RVA структур не должен выходить за допустимые границы image.
SizeOfHeaders	Размер области PE-заголовков.	Некоторые RVA могут указывать в headers, и parser должен учитывать это отдельно от секций.
Exception directory	RVA и Size каталога исключений.	Это входная точка к .pdata и массиву RUNTIME_FUNCTION на x64.

Поле	Смысл	Почему важно
SectionAlignment / FileAlignment	Выравнивание виртуальных и файловых областей.	Помогает отличать корректную раскладку от поврежденной или странной.

Читать этот блок нужно до findings. Например, если finding говорит, что UnwindInfoAddress вышел за image, надо понимать, что граница берется не на глаз, а из SizeOfImage и карты секций.

17.4. Структурная проверка - базовая целостность PE

Структурная проверка показывает, насколько секции и Exception Directory согласованы с физическим файлом и виртуальным образом. Это уровень ниже эвристик. Здесь analyzer проверяет, можно ли вообще доверять координатам, по которым дальше читаются RUNTIME_FUNCTION и UNWIND_INFO.

Поле	Что проверяет	Как интерпретировать
Raw-диапазоны вне файла	Секция заявляет PointerToRawData/SizeOfRawData, которые выходят за размер файла.	Если не 0, секция ссылается на байты, которых физически нет в файле.
Virtual-диапазоны вне image	VirtualAddress/VirtualSize секции выходят за SizeOfImage.	Если не 0, виртуальная карта image противоречит общему размеру образа.
Пересечения raw	Физические диапазоны секций на диске пересекаются.	Один и тот же участок файла как будто принадлежит разным секциям. Это подозрительно или повреждение.
Пересечения virtual	Виртуальные диапазоны секций пересекаются.	В памяти разные секции претендуют на один RVA-диапазон.
Невыровненные секции	Нарушения alignment, порядка или размеров.	Не всегда вредоносно, но снижает доверие к PE-карте.
Exception directory OOB	Каталог исключений недоступен, выходит за границы или не переводится в file offset.	Критично для проекта: без него нельзя нормально прочитать .pdata.

17.5. Таблица Секции - где лежат код, данные и metadata

Таблица секций показывает имя секции, VirtualAddress, VirtualSize, Raw offset, Raw size и права R/W/X. Ее нужно читать как карту местности. Когда finding говорит, что unwind metadata лежит в executable или writable области, именно таблица секций помогает увидеть, почему это странно.

- Имя - условное имя секции, например .text, .rdata, .pdata, .xdata, .data.
- VirtualAddress - RVA начала секции в загруженном image.
- VirtualSize - виртуальный размер секции в памяти.
- Raw offset - физическое смещение секции в файле на диске.
- Raw size - сколько байт секции реально лежит в файле.
- Права - R/W/X. Для .text обычно ожидается execute, для .rdata/.pdata/.xdata обычно read-only.

Если .xdata или .pdata имеют X или W без понятной причины, это не автоматический приговор, но хороший повод проверить findings по placement и integrity. Если raw size равен 0, а virtual size не 0, это может быть zero-fill область: в памяти область есть, а в файле байтов нет.

17.6. Итоговая оценка - зачем она повторяет верхний блок

Раздел Итоговая оценка действительно частично повторяет верхний dashboard. Это сделано не случайно. Верхний блок находится в шапке отчета и нужен для быстрого первого взгляда. Раздел Итоговая оценка находится уже внутри тела отчета, после PE-образа, структурной проверки и секций. Он повторяет ключевой вывод в месте, где читатель уже увидел технический контекст.

Верхний блок	Раздел Итоговая оценка
Быстрый dashboard: что произошло в целом.	Интерпретационный блок: как связать findings, динамику, confidence и корреляцию.
Нужен, чтобы за 5 секунд понять состояние анализа.	Нужен, чтобы после просмотра PE-контекста сформулировать вывод.
Показывает часть summary рядом с метаданными запуска.	Добавляет силу доказательств, корреляцию слоев и план ручной проверки.
Удобен при демонстрации на экране.	Удобен при чтении отчета сверху вниз или печати.

Как это защищать: повтор не является ошибкой архитектуры, потому что эти блоки имеют разные роли. Dashboard - навигационный и обзорный. Итоговая оценка - аналитический summary-блок. В серьезных отчетах это нормальная практика: один и тот же verdict может быть показан в executive summary и затем повторен после доказательной части.

В разделе Итоговая оценка особенно важны поля Вывод, Сила доказательств, Итоговая уверенность, Статика / динамика, Корреляция слоев и Динамические сигналы. Если появляется План ручной проверки, его нужно читать как подсказку аналитику: что проверить вручную в Ghidra, debugger или hex viewer.

17.7. Статические признаки - главная таблица findings

Раздел Статические признаки появляется, если запуск был с --heuristics. Это один из главных разделов отчета. Здесь каждое finding должно отвечать: где найдено, какое правило нарушено, что наблюдалось, как должно быть и почему это важно.

Колонка	Что означает	Как читать
Важность	severity finding: высокая, средняя, низкая, информационная.	Сначала смотреть высокие и средние. Низкие не игнорировать, но они обычно контекстные.
Код	Уникальный код признака.	По нему удобно ссылаться в тексте ВКР и отвечать комиссии: например unwind_rva_oob.
Семейство	Группа признаков: PE structure, unwind integrity, metadata placement, handler layer и т.д.	Помогает понять, на каком уровне возникла проблема.
Затронуто	Сколько объектов попало под finding.	Если затронуто много записей, проблема системная. Если одна, надо смотреть конкретную функцию.
RVA	Представительный адрес проблемы.	Это не file offset. Через секцию можно понять, куда он попадает.
Секция	Секция, куда попадает representative RVA.	Помогает понять контекст: .text, .pdata, .xdata, .rdata или неизвестная область.

Колонка	Что означает	Как читать
Описание	Русское описание плюс evidence block.	Здесь читать Rule, Observed, Expected, Impact. Это основа защиты finding.

Правильная методика чтения finding: сначала код и важность, затем affected count, затем RVA/секция, затем Rule/Observed/Expected/Impact. Если читать только текст finding без evidence, можно упустить, на каких полях PE он основан.

17.8. Правила снижения уверенности и Детали анализа

Иногда рядом со статическими признаками появляется раздел Правила снижения уверенности. Он нужен, чтобы не завышать вывод, когда есть объясняющие или смягчающие обстоятельства. Например, не каждый необычный layout означает вредоносность. Некоторые вещи могут быть следствием компилятора, упаковщика, тестового бинарника или неполной динамической сессии.

Раздел Детали анализа - это компактный текстовый список сработавших findings. Его удобно использовать как короткую выжимку: какие коды сработали и с какой важностью. Но для глубокого ответа комиссии лучше возвращаться к полной таблице Статические признаки, потому что там есть evidence.

17.9. Динамическая проверка - как читать runtime evidence

Раздел Динамическая проверка появляется, если запуск был с --dynamic. Он показывает не статический score, а факты runtime-наблюдения. Сначала читаются карточки-счетчики, затем таблица расшифровки динамических сигналов, затем event table и журнал выполнения.

Поле	Что означает	Как оценивать
Исключения	Сколько exception-событий заметил debugger.	Ноль не всегда плохо: не каждый тестовый сценарий обязан генерировать exception.
Попадания в handler	Сколько раз execution дошел до handler.	Сильный runtime-факт, особенно если handler связан со статически подозрительным unwind/handler placement.
Вызовы runtime-таблиц	RtlAddFunctionTable или RtlInstallFunctionTableCallback.	Важный сигнал динамического управления unwind metadata.
Потерянные события	Сколько событий не удалось сохранить.	Если не 0, динамическая картина может быть неполной.
Инструментация завершена	Удалось ли завершить instrumentation setup.	Если нет, runtime evidence понижается, потому что collector видел не все, что хотел.
Handlers найдены / включены	Сколько handler targets обнаружено и на сколько поставлены breakpoints.	Если найдено больше, чем включено, надо смотреть ошибки breakpoint.
Ошибки breakpoint	Неудачные попытки поставить breakpoint.	Могут ограничить динамическую наблюдаемость.
Покрытие модулей	Пик модулей, ошибки refresh, усечения списка.	Если есть ошибки или усечения, dynamic session менее полная.

Динамическую таблицу событий нужно читать как журнал фактов. Колонка Тип - машинное имя события, Русское название - человекочитаемая расшифровка, Модуль - DLL или EXE, Thread - поток, Exception - код исключения, Chance - first/second chance, Address - адрес события, Arg0-Arg3 -

аргументы API или handler, Смысл события - короткое объяснение, почему событие важно.

> Журнал выполнения нужен для диагностики collector. Там можно увидеть, какие API разрешались, где ставились breakpoints, какие self-check прошли, какие события произошли, почему могли быть предупреждения вроде module not present in target yet. Это не обязательно finding, но важный контекст динамической проверки.

17.10. Таблица RUNTIME_FUNCTION - низкоуровневая карта функций

Таблица RUNTIME_FUNCTION - это техническое ядро отчета. Она показывает каждую запись function table. Если комиссия говорит, что проект просто считает score, нужно открыть эту таблицу: здесь видно, что анализ идет по конкретным Begin/End/Unwind RVA, секциям, флагам, handler и ошибкам разбора.

Колонка	Смысл
#	Индекс записи в массиве RUNTIME_FUNCTION.
Begin / End	RVA начала и конца функции. End должен быть больше Begin.
Size	Размер диапазона функции в байтах.
Секция кода	Секция, куда попадает BeginAddress. Обычно ожидается executable code.
Unwind RVA	RVA структуры UNWIND_INFO для этой функции.
Unwind section	Секция, где лежит UNWIND_INFO. Обычно это .xdata или read-only metadata.
Status	OK или Ошибка. Ошибка означает, что UNWIND_INFO не удалось корректно разобрать.
Version / flags	Версия UNWIND_INFO и флаги: EHANDLER, UHANDLER, CHAININFO и т.д.
Stack	Оценка фиксированного размера стека по unwind-кодам.
Handler	RVA handler, если он есть.
Аномалии / ошибка	Краткое объяснение, что именно не так в этой записи.

Практический порядок чтения таблицы: сначала отфильтровать строки со Status = Ошибка, затем посмотреть Unwind RVA и Unwind section, затем проверить handler, затем связать строку с findings выше. Если finding говорит про unwind_rva_oob, именно здесь можно показать конкретную запись, у которой Unwind RVA не сопоставляется с file offset или выходит за допустимую область.

17.11. Как сделать итоговый вывод по отчету

Итоговый вывод нельзя делать по одной карточке. Правильная оценка строится так:

- Проверить режим запуска по командной строке: только статика, только динамика или смешанный режим.
- Посмотреть верхний dashboard: score, verdict, статика/динамика, статус динамики.
- Проверить PE-координаты: ImageBase, SizeOfImage, Exception Directory, alignment.
- Проверить структурную целостность: OOB, пересечения, misalignment.
- Разобрать статические findings: severity, code, family, evidence block.
- Если включена динамика, разобрать runtime counters, events и журнал.
- Открыть таблицу RUNTIME_FUNCTION и связать findings с конкретными функциями.

- Вернуться к Итоговой оценке и сформулировать вывод: что доказано статикой, что подтверждено динамикой, где остаются ограничения.

Грамотная формулировка результата должна звучать не как 'программа сказала 50 / 100', а так: 'В образе обнаружены статические нарушения unwind metadata такого-то семейства, они затрагивают такие-то записи, динамический запуск дал такие-то runtime-сигналы, корреляция слоев такая-то, поэтому итоговая подозрительность умеренная/высокая/низкая с такой-то уверенностью'.

Раздел 18. Динамические сигналы проекта

Этот раздел добавляется в конец документа 'ГЛУБОКОЕ ОПИСАНИЕ ПРОЕКТА'. Его задача - разобрать динамическую часть так же подробно, как ранее были разобраны статические признаки. В статике проект читает PE-файл и проверяет структуру .pdata/.xdata/UNWIND_INFO без запуска программы. В динамике проект запускает анализируемый бинарник под Windows Debug API и наблюдает, какие exception/unwind-механизмы реально используются во время выполнения.

Важно: динамические события не являются тем же самым, что статические findings. Статический finding обычно означает нарушение или странность в структуре файла. Динамический сигнал означает наблюдение во время выполнения. Он может быть нормальным, нейтральным или подозрительным в зависимости от контекста. Поэтому динамика особенно ценна не как отдельная 'магическая оценка', а как runtime evidence - подтверждение или уточнение того, что статический анализ увидел в PE-структурах.

18.1. Почему динамических типов именно 12

В проекте есть 12 типов событий, которые могут попасть в таблицу динамических событий HTML/JSON-отчета. Они задаются строковым полем DynamicEvent.type и добавляются через dynamic_event_add в src/dynamic_win.c. Эти 12 типов отражают не произвольный список, а реальные точки наблюдения, которые доступны отладчику: исключение, завершение процесса, тайм-аут, попадание в handler из UNWIND_INFO, регистрация и фактическое срабатывание VEH/VCH/UEF, а также регистрация runtime function tables через системные API.

• exception - процесс сгенерировал исключение, не являющееся служебным breakpoint/single-step событием анализатора.

• process_exit - целевой процесс завершился, и можно зафиксировать exit code.

• timeout - динамическая сессия достигла лимита времени --timeout-ms.

• unwind_handler_hit - исполнение дошло до handler, адрес которого был получен из UNWIND_INFO.

• veh_registration - процесс вызвал AddVectoredExceptionHandler и зарегистрировал VEH.

• veh_handler_hit - зарегистрированный VEH реально получил управление.

• vch_registration - процесс вызвал AddVectoredContinueHandler и зарегистрировал VCH.

• vch_handler_hit - зарегистрированный VCH реально получил управление.

• uef_registration - процесс вызвал SetUnhandledExceptionFilter и установил top-level filter.

• uef_handler_hit - установленный UnhandledExceptionFilter реально получил управление.

• rtl_add_function_table - процесс вызвал RtlAddFunctionTable и добавил runtime function table.

• rtl_install_function_table_callback - процесс вызвал RtlInstallFunctionTableCallback и установил callback-based таблицу функций.

В отчете также есть верхняя карточка 'Динамические сигналы'. Она не обязана равняться количеству строк в таблице событий. В report.c функция dynamic_signal_count считает укрупненные группы: handler hits, VEH, VCH, UEF, runtime function table activity и second-chance exception. Поэтому один бинарник может иметь несколько строк событий, но меньшее число укрупненных сигналов. Это сделано специально: верхний блок дает краткую картину, а таблица событий показывает подробную хронологию.

18.2. Где это находится в исходниках

Часть	Файл и строки	Назначение
Модель данных динамики	include/dynamic.h:28-90	Описывает DynamicEvent и DynamicReport: тип события, модуль, поток, excep
Сбор динамики	src/dynamic_win.c:665-1144	Запускает процесс под Debug API, ставит breakpoints на API/handlers, обраба
Тайм-аут	src/dynamic_win.c:768-771	Фиксирует timeout как событие и завершает динамическую сессию без ложно
Исключения	src/dynamic_win.c:834-849	Отделяет реальные исключения от служебных breakpoint/single-step событий,
Регистрации и hits	src/dynamic_win.c:906-1024	Снимает аргументы API из регистров x64 calling convention и ставит breakpoin
Русские расшифровки	src/report.c:1384-1414	Переводит type события в русское название и объяснение смысла.
Сводная таблица динамики	src/report.c:1434-1518	Формирует блок 'Расшифровка динамических сигналов' с правилом, наблюде
Карточки динамики	src/report.c:1818-1875	Показывает exceptions, handler hits, runtime table calls, качество instrumentatio
Уровень runtime evidence	src/report.c:489-535	Считает укрупненные сигналы и уровень доказательности: none, weak, mediu

18.3. Как динамический анализатор работает пошагово

• Шаг 1. Пользователь запускает анализатор с флагом --dynamic. Без этого флага проект выполняет только статический анализ и не имеет runtime evidence.

• Шаг 2. До запуска динамики проект уже выполнил статический разбор PE и UNWIND_INFO. Это нужно, чтобы знать, какие handler RVA существуют в UNWIND_INFO и на какие адреса можно попытаться поставить breakpoints.

• Шаг 3. dynamic_collect_windows запускает целевой EXE под Windows Debug API. Это не эмулятор и не дизассемблер: программа реально стартует, а анализатор выступает как отладчик-наблюдатель.

• Шаг 4. При CREATE_PROCESS_DEBUG_EVENT анализатор получает base address главного модуля. Это позволяет переводить handler RVA из UNWIND_INFO в runtime VA: VA = base_main + handler_rva.

• Шаг 5. Анализатор обновляет список загруженных модулей и пытается поставить breakpoints на интересующие системные API: AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback.

• Шаг 6. На найденные handlers из UNWIND_INFO тоже ставятся breakpoints. Это позволяет проверить не только то, что handler записан в .xdata, но и то, дошло ли до него исполнение.

• Шаг 7. В debug loop анализатор принимает события: загрузка DLL, выгрузка DLL, исключения, breakpoints, single-step, завершение процесса. Часть событий нужна только для обслуживания instrumentation, а часть превращается в DynamicEvent.

• Шаг 8. Когда breakpoint срабатывает на API, анализатор читает регистры потока. На Windows x64 первые четыре аргумента функции обычно идут в RCX, RDX, R8, R9. Поэтому project может записать handler pointer, table pointer, count, base address, callback address.

• Шаг 9. Если API зарегистрировал handler, анализатор ставит дополнительный breakpoint уже на сам handler. Так проект различает 'handler был зарегистрирован' и 'handler реально получил управление'.

• Шаг 10. После попадания в handler анализатор может записать небольшой trace RIP-адресов. Это помогает понять, куда управление пошло после точки обработки исключения.

• Шаг 11. В конце динамики отчет получает не только события, но и качество сессии: была ли instrumentation complete, были ли ошибки установки breakpoint, были ли потерянные события, был ли timeout.

• Шаг 12. HTML-отчет показывает верхние карточки, агрегированную расшифровку сигналов, подробную таблицу событий и журнал выполнения. Читать нужно именно вместе: карточки дают обзор, таблица событий дает факты, журнал объясняет технический ход сессии.

18.4. Почему это не отдельный 'динамический антивирусный verdict'

Динамическая часть проекта не говорит: 'нашел VEH - значит вредонос'. Это было бы неправильно. Windows и нормальные программы могут легально использовать exception handlers и runtime function tables. Задача проекта другая: показать, что во время выполнения программа действительно использует механизмы, которые влияют на exception dispatch, unwind, stack walking или runtime function table. Такой факт особенно важен, когда статический анализ уже нашел странности в .pdata/.xdata/UNWIND_INFO.

В report.c уровень runtime evidence строится осторожно. Фактическое попадание в handler считается сильнее, чем просто регистрация handler. Регистрация runtime function table считается средним сигналом, потому что сама по себе может быть нормальной для JIT/runtime, но она показывает, что статическая .pdata уже не обязательно полная картина. First-chance exception или простая регистрация handler - слабые сигналы, которые требуют контекста. Если динамическая сессия неполная, проект понижает уверенность: неполная instrumentation не должна давать слишком сильный вывод.

18.5. Подробный разбор всех 12 динамических событий

1. exception - исключение в процессе

Где в коде: src/dynamic_win.c:834-849; src/report.c:1386, 1403, 1858-1872

Официальная основа: Microsoft Debug API передает отладчику EXCEPTION_DEBUG_EVENT. Документация по debugger exception handling различает first-chance уведомление до обычных handlers и second-chance уведомление, когда система не нашла обработчик.

Как проект получает сигнал: Когда debug loop получает EXCEPTION_DEBUG_EVENT, код проверяет ExceptionCode. Служебные EXCEPTION_BREAKPOINT и EXCEPTION_SINGLE_STEP не считаются обычными исключениями, потому что они используются самим анализатором для breakpoints и single-step. Все остальные exception codes увеличивают exceptions_seen, first_chance_seen или second_chance_seen, а затем добавляются как DynamicEvent type=exception.

Какие данные попадают в отчет: thread_id показывает поток, где произошло исключение; exception_code показывает код исключения; first_chance показывает стадию; address показывает ExceptionAddress; arg0-arg3 для этого события не несут отдельной нагрузки и обычно равны нулю.

Логический смысл для анализа: Это базовая точка runtime-картины. Если исключений нет, то многие handler registrations остаются только потенциальной возможностью. Если исключения есть, можно смотреть, какие handlers получили управление и была ли обработка на first или second chance.

Когда это может быть нормально: Нормальные программы могут генерировать исключения для обработки ошибок, проверки CPU features, lazy initialization, structured exception handling или runtime diagnostics.

Когда это становится подозрительным: Подозрительность растет, если исключения идут вместе с нестандартными unwind handlers, runtime function tables, handler hits, необычными секциями или статическими findings по .pdata/.xdata. Second-chance исключение сильнее first-chance, потому что оно означает, что обычная обработка не закрыла исключение раньше.

Как это влияет на вывод: Само событие exception не является финальным обвинением. Оно становится важным доказательством, когда показывает реальный маршрут exception dispatch и подтверждает, что найденные статикой handlers действительно могут быть задействованы.

2. process_exit - завершение целевого процесса

Где в коде: src/dynamic_win.c:823-830; src/report.c:1396, 1413, 1858-1872

Официальная основа: Windows Debug API передает EXIT_PROCESS_DEBUG_EVENT, когда отлаживаемый процесс завершается.

Как проект получает сигнал: Когда debug loop получает EXIT_PROCESS_DEBUG_EVENT, проект записывает target_exited=true, сохраняет target_exit_code и добавляет DynamicEvent type=process_exit. После ContinueDebugEvent цикл завершает работу.

Какие данные попадают в отчет: arg0 содержит exit code; thread_id показывает поток debug event; address и exception_code обычно нулевые, потому что это не исключение и не handler address.

Логический смысл для анализа: Это сигнал качества динамической сессии. Он говорит, что процесс дошел до завершения, а анализатор не оборвал его тайм-аутом или внутренней ошибкой instrumentation.

Когда это может быть нормально: Нормальный exit code 0 часто означает штатное завершение. Ненулевой код не всегда плохой: программа могла завершиться с ожидаемой ошибкой из-за отсутствия аргументов, прав, файла конфигурации или среды.

Когда это становится подозрительным: Подозрительность появляется не от самого process_exit, а от сочетания: процесс завершился слишком рано, не успел пройти нужный код, дал crash-like код, либо перед завершением были second-chance exceptions, handler hits или runtime table activity.

Как это влияет на вывод: process_exit помогает понять полноту проверки. Если процесс завершился штатно и instrumentation complete, отсутствие дополнительных dynamic events можно воспринимать надежнее, чем при timeout.

3. timeout - тайм-аут динамической сессии

Где в коде: src/dynamic_win.c:768-771; src/report.c:1397, 1414, 1434-1518

Официальная основа: Это не отдельный Windows exception API, а внутреннее ограничение анализатора. Пользователь задает лимит через --timeout-ms, чтобы debug session не зависала бесконечно.

Как проект получает сигнал: В начале debug loop сохраняется стартовое время. На каждой итерации считается elapsed. Если elapsed больше timeout_ms, проект пишет в журнал '[Dynamic] timeout reached', добавляет DynamicEvent type=timeout, выставляет timed_out=true и выходит из цикла.

Какие данные попадают в отчет: arg0 содержит лимит времени в миллисекундах; остальные поля обычно нулевые. В блоке качества сессии отчет показывает timed_out=да.

Логический смысл для анализа: Это сигнал не о поведении вредоносности, а о надежности наблюдения. Он говорит: динамика была запущена, что-то могло быть найдено, но отсутствие дополнительных событий нельзя считать доказательством их отсутствия.

Когда это может быть нормально: Timeout нормален для программ, которые ждут ввода, работают как сервис, открывают GUI, ждут сеть, специально спят или просто выполняются дольше выбранного лимита.

Когда это становится подозрительным: Подозрительность может расти, если timeout сочетается с признаками anti-debug, бесконечным ожиданием, попытками скрыть handler path или неполной instrumentation. Но сам timeout - это прежде всего ограничение покрытия.

Как это влияет на вывод: Проект не должен завышать уверенность при timeout. Поэтому динамическая интерпретация должна быть осторожной: найденные события полезны, но ненайденные события не являются доказанно отсутствующими.

4. unwind_handler_hit - попадание в handler из UNWIND_INFO

Где в коде: src/dynamic_win.c:740-746, 792-811, 985-995; src/report.c:1387, 1404, 1479-1493

Официальная основа: Microsoft x64 exception handling описывает, что UNWIND_INFO может иметь flags UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER; после unwind codes хранится image-relative handler address. Этот handler участвует в обработке exception/unwind.

Как проект получает сигнал: Сначала статический анализ извлекает handler RVA из UNWIND_INFO. В динамике, когда становится известен base_main, анализатор вычисляет runtime address как base_main + handler_rva и ставит breakpoint. Если этот breakpoint срабатывает, добавляется DynamicEvent type=unwind_handler_hit.

Какие данные попадают в отчет: address показывает фактический VA handler во время выполнения; module помогает понять, в каком модуле произошло попадание; thread_id показывает поток; arg0-arg3 обычно нулевые, потому что событие означает сам факт попадания.

Логический смысл для анализа: Это один из самых сильных динамических сигналов проекта. Статика может сказать: 'в UNWIND_INFO записан handler'. Динамика говорит сильнее: 'исполнение реально дошло до handler'.

Когда это может быть нормально: Нормально для C++ exceptions, SEH-путей, cleanup/finally logic, нормального exception handling и некоторых runtime-библиотек.

Когда это становится подозрительным: Подозрительно, если handler адрес статически выглядел странно: лежал в неожиданной секции, был связан с writable/executable metadata, пересекался с другими аномалиями, использовался многими функциями или вел в нетипичный dispatcher/protection-like код.

Как это влияет на вывод: В runtime_evidence_level попадание в handler относится к сильным наблюдениям. В статико-динамической корреляции оно особенно важно, если статический слой уже нашел handler_nonexec_section, handler_writable_section или handler dominance.

5. veh_registration - регистрация vectored exception handler

Где в коде: src/dynamic_win.c:906-917; src/report.c:1388, 1405, 1464-1477

Официальная основа: Microsoft AddVectoredExceptionHandler регистрирует vectored exception handler. Параметр First задает порядок вызова, а Handler указывает функцию обработчика.

Как проект получает сигнал: Проект ставит breakpoint на AddVectoredExceptionHandler. Когда breakpoint срабатывает, анализатор читает RCX как First и RDX как Handler по x64 calling convention. Затем добавляет DynamicEvent type=veh_registration и, если handler не нулевой, ставит breakpoint на сам handler.

Какие данные попадают в отчет: arg0 = First; arg1 = handler pointer; address = адрес API-вызова/точка breakpoint; module показывает модуль, в котором находилась точка события.

Логический смысл для анализа: VEH находится выше обычных frame-based handlers в маршруте обработки исключений и может видеть исключение до того, как оно дойдет до стандартных обработчиков. Поэтому регистрация VEH показывает наличие runtime-слоя обработки исключений, который не описан только .pdata/.xdata.

Когда это может быть нормально: Нормально для crash reporters, защитных runtime-библиотек, sandboxing, compatibility layers, debuggers, profilers, некоторых JIT/runtime-систем.

Когда это становится подозрительным: Подозрительно, если VEH регистрируется рано, указывает в нетипичную/динамическую/записываемую область, затем реально получает управление, либо сочетается со статическими unwind anomalies. Сам факт регистрации без hit - слабее, чем фактическое попадание.

Как это влияет на вывод: В runtime_evidence_level регистрация VEH обычно слабый сигнал. Она показывает потенциальный route, но не доказывает, что исключение реально было перехвачено этим handler.

6. veh_handler_hit - попадание в VEH handler

Где в коде: src/dynamic_win.c:1000-1008; src/report.c:1389, 1406, 1479-1493

Официальная основа: После регистрации VEH Windows может вызвать handler при обработке исключения. Документация AddVectoredExceptionHandler описывает регистрацию handler; фактический вызов зависит от exception dispatch.

Как проект получает сигнал: После veh_registration проект ставит breakpoint на зарегистрированный handler pointer. Если управление доходит до этой функции, breakpoint срабатывает, увеличивается veh_hits и добавляется DynamicEvent type=veh_handler_hit.

Какие данные попадают в отчет: address показывает адрес VEH handler, thread_id - поток, где handler получил управление. В таблице событий это надо читать вместе с предыдущими exception и veh_registration событиями.

Логический смысл для анализа: Это сильнее, чем регистрация. Регистрация говорит: 'программа поставила обработчик'. Hit говорит: 'обработчик реально участвовал в runtime exception flow'.

Когда это может быть нормально: Нормально для диагностических обработчиков, crash reporters, managed/native runtimes, protection-aware software, которое легально контролирует исключения.

Когда это становится подозрительным: Подозрительно, если handler используется как dispatcher, скрывает control flow, работает вместе с искусственно генерируемыми исключениями, получает управление после статически подозрительных unwind structures или указывает в область с сомнительными правами/происхождением.

Как это влияет на вывод: Попадание в VEH входит в группу handler hits и усиливает runtime evidence. При совпадении со статическими findings оно превращается из изолированного наблюдения в корреляционное доказательство.

7. vch_registration - регистрация vectored continue handler

Где в коде: src/dynamic_win.c:922-933; src/report.c:1390, 1407, 1464-1477

Официальная основа: Microsoft AddVectoredContinueHandler регистрирует vectored continue handler. Он относится к продолжению обработки после exception dispatch и может участвовать в decision path продолжения исполнения.

Как проект получает сигнал: Проект ставит breakpoint на AddVectoredContinueHandler. При срабатывании breakpoint читает RCX как First и RDX как Handler, добавляет DynamicEvent type=vch_registration и ставит breakpoint на handler pointer.

Какие данные попадают в отчет: arg0 = First; arg1 = handler pointer; address = точка API-вызова; module/thread_id дают контекст события.

Логический смысл для анализа: VCH показывает, что программа не только реагирует на исключения, но и может вмешиваться в логику продолжения после исключения. Для reverse engineering это важно, потому что такая логика может менять маршрут исполнения после fault/guard/exception.

Когда это может быть нормально: Нормально для low-level runtime, compatibility code, instrumentation, diagnostics и программ, которые сознательно используют Windows exception machinery.

Когда это становится подозрительным: Подозрительно, если VCH связан с искусственным control flow, anti-debug, exception-driven execution, или если handler pointer находится в необычной области и затем реально получает управление.

Как это влияет на вывод: Регистрация VCH - слабый runtime signal без hit. Она становится сильнее, когда есть vch_handler_hit или correlation со статическими findings.

8. vch_handler_hit - попадание в VCH handler

Где в коде: src/dynamic_win.c:1011-1019; src/report.c:1391, 1408, 1479-1493

Официальная основа: Фактический вызов VCH означает, что зарегистрированный vectored continue handler получил управление в runtime flow.

Как проект получает сигнал: После vch_registration проект ставит breakpoint на handler pointer. Когда breakpoint срабатывает, увеличивается vch_hits, добавляется DynamicEvent type=vch_handler_hit и запускается небольшой trace после handler hit.

Какие данные попадают в отчет: address = адрес VCH handler; thread_id = поток; trace, если включен, показывает следующие RIP после попадания.

Логический смысл для анализа: Это runtime-подтверждение, что VCH не просто зарегистрирован, а реально участвует в обработке/продолжении исполнения.

Когда это может быть нормально: Может быть нормальным в программах с нестандартной диагностикой или низкоуровневым runtime-контролем.

Когда это становится подозрительным: Становится подозрительным, если используется для скрытого перехода управления, exception-based obfuscation, anti-debug или если совпадает с unusual unwind metadata.

Как это влияет на вывод: Входит в сильную группу handler hits. В верхних карточках учитывается вместе с unwind_handler_hits, veh_hits и uef_hits как 'Попадания в handler'.

9. uef_registration - установка top-level UnhandledExceptionFilter

Где в коде: src/dynamic_win.c:936-947; src/report.c:1392, 1409, 1464-1477

Официальная основа: Microsoft SetUnhandledExceptionFilter заменяет top-level unhandled exception filter для текущего процесса. Документация подчеркивает, что filter используется, когда исключение доходит до уровня необработанного исключения; поведение под отладчиком имеет особенности.

Как проект получает сигнал: Проект ставит breakpoint на SetUnhandledExceptionFilter. При срабатывании читает RCX как handler pointer, увеличивает uef_set_calls, добавляет DynamicEvent type=uef_registration и ставит breakpoint на handler pointer.

Какие данные попадают в отчет: arg0 = handler pointer; address = точка вызова SetUnhandledExceptionFilter; module/thread_id дают контекст.

Логический смысл для анализа: UEF - это последняя линия обработки необработанных исключений. Регистрация UEF показывает, что программа управляет top-level реакцией на crash/unhandled exception.

Когда это может быть нормально: Нормально для crash reporters, логгеров аварий, приложений с собственным обработчиком падений, защитных wrappers и продуктов, которые хотят собрать дамп или отчет.

Когда это становится подозрительным: Подозрительно, если UEF используется вместе с искусственными exceptions, anti-debug, скрытым dispatcher, самовосстановлением после fault, либо если handler pointer выглядит нетипично.

Как это влияет на вывод: Регистрация UEF - слабый сигнал. Под отладчиком отсутствие uef_handler_hit нельзя трактовать как гарантированное отсутствие UEF-пути в обычном запуске, потому что debug exception flow отличается от запуска без отладчика.

10. uef_handler_hit - попадание в UnhandledExceptionFilter

Где в коде: src/dynamic_win.c:1022-1030; src/report.c:1393, 1410, 1479-1493

Официальная основа: Если top-level unhandled exception filter реально получает управление, это означает, что исключение дошло до поздней стадии обработки. Документация SetUnhandledExceptionFilter связывает этот путь с необработанными исключениями и особенностями debug/non-debug режима.

Как проект получает сигнал: После uef_registration проект ставит breakpoint на зарегистрированный handler. При попадании увеличивается uef_hits и добавляется DynamicEvent type=uef_handler_hit.

Какие данные попадают в отчет: address = адрес UEF handler; thread_id = поток; nearby events exception/second chance помогают понять, почему handler получил управление.

Логический смысл для анализа: Это сильный runtime-сигнал, потому что показывает не только установку top-level filter, но и реальное прохождение исключения к этому уровню.

Когда это может быть нормально: Может быть нормальным, если программа специально тестирует crash handler или штатно формирует crash report.

Когда это становится подозрительным: Подозрительно, если UEF используется как часть control-flow obfuscation, exception-driven VM/dispatcher, anti-debug или коррелирует с подозрительными статическими handlers.

Как это влияет на вывод: Учитывается как handler hit и усиливает runtime evidence. Но интерпретация должна помнить: запуск под отладчиком меняет exception path, поэтому отсутствие hit не равно доказательству отсутствия UEF-поведения вне отладчика.

11. rtl_add_function_table - динамическая регистрация RUNTIME_FUNCTION table

Где в коде: src/dynamic_win.c:954-965; src/report.c:1394, 1411, 1450-1462

Официальная основа: Microsoft RtlAddFunctionTable добавляет dynamic function table. Документация указывает, что function tables используются на 64-bit Windows для unwind/stack walking, а динамически генерируемый код должен предоставить такую таблицу.

Как проект получает сигнал: Проект ставит breakpoint на RtlAddFunctionTable. При срабатывании читает RCX как table pointer, RDX как entry count, R8 как base address. Затем добавляет DynamicEvent type=rtl_add_function_table.

Какие данные попадают в отчет: arg0 = table pointer; arg1 = count; arg2 = base address; address = точка вызова API.

Логический смысл для анализа: Статическая .pdata описывает function table, находящуюся в файле. RtlAddFunctionTable показывает, что процесс добавляет unwind-карту уже во время выполнения. Поэтому один только статический PE-разбор может не видеть всю runtime-картину.

Когда это может быть нормально: Нормально для JIT-компиляторов, языковых runtime, динамически генерируемого кода, hotpatch/instrumentation механизмов и некоторых защитных библиотек.

Когда это становится подозрительным: Подозрительно, если dynamic function table появляется у программы, где это неожиданно, если base указывает в нетипичную область, count странный, таблица связана с unpacked/generated code, либо статическая .pdata уже имеет anomalies.

Как это влияет на вывод: Это medium runtime signal. Оно не доказывает вредоносность, но существенно меняет модель анализа: проверять нужно не только PE-файл на диске, но и runtime-registered unwind metadata.

12. rtl_install_function_table_callback - callback-based runtime function table

Где в коде: src/dynamic_win.c:971-982; src/report.c:1395, 1412, 1450-1462

Официальная основа: Microsoft RtlInstallFunctionTableCallback устанавливает callback, который может выдавать function table entries для заданного control address. Документация описывает это как механизм для очень динамичного кода.

Как проект получает сигнал: Проект ставит breakpoint на RtlInstallFunctionTableCallback. При срабатывании читает RCX как table identifier, RDX как base address, R8 как length, R9 как callback pointer. Затем добавляет DynamicEvent type=rtl_install_function_table_callback.

Какие данные попадают в отчет: arg0 = table identifier; arg1 = base address; arg2 = length; arg3 = callback pointer.

Логический смысл для анализа: Это еще более динамичный вариант, чем RtlAddFunctionTable: таблица может не лежать готовым массивом, а вычисляться callback-функцией. Для reverse engineering это важно, потому что статический файл может вообще не содержать полной

unwind-карты.

Когда это может быть нормально: Нормально для JIT, runtime-систем, профилировщиков, instrumentation frameworks и кода, который генерирует функции на лету.

Когда это становится подозрительным: Подозрительно, если callback используется для сокрытия реальной карты функций, для защиты/обфускации, если base/length описывают странную область памяти, или если callback address находится в неожиданном модуле/секции.

Как это влияет на вывод: Это medium runtime signal. В сочетании с плохой статикой по .pdata/.xdata оно становится важным аргументом: программа не только имеет подозрительные metadata на диске, но и меняет unwind model во время выполнения.

18.6. Как правильно читать динамическую часть HTML-отчета

• Карточка 'Исключения' показывает, сколько обычных exception events было замечено. Breakpoint и single-step, которые нужны самому анализатору, сюда не входят.

• Карточка 'Попадания в handler' суммирует unwind_handler_hits, veh_hits, vch_hits и uef_hits. Это более сильная группа, чем registrations, потому что handler реально получил управление.

• Карточка 'Вызовы runtime-таблиц' суммирует RtlAddFunctionTable и RtlInstallFunctionTableCallback. Это не всегда плохо, но означает, что runtime unwind map может отличаться от статической .pdata.

• Карточка 'Потерянные события' показывает dropped_events. Если она не нулевая, таблица событий может быть неполной из-за лимита DYNAMIC_MAX_EVENTS.

• Карточка 'Инструментация завершена' показывает, смог ли анализатор нормально поставить/переустановить breakpoints и пройти сессию без внутренних проблем.

• Карточка 'Handlers найдены / включены' показывает, сколько handler RVA было найдено статически и на сколько удалось поставить breakpoints. Если найдено больше, чем включено, часть handler path могла остаться без динамического наблюдения.

• Карточка 'Ошибки breakpoint' показывает проблемы установки breakpoints. Это влияет не на сам бинарник, а на надежность динамического наблюдения.

• Карточка 'Покрытие модулей' показывает, насколько хорошо анализатор видел загруженные модули. Ошибки или усечения списка модулей ухудшают интерпретацию адресов.

• Блок 'Расшифровка динамических сигналов' группирует события по смыслу и объясняет, что найдено, что проверяется и как интерпретировать.

• Таблица событий - самая точная часть. В ней нужно смотреть type, русское название, module, thread, exception, chance, address, arg0-arg3 и смысл события. Именно там видна хронология фактов.

• Журнал выполнения полезен для технической проверки: по нему видно, какие API были resolved, где ставились breakpoints, какие self-check прошли, был ли timeout и почему сессия завершилась.

18.7. Как динамика влияет на итоговую оценку

Проект не должен ставить один общий знак равенства между динамическим событием и вредоносностью. Динамика отвечает на другой вопрос: 'наблюдалось ли во время выполнения поведение, связанное с exception/unwind механизмами?' Поэтому итоговая логика строится через силу evidence и корреляцию со статикой.

• Weak evidence: first-chance exception, регистрация VEH/VCH/UEF без фактического hit. Это говорит о потенциальном runtime path, но не доказывает его использование.

• Medium evidence: second-chance exception или регистрация runtime function table. Это важнее простой регистрации handler, потому что влияет на позднюю обработку исключений или на runtime unwind map.

• Strong evidence: фактическое попадание в unwind/VEH/VCH/UEF handler. Это прямое наблюдение control flow в handler path.

• Correlation: если статика показывает странный handler placement, handler dominance, writable/executable metadata или ошибки UNWIND_INFO, а динамика показывает handler hit или runtime function table activity, вывод становится сильнее.

• Session quality: если есть timeout, dropped_events, breakpoint failures или неполная instrumentation, вывод надо ослаблять. Неполная динамика не имеет права звучать как полное доказательство

отсутствия поведения.

18.8. Границы применимости динамических сигналов

• Динамика видит только тот путь выполнения, который реально прошел в данном запуске. Если программа требует аргументы, файл, сеть, лицензию, GUI-действие или особое окружение, часть behavior может не проявиться.

• Запуск под отладчиком может менять поведение программы. Некоторые программы проверяют наличие отладчика, а exception flow под debug mode имеет особенности.

• Отсутствие handler hit не означает, что handler никогда не будет вызван. Оно означает только: в данном запуске и при данном покрытии анализатор не увидел попадание.

• Регистрация handler не означает вредоносность. Она означает наличие runtime exception-handling layer, который нужно интерпретировать вместе со статикой.

• Runtime function table activity не означает вредоносность. Она нормальна для JIT/runtime, но критична для reverse engineering, потому что статическая .pdata может быть неполной картой функций.

• Timeout и instrumentation failures говорят о качестве эксперимента, а не о свойствах бинарника. Их нельзя смешивать с подозрительными признаками PE.

18.9. Нормативная база Microsoft

Для защиты важно говорить, что динамические сигналы выбраны не 'на глаз', а из официальной модели Windows exception/unwind/runtime function table. Основные первичные источники:

• Microsoft PE Format: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

• Microsoft x64 exception handling: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64

• Debugger Exception Handling: https://learn.microsoft.com/en-us/windows/win32/debug/debugger-exception-handling

• AddVectoredExceptionHandler: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredexceptionhandler

• AddVectoredContinueHandler: https://learn.microsoft.com/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredcontinuehandler

• SetUnhandledExceptionFilter: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-setunhandledexceptionfilter

• RtlAddFunctionTable: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable

• RtlInstallFunctionTableCallback: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback

Итоговая формулировка для комиссии: статические признаки показывают, как устроен PE/unwind metadata на диске; динамические сигналы показывают, какие exception/unwind/runtime-table механизмы реально проявились во время выполнения. Именно сочетание этих двух слоев делает проект не простым набором if-else, а инструментом evidence-based анализа Windows x64 exception/unwind поведения.

Раздел 19. Как читать весь HTML-отчет: каждый блок и элемент

Этот раздел нужен как практический справочник по готовому HTML-отчету EH/Unwind Analyzer. Его задача - объяснить не только динамику, не только статические findings, а весь отчет целиком: каждую карточку, каждую таблицу, каждую строку, каждую колонку и каждый служебный элемент, который пользователь видит в браузере.

Правильная модель чтения отчета такая: верхняя часть дает быстрый обзор; средняя часть раскрывает доказательства; нижняя таблица RUNTIME_FUNCTION показывает первичные технические данные; блоки статических и динамических признаков объясняют, почему инструмент сделал вывод. Нельзя читать только одну карточку 'оценка 50 / 100' и делать вывод. Нужно смотреть цепочку: PE layout -> structural checks -> sections -> findings -> evidence block -> dynamic evidence -> RUNTIME_FUNCTION rows.

19.1. Общая карта HTML-отчета

HTML-отчет строится в src/report.c внутри функции генерации HTML. Он состоит из последовательных экранных блоков. Каждый блок отвечает за отдельный слой анализа. Если объяснять совсем просто: сначала отчет говорит 'что за файл проверяли', потом 'как устроен PE-образ', потом 'какие нарушения найдены', потом 'что происходило в динамике', потом 'какая таблица RUNTIME_FUNCTION лежит в основе всего вывода'.

Порядок	Блок отчета	Зачем нужен
1	Верхняя шапка EH/Unwind Analyzer	Быстрый обзор: путь к файлу, количество записей, ошибки, score, итог, режим анализа, уверенность.
2	Метаданные запуска	Доказывает воспроизводимость: дата, версия, schema, SHA-256, команда, параметры анализа, размер файла.
3	Образ PE	Показывает координатную систему файла: ImageBase, SizeOfImage, SizeOfHeaders, Exception Directory, alignment.
4	Структурная проверка	Показывает, не сломан ли базовый PE layout: OOB, пересечения, выравнивание, Exception Directory OOB.
5	Секции	Показывает все секции, их RVA/raw ranges и права R/W/X.
6	Итоговая оценка	Детальный итог: verdict, сила доказательств, уверенность, статико-динамическая корреляция, план ручной проверки.
7	Статические признаки	Основной список findings, severity, code, family, affected count, RVA, section, evidence block.
8	Правила снижения уверенности	Объясняет, почему score/confidence может быть смягчен, если есть нормальные объяснения.
9	Детали анализа	Короткий машинно-читаемый список найденных статических признаков.
10	Динамическая проверка	Карточки runtime-наблюдений, расшифровка dynamic signals, таблица событий, журнал выполнения.
11	Таблица RUNTIME_FUNCTION	Сырые записи function table: begin/end/unwind RVA, status, flags, stack, handler, anomalies.
12	Фильтр таблицы	Интерактивный поиск по RUNTIME_FUNCTION rows.
13	Footer	Служебная отметка, какой инструмент сформировал отчет.

19.2. Цвета, карточки и визуальный язык отчета

Перед чтением отдельных полей нужно понимать визуальный язык отчета. Он не декоративный: цвет и тип блока помогают быстро оценить состояние, но не заменяют технический анализ. Карточка - это краткое число или статус. Таблица - это доказательная база. Pre-блок журнала - это техническая хронология работы инструмента.

Зеленый / ok

Что означает: Поле или счетчик выглядит штатно для данного режима. Например, ноль ошибок breakpoint или завершенная instrumentation.

Откуда берется: CSS-класс ok в HTML-отчете.

Нормальное чтение: Можно читать дальше без немедленного подозрения.

Когда насторожиться: Зеленый цвет не означает, что весь файл безопасен. Он относится только к конкретному элементу.

Что делать дальше: Не делать вывод по одному зеленому полю, а проверять соседние блоки.

Желтый / warn

Что означает: Есть предупреждение или ненулевой счетчик, который требует внимания, но не всегда означает ошибку или вредоносность.

Откуда берется: CSS-класс warn, часто ставится при anomalies, findings или неполном покрытии.

Нормальное чтение: Желтый значит: 'посмотри подробнее'.

Когда насторожиться: Если желтых элементов много и они коррелируют между собой, это усиливает подозрительность.

Что делать дальше: Открыть соответствующую таблицу и посмотреть конкретные строки.

Красный / danger

Что означает: Высокая серьезность, ошибка разбора, высокий score или критическая проблема.

Откуда берется: CSS-класс danger.

Нормальное чтение: Для нормального файла красных элементов обычно быть не должно.

Когда насторожиться: Красный элемент требует объяснения: повреждение файла, packed/protected sample, ошибка входного файла или реальная аномалия.

Что делать дальше: Смотреть evidence block: Rule, Observed, Expected, Impact.

Синий / neutral

Что означает: Нейтральное состояние: режим не запускался, данных нет, динамика не запрашивалась.

Откуда берется: CSS-класс neutral.

Нормальное чтение: Нейтральное поле не хорошо и не плохо, оно говорит о режиме анализа.

Когда насторожиться: Ошибка будет, если принять 'не запускалась' за 'ничего не найдено'.

Что делать дальше: Проверить команду запуска и флаги --heuristics / --dynamic.

19.3. Верхняя шапка отчета

Верхняя шапка - это первый экран HTML-отчета. Ее задача - дать быстрый ответ: какой файл проверяли, сколько структур удалось разобрать, есть ли явные проблемы, какая общая оценка и был ли динамический запуск. Это не полный анализ. Это навигационная панель для быстрого понимания картины.

Заголовок EH/Unwind Analyzer

Что означает: Название инструмента и начало отчета. Оно показывает, что это отчет именно анализатора exception/unwind metadata, а не общего PE-сканера.

Откуда берется: Жестко записано в HTML-генераторе отчета.

Нормальное чтение: Заголовок просто идентифицирует отчет.

Когда насторожиться: Если отчет открыт не от того инструмента или старой версии, сравни поле 'Инструмент / schema'.

Что делать дальше: Использовать как точку входа, дальше смотреть путь к файлу и metadata.

Путь к анализируемому файлу под заголовком

Что означает: Показывает, какой именно бинарник был передан в анализатор.

Откуда берется: input_path из команды запуска.

Нормальное чтение: Путь должен совпадать с файлом, который ты хотел проверить.

Когда насторожиться: Если путь указывает не на тот EXE, весь вывод относится к другому файлу.

Что делать дальше: Сверить путь, SHA-256 и команду запуска.

Карточка 'Записи RUNTIME_FUNCTION'

Что означает: Количество записей RUNTIME_FUNCTION, прочитанных из Exception Directory. Это число показывает размер статической function table, которую удалось разобрать.

Откуда берется: Переменная n, переданная в HTML report; строки report.c около 1630.

Нормальное чтение: Для x64 non-leaf функций обычно ожидается ненулевая таблица. Маленькие учебные бинарники могут иметь мало записей.

Когда насторожиться: Ноль записей при наличии сложного x64-кода может быть странным; очень большое число само по себе не плохо, но требует контекста.

Что делать дальше: Если число ноль или неожиданное, смотреть Exception Directory, no_runtime_functions, pe_exception_directory_oob и таблицу RUNTIME_FUNCTION.

Карточка 'Ошибки разбора'

Что означает: Сколько UNWIND_INFO записей не удалось разобрать. Это не просто 'ошибка программы', а показатель, что parser дошел до записи, но структура UNWIND_INFO оказалась битой, неполной или недоступной.

Откуда берется: Счетчик parse_failures считается по массиву us_ok.

Нормальное чтение: Ноль ошибок - хорошо: все найденные UNWIND_INFO удалось прочитать.

Когда насторожиться: Ненулевое значение говорит, что часть unwind metadata нельзя надежно интерпретировать.

Что делать дальше: Открыть таблицу RUNTIME_FUNCTION и найти строки со статусом 'Ошибка'.

Карточка 'Аномальные записи'

Что означает: Сколько записей RUNTIME_FUNCTION/UNWIND_INFO имеют anomalies: странные opcodes, flags, stack info, handler data или другие подозрительные свойства.

Откуда берется: Счетчик anomaly_count считается по us[i].anomalies.

Нормальное чтение: Ноль аномальных записей означает, что базовый разбор unwind records выглядит чисто.

Когда насторожиться: Если число больше нуля, надо смотреть не только количество, но и какие именно anomalies в последней колонке таблицы.

Что делать дальше: Перейти к RUNTIME_FUNCTION table и фильтровать по словам 'bad', 'trunc', 'handler', 'chain', 'opcode'.

Карточка 'Эвристические признаки'

Что означает: Количество статических findings, которые сработали в heuristics.c. Это уже не сырой parser status, а результат правил анализа.

Откуда берется: hr->finding_count.

Нормальное чтение: Ноль findings - сильных статических проблем не найдено.

Когда насторожиться: Ненулевое значение означает, что хотя бы одно правило нашло нарушение или подозрительную комбинацию.

Что делать дальше: Перейти в раздел 'Статические признаки' и читать каждую строку с evidence block.

Карточка 'Статическая оценка'

Что означает: Числовой score 0..100 по статическим признакам. Это агрегированная оценка подозрительности на основе PE/unwind structures без учета runtime behavior.

Откуда берется: hr->score, рассчитанный эвристиками.

Нормальное чтение: 0 или низкое значение - мало статических оснований для подозрения.

Когда насторожиться: Высокое значение не значит автоматическую вредоносность; оно значит, что несколько правил дали сильную структурную evidence.

Что делать дальше: Всегда смотреть, какие findings создали score. Число без findings не защищается на комиссии.

Карточка 'Итоговый вывод'

Что означает: Человеческая формулировка результата: например низкая, умеренная или высокая подозрительность. Это итоговая интерпретация, а не отдельное поле PE.

Откуда берется: assessment_summary и ru_verdict_text.

Нормальное чтение: Должна согласовываться со score, findings, dynamic evidence и confidence.

Когда насторожиться: Если итог кажется странным, нужно открыть раздел 'Итоговая оценка' и посмотреть силу доказательств, корреляцию и план ручной проверки.

Что делать дальше: Использовать как краткий вывод для отчета, но защищать через evidence ниже.

Карточка 'Статика / динамика'

Что означает: Показывает силу статического evidence и runtime evidence отдельно. Например 'сильный / слабый'.

Откуда берется: summary.static_evidence_level и summary.runtime_evidence_level.

Нормальное чтение: Полезно, когда запускали смешанный режим: видно, за счет какого слоя возник итог.

Когда насторожиться: Если динамика 'none' или 'слабая', нельзя говорить, что runtime поведение полностью проверено.

Что делать дальше: Сравнить режим запуска, dynamic cards и таблицу событий.

Карточка 'Динамические сигналы'

Что означает: Количество укрупненных runtime-сигналов, а не количество строк динамической таблицы. Считает группы: handler hits, VEH, VCH, UEF, runtime table activity, second-chance exceptions.

Откуда берется: dynamic_signal_count в report.c.

Нормальное чтение: 0 означает, что динамика не дала значимых runtime-групп или не запускалась.

Когда насторожиться: Не путать с количеством DynamicEvent rows. Один тип поведения может создать несколько строк, но одну группу.

Что делать дальше: Если число больше нуля, открыть динамический раздел и смотреть какие именно группы сработали.

Карточка 'Итоговая уверенность'

Что означает: Процент уверенности в итоговом выводе. Это не вероятность вредоносности, а уверенность инструмента в своем verdict с учетом качества evidence.

Откуда берется: summary.final_confidence.

Нормальное чтение: Высокая уверенность возможна при согласованной статике и динамике, низкая - при неполных данных.

Когда насторожиться: Процент может быть снижен из-за неполной динамики, timeout, слабой корреляции или mitigations.

Что делать дальше: На защите говорить: confidence - это надежность вывода, а не 'вероятность вируса'.

Карточка 'Динамическая проверка'

Что означает: Показывает статус runtime-режима: не запрашивалась, завершена, timeout или ошибка.

Откуда берется: ru_status_text(dr).

Нормальное чтение: Если запуск был только статический, 'не запрашивалась' нормально.

Когда насторожиться: Если пользователь ожидал динамику, но видит 'не запрашивалась', значит команда была без --dynamic или запуск не состоялся.

Что делать дальше: Сверить строку 'Команда' и наличие --dynamic.

19.4. Метаданные запуска

Метаданные - это не украшение. Они нужны, чтобы отчет можно было воспроизвести и защитить. На комиссии это отвечает на вопросы: каким инструментом проверяли, какую версию использовали, какой файл был на входе, какими флагами запускали анализ.

Сформировано

Что означает: UTC-время генерации отчета. Показывает, когда был создан результат.

Откуда берется: metadata->generated_at_utc.

Нормальное чтение: Дата должна соответствовать моменту анализа.

Когда насторожиться: Если отчет старый, он мог относиться к предыдущей версии бинарника или инструмента.

Что делать дальше: При сдаче сверить дату с финальным запуском.

Инструмент / schema

Что означает: Версия анализатора и версия схемы отчета. Tool version говорит о версии программы; schema version - о формате JSON/HTML полей.

Откуда берется: metadata->tool_version и metadata->report_schema_version.

Нормальное чтение: Одинаковые версии удобны для сравнения нескольких отчетов.

Когда насторожиться: Если schema отличается между отчетами, поля могли интерпретироваться по-разному.

Что делать дальше: При сравнении результатов указывать версию инструмента.

SHA-256

Что означает: Криптографический хэш проверяемого файла. Он доказывает, что отчет относится к конкретным байтам конкретного бинарника.

Откуда берется: metadata->input_sha256.

Нормальное чтение: Хэш должен совпадать при повторном анализе того же файла.

Когда насторожиться: Если хэш поменялся, это уже другой файл, даже если имя EXE то же самое.

Что делать дальше: Использовать SHA-256 как главный идентификатор образца.

Сборка

Что означает: Показывает платформу, компилятор и режим сборки самого анализатора: например windows-x64 / GCC / release.

Откуда берется: metadata->build_platform, build_compiler, build_mode.

Нормальное чтение: Release-сборка ожидаема для финального использования.

Когда насторожиться: Debug/release различия могут влиять на производительность, но не должны менять смысл анализа.

Что делать дальше: Для сдачи лучше использовать финальную release-сборку.

Параметры анализа

Что означает: Показывает ключевые флаги: strict ABI, follow chain, max depth. Это влияет на разбор chained unwind и строгость интерпретации.

Откуда берется: metadata->strict_abi, follow_chain, max_chain_depth.

Нормальное чтение: Параметры должны соответствовать команде запуска.

Когда насторожиться: Если max depth слишком мал, chained unwind может быть разобран неполно; если режим не strict, часть ABI нарушений может трактоваться мягче.

Что делать дальше: При споре о результате показать параметры анализа.

Размер файла

Что означает: Размер входного файла в байтах. Нужен для проверки raw boundaries и воспроизводимости.

Откуда берется: metadata->input_size.

Нормальное чтение: Должен совпадать с размером файла на диске на момент анализа.

Когда насторожиться: Если размер отличается, отчет относится к другой версии файла.

Что делать дальше: Сверять с SHA-256 и путем.

Команда

Что означает: Полная команда запуска анализатора. Это одно из самых важных полей отчета, потому что оно показывает режим: статика, динамика или смешанный запуск.

Откуда берется: metadata->command_line.

Нормальное чтение: Команда должна содержать нужные флаги: --heuristics, --dynamic, --json, --html, --timeout-ms и путь к бинарнику.

Когда насторожиться: Если в команде нет --dynamic, динамический блок не может быть evidence. Если нет --heuristics, статическая эвристика не будет полноценной.

Что делать дальше: При демонстрации комиссии показывать команду как способ воспроизведения.

19.5. Блок 'Образ PE'

Этот блок задает координатную систему анализа. Без него нельзя правильно объяснить, что такое RVA, VA, raw offset и почему выход за границы считается проблемой. Все последующие проверки опираются на эти поля.

ImageBase

Что означает: Предпочитаемая база загрузки PE-образа. VA во время выполнения часто считается как ImageBase + RVA, если образ загружен по preferred base. При ASLR фактическая база может быть другой, но RVA сохраняет смысл.

Откуда берется: PeImage.image_base из PE Optional Header.

Нормальное чтение: Обычный x64 EXE часто имеет ImageBase вроде 0x140000000.

Когда насторожиться: Сам по себе странный ImageBase не является finding, но важен при объяснении адресов.

Что делать дальше: Использовать для объяснения разницы RVA/VA.

SizeOfImage

Что означает: Размер образа в памяти после загрузки. Это верхняя граница виртуального PE layout.

Откуда берется: PeImage.size_of_image.

Нормальное чтение: Все виртуальные диапазоны секций, Exception Directory, Begin/End RVA и UnwindInfoAddress должны укладываться в SizeOfImage.

Когда насторожиться: Если структуры выходят за SizeOfImage, loader или анализатор не может считать их нормальной частью image.

Что делать дальше: Связывать с findings rf_oob, unwind_rva_oob, pe_section_virtual_oob, pe_exception_directory_oob.

SizeOfHeaders

Что означает: Размер области PE headers в начале образа. Некоторые RVA могут указывать внутрь headers, а не в обычную секцию.

Откуда берется: PeImage.size_of_headers.

Нормальное чтение: Headers должны быть доступны для чтения как часть image.

Когда насторожиться: Если parser игнорирует headers, он может ошибочно считать некоторые RVA unmappable.

Что делать дальше: Использовать при объяснении RVA-to-file-offset logic.

Exception directory

Что означает: Пара RVA + Size из DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]. Для x64 это обычно область .pdata с массивом RUNTIME_FUNCTION.

Откуда берется: img->exception_rva и img->exception_size.

Нормальное чтение: Если RVA и Size ненулевые, область должна быть внутри image и переводиться в читаемый file offset.

Когда насторожиться: Нулевой RVA и Size обычно означает отсутствие каталога; несогласованная пара или OOB - проблема.

Что делать дальше: Смотреть pe_exception_directory_oob, RUNTIME_FUNCTION count и ошибки разбора.

SectionAlignment / FileAlignment

Что означает: Правила выравнивания виртуального layout в памяти и raw layout на диске.

Откуда берется: PE Optional Header.

Нормальное чтение: SectionAlignment обычно больше или равен FileAlignment; секции должны быть согласованы с этими значениями.

Когда насторожиться: Нарушения выравнивания могут означать повреждение, упаковку, ручную модификацию или parser trap.

Что делать дальше: Смотреть pe_section_misaligned и таблицу секций.

19.6. Блок 'Структурная проверка'

Этот блок отвечает на вопрос: можно ли вообще доверять базовой PE-разметке файла. Если здесь есть проблемы, все последующие структуры читаются осторожнее, потому что нарушены фундаментальные границы raw/virtual layout.

Raw-диапазоны вне файла

Что означает: Сколько секций заявляют PointerToRawData + SizeOfRawData за пределами физического размера файла.

Откуда берется: img->section_raw_oob.

Нормальное чтение: Ноль - секции не пытаются читать байты за концом файла.

Когда насторожиться: Ненулевое значение значит, что section table обещает данные, которых на диске нет.

Что делать дальше: Смотреть finding pe_section_raw_oob и конкретную секцию в таблице секций.

Virtual-диапазоны вне image

Что означает: Сколько секций выходят за SizeOfImage в виртуальной раскладке.

Откуда берется: img->section_virtual_oob.

Нормальное чтение: Ноль - virtual ranges укладываются в image.

Когда насторожиться: Выход за SizeOfImage ломает модель загруженного образа.

Что делать дальше: Смотреть pe_section_virtual_oob.

Пересечения raw

Что означает: Сколько raw-диапазонов секций пересекаются на диске.

Откуда берется: img->section_raw_overlaps.

Нормальное чтение: В нормальном PE разные секции обычно не должны перекрывать одни и те же raw bytes.

Когда насторожиться: Пересечения могут быть упаковкой, обфускацией, повреждением или нестандартной ручной сборкой.

Что делать дальше: Смотреть pe_section_raw_overlap и сравнивать raw offset/raw size.

Пересечения virtual

Что означает: Сколько виртуальных диапазонов секций пересекаются в image layout.

Откуда берется: img->section_virtual_overlaps.

Нормальное чтение: Нормальная карта секций не должна создавать неоднозначность, какой секции принадлежит RVA.

Когда насторожиться: Если virtual ranges пересекаются, section lookup становится неоднозначным.

Что делать дальше: Смотреть pe_section_virtual_overlap.

Невыровненные секции

Что означает: Количество нарушений alignment, порядка или согласованности section sizes.

Откуда берется: img->section_misaligned.

Нормальное чтение: Ноль - section table выглядит согласованно.

Когда насторожиться: Ненулевое значение может означать нестандартный/поврежденный PE или ловушку для parser-а.

Что делать дальше: Смотреть pe_section_misaligned.

Exception directory OOB

Что означает: Показывает, выходит ли Exception Directory за границы image/file mapping или не может быть безопасно прочитана.

Откуда берется: img->exception_directory_oob.

Нормальное чтение: Нет - каталог исключений, если заявлен, доступен в допустимой области.

Когда насторожиться: Да - .pdata/RUNTIME_FUNCTION table нельзя считать нормальной и полностью надежной.

Что делать дальше: Смотреть pe_exception_directory_oob, Exception directory RVA/Size и RUNTIME_FUNCTION count.

19.7. Таблица 'Секции'

Каждая строка таблицы секций - это одна IMAGE_SECTION_HEADER из PE. Через эту таблицу отчет связывает адреса findings с реальными областями файла: .text, .rdata, .pdata, .xdata, .data и другими.

Колонка 'Имя'

Что означает: Имя секции. Например .text обычно содержит код, .rdata - read-only data, .pdata - RUNTIME_FUNCTION table, .xdata - UNWIND_INFO.

Откуда берется: IMAGE_SECTION_HEADER.Name.

Нормальное чтение: Имя помогает быстро понять роль секции, но нельзя доверять только имени.

Когда насторожиться: Злоумышленник или packer может назвать секцию как угодно.

Что делать дальше: Сопоставлять имя с правами, virtual/raw ranges и содержимым.

VirtualAddress

Что означает: RVA начала секции в памяти. Это не file offset и не абсолютный VA.

Откуда берется: IMAGE_SECTION_HEADER.VirtualAddress.

Нормальное чтение: RVA секций обычно идут по возрастанию и согласованы с SectionAlignment.

Когда насторожиться: Странный порядок или пересечения могут ломать mapping.

Что делать дальше: Использовать для перевода RVA findings в секцию.

VirtualSize

Что означает: Размер секции в памяти. Может отличаться от raw size, потому что loader может дополнять область нулями.

Откуда берется: IMAGE_SECTION_HEADER.Misc.VirtualSize.

Нормальное чтение: VirtualSize описывает memory layout.

Когда насторожиться: Если RVA попадает в virtual tail без raw bytes, статический parser не должен читать несуществующие байты из файла.

Что делать дальше: Помнить различие virtual range и raw range.

Raw offset

Что означает: Смещение начала секции в файле на диске.

Откуда берется: IMAGE_SECTION_HEADER.PointerToRawData.

Нормальное чтение: Используется для чтения байтов из файла.

Когда насторожиться: Raw offset вне файла или пересекающий другую секцию - structural issue.

Что делать дальше: Связывать с raw OOB/overlap findings.

Raw size

Что означает: Сколько байтов секции реально лежит в файле.

Откуда берется: IMAGE_SECTION_HEADER.SizeOfRawData.

Нормальное чтение: Для чтения из файла parser использует raw size, а не только virtual size.

Когда насторожиться: Raw size 0 при ненулевом VirtualSize может быть нормальным для .bss, но нельзя читать bytes из файла.

Что делать дальше: Объяснять zero-fill и отличие диска от памяти.

Права

Что означает: Комбинация R/W/X: readable, writable, executable. Показывает memory protection role секции.

Откуда берется: IMAGE_SECTION_HEADER.Characteristics.

Нормальное чтение: Код обычно R-X, read-only metadata обычно R--, изменяемые данные RW-.

Когда насторожиться: Unwind metadata в executable или writable секции не всегда вредоносна, но требует проверки.

Что делать дальше: Связывать с unwind_in_exec_section, unwind_writable_section, handler_writable_section.

19.8. Блок 'Итоговая оценка'

Этот блок частично повторяет верхнюю шапку, и это нормально. Верхняя шапка нужна для быстрого обзора сразу при открытии отчета. Блок 'Итоговая оценка' нужен для защиты вывода: он подробнее показывает силу доказательств, корреляцию слоев и план ручной проверки. Это не бессмысленный дубляж, а два уровня представления: summary и reasoning.

Вывод

Что означает: Итоговая словесная оценка подозрительности. Это удобная человеческая формулировка результата.

Откуда берется: summary.final_verdict.

Нормальное чтение: Должна совпадать с общим смыслом findings и runtime evidence.

Когда насторожиться: Если вывод высокий, а findings слабые, надо проверить scoring/correlation.

Что делать дальше: Использовать как короткий ответ, но подкреплять findings ниже.

Сила доказательств

Что означает: Оценивает не просто количество признаков, а насколько они убедительны и согласованы.

Откуда берется: summary.strength и ru_case_strength_text.

Нормальное чтение: Сильная evidence возникает при серьезных findings или корреляции статики и динамики.

Когда насторожиться: Слабая evidence при высоком score требует объяснения, почему уверенность не такая высокая.

Что делать дальше: Защищать через Rule/Observed/Expected/Impact.

Итоговая уверенность

Что означает: Процент надежности verdict. Это не процент вредоносности.

Откуда берется: summary.final_confidence.

Нормальное чтение: Высокая уверенность означает, что данные полные и согласованные.

Когда насторожиться: Timeout, неполная instrumentation или mitigations должны снижать уверенность.

Что делать дальше: На комиссии формулировать: 'уверенность в выводе анализа', не 'вероятность вируса'.

Статика / динамика

Что означает: Показывает отдельно, какой вклад дали статический и runtime-слой.

Откуда берется: summary.static_evidence_level и summary.runtime_evidence_level.

Нормальное чтение: Позволяет честно сказать, на чем основан вывод.

Когда насторожиться: Если динамика слабая, нельзя заявлять сильное runtime-подтверждение.

Что делать дальше: Сравнить с режимом запуска и dynamic section.

Корреляция слоев

Что означает: Показывает, подтвердили ли динамические наблюдения статические странности или усилили их.

Откуда берется: summary.correlated.

Нормальное чтение: Да - сильный аргумент: разные слои указывают на связанную проблему.

Когда насторожиться: Нет - не ошибка, если запуск был только статический или динамика не проявила нужный path.

Что делать дальше: Если корреляция да, показать конкретные static findings и dynamic events.

Динамические сигналы

Что означает: Укрупненное число runtime-групп, как и в верхней шапке.

Откуда берется: summary.dynamic_signals.

Нормальное чтение: Помогает понять, есть ли runtime evidence.

Когда насторожиться: Не равно числу строк в event table.

Что делать дальше: Открыть динамическую таблицу для деталей.

План ручной проверки

Что означает: Список следующих действий для аналитика: что открыть в PE-viewer/Ghidra/debugger и что проверить руками.

Откуда берется: collect_manual_validation_steps.

Нормальное чтение: План появляется, когда есть findings или runtime evidence, требующие подтверждения.

Когда насторожиться: Если план есть, его нельзя игнорировать: автоматический анализ не заменяет reverse engineering.

Что делать дальше: Использовать план как шпаргалку для защиты и ручной верификации.

19.9. Раздел 'Статические признаки'

Это один из главных разделов отчета. Каждая строка таблицы - это finding, то есть сработавшее правило анализа. Важно читать строку целиком, а не только код finding. Код говорит 'что за тип проблемы', но evidence block объясняет 'почему инструмент так решил'.

Сообщение 'Эвристические признаки не обнаружены'

Что означает: Показывается, если hr->finding_count равен нулю.

Откуда берется: Раздел статических признаков.

Нормальное чтение: Это значит, что набор реализованных статических правил не нашел проблем.

Когда насторожиться: Это не доказывает абсолютную безопасность файла и не заменяет динамику.

Что делать дальше: При необходимости запускать dynamic mode и ручной анализ.

Колонка 'Важность'

Что означает: Severity finding: низкая, средняя, высокая. Показывает серьезность конкретного признака.

Откуда берется: finding->severity.

Нормальное чтение: Высокая важность требует первоочередного внимания.

Когда насторожиться: Много средних признаков вместе могут быть сильнее одного слабого.

Что делать дальше: Сортировать внимание по severity, но читать все correlations.

Колонка 'Код'

Что означает: Машинное имя finding: например rf_oob, unwind_parse_fail, handler_nonexec_section.

Откуда берется: finding->code.

Нормальное чтение: Код стабилен и удобен для ссылок в тексте ВКР.

Когда насторожиться: Без описания код непонятен комиссии.

Что делать дальше: Открыть описание в той же строке и раздел 'Тщательно по признакам'.

Колонка 'Семейство'

Что означает: Группа признака: table_integrity, unwind_integrity, metadata_placement, handler_dominance и т.п.

Откуда берется: finding->family.

Нормальное чтение: Семейство показывает, какой слой анализа затронут.

Когда насторожиться: Если разные семейства срабатывают вместе, это усиливает картину.

Что делать дальше: Использовать для объяснения 6-слойной модели findings.

Колонка 'Затронуто'

Что означает: Количество записей или объектов, на которые распространяется finding.

Откуда берется: finding->affected_count.

Нормальное чтение: 1 означает локальную проблему; большое число может указывать на системный паттерн.

Когда насторожиться: affected_count=0 возможно у агрегированных correlation findings, где нет одного representative RVA.

Что делать дальше: Смотреть Observed в evidence block.

Колонка 'RVA'

Что означает: Representative RVA - примерный адрес внутри image, где проявился finding.

Откуда берется: finding->representative_rva при has_location.

Нормальное чтение: Помогает перейти в PE-viewer/Ghidra к конкретному месту.

Когда насторожиться: Прочерк означает, что finding агрегированный или не имеет одного адреса.

Что делать дальше: Если RVA есть, сопоставить с секцией и RUNTIME_FUNCTION row.

Колонка 'Секция'

Что означает: Имя секции, куда попадает representative RVA.

Откуда берется: section_name_or_dash(img, representative_rva).

Нормальное чтение: Помогает понять роль области: код, metadata, data.

Когда насторожиться: Прочерк может означать RVA вне секций или отсутствие location.

Что делать дальше: Если секция неожиданная, читать placement findings.

Колонка 'Описание'

Что означает: Русское объяснение finding плюс evidence block.

Откуда берется: html_append_finding_message_ru и html_append_finding_evidence_block.

Нормальное чтение: Это главный текст для защиты: там должно быть правило, наблюдение, ожидание и влияние.

Когда насторожиться: Если описание не прочитано, score не защищен.

Что делать дальше: Читать полностью, особенно Rule/Observed/Expected/Impact.

19.10. Evidence block внутри finding

Внутри описания finding отчет добавляет доказательный блок. Именно он превращает finding из 'инструмент сказал' в объяснимое доказательство. Хороший ответ комиссии строится по этому блоку.

Rule

Что означает: Какое правило проверялось. Например: диапазон должен быть внутри SizeOfImage, таблица должна быть отсортирована, handler должен указывать в ожидаемую область.

Откуда берется: finding_rule_text.

Нормальное чтение: Rule должно быть формулируемо как технический инвариант.

Когда насторожиться: Если правило непонятно, finding сложно защитить.

Что делать дальше: Перевести Rule на простой язык: 'что должно быть в нормальном PE'.

Observed

Что означает: Что реально увидел проект: конкретные значения счетчиков, RVA, секции, flags, affected_count, source_message.

Откуда берется: Собирается из finding fields и контекста PE/RUNTIME_FUNCTION.

Нормальное чтение: Observed должен содержать факты, а не мнение.

Когда насторожиться: Если observed не связан с rule, finding выглядел бы голословно.

Что делать дальше: Показывать observed как доказательство нарушения.

Expected

Что означает: Что ожидалось для нормального файла. Например: EndAddress > BeginAddress, UnwindInfoAddress внутри image, handler в исполняемой секции, metadata не writable.

Откуда берется: finding_expected_text.

Нормальное чтение: Expected задает нормальную модель.

Когда насторожиться: Если expected спорный, надо объяснять, что это эвристика, а не абсолютный закон.

Что делать дальше: На защите различать обязательные PE-инварианты и эвристические ожидания.

Impact

Что означает: Почему нарушение важно: как оно влияет на unwind, stack walking, exception dispatch или доверие к parser-а.

Откуда берется: finding_impact_text.

Нормальное чтение: Impact отвечает на вопрос 'и что с того?'.

Когда насторожиться: Без impact finding выглядит формальностью.

Что делать дальше: Объяснять последствия для анализа и reverse engineering.

Observed fields типа affected_count, representative_rva, file_offset

Что означает: Числовые доказательства конкретизируют finding.

Откуда берется: FindingLocation, section lookup, RuntimeFunction context.

Нормальное чтение: Чем конкретнее finding, тем легче его проверить руками.

Когда насторожиться: Если finding агрегированный, может не быть одного RVA.

Что делать дальше: Использовать как адрес для Ghidra/PE viewer.

19.11. Правила снижения уверенности

Код mitigation

Что означает: Название правила, которое смягчило интерпретацию или confidence.

Откуда берется: hr->mitigations[i].code.

Нормальное чтение: Mitigation не отменяет finding, а объясняет, почему вывод осторожнее.

Когда насторожиться: Нельзя говорить 'mitigation есть - значит finding ложный'.

Что делать дальше: Читать mitigation вместе с исходным finding.

Пояснение mitigation

Что означает: Русское объяснение, почему уверенность снижена.

Откуда берется: html_append_mitigation_message_ru.

Нормальное чтение: Помогает честно защитить не только сильные, но и слабые стороны анализа.

Когда насторожиться: Если mitigation много, итоговый score надо объяснять аккуратно.

Что делать дальше: Использовать как аргумент, что проект не завышает подозрительность.

19.12. Детали анализа

Pre-блок 'Детали анализа'

Что означает: Краткий список findings в текстовом виде: severity/code/message.

Откуда берется: Формируется после таблицы статических признаков.

Нормальное чтение: Удобен для быстрого копирования в текстовый отчет.

Когда насторожиться: Это сжатая версия, не заменяет таблицу с evidence.

Что делать дальше: Использовать как краткий список, а подробности брать из таблицы.

Строка '- Сильные статические аномалии не обнаружены'

Что означает: Появляется, когда finding_count равен нулю.

Откуда берется: HTML branch для пустого hr.

Нормальное чтение: Означает отсутствие найденных статических findings.

Когда насторожиться: Не означает, что файл полностью безопасен или что динамика ничего не найдет.

Что делать дальше: При необходимости запускать --dynamic.

19.13. Раздел 'Динамическая проверка' как часть полного отчета

Хотя динамические сигналы подробно разобраны в разделе 18, в полном HTML-отчете у динамики есть свои окна и элементы. Здесь объясняется, что означает каждый элемент именно в контексте чтения всего отчета.

Карточка 'Исключения'

Что означает: Сколько реальных исключений увидел debug loop, не считая служебных breakpoint/single-step.

Откуда берется: dr->exceptions_seen.

Нормальное чтение: 0 нормально, если путь выполнения не генерировал exceptions.

Когда насторожиться: Если exceptions много, надо смотреть first/second chance и handlers.

Что делать дальше: Открыть event table и журнал.

Карточка 'Попадания в handler'

Что означает: Сумма unwind_handler_hits, veh_hits, vch_hits, uef_hits. Показывает, что handler реально получил управление.

Откуда берется: dr->unwind_handler_hits + veh_hits + vch_hits + uef_hits.

Нормальное чтение: 0 не доказывает отсутствия handler path, только отсутствие наблюдения в данном запуске.

Когда насторожиться: Ненулевое значение - сильный runtime-факт.

Что делать дальше: Сопоставить с static handler findings.

Карточка 'Вызовы runtime-таблиц'

Что означает: Количество вызовов RtlAddFunctionTable и RtlInstallFunctionTableCallback.

Откуда берется: dr->rtl_add_function_table_calls + dr->rtl_install_function_table_cb_calls.

Нормальное чтение: 0 нормально для обычных статически скомпилированных EXE.

Когда насторожиться: Ненулевое значение означает, что runtime может менять unwind map.

Что делать дальше: Проверить arg0-arg3 в event table.

Карточка 'Потерянные события'

Что означает: Сколько DynamicEvent не попало в массив из-за лимита DYNAMIC_MAX_EVENTS.

Откуда берется: dr->dropped_events.

Нормальное чтение: 0 - таблица событий не была усечена.

Когда насторожиться: Больше 0 - отчет неполный по событиям.

Что делать дальше: Увеличить лимиты в коде или анализировать журнал.

Карточка 'Инструментация завершена'

Что означает: Показывает, удалось ли анализатору корректно поставить и обслужить breakpoints.

Откуда берется: dr->instrumentation_complete.

Нормальное чтение: Да - dynamic evidence надежнее.

Когда насторожиться: Нет - часть runtime path могла остаться ненаблюдаемой.

Что делать дальше: Смотреть ошибки breakpoint, pending rearm, журнал.

Карточка 'Handlers найдены / включены'

Что означает: Первое число - сколько handler RVA найдено статически в UNWIND_INFO; второе - на сколько удалось поставить breakpoints.

Откуда берется: dr->unwind_handlers_discovered и dr->unwind_handler_bp_set.

Нормальное чтение: Идеально, если включено столько же, сколько найдено.

Когда насторожиться: Если включено меньше, часть handlers не покрыта динамикой.

Что делать дальше: Смотреть причины в журнале и статические handler RVAs.

Карточка 'Ошибки breakpoint'

Что означает: Сколько раз не удалось поставить breakpoint.

Откуда берется: dr->breakpoint_set_failures.

Нормальное чтение: 0 - хорошо.

Когда насторожиться: Больше 0 снижает полноту dynamic evidence.

Что делать дальше: Проверить module coverage и адреса.

Карточка 'Покрытие модулей'

Что означает: Пик числа видимых модулей, ошибки обновления списка модулей, усечения списка.

Откуда берется: dr->modules_observed_peak, module_refresh_failures, module_list_truncations.

Нормальное чтение: Ошибки 0 - модульный контекст надежнее.

Когда насторожиться: Ошибки/усечения мешают точно подписывать адреса module/section.

Что делать дальше: Смотреть журнал выполнения.

Блок 'Расшифровка динамических сигналов'

Что означает: Групповое объяснение runtime-наблюдений: signal, observed, rule, interpretation.

Откуда берется: html_append_dynamic_evidence_table.

Нормальное чтение: Помогает читать динамику не по сырым событиям, а по смысловым группам.

Когда насторожиться: Не заменяет event table, потому что скрывает хронологию.

Что делать дальше: Сначала прочитать grouped interpretation, потом event rows.

Таблица динамических событий: 'Тип'

Что означает: Сырой код события: exception, veh_registration, uef_handler_hit и т.д.

Откуда берется: event->type.

Нормальное чтение: Стабильное машинное имя для ссылок.

Когда насторожиться: Без русской расшифровки может быть непонятно.

Что делать дальше: Смотреть соседнюю колонку 'Русское название'.

Таблица динамических событий: 'Русское название'

Что означает: Человеческое имя события.

Откуда берется: dynamic_event_type_ru.

Нормальное чтение: Помогает быстро понять type.

Когда насторожиться: Если type неизвестный, значит report.c не знает новой расшифровки.

Что делать дальше: Обновлять mapping в report.c.

Таблица динамических событий: 'Модуль'

Что означает: Модуль, к которому относится адрес события, если удалось определить.

Откуда берется: dynamic_event_add через module lookup.

Нормальное чтение: Модуль помогает понять, событие в target EXE, ntdll, kernel32 или другой DLL.

Когда насторожиться: Пустой/неизвестный модуль снижает точность интерпретации адреса.

Что делать дальше: Смотреть module coverage и журнал.

Thread

Что означает: ID потока, где произошло событие.

Откуда берется: event->thread_id.

Нормальное чтение: Нужен для понимания многопоточного поведения.

Когда насторожиться: Разные события в разных потоках могут относиться к разным execution paths.

Что делать дальше: При ручной отладке искать тот же thread.

Exception

Что означает: Код исключения для exception events. Для регистраций и process_exit обычно 0.

Откуда берется: event->exception_code.

Нормальное чтение: 0 для не-exception событий нормально.

Когда насторожиться: Ненулевой код нужно интерпретировать по Windows exception codes.

Что делать дальше: Сопоставить с chance и handler hits.

Chance

Что означает: first или second / n/a. Показывает стадию exception notification.

Откуда берется: event->first_chance.

Нормальное чтение: first - исключение только что возникло; second - обычная обработка не закрыла его.

Когда насторожиться: second chance сильнее как тревожный сигнал.

Что делать дальше: Смотреть handlers после этого события.

Address

Что означает: Адрес события: exception address, API breakpoint address или handler address.

Откуда берется: event->address.

Нормальное чтение: Нужен для ручной проверки в debugger/Ghidra.

Когда насторожиться: 0 может быть нормальным для process_exit/timeout.

Что делать дальше: Сопоставить с module и static RVA.

Arg 0 - Arg 3

Что означает: Аргументы события. Для API breakpoints это первые аргументы x64 calling convention: RCX, RDX, R8, R9.

Откуда берется: event->arg0..arg3.

Нормальное чтение: Для RtlAddFunctionTable это table/count/base; для callback - id/base/length/callback; для VEH/VCH - First/Handler.

Когда насторожиться: Непонимание arg fields ведет к неправильной интерпретации динамики.

Что делать дальше: Читать описание type события в разделе 18.

Смысл события

Что означает: Русское объяснение, зачем событие важно.

Откуда берется: dynamic_event_meaning_ru.

Нормальное чтение: Должно объяснять не только 'что', но и 'почему важно'.

Когда насторожиться: Если смысл слишком общий, нужно смотреть журнал и раздел 18.

Что делать дальше: Использовать как короткую расшифровку в отчете.

Журнал выполнения

Что означает: Техническая хронология debug session: CREATE_PROCESS, API resolved, breakpoints, self-check, hits, exit.

Откуда берется: dr->details.

Нормальное чтение: Нужен для проверки, что динамика реально работала.

Когда насторожиться: Ошибки в журнале объясняют неполные данные.

Что делать дальше: При споре показывать журнал как техническое доказательство.

19.14. Таблица RUNTIME_FUNCTION

Это самая низкоуровневая таблица отчета. Каждая строка соответствует одной записи RUNTIME_FUNCTION из Exception Directory. Именно из этой таблицы растут многие статические findings. Если комиссия спрашивает, где 'реальные данные', они здесь.

Поле фильтра

Что означает: Интерактивный поиск по таблице RUNTIME_FUNCTION. Можно вводить RVA, имя секции, flag, слово ошибки.

Откуда берется: HTML input id=function-filter и JavaScript в конце отчета.

Нормальное чтение: Фильтр не меняет анализ, только скрывает/показывает строки.

Когда насторожиться: Если строка пропала, возможно включен фильтр.

Что делать дальше: Очистить фильтр перед демонстрацией полного числа строк.

Счетчик 'X из Y'

Что означает: Показывает, сколько строк видно после фильтра и сколько всего.

Откуда берется: JavaScript function run().

Нормальное чтение: Если фильтр пустой, X должен равняться Y.

Когда насторожиться: Если X меньше Y, ты видишь не всю таблицу.

Что делать дальше: Перед выводами очищать фильтр.

Колонка '#'

Что означает: Номер записи RUNTIME_FUNCTION в массиве.

Откуда берется: Индекс i при генерации таблицы.

Нормальное чтение: Удобен для ссылок: запись 0, запись 1 и т.д.

Когда насторожиться: Номер не является RVA и не является приоритетом.

Что делать дальше: Использовать для связи с JSON/CSV.

Begin

Что означает: BeginAddress функции как RVA. Начало диапазона кода, который описывает RUNTIME_FUNCTION.

Откуда берется: rf[i].begin_rva.

Нормальное чтение: Должен быть меньше End и обычно попадать в executable code section.

Когда насторожиться: Begin вне image, в неправильной секции или дублирующийся Begin может быть finding.

Что делать дальше: Смотреть invalid_ranges, rf_oob, duplicate_begin.

End

Что означает: EndAddress функции как RVA. Конец диапазона функции, не включая сам End.

Откуда берется: rf[i].end_rva.

Нормальное чтение: End должен быть больше Begin.

Когда насторожиться: End <= Begin ломает диапазон функции.

Что делать дальше: Смотреть invalid_ranges.

Size

Что означает: Размер диапазона функции: End - Begin.

Откуда берется: runtime_function_size(&rf[i]).

Нормальное чтение: Положительный размер - нормальное условие.

Когда насторожиться: 0 или отрицательная логика через End <= Begin - ошибка диапазона.

Что делать дальше: Сопоставлять с Begin/End.

Секция кода

Что означает: Секция, куда попадает Begin RVA.

Откуда берется: section_name_or_dash(img, rf[i].begin_rva).

Нормальное чтение: Обычно .text или другая executable section.

Когда насторожиться: Если Begin не попадает в секцию или попадает в data/metadata, это странно.

Что делать дальше: Смотреть rf_oob и placement context.

Unwind RVA

Что означает: UnwindInfoAddress из RUNTIME_FUNCTION. Это RVA структуры UNWIND_INFO.

Откуда берется: rf[i].unwind_rva.

Нормальное чтение: Должен указывать на доступную UNWIND_INFO, обычно в .xdata или read-only metadata.

Когда насторожиться: OOB, unmappable или executable/writable placement может быть finding.

Что делать дальше: Смотреть unwind_rva_oob, unwind_parse_fail, unwind_in_exec_section, unwind_writable_section.

Unwind section

Что означает: Секция, куда попадает Unwind RVA.

Откуда берется: section_name_or_dash(img, rf[i].unwind_rva).

Нормальное чтение: Чаще .xdata или read-only metadata.

Когда насторожиться: Прочерк или неожиданная секция требует проверки.

Что делать дальше: Сопоставить с секциями и findings.

Статус

Что означает: OK или Ошибка. Показывает, удалось ли разобрать UNWIND_INFO для этой записи.

Откуда берется: us_ok[i].

Нормальное чтение: OK означает parser смог прочитать структуру.

Когда насторожиться: Ошибка означает parse_error, truncation, unmappable RVA или другой сбой разбора.

Что делать дальше: Читать последнюю колонку 'Аномалии / ошибка'.

Version / flags

Что означает: Версия UNWIND_INFO и flags: E/U/CHAIN и другие представления.

Откуда берется: us[i].version и us[i].flags.

Нормальное чтение: Обычный x64 unwind часто version 1; flags объясняют наличие handler или chain info.

Когда насторожиться: Неожиданная версия или несовместимые flags могут быть anomaly.

Что делать дальше: Смотреть unwind anomalies и handler layer.

Stack

Что означает: Сколько фиксированных stack bytes вычислил parser из unwind codes.

Откуда берется: us[i].fixed_stack_bytes.

Нормальное чтение: Нужен для понимания prolog/unwind behavior.

Когда насторожиться: Странно большой stack или невозможные unwind codes могут быть признаком ошибки/обфускации.

Что делать дальше: Смотреть anomalies по unwind codes.

Handler

Что означает: Handler RVA, если UNWIND_INFO имеет EHANDLER/UHANDLER.

Откуда берется: us[i].handler_rva при us[i].has_handler.

Нормальное чтение: Прочерк нормально, если handler flags нет.

Когда насторожиться: Handler в неожиданной секции, writable/nonexec/unknown области требует проверки.

Что делать дальше: Смотреть handler_nonexec_section, handler_writable_section, handler_dominance.

Аномалии / ошибка

Что означает: Текстовое описание anomalies и parse_error для конкретной записи.

Откуда берется: anomalies_to_str и us[i].parse_error.

Нормальное чтение: Прочерк означает, что для строки нет ошибки/anomaly.

Когда насторожиться: Любой текст здесь надо читать внимательно, особенно trunc, bad opcode, chain, handler.

Что делать дальше: Связать строку с findings выше.

19.15. Как читать строку отчета по цепочке

Если в отчете есть подозрительный finding, правильный разбор идет не по одному месту, а по цепочке. Ниже универсальный алгоритм чтения любой проблемы.

- Сначала смотри верхнюю шапку: есть ли score, findings, parse errors, dynamic status.

- Потом открой блок 'Образ PE': какие SizeOfImage, Exception directory, alignment.

- Потом смотри 'Структурная проверка': нет ли базовых PE-problems.

- Потом смотри таблицу 'Секции': куда попадают RVA и какие права у секций.

- Потом открой строку finding: severity, code, family, affected, RVA, section.

- Потом читай evidence block: Rule, Observed, Expected, Impact.

- Потом найди соответствующую строку в RUNTIME_FUNCTION table по RVA или Unwind RVA.

- Если запускалась динамика, проверь, есть ли handler hits, runtime table calls или exceptions.

- В конце читай итоговую оценку и план ручной проверки.

- Только после этой цепочки формулируй вывод для комиссии.

19.16. Что нельзя делать при чтении отчета

- Нельзя смотреть только '50 / 100' или '100 / 100' и делать вывод без findings.

- Нельзя путать RVA, VA и file offset.

- Нельзя считать 'динамика не запрашивалась' доказательством отсутствия runtime behavior.

- Нельзя считать регистрацию VEH/VCH/UEF автоматическим доказательством вредоносности.

- Нельзя считать timeout признаком вредоносности. Timeout говорит о качестве эксперимента.

- Нельзя игнорировать mitigation: оно показывает, что инструмент умеет ослаблять вывод.

- Нельзя игнорировать errors в instrumentation: при них динамика может быть неполной.

- Нельзя защищать finding без Rule/Observed/Expected/Impact.

19.17. Короткая формула для защиты

Если нужно кратко объяснить комиссии, как читать отчет, можно сказать так:

Сначала отчет фиксирует, какой конкретно файл проверялся: путь, SHA-256, версия инструмента и команда запуска. Затем он показывает PE-координаты: ImageBase, SizeOfImage, Exception Directory и секции. После этого идет структурная проверка PE layout. Далее отчет перечисляет статические findings с доказательством Rule/Observed/Expected/Impact. Если включена динамика, отчет показывает runtime evidence: исключения, handler hits, runtime function table registrations и качество instrumentation. Внизу находится таблица RUNTIME_FUNCTION - первичные данные, на которых основана значительная часть анализа. Итоговая оценка является не отдельным угадыванием, а сводом этих слоев.

Раздел 20. Откуда берутся поля, описания и рекомендации отчета

Этот раздел отвечает на вопрос комиссии: 'Где в программе формируются значения отчета, описания, рекомендации и итоговые выводы?'. Важно понимать: HTML-отчет не является отдельно написанной презентацией. Он собирается из данных, которые уже посчитаны parser-ом, unwind-разбором, эвристиками и динамическим collector-ом. Текстовые описания и рекомендации действительно лежат в коде, в основном в src/report.c, а фактические числовые данные приходят из src/pe.c, src/unwind.c, src/heuristics.c, src/dynamic_win.c и src/main.c.

На защите это нужно формулировать так: отчет состоит из двух видов информации. Первый вид - наблюдаемые факты: RVA, Size, Section, flags, counters, handler addresses, dynamic events. Они берутся из PE-файла или runtime-сессии. Второй вид - интерпретация: Rule, Expected, Impact, рекомендации ручной проверки, русские расшифровки. Они написаны в report.c как человекочитаемый слой над фактами, чтобы пользователь видел не только число, но и смысл проверки.

20.1. Главная цепочка происхождения отчета

Слой	Файл	Что делает	Что попадает в отчет
CLI и metadata	src/main.c:176, 454; include/report.h:18	Разбирает аргументы, режимы --heuristics/--dynamic, собирает ReportMetadata.	Команда запуска, версия, schema, SHA-256, размер файла, параметры анализа.
PE parser	src/pe.c:36, 230, 250-254, 265, 277	Читает PE headers, sections, DataDirectory, проверяет OOB/overlap/alignment, переводит RVA в file offset.	Образ PE, структурная проверка, секции, section lookup для RVA.
Unwind parser	src/unwind.c:56, 166, 613, 675	Читает RUNTIME_FUNCTION и UNWIND_INFO, разбирает flags, stack, handler, chain, parse errors.	Таблица RUNTIME_FUNCTION, parse failures, anomalies, handler RVA, stack bytes.
Static heuristics	src/heuristics.c:408, 514, 595, 788-1028	Применяет 27 правил, добавляет findings, считает score, verdict, confidence, mitigations.	Статические признаки, score, severity, family, affected count, confidence.
Dynamic collector	src/dynamic_win.c:416, 657, 768-1024	Запускает target под Debug API, ставит breakpoints, фиксирует exceptions, handlers, runtime function tables.	Динамические карточки, event table, журнал выполнения, runtime evidence.
Report renderer	src/report.c:66, 98, 130, 486, 507, 522, 565, 625, 1285, 1348, 1368, 1384, 1401, 1434, 1545	Превращает факты в HTML/JSON, добавляет русские описания, Rule/Expected/Impact, рекомендации и итог.	Полный HTML-отчет и JSON-отчет.

20.2. Почему описания в отчете не 'придуманы отдельно'

Текст в отчете делится на несколько типов. Каждый тип текста имеет свое место в коде и свою роль. Это важно: если комиссия спросит 'откуда вы взяли это описание?', ответ не должен быть 'я так написал'. Правильный ответ: 'описание находится в функции генерации отчета и привязано к конкретному коду finding или dynamic event; данные для него приходят из parser-а/heuristics/dynamic collector'.

Тип текста в отчете	Где в коде	Зачем нужен
Русское описание finding	src/report.c:1285 html_append_finding_message_ru	Кратко объясняет пользователю, что означает конкретный code finding.

Тип текста в отчете	Где в коде	Зачем нужен
Rule	src/report.c:66 finding_rule_text	Формулирует правило или инвариант, который проверялся.
Expected	src/report.c:98 finding_expected_text	Объясняет, что ожидалось в нормальном PE/unwind случае.
Impact	src/report.c:130 finding_impact_text	Объясняет, почему нарушение важно для unwind, exception dispatch или анализа.
Observed/evidence block	src/report.c:1348 html_append_finding_evidence_block	Подставляет факты: affected_count, RVA, section, file offset, related function, source message.
Mitigation explanation	src/report.c:1368 html_append_mitigation_message_ru	Поясняет, почему confidence мог быть снижен.
Manual validation plan	src/report.c:625 collect_manual_validation_steps	Дает человеку следующий шаг ручной проверки.
Dynamic event Russian name	src/report.c:1384 dynamic_event_type_ru	Переводит машинный type события в понятное название.
Dynamic event meaning	src/report.c:1401 dynamic_event_meaning_ru	Объясняет смысл runtime-события.
Dynamic grouped interpretation	src/report.c:1434 html_append_dynamic_evidence_table	Группирует runtime evidence по смыслу: таблицы, handlers, exceptions, качество сессии.
Final verdict/confidence	src/report.c:565 assessment_summary; src/heuristics.c:514 finalize_verdict	Собирает score, findings, dynamic evidence, correlation и quality в итоговый вывод.

20.3. Откуда берутся верхние карточки

Записи RUNTIME_FUNCTION

Какие данные используются: Количество записей n, прочитанных из Exception Directory как массив RUNTIME_FUNCTION.

Где это создается или считается в коде: src/unwind.c:613 unwind_read_runtime_functions_ex читает таблицу; src/unwind.c:675 wrapper возвращает результат.

Где это выводится или объясняется в отчете: src/report.c:1630 выводит карточку 'Записи RUNTIME_FUNCTION'.

Как интерпретировать: Это первичный объем function table. Мало или ноль записей может быть нормально для очень простого/leaf-only образца, но для сложного x64 EXE требует объяснения.

Как защищать на комиссии: Показать, что число не придумано: оно получается из Exception Directory Size / sizeof(RUNTIME_FUNCTION) и чтения .pdata.

Ошибки разбора

Какие данные используются: Счетчик записей, у которых us_ok[i] == false, то есть UNWIND_INFO не был корректно разобран.

Где это создается или считается в коде: src/unwind.c:166 unwind_parse разбирает UNWIND_INFO; ошибки записываются в UnwindSummary.parse_error.

Где это выводится или объясняется в отчете: src/report.c:1580-1585 считает parse_failures; src/report.c:1631-1632 выводит карточку.

Как интерпретировать: Ненулевое значение означает не 'ошибка отчета', а проблему чтения конкретных unwind records.

Как защищать на комиссии: На комиссии говорить: parser не падает, а фиксирует невозможность разбора как evidence.

Аномальные записи

Какие данные используются: Количество UnwindSummary с ненулевым us[i].anomalies.

Где это создается или считается в коде: src/unwind.c:166 unwind_parse выставляет признаки anomalies при разборе UNWIND_INFO.

Где это выводится или объясняется в отчете: src/report.c:1580-1586 считает anomaly_count; src/report.c:1633-1634 выводит карточку.

Как интерпретировать: Это мост между низкоуровневым parser-ом и статическими findings: parser видит странность, heuristics может поднять ее до finding.

Как защищать на комиссии: Показать конкретные строки в таблице RUNTIME_FUNCTION и последнюю колонку 'Аномалии / ошибка'.

Эвристические признаки и статическая оценка

Какие данные используются: finding_count и score из HeuristicReport.

Где это создается или считается в коде: src/heuristics.c:408 add_finding добавляет finding; src/heuristics.c:788-1028 содержит правила и add_score; src/heuristics.c:514 finalize_verdict считает confidence/verdict.

Где это выводится или объясняется в отчете: src/report.c:1636-1641 выводит карточки findings и score.

Как интерпретировать: Это не просто счетчик parser errors, а слой правил: какие инварианты PE/unwind нарушены и насколько это важно.

Как защищать на комиссии: Показать, что за каждым числом стоят конкретные add_finding вызовы и SCORE_* константы.

Итоговый вывод, статика/динамика, динамические сигналы, итоговая уверенность

Какие данные используются: AssessmentSummary собирает статический уровень, runtime level, correlation, dynamic signal count, final confidence.

Где это создается или считается в коде: src/report.c:486 dynamic_signal_count; 507 has_static_dynamic_correlation; 522 runtime_evidence_level; 565 assessment_summary.

Где это выводится или объясняется в отчете: src/report.c:1643-1661 выводит итоговые карточки.

Как интерпретировать: Эти поля являются сводкой нескольких слоев, а не отдельной проверкой PE.

Как защищать на комиссии: Объяснять, что итог берется из модели evidence: static findings + dynamic evidence + качество runtime-сессии + корреляция.

20.4. Откуда берутся PE-блоки и структурные проверки

ImageBase, SizeOfImage, SizeOfHeaders, Exception Directory, Alignments

Какие данные используются: Поля PE Optional Header и DataDirectory[Exception].

Где это создается или считается в коде: src/pe.c:36 pe_parse читает PE; src/pe.c:230 проверяет exception_directory_oob.

Где это выводится или объясняется в отчете: src/report.c:1690-1699 выводит блок 'Образ PE'.

Как интерпретировать: Это координатная система: без нее нельзя объяснить RVA, SizeOfImage, mapping и OOB.

Как защищать на комиссии: На комиссии ссылаться на PE format: эти поля не придуманы проектом, они находятся в PE headers.

Raw/virtual OOB, overlaps, misalignment

Какие данные используются: Счетчики нарушений section table: raw за файл, virtual за image, пересечения raw/virtual, нарушения alignment.

Где это создается или считается в коде: src/pe.c:250-254 заполняет section_raw_oob, section_virtual_oob, section_raw_overlaps, section_virtual_overlaps, section_misaligned.

Где это выводится или объясняется в отчете: src/report.c:1701-1710 выводит блок 'Структурная проверка'.

Как интерпретировать: Эти значения показывают, можно ли доверять базовому PE layout.

Как защищать на комиссии: Объяснить: parser сначала проверяет фундаментальные границы, а уже потом интерпретирует .pdata/.xdata.

Таблица секций

Какие данные используются: Массив PeSection: name, virtual_address, virtual_size, raw_ptr, raw_size, characteristics.

Где это создается или считается в коде: src/pe.c:36 pe_parse заполняет sections; src/pe.c:265 pe_find_section_for_rva ищет секцию по RVA; src/pe.c:277 pe_rva_to_fileoff переводит RVA в file offset.

Где это выводится или объясняется в отчете: src/report.c:1713-1731 выводит таблицу 'Секции'.

Как интерпретировать: Таблица нужна, чтобы понять, где лежит RVA и какие права R/W/X у области.

Как защищать на комиссии: На комиссии показать, что findings не висят в воздухе: representative RVA сопоставляется с секцией через section lookup.

20.5. Откуда берутся статические findings и их объяснения

Сами findings

Какие данные используются: Каждый finding создается add_finding с code, family, severity, affected_count, location, source message.

Где это создается или считается в коде: src/heuristics.c:408 add_finding; src/heuristics.c:788-1028 конкретные правила 27 признаков.

Где это выводится или объясняется в отчете: src/report.c:1758-1786 выводит таблицу 'Статические признаки'.

Как интерпретировать: Finding - это не отдельный текст, а запись в структуре HeuristicReport, созданная конкретным правилом.

Как защищать на комиссии: На комиссии можно открыть heuristics.c и показать add_finding для конкретного code.

Severity, family, affected count, RVA, section

Какие данные используются: Поля HeuristicFinding: severity, family, affected_count, has_location, representative_rva.

Где это создается или считается в коде: src/heuristics.c:408-446 заполняет HeuristicFinding; section вычисляется позже через section_name_or_dash.

Где это выводится или объясняется в отчете: src/report.c:1766-1782 выводит эти колонки; src/report.c:53 section_name_or_dash связывает RVA с секцией.

Как интерпретировать: Эти поля показывают не только 'что найдено', но и где, насколько массово и к какому семейству evidence относится.

Как защищать на комиссии: Подчеркнуть: affected_count и RVA - факты, а не мнение. Severity - инженерная классификация правила.

Rule / Observed / Expected / Impact

Какие данные используются: Rule, Expected и Impact - текстовые объяснения по code finding; Observed строится из фактических полей finding и PE/unwind context.

Где это создается или считается в коде: src/report.c:66 finding_rule_text; 98 finding_expected_text; 130 finding_impact_text; 1348 html_append_finding_evidence_block.

Где это выводится или объясняется в отчете: Внутри колонки 'Описание' в таблице 'Статические признаки'.

Как интерпретировать: Это главный доказательный формат: правило, факт, норма, влияние.

Как защищать на комиссии: Если спрашивают 'правильно ли написано', отвечать: Rule/Expected основаны на PE/x64 unwind invariants, Observed берется из parser-a, Impact объясняет практическое последствие.

Правила снижения уверенности

Какие данные используются: Mitigations добавляются, когда evidence изолированная, выборка маленькая или признаки недостаточно коррелируют.

Где это создается или считается в коде: src/heuristics.c:480 add_mitigation; src/heuristics.c:540-568 weakening rules; src/report.c:1368 html_append_mitigation_message_ru.

Где это выводится или объясняется в отчете: Раздел 'Правила снижения уверенности'.

Как интерпретировать: Это защита от завышения: проект не только повышает score, но и снижает confidence, если evidence слабая.

Как защищать на комиссии: На комиссии это сильный аргумент: система не делает грубый binary verdict, а учитывает качество evidence.

План ручной проверки

Какие данные используются: Рекомендации формируются по наборам findings и dynamic counters.

Где это создается или считается в коде: src/report.c:625 collect_manual_validation_steps.

Где это выводится или объясняется в отчете: Раздел 'План ручной проверки' внутри итоговой оценки.

Как интерпретировать: Это не машинный verdict, а подсказка аналитику, какие места проверить руками.

Как защищать на комиссии: Объяснить: автоматический анализатор не подменяет reverse engineer, а подготавливает evidence и next steps.

20.6. Откуда берется динамическая часть

DynamicEvent и DynamicReport

Какие данные используются: Структуры хранят type, module, thread_id, exception_code, first_chance, address, arg0-arg3, counters, details.

Где это создается или считается в коде: include/dynamic.h:28 DynamicEvent; include/dynamic.h:41 DynamicReport.

Где это выводится или объясняется в отчете: src/report.c использует DynamicReport для карточек, event table и журнала.

Как интерпретировать: Все динамические элементы отчета имеют конкретные поля структуры, а не свободный текст.

Как защищать на комиссии: На комиссии показать include/dynamic.h как контракт данных динамического отчета.

Сбор runtime events

Какие данные используются: Процесс запускается под Windows Debug API; анализатор получает debug events, ставит breakpoints и записывает DynamicEvent.

Где это создается или считается в коде: src/dynamic_win.c:657 dynamic_collect_windows; 416 dynamic_event_add; 768-1024 основные события timeout, process_exit, exception, VEH/VCH/UEF, RtlAddFunctionTable.

Где это выводится или объясняется в отчете: src/report.c:1818-1875 выводит dynamic section; src/report.c:1434 grouped dynamic evidence.

Как интерпретировать: Динамика показывает только то, что реально произошло в данном запуске.

Как защищать на комиссии: Защищать ограничение честно: если timeout или instrumentation incomplete, отсутствие события не доказывает его невозможность.

Русские названия и смыслы dynamic events

Какие данные используются: Машинные type вроде veh_registration переводятся в человекочитаемые русские названия и объяснения.

Где это создается или считается в коде: src/report.c:1384 dynamic_event_type_ru; 1401 dynamic_event_meaning_ru.

Где это выводится или объясняется в отчете: Колонки 'Русское название' и 'Смысл события' в таблице динамических событий.

Как интерпретировать: Это слой объяснения над сырыми event->type и event->arg0..arg3.

Как защищать на комиссии: На комиссии говорить: события собирает dynamic collector, а report.c только объясняет их пользователю.

Расшифровка динамических сигналов

Какие данные используются: Групповая интерпретация runtime evidence: runtime function tables, registrations, handler hits, exceptions, session quality.

Где это создается или считается в коде: src/report.c:1434 html_append_dynamic_evidence_table.

Где это выводится или объясняется в отчете: Блок 'Расшифровка динамических сигналов'.

Как интерпретировать: Этот блок объясняет не каждую строку, а смысловую группу событий.

Как защищать на комиссии: Если спросят, почему dynamic_signal_count меньше event_count: потому что он считает группы evidence, а не строки event log.

20.7. Откуда берется таблица RUNTIME_FUNCTION

Begin, End, Unwind RVA

Какие данные используются: Три DWORD-поля RUNTIME_FUNCTION: BeginAddress, EndAddress, UnwindInfoAddress.

Где это создается или считается в коде: src/unwind.c:613 unwind_read_runtime_functions_ex читает массив; include/unwind.h описывает RuntimeFunction.

Где это выводится или объясняется в отчете: src/report.c:1888-1918 выводит строки таблицы RUNTIME_FUNCTION.

Как интерпретировать: Это первичные данные .pdata. Большая часть статических проверок начинается именно отсюда.

Как защищать на комиссии: На комиссии объяснить: проект не дизассемблирует 'на глаз', а читает официальную x64 function table.

Статус, Version / flags, Stack, Handler, anomalies

Какие данные используются: Результат parse_unwind для каждой записи: удалось ли разобрать UNWIND_INFO, какая version/flags, сколько stack bytes, есть ли handler RVA, какие anomalies/parse_error.

Где это создается или считается в коде: src/unwind.c:166 unwind_parse; src/unwind.c:56 parse_unwindinfo_header; src/report.c:1888-1922 выводит поля.

Где это выводится или объясняется в отчете: Таблица RUNTIME_FUNCTION, последние колонки.

Как интерпретировать: Это низкоуровневое объяснение, почему конкретная запись OK или Ошибка.

Как защищать на комиссии: Если finding связан с unwind_parse_fail или unwind_anomalies, показывать соответствующую строку этой таблицы.

Примечание: если в комиссии спрашивают, почему таблица RUNTIME_FUNCTION в отчете обязательна, ответ такой: это не вспомогательная таблица, а сырьевая доказательная база. Findings и score являются интерпретацией, а RUNTIME_FUNCTION rows показывают исходные данные, из которых эта интерпретация получилась.

20.8. Как доказать правильность описаний и рекомендаций

Чтобы защитить отчет, нужно разделять факты, правила и рекомендации. Факты проверяются по коду parser-а и значениям в отчете. Правила проверяются по PE/x64 unwind модели и реализации в heuristics.c. Рекомендации проверяются по тому, какие findings сработали и какие ручные действия помогают подтвердить или опровергнуть автоматический вывод.

- Если спрашивают 'где вы взяли RVA/Size/Section?' - ответ: PE parser в src/pe.c читает headers и section table, report.c выводит уже прочитанные поля.

- Если спрашивают 'где вы взяли Begin/End/Unwind RVA?' - ответ: unwind.c читает Exception Directory как RUNTIME_FUNCTION table.

- Если спрашивают 'где вы взяли finding?' - ответ: конкретное правило в src/heuristics.c вызывает add_finding с code/severity/family.

- Если спрашивают 'где написано описание finding?' - ответ: src/report.c:1285 для русских сообщений, 66/98/130 для Rule/Expected/Impact.

- Если спрашивают 'где рекомендации?' - ответ: src/report.c:625 collect_manual_validation_steps и src/report.c:1368 mitigation explanations.

- Если спрашивают 'где динамическая расшифровка?' - ответ: dynamic_win.c собирает события, report.c:1384/1401/1434 объясняет их в HTML.

- Если спрашивают 'почему итог такой?' - ответ: heuristics.c считает статический score/confidence, report.c assessment_summary объединяет статику, динамику, корреляцию и качество сессии.

20.9. На чем основаны интерпретации: Microsoft docs или эвристика проекта

Комиссия может спросить: 'Почему вы решили, что именно так надо интерпретировать поле, finding или dynamic event?'. Ответ должен быть честным и разделенным на две части. Первая часть - нормативные правила формата и ABI, они берутся из документации Microsoft. Вторая часть - scoring, веса, confidence, группировка severity и рекомендации ручной проверки, это инженерная модель проекта, построенная поверх этих правил. Microsoft не публикует стандарт 'за rf_oob дать +25 баллов'. Microsoft задает формат и поведение Windows, а проект оценивает нарушения этого формата как evidence.

Что объясняется в отчете	На чем основано	Как правильно защищать
RVA, VA, file offset, sections, DataDirectory, SizeOfImage, SectionAlignment, FileAlignment	Microsoft PE Format: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format	Это официальное описание PE/COFF layout. Проект проверяет, что поля не противоречат этой модели.
.pdata и RUNTIME_FUNCTION table	Microsoft PE Format и Microsoft x64 exception handling: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64	.pdata содержит function table entries; x64 RUNTIME_FUNCTION имеет Begin, End, UnwindInfoAddress, entries image-relative и sorted.
UNWIND_INFO, Version, Flags, EHANDLER, UHANDLER, CHAININFO, handler RVA	Microsoft x64 exception handling	Описания handler flags, chained unwind и handler RVA взяты из официальной структуры UNWIND_INFO.
Unwind codes, stack bytes, prolog effects	Microsoft x64 exception handling	Проект интерпретирует unwind codes как описание действий prolog/unwind. Anomalies возникают, когда структура не читается или выглядит несовместимо.
First chance / second chance exception	Microsoft Exception Handling Functions for Debugging и Debugger Exception Handling	Динамическая интерпретация first/second chance основана на официальной модели уведомлений debugger-а.
VEH registration и handler pointer	Microsoft AddVectoredExceptionHandler	Функция официально регистрирует vectored exception handler; параметры First и Handler объясняют arg0/arg1 в отчете.
VCH registration	Microsoft AddVectoredContinueHandler / Structured Exception Handling Functions	Функция официально регистрирует vectored continue handler, поэтому проект фиксирует это как runtime evidence.
UEF registration	Microsoft SetUnhandledExceptionFilter	Функция заменяет top-level unhandled exception filter; поэтому регистрация и hit интерпретируются как runtime exception-handling layer.
RtlAddFunctionTable	Microsoft RtlAddFunctionTable	Функция добавляет dynamic function table для 64-bit unwind/stack walking, поэтому статическая .pdata может быть неполной runtime-картой.
RtlInstallFunctionTableCallback	Microsoft RtlInstallFunctionTableCallback	Функция задает callback-based dynamic function table для очень динамичного кода.

Что объясняется в отчете	На чем основано	Как правильно защищать
RCX, RDX, R8, R9 как arg0-arg3	Microsoft x64 calling convention	Первые integer/pointer аргументы Windows x64 передаются через RCX, RDX, R8, R9, поэтому dynamic collector читает эти регистры при API breakpoint.
Score, weights, severity, confidence	Инженерная модель проекта, не стандарт Microsoft	Microsoft задает формат и поведение. Проект задает веса для приоритизации evidence. Это нужно защищать как эвристическую scoring-модель, а не как официальный вердикт Windows.
Manual validation recommendations	Методика reverse engineering и структура найденных findings	Рекомендации не являются стандартом Microsoft. Они объясняют, как руками проверить наблюдения: PE viewer, Ghidra, debugger, сопоставление RVA/секций/handlers.

Главный смысл: официальная документация Microsoft отвечает на вопрос 'как должен быть устроен PE/x64 unwind/runtime exception mechanism'. Проект отвечает на следующий вопрос: 'если наблюдаемая структура нарушает или необычно использует эту модель, насколько это важно для анализа'. Поэтому описания формата, полей и API опираются на Microsoft, а баллы и итоговая шкала являются авторской эвристической моделью, которую нужно защищать через прозрачность: код правил, веса, evidence block и ручная проверка.

20.10. Что говорить, если спросят про правильность score и весов

Нужно сказать прямо: Microsoft не устанавливает числовые веса подозрительности для таких признаков. Официальные документы говорят, что такое RUNTIME_FUNCTION, где лежит .pdata, что такое UNWIND_INFO, как работают handlers и dynamic function tables. А проект поверх этой нормативной базы строит scoring model: более сильные нарушения структурных инвариантов получают больший вес, слабые или контекстные признаки получают меньший вес, корреляционные признаки усиливают вывод, mitigation rules снижают confidence.

- Если нарушение делает структуру нечитаемой или выходит за границы image/file, оно весит больше, потому что ломает саму возможность надежного анализа.

- Если признак только контекстный, например unusual section или registration без handler hit, он весит меньше, потому что может иметь нормальное объяснение.

- Если несколько независимых слоев совпадают, например static placement problem плюс dynamic handler hit, итог усиливается, потому что evidence коррелирует.

- Если выборка маленькая, signal одинокий или динамика неполная, confidence снижается, потому что честная модель не должна переутверждать вывод.

- Поэтому score - это не 'истина Windows', а прозрачный способ приоритизировать findings для reverse engineer или malware analyst.

20.11. Формула ответа комиссии

Готовая формулировка: 'В отчете нет произвольных чисел или отдельно придуманных выводов. Сначала PE parser извлекает реальные поля PE: headers, sections, Exception Directory и границы. Затем unwind parser читает RUNTIME_FUNCTION и UNWIND_INFO. Потом heuristics.c применяет набор правил к этим данным и создает findings с severity, family, affected_count и RVA. Динамический модуль, если включен, запускает образ под Debug API и записывает runtime events. report.c не выдумывает данные, а преобразует эти структуры в понятный HTML: добавляет русские описания, Rule, Observed, Expected, Impact и рекомендации ручной проверки. Поэтому каждое окно отчета

можно связать с конкретной структурой данных и конкретным местом в исходном коде.'

20.12. Мини-шпаргалка коротких ответов

- Откуда взяты поля PE? Из официальной структуры PE/COFF, разобранной parser-ом в src/pe.c.

- Откуда взята логика .pdata и RUNTIME_FUNCTION? Из Microsoft PE Format и Microsoft x64 exception handling.

- Откуда взята логика UNWIND_INFO и handler flags? Из Microsoft x64 exception handling.

- Откуда взята динамика VEH/VCH/UEF? Из Microsoft Win32 API для vectored/unhandled exception handlers.

- Откуда взяты arg0-arg3 в динамике? Из Microsoft x64 calling convention: RCX, RDX, R8, R9.

- Откуда взяты баллы? Это не стандарт Microsoft, а прозрачная эвристика проекта для ранжирования evidence.

- Почему можно доверять описанию? Потому что каждое описание привязано к конкретному code finding/event type, а observed values берутся из parser-а или dynamic collector-а.

- Что делать, если комиссия спорит с интерпретацией? Показать Rule/Observed/Expected/Impact, строку кода add_finding и первичный источник Microsoft для соответствующей структуры/API.

20.13. Обоснование фраз из колонки 'Как интерпретировать'

На скриншоте из HTML-отчета видна колонка 'Как интерпретировать'. Это не сырые данные, а аналитическое объяснение. Поэтому его надо защищать отдельно. Важно: эти фразы не утверждают 'вредоносно' или 'безопасно'. Они объясняют, как аналитик должен относиться к наблюдению. Ниже разобраны типовые фразы из этой колонки.

Фраза в отчете	Почему это корректно	На чем основано
Сам факт регистрации может быть нормальным, но он показывает runtime-слой обработки исключений, которого не видно только по PE/.pdata.	Корректно, потому что AddVectoredExceptionHandler, AddVectoredContinueHandler и SetUnhandledExceptionFilter являются штатными Windows API. Их вызов сам по себе не является нарушением. Но регистрация происходит во время выполнения, а не как статическая запись .pdata, поэтому один только PE/.pdata анализ не показывает всю runtime-логику обработки исключений.	Microsoft docs по AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter; Microsoft PE Format и x64 exception handling для .pdata/RUNTIME_FUNCTION.
Это сильнее простой регистрации: handler не только существует, но и реально получил управление.	Корректно, потому что регистрация handler означает только потенциальную возможность вызова. Hit означает, что анализатор поставил breakpoint на handler address и увидел фактическое достижение этого адреса потоком исполнения. Наблюдаемое исполнение сильнее, чем просто наличие указателя.	Windows Debug API / debugger exception handling; логика проекта в dynamic_win.c: после регистрации handler ставится breakpoint, при срабатывании добавляется *_handler_hit.
Если исключений нет, handler registrations остаются контекстом. Если исключения есть, можно проверять реальный маршрут dispatch/unwind.	Корректно, потому что handler вызывается в контексте обработки исключения. Если в данном запуске не было exception events, проект видел регистрацию, но не видел route обработки. Если exception events есть, можно связать их с first/second chance, handler hits и дальнейшим trace.	Microsoft debugger exception handling: first-chance/last-chance exception notifications; Microsoft x64 exception handling: exception dispatch использует function table/unwind information.

Фраза в отчете	Почему это корректно	На чем основано
Если здесь есть предупреждения, это не отменяет найденные события, но снижает полноту runtime-наблюдения.	Корректно, потому что найденное событие остается фактом: если breakpoint сработал, событие было. Но ошибки module refresh, breakpoint failures, dropped events, timeout или incomplete instrumentation означают, что анализатор мог не увидеть часть других событий. Поэтому найденное не отменяется, а отсутствие дополнительных событий становится менее доказательным.	Это методика качества эксперимента проекта, реализованная через DynamicReport fields: timed_out, instrumentation_complete, module_refresh_failures, breakpoint_set_failures, dropped_events. Основано на обычной логике debugger instrumentation: неполное покрытие снижает полноту вывода.
Runtime function tables не обязательно вредоносны, но показывают, что статическая .pdata уже не является полной картой unwind.	Корректно, потому что Microsoft прямо описывает RtlAddFunctionTable и RtlInstallFunctionTableCallback как штатные способы предоставить function table для dynamically generated code. Значит, сам факт вызова может быть нормальным для JIT/runtime. Но он означает, что часть function table появляется во время выполнения, поэтому статический PE-файл может не содержать всей картины.	Microsoft x64 exception handling; Microsoft RtlAddFunctionTable; Microsoft RtlInstallFunctionTableCallback.
Регистрация exception handlers сама по себе слабее, чем handler hit.	Корректно, потому что registration фиксирует намерение/настройку механизма, а hit фиксирует реальное прохождение управления. В терминах evidence: registration - потенциальный route, hit - наблюденный route.	Логика evidence проекта; Debug API breakpoint observation; Microsoft docs подтверждают назначение API регистрации handlers.

Главное, что нужно сказать комиссии: интерпретационные фразы построены не как произвольные утверждения, а как осторожные аналитические выводы из официально описанных механизмов Windows. Где Microsoft говорит 'эта функция регистрирует handler', проект не говорит 'это вредоносно'. Проект говорит: 'это runtime-слой, его надо учитывать'. Где анализатор видит handler hit, проект говорит: 'это сильнее регистрации', потому что это уже наблюдаемое исполнение. Где есть warnings по instrumentation, проект честно снижает полноту вывода. Такая осторожность и делает отчет защищаемым.

20.14. Обоснование evidence-блоков статических findings

На втором скриншоте показан evidence-блок статического finding. У него есть строгая структура: описание, правило, что найдено, как должно быть, почему важно. Такой блок нужно защищать иначе, чем динамическую интерпретацию. Здесь часть текста основана на официальном формате x64 unwind, а часть является фактическим наблюдением parser-a.

Элемент evidence-блока	Откуда берется	Почему это корректно
Описание: UnwindInfoRVA отсутствует или выходит за допустимые границы	Это русское описание finding code unwind_rva_oob из src/report.c:1285. Сам finding создается в src/heuristics.c, когда UnwindInfoAddress не указывает на допустимую область.	Microsoft x64 exception handling описывает, что третье поле RUNTIME_FUNCTION - Unwind info address, image-relative offset. Если этот RVA отсутствует, OOB или не сопоставляется с доступной UNWIND_INFO, Windows unwind/debugger не получает нормальную metadata для функции.

Элемент evidence-блока	Откуда берется	Почему это корректно
Правило: UnwindInfoAddress каждой RUNTIME_FUNCTION должен указывать на доступную UNWIND_INFO внутри image	Rule лежит в src/report.c:66 finding_rule_text. Данные Begin/End/Unwind RVA приходят из src/unwind.c.	Основано на Microsoft x64 exception handling: RUNTIME_FUNCTION содержит BeginAddress, EndAddress, UnwindInfoAddress, все image-relative. UNWIND_INFO должна быть доступной структурой unwind data для функции.
Что найдено: affected_count, representative_rva, file_offset, section, characteristics, related_function	Observed строится в src/report.c:1348 html_append_finding_evidence_block из фактических данных parser-a, section lookup и связанной RUNTIME_FUNCTION entry.	Это не интерпретация и не мнение. Это конкретные значения, которые проект получил из PE/unwind-разбора: какой RVA, какая секция, можно ли перевести RVA в file offset, какая связанная функция.
file_offset=не маппится в файл	Результат pe_rva_to_fileoff из src/pe.c:277. Parser пытается сопоставить RVA с raw-диапазоном файла.	Для статического чтения нужны реальные байты на диске. Если RVA не переводится в file offset, parser не может прочитать UNWIND_INFO из файла. Даже если в памяти loader мог бы создать zero-fill область, на диске этих байтов нет.
section=-	Результат section_name_or_dash/pe_find_section_for_rva. Прочерк означает, что representative RVA не удалось связать с нормальной секцией.	Если RVA не попадает в ожидаемую секцию image, это нарушает доверие к координатной системе. Для unwind metadata ожидается доступная область image, обычно read-only metadata.
Как должно быть: UnwindInfoAddress должен быть RVA внутри image и должен мапиться в байты файла при статическом чтении	Expected лежит в src/report.c:98 finding_expected_text.	Первая часть основана на Microsoft x64 exception handling: UnwindInfoAddress image-relative. Вторая часть - требование статического parser-a: чтобы разобрать структуру без запуска, нужны байты файла, соответствующие RVA.
Почему важно: Windows или эмуляция unwind может не найти metadata функции	Impact лежит в src/report.c:130 finding_impact_text.	Основано на unwind procedure из Microsoft x64 exception handling: Windows ищет RUNTIME_FUNCTION для ControlPc, затем использует UNWIND_INFO, чтобы восстановить context caller-a и обработать exception/unwind.
Описание: Ошибки разбора UNWIND_INFO	Finding unwind_parse_fail появляется, когда unwind_parse не может структурно разобрать UNWIND_INFO.	Microsoft x64 exception handling задает поля UNWIND_INFO: Version, Flags, prolog size, CountOfCodes, unwind codes, optional handler или chained info. Если структура обрывается или противоречит формату, ее нельзя надежно использовать.
Правило: UNWIND_INFO должна структурно разбираться по формату x64 unwind без обрыва и выхода за границы	Rule из src/report.c:66, фактический parse_error из src/unwind.c:166.	Корректно, потому что Windows unwind ожидает валидную структуру. Parser не обязан угадывать битые поля; он фиксирует ошибку как finding.
Как должно быть: Version, Flags, CountOfCodes, массив unwind-кодов, padding и tail-данные должны разбираться без усечения	Expected из src/report.c:98.	Основано на официальном layout UNWIND_INFO. Optional tail зависит от flags: EHANDLER/UHANDLER ведут к handler RVA и language-specific data; CHAININFO ведет к chained RUNTIME_FUNCTION.
Почему важно: Virtual unwind может вычислить неверный caller context или скрыть handler path	Impact из src/report.c:130.	Основано на роли UNWIND_INFO в Windows unwind procedure: она описывает, как восстановить регистры/stack pointer и перейти к caller frame. Ошибка разбора ломает stack walking и exception dispatch analysis.

Итоговая защита таких строк: 'Описание, Правило, Как должно быть и Почему важно основаны на официальной модели PE/x64 unwind. Что найдено - это конкретные значения, которые parser извлек из файла. Если representative_rva не маппится в file offset, это не мнение, а результат функции pe_rva_to_fileoff. Если UNWIND_INFO не разбирается, это не придирка, а нарушение ожидаемой структуры, без которой Windows virtual unwind и stack walking могут работать неверно.'

Раздел 21. Каталог фраз отчета: источник, смысл и условия появления

Этот раздел отвечает на самый строгий вопрос комиссии: 'Все фразы в отчете откуда взяты, почему они правильные, при каких условиях появляются и как их понимать?'. Ниже фразы отчета разобраны не как художественный текст, а как слой доказательной модели. Для каждой группы указано: что основано на официальной документации Microsoft, что является фактическим наблюдением проекта, а что является инженерной осторожной интерпретацией.

Важно: отчет содержит не только однотипные предложения. Есть служебные подписи, фактические значения, правила формата, ожидаемое состояние, влияние на анализ, динамические интерпретации, mitigations, рекомендации и итоговые verdict-фразы. Поэтому их нельзя защищать одним ответом. Правильная защита строится по типу фразы.

21.1. Классификация всех фраз отчета

Тип фразы	Где появляется	Что означает	Основание
Служебная подпись поля	Карточки, таблицы, заголовки: SHA-256, ImageBase, SizeOfImage, Begin, End, Handler.	Называет поле отчета. Сама по себе не является выводом.	Структура HTML-отчета в src/report.c и структуры данных проекта.
Фактическое наблюдение	Что найдено, affected_count, RVA, section, file_offset, event args.	Конкретное значение, полученное parser-ом или dynamic collector-ом.	Код проекта: src/pe.c, src/unwind.c, src/dynamic_win.c.
Правило формата	Rule / Правило.	Как должна быть устроена структура для нормального PE/x64 unwind.	Microsoft PE Format, Microsoft x64 exception handling, Win32 API docs.
Ожидаемое состояние	Expected / Как должно быть.	Нормальная форма структуры или нормальный контекст чтения.	Официальная модель Microsoft плюс требование статического parser-а к доступности байтов.
Impact / Почему важно	Evidence block finding.	Почему нарушение влияет на unwind, stack walking, exception dispatch или полноту анализа.	Microsoft unwind procedure и инженерная логика анализа.
Как интерпретировать	Расшифровка динамических сигналов.	Осторожная оценка силы runtime evidence.	Win32 API docs + Debug API observations + инженерная evidence-модель.
Score / severity / confidence	Итоговая оценка, статическая оценка, важность finding.	Приоритизация признаков и уверенность в выводе.	Эвристическая модель проекта, не стандарт Microsoft.
Рекомендации	План ручной проверки.	Что аналитику проверить руками после автоматического отчета.	Методика reverse engineering, основанная на найденных fields/findings/events.

21.2. Нормативная база Microsoft, на которой стоят фразы отчета

- PE Format: DataDirectory, section table, .pdata, x64 function table entries, RVA, raw section data, alignment.

- x64 exception handling: RUNTIME_FUNCTION, image-relative addresses, UNWIND_INFO, flags EHANDLER/UHANDLER/CHAININFO, unwind procedure.

- Debugger exception handling / first chance exceptions: факт возникновения исключения и стадия обработки.

- x64 calling convention: первые pointer/integer аргументы в RCX, RDX, R8, R9, поэтому dynamic arg0-arg3 читаются из этих регистров.

- AddVectoredExceptionHandler / AddVectoredContinueHandler / SetUnhandledExceptionFilter: официальные API регистрации exception handlers.

- RtlAddFunctionTable / RtlInstallFunctionTableCallback: официальные API dynamic function tables для 64-bit unwind/stack walking.

Все фразы, которые описывают формат и поведение этих механизмов, опираются на Microsoft. Все фразы, которые ранжируют подозрительность, являются инженерной моделью проекта поверх Microsoft-модели. Это честное и правильное разделение для защиты.

21.3. Почему статических признаков ровно 27

Ответ комиссии должен быть очень точным: в проекте не утверждается, что в мире существует только 27 возможных PE/unwind аномалий. Число 27 означает другое: это полный набор статических findings, реализованных в текущей версии проекта для выбранной предметной области - анализа PE32+ x64 Exception Directory, .pdata/RUNTIME_FUNCTION, .xdata/UNWIND_INFO, handler placement, section layout и корреляций между этими слоями. Поэтому правильная формулировка: '27 - это граница реализованной модели ВКР, а не математический предел всех возможных признаков'.

Эти 27 признаков выбраны не случайно. Они покрывают путь анализа от базовой валидности PE-файла до более сложных эвристик по unwind metadata. Сначала проверяется, можно ли доверять PE layout. Потом проверяется таблица RUNTIME_FUNCTION. Потом проверяется UNWIND_INFO. Потом проверяется размещение metadata/handlers. Потом проверяются повторное использование, доминирование handler-ов и корреляционные комбинации. То есть признаки образуют слоистую модель, а не список случайных if-ов.

Слой	Сколько признаков	Какие коды	Почему именно эти
PE / section layout	6	pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob	Без этих проверок нельзя доверять координатам файла: raw/virtual ranges, alignment, Exception Directory.
RUNTIME_FUNCTION table integrity	7	no_runtime_functions, invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin	Это базовые инварианты function table: наличие, диапазоны, границы image, сортировка, отсутствие неоднозначности.
UNWIND_INFO parse/integrity	2	unwind_parse_fail, unwind_anomalies	Проверяет, можно ли разобрать структуру UNWIND_INFO и нет ли странных полей/кодов.
Reuse/dominance	3	unwind_reuse_strong, unwind_reuse_moderate, handler_dominance	Ловит нетипичные паттерны массового переиспользования unwind metadata или одного handler-а.
Metadata / handler placement	5	unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section	Проверяет, соответствует ли роль области ее размещению и правам секции.
Correlation / combo findings	4	placement_exec_nonexec_handler_combo, placement_integrity_combo, reuse_dominance_combo, writable_metadata_handler_combo	Фиксирует случаи, где отдельные слабые/средние признаки вместе дают более сильную картину.
Итого	27	Все add_finding-коды из src/heuristics.c	Полное покрытие выбранной области ВКР: PE layout -> .pdata -> .xdata/UNWIND_INFO -> handlers -> dynamic correlation-ready evidence.

Почему не больше? Потому что проект сознательно ограничен предметной областью EH/unwind анализа x64 PE, а не является универсальным антивирусом или полным PE-linter-ом. Например, проект не обязан включать все возможные признаки импортов, ресурсов, TLS callbacks, overlay, entropy, signatures, packer-detection и т.д., потому что это другая предметная область. Добавить новые признаки можно, но это будет расширение модели, а не исправление ошибки.

Почему не меньше? Потому что если убрать часть этих 27, появятся слепые зоны. Без первых 6 нельзя надежно читать PE layout. Без RUNTIME_FUNCTION checks нельзя доверять .pdata. Без UNWIND_INFO checks нельзя объяснить unwind behavior. Без placement checks нельзя понять, где лежат metadata/handlers. Без combo findings отчет потеряет способность показывать корреляцию между слоями.

21.4. Почему динамических событий ровно 12

С динамикой логика такая же: проект не утверждает, что Windows имеет только 12 возможных runtime-событий. Число 12 означает полный набор DynamicEvent.type, который реализован dynamic collector-ом текущей версии проекта. Эти 12 событий выбраны под цель ВКР: наблюдать runtime evidence, связанное именно с exception handling, handler registration/hit, dynamic function tables, exception flow и качеством dynamic session.

Группа	Сколько событий	Коды	Почему именно эти
Exception flow	1	exception	Без exception event нельзя доказательно говорить, что exception dispatch path реально проявился в данном запуске.
Завершение/качество сессии	2	process_exit, timeout	Нужно отличать штатно завершенную динамику от неполной сессии по лимиту времени.
UNWIND_INFO handler execution	1	unwind_handler_hit	Показывает, что handler из статической UNWIND_INFO реально получил управление.
Vectored exception handling	2	veh_registration, veh_handler_hit	Разделяет потенциальную регистрацию VEH и фактическое попадание в VEH.
Vectored continue handling	2	vch_registration, vch_handler_hit	Разделяет регистрацию VCH и фактическое выполнение VCH handler.
Unhandled exception filter	2	uef_registration, uef_handler_hit	Разделяет установку top-level filter и фактическое попадание в него.
Dynamic function tables	2	rtl_add_function_table, rtl_install_function_table_callback	Показывает, что unwind map может формироваться во время выполнения, а не только из .pdata файла.
Итого	12	Все dynamic_event_add type из src/dynamic_win.c	Полное покрытие runtime-событий, которые проект реально наблюдает в рамках EH/unwind задачи.

Почему не больше? Потому что dynamic collector в этой ВКР наблюдает не все WinAPI-вызовы процесса, а только те, которые относятся к exception/unwind модели: VEH, VCH, UEF, RtlAddFunctionTable, RtlInstallFunctionTableCallback, handler hits, exceptions и состояние сессии. Например, CreateFile, network API, registry API, криптография, memory allocation не включены, потому что они не являются ядром темы EH/unwind.

Почему не меньше? Потому что если убрать регистрацию, мы не увидим потенциальный runtime layer. Если убрать hit, мы не отличим потенциальный route от реально выполненного. Если убрать exception, мы не увидим начало dispatch flow. Если убрать timeout/process_exit, мы не сможем честно оценить полноту dynamic session. Если убрать Rtl* function table events, мы потеряем признак того, что runtime function map меняется после загрузки файла.

21.4.1. Что покрывают 27 + 12 и как это работает как система

Статические 27 findings и динамические 12 event-типов нужно защищать не как два независимых списка, а как единую систему наблюдения. Статика отвечает на вопрос: что сам файл заявляет о своей PE/unwind-структуре и насколько эти заявления внутренне согласованы. Динамика отвечает на вопрос: что реально происходит при запуске под отладочным наблюдением - регистрируются ли обработчики, вызываются ли runtime-таблицы, происходят ли исключения и доходит ли управление до handler-ов. Поэтому верхний отчет может показывать статическую оценку, динамические сигналы и итоговую уверенность одновременно: это разные источники evidence, а не повтор одного и того же числа.

Важно: проект не пытается быть антивирусом общего назначения. Он сознательно сужен до области x64 PE exception/unwind analysis. Поэтому он глубоко проверяет именно те структуры, от которых зависят поиск RUNTIME_FUNCTION, разбор UNWIND_INFO, virtual unwind, stack walking, exception dispatch и интерпретация handler-ов. Импорты, ресурсы, цифровые подписи, поведение сети, файловые операции и packer signatures можно добавить в будущие версии, но они не являются ядром этой ВКР.

Слой покрытия	Что проверяется	Какие finding/event отсюда рождаются	Зачем это нужно
PE layout и section table	Границы raw/virtual секций, пересечения, SizeOfImage, alignment, доступность Exception Directory.	pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob.	Это фундамент координат. Если секции или directory указывают в невозможные места, дальнейший unwind-анализ становится ненадежным.
Exception Directory / .pdata	Наличие каталога, размер таблицы, перевод RVA в file offset, количество RUNTIME_FUNCTION.	no_runtime_functions, pe_exception_directory_oob, часть rf_oob/unwind_rva_oob.	Windows ищет x64 function table через image-relative metadata. Если таблица отсутствует или недоступна, stack walking и exception dispatch теряют карту функций.
RUNTIME_FUNCTION entries	BeginAddress, EndAddress, UnwindInfoAddress, сортировка, пересечения, повторяющиеся BeginAddress.	invalid_ranges, rf_oob, rf_unsorted, rf_overlaps, duplicate_begin, unwind_rva_oob.	Каждая запись должна однозначно связывать диапазон кода с unwind metadata. Нарушения дают неоднозначный или невозможный unwind.
UNWIND_INFO / .xdata	Версия, flags, count of unwind codes, границы массива кодов, padding, handler/chained tail.	unwind_parse_fail, unwind_anomalies.	UNWIND_INFO говорит, как восстановить caller context. Если структура не читается, анализатор не может корректно объяснить выход из кадра.
Placement и права секций	В какой секции лежит UNWIND_INFO, куда указывает handler, есть ли execute/write у metadata или handler.	unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section.	Metadata обычно должна быть читаемой служебной областью, а handler - исполняемым кодом. Нестандартное placement не всегда вредоносно, но требует объяснения.
Повторное использование и корреляции	Много функций с одним UNWIND_INFO, доминирование одного handler-а, сочетания placement/integrity/reuse.	unwind_reuse_strong, unwind_reuse_moderate, handler_dominance, placement_exec_nonexec_handler_combo, placement_integrity_combo, reuse_dominance_combo, writable_metadata_handler_combo.	Одиночный слабый сигнал может быть нормой, но согласованные аномалии образуют более сильный случай. Поэтому есть combo findings.
Runtime-наблюдение	Debug events, breakpoints на handler/API, registrations VEH/VCH/UEF, dynamic function tables, timeout/exit.	12 DynamicEvent.type: exception, handler hits, registrations, RtlAddFunctionTable, RtlInstallFunctionTableCallback, timeout, process_exit.	Динамика показывает runtime-слой, который не всегда виден в статической .pdata. Это особенно важно для JIT/protection-like и callback-based function tables.
Report/evidence	Факты превращаются в Rule, Observed, Expected, Impact, severity, score, confidence и рекомендации.	Все строки таблиц findings/events и итоговые карточки отчета.	Пользователь должен видеть не только verdict, но и доказательную цепочку: где найдено, какое правило нарушено, почему это важно и что проверять дальше.

21.4.2. Как данные проходят от бинарника до конкретной фразы в отчете

- Шаг 1. Пользователь запускает analyzer и выбирает режим: только static, static + heuristics, dynamic-only или combined static + dynamic. Набор флагов определяет, какие блоки будут собирать evidence.

- Шаг 2. PE parser читает DOS Header, PE Signature, COFF File Header, Optional Header, Data Directory и section table. На этом уровне появляются факты про Machine, Magic, ImageBase, SizeOfImage, SizeOfHeaders, DataDirectory[Exception] и секции.

- Шаг 3. Parser переводит RVA в file offset только через допустимые raw-диапазоны. Если RVA существует виртуально, но не имеет байтов в файле, статическое чтение не притворяется успешным. Это важно для zero-fill областей и поврежденных секций.

- Шаг 4. Exception Directory превращается в массив RUNTIME_FUNCTION. Проект проверяет кратность размера, границы каждой записи, сортировку, пересечения и доступность UnwindInfoAddress.

- Шаг 5. UNWIND_INFO parser читает версию, flags, prolog size, CountOfCodes, unwind codes и tail-данные handler/chained info. Если данных не хватает или tail не мапится в файл, появляется

parse/integrity finding.

- Шаг 6. Heuristics module агрегирует наблюдения. Он не просто пишет 'плохо', а связывает факт с кодом finding: например, unmappable UnwindInfoAddress становится unwind_rva_oob, а пересечение function ranges становится rf_overlaps.

- Шаг 7. Placement analyzer связывает RVA с секциями и их правами. Отсюда рождаются формулировки про executable metadata, writable metadata, non-executable handler и unusual section.

- Шаг 8. Correlation layer смотрит на сочетания. Например, если metadata лежит странно и одновременно есть проблемы parse/OOB, это сильнее, чем один placement-only сигнал. Поэтому появляются combo findings.

- Шаг 9. Dynamic collector, если включен, запускает бинарник под наблюдением, ставит breakpoints на релевантные API/handler addresses, пишет events и warnings. Эти события не заменяют static findings, а дополняют их.

- Шаг 10. Report renderer превращает findings/events в HTML: верхние карточки, подробные таблицы, description, rule, observed, expected, impact, interpretation и recommendations. Каждая фраза берется либо из фактических данных анализа, либо из словаря объяснений в коде, либо из итоговой scoring/confidence модели проекта.

21.4.3. Почему такое покрытие достаточно для темы ВКР

Если комиссия спросит, почему проект не проверяет вообще все PE-признаки на свете, правильный ответ такой: тема проекта - не общий PE-антивирус и не универсальный sandbox, а анализ EH/unwind-структур x64 PE. Поэтому критерий полноты здесь тематический: покрыта вся цепочка, которая нужна Windows и анализатору для exception/unwind reasoning. Эта цепочка начинается с PE layout, проходит через Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handler data, section rights и заканчивается runtime-наблюдением handler registrations / hits / dynamic function tables.

С этой точки зрения 27 статических findings закрывают статическую доказательную цепочку от файла до unwind metadata, а 12 динамических events закрывают runtime-доказательную цепочку от запуска до observed exception-handling behavior. Если добавить, например, entropy, imports или TLS callbacks, проект станет шире, но не обязательно глубже именно по EH/unwind. Поэтому текущий набор выбран по принципу: каждый признак должен либо нарушать контракт PE/x64 unwind, либо фиксировать подозрительное размещение/повтор/корреляцию unwind metadata, либо показывать runtime-поведение, которое статический parser не обязан видеть.

Границы между 'официальная норма' и 'инженерная эвристика' в отчете намеренно разделены. Microsoft задает формат и механизм: RVA, .pdata, RUNTIME_FUNCTION, UNWIND_INFO, EHANDLER/UHANDLER, dynamic function tables и API регистрации handler-ов. Проект на этой базе строит аналитические выводы: насколько аномалия сильная, как ее назвать, какой score дать, как объединить несколько сигналов и какую рекомендацию показать. Поэтому на защите нельзя говорить, что Microsoft установила именно score 50 или severity high для конкретного finding. Правильно говорить: Microsoft установила контракт структур и API, а score/severity/confidence - это авторская модель оценки, документированная в evidence matrix и реализованная в коде проекта.

21.5. Статические фразы: PE/section integrity

Finding / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
pe_section_raw_oob	Секция заявляет raw range за пределами размера файла.	PE Format: section data лежит по PointerToRawData и имеет SizeOfRawData.	Классификация как high evidence, потому что parser не может безопасно читать байты.	Файл/section table несогласован с физическим файлом.
pe_section_virtual_oob	Virtual range секции выходит за SizeOfImage.	PE Format: SizeOfImage задает размер image в памяти.	Считать это сильным structural issue.	Секция указывает за пределы загруженного образа.

Finding / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
pe_section_raw_overlap	Raw-диапазоны секций пересекаются.	PE Format описывает section data и порядок секций.	Пересечения трактуются как подозрительная неоднозначность layout.	Одни и те же байты файла принадлежат нескольким секциям.
pe_section_virtual_overlap	Virtual ranges секций пересекаются.	PE image имеет RVA layout секций.	Пересечение трактуется сильнее raw overlap, потому что ломает принадлежность RVA.	По одному RVA нельзя однозначно понять секцию.
pe_section_misaligned	Alignment/порядок/размеры секций не согласованы.	PE Format: SectionAlignment, FileAlignment и constraints section data.	Severity зависит от влияния на чтение.	Section table выглядит нестандартно или поврежденно.
pe_exception_directory_oob	Exception Directory выходит за image/file или не читается.	PE Format: Exception Table entry указывает на .pdata; DataDirectory задает RVA+Size.	High, потому что .pdata является входом для unwind анализа.	Статическая таблица RUNTIME_FUNCTION недоступна или повреждена.

21.6. Статические фразы: RUNTIME_FUNCTION integrity

Finding / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
no_runtime_functions	Exception Directory отсутствует или RUNTIME_FUNCTION count = 0.	x64 EH: nonleaf functions обычно имеют function table entries; leaf functions могут обходиться без записи.	Medium, потому что отсутствие таблицы может быть нормальным для leaf-only, но странным для сложного EXE.	Нужно смотреть контекст: маленький учебный файл или реально отсутствующая unwind map.
invalid_ranges	EndAddress <= BeginAddress.	RUNTIME_FUNCTION задает function start/end address.	Считать недопустимым диапазоном функции.	Запись не описывает нормальный непустой range кода.
rf_oob	Begin/End выходят за SizeOfImage.	Addresses image-relative и должны относиться к image.	High/medium по доле affected entries.	Function table ссылается на код вне образа.
unwind_rva_oob	UnwindInfoAddress отсутствует, OOB или не маппится.	RUNTIME_FUNCTION.UnwindInfoAddress image-relative; должен вести к UNWIND_INFO.	Static parser требует file offset для чтения байтов.	Unwind metadata функции недоступна для надежного разбора.
rf_unsorted	Записи не отсортированы по BeginAddress.	PE Format/x64 EH: function table entries sorted.	Medium, потому что поиск по ControlPc может стать ненадежным.	Таблица нарушает ожидаемый порядок.
rf_overlaps	Function ranges пересекаются.	RUNTIME_FUNCTION описывает ranges функций.	High, потому что один ControlPc может попасть в несколько записей.	Unwind lookup становится неоднозначным.
duplicate_begin	Несколько записей имеют одинаковый BeginAddress.	Function table entries должны описывать функции однозначно.	Medium: может быть об фускация/ошибка/нестандартность.	Две записи конкурируют за один старт функции.

21.7. Статические фразы: UNWIND_INFO, reuse, handler dominance

Finding / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
unwind_parse_fail	UNWIND_INFO не разбирается: truncation, unmappable tail, некорректная структура.	x64 EH задает layout UNWIND_INFO: Version, Flags, CountOfCodes, unwind codes, optional tail.	High/medium по доле failures.	Virtual unwind/stack walking не может надежно использовать metadata.
unwind_anomalies	UNWIND_INFO разбирается, но содержит странные поля/opcodes/комбинации.	x64 EH задает допустимые flags/opcodes/chain rules.	Аномалии не всегда вредоносны, но требуют проверки.	Metadata выглядит нестандартно или противоречиво.

Finding / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
unwind_reuse_strong	Одна UNWIND_INFO чрезмерно переиспользуется многими функциями.	Microsoft допускает chained/common metadata, но RUNTIME_FUNCTION обычно описывает конкретные ranges.	Порог reuse - эвристика проекта.	Может быть нормальная оптимизация или protection/dispatcher pattern.
unwind_reuse_moderate	Переиспользование заметное, но слабее strong.	Та же база x64 unwind.	Средняя severity, потому что возможны нормальные объяснения.	Контекстный сигнал, усиливается другими findings.
handler_dominance	Один handler доминирует у большого числа функций.	EHANDLER/UHANDLER указывают handler RVA.	Dominance threshold - эвристика проекта.	Может указывать на dispatcher/protection-like exception path.

21.8. Статические фразы: placement и корреляционные findings

Finding / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
unwind_in_exec_section	UNWIND_INFO лежит в executable-секции.	PE sections имеют characteristics R/W/X; UNWIND_INFO - metadata, а не код.	High, потому что metadata в executable area нетипична.	Не обязательно вредоносно, но требует ручной проверки.
unwind_unusual_section	UNWIND_INFO лежит не в ожидаемой metadata-секции.	PE/x64 unwind не требует имени .xdata, но metadata должна быть доступной.	Unusual section - эвристическое ожидание.	Контекстный placement signal.
unwind_writable_section	UNWIND_INFO в writable-секции.	Memory protection и section characteristics из PE.	Writable metadata считается рискованнее: может изменяться.	Metadata не должна легко изменяться без причины.
handler_nonexec_section	Handler RVA не попадает в executable-секцию или секция неизвестна.	EHANDLER/UHANDLER указывают адрес handler code.	High: handler должен быть исполняемым кодом.	Exception dispatch может вести в странную область.
handler_writable_section	Handler RVA находится в writable-секции.	Section characteristics R/W/X.	Medium: writable code/handler нетипичен.	Возможен self-modifying/protection context.
placement_exec_nonexec_handler_combo	UNWIND_INFO в executable area плюс handler nonexec/unknown.	База: roles metadata/code и handler address.	Correlation rule проекта.	Два placement-сигнала усиливают друг друга.
placement_integrity_combo	Placement problems сочетаются с parse/OOB/integrity problems.	База: unwind structures должны быть читаемы и согласованы.	Correlation rule проекта.	Не просто необычное место, а еще и проблемы целостности.
reuse_dominance_combo	Reuse UNWIND_INFO сочетается с handler dominance.	База: RUNTIME_FUNCTION/handler fields.	Correlation rule проекта.	Похоже на общий dispatcher/protection pattern.
writable_metadata_handler_combo	Writable metadata сочетается с writable/strange handler.	База: PE section rights и handler metadata.	Correlation rule проекта.	Изменяемая metadata и handler context вместе подозрительнее.

21.9. Динамические event-фразы: 12 типов

Event / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
exception	Debug loop увидел EXCEPTION_DEBUG_EVENT не breakpoint/single-step.	Debugger exception handling / first chance exception model.	Связь с дальнейшим route анализирует проект.	Есть реальный exception flow.
process_exit	Целевой процесс завершился.	Windows Debug API EXIT_PROCESS event.	Используется как качество сессии.	Динамический запуск дошел до завершения.
timeout	Сессия достигла --timeout-ms.	Это не Microsoft event, а ограничение проекта.	Осторожная интерпретация: найденное остается, ненайденное не доказано отсутствующим.	Покрытие динамики неполное.

Event / фраза	Когда появляется	Что из Microsoft	Что инженерное	Как понимать
unwind_handler_hit	Breakpoint на handler из UNWIND_INFO сработал.	EHANDLER/UHANDLER handler RVA из x64 EH.	Hit сильнее наличия указателя.	Handler реально получил управление.
veh_registration	Вызван AddVectoredExceptionHandler.	Win32 API регистрирует VEH.	Registration - контекст, не verdict.	Есть runtime exception-handling layer.
veh_handler_hit	Зарегистрированный VEH достигнут исполнением.	VEH handler зарегистрирован официальным API.	Hit сильнее registration.	VEH реально участвовал в runtime flow.
vch_registration	Вызван AddVectoredContinueHandler.	Win32 API регистрирует VCH.	Контекстный runtime signal.	Программа настраивает continue-handler layer.
vch_handler_hit	Зарегистрированный VCH достигнут исполнением.	VCH handler pointer из API.	Наблюдаемое исполнение сильнее registration.	Continue handler реально получил управление.
uef_registration	Вызван SetUnhandledExceptionFilter.	Win32 API заменяет top-level unhandled exception filter.	Под отладчиком отсутствие hit не доказывает отсутствие UEF вне debug.	Есть top-level crash/exception filter.
uef_handler_hit	UEF handler достигнут исполнением.	UEF вызывается при unhandled exception path, с особенностями debug/non-debug.	Сильный runtime signal, но нужен контекст.	Исключение дошло до top-level path.
rtl_add_function_table	Вызван RtlAddFunctionTable.	Microsoft: adds dynamic function table; function tables used for 64-bit unwind/stack walking.	Не вредоносно само по себе, но static .pdata неполна.	Runtime добавляет unwind map.
rtl_install_function_table_callback	Вызван RtlInstallFunctionTableCallback.	Microsoft: callback-based function table for very dynamic code.	Medium signal: может быть JIT/runtime/protection.	Function table может выдаваться callback-ом.

21.10. Групповые фразы 'Как интерпретировать'

Фраза 'Как интерпретировать'	Условия появления	Документация	Инженерный смысл
Регистрация может быть нормальной, но показывает runtime-слой, не видимый только по PE/.pdata.	Есть VEH/VCH/UEF registration.	Win32 APIs register handlers; PE/.pdata описывает static function table.	API штатный, не вредоносный сам по себе; но он добавляет runtime behavior.
Handler hit сильнее простой регистрации.	Есть unwind/VEH/VCH/UEF handler hit.	Debugger observation + handler APIs.	Фактическое достижение адреса сильнее наличия pointer-а.
Если исключений нет, registrations остаются контекстом; если есть, можно проверять маршрут dispatch/unwind.	Есть registrations и/или exception events.	Debugger exception handling; x64 exception dispatch.	Без exception не видно маршрута, с exception можно связать event -> handler.
Warnings не отменяют найденные события, но снижают полноту runtime-наблюдения.	timed_out, instrumentation incomplete, breakpoint/module/dropped warnings.	Это методика качества эксперимента, не отдельный Microsoft API.	Найденное остается фактом, но отсутствие других событий менее доказательно.
Runtime function tables не обязательно вредоносны, но static .pdata уже не полная карта.	Есть RtlAddFunctionTable/RtlInstallFunctionTableCallback.	Microsoft docs по dynamic function tables.	Для JIT/runtime это нормально, для анализа важно как runtime metadata.
Dynamic check not requested / no runtime signals.	Нет dr или signal_count=0.	Не документация Microsoft, а режим работы инструмента.	Не запускалось или не проявилось; нельзя путать с доказанным отсутствием behavior.

21.11. Evidence-блоки: Правило / Что найдено / Как должно быть / Почему важно

Элемент	Что означает	Источник правильности	Как защищать
Описание	Короткая русская формулировка finding.	Привязано к finding code в report.c; сам finding создан heuristics.c.	Это не отдельное доказательство, а название проблемы.

Элемент	Что означает	Источник правильности	Как защищать
Правило	Какой инвариант проверялся.	Для PE/unwind - Microsoft docs; для scoring combos - эвристика проекта.	Сказать: rule либо нормативный, либо явно эвристический.
Что найдено / Observed	Фактические значения: count, RVA, section, file_offset, related function.	Parser/dynamic collector; report.c только выводит.	Это главное доказательство, не мнение.
Как должно быть / Expected	Нормальное состояние структуры или доступность для parser-а.	Microsoft docs + требования статического чтения файла.	Объяснить, какая часть нормативная, какая parser-specific.
Почему важно / Impact	Практическое последствие для unwind/stack walking/dispatch/анализа.	Microsoft unwind procedure + инженерная логика анализа.	Показывает, почему finding не формальность.

21.12. Mitigation, recommendations, verdict-фразы

Группа фраз	Когда появляется	Что означает	Основание
Mitigation / снижение уверенности	Одинокий сигнал, маленькая .pdata выборка, placement-only, single family.	Проект уменьшает confidence, чтобы не переутверждать вывод.	Инженерная осторожность, не Microsoft standard.
План ручной проверки	Есть findings/dynamic signals.	Что открыть в PE viewer/Ghidra/debugger и что подтвердить.	Методика reverse engineering по найденной evidence.
Вердикт low/moderate/high	По score/static+dynamic summary.	Словесная сводка подозрительности.	Эвристика проекта; должна защищаться findings/evidence.
Уровни none/weak/medium/strong	Для static/runtime evidence.	Сила доказательной базы, а не вероятность вируса.	Инженерная модель evidence.
Confidence %	Итоговый confidence.	Уверенность в выводе анализа, не вероятность вредоносности.	Считается по модели проекта; снижается при слабой/неполной evidence.
Корреляция слоев	Статика и динамика подтверждают друг друга.	Разные независимые наблюдения усиливают случай.	Инженерная логика evidence correlation.

21.13. Как отвечать комиссии про любую фразу отчета

- Сначала определить тип фразы: поле, факт, правило, expected, impact, interpretation, score или recommendation.

- Если это поле PE/x64 unwind/API - сослаться на Microsoft PE Format, x64 exception handling или Win32 API docs.

- Если это Observed - показать, где parser/dynamic collector получил значение.

- Если это 'как интерпретировать' - объяснить, что это осторожная evidence-логика, а не обвинение во вредоносности.

- Если это score/confidence - честно сказать, что это авторская эвристика проекта поверх официальных правил Windows.

- Если комиссия спорит - показать связку: код finding/event -> Rule/Observed/Expected/Impact -> официальный источник Microsoft -> ручная проверка.

Итоговая формула: официальная документация Microsoft задает устройство механизмов; проект наблюдает конкретные значения и события; отчет объясняет эти наблюдения понятным языком; score и рекомендации являются прозрачной инженерной моделью, которая помогает аналитику приоритизировать проверку, но не подменяет ручную экспертизу.

21.14. Ссылки на официальные источники

- Microsoft PE Format: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

- Microsoft x64 exception handling: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64

- Microsoft x64 architecture/calling convention registers: https://learn.microsoft.com/windows-hardware/drivers/debugger/x64-architecture

- Microsoft Visual Studio first chance exceptions: https://learn.microsoft.com/en-us/visualstudio/debugger/managing-exceptions-with-the-debugger

- AddVectoredExceptionHandler: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredexceptionhandler

- AddVectoredContinueHandler: https://learn.microsoft.com/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredcontinuehandler

- SetUnhandledExceptionFilter: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-setunhandledexceptionfilter

- RtlAddFunctionTable: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable

- RtlInstallFunctionTableCallback: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback